



# IEC 62443-4-2

Companion Guide for VMware Cloud Foundation 9.1

June 2026

## Table of Contents

|                                            |     |
|--------------------------------------------|-----|
| Disclaimer.....                            | 8   |
| Version.....                               | 9   |
| Introduction.....                          | 9   |
| Third Party Identifiers and Mappings ..... | 9   |
| Coverage Levels .....                      | 9   |
| Controls.....                              | 10  |
| Control CR 1.1                             | 10  |
| Control CR 1.1(1)                          | 15  |
| Control CR 1.1(2)                          | 20  |
| Control CR 1.2                             | 21  |
| Control CR 1.2(1)                          | 25  |
| Control CR 1.3                             | 29  |
| Control CR 1.4                             | 33  |
| Control CR 1.5                             | 36  |
| Control CR 1.5(1)                          | 41  |
| Control CR 1.5(a)                          | 43  |
| Control CR 1.5(b)                          | 47  |
| Control CR 1.5(c)                          | 51  |
| Control CR 1.5(d)                          | 54  |
| Control CR 1.6                             | 58  |
| Control CR 1.7                             | 59  |
| Control CR 1.7(1)                          | 62  |
| Control CR 1.7(2)                          | 65  |
| Control CR 1.8                             | 68  |
| Control CR 1.9                             | 71  |
| Control CR 1.9(1)                          | 76  |
| Control CR 1.9(a)                          | 78  |
| Control CR 1.9(b)                          | 84  |
| Control CR 1.9(c)                          | 90  |
| Control CR 1.9(d)                          | 96  |
| Control CR 1.9(e)                          | 101 |
| Control CR 1.9(f)                          | 106 |
| Control CR 1.10                            | 111 |

|                    |     |
|--------------------|-----|
| Control CR 1.11    | 114 |
| Control CR 1.11(a) | 116 |
| Control CR 1.11(b) | 118 |
| Control CR 1.12    | 120 |
| Control CR 1.14    | 122 |
| Control CR 1.14(1) | 126 |
| Control CR 1.14(a) | 128 |
| Control CR 1.14(b) | 132 |
| Control CR 1.14(c) | 136 |
| Control CR 1.14(d) | 140 |
| Control CR 2.1     | 144 |
| Control CR 2.1(1)  | 150 |
| Control CR 2.1(2)  | 156 |
| Control CR 2.1(3)  | 162 |
| Control CR 2.1(4)  | 163 |
| Control CR 2.2     | 165 |
| Control CR 2.5     | 171 |
| Control CR 2.5(a)  | 173 |
| Control CR 2.5(b)  | 175 |
| Control CR 2.6     | 177 |
| Control CR 2.7     | 180 |
| Control CR 2.8     | 182 |
| Control CR 2.9(1)  | 188 |
| Control CR 2.9(a)  | 192 |
| Control CR 2.9(b)  | 197 |
| Control CR 2.10(a) | 202 |
| Control CR 2.10(b) | 206 |
| Control CR 2.11    | 211 |
| Control CR 2.11(1) | 215 |
| Control CR 2.11(2) | 218 |
| Control CR 2.12    | 222 |
| Control CR 2.12(1) | 227 |
| Control CR 3.1     | 232 |
| Control CR 3.1(1)  | 237 |

|                   |     |
|-------------------|-----|
| Control CR 3.3    | 242 |
| Control CR 3.3(1) | 246 |
| Control CR 3.4    | 250 |
| Control CR 3.4(1) | 252 |
| Control CR 3.4(2) | 254 |
| Control CR 3.5    | 256 |
| Control CR 3.6    | 261 |
| Control CR 3.7    | 265 |
| Control CR 3.8    | 270 |
| Control CR 3.8(a) | 275 |
| Control CR 3.8(b) | 280 |
| Control CR 3.8(c) | 285 |
| Control CR 3.9    | 290 |
| Control CR 3.9(1) | 295 |
| Control CR 3.10   | 296 |
| Control CR 3.11   | 297 |
| Control CR 3.12   | 300 |
| Control CR 3.13   | 302 |
| Control CR 3.14   | 304 |
| Control CR 4.1(a) | 306 |
| Control CR 4.1(b) | 309 |
| Control CR 4.2    | 314 |
| Control CR 4.2(1) | 316 |
| Control CR 4.2(2) | 318 |
| Control CR 4.3    | 320 |
| Control CR 5.1    | 327 |
| Control CR 5.2    | 331 |
| Control CR 5.3    | 336 |
| Control CR 6.1    | 340 |
| Control CR 6.1(1) | 345 |
| Control CR 6.2    | 350 |
| Control CR 7.1    | 356 |
| Control CR 7.1(1) | 361 |
| Control CR 7.2    | 365 |

|                     |     |
|---------------------|-----|
| Control CR 7.3      | 370 |
| Control CR 7.3(1)   | 376 |
| Control CR 7.4      | 381 |
| Control CR 7.6      | 386 |
| Control CR 7.6(1)   | 392 |
| Control CR 7.7      | 396 |
| Control CR 7.8      | 400 |
| Control EDR 2.4     | 404 |
| Control EDR 2.4(1)  | 405 |
| Control EDR 2.4(a)  | 407 |
| Control EDR 2.4(b)  | 408 |
| Control EDR 2.4(c)  | 409 |
| Control EDR 2.13    | 410 |
| Control EDR 2.13(1) | 411 |
| Control EDR 3.2     | 413 |
| Control EDR 3.10    | 414 |
| Control EDR 3.10(1) | 415 |
| Control EDR 3.11    | 416 |
| Control EDR 3.11(1) | 419 |
| Control EDR 3.12    | 420 |
| Control EDR 3.13(a) | 422 |
| Control EDR 3.13(b) | 424 |
| Control EDR 3.14    | 426 |
| Control EDR 3.14(1) | 428 |
| Control HDR 2.4     | 430 |
| Control HDR 2.4(1)  | 431 |
| Control HDR 2.4(a)  | 432 |
| Control HDR 2.4(b)  | 433 |
| Control HDR 2.4(c)  | 434 |
| Control HDR 2.13    | 435 |
| Control HDR 2.13(1) | 436 |
| Control HDR 3.2     | 438 |
| Control HDR 3.2(1)  | 442 |
| Control HDR 3.10    | 444 |

|                     |     |
|---------------------|-----|
| Control HDR 3.10(1) | 445 |
| Control HDR 3.11    | 447 |
| Control HDR 3.11(1) | 450 |
| Control HDR 3.12    | 453 |
| Control HDR 3.13(a) | 455 |
| Control HDR 3.13(b) | 457 |
| Control HDR 3.14    | 460 |
| Control HDR 3.14(1) | 462 |
| Control NDR 1.6     | 464 |
| Control NDR 1.6(1)  | 465 |
| Control NDR 1.13    | 466 |
| Control NDR 1.13(1) | 470 |
| Control NDR 2.4     | 473 |
| Control NDR 2.4(1)  | 474 |
| Control NDR 2.4(a)  | 475 |
| Control NDR 2.4(b)  | 476 |
| Control NDR 2.4(c)  | 477 |
| Control NDR 2.13    | 478 |
| Control NDR 2.13(1) | 479 |
| Control NDR 3.2     | 481 |
| Control NDR 3.10    | 485 |
| Control NDR 3.10(1) | 486 |
| Control NDR 3.11    | 488 |
| Control NDR 3.11(1) | 491 |
| Control NDR 3.12    | 494 |
| Control NDR 3.13(a) | 496 |
| Control NDR 3.13(b) | 498 |
| Control NDR 3.14    | 500 |
| Control NDR 3.14(1) | 502 |
| Control NDR 5.2     | 504 |
| Control NDR 5.2(1)  | 509 |
| Control NDR 5.2(2)  | 513 |
| Control NDR 5.2(3)  | 517 |
| Control NDR 5.3     | 521 |

|                    |     |
|--------------------|-----|
| Control SAR 2.4    | 525 |
| Control SAR 2.4(1) | 526 |
| Control SAR 2.4(a) | 527 |
| Control SAR 2.4(b) | 528 |
| Control SAR 2.4(c) | 529 |
| Control SAR 3.2    | 530 |

## Disclaimer

This document is intended to provide general guidance for organizations that are considering Broadcom solutions. The information contained in this document is for educational and informational purposes only. This document is not intended to provide advice and is provided “AS IS.” Broadcom makes no claims, promises, or guarantees about the accuracy, completeness, or adequacy of the information contained herein. Organizations should engage appropriate legal, business, technical, and audit expertise within their specific organization for review of requirements and effectiveness of implementations.



## Version

910-20260612-01

## Introduction

This document describes how VMware Cloud Foundation (VCF) 9.1 and the separately licensed advanced services assessed alongside it, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, and VMware Avi Load Balancer, can support organizations working toward compliance with IEC 62443-4-2. This standard, part of the ISA/IEC 62443 series for industrial automation and control system (IACS) security, specifies technical security requirements at the component level, including authentication, authorization, data integrity, and data confidentiality. Each assessed product is reported separately for every control, with VCF covering the full platform, including its consumption services and Private AI Services capabilities. The responsibility for secure integration, configuration, and achieving the target security level remains with the system integrator or asset owner.

<https://webstore.iec.ch/en/publication/34421>

This guidance evolves. Please check the current revision at: <https://bcm.tech/vcf-compliance>

## Third Party Identifiers and Mappings

This document includes regulatory compliance and security control identifiers from external sources as a convenience to end users. This does not constitute endorsement, in either direction.

There is not a one-to-one mapping of product capabilities to third-party controls. A product capability, or set of capabilities, may be applicable to multiple controls. Conversely, a control may be satisfied with the use of multiple capabilities.

Control identifier numbers have been included from the Secure Controls Framework, version 2026.1, under the terms of the Creative Commons Attribution-NoDerivatives 4.0 International Public License. No modifications have been made to the control identifier numbers under the terms of the license.

<https://securecontrolsframework.com/>

## Coverage Levels

Each control entry below carries the framework control text, the SCF control or controls it maps to, and an Aggregate Coverage line showing the strongest position any one of the assessed products holds for that control. Products that meet or contribute to the control follow as subsections, each with its own coverage level and response. Products the control does not apply to are listed together at the end of the entry, with a brief explanation. The coverage levels are:

- **Meets** -- the product directly provides the capability described in the control. No additional organizational process, external product, or human activity is needed beyond configuring the product itself.
- **Contributes** -- the product provides technical capabilities that help satisfy the control, but the control also requires something outside the product to be fully met, such as an organizational process, human activity, or an additional tool.
- **Not Applicable** -- the control does not apply to the product. Physical facility controls, staffing decisions, organizational governance processes, and domains the product does not operate in fall here.
- **Not Supported** -- the control is relevant to the product's domain (infrastructure, networking, security), but the product lacks the specific capability the control requires.
- **Not Assessed** -- the control has not yet been evaluated against the product.

## Controls

### Control CR 1.1

> Human user identification and authentication: Provide the capability to identify and authenticate all human users on all interfaces capable of human user access. Unique identification and authentication: Capability to uniquely identify and authenticate all human users. Multifactor authentication for untrusted networks: Capability to employ multi-factor authentication for all human user access from an untrusted network. Authenticator feedback: Obscure feedback of authentication information (e.g., asterisks) during entry. Unsuccessful login attempts: Enforce a limit of consecutive invalid login attempts before lockout or delay. System use notification: Display a notification message before establishing user identification. Access via untrusted networks: Monitor and control all access attempts from untrusted networks. Strength of symmetric key-based authentication: Capability to manage symmetric keys securely. Symmetric key protection: Hardware-based protection for symmetric keys.

**SCF Controls:** IAC-02, IAC-03

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides centralized authentication, authorization, and auditing through VCF SSO, which acts as the identity broker for all VCF management components. All user and service authentication flows through VCF SSO, providing a single point of control for identity management across VMware vCenter, VMware ESX, NSX, VCF Operations, and VCF Automation. VCF SSO is configured via the VCF Operations console and provides a unified authentication source for VCF components.

VCF supports multiple authentication mechanisms for organizational users. VCF SSO issues SAML tokens for authenticated sessions and supports multiple authentication context methods, including PasswordProtectedTransport (password over HTTPS) and holder-of-key tokens that embed security artifacts for delegation scenarios. vCenter supports built-in identity source authentication using local domain credentials and basic authentication through the OAuth 2.0 Password grant type when federated to an external identity provider. VCF SSO also supports the use of SSPI (Security Support Provider Interface) for client authentication.

Identity federation with enterprise identity providers (via SAML or OIDC) enables organizations to extend their existing AAA infrastructure to VCF. Federating vCenter to an enterprise identity provider supports multifactor authentication and automatic registration and termination of users across services. vCenter has documented federation configurations for specific providers including Okta and PingFederate. VCF SSO also supports the System for Cross-domain Identity Management (SCIM) protocol for automating the exchange of user identity information between identity domains and external identity providers.

Organizations using Microsoft Active Directory can use user identities contained in a Windows Server domain controller or Active Directory service across their virtual infrastructure. VCF Automation allows each organization to define its own identity provider that it shares with other applications; users authenticate to the identity provider to obtain a token that they then use to log in to the organization. Every organization in VCF Automation has its own identity source for user import. The Supervisor component supports registration of external OIDC identity providers for authentication using Pinniped, which inspects upstream identity tokens from the external provider to determine user identity and group membership before passing that information to the Kubernetes API server. Supervisor can be configured with any OIDC-compliant provider, including Okta, Workspace ONE Access, and Dex, to authenticate users connecting to VMware Kubernetes Service (VKS) clusters. Custom ClusterClass configurations for VKS clusters can be used to authenticate developers using existing corporate identity systems. The Kubernetes API server also supports integration with organizational authentication infrastructure such as LDAP or Kerberos through authentication plugins, enabling Supervisor to participate in existing centralized identity management. An Identity API is available to programmatically authenticate and manage the authorization provider, supporting automated identity lifecycle management.

Authorization is enforced through role-based access control across all VCF components. vCenter maps authorization levels to groups and roles, providing consistent security controls regardless of operational context. VCF verifies what objects principals have access to as part of its authorization model. NSX has a set of built-in roles including Enterprise Admin, Auditor, Cloud Admin, Cloud Operator, Network Admin, Network Operator, Project Admin, Security Admin, and others. Local guest user

accounts in NSX have the Auditor role by default, and only a user with the Enterprise Admin role can create new local guest user accounts. VCF Automation provides Organization Administrator, Organization Auditor (read-only), and Organization Owner roles, along with the ability to create custom roles scoped to an organization. Organization Administrators can view the rights associated with roles and create roles local to their organization. For Kubernetes workloads, vSphere Supervisor supports Namespace Owner, Edit, and View permission roles that allow consumers of vSphere Namespaces to provision VKS clusters and vSphere Pods using `kubectl`. Kubernetes RBAC evaluates Roles, ClusterRoles, RoleBindings, and ClusterRoleBindings against the username, UID, group membership, and extra fields supplied by authenticator modules to determine whether a request is permitted. Users authenticated through external OIDC providers are referenced in ClusterRoleBindings using the `oidc:user@example.com` naming convention; users authenticated through VCF SSO use the `sso:<username>@<domain>` convention. The `AuthorizeWithSelectors` feature gate passes field and label selector information to authorization webhooks, supporting more granular per-resource authorization decisions. User impersonation is governed by Kubernetes RBAC: the "impersonate" verb on user, group, and ServiceAccount resources under `authentication.k8s.io` allows designated administrators to act on behalf of other principals in a traceable manner. Kubernetes also distinguishes human user accounts, whose identities are managed externally by the identity provider, from ServiceAccounts, which are namespace-scoped API objects used by automated workloads and processes; separating their lifecycle management supports differentiated audit strategies for each category. IAM role trust policies for VKS clusters can be scoped to specific service accounts and namespaces to enforce least privilege.

VCF provides role-based visibility into policy enforcements. The Organization Administrator and Organization Auditor roles can view policy enforcements within their organizations. VCF's logging infrastructure captures authentication and authorization events across management components. The Kubernetes API server records API server activity with structured per-event metadata. Audit annotations include `authorization.k8s.io/decision` and `authorization.k8s.io/reason`, which record each authorization decision and its human-readable justification in the audit log, providing per-request traceability of who performed what action and how the authorization decision was reached. An audit event is logged for each impersonation request, enabling tracking of how impersonation is used over time. Audit rules can be configured to alert on specific events, such as concurrent reading of multiple Secrets by a single user. Authentication context, including username and group memberships, can be retrieved and logged using `kubectl auth whoami` for verification purposes. Administrators can use `kubectl auth can-i` with user impersonation to determine what actions other users are permitted to perform.

Service-to-service authentication uses certificate-based Principal Identity users rather than password credentials. Principal identity user certificates must be SHA256-based and use RSA/DSA message algorithm with 2048 bits or above key size. Each VCF component registers with VCF SSO as a solution user. VCF Operations integrates with VCF SSO for centralized user and group authentication and management, and also supports LDAP authentication sources for additional monitoring and management access.

Additional Consumption-layer services extend the AAA model to workload infrastructure. Harbor Registry, available as a standard package on VKS clusters, enforces role-based access control for container image artifacts and artifact security policies. The Secret Store Supervisor Service enforces RBAC across administrators, DevOps engineers, and application tenants for access to secrets and credentials. Argo CD, deployable as a Supervisor Service, supports OIDC authentication with PKCE via the `enablePKCEAuthentication` field and defines access through global RBAC policies using policy rules and group mappings.

While VCF provides the technical mechanisms for authentication, authorization, and audit, centralizing these capabilities requires the organization to deploy and manage an external identity provider (such as Active Directory or an OIDC-compliant provider), define identity governance policies, and establish processes for user lifecycle management including provisioning, periodic access reviews, and de-provisioning. Role assignment procedures and access review processes are organizational responsibilities that fall outside VCF's direct control.

VMware Private AI Services (PAIS) integrates with an external OIDC identity provider to authenticate organizational users accessing AI infrastructure on VCF. The `PAISConfiguration` custom resource is configured with an OIDC authentication block specifying the issuer URL, the client ID (`'pais-authentication-id'`), and the scopes `'openid'`, `'groups'`, and `'offline_access'`. When a region is created for a supervisor in PAIS, an identity provider is automatically created in VMware vCenter for that

supervisor, establishing a consistent authentication entry point for AI workloads. Activating PAIS in an organization namespace requires coordination with the OIDC provider administrator, who supplies the issuer URL and client ID.

Authorization for PAIS is enforced through OIDC group-based access control. The ``authorizedGroups`` setting in the `PAISConfiguration` restricts service access to user accounts that belong to designated OIDC groups (for example, ``group-for-pais-access-01`` or ``group-for-pais-access-02``). The ``oidc-groups-claim`` setting names the claim used to identify group membership in the Access Token, and ``oidc-extra-audiences`` accepts additional OAuth 2.0 client IDs that PAIS will recognize when validating tokens. Grafana, used for PAIS metrics visualization, extends the same model through the ``grafana-oidc-role-mapping`` configuration: users in a designated PAIS access group are assigned the Admin role automatically, while remaining authenticated users receive the Viewer role.

Harbor Registry imposes its own credential boundaries for AI artifact access. PAIS integrates with Harbor registries as a model storage backend, requiring authentication and certificate-based trust for model retrieval. Processes that push artifacts to Harbor authenticate using the ``oras login`` command with username and password credentials. Knowledge Base integrations support per-source authentication: Confluence connections accept a user name and password or a personal access token, and SharePoint On-Premises connections use user name and password authentication. These mechanisms restrict access to organizational AI artifacts and reference data to authorized accounts.

### **VMware vDefend (Contributes)**

VMware vDefend contributes to this control through two mechanisms: management plane authentication for its Security Services Platform (SSP) and Identity Firewall (IDFW) for network policy enforcement.

SSP provides two built-in local user accounts configured during deployment: an admin account with the Enterprise Admin role, which has full privileges to deploy, manage, upgrade, and configure authentication and authorization settings, and an audit account with the Auditor role, which provides read-only visibility into system activity and API access for auditing without the ability to make configuration changes. The SSP License Hub implements role-based access control with three defined roles: Enterprise Admin, Auditor, and Support Bundle Collector. Roles can be assigned to individual users or groups defined in an external identity provider, allowing organizations to integrate Active Directory or LDAP group membership into access control for the platform.

SSP supports directory-backed authentication through Active Directory over LDAP and OpenLDAP in addition to local accounts. The platform uses Authelia, an open-source authentication and authorization server, to provide two-factor authentication and single sign-on capabilities for management access, allowing organizations to require multiple authentication factors for access to vDefend's security management functions.

Identity Firewall processes user identity at the source workload to enforce DFW and gateway firewall rules based on Active Directory user and group membership. This allows AD users and groups to be used as source criteria in firewall rules, enabling network access controls based on authenticated user identity rather than IP address. Security administrators are directed to distinguish between two primary administrative personas when managing vDefend access, maintaining security boundaries between different projects and the broader organization.

Centralized AAA for organizational users across the broader VCF environment, including single sign-on and multi-factor authentication for all users, is provided by VMware vCenter and VCF SSO and is outside vDefend's scope.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) addresses the authentication component of this control through a two-tier identity model. DSM supports both local user accounts and LDAPS-integrated directory service accounts, and administrators can configure one or both identity sources simultaneously. Local users are authenticated by matching their email address and validating their password directly within DSM. LDAPS users are authenticated by matching the email domain name against the registered LDAPS server and validating credentials against the directory service. DSM can be configured to use an LDAP server for both authentication and authorization of platform-level users.

Authentication for the managed databases is handled through database-native mechanisms that DSM exposes and configures. PostgreSQL databases use host-based authentication (HBA), with rules managed through DSM. The HBA configuration

applies ordered matching rules where the first entry matching the connection type, client address, requested database, and user name is applied. The default final entry uses SCRAM-SHA-256, providing strong password-based authentication; an optional LDAP entry can also be included. MySQL databases support LDAPS integration using a simple authentication plug-in, with a unique LDAPS user entry per user and support for Active Directory authentication. PostgreSQL and MySQL databases also support LDAPS user accounts linked to organizational directory services, with LDAPS user creation managed through the DSM interface.

DSM adds SQL Server as a supported database engine. SQL Server clusters support two administrator identity types: SQL User credentials and Windows Principal accounts for clusters integrated with Active Directory. DSM's Active Directory integration supports Kerberos-based authentication for domain-joined SQL Server clients, allowing tools such as SQL Server Management Studio running on a Windows machine to authenticate using domain credentials. DSM maps the dbo user, the database object owner, to administrator-controlled identities for SQL Server database access. SQL Server clusters can also use contained database users, where each user authenticates directly against the database rather than a server-level login, keeping identity information self-contained within the database.

Authorization is handled through a role-based model. DSM provides two platform-level roles, DSM Admin and DSM User, which determine the capabilities available to each user within the DSM management interface and API. Custom user roles can be configured for directory service group members, providing scoped monitoring and management access. The Permissions view in DSM supports the creation, monitoring, and management of configured users, giving administrators visibility into who has access. Role assignments also affect behavior in database operations; DSM automatically establishes trust between primary and secondary PostgreSQL instances for Admin-role users, while User-role accounts require manual trust establishment.

DSM produces audit records covering both user-initiated and system-initiated actions. Audit events are generated for actions performed through the DSM UI and API, and each audit log entry includes a Source field identifying the initiating entity: either a user's email address or "System" when the action was initiated by DSM itself. Gateway Audit Logs provide visibility into gateway-level events, and the System Audit Events view is the designated location in the DSM UI for tracking nondatabase-related operations and tasks. For SQL Server clusters, DSM provides configurable server auditing with preset audits for login tracking and log retention; the Login Auditing option can automatically create a simple audit for login events.

In self-service consumption deployments, the DSM Consumption Operator authenticates with the DSM infrastructure using a dedicated authentication secret (dsm-auth-creds), providing a defined authentication path for Kubernetes-native database provisioning.

DSM's authentication, authorization, and audit capabilities are scoped to the DSM platform and its managed databases. Centralized AAA for the broader organizational user population, including account lifecycle management, privileged access governance, and enterprise-wide log aggregation, requires organizational identity governance programs and security tooling outside DSM's scope.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) maintains its own administrative identity plane with mechanisms covering all three AAA elements: authentication, authorization, and audit. User accounts fall into two categories: local users and remote users. Remote users are authenticated through external AAA servers, with support for LDAP, TACACS+, and SAML-based authentication profiles. The product documentation recommends federated authentication as the preferred approach, delegating the authentication process to an organization's central identity provider rather than managing credentials locally within Avi.

For authorization, the Avi Controller evaluates authorization mapping rules on every user login, updating the user record with tenant and role pairs. Auth Mapping Profiles can map multiple user groups to different roles with tenant isolation, supporting both simple single-role assignments and multi-tenant configurations where a single user requires access across multiple administrative boundaries. The admin user is authorized and responsible for the highest-level configuration operations, including user creation and authorization and tenant management. The admin account is maintained as a locally authenticated account even in a system configured for remote authentication, providing a break-glass path for Controller access.

The Avi Controller License Hub provides dedicated role-based accounts for administration and auditing. The Admin account holds the `enterprise_admin` role with full access to deploy and manage security services, perform upgrades, and configure authentication and authorization. The Audit account holds the `auditor` role with full access to APIs and services for system auditing. LDAP can be configured as an identity provider for License Hub authentication, and the Admin user can reset the Audit user's password.

Avi also supports OAuth and OpenID Connect (OIDC) for client authentication at the application proxy layer. When configured, the Controller acts as a resource server and retrieves identity information from the OIDC `UserInfo` endpoint or from ID token claims, enabling per-user identity context to be associated with proxied application traffic. DataScript can check if a user ID is in an approved group list and redirect unauthenticated users to a login page, providing identity-based access control at the virtual service layer.

In the Avi for VCF Organization-Managed Mode, the organization administrator must be authenticated to edit Service Engine Groups and manage their own infrastructure configurations, reinforcing role-based access enforcement at the VCF integration layer.

The control asks whether the organization centrally manages AAA across its environment. Avi provides the mechanisms for its own administrative plane and supports integration with a central identity provider, but the organization must configure that integration, maintain the identity provider, and establish processes for reviewing audit log output.

Not applicable for this control: VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.



## Control CR 1.1(1)

> Human user identification and authentication: Provide the capability to identify and authenticate all human users on all interfaces capable of human user access. Unique identification and authentication: Capability to uniquely identify and authenticate all human users. Multifactor authentication for untrusted networks: Capability to employ multi-factor authentication for all human user access from an untrusted network. Authenticator feedback: Obscure feedback of authentication information (e.g., asterisks) during entry. Unsuccessful login attempts: Enforce a limit of consecutive invalid login attempts before lockout or delay. System use notification: Display a notification message before establishing user identification. Access via untrusted networks: Monitor and control all access attempts from untrusted networks. Strength of symmetric key-based authentication: Capability to manage symmetric keys securely. Symmetric key protection: Hardware-based protection for symmetric keys.

**SCF Controls:** IAC-03

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides a centralized identity federation architecture that supports authentication, authorization, and auditing of both organizational and non-organizational users. Third-party users and service accounts can be uniquely identified and managed through integration with external identity providers, giving administrators a single point of control over who accesses the platform and what they can do.

VCF SSO is the central authentication service for the platform. All user authentication flows through VCF SSO, which issues tokens that establish identity and trust with API endpoints. VCF SSO supports federation with enterprise identity providers through OIDC, OAuth 2.0, and Generic SAML 2.0, so organizations can connect their existing directory services and enforce their own authentication policies, including multifactor authentication. Federating VMware vCenter to an enterprise identity provider alleviates the burden of identity management and provides flexible options such as multifactor authentication and automatic registration and termination of users across services. This federation model allows organizations to manage third-party user identities in their existing identity provider and have those identities flow through to VCF without creating local accounts. The VCF Identity Broker provisions users and groups from the configured identity provider, enabling centralized access control across the platform. vCenter also supports the System for Cross-domain Identity Management (SCIM) protocol for automating user and group synchronization from external identity providers, with explicitly documented federation configurations for Okta and PingFederate as named OIDC partners.

Multiple VCF components support external identity provider integration. NSX Manager can be configured to authenticate users from identity management providers and integrates with LDAP, vIDM, and OIDC to verify user identities and determine access to services based on user group membership. NSX Manager also exposes API endpoints for authentication policy settings, allowing administrators to validate authentication configuration across nodes programmatically. The Supervisor component supports registration of external identity providers using OAuth 2.0 and OIDC protocols, and uses the Pinniped Supervisor and Concierge components to facilitate federated authentication for users and groups when an external identity provider is configured. Supervisor can be configured with any OIDC-compliant identity provider such as Okta, Workspace ONE Access, or Dex to authenticate users connecting to VMware Kubernetes Service (VKS) clusters, and custom ClusterClass configurations can authenticate developers using their existing corporate identity systems. Once authenticated via OIDC, third-party users are subject to Kubernetes RBAC; ClusterRoleBindings grant roles to OIDC-authenticated users via identifiers in the format `oidc:user@example.com`, giving administrators precisely scoped access to specific namespaces or resources without extending broader platform privileges. Kubernetes OIDC authentication produces a `UserInfo` object derived from the token subject and roles claims, and `userValidationRules` can enforce constraints on mapped identities, such as preventing the reserved `system:` prefix in usernames. For production clusters with multiple users accessing the Kubernetes API directly, OIDC is the recommended authentication mechanism; OIDC tokens are short-lived and subject to the identity provider's revocation mechanisms, so deprovisioning a user in the upstream identity provider removes their ability to authenticate to the cluster. VCF Automation allows each organization to define an identity provider that it shares with other applications, so that users authenticate to the identity provider to obtain a token they can use to access the organization. VCF Automation also supports importing provider users from SAML, OIDC, or VCF SSO identity providers, with configurable authentication mechanisms.

including redirections to external identity providers. Imported users log in with the credentials established in their OIDC identity provider. The vCenter Client Credentials grant type allows application-to-identity-provider authentication without a human user present, supporting machine-to-machine authentication scenarios.

For environments using Microsoft Active Directory, organizations can use user identities contained in a Windows Server domain controller or Active Directory service across their virtual infrastructure, providing another path for centrally managing third-party user identities through an established directory service. VCF SSO also supports user and group import for access control from configured identity sources. VMware ESX and vCenter validate user credentials against configured identity stores.

For programmatic and service-to-service access, Principal Identity users authenticate using certificates, providing unique identification and auditability for automated processes. A Principal Identity user's certificate must be SHA256 based. NSX Federation creates dedicated Principal Identity users with specific roles (such as Auditor) on each Global Manager when a Local Manager is registered, demonstrating how even automated inter-component communication uses uniquely identifiable accounts. The VCF Automation API supports endpoints for retrieving organizational context for authenticated users, and the Identity API authenticates and manages the authorization provider, giving programmatic control over identity and access management. Valid session identifiers establish trust with API endpoints, providing traceability for each authenticated session.

For non-human processes running inside VKS clusters, Kubernetes ServiceAccounts provide a distinct, namespaced identity separate from human user accounts. Each ServiceAccount authenticates with the username `system:serviceaccount:(NAMESPACE):(SERVICEACCOUNT)` and is assigned to namespace-scoped groups, enabling RBAC policies to authorize specific API operations without sharing human user credentials. Scoping IAM role trust policies to specific ServiceAccounts and namespaces enforces least privilege for workloads that access external services. Where VKS clusters integrate with AWS cloud services, IAM Roles for Service Accounts (IRSA) replaces long-lived static access keys with an OIDC-based identity federation model that provides workloads with short-lived, least-privileged credentials on demand. Separating ServiceAccount creation from human user onboarding supports differentiated audit strategies, since audit events carry the ServiceAccount identity rather than a generic shared credential. Third-party security software running in the cluster can use ServiceAccount identities to group pods into distinct security contexts for access control purposes. ArgoCD, deployable as a VCF Service on VKS clusters, supports OIDC authentication with global RBAC policies for federated identity and role-based access control, with both OIDC and local account authentication available depending on deployment requirements.

On the authorization side, VCF enforces role-based access control where authenticated users can view objects or invoke operations on the server depending on the permissions associated with their account. The vCenter authorization model provides consistent security control regardless of how users connect to the environment. Organization owners and Enterprise Admins can add, manage, and remove users, controlling what level of access each user receives. NSX Manager local user accounts are created with a default Auditor role, and role assignments can be changed as needed. NSX authentication best practices include defining access controls based on user roles and permissions and integrating with identity providers.

Each organization within VCF Automation has its own identity source and organization portal where users log in to consume resources and services. This multi-tenant isolation helps organizations maintain separation between internal and third-party user populations while still leveraging centralized identity infrastructure.

Audit logging captures authentication and authorization events across VCF components, providing a record of third-party user activity for review and compliance purposes. All API calls to NSX Manager are subject to authentication and authorization and generate audit logs. NSX generates audit logs for events such as logon, logoff, and access to resources, with each audit log containing the timestamp, source, result, and a description of the event. NSX Manager and NSX Edge both include a dedicated audit role with separate credentials; the NSX Manager audit user has read-only privileges to the NSX environment and is not active by default, requiring explicit activation after installation by an admin through the CLI. This separation of the audit role from administrative roles supports segregation of duties for audit functions. VCF Operations generates audit events across categories including access, access control, account management, configuration, data access, network, notification, permissions, session, system, policy, and firewall actions, which can be filtered, searched, and reviewed in detail. VCF Operations Orchestrator tracks the user who initiated each workflow run, providing audit accountability for automated management tasks executed against the platform. At the Kubernetes layer, the cluster API server records authentication



events, API requests, and authorization decisions for all users and ServiceAccounts. Kubernetes audit annotations include `authorization.k8s.io/decision` to record each authorization decision; for OIDC-authenticated third-party users, audit entries capture the username and groups asserted by the identity provider, enabling traceability back to the individual; for ServiceAccounts, entries capture the service account name and namespace. Organizations can configure the Kubernetes audit policy to define which request types and API verbs are logged and at what verbosity, and route those events to a centralized log collector.

While VCF provides the technical infrastructure for centralized AAA of third-party users, the organization is responsible for defining policies that govern which third-party users receive access, establishing onboarding and offboarding procedures, conducting periodic access reviews, configuring the external identity provider with appropriate authentication strength requirements, and determining which AAA solutions to approve and how to handle AAA for external service providers.

VMware Private AI Services (PAIS) supports identification and access management for third-party and non-organizational users of AI services through OpenID Connect (OIDC) integration. PAIS activation requires coordination with the OIDC provider administrator, and the PAISConfiguration resource specifies an issuer URL, an OIDC client ID (the `pais-authentication-id` value provided by the OIDC administrator), and the scopes `openid`, `groups`, and `offline_access`. The OIDC client ID, the groups claim name, and additional accepted OAuth 2.0 audiences are individually configurable so that PAIS can interoperate with an external provider's existing tenant model rather than requiring a PAIS-specific identity store.

Authorization for PAIS is enforced through OIDC group membership. The `authorizedGroups` setting in the PAISConfiguration restricts access to user accounts whose OIDC Access Token includes membership in one or more designated groups, with example group names such as `group-for-pais-access-01`. The groups claim is included in the Access Token, and a corresponding groups scope is created in the OIDC configuration. This is the primary PAIS-native mechanism for uniquely identifying non-organizational users by OIDC identity claims and authorizing their access based on group membership in the external provider.

The Grafana observability interface for PAIS authenticates through the same OIDC provider, so visualization of PAIS metrics for monitoring and audit purposes is gated by the same identity and group controls. PAIS metrics can be exported to observability platforms to support monitoring and audit review of third-party user activity. Audit recording of authentication and authorization events at the platform layer is provided by the underlying VCF infrastructure rather than by PAIS-native audit trails dedicated to non-organizational user activity.

For third-party data sources and registries, PAIS Knowledge Bases authenticate to Confluence using either a user name and password or a personal access token, and to Microsoft SharePoint On-Premises using a user name and password. PAIS integrates with Harbor registries as a model storage backend, requiring authentication and certificate-based trust for model retrieval, and the `oras login` command authenticates to a private Harbor registry using user name and password credentials for catalog and image operations. These mechanisms address the identification and access control dimensions of the control for the third-party services that PAIS itself reaches out to.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) supports authentication and authorization of users who access database services, including third-party users and service accounts that connect to managed databases. DSM's identity mechanisms apply to all user types, and authentication controls can be configured to restrict access based on identity, source address, and authentication method.

For administrative access to the DSM management plane, DSM supports both local user accounts and LDAPS integration with an enterprise directory service (LDAP or Active Directory). An installation can choose to use one or both identity sources, and DSM manages them independently. DSM can be configured to use an LDAP server for both authentication and authorization. When Directory Service credentials are unavailable, DSM falls back to local user authentication.

For PostgreSQL database instances, DSM provides host-based authentication (HBA) configuration. The `pg_hba.conf` file controls which clients may connect to a database by specifying connection type, client address, requested database, and user name. Each connection attempt is matched against an ordered list of rules, and the first matching record determines the authentication method applied. Default entries use SCRAM-SHA-256 for local accounts including the `pgadmin` and

postgres\_exporter service accounts, and administrators can add LDAP entries to authenticate database users through Active Directory. Database instances can also be configured to use directory service authentication after initial creation using the directory-service-authentication setting. MySQL instances similarly support Active Directory authentication via LDAPS.

DSM adds SQL Server as a managed database type with its own authentication model. SQL Server clusters use contained database users, where users authenticate directly against the database and identity information resides within the database rather than at a server level. SQL Server clusters can be integrated with Active Directory, enabling Kerberos-based authentication and Windows Authentication for server-level logins. Windows Principals can be authorized as cluster administrators through T-SQL or the DSM API. Active Directory integration also supports single sign-on for clients that use Kerberos authentication, such as SQL Server Management Studio running on a Windows machine joined to the same domain. SQL Server Authentication logins can be created for clusters that do not require Windows Authentication. DSM supports two primary administrator user identities for SQL Server: SQL User or Windows Principal, with Windows Principal available for clusters integrated with Active Directory.

Third-party processes and service integrations that receive DSM event notifications through webhooks are authenticated using credentials. DSM webhooks support basic authentication with an Authorization User and Authorization Password, restricting webhook delivery to authorized endpoints.

Metrics forwarding to Prometheus-compatible endpoints supports configurable authentication, including certificate-only, basic authentication, or a combination, allowing organizations to control which external monitoring systems can pull DSM metrics.

The DSM Consumption Operator, which enables self-service database provisioning through Kubernetes-native interfaces, authenticates with DSM infrastructure using an authentication secret named dsm-auth-creds.

DSM audit logs track user actions performed through the UI and API, recording the source of each operation as either a user identifier (Email ID) for user-initiated events or System for events initiated by DSM without user intervention. For SQL Server clusters, DSM provides configurable server auditing with preset audits for login tracking and log retention, and can create a simple login event audit based on the Login Auditing option selection.

DSM does not itself categorize users as organizational versus non-organizational. Meeting this control fully requires organizational processes beyond what DSM provides: identifying which users or processes are third-party, ensuring they receive appropriately scoped accounts or directory entries, and reviewing audit logs to confirm that third-party access remains within authorized boundaries. DSM provides the authentication and access control mechanisms that support those organizational processes.

### **VMware Avi Load Balancer (Contributes)**

Avi Load Balancer provides authentication and authorization mechanisms across three planes: the administrative control plane, the License Hub component, and the application data plane.

For the administrative control plane, Avi supports federated authentication by delegating identity verification to external AAA servers rather than managing credentials locally. The Controller's Authentication Profile can be configured with LDAP or SAML providers, allowing remote users to authenticate against a central directory or identity service. Authorization Mapping Profiles provide dynamic tenant and role assignment during login using group membership evaluation, so group memberships retrieved from an LDAP server translate directly into Controller permissions without manual per-user assignment. HTTP authentication on virtual service policies supports a Required User Group Membership configuration that restricts access to members of specific groups.

The Avi License Hub includes its own authentication layer supporting Active Directory and OpenLDAP as identity sources for user authentication. License Hub also maintains a dedicated local Audit account with the auditor role, which has full access to APIs and services for system auditing, and a local Admin account responsible for user creation, authorization, and tenant management.

For the application data plane, Avi virtual services can enforce client authentication before traffic reaches back-end workloads. OAuth 2.0 provides secure delegated access where an authorization server issues tokens to clients on behalf of resource

owners; Avi validates identity by consulting the UserInfo endpoint or reading claims from the ID token when configured for OIDC. OAuth is the recommended approach when third-party applications need access, as it avoids storing resource owner credentials in clear text on those applications. SAML securely verifies user identity without storing credentials on multiple untrusted devices. DataScript can check whether a user ID appears in an approved group list and redirect unauthenticated users to a login page, enabling identity-based access control at the proxy layer.

In Kubernetes environments managed through the Avi Kubernetes Operator (AKO), the SSORule custom resource definition configures either OAuth/OIDC properties or SAML service provider properties along with the SSO policy, extending centralized authentication to Kubernetes ingress virtual services. Avi GSLB supports creation of non-admin users with assigned roles, providing granular access control for global server load balancing operations.

Together these mechanisms support the authentication and authorization dimensions the control requires. Authentication is delegated to a central identity provider; authorization is enforced through role and tenant mapping at login; and Avi's audit and event logging captures administrative and configuration activity. For the organizational dimension of this control, the external identity providers must be deployed and operated by the organization, role mapping rules must be defined for each population of third-party users, and the governance program determining which third parties are authorized to access services must be maintained separately.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 1.1(2)

> Human user identification and authentication: Provide the capability to identify and authenticate all human users on all interfaces capable of human user access. Unique identification and authentication: Capability to uniquely identify and authenticate all human users. Multifactor authentication for untrusted networks: Capability to employ multi-factor authentication for all human user access from an untrusted network. Authenticator feedback: Obscure feedback of authentication information (e.g., asterisks) during entry. Unsuccessful login attempts: Enforce a limit of consecutive invalid login attempts before lockout or delay. System use notification: Display a notification message before establishing user identification. Access via untrusted networks: Monitor and control all access attempts from untrusted networks. Strength of symmetric key-based authentication: Capability to manage symmetric keys securely. Symmetric key protection: Hardware-based protection for symmetric keys.

**SCF Controls:** IAC-06

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 1.2

> Software process and device identification and authentication: Capability to identify and authenticate all software processes and devices. Unique identification and authentication: Capability to uniquely identify and authenticate all software processes and devices.

**SCF Controls:** IAC-04

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides multiple mechanisms for uniquely identifying and authenticating devices, services, and infrastructure components before establishing connections, using cryptographically-based bidirectional authentication.

VCF SSO is the centralized authentication, authorization, and auditing backbone across the platform. VCF SSO supports both SAML and OIDC authentication protocols, which incorporate replay resistance through nonces, timestamps, and token expiration. Organizations can integrate external identity providers such as Microsoft Entra ID or Ping Identity through these protocols, enabling centralized identity management for both users and service accounts. VCF Operations, VCF Log Management, and other platform components integrate with VCF SSO, providing a single point of AAA policy enforcement across the environment. VMware Kubernetes Service (VKS) clusters running on vSphere Supervisor support both VCF SSO and OIDC-compliant external identity providers for cluster authentication. Supervisor uses Pinnipd to broker OAuth 2.0-based user authentication and group membership resolution with external providers, and cluster access is distributed via kubeconfig files that carry the federated identity configuration. The Kubernetes JWT authenticator verifies that tokens presented to the API server have been signed by the configured issuer and resolves the public signing key from the issuer's OIDC discovery endpoint; multiple JWT issuers can be configured to support multiple identity providers simultaneously.

VMware ESX hosts authenticate to VMware vCenter using cryptographic credentials during the host addition process. Each ESX host has a unique machine SSL certificate issued by a Certificate Authority, and the CommonName in the certificate must match the host's FQDN or IP address used to add it to the vCenter system. vCenter validates this certificate before accepting the host into the managed inventory, and the root certificate of the CA must be present in the trusted root store of both vCenter and connected ESX hosts. This mutual TLS authentication is bidirectional: the host verifies it is communicating with a legitimate vCenter, and vCenter verifies the host's identity through its certificate.

For machine-to-machine communication, NSX supports Principal Identity certificates as the preferred approach for authenticating automated clients and service integrations. Any NSX Manager client can create and use a Principal Identity certificate, providing certificate-based mutual authentication that is inherently bidirectional and cryptographically-based. NSX requires authentication prior to establishing any management connection for administrative access. For network-layer device authentication, NSX BGP neighbor communication can be configured with password-based authentication to verify the identity of routing peers before establishing sessions.

VCF enforces TLS as the transport security protocol for inter-component communication. Components such as Telegraf are configured with a minimum TLS version that provides bidirectional authentication through the TLS handshake, cryptographic key exchange, and built-in replay protection. vCenter supports holder-of-key security token authentication through the LoginByToken method, which binds the authentication token to a specific cryptographic key held by the client, providing proof of possession that is resistant to token replay. Valid session identifiers in vSphere verify the identity of principals and establish trust with API endpoints.

Within the Consumption layer of the VCF platform, VKS clusters on vSphere Supervisor implement bidirectional cryptographic authentication between all Kubernetes cluster components. Each kubelet presents an X.509 client certificate to the Kubernetes API server during connection establishment, and the API server presents its own certificate in return, forming a mutual TLS handshake that is cryptographically based and replay resistant. Operators can restrict the cipher suites accepted for API server connections using the kube-apiserver `--tls-cipher-suites` parameter, set a minimum protocol version using `--tls-min-version`, and apply equivalent constraints at the node level via the KubeletConfiguration `tlsCipherSuites` and `tlsMinVersion` fields. The Kubernetes Scheduler exposes an equivalent `--tls-cipher-suites` parameter for its own API surface. VKS clusters

include TLS certificates supporting cryptographic authentication among all cluster components, and the guidance recommends configuring short certificate lifetimes and automating rotation to reduce the exposure window for any compromised credential. For ingress, the Contour ingress controller available as a standard VKS package supports a configurable `tls.minimum-protocol-version` parameter. For service-to-service authentication within VKS workload clusters, Istio is available as a standard package and enforces mutual TLS between services using X.509 certificates issued by Istio's built-in certificate authority; the minimum TLS protocol version for the mesh is configurable via `meshConfig.meshMTLS.minProtocolVersion`, which defaults to TLS 1.2 and supports TLS 1.3. Antrea, the default CNI for Supervisor, supports configurable TLS cipher suites for its own control-plane communication. The Supervisor secret injection agent automatically configures mutual TLS for communication with the vault-agent-injector service using auto-generated certificates. For environments subject to strict cryptographic requirements, VKS ClusterClass variables support enabling FIPS mode via the `fips.enabled` setting, which restricts the cluster to FIPS-validated cryptographic modules for all component communication. Harbor, available as a Supervisor service, presents a TLS certificate to clients before any image pull or push operation, providing cryptographic authentication for the container registry endpoint.

Network devices discovered and managed through VCF Operations are authenticated using credentials appropriate to their connection type. VCF Operations supports SSH, SNMP v2, SNMPv3, and API-based access credential types for device discovery and ongoing management. SNMPv3 includes authentication and privacy parameters that provide cryptographic verification of device identity. VCF Operations also supports detection and remediation of default manufacturer passwords on network devices, helping to ensure that devices are uniquely authenticated rather than relying on shared default credentials.

For storage devices, VCF supports mutual CHAP authentication, which provides bidirectional cryptographic verification between initiators and storage targets before establishing a connection. The vSAN iSCSI target service supports both CHAP and Mutual CHAP authentication, and the vSphere API provides the `UpdateInternetScsiAuthenticationProperties` method for configuring iSCSI authentication parameters.

The cryptographic foundation underpinning these authentication mechanisms uses FIPS 140-3 validated modules. NSX is configured to use FIPS 140-3 validated cryptographic modules by default, with validation performed through the NIST Cryptographic Module Validation Program. VCF Automation uses the BouncyCastle FIPS 2.0.0 cryptographic module, which is compliant with FIPS 140-3. This validated cryptographic foundation supports the integrity of all certificate-based and token-based authentication operations across the platform. The vCenter appliance uses VMware's Boring Crypto Module version 6.0 and the OpenSSL FIPS Object Module version 3.0, both FIPS-validated, supporting algorithms including AES, ECDSA, HMAC, RSA, and SHA variants for cryptographic operations. Starting in vSphere 9.0, all TLS profiles are FIPS compliant, and the COMPATIBLE profile supports modern TLS cipher suites and elliptic curves. For VKS clusters, FIPS mode can be enabled via the `fips.enabled` ClusterClass variable, which restricts the cluster to FIPS-validated cryptographic modules for all component communication.

VCF Automation also provides a device authorization OAuth endpoint that implements the OAuth 2.0 Device Authorization Grant flow. This endpoint accepts a client identifier and returns a device code, user code, verification URI, expiration time, and polling interval, enabling devices with limited input capability to authenticate through a secure, time-bounded authorization process.

### **VMware vDefend (Contributes)**

VMware vDefend contributes to device authentication through its NDR Sensor onboarding process and ongoing sensor-to-platform communication. NDR Sensors connect to Security Services Platform using Mutual TLS (mTLS), providing bidirectional, cryptographically-based authentication before establishing the management connection. During initial registration, Security Services Platform generates a unique registration token that is valid for 30 minutes; the sensor presents this time-limited token as part of the mTLS-secured registration sequence, providing replay resistance for the onboarding step. Security Services Platform and NDR Sensor components implement FIPS 140-2 and FIPS 140-3 validated cryptographic modules for the underlying cryptographic operations, including the VMware BouncyCastle Module (Certificate #4943) and the VMware OpenSSL FIPS provider Object Module (Certificate #4861).

Beyond this sensor authentication, vDefend does not provide centralized AAA for general network devices. vDefend enforces policy on traffic flows based on identity, application, and network context, but it does not issue device certificates or manage a device identity registry for arbitrary network endpoints. That capability falls outside vDefend's scope and would be addressed by network infrastructure and PKI controls at the platform level.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) provides cryptographic, replay-resistant authentication for its own inter-site components. When two sites are paired for recovery operations, PNR validates the identity of each peer through certificate-based authentication and supports importing certificate-authority-signed server certificates through the PNR Appliance Management Interface. Site pairing, site reconnection, and site pair break operations all require credential authentication, and the PNR Appliance Management Interface itself requires authentication as the admin user for service configuration, network configuration, hostname changes, and disconnection from the PNR Global Console.

For inter-site service authentication, PNR servers acquire a holder-of-key SAML token from VCF SSO using token-based authentication, then send cryptographically signed requests carrying that token to remote services. The remote service validates the token to establish the identity of the calling service before the connection proceeds, supplying bidirectional, replay-resistant authentication between PNR components without transmitting passwords across site boundaries. The VMware vCenter appliance handles authentication between the PNR server and vCenter at the VCF SSO level, and in an Enhanced Linked Mode environment the remote site reuses the same service account to authenticate to local services.

PNR also implements an OAuth 2.0 flow for remote site authentication, with a configurable authorization request lifetime through the `oauth2.authRequestLifetime` setting (default 60 seconds). The Cyber Recovery Connector authenticates with an OAuth app secret that Broadcom rotates every three months. Management communications are carried over TLS, with the minimum protocol version configurable through the `envoy-proxy-config.json` file. The PNR user interface maintains logs that record certificate validation errors and connection failures during authentication attempts, supporting the audit dimension of AAA for PNR's own management traffic.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer contributes to device identification and authentication at the application delivery layer through configurable SSL/TLS profiles, FIPS 140-3 mode cryptographic enforcement, Hardware Security Module (HSM) integration for hardware-rooted key protection, and support for federated client authentication protocols including SAML, OAuth, and OpenID Connect (OIDC).

Avi's SSL/TLS Profile mechanism, configured under **Templates > Security > SSL/TLS Profile**, defines accepted protocol versions and cipher suites for each virtual service. Administrators can restrict accepted options to those meeting organizational policy for strong, replay-resistant authentication. In 9.1, the SSL Profile includes post-quantum cryptography options for TLS 1.3, providing a path to quantum-resistant authentication at the application delivery boundary. The same profile mechanism applies to the Controller's own management-plane connections, giving administrators a consistent control point for the cryptographic posture of both the data plane and the management interface.

When FIPS mode is enabled on the Avi Controller, all cryptographic operations are restricted to FIPS 140-3 approved algorithms. The Avi Controller uses an OpenSSL FIPS Object Module validated at FIPS 140-3 Security Level 1 (CMVP Certificate #4985). FIPS mode is enabled at the Controller cluster level and scopes cipher suite choices for all Service Engine connections, satisfying the cryptographically-based and replay-resistant dimension of the control for traffic processed at the Avi layer.

For hardware-backed device authentication, Avi Service Engines support integration with Hardware Security Modules, including Thales Luna HSM, through HSM Groups configured under **Templates > Security > HSM Groups**. The Avi Controller performs mutual authentication with HSM devices using client certificate registration and verification, providing hardware-rooted key storage for SSL certificates and supporting the bidirectional authentication requirement.

At the application delivery layer, Avi supports client authentication through SAML 2.0 (with identity providers such as Okta and PingFederate), OAuth 2.0, and OIDC, configured through SSO Policies referencing authentication profiles. These



mechanisms allow Avi virtual services to enforce identity verification before forwarding connections, supporting the Authenticate and Authorize dimensions of the control for workloads behind Avi. LDAP-based federated authentication is also available for environments using directory-integrated identity.

Avi's audit logging infrastructure records authentication failures for SSH and TLS sessions, logging and persisting those records for auditing purposes, covering the Audit dimension of the AAA requirement for authentication events that pass through Avi.

The central device identity infrastructure, including the PKI, certificate authority, and device enrollment and management functions, is provided by VCF rather than by Avi itself. Avi validates certificates at the application proxy boundary but does not operate the device identity registry or provide the central AAA framework for the broader infrastructure. Organizations must ensure that the PKI and device identity management components in VCF are configured to issue and manage certificates for devices connecting through Avi virtual services.

Not applicable for this control: VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.



## Control CR 1.2(1)

> Software process and device identification and authentication: Capability to identify and authenticate all software processes and devices. Unique identification and authentication: Capability to uniquely identify and authenticate all software processes and devices.

**SCF Controls:** IAC-04

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides multiple mechanisms for uniquely identifying and authenticating devices, services, and infrastructure components before establishing connections, using cryptographically-based bidirectional authentication.

VCF SSO is the centralized authentication, authorization, and auditing backbone across the platform. VCF SSO supports both SAML and OIDC authentication protocols, which incorporate replay resistance through nonces, timestamps, and token expiration. Organizations can integrate external identity providers such as Microsoft Entra ID or Ping Identity through these protocols, enabling centralized identity management for both users and service accounts. VCF Operations, VCF Log Management, and other platform components integrate with VCF SSO, providing a single point of AAA policy enforcement across the environment. VMware Kubernetes Service (VKS) clusters running on vSphere Supervisor support both VCF SSO and OIDC-compliant external identity providers for cluster authentication. Supervisor uses Pinnipd to broker OAuth 2.0-based user authentication and group membership resolution with external providers, and cluster access is distributed via kubeconfig files that carry the federated identity configuration. The Kubernetes JWT authenticator verifies that tokens presented to the API server have been signed by the configured issuer and resolves the public signing key from the issuer's OIDC discovery endpoint; multiple JWT issuers can be configured to support multiple identity providers simultaneously.

VMware ESX hosts authenticate to VMware vCenter using cryptographic credentials during the host addition process. Each ESX host has a unique machine SSL certificate issued by a Certificate Authority, and the CommonName in the certificate must match the host's FQDN or IP address used to add it to the vCenter system. vCenter validates this certificate before accepting the host into the managed inventory, and the root certificate of the CA must be present in the trusted root store of both vCenter and connected ESX hosts. This mutual TLS authentication is bidirectional: the host verifies it is communicating with a legitimate vCenter, and vCenter verifies the host's identity through its certificate.

For machine-to-machine communication, NSX supports Principal Identity certificates as the preferred approach for authenticating automated clients and service integrations. Any NSX Manager client can create and use a Principal Identity certificate, providing certificate-based mutual authentication that is inherently bidirectional and cryptographically-based. NSX requires authentication prior to establishing any management connection for administrative access. For network-layer device authentication, NSX BGP neighbor communication can be configured with password-based authentication to verify the identity of routing peers before establishing sessions.

VCF enforces TLS as the transport security protocol for inter-component communication. Components such as Telegraf are configured with a minimum TLS version that provides bidirectional authentication through the TLS handshake, cryptographic key exchange, and built-in replay protection. vCenter supports holder-of-key security token authentication through the LoginByToken method, which binds the authentication token to a specific cryptographic key held by the client, providing proof of possession that is resistant to token replay. Valid session identifiers in vSphere verify the identity of principals and establish trust with API endpoints.

Within the Consumption layer of the VCF platform, VKS clusters on vSphere Supervisor implement bidirectional cryptographic authentication between all Kubernetes cluster components. Each kubelet presents an X.509 client certificate to the Kubernetes API server during connection establishment, and the API server presents its own certificate in return, forming a mutual TLS handshake that is cryptographically based and replay resistant. Operators can restrict the cipher suites accepted for API server connections using the kube-apiserver `--tls-cipher-suites` parameter, set a minimum protocol version using `--tls-min-version`, and apply equivalent constraints at the node level via the KubeletConfiguration `tlsCipherSuites` and `tlsMinVersion` fields. The Kubernetes Scheduler exposes an equivalent `--tls-cipher-suites` parameter for its own API surface. VKS clusters

include TLS certificates supporting cryptographic authentication among all cluster components, and the guidance recommends configuring short certificate lifetimes and automating rotation to reduce the exposure window for any compromised credential. For ingress, the Contour ingress controller available as a standard VKS package supports a configurable `tls.minimum-protocol-version` parameter. For service-to-service authentication within VKS workload clusters, Istio is available as a standard package and enforces mutual TLS between services using X.509 certificates issued by Istio's built-in certificate authority; the minimum TLS protocol version for the mesh is configurable via `meshConfig.meshMTLS.minProtocolVersion`, which defaults to TLS 1.2 and supports TLS 1.3. Antrea, the default CNI for Supervisor, supports configurable TLS cipher suites for its own control-plane communication. The Supervisor secret injection agent automatically configures mutual TLS for communication with the vault-agent-injector service using auto-generated certificates. For environments subject to strict cryptographic requirements, VKS ClusterClass variables support enabling FIPS mode via the `fips.enabled` setting, which restricts the cluster to FIPS-validated cryptographic modules for all component communication. Harbor, available as a Supervisor service, presents a TLS certificate to clients before any image pull or push operation, providing cryptographic authentication for the container registry endpoint.

Network devices discovered and managed through VCF Operations are authenticated using credentials appropriate to their connection type. VCF Operations supports SSH, SNMP v2, SNMPv3, and API-based access credential types for device discovery and ongoing management. SNMPv3 includes authentication and privacy parameters that provide cryptographic verification of device identity. VCF Operations also supports detection and remediation of default manufacturer passwords on network devices, helping to ensure that devices are uniquely authenticated rather than relying on shared default credentials.

For storage devices, VCF supports mutual CHAP authentication, which provides bidirectional cryptographic verification between initiators and storage targets before establishing a connection. The vSAN iSCSI target service supports both CHAP and Mutual CHAP authentication, and the vSphere API provides the `UpdateInternetScsiAuthenticationProperties` method for configuring iSCSI authentication parameters.

The cryptographic foundation underpinning these authentication mechanisms uses FIPS 140-3 validated modules. NSX is configured to use FIPS 140-3 validated cryptographic modules by default, with validation performed through the NIST Cryptographic Module Validation Program. VCF Automation uses the BouncyCastle FIPS 2.0.0 cryptographic module, which is compliant with FIPS 140-3. This validated cryptographic foundation supports the integrity of all certificate-based and token-based authentication operations across the platform. The vCenter appliance uses VMware's Boring Crypto Module version 6.0 and the OpenSSL FIPS Object Module version 3.0, both FIPS-validated, supporting algorithms including AES, ECDSA, HMAC, RSA, and SHA variants for cryptographic operations. Starting in vSphere 9.0, all TLS profiles are FIPS compliant, and the COMPATIBLE profile supports modern TLS cipher suites and elliptic curves. For VKS clusters, FIPS mode can be enabled via the `fips.enabled` ClusterClass variable, which restricts the cluster to FIPS-validated cryptographic modules for all component communication.

VCF Automation also provides a device authorization OAuth endpoint that implements the OAuth 2.0 Device Authorization Grant flow. This endpoint accepts a client identifier and returns a device code, user code, verification URI, expiration time, and polling interval, enabling devices with limited input capability to authenticate through a secure, time-bounded authorization process.

### **VMware vDefend (Contributes)**

VMware vDefend contributes to device authentication through its NDR Sensor onboarding process and ongoing sensor-to-platform communication. NDR Sensors connect to Security Services Platform using Mutual TLS (mTLS), providing bidirectional, cryptographically-based authentication before establishing the management connection. During initial registration, Security Services Platform generates a unique registration token that is valid for 30 minutes; the sensor presents this time-limited token as part of the mTLS-secured registration sequence, providing replay resistance for the onboarding step. Security Services Platform and NDR Sensor components implement FIPS 140-2 and FIPS 140-3 validated cryptographic modules for the underlying cryptographic operations, including the VMware BouncyCastle Module (Certificate #4943) and the VMware OpenSSL FIPS provider Object Module (Certificate #4861).

Beyond this sensor authentication, vDefend does not provide centralized AAA for general network devices. vDefend enforces policy on traffic flows based on identity, application, and network context, but it does not issue device certificates or manage a device identity registry for arbitrary network endpoints. That capability falls outside vDefend's scope and would be addressed by network infrastructure and PKI controls at the platform level.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) provides cryptographic, replay-resistant authentication for its own inter-site components. When two sites are paired for recovery operations, PNR validates the identity of each peer through certificate-based authentication and supports importing certificate-authority-signed server certificates through the PNR Appliance Management Interface. Site pairing, site reconnection, and site pair break operations all require credential authentication, and the PNR Appliance Management Interface itself requires authentication as the admin user for service configuration, network configuration, hostname changes, and disconnection from the PNR Global Console.

For inter-site service authentication, PNR servers acquire a holder-of-key SAML token from VCF SSO using token-based authentication, then send cryptographically signed requests carrying that token to remote services. The remote service validates the token to establish the identity of the calling service before the connection proceeds, supplying bidirectional, replay-resistant authentication between PNR components without transmitting passwords across site boundaries. The VMware vCenter appliance handles authentication between the PNR server and vCenter at the VCF SSO level, and in an Enhanced Linked Mode environment the remote site reuses the same service account to authenticate to local services.

PNR also implements an OAuth 2.0 flow for remote site authentication, with a configurable authorization request lifetime through the `oauth2.authRequestLifetime` setting (default 60 seconds). The Cyber Recovery Connector authenticates with an OAuth app secret that Broadcom rotates every three months. Management communications are carried over TLS, with the minimum protocol version configurable through the `envoy-proxy-config.json` file. The PNR user interface maintains logs that record certificate validation errors and connection failures during authentication attempts, supporting the audit dimension of AAA for PNR's own management traffic.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer contributes to device identification and authentication at the application delivery layer through configurable SSL/TLS profiles, FIPS 140-3 mode cryptographic enforcement, Hardware Security Module (HSM) integration for hardware-rooted key protection, and support for federated client authentication protocols including SAML, OAuth, and OpenID Connect (OIDC).

Avi's SSL/TLS Profile mechanism, configured under **Templates > Security > SSL/TLS Profile**, defines accepted protocol versions and cipher suites for each virtual service. Administrators can restrict accepted options to those meeting organizational policy for strong, replay-resistant authentication. In 9.1, the SSL Profile includes post-quantum cryptography options for TLS 1.3, providing a path to quantum-resistant authentication at the application delivery boundary. The same profile mechanism applies to the Controller's own management-plane connections, giving administrators a consistent control point for the cryptographic posture of both the data plane and the management interface.

When FIPS mode is enabled on the Avi Controller, all cryptographic operations are restricted to FIPS 140-3 approved algorithms. The Avi Controller uses an OpenSSL FIPS Object Module validated at FIPS 140-3 Security Level 1 (CMVP Certificate #4985). FIPS mode is enabled at the Controller cluster level and scopes cipher suite choices for all Service Engine connections, satisfying the cryptographically-based and replay-resistant dimension of the control for traffic processed at the Avi layer.

For hardware-backed device authentication, Avi Service Engines support integration with Hardware Security Modules, including Thales Luna HSM, through HSM Groups configured under **Templates > Security > HSM Groups**. The Avi Controller performs mutual authentication with HSM devices using client certificate registration and verification, providing hardware-rooted key storage for SSL certificates and supporting the bidirectional authentication requirement.

At the application delivery layer, Avi supports client authentication through SAML 2.0 (with identity providers such as Okta and PingFederate), OAuth 2.0, and OIDC, configured through SSO Policies referencing authentication profiles. These

mechanisms allow Avi virtual services to enforce identity verification before forwarding connections, supporting the Authenticate and Authorize dimensions of the control for workloads behind Avi. LDAP-based federated authentication is also available for environments using directory-integrated identity.

Avi's audit logging infrastructure records authentication failures for SSH and TLS sessions, logging and persisting those records for auditing purposes, covering the Audit dimension of the AAA requirement for authentication events that pass through Avi.

The central device identity infrastructure, including the PKI, certificate authority, and device enrollment and management functions, is provided by VCF rather than by Avi itself. Avi validates certificates at the application proxy boundary but does not operate the device identity registry or provide the central AAA framework for the broader infrastructure. Organizations must ensure that the PKI and device identity management components in VCF are configured to issue and manage certificates for devices connecting through Avi virtual services.

Not applicable for this control: VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 1.3

> Account management: Support management of all accounts by authorized users (adding, activating, modifying, disabling, removing).

**SCF Controls:** IAC-01

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides a layered identity and access management architecture spanning centralized identity services, role-based access control, group-based access policies, identity provider federation, and programmatic access management through APIs.

VCF SSO is the centralized authentication service. VCF SSO issues security tokens that grant authenticated users access to VMware vCenter, NSX Manager, VCF Operations, and other platform components without requiring separate credentials for each service. VCF Operations supports importing users and groups from VCF SSO for consistent identity management across the stack. VMware Identity Manager provides a platform for managing users and groups, managing resources and user authentication, and configuring access policies and resource entitlements. vCenter also supports built-in identity source authentication using local domain credentials. The built-in identity provider can also be configured to use Active Directory as an identity source through LDAP/S, OpenLDAP/S, or Integrated Windows Authentication (IWA), depending on the directory environment.

VCF supports federation with enterprise identity providers. vCenter can be federated to providers such as Microsoft Entra ID, ADFS, and Okta, which enables multifactor authentication and automatic user synchronization with the organization's directory. PingFederate is also supported as a federated identity provider; for these integrations, vCenter uses System for Cross-domain Identity Management (SCIM) to automate the exchange of user and group identity information between the external identity domain and vCenter. VMware Identity Broker allows principals to authenticate to vCenter through an external identity provider. NSX Manager can be configured to authenticate users from identity management providers and can be integrated with Workspace ONE Access for identity management services. The Supervisor component supports registration of OIDC-compliant external identity providers for authentication to VMware Kubernetes Service (VKS) clusters; supported providers include Okta, Workspace ONE Access, and Dex, configured by supplying the Supervisor callback URL to the identity provider.

Role-based access control is enforced across all VCF components. In vCenter, permissions associate a user or group with a role on a specific inventory object, and predefined roles cover common administrative functions. Custom roles can be created with only the minimum required privileges for a given workflow. Managing identity provider configuration in vCenter requires the VcIdentityProviders Manage privilege. VCF Operations supports group-based access control, allowing administrators to configure groups and assign roles that control access to dashboards, reports, policies, alerts, and administration functions, as well as granular role-based controls for platform operations including approvals, audits, certificates, and content libraries.

VCF Automation extends access management through organization-level and project-level controls. The VCF Automation Access Control service supports identity operations for managing groups and users, including editing user names, passwords, and role assignments. The organization portal enforces role-based access control, requiring either Organization Administrator or Organization Auditor roles to manage policy templates. Organization administrators can distribute infrastructure and configure group-level access through the organization management functionality. Project access is governed by predefined roles. Users and groups can be disabled at the organization level to revoke access without deleting accounts. The System Administrator role has the right to manage and view Access Control Lists, and the accessControls API allows administrators to view and manage ACL entries for defined entities.

For Kubernetes workloads within the VCF consumption layer, VKS supports two authentication methods: VCF SSO and OIDC-compliant external identity providers. The OIDC path is implemented through Pinniped and supports providers including Okta, Workspace ONE Access, Dex, GitLab, and Google OAuth. Configuring an external provider requires setting up an OpenID Connect ID Token with a Groups claim type filter to enable group-based access control tied to the organization's directory.

Custom ClusterClass configurations allow clusters to authenticate developers using existing corporate identity systems. VKS supports namespace permission roles: Owner (full control including deletion), Can Edit (provisioning and operating workloads), and Can View (read-only access). These roles map to Kubernetes ClusterRoles through RoleBindings, and RoleBinding objects can reference VCF SSO users and groups directly to integrate SSO identities with Kubernetes role-based access control. A user with the Namespaces.Edit privilege on a user-created Namespace folder can perform CRUD operations on Kubernetes API resources in the Supervisor. Namespace Self-Service permissions control which users can self-provision namespaces through kubectl.

Kubernetes RBAC is the primary authorization mechanism within VKS, using Roles, ClusterRoles, RoleBindings, and ClusterRoleBindings to bind permissions to users, groups, or service accounts. Kubernetes Attribute-Based Access Control (ABAC) is also available, defining access rights through policies that combine user, namespace, resource, and access mode attributes. Authorization webhooks can interpret field and label selectors to enforce fine-grained access control based on resource attributes such as node names and labels.

Service accounts provide namespaced workload identity within VKS clusters. For Windows containers, the Group Managed Service Account (gMSA) Webhook allows containers to run as Active Directory identities. VKS service accounts can be annotated with IAM role ARNs to enable pod identity and role assumption for cloud resource access through OIDC federation, replacing long-lived static credentials with short-lived, least-privileged credentials on demand. IAM role trust policies can be scoped to specific service accounts and namespaces to enforce least privilege.

Several supplementary services within the VKS layer extend the IAM surface. Harbor Registry, available as a standard package on VKS clusters, enforces role-based access control to manage access to container image artifacts. The Secret Store Supervisor Service enforces RBAC for administrators, DevOps engineers, and application tenants managing sensitive data. Argo CD supports both OIDC configuration for external identity providers and local account configuration with global RBAC policies that define role-based access through policy rules and group mappings. The Istio service mesh package includes a pluggable policy layer supporting access controls, rate limits, and quotas at the service-to-service level.

Programmatic access management is available through several APIs. The Identity API authenticates and manages the authorization provider. The Identity Service and Infrastructure as a Service APIs are used to acquire access tokens for REST API authentication. Principal Identity users authenticate using client certificates for service-to-service communications between VCF components.

NSX integrates with LDAP and external identity management services for administrative access to networking configuration. NSX Manager enforces role-based access control with roles such as auditor and network administrator assignable to LDAP users. vDefend, a separate product that extends NSX with security capabilities, offers an Identity Firewall that integrates with Active Directory for identity-based firewall rules.

VMware Private AI Services (PAIS) provides OIDC-based authentication and group-based authorization for AI workloads, the primary IAM mechanisms controlling access to model endpoints, the model runtime, Agent Builder, and Data Indexing and Retrieval capabilities. PAIS authentication is configured with an issuer URL, a client ID (pais-authentication-id), and the openid, groups, and offline\_access scopes. The OIDC provider must support the .well-known/openid-configuration discovery endpoint. Authorization restricts access to user accounts that are members of named groups listed in the authorizedGroups setting (such as group-for-pais-access-01 or group-for-pais-access-02). A groups claim is included in the Access Token and a corresponding groups scope is created in the OIDC configuration, while the oidc-groups-claim setting selects which claim name carries group membership when the OIDC provider uses a non-default name. The oidc-extra-audiences setting accepts additional OAuth 2.0 client identifiers that PAIS will accept when validating Access Tokens, supporting integration with downstream clients that present their own audience. Activating PAIS in an organization namespace requires coordination with the OIDC provider administrator and an organization administrator account in VCF Automation.

Identity provider integration for PAIS deployments is configured at the Supervisor level under Workload Management > Supervisors > Configure > Identity Providers. When a PAIS region is created for a Supervisor, an identity provider entry is automatically created in VMware vCenter for that Supervisor, linking the Supervisor namespace to the configured OIDC source.



The Model Gallery, the curated repository for ML models in PAIS, restricts access to model artifacts and the training data used to tune them through Harbor project access controls. These controls scope which users and service accounts can read, push, or manage sensitive model data, adding an IAM enforcement layer to the model supply chain. A Harbor registry is recommended for PAIS deployments, and OCI-compliant registries from other vendors are also supported.

PAIS Knowledge Bases extend IAM to ingested data sources. Confluence data sources authenticate with a user name and password or a personal access token, while Microsoft SharePoint On-Premises data sources authenticate with a user name and password. Personal access tokens for Knowledge Bases are generated from user account settings and used for programmatic access to linked data sources.

PAIS integrates with Grafana for metrics visualization using the same OIDC provider, extending the IAM model to the observability layer so that operators authenticate through the configured identity provider before accessing PAIS dashboards.

### **VMware vDefend (Contributes)**

VMware vDefend contributes to identity and access management by providing identity-aware network enforcement through Identity Firewall, which integrates with Active Directory to apply firewall policy based on authenticated user identity rather than IP address alone. Identity Firewall supports assigning Active Directory users and groups as source criteria in both distributed firewall rules and gateway firewall rules, applying to tier-0 and tier-1 gateways. This allows organizations to enforce network access policy by user or group membership rather than relying solely on IP-based controls.

Within its management plane, the Security Services Platform implements role-based access control with five defined roles: Enterprise Admin, Auditor, Security Engineer, Security Operator, and Support Bundle Collector. These roles govern access to security policy management, analytics, incident response, and diagnostic functions across the platform. Network Detection and Response enforces the same five-role RBAC model for triage operations such as viewing detections and campaigns, and for configuration operations including SIEM integration management. For authentication, the Security Services Platform supports Active Directory and OpenLDAP as external identity sources, with one identity source configurable at a time. The License Hub integrates with external identity providers to store and manage user and group information for access delegation.

Centralized IAM for organizational users, including VCF SSO, MFA, and directory lifecycle management, is handled by VCF and falls outside vDefend's scope.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) incorporates multiple identification and access management mechanisms that support organization-wide IAM controls within the data services layer.

DSM implements role-based access control with two defined roles: the DSM Admin role, which can create, monitor, and manage all data services in the environment, and the DSM User role, which is restricted to managing and monitoring data services within the namespaces to which the user has been granted access. These roles are assigned at user creation time and govern what each user can see and do within the DSM interface. The Permissions view in the DSM dashboard provides administrators with a centralized location for creating, monitoring, and managing all configured users.

DSM supports two identity sources, which can be used independently or together. Local users are created directly within DSM, with credentials validated by matching email address and password. LDAPS users are sourced from a configured LDAP identity provider, allowing organizations to extend their existing directory service into DSM. When LDAPS is configured, administrators can import users from the directory provider and assign them DSM Admin or DSM User roles. DSM also supports group-based access control: Directory Service Groups from the LDAP provider can be assigned roles directly, so group membership in the external directory determines access level in DSM without requiring per-user assignments.

At the database level, DSM extends authentication controls beyond the platform management layer. MySQL database instances support LDAPS authentication using Active Directory. PostgreSQL and other database instances support host-based authentication (HBA) rule configuration, which determines which clients are permitted to authenticate to a given database instance. SQL Server instances support Windows Authentication through Active Directory integration, enabling Kerberos-based authentication so that clients such as SQL Server Management Studio running on domain-joined machines can authenticate via single sign-on. SQL Server also uses contained database users, where identity information resides within

the database itself rather than at the server level. Administrators can configure SQL Server instances with either a SQL User or Windows Principal as the primary administrator identity, with Windows Principal available for Active Directory-integrated clusters. An internal `dsm_mgmt` management login restricts users from accessing or modifying system-level objects, keeping administrative operations within an isolated context.

Namespace isolation adds a further access boundary. DSM namespaces enforce role-based access control that allows multiple users to work jointly on data services within a namespace while remaining isolated from data services in other namespaces. This supports least-privilege access within multi-tenant deployments.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides a dedicated administrative identity and access management plane through the Avi Controller, supporting multiple authentication methods and fine-grained authorization controls that allow organizations to implement IAM policies across both the management plane and the application delivery layer.

For administrative access, the Avi Controller supports local user accounts, LDAP and LDAPS directory integration, SAML-based federated authentication (including integration with Workspace ONE Access for catalog-based application access), basic authentication, and OAuth/OIDC for API access. Auth Mapping Profiles allow organizations to map external directory groups to specific roles and tenants, so that group membership in an external identity provider directly controls what a user can access within Avi.

Authorization is handled through role-based access control (RBAC) with object-level granularity. Avi's extended granular RBAC uses markers, which are labels assigned to objects, to enable fine-grained authorization based on object ownership and classification. Permission categories cover Operations (including alerts, actions, and traffic capture) and Administration (including controller, tenant, user, role, and system configuration management). This allows restrictions so that a user's permissions apply only to the specific resources matching the markers assigned to their role. RBAC covers object categories including Virtual Services, Application Profiles, SSL/TLS Certificates, Certificate Management Profiles, Cloud definitions, TCP/UDP Profiles, Persistence Profiles, Health Monitors, Analytics Profiles, IP Address Groups, String Groups, SSL/TLS Profiles, PKI Profiles, and Authentication Profiles, with distinct read and write access levels configured per object type.

For multi-tenant deployments, Auth Mapping Profiles support tenant isolation with configurable user group to role mappings, allowing organizations to enforce separation between teams or business units operating within the same Avi Controller.

The Avi License Hub supports role-based access control through an external identity provider, with three predefined roles: Enterprise Admin (full access for all operations, including upgrades and authentication configuration), Auditor (view-only access for tracking activity without making changes), and Support Bundle Collector (restricted to collecting system logs and diagnostics). Management interface access is further controlled through restrictions on allowed ciphers and HMACs for management sessions.

For deployments involving the Avi Kubernetes Operator (AKO), Kubernetes RBAC policies can be applied to AKO Custom Resource Definitions (CRDs) to enforce stricter access control over routing configuration for specific user groups. The Avi Controller also implements RBAC at the Kubernetes and OpenShift cluster level by assigning tenants and roles to user accounts.

At the application layer, Avi can enforce client authentication using OAuth, SAML, JWT token validation, and mutual TLS with client certificates, extending access control to the traffic flowing through virtual services rather than only to the management interface.

Organizations are responsible for defining roles, mapping directory groups to permissions, establishing naming conventions for markers, and maintaining user lifecycle processes. Avi provides the technical mechanisms; the governance and operational processes for identity management must be organized by the teams managing the deployment.

Not applicable for this control: VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.



## Control CR 1.4

> Identifier management: Support the management of identifiers to ensure uniqueness over time.

**SCF Controls:** IAC-09, IAC-09.1

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides a layered set of identity management capabilities for platform administrators and non-consumer users across all component products. The platform's identity architecture is built around VCF SSO, the identity management hub, and the concept of identity sources, where a directory service stores and manages principals along with attributes such as name, address, email, and group membership. VCF Identity Broker appliances manage the mapping between external identity providers and internal directory services, using unique identifiers that must exactly match the values configured in the OIDC identity provider for proper attribute mapping from Active Directory. VCF SSO supports federation with enterprise identity providers through OIDC, SAML, and AD/LDAP protocols. VMware Identity Services supports System for Cross-domain Identity Management (SCIM) for automating user and group synchronization with external identity providers, with documented support for Okta, Microsoft Entra ID, and PingFederate.

At the VMware vCenter level, organizations using Microsoft Active Directory can leverage user identities contained in a Windows Server domain controller across their virtual infrastructure. Federating vCenter to an enterprise identity provider provides flexible options such as multifactor authentication and enables automatic registration and termination of users across services, reducing the identity management burden. The identity provider federation configuration is found in the vCenter administration settings. As a security best practice, a dedicated organizational unit should be pre-created in Active Directory for VCF-related accounts, with appropriate administrative control over that organizational unit. For VMware ESX hosts, using Active Directory for host-level user management is recommended over creating local user accounts on each host, which avoids the complexity of synchronizing account credentials across multiple hosts.

VCF supports Just-In-Time (JIT) Group Provisioning, where groups are created dynamically based on SAML or OIDC token assertions when the user logs in for the first time. This allows users to receive appropriate role assignments at login based on their group membership in the external identity provider, without requiring pre-provisioned groups in VCF. If JIT provisioning is configured without specifying any groups, users and groups must attempt login from the associated domain to be dynamically provisioned before service roles can be assigned.

VMware Identity Manager is a platform for managing users and groups, managing resources and user authentication, and configuring access policies and entitlements. VCF Operations requires an AD/LDAP identity provider to be configured before identity management can begin, and supports importing users from VCF SSO and LDAP sources with role-based access control assignment.

VCF Automation extends the identity architecture by allowing each organization to connect to its own identity source and import users from it. Organization administrators can import users from configured identity providers and assign one or more roles to control their permissions. System administrators can create other system administrator and user accounts in the provider organization, and organization owners can add and manage other users within their scope. VCF Automation also supports API-based remapping of users between identity providers, with both individual and bulk remapping capabilities using SAML, LDAP, and OAUTH provider types. The Identity API authenticates and manages the authorization provider programmatically, supporting automation of identity management tasks.

It is important to note that identity provider role mappings must be configured correctly. If an identity provider associates a user with a system-level role such as System Administrator but does not grant the appropriate rights within the VCF organization, the user can log in but has no operational permissions. User accounts should be given the minimal number of privileges required for their job functions.

NSX Manager can be configured to authenticate users from identity management providers and supports mechanisms to remove local user accounts from the appliances. Users with the Enterprise Administrator role can create new local users, delete users, activate and deactivate accounts, reset passwords, change usernames, assign roles, and configure password

expiration. NSX Manager user passwords expire by default after 90 days, and password expiration can be changed or deactivated for each user within a valid range of 1 to 9999 days. The admin can deactivate and then activate a local user to reset the user's password expiration date. Principal Identity management in NSX provides dedicated service account identities for third-party integrations and automation platforms, and the Enterprise Administrator role can modify or delete objects owned by principal identities.

For Kubernetes workloads running on vSphere Supervisor, external OIDC identity providers can be registered for authentication. When an external identity provider is configured, the Pinniped Supervisor and Concierge components facilitate federated authentication for users and groups. Users and groups should be defined using identity source tools such as Active Directory rather than managed locally on the Supervisor.

While VCF provides robust technical capabilities for user identity management across its components, the control requires organizational mechanisms such as identity governance processes, joiner-mover-leaver workflows, periodic access reviews, and identity lifecycle policies that fall outside the scope of the platform itself. Organizations must establish these processes using their enterprise identity provider and governance tooling, with VCF as the consumer of those centrally managed identities.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) supports two categories of user identity: local users, whose accounts are stored in the DSM internal database, and LDAPS users, whose identities are imported from a configured LDAP Secure (LDAPS) directory server. These two account types are managed independently, and administrators may configure the system to use one or both sources in a given deployment.

For local users, DSM uses the email address as the unique account identifier, which must be unique across the system. Each local account requires an email address, a role assignment, and a password at creation time. Local user authentication proceeds by matching the email ID and validating the password before any LDAPS processing occurs. The first local user account created during deployment of the DSM Provider Appliance is the administrator account; this account is mandatory and cannot be deleted.

For LDAPS users, DSM integrates with an enterprise directory such as Active Directory or OpenLDAP. Administrators configure a Directory Service connection that specifies the Base Distinguished Name, the bind credentials, and the User Search Attribute. The User Search Attribute, which defaults to `userPrincipalName` but may also be set to `sAMAccountName` or `uid` for OpenLDAP environments, determines how users are identified in both the DSM appliance and the managed databases. Users imported from the LDAPS provider are assigned the DSM Admin or DSM User role, and imported LDAP users cannot be edited or deleted from within DSM; lifecycle changes must be made in the authoritative directory.

At the database level, DSM provides identity management across multiple database engines. For MySQL instances, DSM supports LDAPS-based authentication using the `authentication_ldap_simple` plug-in, with the resulting identifier derived from the Directory Service User Search Attribute. For SQL Server instances, DSM supports two administrator user identity types: SQL User authentication using standard SQL Server login credentials, and Windows Principal authentication using Active Directory credentials, available for clusters integrated with Active Directory. SQL Server databases in DSM use contained database users, where identity information resides within the database itself rather than at the server level, so user permissions remain active after a node failover. For SQL Server deployments using Windows Authentication, DSM requires a Privileged Account in Active Directory to support automated identity management. The `dbo` user (database object owner) is mapped to administrator-controlled identities. Administrators can control which user types are permitted in databases provisioned under a policy, selecting Windows Principal authentication, SQL User authentication, or both.

In deployments integrated with VCF Automation through the Consumption Operator, the DSM user account configured for Consumption Operator authentication must hold the DSM Admin role.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer provides a multi-layered user identity management framework for administrators and non-consumer users of the Controller and GSLB management plane. User accounts are maintained in two categories: local users

stored in the Controller's internal database, and remote users authenticated through external AAA servers. Local accounts are created directly in the Controller and associated with a User Profile that governs account attributes and access controls.

For directory-based identity, Avi's Authentication Profile supports LDAP integration. The LDAP Authentication Profile defines a User ID Attribute that uniquely identifies each user record in the directory and must match the username supplied at the login prompt. The Controller can also require group membership, enforcing that users must belong to specified groups identified by distinguished name before they can authenticate. Avi recommends using federated authentication to centralize the authentication process by delegating to an identity provider rather than managing authentication locally.

Avi supports SAML-based federated identity, where the Controller acts as a Service Provider and delegates authentication to a trusted Identity Provider. The Entity ID must match in both the Identity Provider and Service Provider configurations for SAML communication to proceed. SAML verifies user identity without storing credentials on the Controller itself. For OAuth and OIDC scenarios, Avi can extract user identity from the UserInfo endpoint by presenting an access token, or from claims in the id\_token when the UserInfo flag is not enabled.

TACACS+ integration allows a TACACS+ server to supply authorization attributes that Avi maps to specific roles and tenants through Auth Mapping Profiles. The Auth Mapping Profile also supports an "Any Group or Any Attribute" rule that assigns all authenticated users to a least-privileged role and tenant by default, providing a baseline authorization binding for federated accounts. Auth Mapping Profiles should be configured to enforce tenant isolation except for designated superusers who require cross-tenant access.

Each user account can be scoped to a specific tenant and assigned a role with limited access. Administrators can restrict superuser access by setting the is\_superuser flag to False on local accounts, which limits access to only the permitted tenants and roles. Custom User Account Security profiles let administrators configure per-profile thresholds for login failures, account lockout duration, password history, concurrent sessions, and credential timeout, beyond the system-wide Default-User-Account-Profile.

Avi GSLB supports creation of non-admin users with assigned roles for granular access control on the global load balancing management plane. The Avi Service Engine supports user ID population for application-layer identity tracking through three methods: custom DataScript assignment via `avi.http.set_userid()`, HTTP Basic Auth username extraction, or client certificate subject name extraction during the SSL/TLS handshake. These controls together provide the technical mechanisms for unique user identification, account scoping, and lifecycle configuration.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 1.5

> Authenticator management: Support the management of authenticators (passwords, tokens, keys) used by the component. Hardware security for authenticators: Capability to protect authenticators using hardware mechanisms (e.g., TPM). Authenticator management enhancement: Additional requirements for secure authenticator storage.

**SCF Controls:** IAC-10

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides a layered authentication infrastructure that supports secure management of authenticators across the platform's components and the Kubernetes workload consumption layer, though the broader organizational policies governing authenticator lifecycle management require processes and governance beyond what VCF itself provides.

VCF Identity Broker integrates with a selected identity provider for user authentication, establishing trust through metadata exchange between the service provider and the identity provider. This centralized approach allows organizations to manage authenticators through their existing identity management systems rather than maintaining separate credential stores for each VCF component. VCF SSO provides a single sign-on environment where a user provides credentials once, and components in the environment perform operations based on the original authentication. Administrators can federate VMware vCenter with an enterprise identity provider, which enables multifactor authentication (MFA), automatic user provisioning and deprovisioning, and token-based authentication that reduces the risk of credential exposure. VCF SSO supports SCIM (System for Cross-domain Identity Management) for automating user identity exchange across identity domains, and named providers including Okta are supported through the identity provider federation mechanism. When configured for identity provider federation, the external provider handles all authentication mechanisms, including passwords, MFA, and biometrics, delegating full authenticator management to the organization's existing identity infrastructure. The Authorization Code grant type in VMware Identity Broker supports MFA workflows, while the Password OAuth 2.0 grant type is incompatible with MFA requirements, so organizations should use the Authorization Code flow when MFA is needed.

VMware Kubernetes Service (VKS) extends this identity architecture to the workload consumption layer. VKS clusters support two authentication methods: VCF SSO and any OIDC-compliant external identity provider, including Okta, so identity lifecycle management, including onboarding, offboarding, and group membership changes, can remain in the organization's central identity system. Users authenticated through an external identity provider are granted Kubernetes RBAC permissions using the `sso:<username>@<domain>` naming convention in ClusterRoleBindings. Custom ClusterClass configurations allow organizations to authenticate developers against existing corporate identity systems. When provisioning clusters based on a custom ClusterClass, the authentication Secret can be configured with multiple JWT issuers to support distinct identity providers simultaneously, such as Keycloak for contractors and Microsoft Entra ID for employees. At the kube-apiserver level, VKS supports multiple authenticator types: client certificates, bearer tokens, an authenticating proxy, and HTTP basic auth. The `--authentication-config` file provides structured control, allowing multiple JWT authenticators each with a unique issuer URL and discovery URL, and CEL expressions to map claims to user attributes and validate claims. When multiple authenticator modules are active, the first to successfully authenticate a request short-circuits further evaluation, making authenticator order significant; Kubernetes hardening guidance recommends enabling as few authentication mechanisms as possible to simplify user management and reduce the risk of users retaining access after it is no longer needed. The kube-apiserver can also leverage existing authentication methods such as LDAP or Kerberos through plugins, and kubectrl integrates with credential plugins for centralized operator credential management. The `system:auth-delegator` ClusterRole provides unified authentication and authorization flows for add-on API servers deployed into VKS clusters. Production VKS clusters should enable Kubelet authentication and authorization to restrict direct access to the kubelet API. For Windows node pools, VKS clusters support Group Managed Service Accounts (gMSA) for authenticating Windows nodes to Active Directory; gMSAs are managed by security groups that control password retrieval and delegation permissions, allowing Windows workloads to use Active Directory-managed identities.

Standard packages deployable on VKS clusters extend authenticator management to workload-level services. Argo CD, available as a standard package, supports local account authentication with global RBAC policies and OIDC authentication for

federated identity, allowing GitOps workflow access to be governed through the same identity infrastructure used for cluster access. The Harbor container registry package provides authentication and token management for artifact access, with role-based access control restricting image push and pull operations.

VCF Operations integrates with VCF SSO for centralized authentication through an identity provider. NSX Manager can be configured to authenticate users from identity management providers or through a directory service such as Active Directory over LDAP or OpenLDAP. NSX authentication best practices include integrating with identity providers, enabling multi-factor authentication, and defining access controls based on user roles and permissions. NSX Manager requires authentication prior to establishing a management connection for administrative access. The VCF Automation Provider Management Portal supports authentication via local users and SAML users, and Organization Administrators use credentials configured during organization creation to access their respective organizations.

For local accounts, each VCF component enforces password complexity and lifecycle policies. NSX Manager requires passwords of at least 12 characters with uppercase, lowercase, numeric, and special character requirements, enforced through Linux PAM modules. Each NSX user password has an expiration date tracked by NSX, with a default expiration period of 90 days. Password expiration is configurable between 1 and 9,999 days. VCF Operations enforces a minimum of 15 characters with similar complexity rules. VCF Automation passwords must also be at least 15 characters in length.

VCF provides configurable account lockout policies across its components. VCF Operations locks accounts after five consecutive failed login attempts for 15 minutes. A separate policy locks VCF Operations accounts if a user enters an incorrect password three times within five minutes. VCF Automation supports account lockout in password policy system settings, which locks user accounts after a configurable number of invalid login attempts. VCF Automation also provides a configurable account lockout interval that specifies the duration before an account is automatically unlocked, and provider administrators can unlock a user account manually without waiting for that interval to expire.

For service-to-service authentication, VCF uses certificate-based credentials. VCF SSO supports X.509 certificate authentication, and vCenter extensions can authenticate using `loginExtensionByCertificate()` with an extension key certificate. Provider management accounts authenticate using API access tokens that rotate after each use. VCF Automation provides service accounts with the same access restrictions as user API tokens, so service accounts cannot perform user management or create refresh tokens. NSX supports Principal Identity certificates for cloud management platform integration, requiring SHA256-based certificates with RSA/DSA message algorithm and 2048-bit or larger key sizes. At the Kubernetes layer, service account authentication uses bearer token secrets generated automatically for service accounts. Kubernetes hardening guidance recommends setting short lifetimes on certificates and automating their rotation, and recommends using an authentication provider that controls how long issued tokens remain valid; using short-lived Secrets helps detect or limit the impact of credential exposure. The `system:masters` group functions as a break-glass super-user group that bypasses the RBAC authorization layer and should be reserved for emergency access scenarios only.

VCF Operations provides API endpoints for trusted certificate management and displays certificate expiration status for each workload domain through its Certificates tab. NSX supports non-disruptive certificate rotation in VCF and later, allowing certificate replacement without service interruption.

VCF also provides visibility into authenticator usage through audit logging. User authentication details are viewable in VCF Operations, including user name, successful or failed authentications, and events such as user login and guest operation authentication. This logging supports detection of compromised or misused authenticators.

While VCF provides the technical controls for authenticator strength, complexity, expiration, lockout, and rotation within its own components, as well as identity provider integration for MFA enforcement and centralized authentication, and the VKS consumption layer provides configurable multi-method kube-apiserver authentication, external identity provider integration, short-lived credential support, and gMSA for Windows workloads, the broader organizational policies for initial authenticator issuance, periodic credential rotation schedules, revocation upon personnel changes, and mapping authentication strength requirements to data classification levels require processes and governance outside VCF's scope.

VMware Private AI Services (PAIS) provides authentication mechanisms specific to its AI infrastructure components, integrating with externally managed identity systems to control access to AI services, model endpoints, and training data.

PAIS uses OIDC-based authentication as its primary access control mechanism. The PAISConfiguration custom resource configures the OIDC provider with issuerUrl, clientId, and scopes including openid, groups, and offline\_access. Access is restricted to user accounts whose membership matches OIDC groups listed in the authorizedGroups field, and the Access Token carries a groups claim that PAIS validates against the authorized group list. The OIDC configuration also supports configurable group claim names and Extra Audiences for additional OAuth2.0 clients that PAIS will accept when validating the Access Token.

The PAISConfiguration custom resource supports client TLS for outbound authentication to the OIDC provider and Harbor registry, configured through the worker.clientTls.caBundleRefs field. This binds certificate-based trust to identity provider and registry connections so that authentication exchanges are integrity-checked against approved CAs.

Authenticator management for the Harbor registry that stores AI models and artifacts follows a credential-based pattern. The oras login command authenticates to a private Harbor registry using username and password credentials for artifact push operations, and docker login is used for pushing models to Harbor projects through the Model Gallery. Harbor project access capabilities restrict access to the sensitive data that is used to train and tune models. Model endpoints support authentication through harbor-pull-secret-readonly to control access to container images stored in private registries, and Harbor model retrieval requires certificate-based trust in addition to credential authentication.

Internal service-to-service authentication is addressed at the component level. PAIS authenticates to the OpenTelemetry Collector using a Kubernetes Secret containing Authorization Bearer tokens and custom headers, and the LLM Traces flow uses the same pattern with semi-colon separated headers. The PAIS MCP server supports custom HTTP headers including User-Agent and Authorization headers for API authentication, and remote MCP servers can optionally use static authentication tokens. PAIS integrates with Grafana for metrics visualization using OIDC provider authentication.

For Knowledge Base data sources, PAIS supports authentication to Confluence using either a username and password or a personal access token, configured according to the Confluence administrator's settings. Personal access tokens are generated from user account settings and used for programmatic access to Knowledge Bases and data sources. Microsoft SharePoint data sources require username and password authentication, with support limited to SharePoint On-Premises deployments.

Deep Learning VMs deployed through self-service catalog items in VCF Automation, when configured with the PyTorch NGC software bundle, support optional JupyterLab authentication activation for interactive notebook access.

The underlying user accounts, credential storage, password policies, and OIDC provider administration are owned by VCF SSO or an external identity provider. PAIS configures and enforces authentication using those systems but does not manage authenticator lifecycle itself.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides several mechanisms for managing authenticators across both the administrative plane and the database layer, supporting the secure credential management aspect of this control.

For administrative access to the DSM platform, the system supports two authentication paths: local user accounts authenticated by email ID and password, and LDAPS-integrated accounts authenticated by matching the email domain against a registered LDAPS server. When directory service credentials are non-functional, DSM falls back to local user authentication, maintaining access continuity. Administrators configure and manage these authentication sources through the DSM Admin UI, which also provides a Permissions view for creating, monitoring, and managing configured users. LDAPS integration supports certificate-based validation of the directory service connection, with administrators able to upload directory service server certificates through the Admin UI to establish trusted connections. LDAP groups can be assigned roles within DSM, enabling group-based access control aligned with directory service group membership.

At the database layer, DSM supports directory service authentication for PostgreSQL, MySQL, SQL Server, and MinIO databases. For PostgreSQL, host-based authentication (HBA) rules determine how clients are authenticated, using ordered matching rules that evaluate connection type, client address, requested database, and user name. The default HBA configuration includes a final catch-all entry using scram-sha-256, with optional LDAP entries for directory-integrated authentication. This HBA configuration can be modified through the DSM interface. For MySQL databases, LDAPS



authentication is available using Active Directory Global Catalog, with support for multi-domain controller deployments via the AD Secure/SSL Global Catalog on port 3269. Directory service authentication for individual database instances can be enabled or disabled through the Enable Directory Service Authentication toggle in the database settings.

For SQL Server databases, DSM uses contained database users, where users authenticate directly against the database and identity information resides within the database. DSM supports Windows Authentication for SQL Server server-level logins through Active Directory integration, enabling Kerberos-based single sign-on for clients joined to the same domain. SQL Server Authentication logins can also be created on all DSM clusters. Administrators configure which user types are permitted per data service policy, selecting between Windows Principal authentication using Active Directory credentials and SQL User authentication using standard SQL Server logins. An internal management login (`dsm_mgmt`) restricts access to system-level objects, keeping administrative operations within an isolated context.

DSM enforces password policies for local accounts, including complexity requirements, and applies a root password expiry on a three-month rotation cycle. For service-to-service communication, DSM uses authentication secrets (`dsm-auth-creds`) containing endpoint credentials and root CA certificates, with automatic secret generation available through the Consumption Operator installation. Prometheus metrics forwarding supports configurable authentication options, including no authentication, certificate-only authentication, basic authentication with username and password, or a combination of certificate and basic authentication.

Regarding authentication strength appropriate to data classification, DSM provides the authentication mechanisms that administrators can configure to match organizational security policies, but it does not automatically enforce differentiated authentication strength based on data classification. Organizations must define their data classification scheme and determine which DSM authentication options (local credentials, LDAPS, HBA rules, certificate validation, Kerberos) apply to each tier of sensitivity. DSM contributes the technical controls; the alignment between authentication strength and data classification requires organizational governance to establish and maintain.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer provides mechanisms to securely manage authenticators across two distinct planes: the Controller management plane (admin access) and the application data plane (end-user client authentication for load-balanced applications).

For the management plane, Avi maintains user accounts in two categories: local users managed directly on the Controller, and remote users authenticated through an external authentication, authorization, and accounting (AAA) server. Avi uses Authentication Profiles to configure how administrators authenticate to the Controller. Authentication Profiles support LDAP for credential validation against a directory service, SAML for federated authentication that delegates the process to a centralized identity provider (IdP), and basic authentication. The recommended approach is federated authentication, which centralizes the authentication process by delegating to an external IdP rather than managing credentials locally on the Controller. SAML profiles verify user identity without storing credentials on untrusted systems, and Avi Controller SAML authentication profiles can be integrated with Workspace ONE Access for catalog-based application access. Admin users can also authenticate using password-less SSH-based login as an alternative to password-based authentication. On every user login, the Auth Mapping Profile evaluates authorization mapping rules and updates the user record with tenant and role pairs drawn from the IdP's assertions.

For the application data plane, Avi acts as a proxy that applies client authentication policies to inbound application traffic. Supported methods include LDAP authentication (checking client credentials against a directory service), SAML single sign-on (enabling federated application access with a single credential set), OAuth and OIDC (where Avi acts as a client and resource server, with the authorization server authenticating clients and validating authorization codes before issuing tokens), and JSON Web Token (JWT) validation. The SSO Policy supports configurable Authentication Rules and Authorization Rules with path-based matching criteria, allowing different authentication requirements to be applied to different application paths. More granular authorization in OAuth/OIDC flows is available through Authorization Policies or DataScript.

Organizations configure which authentication method applies to each virtual service or application based on the sensitivity of the data accessed. Avi's Authentication Profiles and SSO Policies provide the technical enforcement points to apply different

authentication strengths across the application portfolio, but the mapping from data classification tier to required authentication method is an organizational governance decision that must be defined and maintained by the organization.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.



## Control CR 1.5(1)

> Authenticator management: Support the management of authenticators (passwords, tokens, keys) used by the component. Hardware security for authenticators: Capability to protect authenticators using hardware mechanisms (e.g., TPM). Authenticator management enhancement: Additional requirements for secure authenticator storage.

**SCF Controls:** IAC-12.1

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides several integration points for Hardware Security Modules (HSMs) and hardware-rooted trust mechanisms that support the protection of authenticators and cryptographic material, though VCF itself does not include a built-in HSM. The platform is designed to work with external HSM devices and hardware security components to strengthen key management, entropy sourcing, and boot integrity.

VCF integrates with external Key Management Servers (KMS) through the KMIP 1.1 protocol, providing the mechanism through which organizations can connect HSM-backed key management infrastructure to the platform's encryption services. The external KMS is the component that would be backed by an HSM to protect the root encryption keys on which VCF relies for vSAN data-at-rest encryption, VM Encryption, and other cryptographic operations. The KMS must support KMIP 1.1 and be capable of acting as a symmetric key foundry and server. VCF supports Standard Key Providers that connect to external KMIP-compliant KMS systems, where the KMS can be backed by HSMs for hardware-protected key storage. VMware vCenter establishes a trusted connection with the KMS by uploading the KMS server certificate and private key, and provisions encryption keys from the key provider. Each VMware ESX host uses the Key Encryption Key (KEK) to encrypt its Data Encryption Keys (DEKs) and stores the encrypted DEKs on disk, but does not store the KEK on disk. When a host reboots, it requests the KEK with the corresponding ID from the KMS and decrypts its DEKs as needed. The vSAN KMS configuration also supports an optional proxy address for environments that require it. When a user performs an encryption task such as creating an encrypted virtual machine, vCenter requests a new key from the default KMS to use as the KEK.

ESX supports the use of external HSM devices that are FIPS 140-3 and EAL4 certified as sources of external entropy data. Starting with vSphere 8.0 Update 1, administrators can add external entropy sources to an ESX host to feed high-quality, hardware-generated randomness into the host's cryptographic subsystem. This capability directly strengthens the generation of keys and other authenticators used across the platform by supplementing the host's own entropy sources.

ESX host hardening guidance includes enabling the Trusted Platform Module (TPM), a hardware-based security component embedded in the physical server. TPM provides a hardware root of trust for secure boot attestation and can store cryptographic keys and measurements in tamper-resistant hardware. When TPM is enabled and configured as part of the ESX hardening baseline, it provides hardware-backed protection for host-level authenticators and boot integrity verification. The ESX hardening guidance covers configuring authentication and authorization settings, enabling Secure Boot, disabling unnecessary services, enabling TPM, and implementing network security measures as part of a layered security posture. TPM modules must run the latest 2.0 firmware when in use, or be disabled in the BIOS if not in use. The physical TPM on ESX hosts also supports sealing policies based on Platform Configuration Register (PCR) measurements, which restrict access to sensitive data stored in the TPM to configurations that match a known-good platform state. Beyond host-level TPM, vSphere supports Virtual Trusted Platform Modules (vTPM) for virtual machines, allowing guest operating systems to use hardware-rooted cryptographic security without requiring a dedicated physical TPM per VM; vTPM keys are not exposed to the guest operating system itself, which reduces the attack surface at the VM level.

VCF also supports hardware memory encryption through confidential vSphere Pods. Confidential vSphere Pods use hardware technology such as AMD SEV-ES to keep guest OS memory encrypted, protecting it against access from the hypervisor. This capability addresses workload confidentiality rather than authenticator storage, but is part of the platform's hardware-rooted security mechanisms.

Organizations deploying VCF in environments that require HSM-protected authenticators should procure and integrate external HSM appliances, configure ESX hosts to use them for entropy and key management, enable TPM on all hosts, and

establish trusted connections between vCenter and the KMS infrastructure. The platform provides the integration interfaces, protocol support, and configuration options, but the HSM hardware itself, its lifecycle management, and the policies governing its use are organizational responsibilities.

#### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer supports Hardware Security Module (HSM) integration as a native capability for protecting the cryptographic keys that Avi's TLS termination and certificate operations depend on. The Avi Controller integrates with Thales Luna (formerly SafeNet Luna) HSM appliances, enabling hardware-backed storage and management of SSL certificates and private keys rather than software-based key storage.

The primary configuration mechanism is the HSM Group, configured in the Service Engine Group settings under the Advanced Settings tab in the Avi Controller UI or through the API. An HSM Group defines how Avi Controller and Service Engine nodes reach and authenticate with the external HSM device, including registering the IP addresses of all Controllers and Service Engines authorized to interact with it. The HSM Group is assigned per Service Engine Group and acts as a template of HSM settings applied uniformly to every Service Engine in that group. Mutual certificate authentication is established between the Avi Controller and the HSM during initial registration, so that only authorized nodes can request cryptographic operations from the device.

Avi Service Engines support dedicated network interfaces for HSM communication, separate from their standard management and data-plane interfaces. This interface isolation keeps HSM traffic off the general management network. Avi Controller cluster nodes similarly use dedicated network interfaces labeled HSM for hardware security module communication, and the HSM interface can be moved to a secondary interface if needed. High availability across multiple HSM devices is supported, allowing organizations to add redundant HSM nodes to an HSM Group to reduce single points of failure in key availability.

Avi Load Balancer's FIPS 140-3 compliance posture also incorporates dedicated HSM interfaces on Service Engines for secure cryptographic operations. Supported HSM clients include Thales Luna HSM client 7.3 and Universal Client v 10.4.1 for key management integration.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 1.5(a)

> Sub-item (a) of CR 1.5 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 1.5 requirement in the standard.

**SCF Controls:** IAC-10.8

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF requires administrators to establish unique credentials at every stage of platform deployment rather than shipping with pre-set default passwords. The deployment architecture enforces this by making credential configuration a mandatory step that must be completed before the platform becomes operational.

During initial installation, the VCF Installer Deployment Wizard requires the administrator to specify passwords for all components or to auto-generate them. Each appliance, including VCF Operations, VMware vCenter, NSX, VMware ESX hosts, and VCF Automation, requires its own password configuration at deployment time, and the deployment process will not proceed without it. The wizard allows administrators to view generated passwords at the end of the deployment process, so auto-generated credentials are recorded and known from the start. Authentication sources for VCF Operations are also defined during initial server setup, requiring the deployer to make deliberate authentication configuration decisions before the system enters production. When organizations are created within VCF Automation, organization administrators use credentials that are configured during the organization creation process itself, so every administrative account begins with purposefully chosen authenticators rather than vendor-supplied defaults.

After deployment, VCF Operations provides centralized password management for all VCF components. Administrators can rotate passwords on demand through the Password Management page or schedule automatic rotation on a recurring basis. During rotation, VCF Operations generates a randomized password that conforms to the component's password policy and re-establishes the relevant component integrations automatically. The VCF Operations API also supports scheduling rotation at custom intervals for automation workflows. VCF Operations also provides credential rotation for system-managed endpoint credentials through a Rotate function, and administrators are required to rotate these credentials for individual endpoints on a regular schedule.

For programmatic access, service accounts in VCF Automation authenticate using API access tokens that rotate after each use, as specified by RFC 6749 section 6. Access tokens normally last one hour and are based on a refresh token that normally lasts 90 days but can be revoked earlier. When rotation is activated on a service account, each refresh token grant call returns a new access token along with a new refresh token, and the original refresh token is invalidated. This prevents token reuse and limits the window of exposure for any single credential.

VCF SSO allows configuration changes to the identity provider after initial setup, giving administrators the ability to migrate from locally defined credentials to enterprise identity sources as the environment matures. When the identity broker is changed for VCF Operations or VCF Automation, the previous authentication source is deactivated and existing provisioned users and groups can no longer be used. Roles must then be reassigned to users and groups from the new authentication source, which enforces a clean transition rather than leaving stale default credentials in place. Administrators should document substantial modifications to the initial identity source configuration so that the VCF SSO configuration can be reverted to a recognized and stable state if necessary.

Within the VMware Kubernetes Service (VKS) and vSphere Supervisor layer, the same principle of replacing default credentials at installation time extends to components deployed on Kubernetes workload clusters. Harbor Registry, available as a standard package or as a Supervisor service on VKS clusters, ships with a known default administrative password and requires administrators to set the harborAdminPassword to a strong, unique value during installation; the documented default is Harbor12345, and the installation documentation requires this to be changed before the registry enters production. Harbor provides a generate-passwords.sh script bundled with the Harbor package image that auto-generates all mandatory passwords and secrets, and the documentation recommends using this script rather than manually specifying values. The Harbor inter-component secrets, specifically core.secret, jobservice.secret, and registry.secret, must each be changed from

their default values during installation; changing `core.secret` after deployment breaks inter-component authentication and logs out all active users, creating a strong operational incentive to configure these values correctly at install time rather than relying on a post-deployment correction.

At the Kubernetes API server level, anonymous authentication is enabled by default and must be explicitly configured or restricted. The `AuthenticationConfiguration` file provides control over which endpoints support anonymous access, allowing administrators to disable anonymous authentication or limit it to specific paths; if the `anonymous` field is set in `AuthenticationConfiguration`, the command-line `anonymous-auth` flag cannot be used, so all anonymous access control flows through the configuration file. The Kubernetes hardening guidance recommends enabling as few authentication mechanisms as possible to reduce the surface area of default credential exposure. Similarly, the Kubelet's authorization mode defaults to `AlwaysAllow`, a configuration that organizations deploying VKS clusters are expected to change to `Webhook` mode during cluster setup; operational runbooks should confirm this change before clusters enter production. Infrastructure credentials within VKS clusters benefit from short certificate lifetimes with automated rotation to reduce the window of exposure for any single credential.

For developer access to VKS workload clusters, DevOps users authenticate through VCF SSO using their identity provider credentials, and custom `ClusterClass` configurations can direct developers to authenticate through existing corporate identity systems. This integration shifts user credential management to the external identity provider, reducing the scope of locally managed default credentials on the Kubernetes platform to system service accounts and component secrets that are covered by the installation-time requirements above.

VMware Private AI Services (PAIS) includes several deployment-time mechanisms that require credential setup or prompt credential changes as part of system installation.

When Deep Learning VMs are deployed through VCF Automation self-service catalog items, PAIS requires a password to be specified for the default `vmware` user account via OVF properties at deployment time. Deep Learning VMs also accept a `public-keys` OVF property that populates the default user's `SSH authorized_keys` with provided SSH public key values, supporting key-based access without leaving default keys in place. Deep Learning VM workload deployment fails when invalid authentication credentials are provided for downloading DL workloads, so placeholder or blank credentials cannot persist after first deployment.

PAIS organization-level authentication has no operational default. PAIS authentication is configured with OIDC using an issuer URL, client ID `pais-authentication-id`, and scopes including `openid`, `groups`, and `offline_access`. The `jwtauthenticator` configuration requires specification of an issuer URL, audience ID, username claim field set to `preferred_username`, groups claim field set to `groups`, and TLS certificate authority data. PAIS activation requires coordination with the OIDC provider administrator to set up identity and access management, so authentication is non-functional until these values are configured. OIDC authentication also supports configuration of Extra Audiences for additional OAuth 2.0 clients when required by the OIDC provider. The `vcf cluster register-vcfa-jwt-authenticator` command enables JWT authentication for VKS cluster login to PAIS.

Registry credential configuration is required throughout the PAIS setup workflow. The `oras login` command authenticates to a private Harbor registry using username and password credentials for subsequent image operations. PAIS air-gapped deployments require the location of a self-hosted registry to be provided, and login credentials must be supplied if the registry requires authentication. All passwords entered in the PAIS quickstart, including the private container registry password and proxy password, are saved as secrets in VCF Automation rather than left in plain text. For Model Runtime image pulls, a robot account must be configured in the Harbor registry, and model endpoints deployed via `kubectl` can specify an optional `pullSecrets` image pull secret for OCI registry authentication.

PAIS observability and tooling integrations use bearer-token credentials configured at setup time. PAIS LLM Traces authentication to the OpenTelemetry collector is configured through a Kubernetes Secret containing authorization headers, including a Bearer token and custom headers. Authentication to VCF Automation using the VCF Consumption CLI requires generation of an API token for the user's account in their VCF Automation organization. Knowledge Base data source

authentication for Confluence uses either a user name and password or a personal access token, configured according to the Confluence administrator's settings. Remote MCP servers connected to PAIS can optionally use static authentication tokens.

One default that requires explicit organizational action: PAIS Prometheus TLS client authentication and certificate validation are turned off by default in the Grafana setup for PAIS metrics. Operators who require client-certificate authentication on the metrics path must turn this on as part of the deployment process.

The underlying identity framework, including identity federation, organizational user management, and single sign-on, is provided by VCF SSO and VCF Automation rather than by PAIS itself.

### **VMware vDefend (Contributes)**

The Security Services Platform (SSP) component of VMware vDefend (vDefend) requires mandatory password configuration for its local accounts during initial deployment. When deploying the SSP Installer OVA, the administrator must set passwords for the admin account, which has full privileges to manage the platform, and the audit account, which has access to APIs and services for auditing the system. A password for the sysadmin appliance management account is also required before deployment completes. All passwords must conform to specified password rules at the time of configuration, making it structurally impossible to leave these accounts with default credentials after installation.

After initial deployment, the admin and audit account passwords can be changed by an authorized administrator through either the SSP Installer UI or the SSP Platform UI. The installer user password can also be changed via the command-line interface using the `-u` flag. Any password set or changed through these interfaces must continue to meet the platform's defined password criteria.

The SSP Installer also requires configuration of a System GRUB password during deployment, which controls access to the appliance-level password reset mechanism for local system users. If no GRUB password is explicitly configured during deployment, the SSP Installer applies a default value. Administrators should set a custom GRUB password during deployment to address this additional default authenticator.

vDefend's contribution is scoped to its own management plane. The organizational requirement to change default authenticators on all other systems and accounts across the environment is outside vDefend's scope.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides several mechanisms that support changing default authenticators during account creation and system installation.

For local user accounts, DSM includes a `force_password_change` flag. When set to true, this flag requires the user to change their password at next login. Administrators configure this flag when creating or resetting local user accounts through the DSM administration interface (Administering Data Services Manager > Managing LDAP Users in Data Services Manager). The flag can also be set to false, which means enforcement depends on the administrative policies organizations establish at account creation time.

For MySQL databases, DSM creates a default database user with a defined privilege set, including a default username that can be changed before provisioning the database. When credentials are not manually specified during creation, DSM automatically generates a unique database engine administrator username and password, removing the reliance on a static default credential that would otherwise need to be changed post-deployment.

DSM adds SQL Server as a supported database engine. During SQL Server cluster creation, DSM automatically creates a cluster administrator login and can automatically generate the administrator username and password if they are not manually provided. SQL Server Authentication logins can be created for individual users, and Active Directory integration is available for Windows Authentication when a Privileged Account is configured.

PostgreSQL instances provisioned through DSM use `pg_hba.conf` host-based authentication configuration, which defines authentication methods for specific users and connection types. The default configuration uses `scram-sha-256` as the password authentication method, with `reject` and `cert` as alternatives. Custom HBA entries can be configured at provisioning

time and adjusted in the Additional Settings step of the VCF Automation provisioning interface. Organizations should review and customize HBA configurations after deployment to match their access control requirements.

DSM supports LDAPS integration as an alternative to local user accounts, delegating authentication and authorization to the configured directory service. LDAP or Active Directory-based authentication can also be enabled for individual MySQL and PostgreSQL database instances, either during initial provisioning or during cloning from a snapshot. When directory service authentication is in use, password lifecycle management, including change-at-first-login requirements, is governed by the directory service's own policies.

Auditors should verify that the `force_password_change` flag is used during initial local account setup, that MySQL default database usernames are changed or that auto-generated credentials are used before provisioning, that SQL Server cluster credentials are reviewed after creation, and that PostgreSQL HBA configurations are customized post-deployment to match site security requirements.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) contributes to this control through credential management mechanisms that address the intent of avoiding shared, well-known defaults across deployments.

For administrative access, local authentication is the default method on the Avi Controller. The default deployment creates an admin account that does not mandate specific password requirements at the account level; organizations must configure strong password enforcement separately through User Account Security profiles. When a new user is created and no profile is explicitly selected, the system automatically assigns the Default-User-Account-Profile. Administrators can create custom User Account Security profiles under Administration > User Account Security in the Controller UI, specifying password history, login failure thresholds, and account lock settings beyond what the default profile provides. In multi-node controller clusters, follower nodes may use a default initial admin password or a user-configured one, so post-deployment credential rotation should address all cluster members.

A specific credential-change requirement applies to the audit user account: the Avi Controller audit user password must be changed and must meet required criteria before the system is placed in production.

Default system accounts in Avi cannot be deactivated or removed. Organizations must establish a post-deployment rotation procedure that explicitly addresses these accounts. Locally authenticated users can change their own passwords through My Account in the Controller UI by entering the old password followed by the new one. Administrators can also generate a random password or set a new password manually during a password reset operation.

Because Avi does not enforce a mandatory credential rotation step at first login or as part of account creation for the general admin account, and because default system accounts cannot be disabled, organizations are responsible for the post-deployment procedures that satisfy this control.

Not applicable for this control: VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.



## Control CR 1.5(b)

> Sub-item (b) of CR 1.5 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 1.5 requirement in the standard.

**SCF Controls:** IAC-10.8

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF requires administrators to establish unique credentials at every stage of platform deployment rather than shipping with pre-set default passwords. The deployment architecture enforces this by making credential configuration a mandatory step that must be completed before the platform becomes operational.

During initial installation, the VCF Installer Deployment Wizard requires the administrator to specify passwords for all components or to auto-generate them. Each appliance, including VCF Operations, VMware vCenter, NSX, VMware ESX hosts, and VCF Automation, requires its own password configuration at deployment time, and the deployment process will not proceed without it. The wizard allows administrators to view generated passwords at the end of the deployment process, so auto-generated credentials are recorded and known from the start. Authentication sources for VCF Operations are also defined during initial server setup, requiring the deployer to make deliberate authentication configuration decisions before the system enters production. When organizations are created within VCF Automation, organization administrators use credentials that are configured during the organization creation process itself, so every administrative account begins with purposefully chosen authenticators rather than vendor-supplied defaults.

After deployment, VCF Operations provides centralized password management for all VCF components. Administrators can rotate passwords on demand through the Password Management page or schedule automatic rotation on a recurring basis. During rotation, VCF Operations generates a randomized password that conforms to the component's password policy and re-establishes the relevant component integrations automatically. The VCF Operations API also supports scheduling rotation at custom intervals for automation workflows. VCF Operations also provides credential rotation for system-managed endpoint credentials through a Rotate function, and administrators are required to rotate these credentials for individual endpoints on a regular schedule.

For programmatic access, service accounts in VCF Automation authenticate using API access tokens that rotate after each use, as specified by RFC 6749 section 6. Access tokens normally last one hour and are based on a refresh token that normally lasts 90 days but can be revoked earlier. When rotation is activated on a service account, each refresh token grant call returns a new access token along with a new refresh token, and the original refresh token is invalidated. This prevents token reuse and limits the window of exposure for any single credential.

VCF SSO allows configuration changes to the identity provider after initial setup, giving administrators the ability to migrate from locally defined credentials to enterprise identity sources as the environment matures. When the identity broker is changed for VCF Operations or VCF Automation, the previous authentication source is deactivated and existing provisioned users and groups can no longer be used. Roles must then be reassigned to users and groups from the new authentication source, which enforces a clean transition rather than leaving stale default credentials in place. Administrators should document substantial modifications to the initial identity source configuration so that the VCF SSO configuration can be reverted to a recognized and stable state if necessary.

Within the VMware Kubernetes Service (VKS) and vSphere Supervisor layer, the same principle of replacing default credentials at installation time extends to components deployed on Kubernetes workload clusters. Harbor Registry, available as a standard package or as a Supervisor service on VKS clusters, ships with a known default administrative password and requires administrators to set the harborAdminPassword to a strong, unique value during installation; the documented default is Harbor12345, and the installation documentation requires this to be changed before the registry enters production. Harbor provides a generate-passwords.sh script bundled with the Harbor package image that auto-generates all mandatory passwords and secrets, and the documentation recommends using this script rather than manually specifying values. The Harbor inter-component secrets, specifically core.secret, jobservice.secret, and registry.secret, must each be changed from

their default values during installation; changing `core.secret` after deployment breaks inter-component authentication and logs out all active users, creating a strong operational incentive to configure these values correctly at install time rather than relying on a post-deployment correction.

At the Kubernetes API server level, anonymous authentication is enabled by default and must be explicitly configured or restricted. The `AuthenticationConfiguration` file provides control over which endpoints support anonymous access, allowing administrators to disable anonymous authentication or limit it to specific paths; if the `anonymous` field is set in `AuthenticationConfiguration`, the command-line `anonymous-auth` flag cannot be used, so all anonymous access control flows through the configuration file. The Kubernetes hardening guidance recommends enabling as few authentication mechanisms as possible to reduce the surface area of default credential exposure. Similarly, the Kubelet's authorization mode defaults to `AlwaysAllow`, a configuration that organizations deploying VKS clusters are expected to change to `Webhook` mode during cluster setup; operational runbooks should confirm this change before clusters enter production. Infrastructure credentials within VKS clusters benefit from short certificate lifetimes with automated rotation to reduce the window of exposure for any single credential.

For developer access to VKS workload clusters, DevOps users authenticate through VCF SSO using their identity provider credentials, and custom `ClusterClass` configurations can direct developers to authenticate through existing corporate identity systems. This integration shifts user credential management to the external identity provider, reducing the scope of locally managed default credentials on the Kubernetes platform to system service accounts and component secrets that are covered by the installation-time requirements above.

VMware Private AI Services (PAIS) includes several deployment-time mechanisms that require credential setup or prompt credential changes as part of system installation.

When Deep Learning VMs are deployed through VCF Automation self-service catalog items, PAIS requires a password to be specified for the default `vmware` user account via OVF properties at deployment time. Deep Learning VMs also accept a `public-keys` OVF property that populates the default user's `SSH authorized_keys` with provided SSH public key values, supporting key-based access without leaving default keys in place. Deep Learning VM workload deployment fails when invalid authentication credentials are provided for downloading DL workloads, so placeholder or blank credentials cannot persist after first deployment.

PAIS organization-level authentication has no operational default. PAIS authentication is configured with OIDC using an issuer URL, client ID `pais-authentication-id`, and scopes including `openid`, `groups`, and `offline_access`. The `jwtauthenticator` configuration requires specification of an issuer URL, audience ID, username claim field set to `preferred_username`, groups claim field set to `groups`, and TLS certificate authority data. PAIS activation requires coordination with the OIDC provider administrator to set up identity and access management, so authentication is non-functional until these values are configured. OIDC authentication also supports configuration of Extra Audiences for additional OAuth 2.0 clients when required by the OIDC provider. The `vcf cluster register-vcfa-jwt-authenticator` command enables JWT authentication for VKS cluster login to PAIS.

Registry credential configuration is required throughout the PAIS setup workflow. The `oras login` command authenticates to a private Harbor registry using username and password credentials for subsequent image operations. PAIS air-gapped deployments require the location of a self-hosted registry to be provided, and login credentials must be supplied if the registry requires authentication. All passwords entered in the PAIS quickstart, including the private container registry password and proxy password, are saved as secrets in VCF Automation rather than left in plain text. For Model Runtime image pulls, a robot account must be configured in the Harbor registry, and model endpoints deployed via `kubectl` can specify an optional `pullSecrets` image pull secret for OCI registry authentication.

PAIS observability and tooling integrations use bearer-token credentials configured at setup time. PAIS LLM Traces authentication to the OpenTelemetry collector is configured through a Kubernetes Secret containing authorization headers, including a Bearer token and custom headers. Authentication to VCF Automation using the VCF Consumption CLI requires generation of an API token for the user's account in their VCF Automation organization. Knowledge Base data source



authentication for Confluence uses either a user name and password or a personal access token, configured according to the Confluence administrator's settings. Remote MCP servers connected to PAIS can optionally use static authentication tokens.

One default that requires explicit organizational action: PAIS Prometheus TLS client authentication and certificate validation are turned off by default in the Grafana setup for PAIS metrics. Operators who require client-certificate authentication on the metrics path must turn this on as part of the deployment process.

The underlying identity framework, including identity federation, organizational user management, and single sign-on, is provided by VCF SSO and VCF Automation rather than by PAIS itself.

### **VMware vDefend (Contributes)**

The Security Services Platform (SSP) component of VMware vDefend (vDefend) requires mandatory password configuration for its local accounts during initial deployment. When deploying the SSP Installer OVA, the administrator must set passwords for the admin account, which has full privileges to manage the platform, and the audit account, which has access to APIs and services for auditing the system. A password for the sysadmin appliance management account is also required before deployment completes. All passwords must conform to specified password rules at the time of configuration, making it structurally impossible to leave these accounts with default credentials after installation.

After initial deployment, the admin and audit account passwords can be changed by an authorized administrator through either the SSP Installer UI or the SSP Platform UI. The installer user password can also be changed via the command-line interface using the `-u` flag. Any password set or changed through these interfaces must continue to meet the platform's defined password criteria.

The SSP Installer also requires configuration of a System GRUB password during deployment, which controls access to the appliance-level password reset mechanism for local system users. If no GRUB password is explicitly configured during deployment, the SSP Installer applies a default value. Administrators should set a custom GRUB password during deployment to address this additional default authenticator.

vDefend's contribution is scoped to its own management plane. The organizational requirement to change default authenticators on all other systems and accounts across the environment is outside vDefend's scope.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides several mechanisms that support changing default authenticators during account creation and system installation.

For local user accounts, DSM includes a `force_password_change` flag. When set to true, this flag requires the user to change their password at next login. Administrators configure this flag when creating or resetting local user accounts through the DSM administration interface (Administering Data Services Manager > Managing LDAP Users in Data Services Manager). The flag can also be set to false, which means enforcement depends on the administrative policies organizations establish at account creation time.

For MySQL databases, DSM creates a default database user with a defined privilege set, including a default username that can be changed before provisioning the database. When credentials are not manually specified during creation, DSM automatically generates a unique database engine administrator username and password, removing the reliance on a static default credential that would otherwise need to be changed post-deployment.

DSM adds SQL Server as a supported database engine. During SQL Server cluster creation, DSM automatically creates a cluster administrator login and can automatically generate the administrator username and password if they are not manually provided. SQL Server Authentication logins can be created for individual users, and Active Directory integration is available for Windows Authentication when a Privileged Account is configured.

PostgreSQL instances provisioned through DSM use `pg_hba.conf` host-based authentication configuration, which defines authentication methods for specific users and connection types. The default configuration uses `scram-sha-256` as the password authentication method, with `reject` and `cert` as alternatives. Custom HBA entries can be configured at provisioning

time and adjusted in the Additional Settings step of the VCF Automation provisioning interface. Organizations should review and customize HBA configurations after deployment to match their access control requirements.

DSM supports LDAPS integration as an alternative to local user accounts, delegating authentication and authorization to the configured directory service. LDAP or Active Directory-based authentication can also be enabled for individual MySQL and PostgreSQL database instances, either during initial provisioning or during cloning from a snapshot. When directory service authentication is in use, password lifecycle management, including change-at-first-login requirements, is governed by the directory service's own policies.

Auditors should verify that the `force_password_change` flag is used during initial local account setup, that MySQL default database usernames are changed or that auto-generated credentials are used before provisioning, that SQL Server cluster credentials are reviewed after creation, and that PostgreSQL HBA configurations are customized post-deployment to match site security requirements.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) contributes to this control through credential management mechanisms that address the intent of avoiding shared, well-known defaults across deployments.

For administrative access, local authentication is the default method on the Avi Controller. The default deployment creates an admin account that does not mandate specific password requirements at the account level; organizations must configure strong password enforcement separately through User Account Security profiles. When a new user is created and no profile is explicitly selected, the system automatically assigns the Default-User-Account-Profile. Administrators can create custom User Account Security profiles under Administration > User Account Security in the Controller UI, specifying password history, login failure thresholds, and account lock settings beyond what the default profile provides. In multi-node controller clusters, follower nodes may use a default initial admin password or a user-configured one, so post-deployment credential rotation should address all cluster members.

A specific credential-change requirement applies to the audit user account: the Avi Controller audit user password must be changed and must meet required criteria before the system is placed in production.

Default system accounts in Avi cannot be deactivated or removed. Organizations must establish a post-deployment rotation procedure that explicitly addresses these accounts. Locally authenticated users can change their own passwords through My Account in the Controller UI by entering the old password followed by the new one. Administrators can also generate a random password or set a new password manually during a password reset operation.

Because Avi does not enforce a mandatory credential rotation step at first login or as part of account creation for the general admin account, and because default system accounts cannot be disabled, organizations are responsible for the post-deployment procedures that satisfy this control.

Not applicable for this control: VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 1.5(c)

> Sub-item (c) of CR 1.5 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 1.5 requirement in the standard.

**SCF Controls:** IAC-10.1

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF enforces password complexity, length, and lifespan requirements across its component stack, with each component providing configurable password policies.

At the platform level, VCF uses the Linux PAM (Pluggable Authentication Module) framework to enforce default password complexity rules. The default policy requires passwords of at least 12 characters containing at least one lowercase letter, one uppercase letter, one digit, one special character, and at least five different characters. The PAM module limits password entry attempts to a maximum of three retries before returning an error. On VMware ESX hosts, the password complexity policy is enforced for the root user, helping maintain strong credentials on the hypervisor layer.

VMware vCenter's built-in authentication service, VCF SSO, enforces a password policy for local domain accounts with a default password lifetime of 90 days, configurable through the VCF SSO password policy settings in vCenter. For users authenticated through external identity sources such as Active Directory, password restrictions, expiration, and account lockout are governed by the identity source domain rather than VCF SSO policies.

NSX Manager and NSX Edge enforce password complexity through Linux PAM modules. Default requirements include a minimum of 12 characters (maximum 128), at least one lowercase letter, one uppercase letter, one numeric character, one special character, and at least five different characters. NSX Edge adds restrictions against dictionary words, palindromes, and monotonic character sequences of more than four. All of these parameters are configurable through the NSX Authentication Policy Settings or the NSX Manager API, including minimum and maximum password length, required count of uppercase, lowercase, special, and numeric characters, password uniqueness, and hashing algorithm. NSX Manager enforces a retry limit of three attempts when setting a new password. For password lifespan, NSX Manager supports configuring password expiration with a range from 1 to 9,999 days.

VCF Operations applies password complexity requirements at multiple levels. The admin user account requires a minimum of 15 characters with at least one uppercase letter, one lowercase letter, one digit, and one special character. The default password strength policy allows configuration of the minimum password length starting at eight characters. The password policy settings are managed through Administration > Global Settings > User Access > Password Policy, which provides controls for password strength, password change policy, and account lockout. VCF Operations locks user accounts based on the password lockout policy, with a default threshold of seven failed login attempts, or if a user enters an incorrect password three times within five minutes. VCF Operations for Networks locks a user account after five consecutive failed login attempts for 15 minutes.

VCF Automation supports its own password policy system settings, including account lockout after a configurable number of invalid login attempts. The lockout interval specifying the duration before automatic unlock is also configurable. Provider administrators can unlock a user account manually without waiting for the lockout interval to expire. VCF Automation passwords require a minimum of 15 characters.

VCF Operations provides centralized password lifecycle management for all VCF components. Administrators can rotate passwords on demand or schedule automatic rotation on a recurring basis. During rotation, VCF Operations generates a randomized password that conforms to the component's password policy and re-establishes the relevant component integrations automatically. The VCF Operations UI offers preset rotation intervals, and the API supports custom scheduling. VCF Operations also provides a password remediation capability through the Security > Password management interface, allowing administrators to update service account credentials and synchronize passwords when they have drifted out of sync between components.

The VCF API supports managing the local accounts global password policy in vCenter. The `Invoke-GetLocalAccountsGlobalPolicy` and `Invoke-SetLocalAccountsGlobalPolicy` operations allow administrators to retrieve and update the vCenter password policy programmatically through the vSphere Automation SDK for PowerShell.

VCF documentation recommends setting passwords following a common set of requirements across all accounts. While individual components support different password parameters and defaults, administrators can align password policies across the environment to establish a consistent security posture. The combination of enforced defaults, per-component customization options, password expiration controls, and centralized rotation provides the mechanisms needed to enforce complexity, length, and lifespan requirements for password-based authentication throughout the VCF stack.

### **VMware vDefend (Contributes)**

VMware vDefend (vDefend) enforces password complexity, length, and lifespan requirements for its management interface accounts, contributing to the organization's password-based authentication posture.

Security Services Platform (SSP) is the centralized management console for vDefend. SSP enforces password policies for local user accounts: admin and audit user passwords require a minimum of 12 characters and a maximum of 128 characters, and must contain at least one uppercase letter, one lowercase letter, one numeric character, and one special character. The same complexity criteria apply during password changes and resets. Local user passwords are valid for 180 days; a notification banner appears in the SSP UI 30 days before expiration to prompt users to update their password. Installer user passwords are valid for 60 days. The License Hub component of SSP also enforces password complexity requirements during local password change operations.

SSP supports integration with directory services, including Active Directory over LDAP and OpenLDAP. When remote users or remote user groups are configured through LDAP, SSP applies the password expiration settings configured in the directory service; password expiry for these accounts cannot be modified from the SSP UI. SSP uses Authelia, an open-source authentication and authorization server, to handle authentication requests, including two-factor authentication and single sign-on.

For Identity Firewall, vDefend uses an LDAP password to authenticate connections to the directory service used for identity-based policy enforcement.

vDefend is a network security product and not a general-purpose identity management system. Its management interfaces enforce password complexity, length, and lifespan policies for vDefend administrative access. Satisfying this control fully requires applying equivalent password policies across all systems within the organization's environment, which is an organizational responsibility beyond vDefend's scope.

### **VMware Data Services Manager (Meets)**

VMware Data Services Manager (DSM) enforces password complexity and length requirements for all local user accounts. Local user passwords must contain a minimum of 8 and a maximum of 20 characters, including at least one uppercase letter, one lowercase letter, one number, and one special character from the set `(?!@#$$%^&*)`. These requirements apply both at account creation and at every subsequent password reset; DSM rejects any new password that does not satisfy all criteria before accepting the change. Administrators can set the `force_password_change` flag on individual local user accounts to require a password reset at the next login, supporting periodic credential rotation and post-provisioning hygiene workflows.

The root account for the DSM provider VM carries its own complexity requirements: a minimum of 15 and a maximum of 32 characters, requiring at least one uppercase letter, one lowercase letter, one number, and one special character. These constraints are enforced during OVA deployment, so the appliance cannot be instantiated with a weak root credential. The root password expiry date must be set to a valid future date and cannot be configured to never expire, making root credential rotation mandatory. DSM checks root password status at regular intervals and generates a `ROOT_USER_PASSWORD_EXPIRE_SOON` alert when the credential is approaching expiration, giving administrators advance notice before a reset is required.

For PostgreSQL database instances managed by DSM, the default host-based authentication (HBA) configuration applies the `scram-sha-256` algorithm, a salted challenge-response mechanism that avoids transmitting cleartext passwords during

authentication. Custom `pg_hba.conf` entries also default to `scram-sha-256` as the password-based authentication method. Local user management passwords within the DSM platform are encrypted using PGP symmetric encryption with dedicated keys (`PGCRYPTO_PASSWORD` and `PGCRYPTO_ALGORITHM`).

DSM supports both local user authentication and LDAPS-based directory service authentication for platform access. Local users are authenticated by validating the email address and password against DSM's internal store. When LDAPS is configured, password lifecycle management, including organizational complexity and lifespan policies, is delegated to the directory service. PostgreSQL and MySQL database instances can independently be configured for LDAP or Active Directory authentication, allowing database-level credential management to align with enterprise directory policies. For PostgreSQL, the Enable Directory Service Authentication toggle on the database creation form activates this integration when a default directory service is configured. Administrators should configure LDAPS where enterprise password governance policies must extend to database-level credentials, or rely on DSM's built-in local user password policy where directory integration is not used.

For SQL Server database instances, DSM supports SQL User authentication with standard SQL Server logins and passwords. The DSM privileged service account used for automated SQL Server tasks must be configured with its password policy set to Never Expire to prevent service interruptions; administrators should account for this service account exception when reviewing password lifespan policies across DSM-managed database instances.

### **VMware Avi Load Balancer (Meets)**

VMware Avi Load Balancer (Avi) enforces password-based authentication controls through the Avi Controller's administrative identity plane, addressing complexity, length, and lifespan for locally authenticated accounts.

The Controller's strong password enforcement feature requires each password to contain at least one uppercase character, at least one lowercase character, at least one numeric character, and at least one special character from the allowed set. The Controller rejects passwords consisting of sequences of adjacent keyboard keys. These requirements apply to both newly created accounts and to existing accounts when a user changes their password. Complexity enforcement is controlled through the `password_strength_check` setting, which the Controller applies at the portal level regardless of the administrative interface used.

The minimum permissible password length is configurable through the `minimum_password_length` setting. The Controller enforces a minimum of 8 characters by default; administrators can raise this threshold via the REST API or CLI. The Avi License Hub enforces stricter length requirements for its admin and audit accounts: a minimum of 12 characters and a maximum of 128 characters.

Password expiration is enforced through user account profiles using the `password_expiration_days` setting. The Default-User-Account-Profile sets password expiration at 365 days. The Controller displays a notification banner in the UI 30 days before a password expires to prompt renewal. The expiration period is configurable per profile.

Password history enforcement restricts users from reusing recent passwords by comparing proposed updates against a configurable list of previous password hashes. The history count is configurable from 1 to 10 previous passwords, with a default of 5.

Locally authenticated users change their own password through the My Account menu, which requires entry of the current password before accepting a new one. Administrators can generate a random password for a local user account or input a new password manually during a password reset. User-initiated password recovery through the GUI is supported when SMTP email is configured.

These controls apply to locally authenticated Avi Controller accounts. For administrators who authenticate against external directories such as LDAP, password complexity and lifespan are managed by those external directories.

Not applicable for this control: VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 1.5(d)

> Sub-item (d) of CR 1.5 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 1.5 requirement in the standard.

**SCF Controls:** IAC-10.5

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF protects authenticators at rest, in transit, and during use through multiple mechanisms across its component stack. The platform's approach spans credential storage and hashing, token-based session management, certificate-based service authentication, identity federation, and support for stronger authentication methods that reduce reliance on static credentials.

Passwords stored on VCF components are protected using one-way hashing. NSX Manager encrypts all passwords in its system (databases, configuration files, and log files) using a strong one-way hashing algorithm with a salt. Local user passwords on NSX appliances are secured using Linux/PAM libraries, which store hashed and salted representations in `/etc/shadow`. The NSX Manager password hash algorithm is configured to use SHA512. Other internal passwords are encrypted using a random key stored in the local filesystem.

Credential stores used by the vSphere Web Services SDK protect passwords using filesystem-level permissions, with each user having a separate backing file. The CredentialStoreCipher component encrypts and decrypts credential data using the `javax.crypto` libraries. Java keystores used for certificate management are password-protected. At the workload consumption layer, private container image registry credentials are stored in Kubernetes Secret objects and referenced in workload specifications to support authenticated image pulls without embedding credentials in configuration directly. Harbor Registry, deployable as a Supervisor service on VMware Kubernetes Service (VKS) clusters, enforces role-based access control to secure artifact management, and its own sensitive internal credentials (including the admin password, secret key, database password, and session integrity tokens) are scoped to specific authentication functions within the registry.

Service-to-service authentication within VCF uses certificates rather than passwords. Solution users authenticate using X.509 certificates for inter-component communication, which avoids transmitting password-based credentials between services. VCF SSO issues time-limited tokens (bearer tokens for short-term single operations, holder-of-key tokens for delegation scenarios) so that long-lived credentials are not passed repeatedly across the network. The `loginByToken` method authenticates clients using SAML tokens obtained from the SSO server. For programmatic and API-based access, VCF employs holder-of-key security tokens through the `LoginByToken` method, which binds authentication to a specific cryptographic key. This approach provides stronger authenticator protection than simple bearer tokens, which authenticate based solely on possession of the token. The distinction between these token types allows administrators to select the appropriate authenticator strength based on the sensitivity of the operations being performed.

VCF supports identity provider federation, which uses token-based authentication to minimize the risk of credential exposure. By federating authentication to an external identity provider through VCF SSO and the VCF Identity Broker, user credentials such as passwords are not stored or transmitted directly through VMware vCenter or other VCF components. This architecture reduces the attack surface for credential theft by keeping sensitive authenticators within the identity provider's controlled environment. VCF supports SAML and OIDC protocols for federation. The platform also supports the OAuth 2.0 Authorization Code grant type, which protects user credentials by keeping them within the API environment rather than exposing them over the network. This grant type is compatible with multi-factor authentication (MFA), enabling organizations to layer additional authenticator protections for access to sensitive resources.

At the workload consumption layer, VKS clusters running on vSphere Supervisor extend authenticator protection to containerized workloads. Kubernetes Secrets provide the foundational mechanism for storing authenticators such as passwords, tokens, X.509 private keys, and registry credentials; access to Secret objects is governed by Kubernetes RBAC, so users and service accounts must have explicit permissions for specific Secrets or Secret-bearing namespaces. The Secret Store service, available as a Supervisor service, provides an externalized secret management layer with its own RBAC for administrators, DevOps engineers, and application tenants, separating secret lifecycle management from application



configuration. The Kubernetes hardening documentation recommends enabling as few authentication mechanisms as possible within a cluster so that, when multiple authenticator modules are configured, the first module to successfully authenticate a request short-circuits further evaluation, reducing the surface area for credential misuse. Each JWT authenticator configured in the API server must have a unique issuer URL, and at most one JWT authenticator will attempt to cryptographically validate a given token, preventing ambiguity in validation paths. When using TokenRequest tokens for service account authentication, configuring a short token lifespan limits the window of exposure if a token is compromised. The Argo CD Supervisor Service manages CI/CD pipeline secrets outside of Git repositories with access controls enforced across all components, and supports both local account authentication with global RBAC policies and OIDC authentication for federated identity integration.

VCF supports multiple authentication methods including smart cards and certificate-based authentication, enabling organizations to deploy hardware-based authenticators for high-sensitivity environments. vSphere also supports Windows Virtualization-based Security for Windows VMs, which enables Credential Guard to isolate and harden key system and user secrets against compromise within the guest environment. The platform documentation recommends using stricter security practices than basic authentication in production environments, and notes that allowing users to accept self-signed certificates or to select certificate authorization options provides reduced security compared to other methods. These warnings guide administrators toward authenticator configurations that match the sensitivity of the data being protected.

The vSphere Web Services API supports multiple guest authentication types including NamePasswordAuthentication, SAMLTokenAuthentication, SSPIAuthentication, and TicketedSessionAuthentication. This range of authenticator types allows administrators to select authentication mechanisms appropriate to the trust level and sensitivity requirements of each use case, from integrated Windows authentication for domain-joined environments to SAML tokens for federated identity scenarios.

All management communication channels use TLS encryption, protecting authenticators during transmission. NSX Manager API service supports TLS for API communications. VCF Operations orchestrator uses TLS for encrypted communications between deployment components and HTTPS access to the web front end. VCF supports automatic renewal of TLS certificates for components using VMCA, Microsoft CA, OpenSSL, or self-signed certificates, which helps keep authentication material current and reduces the window of exposure for any single credential. Where Istio is deployed on VKS clusters, the enableStrictMTLS setting enforces mutual TLS authentication for all service-to-service communication within the mesh, protecting service account credentials from being accepted by unauthorized endpoints. The Kubernetes hardening documentation recommends setting short lifetimes on certificates and automating their rotation to limit the window of exposure if credentials are compromised.

For storage-layer authentication, vSphere supports Kerberos integration with Active Directory for NFS datastore access, providing an additional layer of protection when storage traffic traverses the network. For iSCSI storage, vSphere hardening guidance recommends configuring unique CHAP secrets per VMware ESX host so that compromise of one host cannot be used to authenticate to the storage array from another.

Backup files that may contain credential material are encrypted with a user-specified encryption passphrase, and administrators are directed to store that passphrase safely since it is required during restore operations. NSX Manager backup data can also be encrypted with a passphrase that must be provided during the restore process. Applications consuming Secrets via environment variables or volume mounts on VKS clusters are responsible for protecting authenticator values after reading them, including avoiding logging secret data in clear text or transmitting it to untrusted parties; the platform provides the access control and storage protections, while commensurate-sensitivity handling at the application level remains an organizational responsibility.

#### **VMware vDefend (Contributes)**

Within vDefend's own management interface, Security Services Platform (SSP) protects authenticators by supporting both local account credentials and directory-managed authentication via Active Directory over LDAP or OpenLDAP, with the SSP Installer enforcing authentication, authorization, and access control as critical elements of user management. In 9.1, SSP uses Authelia, an open-source authentication and authorization server that provides two-factor authentication (2FA) and single



sign-on (SSO) capabilities for platform administrators, giving the management plane a stronger authenticator posture commensurate with the sensitivity of the systems administrators access.

SSP implements role-based access control to limit system access based on user roles, with local user management in the License Hub restricting actions to authorized users only. Local administrative and audit user passwords are established during deployment and can be changed only by authorized users through the SSP UI or Installer UI, restricting credential modification to authenticated administrators.

At the network layer, vDefend's Identity Firewall maps authenticated user identity to IP addresses and applies Distributed Firewall rules based on Active Directory user and group membership rather than source IP address alone. Administrators can create firewall rules using AD users and groups as source criteria, enforce access controls on virtual machines and physical devices through Event Log Scraping for login detection, and apply security policies to virtual machines based on dynamic name-based criteria combined with AD group membership. These mechanisms tie network access decisions to the authenticated identity of the originating user, supporting the broader goal of restricting access to sensitive systems in proportion to their classification.

Protecting authenticators for systems and applications outside of vDefend's management plane, such as enforcing password storage standards or token lifecycle management across the broader organization, remains outside vDefend's scope.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) protects authenticators through encrypted transport protocols, strong credential hashing, and certificate-based directory service validation across its supported database engines. These mechanisms help reduce the risk of credential exposure in transit and during authentication exchanges for both platform access and individual database connections.

For directory service integration, DSM supports LDAPS (LDAP over SSL/TLS) for both the platform UI and individual database instances. MySQL database instances support LDAPS authentication with LDAPS-specific user entries, and Active Directory authentication is conducted over the AD Secure/SSL Global Catalog port 3269 for multi-domain controller deployments. Directory service authentication is enabled or disabled per database through the "Enable Directory Service Authentication" toggle in the database settings, including on existing databases after creation. When Directory Service credentials are non-functional, DSM falls back to local user authentication to maintain administrative access continuity.

For PostgreSQL database instances, DSM configures host-based authentication (HBA) using scram-sha-256 as the default credential hashing method. Default HBA entries cover the postgres\_exporter and pgadmin accounts using SCRAM-SHA-256, along with a catchall entry applying scram-sha-256 to all remaining users. Custom HBA entries are restricted to scram-sha-256, reject, and cert as the permitted authentication methods. HBA rules use ordered matching based on connection type, client address, requested database, and user name, giving administrators control over which authentication method applies to which connection patterns.

For SQL Server database instances, DSM data service policies can be configured to allow Windows Principal authentication, enabling users to authenticate using their Active Directory credentials, or SQL User authentication using standard SQL Server logins and passwords. DSM enforces an isolated internal management login (dsm\_mgmt) that restricts user access to system-level objects, keeping administrative operations within a secured context separate from database user credentials.

For the platform UI, DSM first attempts to authenticate users as local users by matching the email address and validating the password. If a local user match is found, no further LDAPS processing occurs. LDAPS users are authenticated by email domain matching against the registered LDAPS server. DSM also supports mutual TLS (mTLS) authentication for metrics forwarding, enabled through the Client Certificate Authentication toggle with a client certificate and private key.

Full satisfaction of this control requires organizational policies that classify the sensitivity of information stored in DSM-managed databases, establish commensurate authenticator protection requirements such as password strength, rotation schedules, and acceptable credential storage practices, and govern credential lifecycle management beyond what the platform itself enforces. DSM provides the technical mechanisms to implement strong authenticator protection, but determining what protection level is appropriate for a given database workload is an organizational responsibility.

### VMware Avi Load Balancer (Contributes)

VMware Avi Load Balancer provides authenticator protection mechanisms across both its management plane and application delivery layer.

For the Avi Controller management plane, the product supports federated authentication via SAML, LDAP, OAuth 2.0, and OpenID Connect. SAML authentication implements federated authentication by delegating the process to a centralized identity provider (IdP), which verifies user identity without storing credentials on multiple untrusted devices. Avi guidance explicitly recommends federated authentication as the preferred pattern to centralize credential management and reduce the footprint of locally held authenticators. SAML authentication rules on the Controller support two actions: Skip Authentication, which bypasses authentication when a rule matches, and Default Authentication, which enforces SAML for matched paths, enabling sensitivity-based application of authentication requirements. LDAP authentication validates user identity against a directory service, centralizing credential storage in the organization's existing identity infrastructure. The LDAP configuration also supports Required User Group Membership enforcement, restricting access to users who belong to designated groups.

Auth Mapping Profiles provide additional control over authenticated sessions. The Avi Controller can map LDAP group memberships to specific roles and tenants, and can deny privileges entirely to specific users by updating Auth Mapping rules, preventing those users from obtaining any role or tenant assignment upon login.

For application traffic, Avi supports OAuth/OIDC flows where the authorization server authenticates the resource owner before issuing access tokens. OAuth provides secure delegated access by allowing the identity provider to issue tokens to authorized parties without requiring users to share credentials with intermediary applications. Granular authorization policies can apply additional controls on top of token-based access. JWT validation is available through configurable SSO Policies with named Authentication Rules and Authorization Rules applied per request path, allowing different authentication profiles to be assigned based on the sensitivity of the resource.

Access to SSL/TLS certificate objects, which serve as machine authenticators for virtual services, is scoped by role-based access control: only administrator accounts with write-access to certificates for the required tenant can manage those objects, reducing exposure of certificate-based authenticators.

The Avi Controller VM and Service Engine VMs rely on the underlying VCF platform for data-at-rest encryption of disk contents, including any stored credential material. Avi does not manage its own at-rest encryption; that layer is inherited from VCF's VM Encryption or vSAN Encryption capabilities.

Applying authentication protection commensurate with information sensitivity requires the organization to classify assets and define appropriate authentication requirements for each level. Avi provides the mechanisms; the organization selects and applies them based on its asset classification and risk management process.

Not applicable for this control: VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 1.6

> Wireless access management: Capability to identify and authenticate all users and devices on wireless networks.

**SCF Controls:** CRY-07

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 1.7

> Password strength enhancement: Capability to enforce specific password complexity rules. Strength of password-based authentication: Enforce configurable password strength requirements.

**SCF Controls:** IAC-10.1

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF enforces password complexity, length, and lifespan requirements across its component stack, with each component providing configurable password policies.

At the platform level, VCF uses the Linux PAM (Pluggable Authentication Module) framework to enforce default password complexity rules. The default policy requires passwords of at least 12 characters containing at least one lowercase letter, one uppercase letter, one digit, one special character, and at least five different characters. The PAM module limits password entry attempts to a maximum of three retries before returning an error. On VMware ESX hosts, the password complexity policy is enforced for the root user, helping maintain strong credentials on the hypervisor layer.

VMware vCenter's built-in authentication service, VCF SSO, enforces a password policy for local domain accounts with a default password lifetime of 90 days, configurable through the VCF SSO password policy settings in vCenter. For users authenticated through external identity sources such as Active Directory, password restrictions, expiration, and account lockout are governed by the identity source domain rather than VCF SSO policies.

NSX Manager and NSX Edge enforce password complexity through Linux PAM modules. Default requirements include a minimum of 12 characters (maximum 128), at least one lowercase letter, one uppercase letter, one numeric character, one special character, and at least five different characters. NSX Edge adds restrictions against dictionary words, palindromes, and monotonic character sequences of more than four. All of these parameters are configurable through the NSX Authentication Policy Settings or the NSX Manager API, including minimum and maximum password length, required count of uppercase, lowercase, special, and numeric characters, password uniqueness, and hashing algorithm. NSX Manager enforces a retry limit of three attempts when setting a new password. For password lifespan, NSX Manager supports configuring password expiration with a range from 1 to 9,999 days.

VCF Operations applies password complexity requirements at multiple levels. The admin user account requires a minimum of 15 characters with at least one uppercase letter, one lowercase letter, one digit, and one special character. The default password strength policy allows configuration of the minimum password length starting at eight characters. The password policy settings are managed through Administration > Global Settings > User Access > Password Policy, which provides controls for password strength, password change policy, and account lockout. VCF Operations locks user accounts based on the password lockout policy, with a default threshold of seven failed login attempts, or if a user enters an incorrect password three times within five minutes. VCF Operations for Networks locks a user account after five consecutive failed login attempts for 15 minutes.

VCF Automation supports its own password policy system settings, including account lockout after a configurable number of invalid login attempts. The lockout interval specifying the duration before automatic unlock is also configurable. Provider administrators can unlock a user account manually without waiting for the lockout interval to expire. VCF Automation passwords require a minimum of 15 characters.

VCF Operations provides centralized password lifecycle management for all VCF components. Administrators can rotate passwords on demand or schedule automatic rotation on a recurring basis. During rotation, VCF Operations generates a randomized password that conforms to the component's password policy and re-establishes the relevant component integrations automatically. The VCF Operations UI offers preset rotation intervals, and the API supports custom scheduling. VCF Operations also provides a password remediation capability through the Security > Password management interface, allowing administrators to update service account credentials and synchronize passwords when they have drifted out of sync between components.

The VCF API supports managing the local accounts global password policy in vCenter. The `Invoke-GetLocalAccountsGlobalPolicy` and `Invoke-SetLocalAccountsGlobalPolicy` operations allow administrators to retrieve and update the vCenter password policy programmatically through the vSphere Automation SDK for PowerShell.

VCF documentation recommends setting passwords following a common set of requirements across all accounts. While individual components support different password parameters and defaults, administrators can align password policies across the environment to establish a consistent security posture. The combination of enforced defaults, per-component customization options, password expiration controls, and centralized rotation provides the mechanisms needed to enforce complexity, length, and lifespan requirements for password-based authentication throughout the VCF stack.

### **VMware vDefend (Contributes)**

VMware vDefend (vDefend) enforces password complexity, length, and lifespan requirements for its management interface accounts, contributing to the organization's password-based authentication posture.

Security Services Platform (SSP) is the centralized management console for vDefend. SSP enforces password policies for local user accounts: admin and audit user passwords require a minimum of 12 characters and a maximum of 128 characters, and must contain at least one uppercase letter, one lowercase letter, one numeric character, and one special character. The same complexity criteria apply during password changes and resets. Local user passwords are valid for 180 days; a notification banner appears in the SSP UI 30 days before expiration to prompt users to update their password. Installer user passwords are valid for 60 days. The License Hub component of SSP also enforces password complexity requirements during local password change operations.

SSP supports integration with directory services, including Active Directory over LDAP and OpenLDAP. When remote users or remote user groups are configured through LDAP, SSP applies the password expiration settings configured in the directory service; password expiry for these accounts cannot be modified from the SSP UI. SSP uses Authelia, an open-source authentication and authorization server, to handle authentication requests, including two-factor authentication and single sign-on.

For Identity Firewall, vDefend uses an LDAP password to authenticate connections to the directory service used for identity-based policy enforcement.

vDefend is a network security product and not a general-purpose identity management system. Its management interfaces enforce password complexity, length, and lifespan policies for vDefend administrative access. Satisfying this control fully requires applying equivalent password policies across all systems within the organization's environment, which is an organizational responsibility beyond vDefend's scope.

### **VMware Data Services Manager (Meets)**

VMware Data Services Manager (DSM) enforces password complexity and length requirements for all local user accounts. Local user passwords must contain a minimum of 8 and a maximum of 20 characters, including at least one uppercase letter, one lowercase letter, one number, and one special character from the set `(?!@#$$%^&*)`. These requirements apply both at account creation and at every subsequent password reset; DSM rejects any new password that does not satisfy all criteria before accepting the change. Administrators can set the `force_password_change` flag on individual local user accounts to require a password reset at the next login, supporting periodic credential rotation and post-provisioning hygiene workflows.

The root account for the DSM provider VM carries its own complexity requirements: a minimum of 15 and a maximum of 32 characters, requiring at least one uppercase letter, one lowercase letter, one number, and one special character. These constraints are enforced during OVA deployment, so the appliance cannot be instantiated with a weak root credential. The root password expiry date must be set to a valid future date and cannot be configured to never expire, making root credential rotation mandatory. DSM checks root password status at regular intervals and generates a `ROOT_USER_PASSWORD_EXPIRE_SOON` alert when the credential is approaching expiration, giving administrators advance notice before a reset is required.

For PostgreSQL database instances managed by DSM, the default host-based authentication (HBA) configuration applies the `scram-sha-256` algorithm, a salted challenge-response mechanism that avoids transmitting cleartext passwords during

authentication. Custom `pg_hba.conf` entries also default to `scram-sha-256` as the password-based authentication method. Local user management passwords within the DSM platform are encrypted using PGP symmetric encryption with dedicated keys (`PGCRYPTO_PASSWORD` and `PGCRYPTO_ALGORITHM`).

DSM supports both local user authentication and LDAPS-based directory service authentication for platform access. Local users are authenticated by validating the email address and password against DSM's internal store. When LDAPS is configured, password lifecycle management, including organizational complexity and lifespan policies, is delegated to the directory service. PostgreSQL and MySQL database instances can independently be configured for LDAP or Active Directory authentication, allowing database-level credential management to align with enterprise directory policies. For PostgreSQL, the Enable Directory Service Authentication toggle on the database creation form activates this integration when a default directory service is configured. Administrators should configure LDAPS where enterprise password governance policies must extend to database-level credentials, or rely on DSM's built-in local user password policy where directory integration is not used.

For SQL Server database instances, DSM supports SQL User authentication with standard SQL Server logins and passwords. The DSM privileged service account used for automated SQL Server tasks must be configured with its password policy set to Never Expire to prevent service interruptions; administrators should account for this service account exception when reviewing password lifespan policies across DSM-managed database instances.

### **VMware Avi Load Balancer (Meets)**

VMware Avi Load Balancer (Avi) enforces password-based authentication controls through the Avi Controller's administrative identity plane, addressing complexity, length, and lifespan for locally authenticated accounts.

The Controller's strong password enforcement feature requires each password to contain at least one uppercase character, at least one lowercase character, at least one numeric character, and at least one special character from the allowed set. The Controller rejects passwords consisting of sequences of adjacent keyboard keys. These requirements apply to both newly created accounts and to existing accounts when a user changes their password. Complexity enforcement is controlled through the `password_strength_check` setting, which the Controller applies at the portal level regardless of the administrative interface used.

The minimum permissible password length is configurable through the `minimum_password_length` setting. The Controller enforces a minimum of 8 characters by default; administrators can raise this threshold via the REST API or CLI. The Avi License Hub enforces stricter length requirements for its admin and audit accounts: a minimum of 12 characters and a maximum of 128 characters.

Password expiration is enforced through user account profiles using the `password_expiration_days` setting. The Default-User-Account-Profile sets password expiration at 365 days. The Controller displays a notification banner in the UI 30 days before a password expires to prompt renewal. The expiration period is configurable per profile.

Password history enforcement restricts users from reusing recent passwords by comparing proposed updates against a configurable list of previous password hashes. The history count is configurable from 1 to 10 previous passwords, with a default of 5.

Locally authenticated users change their own password through the My Account menu, which requires entry of the current password before accepting a new one. Administrators can generate a random password for a local user account or input a new password manually during a password reset. User-initiated password recovery through the GUI is supported when SMTP email is configured.

These controls apply to locally authenticated Avi Controller accounts. For administrators who authenticate against external directories such as LDAP, password complexity and lifespan are managed by those external directories.

Not applicable for this control: VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 1.7(1)

> Password strength enhancement: Capability to enforce specific password complexity rules. Strength of password-based authentication: Enforce configurable password strength requirements.

**SCF Controls:** IAC-10.1

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF enforces password complexity, length, and lifespan requirements across its component stack, with each component providing configurable password policies.

At the platform level, VCF uses the Linux PAM (Pluggable Authentication Module) framework to enforce default password complexity rules. The default policy requires passwords of at least 12 characters containing at least one lowercase letter, one uppercase letter, one digit, one special character, and at least five different characters. The PAM module limits password entry attempts to a maximum of three retries before returning an error. On VMware ESX hosts, the password complexity policy is enforced for the root user, helping maintain strong credentials on the hypervisor layer.

VMware vCenter's built-in authentication service, VCF SSO, enforces a password policy for local domain accounts with a default password lifetime of 90 days, configurable through the VCF SSO password policy settings in vCenter. For users authenticated through external identity sources such as Active Directory, password restrictions, expiration, and account lockout are governed by the identity source domain rather than VCF SSO policies.

NSX Manager and NSX Edge enforce password complexity through Linux PAM modules. Default requirements include a minimum of 12 characters (maximum 128), at least one lowercase letter, one uppercase letter, one numeric character, one special character, and at least five different characters. NSX Edge adds restrictions against dictionary words, palindromes, and monotonic character sequences of more than four. All of these parameters are configurable through the NSX Authentication Policy Settings or the NSX Manager API, including minimum and maximum password length, required count of uppercase, lowercase, special, and numeric characters, password uniqueness, and hashing algorithm. NSX Manager enforces a retry limit of three attempts when setting a new password. For password lifespan, NSX Manager supports configuring password expiration with a range from 1 to 9,999 days.

VCF Operations applies password complexity requirements at multiple levels. The admin user account requires a minimum of 15 characters with at least one uppercase letter, one lowercase letter, one digit, and one special character. The default password strength policy allows configuration of the minimum password length starting at eight characters. The password policy settings are managed through Administration > Global Settings > User Access > Password Policy, which provides controls for password strength, password change policy, and account lockout. VCF Operations locks user accounts based on the password lockout policy, with a default threshold of seven failed login attempts, or if a user enters an incorrect password three times within five minutes. VCF Operations for Networks locks a user account after five consecutive failed login attempts for 15 minutes.

VCF Automation supports its own password policy system settings, including account lockout after a configurable number of invalid login attempts. The lockout interval specifying the duration before automatic unlock is also configurable. Provider administrators can unlock a user account manually without waiting for the lockout interval to expire. VCF Automation passwords require a minimum of 15 characters.

VCF Operations provides centralized password lifecycle management for all VCF components. Administrators can rotate passwords on demand or schedule automatic rotation on a recurring basis. During rotation, VCF Operations generates a randomized password that conforms to the component's password policy and re-establishes the relevant component integrations automatically. The VCF Operations UI offers preset rotation intervals, and the API supports custom scheduling. VCF Operations also provides a password remediation capability through the Security > Password management interface, allowing administrators to update service account credentials and synchronize passwords when they have drifted out of sync between components.



The VCF API supports managing the local accounts global password policy in vCenter. The `Invoke-GetLocalAccountsGlobalPolicy` and `Invoke-SetLocalAccountsGlobalPolicy` operations allow administrators to retrieve and update the vCenter password policy programmatically through the vSphere Automation SDK for PowerShell.

VCF documentation recommends setting passwords following a common set of requirements across all accounts. While individual components support different password parameters and defaults, administrators can align password policies across the environment to establish a consistent security posture. The combination of enforced defaults, per-component customization options, password expiration controls, and centralized rotation provides the mechanisms needed to enforce complexity, length, and lifespan requirements for password-based authentication throughout the VCF stack.

### **VMware vDefend (Contributes)**

VMware vDefend (vDefend) enforces password complexity, length, and lifespan requirements for its management interface accounts, contributing to the organization's password-based authentication posture.

Security Services Platform (SSP) is the centralized management console for vDefend. SSP enforces password policies for local user accounts: admin and audit user passwords require a minimum of 12 characters and a maximum of 128 characters, and must contain at least one uppercase letter, one lowercase letter, one numeric character, and one special character. The same complexity criteria apply during password changes and resets. Local user passwords are valid for 180 days; a notification banner appears in the SSP UI 30 days before expiration to prompt users to update their password. Installer user passwords are valid for 60 days. The License Hub component of SSP also enforces password complexity requirements during local password change operations.

SSP supports integration with directory services, including Active Directory over LDAP and OpenLDAP. When remote users or remote user groups are configured through LDAP, SSP applies the password expiration settings configured in the directory service; password expiry for these accounts cannot be modified from the SSP UI. SSP uses Authelia, an open-source authentication and authorization server, to handle authentication requests, including two-factor authentication and single sign-on.

For Identity Firewall, vDefend uses an LDAP password to authenticate connections to the directory service used for identity-based policy enforcement.

vDefend is a network security product and not a general-purpose identity management system. Its management interfaces enforce password complexity, length, and lifespan policies for vDefend administrative access. Satisfying this control fully requires applying equivalent password policies across all systems within the organization's environment, which is an organizational responsibility beyond vDefend's scope.

### **VMware Data Services Manager (Meets)**

VMware Data Services Manager (DSM) enforces password complexity and length requirements for all local user accounts. Local user passwords must contain a minimum of 8 and a maximum of 20 characters, including at least one uppercase letter, one lowercase letter, one number, and one special character from the set `(?!@#$$%^&*)`. These requirements apply both at account creation and at every subsequent password reset; DSM rejects any new password that does not satisfy all criteria before accepting the change. Administrators can set the `force_password_change` flag on individual local user accounts to require a password reset at the next login, supporting periodic credential rotation and post-provisioning hygiene workflows.

The root account for the DSM provider VM carries its own complexity requirements: a minimum of 15 and a maximum of 32 characters, requiring at least one uppercase letter, one lowercase letter, one number, and one special character. These constraints are enforced during OVA deployment, so the appliance cannot be instantiated with a weak root credential. The root password expiry date must be set to a valid future date and cannot be configured to never expire, making root credential rotation mandatory. DSM checks root password status at regular intervals and generates a `ROOT_USER_PASSWORD_EXPIRE_SOON` alert when the credential is approaching expiration, giving administrators advance notice before a reset is required.

For PostgreSQL database instances managed by DSM, the default host-based authentication (HBA) configuration applies the `scram-sha-256` algorithm, a salted challenge-response mechanism that avoids transmitting cleartext passwords during

authentication. Custom `pg_hba.conf` entries also default to `scram-sha-256` as the password-based authentication method. Local user management passwords within the DSM platform are encrypted using PGP symmetric encryption with dedicated keys (`PGCRYPTO_PASSWORD` and `PGCRYPTO_ALGORITHM`).

DSM supports both local user authentication and LDAPS-based directory service authentication for platform access. Local users are authenticated by validating the email address and password against DSM's internal store. When LDAPS is configured, password lifecycle management, including organizational complexity and lifespan policies, is delegated to the directory service. PostgreSQL and MySQL database instances can independently be configured for LDAP or Active Directory authentication, allowing database-level credential management to align with enterprise directory policies. For PostgreSQL, the Enable Directory Service Authentication toggle on the database creation form activates this integration when a default directory service is configured. Administrators should configure LDAPS where enterprise password governance policies must extend to database-level credentials, or rely on DSM's built-in local user password policy where directory integration is not used.

For SQL Server database instances, DSM supports SQL User authentication with standard SQL Server logins and passwords. The DSM privileged service account used for automated SQL Server tasks must be configured with its password policy set to Never Expire to prevent service interruptions; administrators should account for this service account exception when reviewing password lifespan policies across DSM-managed database instances.

### **VMware Avi Load Balancer (Meets)**

VMware Avi Load Balancer (Avi) enforces password-based authentication controls through the Avi Controller's administrative identity plane, addressing complexity, length, and lifespan for locally authenticated accounts.

The Controller's strong password enforcement feature requires each password to contain at least one uppercase character, at least one lowercase character, at least one numeric character, and at least one special character from the allowed set. The Controller rejects passwords consisting of sequences of adjacent keyboard keys. These requirements apply to both newly created accounts and to existing accounts when a user changes their password. Complexity enforcement is controlled through the `password_strength_check` setting, which the Controller applies at the portal level regardless of the administrative interface used.

The minimum permissible password length is configurable through the `minimum_password_length` setting. The Controller enforces a minimum of 8 characters by default; administrators can raise this threshold via the REST API or CLI. The Avi License Hub enforces stricter length requirements for its admin and audit accounts: a minimum of 12 characters and a maximum of 128 characters.

Password expiration is enforced through user account profiles using the `password_expiration_days` setting. The Default-User-Account-Profile sets password expiration at 365 days. The Controller displays a notification banner in the UI 30 days before a password expires to prompt renewal. The expiration period is configurable per profile.

Password history enforcement restricts users from reusing recent passwords by comparing proposed updates against a configurable list of previous password hashes. The history count is configurable from 1 to 10 previous passwords, with a default of 5.

Locally authenticated users change their own password through the My Account menu, which requires entry of the current password before accepting a new one. Administrators can generate a random password for a local user account or input a new password manually during a password reset. User-initiated password recovery through the GUI is supported when SMTP email is configured.

These controls apply to locally authenticated Avi Controller accounts. For administrators who authenticate against external directories such as LDAP, password complexity and lifespan are managed by those external directories.

Not applicable for this control: VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 1.7(2)

> Requirement enhancement (2) of CR 1.7 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 1.7 requirement in the standard.

**SCF Controls:** IAC-10.1

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF enforces password complexity, length, and lifespan requirements across its component stack, with each component providing configurable password policies.

At the platform level, VCF uses the Linux PAM (Pluggable Authentication Module) framework to enforce default password complexity rules. The default policy requires passwords of at least 12 characters containing at least one lowercase letter, one uppercase letter, one digit, one special character, and at least five different characters. The PAM module limits password entry attempts to a maximum of three retries before returning an error. On VMware ESX hosts, the password complexity policy is enforced for the root user, helping maintain strong credentials on the hypervisor layer.

VMware vCenter's built-in authentication service, VCF SSO, enforces a password policy for local domain accounts with a default password lifetime of 90 days, configurable through the VCF SSO password policy settings in vCenter. For users authenticated through external identity sources such as Active Directory, password restrictions, expiration, and account lockout are governed by the identity source domain rather than VCF SSO policies.

NSX Manager and NSX Edge enforce password complexity through Linux PAM modules. Default requirements include a minimum of 12 characters (maximum 128), at least one lowercase letter, one uppercase letter, one numeric character, one special character, and at least five different characters. NSX Edge adds restrictions against dictionary words, palindromes, and monotonic character sequences of more than four. All of these parameters are configurable through the NSX Authentication Policy Settings or the NSX Manager API, including minimum and maximum password length, required count of uppercase, lowercase, special, and numeric characters, password uniqueness, and hashing algorithm. NSX Manager enforces a retry limit of three attempts when setting a new password. For password lifespan, NSX Manager supports configuring password expiration with a range from 1 to 9,999 days.

VCF Operations applies password complexity requirements at multiple levels. The admin user account requires a minimum of 15 characters with at least one uppercase letter, one lowercase letter, one digit, and one special character. The default password strength policy allows configuration of the minimum password length starting at eight characters. The password policy settings are managed through Administration > Global Settings > User Access > Password Policy, which provides controls for password strength, password change policy, and account lockout. VCF Operations locks user accounts based on the password lockout policy, with a default threshold of seven failed login attempts, or if a user enters an incorrect password three times within five minutes. VCF Operations for Networks locks a user account after five consecutive failed login attempts for 15 minutes.

VCF Automation supports its own password policy system settings, including account lockout after a configurable number of invalid login attempts. The lockout interval specifying the duration before automatic unlock is also configurable. Provider administrators can unlock a user account manually without waiting for the lockout interval to expire. VCF Automation passwords require a minimum of 15 characters.

VCF Operations provides centralized password lifecycle management for all VCF components. Administrators can rotate passwords on demand or schedule automatic rotation on a recurring basis. During rotation, VCF Operations generates a randomized password that conforms to the component's password policy and re-establishes the relevant component integrations automatically. The VCF Operations UI offers preset rotation intervals, and the API supports custom scheduling. VCF Operations also provides a password remediation capability through the Security > Password management interface, allowing administrators to update service account credentials and synchronize passwords when they have drifted out of sync between components.

The VCF API supports managing the local accounts global password policy in vCenter. The `Invoke-GetLocalAccountsGlobalPolicy` and `Invoke-SetLocalAccountsGlobalPolicy` operations allow administrators to retrieve and update the vCenter password policy programmatically through the vSphere Automation SDK for PowerShell.

VCF documentation recommends setting passwords following a common set of requirements across all accounts. While individual components support different password parameters and defaults, administrators can align password policies across the environment to establish a consistent security posture. The combination of enforced defaults, per-component customization options, password expiration controls, and centralized rotation provides the mechanisms needed to enforce complexity, length, and lifespan requirements for password-based authentication throughout the VCF stack.

### **VMware vDefend (Contributes)**

VMware vDefend (vDefend) enforces password complexity, length, and lifespan requirements for its management interface accounts, contributing to the organization's password-based authentication posture.

Security Services Platform (SSP) is the centralized management console for vDefend. SSP enforces password policies for local user accounts: admin and audit user passwords require a minimum of 12 characters and a maximum of 128 characters, and must contain at least one uppercase letter, one lowercase letter, one numeric character, and one special character. The same complexity criteria apply during password changes and resets. Local user passwords are valid for 180 days; a notification banner appears in the SSP UI 30 days before expiration to prompt users to update their password. Installer user passwords are valid for 60 days. The License Hub component of SSP also enforces password complexity requirements during local password change operations.

SSP supports integration with directory services, including Active Directory over LDAP and OpenLDAP. When remote users or remote user groups are configured through LDAP, SSP applies the password expiration settings configured in the directory service; password expiry for these accounts cannot be modified from the SSP UI. SSP uses Authelia, an open-source authentication and authorization server, to handle authentication requests, including two-factor authentication and single sign-on.

For Identity Firewall, vDefend uses an LDAP password to authenticate connections to the directory service used for identity-based policy enforcement.

vDefend is a network security product and not a general-purpose identity management system. Its management interfaces enforce password complexity, length, and lifespan policies for vDefend administrative access. Satisfying this control fully requires applying equivalent password policies across all systems within the organization's environment, which is an organizational responsibility beyond vDefend's scope.

### **VMware Data Services Manager (Meets)**

VMware Data Services Manager (DSM) enforces password complexity and length requirements for all local user accounts. Local user passwords must contain a minimum of 8 and a maximum of 20 characters, including at least one uppercase letter, one lowercase letter, one number, and one special character from the set `(?!@#$$%^&*)`. These requirements apply both at account creation and at every subsequent password reset; DSM rejects any new password that does not satisfy all criteria before accepting the change. Administrators can set the `force_password_change` flag on individual local user accounts to require a password reset at the next login, supporting periodic credential rotation and post-provisioning hygiene workflows.

The root account for the DSM provider VM carries its own complexity requirements: a minimum of 15 and a maximum of 32 characters, requiring at least one uppercase letter, one lowercase letter, one number, and one special character. These constraints are enforced during OVA deployment, so the appliance cannot be instantiated with a weak root credential. The root password expiry date must be set to a valid future date and cannot be configured to never expire, making root credential rotation mandatory. DSM checks root password status at regular intervals and generates a `ROOT_USER_PASSWORD_EXPIRE_SOON` alert when the credential is approaching expiration, giving administrators advance notice before a reset is required.

For PostgreSQL database instances managed by DSM, the default host-based authentication (HBA) configuration applies the `scram-sha-256` algorithm, a salted challenge-response mechanism that avoids transmitting cleartext passwords during

authentication. Custom `pg_hba.conf` entries also default to `scram-sha-256` as the password-based authentication method. Local user management passwords within the DSM platform are encrypted using PGP symmetric encryption with dedicated keys (`PGCRYPTO_PASSWORD` and `PGCRYPTO_ALGORITHM`).

DSM supports both local user authentication and LDAPS-based directory service authentication for platform access. Local users are authenticated by validating the email address and password against DSM's internal store. When LDAPS is configured, password lifecycle management, including organizational complexity and lifespan policies, is delegated to the directory service. PostgreSQL and MySQL database instances can independently be configured for LDAP or Active Directory authentication, allowing database-level credential management to align with enterprise directory policies. For PostgreSQL, the Enable Directory Service Authentication toggle on the database creation form activates this integration when a default directory service is configured. Administrators should configure LDAPS where enterprise password governance policies must extend to database-level credentials, or rely on DSM's built-in local user password policy where directory integration is not used.

For SQL Server database instances, DSM supports SQL User authentication with standard SQL Server logins and passwords. The DSM privileged service account used for automated SQL Server tasks must be configured with its password policy set to Never Expire to prevent service interruptions; administrators should account for this service account exception when reviewing password lifespan policies across DSM-managed database instances.

### **VMware Avi Load Balancer (Meets)**

VMware Avi Load Balancer (Avi) enforces password-based authentication controls through the Avi Controller's administrative identity plane, addressing complexity, length, and lifespan for locally authenticated accounts.

The Controller's strong password enforcement feature requires each password to contain at least one uppercase character, at least one lowercase character, at least one numeric character, and at least one special character from the allowed set. The Controller rejects passwords consisting of sequences of adjacent keyboard keys. These requirements apply to both newly created accounts and to existing accounts when a user changes their password. Complexity enforcement is controlled through the `password_strength_check` setting, which the Controller applies at the portal level regardless of the administrative interface used.

The minimum permissible password length is configurable through the `minimum_password_length` setting. The Controller enforces a minimum of 8 characters by default; administrators can raise this threshold via the REST API or CLI. The Avi License Hub enforces stricter length requirements for its admin and audit accounts: a minimum of 12 characters and a maximum of 128 characters.

Password expiration is enforced through user account profiles using the `password_expiration_days` setting. The Default-User-Account-Profile sets password expiration at 365 days. The Controller displays a notification banner in the UI 30 days before a password expires to prompt renewal. The expiration period is configurable per profile.

Password history enforcement restricts users from reusing recent passwords by comparing proposed updates against a configurable list of previous password hashes. The history count is configurable from 1 to 10 previous passwords, with a default of 5.

Locally authenticated users change their own password through the My Account menu, which requires entry of the current password before accepting a new one. Administrators can generate a random password for a local user account or input a new password manually during a password reset. User-initiated password recovery through the GUI is supported when SMTP email is configured.

These controls apply to locally authenticated Avi Controller accounts. For administrators who authenticate against external directories such as LDAP, password complexity and lifespan are managed by those external directories.

Not applicable for this control: VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 1.8

> Public key infrastructure certificates: Capability to use and validate digital certificates.

**SCF Controls:** CRY-08

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF includes a purpose-built internal certificate authority and certificate lifecycle management capabilities that directly address the need for a securely implemented Public Key Infrastructure.

At the core of VCF's PKI capabilities is the VMware Certificate Authority (VMCA), an internal certificate authority that ships with every vSphere deployment and is designed to serve the certificate needs of the VCF environment. VMCA provides certificate issuance and signing without requiring an external CA. It can function as the sole certificate authority for all VCF components, or organizations can configure it as a subordinate CA under an external root, integrating into an existing enterprise PKI hierarchy. When external certificates are provided during deployment, they are not replaced by VMCA-signed certificates, preserving the organization's PKI chain of trust. Organizations can also adopt a hybrid model, using third-party certificates for externally facing services while relying on VMCA internally.

For organizations that require externally issued certificates, VCF integrates with multiple external PKI providers. The certificate management system supports Microsoft Certificate Authority (MSCA), OpenSSL CA, and self-signed certificates alongside VMCA. Microsoft Active Directory Certificate Services integration is directly supported, with VCF Operations connecting to a Microsoft CA using basic authentication to manage the full certificate request and issuance workflow. A dedicated svc-vcf-ca service account is configured with least-privilege permissions on the Microsoft CA template to handle certificate operations. Administrators can also use any external CA by generating Certificate Signing Requests (CSRs) through VCF Operations or the vSphere Automation API, submitting them to the CA of their choice, and importing the signed certificates back into the environment. VCF Certificate Management allows importing trusted certificates through the Provider Management Portal, where certificates can be uploaded as PEM files containing certificate chains and optionally associated private keys. The Certificates Library in VCF Operations also supports importing certificate chains from PEM files along with their associated private keys.

VCF provides automated certificate lifecycle management across the platform. Automatic renewal of TLS certificates occurs 60 days prior to certificate expiration for components that have VMCA, Microsoft CA, OpenSSL, or self-signed certificates and support non-disruptive certificate updates. For components that require coordinated maintenance windows, VCF Operations supports manual renewal of TLS certificates. Machine SSL certificate renewal duration is configurable, with a maximum of 730 days and a default of 730 days if no value is specified. The VMCA root certificate expires after 10 years by default, and all certificates signed by VMCA expire no later than when the root certificate expires. The VMCARoot API endpoint allows administrators to replace the VMCA root certificate with a new certificate, which triggers a restart of services using that certificate to pick up the change. The vSphere Automation API can refresh VMCA-issued certificates and add external and third-party certificates to the environment, providing programmatic control over certificate operations. PowerCLI exposes the New-VMachineCertificateSigningRequest command, which generates CSRs for VMware ESX hosts with configurable parameters including country and locality.

VCF Operations provides centralized certificate visibility and management from a single interface. Administrators can view certificate status, generate CSRs, import certificates from PEM files or URLs, replace expiring certificates, and enable automatic renewal. A Certificates widget displays certificates that require attention based on their expiration date, and VCF Health enables administrators to see all expiring certificates at a glance. The Certificates tab in VCF Operations for each workload domain displays certificate expiration status. The SoS Utility includes a --certificate-health option that verifies component certificates are valid and within the expiry date.

Certificate management is available at multiple layers of the stack. VCF Operations handles fleet-wide certificate operations, with Platform Operators able to manage certificates across the entire VCF fleet from a central interface. VMware vCenter manages machine SSL certificates and VMCA-issued certificates for its connected ESX hosts. ESX hosts support multiple



certificate modes; organizations using custom third-party certificates should configure the ESX certificate mode to Custom Certificate Authority to prevent VMCA from automatically replacing those certificates during renewal operations. NSX maintains its own PKI capabilities: administrators can generate root CA certificates and private keys based on RSA, and these CA certificates can sign other certificates within the NSX domain. The NSX management plane truststore APIs manage non-service certificates. NSX also generates Certificate Expiry alarms with details on certificate ID, severity, node, first and last report time, and recommended action. The platform recommends replacing default self-signed certificates with CA-signed certificates to strengthen the security posture.

VCF Operations orchestrator provides its own PKI layer to sign and encrypt content imported and exported between servers, adding a layer of integrity and confidentiality verification to inter-server communication. Cluster CA certificates use the RSA public key algorithm and include subject distinguished names with organizational metadata. VCF Operations supports installation of custom PEM certificates, and the orchestrator can import required TLS certificates from a URL or a PEM-encoded file to establish secure connections with external systems.

VCF Automation supports uploading public key and private key PEM files for OIDC proxy key management, and SSL certificates are public key certificates that verify OIDC identity. For key management server integration, vCenter can establish trust with a Key Management Server (KMS) by uploading a KMS certificate in X509 PEM format. For stronger security, administrators should upload a leaf certificate or an intermediate CA certificate controlled by the KMS vendor rather than the root CA certificate. The vSphere Automation SDKs recommend using an external utility such as keytool to create a certificate truststore with password protection for additional security.

The Consumption layer of VCF extends PKI capabilities into the Kubernetes environment. VMware Kubernetes Service (VKS) clusters incorporate a certificate-based PKI anchored to VMCA: TLS certificates for the Supervisor control plane are derived from the VMCA certificate, integrating Kubernetes PKI into the broader VCF chain of trust. VKS Service Cluster control plane components (kube-apiserver, kubelet, etcd, scheduler, and front-proxy) each use TLS certificates with certificate authorities carrying a nine-year validity and leaf certificates carrying a validity of approximately one year. The etcd data store uses x509 PKI to establish secure communication channels, including peer-to-peer mutual TLS authentication using dedicated peer key pairs stored at `/etc/kubernetes/pki/etcd/`.

VKS clusters support automatic certificate rotation for control plane nodes through the `certificateRotation ClusterClass` variable. The `renewalDaysBeforeExpiry` field defaults to 90 days before expiry and is configurable via the ClusterClass API at cluster provisioning or update time, triggering a rolling deployment of renewed certificates before they expire. For VKS Service Clusters, administrators can also perform manual rotation using `kubeadm certs renew` commands; manual rotation must be initiated while certificates remain valid. Administrators can verify the current expiration status of control plane certificates using the `kubeadm certs check-expiration` command, which reports expiration dates and residual time for certificates including `admin.conf`, `apiserver`, `apiserver-etcd-client`, `apiserver-kubelet-client`, and `controller-manager.conf`, supporting proactive monitoring prior to scheduled rotations. Kubelet TLS bootstrapping supports automatic renewal requests as certificates approach expiry, with approval configurable as automatic or manual depending on organizational policy.

For workload certificates, `cert-manager` is available as a standard package for VKS clusters. It creates TLS certificates for Kubernetes workloads and renews them automatically before expiration, with the renewal window governed by the `certificates.renewBefore` field in each certificate resource. VKS clusters can be provisioned to trust additional CA certificates for SSL/TLS communication with private container registries and other enterprise systems. The Kubernetes `CertificateSigningRequest` API provides a managed path for certificate requests to an external signing authority, with certificate lifetimes either fixed by the signer or configurable by the administrator. The VKS `AuthenticationConfiguration Secret` supports JWT authentication with an optional certificate authority field for internal PKI integration.

### VMware vDefend (Contributes)

VMware vDefend implements an internal Public Key Infrastructure within the Security Services Platform (SSP) to secure all communications between vDefend components. The SSP includes a certificate manager that provisions platform certificates internally, covering both incoming (ingress) and outgoing (egress) traffic. The SSP generates an Intermediate Certificate



Authority that establishes trust relationships with external nodes such as bare metal servers and NDR sensors, then provisions and updates certificates on those nodes automatically. Certificate management within SSP is handled through CertWatcher, which loads and validates root CA certificates and keys during initialization. SSP enforces strong cryptographic standards and rejects any certificate or configuration that uses deprecated algorithms, insufficient key lengths, or older protocol versions.

The SSP automatically initiates a certificate refresh 60 days before expiration for all internal entities and Intermediate CAs and raises an alert 67 days before the Platform CA certificate expires. Certificate refresh can also be triggered manually through the System > Certificates interface. The License Hub component within SSP manages its own Platform CA certificate lifecycle with the same automated refresh and expiration alerting thresholds. The System > Certificates dashboard provides a certificate inventory overview including expired certificates and certificates expiring within 30 days.

The SSP provides certificate lifecycle management capabilities including certificate export, CSR generation, and CA-signed certificate replacement. CSR generation for ingress certificates supports RSA with key sizes of 2048 or 3072 bits. Administrators can replace certificates with locally stored certificates signed by their organization's CA, supplying a private key and optional passphrase. The platform supports importing ingress certificates signed by a custom Certificate Authority, allowing organizations to bring vDefend's externally facing certificates under their enterprise PKI hierarchy. When importing CA-signed ingress certificates, the full certificate chain must be provided in order: the Ingress Certificate, any Intermediate Certificate(s), and the Root CA Certificate.

For outbound communication, the SSP supports importing web proxy certificates and uses CA bundles to build trust chains for communicating with external services. CA bundles have a separate lifecycle from platform certificates and are not automatically refreshed by SSP; their validity status is visible through the CA Bundle tab. Certificates within SSP identify client and host machines, establish trust, and secure communication channels.

The TLS Inspection subsystem maintains its own certificate management capabilities for inspecting encrypted traffic flows. TLS Inspection supports importing or generating certificates (including self-signed and CA-signed CSR generation), importing or updating trusted CA bundles and Certificate Revocation Lists (CRLs) for server certificate validation, and exporting existing certificates. CA certificate bundles are validated as well-formed PEM-encoded concatenations upon upload and are rejected if malformed.

VMware vDefend Malware Prevention uses separate public-private key pairs for service deployment across compute clusters, with each cluster able to use its own key pair for isolation. A public key is required when deploying the Malware Prevention Service on an ESX host cluster, and a private key is required to establish an SSH session to the service VM. SSH public keys must adhere to OpenSSH format.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 1.9

> Strength of function: Ensure that authentication mechanisms are robust against brute force attacks. Strength of function enhancement: Specific metrics for resistance to automated login attempts.

**SCF Controls:** IAC-10

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides a layered authentication infrastructure that supports secure management of authenticators across the platform's components and the Kubernetes workload consumption layer, though the broader organizational policies governing authenticator lifecycle management require processes and governance beyond what VCF itself provides.

VCF Identity Broker integrates with a selected identity provider for user authentication, establishing trust through metadata exchange between the service provider and the identity provider. This centralized approach allows organizations to manage authenticators through their existing identity management systems rather than maintaining separate credential stores for each VCF component. VCF SSO provides a single sign-on environment where a user provides credentials once, and components in the environment perform operations based on the original authentication. Administrators can federate VMware vCenter with an enterprise identity provider, which enables multifactor authentication (MFA), automatic user provisioning and deprovisioning, and token-based authentication that reduces the risk of credential exposure. VCF SSO supports SCIM (System for Cross-domain Identity Management) for automating user identity exchange across identity domains, and named providers including Okta are supported through the identity provider federation mechanism. When configured for identity provider federation, the external provider handles all authentication mechanisms, including passwords, MFA, and biometrics, delegating full authenticator management to the organization's existing identity infrastructure. The Authorization Code grant type in VMware Identity Broker supports MFA workflows, while the Password OAuth 2.0 grant type is incompatible with MFA requirements, so organizations should use the Authorization Code flow when MFA is needed.

VMware Kubernetes Service (VKS) extends this identity architecture to the workload consumption layer. VKS clusters support two authentication methods: VCF SSO and any OIDC-compliant external identity provider, including Okta, so identity lifecycle management, including onboarding, offboarding, and group membership changes, can remain in the organization's central identity system. Users authenticated through an external identity provider are granted Kubernetes RBAC permissions using the `sso:<username>@<domain>` naming convention in ClusterRoleBindings. Custom ClusterClass configurations allow organizations to authenticate developers against existing corporate identity systems. When provisioning clusters based on a custom ClusterClass, the authentication Secret can be configured with multiple JWT issuers to support distinct identity providers simultaneously, such as Keycloak for contractors and Microsoft Entra ID for employees. At the kube-apiserver level, VKS supports multiple authenticator types: client certificates, bearer tokens, an authenticating proxy, and HTTP basic auth. The `--authentication-config` file provides structured control, allowing multiple JWT authenticators each with a unique issuer URL and discovery URL, and CEL expressions to map claims to user attributes and validate claims. When multiple authenticator modules are active, the first to successfully authenticate a request short-circuits further evaluation, making authenticator order significant; Kubernetes hardening guidance recommends enabling as few authentication mechanisms as possible to simplify user management and reduce the risk of users retaining access after it is no longer needed. The kube-apiserver can also leverage existing authentication methods such as LDAP or Kerberos through plugins, and kubectrl integrates with credential plugins for centralized operator credential management. The `system:auth-delegator` ClusterRole provides unified authentication and authorization flows for add-on API servers deployed into VKS clusters. Production VKS clusters should enable Kubelet authentication and authorization to restrict direct access to the kubelet API. For Windows node pools, VKS clusters support Group Managed Service Accounts (gMSA) for authenticating Windows nodes to Active Directory; gMSAs are managed by security groups that control password retrieval and delegation permissions, allowing Windows workloads to use Active Directory-managed identities.

Standard packages deployable on VKS clusters extend authenticator management to workload-level services. Argo CD, available as a standard package, supports local account authentication with global RBAC policies and OIDC authentication for federated identity, allowing GitOps workflow access to be governed through the same identity infrastructure used for cluster

access. The Harbor container registry package provides authentication and token management for artifact access, with role-based access control restricting image push and pull operations.

VCF Operations integrates with VCF SSO for centralized authentication through an identity provider. NSX Manager can be configured to authenticate users from identity management providers or through a directory service such as Active Directory over LDAP or OpenLDAP. NSX authentication best practices include integrating with identity providers, enabling multi-factor authentication, and defining access controls based on user roles and permissions. NSX Manager requires authentication prior to establishing a management connection for administrative access. The VCF Automation Provider Management Portal supports authentication via local users and SAML users, and Organization Administrators use credentials configured during organization creation to access their respective organizations.

For local accounts, each VCF component enforces password complexity and lifecycle policies. NSX Manager requires passwords of at least 12 characters with uppercase, lowercase, numeric, and special character requirements, enforced through Linux PAM modules. Each NSX user password has an expiration date tracked by NSX, with a default expiration period of 90 days. Password expiration is configurable between 1 and 9,999 days. VCF Operations enforces a minimum of 15 characters with similar complexity rules. VCF Automation passwords must also be at least 15 characters in length.

VCF provides configurable account lockout policies across its components. VCF Operations locks accounts after five consecutive failed login attempts for 15 minutes. A separate policy locks VCF Operations accounts if a user enters an incorrect password three times within five minutes. VCF Automation supports account lockout in password policy system settings, which locks user accounts after a configurable number of invalid login attempts. VCF Automation also provides a configurable account lockout interval that specifies the duration before an account is automatically unlocked, and provider administrators can unlock a user account manually without waiting for that interval to expire.

For service-to-service authentication, VCF uses certificate-based credentials. VCF SSO supports X.509 certificate authentication, and vCenter extensions can authenticate using `loginExtensionByCertificate()` with an extension key certificate. Provider management accounts authenticate using API access tokens that rotate after each use. VCF Automation provides service accounts with the same access restrictions as user API tokens, so service accounts cannot perform user management or create refresh tokens. NSX supports Principal Identity certificates for cloud management platform integration, requiring SHA256-based certificates with RSA/DSA message algorithm and 2048-bit or larger key sizes. At the Kubernetes layer, service account authentication uses bearer token secrets generated automatically for service accounts. Kubernetes hardening guidance recommends setting short lifetimes on certificates and automating their rotation, and recommends using an authentication provider that controls how long issued tokens remain valid; using short-lived Secrets helps detect or limit the impact of credential exposure. The `system:masters` group functions as a break-glass super-user group that bypasses the RBAC authorization layer and should be reserved for emergency access scenarios only.

VCF Operations provides API endpoints for trusted certificate management and displays certificate expiration status for each workload domain through its Certificates tab. NSX supports non-disruptive certificate rotation in VCF and later, allowing certificate replacement without service interruption.

VCF also provides visibility into authenticator usage through audit logging. User authentication details are viewable in VCF Operations, including user name, successful or failed authentications, and events such as user login and guest operation authentication. This logging supports detection of compromised or misused authenticators.

While VCF provides the technical controls for authenticator strength, complexity, expiration, lockout, and rotation within its own components, as well as identity provider integration for MFA enforcement and centralized authentication, and the VKS consumption layer provides configurable multi-method kube-apiserver authentication, external identity provider integration, short-lived credential support, and gMSA for Windows workloads, the broader organizational policies for initial authenticator issuance, periodic credential rotation schedules, revocation upon personnel changes, and mapping authentication strength requirements to data classification levels require processes and governance outside VCF's scope.

VMware Private AI Services (PAIS) provides authentication mechanisms specific to its AI infrastructure components, integrating with externally managed identity systems to control access to AI services, model endpoints, and training data.

PAIS uses OIDC-based authentication as its primary access control mechanism. The PAISConfiguration custom resource configures the OIDC provider with issuerUrl, clientId, and scopes including openid, groups, and offline\_access. Access is restricted to user accounts whose membership matches OIDC groups listed in the authorizedGroups field, and the Access Token carries a groups claim that PAIS validates against the authorized group list. The OIDC configuration also supports configurable group claim names and Extra Audiences for additional OAuth2.0 clients that PAIS will accept when validating the Access Token.

The PAISConfiguration custom resource supports client TLS for outbound authentication to the OIDC provider and Harbor registry, configured through the worker.clientTls.caBundleRefs field. This binds certificate-based trust to identity provider and registry connections so that authentication exchanges are integrity-checked against approved CAs.

Authenticator management for the Harbor registry that stores AI models and artifacts follows a credential-based pattern. The oras login command authenticates to a private Harbor registry using username and password credentials for artifact push operations, and docker login is used for pushing models to Harbor projects through the Model Gallery. Harbor project access capabilities restrict access to the sensitive data that is used to train and tune models. Model endpoints support authentication through harbor-pull-secret-readonly to control access to container images stored in private registries, and Harbor model retrieval requires certificate-based trust in addition to credential authentication.

Internal service-to-service authentication is addressed at the component level. PAIS authenticates to the OpenTelemetry Collector using a Kubernetes Secret containing Authorization Bearer tokens and custom headers, and the LLM Traces flow uses the same pattern with semi-colon separated headers. The PAIS MCP server supports custom HTTP headers including User-Agent and Authorization headers for API authentication, and remote MCP servers can optionally use static authentication tokens. PAIS integrates with Grafana for metrics visualization using OIDC provider authentication.

For Knowledge Base data sources, PAIS supports authentication to Confluence using either a username and password or a personal access token, configured according to the Confluence administrator's settings. Personal access tokens are generated from user account settings and used for programmatic access to Knowledge Bases and data sources. Microsoft SharePoint data sources require username and password authentication, with support limited to SharePoint On-Premises deployments.

Deep Learning VMs deployed through self-service catalog items in VCF Automation, when configured with the PyTorch NGC software bundle, support optional JupyterLab authentication activation for interactive notebook access.

The underlying user accounts, credential storage, password policies, and OIDC provider administration are owned by VCF SSO or an external identity provider. PAIS configures and enforces authentication using those systems but does not manage authenticator lifecycle itself.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides several mechanisms for managing authenticators across both the administrative plane and the database layer, supporting the secure credential management aspect of this control.

For administrative access to the DSM platform, the system supports two authentication paths: local user accounts authenticated by email ID and password, and LDAPS-integrated accounts authenticated by matching the email domain against a registered LDAPS server. When directory service credentials are non-functional, DSM falls back to local user authentication, maintaining access continuity. Administrators configure and manage these authentication sources through the DSM Admin UI, which also provides a Permissions view for creating, monitoring, and managing configured users. LDAPS integration supports certificate-based validation of the directory service connection, with administrators able to upload directory service server certificates through the Admin UI to establish trusted connections. LDAP groups can be assigned roles within DSM, enabling group-based access control aligned with directory service group membership.

At the database layer, DSM supports directory service authentication for PostgreSQL, MySQL, SQL Server, and MinIO databases. For PostgreSQL, host-based authentication (HBA) rules determine how clients are authenticated, using ordered matching rules that evaluate connection type, client address, requested database, and user name. The default HBA configuration includes a final catch-all entry using scram-sha-256, with optional LDAP entries for directory-integrated authentication. This HBA configuration can be modified through the DSM interface. For MySQL databases, LDAPS

authentication is available using Active Directory Global Catalog, with support for multi-domain controller deployments via the AD Secure/SSL Global Catalog on port 3269. Directory service authentication for individual database instances can be enabled or disabled through the Enable Directory Service Authentication toggle in the database settings.

For SQL Server databases, DSM uses contained database users, where users authenticate directly against the database and identity information resides within the database. DSM supports Windows Authentication for SQL Server server-level logins through Active Directory integration, enabling Kerberos-based single sign-on for clients joined to the same domain. SQL Server Authentication logins can also be created on all DSM clusters. Administrators configure which user types are permitted per data service policy, selecting between Windows Principal authentication using Active Directory credentials and SQL User authentication using standard SQL Server logins. An internal management login (`dsm_mgmt`) restricts access to system-level objects, keeping administrative operations within an isolated context.

DSM enforces password policies for local accounts, including complexity requirements, and applies a root password expiry on a three-month rotation cycle. For service-to-service communication, DSM uses authentication secrets (`dsm-auth-creds`) containing endpoint credentials and root CA certificates, with automatic secret generation available through the Consumption Operator installation. Prometheus metrics forwarding supports configurable authentication options, including no authentication, certificate-only authentication, basic authentication with username and password, or a combination of certificate and basic authentication.

Regarding authentication strength appropriate to data classification, DSM provides the authentication mechanisms that administrators can configure to match organizational security policies, but it does not automatically enforce differentiated authentication strength based on data classification. Organizations must define their data classification scheme and determine which DSM authentication options (local credentials, LDAPS, HBA rules, certificate validation, Kerberos) apply to each tier of sensitivity. DSM contributes the technical controls; the alignment between authentication strength and data classification requires organizational governance to establish and maintain.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer provides mechanisms to securely manage authenticators across two distinct planes: the Controller management plane (admin access) and the application data plane (end-user client authentication for load-balanced applications).

For the management plane, Avi maintains user accounts in two categories: local users managed directly on the Controller, and remote users authenticated through an external authentication, authorization, and accounting (AAA) server. Avi uses Authentication Profiles to configure how administrators authenticate to the Controller. Authentication Profiles support LDAP for credential validation against a directory service, SAML for federated authentication that delegates the process to a centralized identity provider (IdP), and basic authentication. The recommended approach is federated authentication, which centralizes the authentication process by delegating to an external IdP rather than managing credentials locally on the Controller. SAML profiles verify user identity without storing credentials on untrusted systems, and Avi Controller SAML authentication profiles can be integrated with Workspace ONE Access for catalog-based application access. Admin users can also authenticate using password-less SSH-based login as an alternative to password-based authentication. On every user login, the Auth Mapping Profile evaluates authorization mapping rules and updates the user record with tenant and role pairs drawn from the IdP's assertions.

For the application data plane, Avi acts as a proxy that applies client authentication policies to inbound application traffic. Supported methods include LDAP authentication (checking client credentials against a directory service), SAML single sign-on (enabling federated application access with a single credential set), OAuth and OIDC (where Avi acts as a client and resource server, with the authorization server authenticating clients and validating authorization codes before issuing tokens), and JSON Web Token (JWT) validation. The SSO Policy supports configurable Authentication Rules and Authorization Rules with path-based matching criteria, allowing different authentication requirements to be applied to different application paths. More granular authorization in OAuth/OIDC flows is available through Authorization Policies or DataScript.

Organizations configure which authentication method applies to each virtual service or application based on the sensitivity of the data accessed. Avi's Authentication Profiles and SSO Policies provide the technical enforcement points to apply different

authentication strengths across the application portfolio, but the mapping from data classification tier to required authentication method is an organizational governance decision that must be defined and maintained by the organization.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.



## Control CR 1.9(1)

> Strength of function: Ensure that authentication mechanisms are robust against brute force attacks. Strength of function enhancement: Specific metrics for resistance to automated login attempts.

**SCF Controls:** IAC-12.1

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides several integration points for Hardware Security Modules (HSMs) and hardware-rooted trust mechanisms that support the protection of authenticators and cryptographic material, though VCF itself does not include a built-in HSM. The platform is designed to work with external HSM devices and hardware security components to strengthen key management, entropy sourcing, and boot integrity.

VCF integrates with external Key Management Servers (KMS) through the KMIP 1.1 protocol, providing the mechanism through which organizations can connect HSM-backed key management infrastructure to the platform's encryption services. The external KMS is the component that would be backed by an HSM to protect the root encryption keys on which VCF relies for vSAN data-at-rest encryption, VM Encryption, and other cryptographic operations. The KMS must support KMIP 1.1 and be capable of acting as a symmetric key foundry and server. VCF supports Standard Key Providers that connect to external KMIP-compliant KMS systems, where the KMS can be backed by HSMs for hardware-protected key storage. VMware vCenter establishes a trusted connection with the KMS by uploading the KMS server certificate and private key, and provisions encryption keys from the key provider. Each VMware ESX host uses the Key Encryption Key (KEK) to encrypt its Data Encryption Keys (DEKs) and stores the encrypted DEKs on disk, but does not store the KEK on disk. When a host reboots, it requests the KEK with the corresponding ID from the KMS and decrypts its DEKs as needed. The vSAN KMS configuration also supports an optional proxy address for environments that require it. When a user performs an encryption task such as creating an encrypted virtual machine, vCenter requests a new key from the default KMS to use as the KEK.

ESX supports the use of external HSM devices that are FIPS 140-3 and EAL4 certified as sources of external entropy data. Starting with vSphere 8.0 Update 1, administrators can add external entropy sources to an ESX host to feed high-quality, hardware-generated randomness into the host's cryptographic subsystem. This capability directly strengthens the generation of keys and other authenticators used across the platform by supplementing the host's own entropy sources.

ESX host hardening guidance includes enabling the Trusted Platform Module (TPM), a hardware-based security component embedded in the physical server. TPM provides a hardware root of trust for secure boot attestation and can store cryptographic keys and measurements in tamper-resistant hardware. When TPM is enabled and configured as part of the ESX hardening baseline, it provides hardware-backed protection for host-level authenticators and boot integrity verification. The ESX hardening guidance covers configuring authentication and authorization settings, enabling Secure Boot, disabling unnecessary services, enabling TPM, and implementing network security measures as part of a layered security posture. TPM modules must run the latest 2.0 firmware when in use, or be disabled in the BIOS if not in use. The physical TPM on ESX hosts also supports sealing policies based on Platform Configuration Register (PCR) measurements, which restrict access to sensitive data stored in the TPM to configurations that match a known-good platform state. Beyond host-level TPM, vSphere supports Virtual Trusted Platform Modules (vTPM) for virtual machines, allowing guest operating systems to use hardware-rooted cryptographic security without requiring a dedicated physical TPM per VM; vTPM keys are not exposed to the guest operating system itself, which reduces the attack surface at the VM level.

VCF also supports hardware memory encryption through confidential vSphere Pods. Confidential vSphere Pods use hardware technology such as AMD SEV-ES to keep guest OS memory encrypted, protecting it against access from the hypervisor. This capability addresses workload confidentiality rather than authenticator storage, but is part of the platform's hardware-rooted security mechanisms.

Organizations deploying VCF in environments that require HSM-protected authenticators should procure and integrate external HSM appliances, configure ESX hosts to use them for entropy and key management, enable TPM on all hosts, and establish trusted connections between vCenter and the KMS infrastructure. The platform provides the integration interfaces,



protocol support, and configuration options, but the HSM hardware itself, its lifecycle management, and the policies governing its use are organizational responsibilities.

#### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer supports Hardware Security Module (HSM) integration as a native capability for protecting the cryptographic keys that Avi's TLS termination and certificate operations depend on. The Avi Controller integrates with Thales Luna (formerly SafeNet Luna) HSM appliances, enabling hardware-backed storage and management of SSL certificates and private keys rather than software-based key storage.

The primary configuration mechanism is the HSM Group, configured in the Service Engine Group settings under the Advanced Settings tab in the Avi Controller UI or through the API. An HSM Group defines how Avi Controller and Service Engine nodes reach and authenticate with the external HSM device, including registering the IP addresses of all Controllers and Service Engines authorized to interact with it. The HSM Group is assigned per Service Engine Group and acts as a template of HSM settings applied uniformly to every Service Engine in that group. Mutual certificate authentication is established between the Avi Controller and the HSM during initial registration, so that only authorized nodes can request cryptographic operations from the device.

Avi Service Engines support dedicated network interfaces for HSM communication, separate from their standard management and data-plane interfaces. This interface isolation keeps HSM traffic off the general management network. Avi Controller cluster nodes similarly use dedicated network interfaces labeled HSM for hardware security module communication, and the HSM interface can be moved to a secondary interface if needed. High availability across multiple HSM devices is supported, allowing organizations to add redundant HSM nodes to an HSM Group to reduce single points of failure in key availability.

Avi Load Balancer's FIPS 140-3 compliance posture also incorporates dedicated HSM interfaces on Service Engines for secure cryptographic operations. Supported HSM clients include Thales Luna HSM client 7.3 and Universal Client v 10.4.1 for key management integration.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 1.9(a)

> Sub-item (a) of CR 1.9 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 1.9 requirement in the standard.

**SCF Controls:** IAC-10.2

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF includes the VMware Certificate Authority (VMCA), a built-in internal certificate authority integral with every vSphere deployment that is the foundation for constructing and verifying certification paths within the VCF infrastructure. Components that obtain certificates from VMCA have an inherent chain of trust back to this root. Administrators can replace the VMCA root certificate when organizational policy requires alignment with an enterprise PKI hierarchy, at which point VMCA operates as a subordinate CA, and all certificates it issues chain back to the enterprise root. Replacing the VMCA root certificate triggers a restart of services that use that certificate, and the VMware vCenter Certificate Management API provides a dedicated endpoint for this operation. Management of both VMCA-issued and third-party certificates is supported across the environment.

The VMware Kubernetes Service (VKS) platform, which provides Kubernetes cluster provisioning within the VCF consumption layer, extends this VMCA-anchored trust chain to cluster infrastructure. TLS certificates for vSphere Supervisor and VKS cluster components, including the Kubernetes API server, kubelet, etcd, scheduler, and front-proxy, derive from the VMCA certificate. A CA-signed certificate can also be uploaded to replace the default certificate on the Supervisor Kubernetes control plane, supporting organizations that require alignment with an external PKI hierarchy at the Supervisor infrastructure level. VKS Service Cluster certificate authorities have a 9-year lifetime; leaf certificates have a 364-day validity period. Trust configuration for VKS clusters supports specifying additional trusted certificate authorities through the `TkgServiceConfiguration additionalTrustedCAs` field and the `TanzuKubernetesCluster` API's corresponding trust configuration, expressed as base64-encoded PEM certificate data, allowing administrators to extend trust to enterprise CA hierarchies within the Kubernetes cluster context. The `Kubernetes ClusterTrustBundle` API provides a structured mechanism for storing and distributing trust anchors to workloads and cluster components, decoupling trust anchor distribution from the certificate-signing pipeline. The `VKS AuthenticationConfiguration Secret` also accepts an optional certificate authority field to support JWT-based authentication flows backed by an internal PKI.

VCF SSO supports certificate-based authentication through holder-of-key tokens, where security artifacts embedded in the token are validated against trusted certificate authorities. The VCF SSO client can obtain a holder-of-key token by sending a token request that uses a certificate, and the SSO SDK includes samples demonstrating how solution components use their private key and certificate to acquire holder-of-key tokens. Solution user accounts, which handle service-to-service communication between vCenter, VCF Operations, and other platform components, authenticate using certificates rather than passwords, with the platform validating the certificate chain for each connection. Requests are signed with a user certificate private key through a security signature handler.

Trust store management across VCF components allows administrators to define accepted trust anchors. VCF Automation allows certificates to be added to the trust store through the UI or the VCF Automation API. The vSphere Automation SDKs support creating certificate stores using `keytool` to import certificates into a truststore with a password, providing a structured approach to managing which root and intermediate certificates are trusted. The `dir-cli trustedcert publish` command provides a CLI path for adding certificate authority certificates, including intermediate CA certificates, to the `VECS TRUSTED_ROOTS` store on vCenter. The vSphere Automation API exposes certificate management interfaces including `TrustedRootChains`, `VMCARoot`, and TLS certificate operations, allowing programmatic management of certificate trust. The NSX management plane and truststore APIs provide additional certificate management for network infrastructure components, including management of non-service certificates.

VCF Automation provides automated certificate validation capabilities, including an API endpoint that validates certificates and returns a resource with the certificate ID upon successful validation. When establishing trust with remote servers, VCF Automation can verify certificate information for SSL communication and retrieve intermediate and leaf certificates from

issuing certificate authorities if the certificate chain is incomplete, allowing administrators to review the full path before making a trust decision and make selective choices about which certificates in a chain to trust. This chain retrieval capability supports the construction of certification paths from an end-entity certificate back to a trusted root. VCF Automation also validates server certificates during LDAPS connections through a Trust Certificate window, applying certificate verification to directory service authentication paths.

VCF Operations for Networks offers configurable certificate validation with two modes: Automatic Acceptance, where the system automatically accepts all detected certificate changes, and Manual Acceptance, where the system presents a notification for an administrator to review and accept certificates individually. Standard certificate validation can be activated in VCF Operations to verify certificates during connection testing, using a standard verification flow that checks the certificate chain against configured trust anchors. When standard certificate validation is activated, the Certificate Revocation List (CRL) tab becomes available, allowing administrators to upload and manage revocation lists for digital certificates that have been revoked by the issuing CA before their scheduled expiration date.

NSX also supports certificate revocation checking. NSX Manager can import certificate revocation lists in PEM format through its API and can enable or disable Certificate Revocation List Distribution Point (CDP) validation checks during certificate import. The NSX API provides endpoints to configure NSX Manager to retrieve CRLs, supporting ongoing revocation validation for network infrastructure certificates.

VCF applies proper certificate validation practices in its own operations. The platform documentation explicitly states that trust-all trust managers, which accept all certificates without validation, are appropriate only for development environments and should not be used in production. Production implementations should use proper certificate support with validated trust paths.

Automated certificate lifecycle management further supports PKI operations. VCF supports automatic renewal of TLS certificates for components using VMCA, Microsoft CA, OpenSSL CA, or self-signed certificates, with renewal triggered 60 days before certificate expiration. The Machine SSL certificate can be renewed for a specified period up to 730 days using the vSphere Automation API, and the auto-renewal toggle in VCF Operations console (Fleet Management, Certificates) allows administrators to activate automatic renewal. By default, the VMCA root certificate has a 10-year lifetime, and all certificates that VMCA signs expire no later than the root certificate; organizations planning to use VMCA long-term should account for root CA rotation before that threshold. VCF Operations provides a Certificates tab for each workload domain that displays certificate expiration status, supporting monitoring of certificate health.

Within VKS clusters, certificate lifecycle management operates through dedicated automation. The ClusterClass certificateRotation parameter governs automated control-plane certificate rotation, and the renewalDaysBeforeExpiry field defaults to 90 days and triggers a rolling rollout of updated certificates before they expire; this value is configurable per ClusterClass to align with organizational PKI policy. Certificate rotation is enabled by default across VKS ClusterClass versions, including builtin-generic-v3.2.0, v3.3.0, and v3.4.0 and later, so automated pre-expiry renewal is active without explicit configuration. Kubelet TLS bootstrapping automates initial certificate distribution and signing for new nodes joining a VKS cluster and can be configured to automatically request renewal when the kubelet certificate approaches expiry, with renewal approved automatically or manually depending on configuration. For application workloads, cert-manager is available as a standard VKS package; it creates TLS certificates for workloads and renews them automatically before expiration. CertificateSigningRequest resources from both the native Kubernetes certificates.k8s.io API group and the cert-manager.io API group are supported for certificate lifecycle management within VKS clusters. Cert-manager is also a prerequisite for several standard VKS addons including Contour, ExternalDNS, Prometheus, and Harbor. Harbor, available as a standard VKS package for container image registry services, supports TLS certificate configuration through cert-manager integration, a custom certificate provided via the tlsCertificate field, or a reference to an existing Kubernetes secret; remote registry certificate verification is configurable through the Verify Remote Cert option in Harbor's registry configuration. VKS clusters can be configured to trust self-signed CA certificates for private registries, and from ClusterClass builtin-generic-v3.2.0 onward, Harbor services with custom certificates are automatically trusted by VKS clusters through deterministic ordering of trusted certificates. The VCF Consumption CLI's imgpkg tool provides --registry-verify-certs and --registry-ca-cert-path flags for validating registry connections against configured trust anchors. Certificate expiration status across VKS control-plane

certificates can be monitored using the `kubeadm certs check-expiration` command. For manual certificate renewal, the `kubeadm certs renew` command can renew all or individual cluster certificates; in replicated control-plane deployments, it must be executed on each control-plane node. Runtime OCSP or CRL revocation checking is not enforced natively by the Kubernetes API server at authentication time; organizations with requirements for runtime revocation validation integrate external PKI systems at the application or service mesh layer.

Together, these mechanisms construct and verify certification paths to accepted trust anchors across VCF components and check certificate status through CRL import and Certificate Revocation List Distribution Point validation in VCF Operations and NSX. The VKS consumption layer extends these PKI foundations through VMCA-anchored cluster component certificates, ClusterTrustBundle-based trust anchor distribution, and automated rotation at both the platform and workload levels, covering the automated certification path validation capabilities the control asks about.

VMware Private AI Services (PAIS) contributes to PKI-based authentication through a CA trust bundle management layer that spans its AI service components. PAIS validates certificates through configurable CA trust bundles stored as ConfigMaps in the VCF Automation namespace, with bundles supplied either as certificate files or ConfigMap resources by the administrator of the Harbor registry or content management system that PAIS connects to. The PAISConfiguration custom resource's `spec.clientTls.caBundleRefs` field references those ConfigMaps for client TLS authentication with OIDC providers and Harbor registries, and PAIS is updated with a new ConfigMap resource when a CA certificate is selected and added from the namespace. Trust bundles are organized per application and rotated through the Service Configuration tab by deleting the bundle containing the old certificate and adding a new bundle with the updated certificate before expiration.

PAIS requires CA trust bundles to be configured for the external entities it connects to: OIDC providers for identity federation, Harbor registries for model artifact retrieval, content management systems serving as Knowledge Base data sources (such as Confluence sites and Microsoft SharePoint On-Premises deployments), and remote MCP servers integrated for specialized data and capabilities. On Deep Learning VMs, Harbor registry CA certificates are copied to the `/usr/local/share/ca-certificates` directory and added to the system trust store using the `update-ca-certificates` command.

Certificate validation is enforced through several PAIS authentication paths. The VCF Consumption CLI accepts the `--ca-certificate` option to validate the VCF Automation endpoint certificate when it is not already in the local truststore, covering namespace login, VKS cluster provisioning with vGPU or GPU passthrough in both connected and disconnected environments, and deployment of completion or embedding model endpoints by using `kubectrl`. The VCF Automation instance certificate must be saved locally on the machine deploying model endpoints or provisioning VKS clusters. The VCF Automation Provider Management UI requires trusted certificate import via Certificate Management > Trusted Certificates and certificate acceptance during the namespace import process for PAIS integration with VMware vCenter, and records a `TM_CERT_BASE64` credential captured by running `openssl s_client` against the VCF Automation FQDN on port 443. PAIS OIDC authentication supports configuring Extra Audiences so the service accepts additional OAuth 2.0 clients during access token validation, and Grafana OIDC authentication for PAIS supports PKCE and refresh tokens. The PAIS ingress self-signed certificate is retrievable from the `pais-ingress-default` Kubernetes secret for downstream TLS configuration.

The actual certification path construction and certificate status checking at the network layer is performed by the underlying VKS and Kubernetes platform rather than by PAIS directly. The PAIS contribution is the CA trust bundle configuration layer, managed through ConfigMaps, the PAISConfiguration custom resource, and the VCF Automation Provider Management UI, that populates the trust stores consumed by those platform-level PKI validation mechanisms across its AI service components.

### **VMware vDefend (Contributes)**

VMware vDefend uses PKI-based certificate validation within the Security Services Platform (SSP) to authenticate connections between platform components and to secure communications with external identity providers. The platform maintains a certificate hierarchy that supports automated certification path construction and verification against trusted anchors.

The Security Services Platform maintains its own certificate infrastructure, including Web Proxy, Platform Certificates, and Intermediate CA certificates. A CertWatcher component loads and validates root CA certificates and keys during initialization, maintaining a CA certificate pool and truststore for validating cluster API certificates. The platform monitors certificate status, identifying certificates that are expired, expiring soon, or valid, and generates alarms when certificates require attention.

Administrators can view certificate details, replace certificates, generate certificate signing requests, and import certificates signed by a certificate authority of their choice. The Security Services Platform proactively manages its Platform CA certificate lifecycle, raising an alert 67 days before expiration and automatically initiating a certificate refresh 60 days before expiration for all internal entities and Intermediate CAs. A Certificates UI page provides visibility into Certificate Refresh Status and External Rollout details, including Bare Metal server certificate validity status.

For network detection components, NDR Sensors connect to the Security Services Platform using Mutual TLS (mTLS) authentication. PKIX certificate path validation is used to authenticate these connections, and authentication failures are logged with specific error messages indicating PKIX path validation or access authorization issues. This validation occurs automatically during sensor registration and ongoing communication.

The Security Services Platform validates certificates when establishing secure connections to external identity providers. When configured for LDAPS authentication, the platform requires an LDAP server certificate to authenticate directory service users, establishing a verified connection. Certificate import capabilities allow administrators to identify web proxy or ingress certificates signed by a certificate authority of their choice, extending the trust model to external systems.

For deployments using TLS Inspection, vDefend provides additional certificate path validation capabilities at the network layer. TLS Inspection certificate management supports importing or updating trusted CA bundles and a default CRL for server certificate validation during traffic inspection. In 9.1, TLS Inspection supports OCSP stapling options for certificate validation, providing a mechanism for checking certificate revocation status in addition to CRL-based checking. When inspecting encrypted traffic to external services, server certificate validation can be enforced against the configured trusted CA bundle and revocation information. The TLS proxy supports Enterprise issuing CA certificates, where a trusted subordinate CA certificate is signed by an Enterprise issuing CA through a CSR workflow, allowing integration with an organization's existing enterprise PKI infrastructure.

vDefend contributes to this control through its internal certificate path validation, automated certificate chain management for SSP component authentication, PKIX-based authentication for NDR Sensor connections via mTLS, CRL-based revocation checking, and OCSP stapling support for TLS Inspection server certificate validation. Full enterprise PKI integration for administrator authentication relies on the organization's LDAP-based identity infrastructure rather than vDefend alone, and comprehensive certificate path validation for all external connections depends on the broader VCF certificate infrastructure.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) uses PKI-based authentication for site pairing, appliance management, and management-plane communication. PNR uses TLS certificates and private keys to protect network communication and establish authentication with other servers, and treats certificates and keys as part of its core security infrastructure.

The PNR appliance supports SSL/TLS server endpoint certificates that can be either self-signed or custom certificates signed by a certificate authority. The PNR self-signed certificate expires five years from the first boot of the appliance. Custom certificates may be intermediate CA certificates that are not the root of a trust chain, supporting full certification path construction to an accepted trust anchor. Custom SSL/TLS certificates must meet specific requirements to work with PNR, and the current time must be within the certificate's validity period for authentication to succeed.

The PNR Configuration REST API provides endpoints to add certificate authority (CA) certificates and to retrieve installed CA certificates used to validate other servers' certificates. PNR also includes Authentication APIs as part of its REST API offerings. The PNR user interface validates host certificates against subject alternative names during connection to Platform Services Controller hosts. The PNR server establishes authenticated communication channels to remote services by acquiring a holder-of-key SAML token from VCF Single Sign-On (VCF SSO), then sends this token in a cryptographically signed message.

PNR includes certificate status monitoring. PNR raises a certificate expiring event when the certificate expiration date is within the configured number of days, with a default value of 30 days controlled by the `localSiteStatus.minCertRemainingTime` setting. PNR generates `SrmCertificateExpiringEvent` when the SSL/TLS certificate for a specified Site Recovery Manager (SRM) Server is approaching expiration, and `SrmCertificateExpiredEvent` when the certificate has expired. Certificate verification errors are recorded in security-related log files for audit-trail purposes.

PNR supports certificate-based authentication with storage providers through either direct certificate specification or thumbprint verification. During site registration and remote site management, PNR can accept or reject remote VMware vCenter certificates and prompts the administrator before adding a certificate to the trusted store. The Configure Local Sites workflow and the VCF Operations orchestrator plug-in include an option to ignore certificate warnings, which silently adds the certificate to the trusted store; administrators should use this option carefully because it bypasses interactive verification.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) includes several certificate management capabilities that contribute to PKI-based authentication validation. DSM maintains a Trusted Root Certificates store where administrators can add PEM-encoded Root and Intermediate CA certificates for directory service connections, database cluster connections, and connections to external systems such as backup repositories. The store displays the expiration time and date for each stored certificate, giving administrators visibility into certificate lifecycle status. Custom trusted certificates added to DSM must comply with FIPS requirements. Certificate validation behavior is configurable through the advanced-system-config ConfigMap, which supports two validation modes; the "latest" mode is the default for version 9.1 and later.

When clients connect to a DSM database cluster, DSM stores PEM-encoded certificate authorities in the `ca.crt` property of a Kubernetes Secret. Clients must add this CA certificate to their trusted CA list or operating system trust store before a connection is established. Connections from DSM to the VCF Automation Provider also require certificate validation and trust establishment before the connection becomes active. The Consumption Operator requires a TLS certificate for secure communication between the Consumption Operator and the DSM provider. DSM validates certificate status and requires explicit trust before the connection proceeds.

DSM supports custom certificates for both the Provider VM and for individual database clusters. Administrators configure custom certificates in PEM format, including the full certificate chain with its associated private key and CA certificate. The DSM API provides programmatic capability to configure custom certificates for database clusters, supporting integration with certificate management workflows. Administrators with restrictive certificate policies should replace the default DSM certificate with a CA-issued custom certificate. Custom certificate configuration for the Provider VM can only be performed on a fresh installation where no data services are deployed. If the Provider VM certificate expires, access to the DSM UI, database cluster upgrades or restarts, and cluster metrics may all be affected.

Certificate-based authentication is supported for PostgreSQL databases through host-based authentication (HBA) configuration, which accepts `cert` as an authentication method alongside `SCRAM-SHA-256` and `reject` options. DSM supports mutual TLS (mTLS) authentication for metrics forwarding by enabling the Client Certificate Authentication toggle and providing the client certificate and private key. For SQL Server databases, DSM supports Custom CA as a Certificate Trust Source option for Active Directory domain connections when the system trust store cannot verify the server certificate.

For backup repository connections, DSM supports Trust On First Use (TOFU) configuration to extract and trust certificates for specific backup locations. DSM also introduced updated cryptographic algorithm requirements for certificates; administrators should verify that certificates comply with these requirements before or after upgrading, as noncompliant certificates can cause validation errors, service startup failures, or TLS handshake issues.

Satisfying the full control intent requires the organization to implement automated certificate renewal and revocation-checking processes. Certificate renewal for the Provider VM and database cluster certificates is a manual administrator responsibility; the documentation explicitly places this obligation on the administrator rather than on DSM itself. DSM does not provide built-in automated mechanisms for OCSP or CRL-based certificate status checking; these must be provided by the organization's PKI infrastructure or by PKI automation tooling configured with an appropriate issuer.

### **VMware Avi Load Balancer (Meets)**

VMware Avi Load Balancer provides native PKI-based certificate validation through the PKI Profile, a reusable configuration object that implements the full certification path construction and status-checking pipeline required by this control.

The PKI Profile is configured under Security > Client SSL Certificate Validation > PKI Profile in the Avi Controller UI or API. It stores one or more certificate authority (CA) certificates that serve as trust anchors. When a client presents a certificate for



authentication, Avi constructs the certification path from the presented certificate through any intermediate CAs to the root trust anchor stored in the PKI Profile. Each link in the intermediate chain must be present in the PKI Profile for the validation to succeed, enforcing complete chain construction to a known anchor.

Certificate status checking is handled through Certificate Revocation List (CRL) support built into the PKI Profile. The profile stores CRLs from issuing certificate authorities, including CRLs covering both end-entity and intermediate certificates. The Enable CRL Check option in the PKI Profile controls full-chain CRL checking for client certificate validation. When enabled, Avi checks the revocation status of the presented certificate and each intermediate against the configured CRLs before allowing the connection to proceed. A separate Ignore Peer Chain setting is available to control whether intermediate certificate validation is required, providing explicit configuration control over path depth.

The PKI Profile is attached to virtual service configurations through the HTTP Application Profile's Authentication tab, where it is referenced alongside mutual TLS settings to require and validate client certificates. The `avi.ssl.set_pki_profile()` DataScript function allows dynamic assignment of a PKI Profile and client certificate verification mode on a per-connection basis for advanced traffic-handling scenarios. A single PKI Profile object can be shared across multiple HTTP profiles within a tenant, centralizing trust anchor and revocation list management.

In Kubernetes environments, the Avi Kubernetes Operator (AKO) exposes PKI-based authentication through the PKIProfile custom resource definition (CRD). The PKIProfile CRD defines a list of trusted Certificate Authorities (Root and Intermediate) through the required `ca_certs` field and integrates with RouteBackendExtension to enforce certificate validation for backend TLS connections. The System-Standard SSL Profile works in conjunction with the PKIProfile CRD for certificate validation and trust management across Kubernetes ingress configurations. This extends the same PKI validation pipeline to cloud-native deployment patterns without requiring configuration outside Kubernetes-native tooling.

In VCF deployments, the PKI Profile is available as a custom resource through the IaaS Service Console, where it manages the list of trusted authorities permitted to sign client certificates. This supports Zero Trust security patterns between client and server across the VCF-integrated load balancing layer.

For NSX and vCenter cloud integrations, the Avi Controller provides a Truststore PKI Profile setting under System Settings on the Access tab to enable certificate verification and supports CA certificate import for client SSL certificate validation in those cloud contexts.



## Control CR 1.9(b)

> Sub-item (b) of CR 1.9 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 1.9 requirement in the standard.

**SCF Controls:** IAC-10.2

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF includes the VMware Certificate Authority (VMCA), a built-in internal certificate authority integral with every vSphere deployment that is the foundation for constructing and verifying certification paths within the VCF infrastructure. Components that obtain certificates from VMCA have an inherent chain of trust back to this root. Administrators can replace the VMCA root certificate when organizational policy requires alignment with an enterprise PKI hierarchy, at which point VMCA operates as a subordinate CA, and all certificates it issues chain back to the enterprise root. Replacing the VMCA root certificate triggers a restart of services that use that certificate, and the VMware vCenter Certificate Management API provides a dedicated endpoint for this operation. Management of both VMCA-issued and third-party certificates is supported across the environment.

The VMware Kubernetes Service (VKS) platform, which provides Kubernetes cluster provisioning within the VCF consumption layer, extends this VMCA-anchored trust chain to cluster infrastructure. TLS certificates for vSphere Supervisor and VKS cluster components, including the Kubernetes API server, kubelet, etcd, scheduler, and front-proxy, derive from the VMCA certificate. A CA-signed certificate can also be uploaded to replace the default certificate on the Supervisor Kubernetes control plane, supporting organizations that require alignment with an external PKI hierarchy at the Supervisor infrastructure level. VKS Service Cluster certificate authorities have a 9-year lifetime; leaf certificates have a 364-day validity period. Trust configuration for VKS clusters supports specifying additional trusted certificate authorities through the `TkgServiceConfiguration additionalTrustedCAs` field and the `TanzuKubernetesCluster` API's corresponding trust configuration, expressed as base64-encoded PEM certificate data, allowing administrators to extend trust to enterprise CA hierarchies within the Kubernetes cluster context. The `Kubernetes ClusterTrustBundle` API provides a structured mechanism for storing and distributing trust anchors to workloads and cluster components, decoupling trust anchor distribution from the certificate-signing pipeline. The `VKS AuthenticationConfiguration Secret` also accepts an optional certificate authority field to support JWT-based authentication flows backed by an internal PKI.

VCF SSO supports certificate-based authentication through holder-of-key tokens, where security artifacts embedded in the token are validated against trusted certificate authorities. The VCF SSO client can obtain a holder-of-key token by sending a token request that uses a certificate, and the SSO SDK includes samples demonstrating how solution components use their private key and certificate to acquire holder-of-key tokens. Solution user accounts, which handle service-to-service communication between vCenter, VCF Operations, and other platform components, authenticate using certificates rather than passwords, with the platform validating the certificate chain for each connection. Requests are signed with a user certificate private key through a security signature handler.

Trust store management across VCF components allows administrators to define accepted trust anchors. VCF Automation allows certificates to be added to the trust store through the UI or the VCF Automation API. The vSphere Automation SDKs support creating certificate stores using `keytool` to import certificates into a truststore with a password, providing a structured approach to managing which root and intermediate certificates are trusted. The `dir-cli trustedcert publish` command provides a CLI path for adding certificate authority certificates, including intermediate CA certificates, to the `VECS TRUSTED_ROOTS` store on vCenter. The vSphere Automation API exposes certificate management interfaces including `TrustedRootChains`, `VMCARoot`, and TLS certificate operations, allowing programmatic management of certificate trust. The NSX management plane and truststore APIs provide additional certificate management for network infrastructure components, including management of non-service certificates.

VCF Automation provides automated certificate validation capabilities, including an API endpoint that validates certificates and returns a resource with the certificate ID upon successful validation. When establishing trust with remote servers, VCF Automation can verify certificate information for SSL communication and retrieve intermediate and leaf certificates from

issuing certificate authorities if the certificate chain is incomplete, allowing administrators to review the full path before making a trust decision and make selective choices about which certificates in a chain to trust. This chain retrieval capability supports the construction of certification paths from an end-entity certificate back to a trusted root. VCF Automation also validates server certificates during LDAPS connections through a Trust Certificate window, applying certificate verification to directory service authentication paths.

VCF Operations for Networks offers configurable certificate validation with two modes: Automatic Acceptance, where the system automatically accepts all detected certificate changes, and Manual Acceptance, where the system presents a notification for an administrator to review and accept certificates individually. Standard certificate validation can be activated in VCF Operations to verify certificates during connection testing, using a standard verification flow that checks the certificate chain against configured trust anchors. When standard certificate validation is activated, the Certificate Revocation List (CRL) tab becomes available, allowing administrators to upload and manage revocation lists for digital certificates that have been revoked by the issuing CA before their scheduled expiration date.

NSX also supports certificate revocation checking. NSX Manager can import certificate revocation lists in PEM format through its API and can enable or disable Certificate Revocation List Distribution Point (CDP) validation checks during certificate import. The NSX API provides endpoints to configure NSX Manager to retrieve CRLs, supporting ongoing revocation validation for network infrastructure certificates.

VCF applies proper certificate validation practices in its own operations. The platform documentation explicitly states that trust-all trust managers, which accept all certificates without validation, are appropriate only for development environments and should not be used in production. Production implementations should use proper certificate support with validated trust paths.

Automated certificate lifecycle management further supports PKI operations. VCF supports automatic renewal of TLS certificates for components using VMCA, Microsoft CA, OpenSSL CA, or self-signed certificates, with renewal triggered 60 days before certificate expiration. The Machine SSL certificate can be renewed for a specified period up to 730 days using the vSphere Automation API, and the auto-renewal toggle in VCF Operations console (Fleet Management, Certificates) allows administrators to activate automatic renewal. By default, the VMCA root certificate has a 10-year lifetime, and all certificates that VMCA signs expire no later than the root certificate; organizations planning to use VMCA long-term should account for root CA rotation before that threshold. VCF Operations provides a Certificates tab for each workload domain that displays certificate expiration status, supporting monitoring of certificate health.

Within VKS clusters, certificate lifecycle management operates through dedicated automation. The ClusterClass certificateRotation parameter governs automated control-plane certificate rotation, and the renewalDaysBeforeExpiry field defaults to 90 days and triggers a rolling rollout of updated certificates before they expire; this value is configurable per ClusterClass to align with organizational PKI policy. Certificate rotation is enabled by default across VKS ClusterClass versions, including builtin-generic-v3.2.0, v3.3.0, and v3.4.0 and later, so automated pre-expiry renewal is active without explicit configuration. Kubelet TLS bootstrapping automates initial certificate distribution and signing for new nodes joining a VKS cluster and can be configured to automatically request renewal when the kubelet certificate approaches expiry, with renewal approved automatically or manually depending on configuration. For application workloads, cert-manager is available as a standard VKS package; it creates TLS certificates for workloads and renews them automatically before expiration. CertificateSigningRequest resources from both the native Kubernetes certificates.k8s.io API group and the cert-manager.io API group are supported for certificate lifecycle management within VKS clusters. Cert-manager is also a prerequisite for several standard VKS addons including Contour, ExternalDNS, Prometheus, and Harbor. Harbor, available as a standard VKS package for container image registry services, supports TLS certificate configuration through cert-manager integration, a custom certificate provided via the tlsCertificate field, or a reference to an existing Kubernetes secret; remote registry certificate verification is configurable through the Verify Remote Cert option in Harbor's registry configuration. VKS clusters can be configured to trust self-signed CA certificates for private registries, and from ClusterClass builtin-generic-v3.2.0 onward, Harbor services with custom certificates are automatically trusted by VKS clusters through deterministic ordering of trusted certificates. The VCF Consumption CLI's imgpkg tool provides --registry-verify-certs and --registry-ca-cert-path flags for validating registry connections against configured trust anchors. Certificate expiration status across VKS control-plane

certificates can be monitored using the `kubeadm certs check-expiration` command. For manual certificate renewal, the `kubeadm certs renew` command can renew all or individual cluster certificates; in replicated control-plane deployments, it must be executed on each control-plane node. Runtime OCSP or CRL revocation checking is not enforced natively by the Kubernetes API server at authentication time; organizations with requirements for runtime revocation validation integrate external PKI systems at the application or service mesh layer.

Together, these mechanisms construct and verify certification paths to accepted trust anchors across VCF components and check certificate status through CRL import and Certificate Revocation List Distribution Point validation in VCF Operations and NSX. The VKS consumption layer extends these PKI foundations through VMCA-anchored cluster component certificates, ClusterTrustBundle-based trust anchor distribution, and automated rotation at both the platform and workload levels, covering the automated certification path validation capabilities the control asks about.

VMware Private AI Services (PAIS) contributes to PKI-based authentication through a CA trust bundle management layer that spans its AI service components. PAIS validates certificates through configurable CA trust bundles stored as ConfigMaps in the VCF Automation namespace, with bundles supplied either as certificate files or ConfigMap resources by the administrator of the Harbor registry or content management system that PAIS connects to. The PAISConfiguration custom resource's `spec.clientTls.caBundleRefs` field references those ConfigMaps for client TLS authentication with OIDC providers and Harbor registries, and PAIS is updated with a new ConfigMap resource when a CA certificate is selected and added from the namespace. Trust bundles are organized per application and rotated through the Service Configuration tab by deleting the bundle containing the old certificate and adding a new bundle with the updated certificate before expiration.

PAIS requires CA trust bundles to be configured for the external entities it connects to: OIDC providers for identity federation, Harbor registries for model artifact retrieval, content management systems serving as Knowledge Base data sources (such as Confluence sites and Microsoft SharePoint On-Premises deployments), and remote MCP servers integrated for specialized data and capabilities. On Deep Learning VMs, Harbor registry CA certificates are copied to the `/usr/local/share/ca-certificates` directory and added to the system trust store using the `update-ca-certificates` command.

Certificate validation is enforced through several PAIS authentication paths. The VCF Consumption CLI accepts the `--ca-certificate` option to validate the VCF Automation endpoint certificate when it is not already in the local truststore, covering namespace login, VKS cluster provisioning with vGPU or GPU passthrough in both connected and disconnected environments, and deployment of completion or embedding model endpoints by using `kubectrl`. The VCF Automation instance certificate must be saved locally on the machine deploying model endpoints or provisioning VKS clusters. The VCF Automation Provider Management UI requires trusted certificate import via Certificate Management > Trusted Certificates and certificate acceptance during the namespace import process for PAIS integration with VMware vCenter, and records a `TM_CERT_BASE64` credential captured by running `openssl s_client` against the VCF Automation FQDN on port 443. PAIS OIDC authentication supports configuring Extra Audiences so the service accepts additional OAuth 2.0 clients during access token validation, and Grafana OIDC authentication for PAIS supports PKCE and refresh tokens. The PAIS ingress self-signed certificate is retrievable from the `pais-ingress-default` Kubernetes secret for downstream TLS configuration.

The actual certification path construction and certificate status checking at the network layer is performed by the underlying VKS and Kubernetes platform rather than by PAIS directly. The PAIS contribution is the CA trust bundle configuration layer, managed through ConfigMaps, the PAISConfiguration custom resource, and the VCF Automation Provider Management UI, that populates the trust stores consumed by those platform-level PKI validation mechanisms across its AI service components.

### VMware vDefend (Contributes)

VMware vDefend uses PKI-based certificate validation within the Security Services Platform (SSP) to authenticate connections between platform components and to secure communications with external identity providers. The platform maintains a certificate hierarchy that supports automated certification path construction and verification against trusted anchors.

The Security Services Platform maintains its own certificate infrastructure, including Web Proxy, Platform Certificates, and Intermediate CA certificates. A CertWatcher component loads and validates root CA certificates and keys during initialization, maintaining a CA certificate pool and truststore for validating cluster API certificates. The platform monitors certificate status, identifying certificates that are expired, expiring soon, or valid, and generates alarms when certificates require attention.

Administrators can view certificate details, replace certificates, generate certificate signing requests, and import certificates signed by a certificate authority of their choice. The Security Services Platform proactively manages its Platform CA certificate lifecycle, raising an alert 67 days before expiration and automatically initiating a certificate refresh 60 days before expiration for all internal entities and Intermediate CAs. A Certificates UI page provides visibility into Certificate Refresh Status and External Rollout details, including Bare Metal server certificate validity status.

For network detection components, NDR Sensors connect to the Security Services Platform using Mutual TLS (mTLS) authentication. PKIX certificate path validation is used to authenticate these connections, and authentication failures are logged with specific error messages indicating PKIX path validation or access authorization issues. This validation occurs automatically during sensor registration and ongoing communication.

The Security Services Platform validates certificates when establishing secure connections to external identity providers. When configured for LDAPS authentication, the platform requires an LDAP server certificate to authenticate directory service users, establishing a verified connection. Certificate import capabilities allow administrators to identify web proxy or ingress certificates signed by a certificate authority of their choice, extending the trust model to external systems.

For deployments using TLS Inspection, vDefend provides additional certificate path validation capabilities at the network layer. TLS Inspection certificate management supports importing or updating trusted CA bundles and a default CRL for server certificate validation during traffic inspection. In 9.1, TLS Inspection supports OCSP stapling options for certificate validation, providing a mechanism for checking certificate revocation status in addition to CRL-based checking. When inspecting encrypted traffic to external services, server certificate validation can be enforced against the configured trusted CA bundle and revocation information. The TLS proxy supports Enterprise issuing CA certificates, where a trusted subordinate CA certificate is signed by an Enterprise issuing CA through a CSR workflow, allowing integration with an organization's existing enterprise PKI infrastructure.

vDefend contributes to this control through its internal certificate path validation, automated certificate chain management for SSP component authentication, PKIX-based authentication for NDR Sensor connections via mTLS, CRL-based revocation checking, and OCSP stapling support for TLS Inspection server certificate validation. Full enterprise PKI integration for administrator authentication relies on the organization's LDAP-based identity infrastructure rather than vDefend alone, and comprehensive certificate path validation for all external connections depends on the broader VCF certificate infrastructure.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) uses PKI-based authentication for site pairing, appliance management, and management-plane communication. PNR uses TLS certificates and private keys to protect network communication and establish authentication with other servers, and treats certificates and keys as part of its core security infrastructure.

The PNR appliance supports SSL/TLS server endpoint certificates that can be either self-signed or custom certificates signed by a certificate authority. The PNR self-signed certificate expires five years from the first boot of the appliance. Custom certificates may be intermediate CA certificates that are not the root of a trust chain, supporting full certification path construction to an accepted trust anchor. Custom SSL/TLS certificates must meet specific requirements to work with PNR, and the current time must be within the certificate's validity period for authentication to succeed.

The PNR Configuration REST API provides endpoints to add certificate authority (CA) certificates and to retrieve installed CA certificates used to validate other servers' certificates. PNR also includes Authentication APIs as part of its REST API offerings. The PNR user interface validates host certificates against subject alternative names during connection to Platform Services Controller hosts. The PNR server establishes authenticated communication channels to remote services by acquiring a holder-of-key SAML token from VCF Single Sign-On (VCF SSO), then sends this token in a cryptographically signed message.

PNR includes certificate status monitoring. PNR raises a certificate expiring event when the certificate expiration date is within the configured number of days, with a default value of 30 days controlled by the `localSiteStatus.minCertRemainingTime` setting. PNR generates `SrmCertificateExpiringEvent` when the SSL/TLS certificate for a specified Site Recovery Manager (SRM) Server is approaching expiration, and `SrmCertificateExpiredEvent` when the certificate has expired. Certificate verification errors are recorded in security-related log files for audit-trail purposes.

PNR supports certificate-based authentication with storage providers through either direct certificate specification or thumbprint verification. During site registration and remote site management, PNR can accept or reject remote VMware vCenter certificates and prompts the administrator before adding a certificate to the trusted store. The Configure Local Sites workflow and the VCF Operations orchestrator plug-in include an option to ignore certificate warnings, which silently adds the certificate to the trusted store; administrators should use this option carefully because it bypasses interactive verification.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) includes several certificate management capabilities that contribute to PKI-based authentication validation. DSM maintains a Trusted Root Certificates store where administrators can add PEM-encoded Root and Intermediate CA certificates for directory service connections, database cluster connections, and connections to external systems such as backup repositories. The store displays the expiration time and date for each stored certificate, giving administrators visibility into certificate lifecycle status. Custom trusted certificates added to DSM must comply with FIPS requirements. Certificate validation behavior is configurable through the advanced-system-config ConfigMap, which supports two validation modes; the "latest" mode is the default for version 9.1 and later.

When clients connect to a DSM database cluster, DSM stores PEM-encoded certificate authorities in the ca.crt property of a Kubernetes Secret. Clients must add this CA certificate to their trusted CA list or operating system trust store before a connection is established. Connections from DSM to the VCF Automation Provider also require certificate validation and trust establishment before the connection becomes active. The Consumption Operator requires a TLS certificate for secure communication between the Consumption Operator and the DSM provider. DSM validates certificate status and requires explicit trust before the connection proceeds.

DSM supports custom certificates for both the Provider VM and for individual database clusters. Administrators configure custom certificates in PEM format, including the full certificate chain with its associated private key and CA certificate. The DSM API provides programmatic capability to configure custom certificates for database clusters, supporting integration with certificate management workflows. Administrators with restrictive certificate policies should replace the default DSM certificate with a CA-issued custom certificate. Custom certificate configuration for the Provider VM can only be performed on a fresh installation where no data services are deployed. If the Provider VM certificate expires, access to the DSM UI, database cluster upgrades or restarts, and cluster metrics may all be affected.

Certificate-based authentication is supported for PostgreSQL databases through host-based authentication (HBA) configuration, which accepts cert as an authentication method alongside SCRAM-SHA-256 and reject options. DSM supports mutual TLS (mTLS) authentication for metrics forwarding by enabling the Client Certificate Authentication toggle and providing the client certificate and private key. For SQL Server databases, DSM supports Custom CA as a Certificate Trust Source option for Active Directory domain connections when the system trust store cannot verify the server certificate.

For backup repository connections, DSM supports Trust On First Use (TOFU) configuration to extract and trust certificates for specific backup locations. DSM also introduced updated cryptographic algorithm requirements for certificates; administrators should verify that certificates comply with these requirements before or after upgrading, as noncompliant certificates can cause validation errors, service startup failures, or TLS handshake issues.

Satisfying the full control intent requires the organization to implement automated certificate renewal and revocation-checking processes. Certificate renewal for the Provider VM and database cluster certificates is a manual administrator responsibility; the documentation explicitly places this obligation on the administrator rather than on DSM itself. DSM does not provide built-in automated mechanisms for OCSP or CRL-based certificate status checking; these must be provided by the organization's PKI infrastructure or by PKI automation tooling configured with an appropriate issuer.

### **VMware Avi Load Balancer (Meets)**

VMware Avi Load Balancer provides native PKI-based certificate validation through the PKI Profile, a reusable configuration object that implements the full certification path construction and status-checking pipeline required by this control.

The PKI Profile is configured under Security > Client SSL Certificate Validation > PKI Profile in the Avi Controller UI or API. It stores one or more certificate authority (CA) certificates that serve as trust anchors. When a client presents a certificate for

authentication, Avi constructs the certification path from the presented certificate through any intermediate CAs to the root trust anchor stored in the PKI Profile. Each link in the intermediate chain must be present in the PKI Profile for the validation to succeed, enforcing complete chain construction to a known anchor.

Certificate status checking is handled through Certificate Revocation List (CRL) support built into the PKI Profile. The profile stores CRLs from issuing certificate authorities, including CRLs covering both end-entity and intermediate certificates. The Enable CRL Check option in the PKI Profile controls full-chain CRL checking for client certificate validation. When enabled, Avi checks the revocation status of the presented certificate and each intermediate against the configured CRLs before allowing the connection to proceed. A separate Ignore Peer Chain setting is available to control whether intermediate certificate validation is required, providing explicit configuration control over path depth.

The PKI Profile is attached to virtual service configurations through the HTTP Application Profile's Authentication tab, where it is referenced alongside mutual TLS settings to require and validate client certificates. The `avi.ssl.set_pki_profile()` DataScript function allows dynamic assignment of a PKI Profile and client certificate verification mode on a per-connection basis for advanced traffic-handling scenarios. A single PKI Profile object can be shared across multiple HTTP profiles within a tenant, centralizing trust anchor and revocation list management.

In Kubernetes environments, the Avi Kubernetes Operator (AKO) exposes PKI-based authentication through the PKIProfile custom resource definition (CRD). The PKIProfile CRD defines a list of trusted Certificate Authorities (Root and Intermediate) through the required `ca_certs` field and integrates with RouteBackendExtension to enforce certificate validation for backend TLS connections. The System-Standard SSL Profile works in conjunction with the PKIProfile CRD for certificate validation and trust management across Kubernetes ingress configurations. This extends the same PKI validation pipeline to cloud-native deployment patterns without requiring configuration outside Kubernetes-native tooling.

In VCF deployments, the PKI Profile is available as a custom resource through the IaaS Service Console, where it manages the list of trusted authorities permitted to sign client certificates. This supports Zero Trust security patterns between client and server across the VCF-integrated load balancing layer.

For NSX and vCenter cloud integrations, the Avi Controller provides a Truststore PKI Profile setting under System Settings on the Access tab to enable certificate verification and supports CA certificate import for client SSL certificate validation in those cloud contexts.



## Control CR 1.9(c)

> Sub-item (c) of CR 1.9 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 1.9 requirement in the standard.

**SCF Controls:** IAC-10.2

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF includes the VMware Certificate Authority (VMCA), a built-in internal certificate authority integral with every vSphere deployment that is the foundation for constructing and verifying certification paths within the VCF infrastructure. Components that obtain certificates from VMCA have an inherent chain of trust back to this root. Administrators can replace the VMCA root certificate when organizational policy requires alignment with an enterprise PKI hierarchy, at which point VMCA operates as a subordinate CA, and all certificates it issues chain back to the enterprise root. Replacing the VMCA root certificate triggers a restart of services that use that certificate, and the VMware vCenter Certificate Management API provides a dedicated endpoint for this operation. Management of both VMCA-issued and third-party certificates is supported across the environment.

The VMware Kubernetes Service (VKS) platform, which provides Kubernetes cluster provisioning within the VCF consumption layer, extends this VMCA-anchored trust chain to cluster infrastructure. TLS certificates for vSphere Supervisor and VKS cluster components, including the Kubernetes API server, kubelet, etcd, scheduler, and front-proxy, derive from the VMCA certificate. A CA-signed certificate can also be uploaded to replace the default certificate on the Supervisor Kubernetes control plane, supporting organizations that require alignment with an external PKI hierarchy at the Supervisor infrastructure level. VKS Service Cluster certificate authorities have a 9-year lifetime; leaf certificates have a 364-day validity period. Trust configuration for VKS clusters supports specifying additional trusted certificate authorities through the `TkgServiceConfiguration additionalTrustedCAs` field and the `TanzuKubernetesCluster` API's corresponding trust configuration, expressed as base64-encoded PEM certificate data, allowing administrators to extend trust to enterprise CA hierarchies within the Kubernetes cluster context. The `Kubernetes ClusterTrustBundle` API provides a structured mechanism for storing and distributing trust anchors to workloads and cluster components, decoupling trust anchor distribution from the certificate-signing pipeline. The `VKS AuthenticationConfiguration Secret` also accepts an optional certificate authority field to support JWT-based authentication flows backed by an internal PKI.

VCF SSO supports certificate-based authentication through holder-of-key tokens, where security artifacts embedded in the token are validated against trusted certificate authorities. The VCF SSO client can obtain a holder-of-key token by sending a token request that uses a certificate, and the SSO SDK includes samples demonstrating how solution components use their private key and certificate to acquire holder-of-key tokens. Solution user accounts, which handle service-to-service communication between vCenter, VCF Operations, and other platform components, authenticate using certificates rather than passwords, with the platform validating the certificate chain for each connection. Requests are signed with a user certificate private key through a security signature handler.

Trust store management across VCF components allows administrators to define accepted trust anchors. VCF Automation allows certificates to be added to the trust store through the UI or the VCF Automation API. The vSphere Automation SDKs support creating certificate stores using `keytool` to import certificates into a truststore with a password, providing a structured approach to managing which root and intermediate certificates are trusted. The `dir-cli trustedcert publish` command provides a CLI path for adding certificate authority certificates, including intermediate CA certificates, to the `VECS TRUSTED_ROOTS` store on vCenter. The vSphere Automation API exposes certificate management interfaces including `TrustedRootChains`, `VMCARoot`, and TLS certificate operations, allowing programmatic management of certificate trust. The NSX management plane and truststore APIs provide additional certificate management for network infrastructure components, including management of non-service certificates.

VCF Automation provides automated certificate validation capabilities, including an API endpoint that validates certificates and returns a resource with the certificate ID upon successful validation. When establishing trust with remote servers, VCF Automation can verify certificate information for SSL communication and retrieve intermediate and leaf certificates from



issuing certificate authorities if the certificate chain is incomplete, allowing administrators to review the full path before making a trust decision and make selective choices about which certificates in a chain to trust. This chain retrieval capability supports the construction of certification paths from an end-entity certificate back to a trusted root. VCF Automation also validates server certificates during LDAPS connections through a Trust Certificate window, applying certificate verification to directory service authentication paths.

VCF Operations for Networks offers configurable certificate validation with two modes: Automatic Acceptance, where the system automatically accepts all detected certificate changes, and Manual Acceptance, where the system presents a notification for an administrator to review and accept certificates individually. Standard certificate validation can be activated in VCF Operations to verify certificates during connection testing, using a standard verification flow that checks the certificate chain against configured trust anchors. When standard certificate validation is activated, the Certificate Revocation List (CRL) tab becomes available, allowing administrators to upload and manage revocation lists for digital certificates that have been revoked by the issuing CA before their scheduled expiration date.

NSX also supports certificate revocation checking. NSX Manager can import certificate revocation lists in PEM format through its API and can enable or disable Certificate Revocation List Distribution Point (CDP) validation checks during certificate import. The NSX API provides endpoints to configure NSX Manager to retrieve CRLs, supporting ongoing revocation validation for network infrastructure certificates.

VCF applies proper certificate validation practices in its own operations. The platform documentation explicitly states that trust-all trust managers, which accept all certificates without validation, are appropriate only for development environments and should not be used in production. Production implementations should use proper certificate support with validated trust paths.

Automated certificate lifecycle management further supports PKI operations. VCF supports automatic renewal of TLS certificates for components using VMCA, Microsoft CA, OpenSSL CA, or self-signed certificates, with renewal triggered 60 days before certificate expiration. The Machine SSL certificate can be renewed for a specified period up to 730 days using the vSphere Automation API, and the auto-renewal toggle in VCF Operations console (Fleet Management, Certificates) allows administrators to activate automatic renewal. By default, the VMCA root certificate has a 10-year lifetime, and all certificates that VMCA signs expire no later than the root certificate; organizations planning to use VMCA long-term should account for root CA rotation before that threshold. VCF Operations provides a Certificates tab for each workload domain that displays certificate expiration status, supporting monitoring of certificate health.

Within VKS clusters, certificate lifecycle management operates through dedicated automation. The ClusterClass certificateRotation parameter governs automated control-plane certificate rotation, and the renewalDaysBeforeExpiry field defaults to 90 days and triggers a rolling rollout of updated certificates before they expire; this value is configurable per ClusterClass to align with organizational PKI policy. Certificate rotation is enabled by default across VKS ClusterClass versions, including builtin-generic-v3.2.0, v3.3.0, and v3.4.0 and later, so automated pre-expiry renewal is active without explicit configuration. Kubelet TLS bootstrapping automates initial certificate distribution and signing for new nodes joining a VKS cluster and can be configured to automatically request renewal when the kubelet certificate approaches expiry, with renewal approved automatically or manually depending on configuration. For application workloads, cert-manager is available as a standard VKS package; it creates TLS certificates for workloads and renews them automatically before expiration. CertificateSigningRequest resources from both the native Kubernetes certificates.k8s.io API group and the cert-manager.io API group are supported for certificate lifecycle management within VKS clusters. Cert-manager is also a prerequisite for several standard VKS addons including Contour, ExternalDNS, Prometheus, and Harbor. Harbor, available as a standard VKS package for container image registry services, supports TLS certificate configuration through cert-manager integration, a custom certificate provided via the tlsCertificate field, or a reference to an existing Kubernetes secret; remote registry certificate verification is configurable through the Verify Remote Cert option in Harbor's registry configuration. VKS clusters can be configured to trust self-signed CA certificates for private registries, and from ClusterClass builtin-generic-v3.2.0 onward, Harbor services with custom certificates are automatically trusted by VKS clusters through deterministic ordering of trusted certificates. The VCF Consumption CLI's imgpkg tool provides --registry-verify-certs and --registry-ca-cert-path flags for validating registry connections against configured trust anchors. Certificate expiration status across VKS control-plane

certificates can be monitored using the `kubeadm certs check-expiration` command. For manual certificate renewal, the `kubeadm certs renew` command can renew all or individual cluster certificates; in replicated control-plane deployments, it must be executed on each control-plane node. Runtime OCSP or CRL revocation checking is not enforced natively by the Kubernetes API server at authentication time; organizations with requirements for runtime revocation validation integrate external PKI systems at the application or service mesh layer.

Together, these mechanisms construct and verify certification paths to accepted trust anchors across VCF components and check certificate status through CRL import and Certificate Revocation List Distribution Point validation in VCF Operations and NSX. The VKS consumption layer extends these PKI foundations through VMCA-anchored cluster component certificates, ClusterTrustBundle-based trust anchor distribution, and automated rotation at both the platform and workload levels, covering the automated certification path validation capabilities the control asks about.

VMware Private AI Services (PAIS) contributes to PKI-based authentication through a CA trust bundle management layer that spans its AI service components. PAIS validates certificates through configurable CA trust bundles stored as ConfigMaps in the VCF Automation namespace, with bundles supplied either as certificate files or ConfigMap resources by the administrator of the Harbor registry or content management system that PAIS connects to. The PAISConfiguration custom resource's `spec.clientTls.caBundleRefs` field references those ConfigMaps for client TLS authentication with OIDC providers and Harbor registries, and PAIS is updated with a new ConfigMap resource when a CA certificate is selected and added from the namespace. Trust bundles are organized per application and rotated through the Service Configuration tab by deleting the bundle containing the old certificate and adding a new bundle with the updated certificate before expiration.

PAIS requires CA trust bundles to be configured for the external entities it connects to: OIDC providers for identity federation, Harbor registries for model artifact retrieval, content management systems serving as Knowledge Base data sources (such as Confluence sites and Microsoft SharePoint On-Premises deployments), and remote MCP servers integrated for specialized data and capabilities. On Deep Learning VMs, Harbor registry CA certificates are copied to the `/usr/local/share/ca-certificates` directory and added to the system trust store using the `update-ca-certificates` command.

Certificate validation is enforced through several PAIS authentication paths. The VCF Consumption CLI accepts the `--ca-certificate` option to validate the VCF Automation endpoint certificate when it is not already in the local truststore, covering namespace login, VKS cluster provisioning with vGPU or GPU passthrough in both connected and disconnected environments, and deployment of completion or embedding model endpoints by using `kubectrl`. The VCF Automation instance certificate must be saved locally on the machine deploying model endpoints or provisioning VKS clusters. The VCF Automation Provider Management UI requires trusted certificate import via Certificate Management > Trusted Certificates and certificate acceptance during the namespace import process for PAIS integration with VMware vCenter, and records a `TM_CERT_BASE64` credential captured by running `openssl s_client` against the VCF Automation FQDN on port 443. PAIS OIDC authentication supports configuring Extra Audiences so the service accepts additional OAuth 2.0 clients during access token validation, and Grafana OIDC authentication for PAIS supports PKCE and refresh tokens. The PAIS ingress self-signed certificate is retrievable from the `pais-ingress-default` Kubernetes secret for downstream TLS configuration.

The actual certification path construction and certificate status checking at the network layer is performed by the underlying VKS and Kubernetes platform rather than by PAIS directly. The PAIS contribution is the CA trust bundle configuration layer, managed through ConfigMaps, the PAISConfiguration custom resource, and the VCF Automation Provider Management UI, that populates the trust stores consumed by those platform-level PKI validation mechanisms across its AI service components.

### **VMware vDefend (Contributes)**

VMware vDefend uses PKI-based certificate validation within the Security Services Platform (SSP) to authenticate connections between platform components and to secure communications with external identity providers. The platform maintains a certificate hierarchy that supports automated certification path construction and verification against trusted anchors.

The Security Services Platform maintains its own certificate infrastructure, including Web Proxy, Platform Certificates, and Intermediate CA certificates. A CertWatcher component loads and validates root CA certificates and keys during initialization, maintaining a CA certificate pool and truststore for validating cluster API certificates. The platform monitors certificate status, identifying certificates that are expired, expiring soon, or valid, and generates alarms when certificates require attention.

Administrators can view certificate details, replace certificates, generate certificate signing requests, and import certificates signed by a certificate authority of their choice. The Security Services Platform proactively manages its Platform CA certificate lifecycle, raising an alert 67 days before expiration and automatically initiating a certificate refresh 60 days before expiration for all internal entities and Intermediate CAs. A Certificates UI page provides visibility into Certificate Refresh Status and External Rollout details, including Bare Metal server certificate validity status.

For network detection components, NDR Sensors connect to the Security Services Platform using Mutual TLS (mTLS) authentication. PKIX certificate path validation is used to authenticate these connections, and authentication failures are logged with specific error messages indicating PKIX path validation or access authorization issues. This validation occurs automatically during sensor registration and ongoing communication.

The Security Services Platform validates certificates when establishing secure connections to external identity providers. When configured for LDAPS authentication, the platform requires an LDAP server certificate to authenticate directory service users, establishing a verified connection. Certificate import capabilities allow administrators to identify web proxy or ingress certificates signed by a certificate authority of their choice, extending the trust model to external systems.

For deployments using TLS Inspection, vDefend provides additional certificate path validation capabilities at the network layer. TLS Inspection certificate management supports importing or updating trusted CA bundles and a default CRL for server certificate validation during traffic inspection. In 9.1, TLS Inspection supports OCSP stapling options for certificate validation, providing a mechanism for checking certificate revocation status in addition to CRL-based checking. When inspecting encrypted traffic to external services, server certificate validation can be enforced against the configured trusted CA bundle and revocation information. The TLS proxy supports Enterprise issuing CA certificates, where a trusted subordinate CA certificate is signed by an Enterprise issuing CA through a CSR workflow, allowing integration with an organization's existing enterprise PKI infrastructure.

vDefend contributes to this control through its internal certificate path validation, automated certificate chain management for SSP component authentication, PKIX-based authentication for NDR Sensor connections via mTLS, CRL-based revocation checking, and OCSP stapling support for TLS Inspection server certificate validation. Full enterprise PKI integration for administrator authentication relies on the organization's LDAP-based identity infrastructure rather than vDefend alone, and comprehensive certificate path validation for all external connections depends on the broader VCF certificate infrastructure.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) uses PKI-based authentication for site pairing, appliance management, and management-plane communication. PNR uses TLS certificates and private keys to protect network communication and establish authentication with other servers, and treats certificates and keys as part of its core security infrastructure.

The PNR appliance supports SSL/TLS server endpoint certificates that can be either self-signed or custom certificates signed by a certificate authority. The PNR self-signed certificate expires five years from the first boot of the appliance. Custom certificates may be intermediate CA certificates that are not the root of a trust chain, supporting full certification path construction to an accepted trust anchor. Custom SSL/TLS certificates must meet specific requirements to work with PNR, and the current time must be within the certificate's validity period for authentication to succeed.

The PNR Configuration REST API provides endpoints to add certificate authority (CA) certificates and to retrieve installed CA certificates used to validate other servers' certificates. PNR also includes Authentication APIs as part of its REST API offerings. The PNR user interface validates host certificates against subject alternative names during connection to Platform Services Controller hosts. The PNR server establishes authenticated communication channels to remote services by acquiring a holder-of-key SAML token from VCF Single Sign-On (VCF SSO), then sends this token in a cryptographically signed message.

PNR includes certificate status monitoring. PNR raises a certificate expiring event when the certificate expiration date is within the configured number of days, with a default value of 30 days controlled by the `localSiteStatus.minCertRemainingTime` setting. PNR generates `SrmCertificateExpiringEvent` when the SSL/TLS certificate for a specified Site Recovery Manager (SRM) Server is approaching expiration, and `SrmCertificateExpiredEvent` when the certificate has expired. Certificate verification errors are recorded in security-related log files for audit-trail purposes.

PNR supports certificate-based authentication with storage providers through either direct certificate specification or thumbprint verification. During site registration and remote site management, PNR can accept or reject remote VMware vCenter certificates and prompts the administrator before adding a certificate to the trusted store. The Configure Local Sites workflow and the VCF Operations orchestrator plug-in include an option to ignore certificate warnings, which silently adds the certificate to the trusted store; administrators should use this option carefully because it bypasses interactive verification.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) includes several certificate management capabilities that contribute to PKI-based authentication validation. DSM maintains a Trusted Root Certificates store where administrators can add PEM-encoded Root and Intermediate CA certificates for directory service connections, database cluster connections, and connections to external systems such as backup repositories. The store displays the expiration time and date for each stored certificate, giving administrators visibility into certificate lifecycle status. Custom trusted certificates added to DSM must comply with FIPS requirements. Certificate validation behavior is configurable through the advanced-system-config ConfigMap, which supports two validation modes; the "latest" mode is the default for version 9.1 and later.

When clients connect to a DSM database cluster, DSM stores PEM-encoded certificate authorities in the `ca.crt` property of a Kubernetes Secret. Clients must add this CA certificate to their trusted CA list or operating system trust store before a connection is established. Connections from DSM to the VCF Automation Provider also require certificate validation and trust establishment before the connection becomes active. The Consumption Operator requires a TLS certificate for secure communication between the Consumption Operator and the DSM provider. DSM validates certificate status and requires explicit trust before the connection proceeds.

DSM supports custom certificates for both the Provider VM and for individual database clusters. Administrators configure custom certificates in PEM format, including the full certificate chain with its associated private key and CA certificate. The DSM API provides programmatic capability to configure custom certificates for database clusters, supporting integration with certificate management workflows. Administrators with restrictive certificate policies should replace the default DSM certificate with a CA-issued custom certificate. Custom certificate configuration for the Provider VM can only be performed on a fresh installation where no data services are deployed. If the Provider VM certificate expires, access to the DSM UI, database cluster upgrades or restarts, and cluster metrics may all be affected.

Certificate-based authentication is supported for PostgreSQL databases through host-based authentication (HBA) configuration, which accepts `cert` as an authentication method alongside `SCRAM-SHA-256` and `reject` options. DSM supports mutual TLS (mTLS) authentication for metrics forwarding by enabling the Client Certificate Authentication toggle and providing the client certificate and private key. For SQL Server databases, DSM supports Custom CA as a Certificate Trust Source option for Active Directory domain connections when the system trust store cannot verify the server certificate.

For backup repository connections, DSM supports Trust On First Use (TOFU) configuration to extract and trust certificates for specific backup locations. DSM also introduced updated cryptographic algorithm requirements for certificates; administrators should verify that certificates comply with these requirements before or after upgrading, as noncompliant certificates can cause validation errors, service startup failures, or TLS handshake issues.

Satisfying the full control intent requires the organization to implement automated certificate renewal and revocation-checking processes. Certificate renewal for the Provider VM and database cluster certificates is a manual administrator responsibility; the documentation explicitly places this obligation on the administrator rather than on DSM itself. DSM does not provide built-in automated mechanisms for OCSP or CRL-based certificate status checking; these must be provided by the organization's PKI infrastructure or by PKI automation tooling configured with an appropriate issuer.

### **VMware Avi Load Balancer (Meets)**

VMware Avi Load Balancer provides native PKI-based certificate validation through the PKI Profile, a reusable configuration object that implements the full certification path construction and status-checking pipeline required by this control.

The PKI Profile is configured under Security > Client SSL Certificate Validation > PKI Profile in the Avi Controller UI or API. It stores one or more certificate authority (CA) certificates that serve as trust anchors. When a client presents a certificate for

authentication, Avi constructs the certification path from the presented certificate through any intermediate CAs to the root trust anchor stored in the PKI Profile. Each link in the intermediate chain must be present in the PKI Profile for the validation to succeed, enforcing complete chain construction to a known anchor.

Certificate status checking is handled through Certificate Revocation List (CRL) support built into the PKI Profile. The profile stores CRLs from issuing certificate authorities, including CRLs covering both end-entity and intermediate certificates. The Enable CRL Check option in the PKI Profile controls full-chain CRL checking for client certificate validation. When enabled, Avi checks the revocation status of the presented certificate and each intermediate against the configured CRLs before allowing the connection to proceed. A separate Ignore Peer Chain setting is available to control whether intermediate certificate validation is required, providing explicit configuration control over path depth.

The PKI Profile is attached to virtual service configurations through the HTTP Application Profile's Authentication tab, where it is referenced alongside mutual TLS settings to require and validate client certificates. The `avi.ssl.set_pki_profile()` DataScript function allows dynamic assignment of a PKI Profile and client certificate verification mode on a per-connection basis for advanced traffic-handling scenarios. A single PKI Profile object can be shared across multiple HTTP profiles within a tenant, centralizing trust anchor and revocation list management.

In Kubernetes environments, the Avi Kubernetes Operator (AKO) exposes PKI-based authentication through the PKIProfile custom resource definition (CRD). The PKIProfile CRD defines a list of trusted Certificate Authorities (Root and Intermediate) through the required `ca_certs` field and integrates with RouteBackendExtension to enforce certificate validation for backend TLS connections. The System-Standard SSL Profile works in conjunction with the PKIProfile CRD for certificate validation and trust management across Kubernetes ingress configurations. This extends the same PKI validation pipeline to cloud-native deployment patterns without requiring configuration outside Kubernetes-native tooling.

In VCF deployments, the PKI Profile is available as a custom resource through the IaaS Service Console, where it manages the list of trusted authorities permitted to sign client certificates. This supports Zero Trust security patterns between client and server across the VCF-integrated load balancing layer.

For NSX and vCenter cloud integrations, the Avi Controller provides a Truststore PKI Profile setting under System Settings on the Access tab to enable certificate verification and supports CA certificate import for client SSL certificate validation in those cloud contexts.

## Control CR 1.9(d)

> Sub-item (d) of CR 1.9 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 1.9 requirement in the standard.

**SCF Controls:** CRY-09

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides cryptographic key management controls that protect the confidentiality, integrity, and availability of keys through multiple integrated mechanisms spanning key storage, access control, lifecycle management, and availability safeguards.

At the core of VCF's key management architecture is the CryptoManager managed object hierarchy in the vSphere API. The CryptoManager and its child object CryptoManagerKmp provide programmatic interfaces for managing cryptographic keys, integrating with external key management servers, and performing cryptographic operations. The API exposes methods for adding, listing, and removing keys, giving administrators and automation tools full control over key lifecycle operations.

VCF supports two models for key storage. Organizations can integrate with external Key Management Servers that support the Key Management Interoperability Protocol (KMIP) 1.1 standard, allowing interoperability with a wide range of third-party KMS products. Alternatively, the vSphere Native Key Provider (NKP) is included in all vSphere editions and does not require an external key server. NKP can be added, updated, and removed through the Providers interface in the vSphere API, and its configuration must be exported at least once before it can be used. For organizations that choose an external KMS, administrators establish a trust relationship by uploading a certificate and key file to the KMS vendor, and then select a KMS cluster during encryption configuration. The KMS must appear in the VMware Compatibility Guide for Key Management Servers and be KMIP 1.1 compliant; that guide provides the authoritative list of tested and supported third-party KMS products. Some KMS vendors allow further isolation of encryption keys by specifying dedicated credentials during configuration. Administrators can also configure separate key providers for different users or departments, providing more granular access control over encryption keys by organizational unit. VCF establishes a cryptographic domain of trust between key providers, VMware vCenter, and VMware ESX hosts so that keys are accessible only to authorized components.

For organizations consuming VCF services through the multi-tenant consumption layer, VCF Encryption Management extends key management to organization administrators. This feature allows organization administrators to use encryption keys from their own key providers for encryption of virtual machines and disks within their VCF environment, enabling tenant-controlled key custody independent of the platform provider's key management infrastructure.

VCF uses a layered key hierarchy for encryption. Key Encryption Keys (KEK) wrap Disk Encryption Keys (DEK), with optional wrapping keys protecting KEKs. Disk encryption keys use XTS-AES-256. This layered approach means that rotating or revoking a higher-level key protects all keys beneath it without requiring re-encryption of the underlying data. ESX hosts use the KEK to encrypt their internal keys and store only the encrypted internal keys on disk, not the KEK itself.

For virtual machines, the vSphere API supports two reencrypt operations: shallow reencrypt replaces only the KEK and can be performed while the virtual machine is powered on, while deep reencrypt replaces both KEKs and DEKs but requires the virtual machine to be powered off. For vSAN data-at-rest encryption, administrators can generate new KEKs or new DEKs with the option to re-encrypt all data on the storage using new keys. vSAN encryption supports configurable rekey intervals that can be set to default or custom values. vSAN data-in-transit encryption also supports a configurable rekey interval, defaulting to 1,440 minutes (24 hours) with a range from 30 minutes to 7 days. An Allow Reduced Redundancy option is available for clusters with limited resources during key rotation operations.

Workloads running in VMware Kubernetes Service (VKS) clusters on Supervisor benefit from an additional layer of key management integrated with the same external KMS infrastructure. VKS uses the Kubernetes KMS provider plugin interface to protect data at rest in etcd. The EncryptionConfiguration resource configures a provider of type kms in the providers array on the API server; the plugin communicates with an external Key Management Service via a Unix domain socket endpoint, keeping all key material outside the cluster. The current stable implementation is KMS v2 (available from Kubernetes v1.29),



which uses an envelope encryption scheme where each resource is encrypted under a Data Encryption Key (DEK) that is itself wrapped by a Key Encryption Key (KEK) held in the remote KMS. KMS v2 employs a key derivation function to generate single-use DEKs from a secret seed combined with random data, reducing the frequency of external KMS calls while maintaining per-encryption key uniqueness. KMS plugins are required to verify that a `key_id` presented for decryption was encrypted by them, providing a chain-of-custody check that restricts decryption to the originating plugin. The KMS provider must be listed first in the `EncryptionConfiguration` file to take precedence over any local encryption providers. VKS administrators manage sensitive material such as passwords, tokens, and SSH keys through Kubernetes Secret objects, which are protected at rest by the cluster's encryption-at-rest configuration. For environments requiring centralized secrets management beyond what Kubernetes natively provides, Supervisor and VKS clusters support integration with external vault systems. In CI/CD architectures built on Supervisor through integrations such as Harness or Argo CD, HashiCorp Vault provides dynamic secret generation, access policies, and a centralized store for API keys, certificates, and credentials; dynamic secrets reduce exposure from long-lived credentials by generating short-lived, automatically expiring credentials per service. Secrets are managed outside of source control, with access controls enforced across all components.

VKS node pools can be provisioned using Kubernetes releases that include FIPS cryptographic module support. The `tkr.reference.name` field in the `TanzuKubernetesCluster` specification selects a FIPS-enabled release so that cryptographic operations within the cluster use validated modules. For Windows node pools using Group Managed Service Accounts, a KDS Root Encryption Key must be provisioned on each domain controller by an Enterprise Administrator before Group Managed Service Account credentials can be derived; this key is managed in Active Directory and forms the cryptographic foundation for credential derivation. Certificate lifecycle management for VKS clusters is provided by Cert Manager, which handles issuance and renewal of certificates across cluster workloads. VKS clusters include automatic certificate rotation enabled by default, with renewal initiated 90 days before certificate expiry; this interval is configurable per cluster using the `renewalDaysBeforeExpiry` parameter in the ClusterClass `certificateRotation` variable. Administrators can also rotate certificates manually using the `kubeadm certs renew` command, and the `kubeadm certs check-expiration` command reports expiration dates and residual time for all control plane certificates, including the API server, etcd client, and controller-manager certificates. The Supervisor Kubernetes control plane supports replacing its default certificate with one signed by an external certificate authority, and `CertificateSigningRequest` resources from both the native `certificates.k8s.io` API group and the `cert-manager.io` API group are supported for certificate provisioning and rotation.

Access to key management functions is controlled through vCenter's privilege system. The Cryptographic Operations privilege set provides fine-grained control over encryption tasks, including separate privileges for individual operations. The `Cryptographer.ManageKeyServers` permission is specifically required to manage key servers. A dedicated No Cryptography Administrator system role includes all Administrator privileges except Cryptographic Operations privileges, enforcing separation of duties between key managers and system administrators. Administrators can clone the No Cryptography Administrator role to create custom roles with only selected Cryptographic Operations privileges, such as allowing encryption but not decryption of virtual machines.

For key availability, VCF supports several protective measures. Administrators should deploy highly available KMS clusters when configuring a standard key provider. KMS servers should not be deployed on the same vSAN datastore or cluster that is being encrypted, to avoid circular dependencies that could make keys inaccessible. VCF validates that the KMS is healthy before encryption can be enabled. On ESX hosts, the `esxcli system security keypersistence enable` command allows encryption key persistence, so that hosts can access encrypted workloads after a reboot without requiring immediate KMS connectivity.

Key lifecycle management extends to individual virtual machines and their disks, where different keys may be in use simultaneously. Administrators should take care when managing the lifecycle of these keys to avoid accidental data loss. VCF Operations also provides Keystore Workflows through the orchestrator that help manage certificates and keys within keystores. The vSAN Security Configuration Guide provides additional guidance on selecting appropriate encryption algorithms and managing KMS integration.



When an external key management system is used, responsibility for the lifecycle of keys held in that system, including generation, rotation, escrow, and destruction, rests with the organization operating the KMS; the platform integrates with the KMS through the standard key provider interface but does not manage the external key lifecycle on the organization's behalf.

### **VMware vDefend (Contributes)**

VMware vDefend contributes to cryptographic key management through certificate lifecycle management within the Security Services Platform (SSP) and key pair controls for the Malware Prevention service.

The Security Services Platform manages certificates that identify client and host machines, establish trust between components, secure inter-component communication channels, and control access levels. SSP provides certificate management capabilities including export, certificate signing request (CSR) generation with configurable cryptographic algorithm selection and key size, and replacement with CA-signed certificates. Certificate initialization is handled by CertWatcher, which loads and validates root CA certificates and keys during SSP startup. Administrators should confirm certificate consistency between backup and restored SSP environments to avoid service disruptions during recovery scenarios.

SSP's cryptographic operations use FIPS 140-2 and FIPS 140-3 validated modules: the VMware BouncyCastle Module (Certificate #4943) and the VMware OpenSSL FIPS provider Object Module (Certificate #4861). These validated modules underpin certificate operations and inter-component communications for SSP and its embedded services, including the NDR Sensor for vDefend. Administrators should be aware that TLS protocol version and cipher suite configuration on SSP Installer and SSP cannot be modified; these are fixed by the platform at deployment time. When deploying SSP on vSAN, administrators should activate the data-at-rest encryption option in the vSAN storage policy to provide encryption for SSP data at rest.

For Distributed Malware Prevention, vDefend manages public-private key pairs that secure communication between the malware analysis infrastructure and the hosts it protects. Administrators can configure whether a new key pair is generated per cluster or shared across all service deployments. Private keys must be stored securely, and the deployment strategy gives organizations control over the scope of any single key exposure.

vDefend TLS Inspection includes certificate management as part of its traffic inspection infrastructure. Administrators can manage certificate revocation within the TLS inspection path, and TLS Inspection policies include a Crypto Enforcement option that allows administrators to define the acceptable range of cryptographic strength for inspected connections.

These capabilities address certificate and key material that vDefend generates, consumes, and rotates as part of its security service infrastructure. Broader enterprise key management functions, including external key management server integration, key hierarchy management, and configurable rotation intervals, are provided by VCF rather than by vDefend.

### **VCF Protection and Recovery (Meets)**

VCF Protection and Recovery (PNR) enforces cryptographic key management requirements across both replication and recovery operations, making key availability a technical prerequisite for protecting and recovering encrypted virtual machines.

For encrypted virtual machine protection, PNR requires that the recovery and protected sites share a common Key Management Server (KMS) or that the KMS clusters at both sites use common encryption keys. The KMS cluster must be registered with the same name on both the protected and recovery sites. PNR maintains a trusted connection between VMware vCenter and the KMS, through which vCenter retrieves encryption keys as needed for encrypted virtual machine disks and other files. If the KMS is not available at the recovery site, encrypted virtual machine recovery fails at the power-on step, making key availability a hard technical requirement for successful failover.

vSphere Replication imposes the same requirements when replicating encrypted virtual machines: the KMS server must be registered with the same name at both source and target sites, and common encryption keys must be present at both locations. Encrypted virtual machines can also be protected using array-based replication protection groups within PNR.

PNR supports the vSphere Native Key Provider as an alternative to external KMS infrastructure. The Native Key Provider enables encryption-related functionality for virtual machines and virtual disks without requiring an external key server, and

must be manually configured on both the local and remote sites. The vSphere Native Key Provider requires vSphere Enterprise+ edition.

For its own communications, PNR enforces certificate-based controls. Server SSL/TLS certificates must be x509v3 server certificates with Extended Key Usage attributes specifying both TLS Web Server Authentication (serverAuth) and client authentication (clientAuth). PNR supports custom certificates signed by a certificate authority as an alternative to self-signed certificates. PNR defaults to a strong TLS protocol version for all connections, with the minimum acceptable version configurable through the `envoy-proxy-config.json` file using the `tls_minimum_protocol_version` parameter.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides several mechanisms that support cryptographic key management for database infrastructure. These center on certificate and private key handling for the DSM Provider VM and managed database clusters.

Certificate storage in DSM follows the Kubernetes TLS Secret model. Custom certificates for both the DSM Provider VM and individual database clusters are stored as Kubernetes TLS Secrets in the appropriate namespaces. For database clusters, the TLS Secret must reside in the same namespace as the cluster and include a Base64-encoded PEM signed certificate chain, a Base64-encoded PEM private key, and a Base64-encoded PEM CA certificate. DSM stores CA certificates as PEM-encoded data in Kubernetes Secrets within the cluster's namespace, making them available for client trust store configuration.

DSM supports configurable key algorithms for certificate generation. Administrators can specify RSA or ECDSA as the private key algorithm, with supported key sizes of 256 and 384 bits. Starting with version 9.1, DSM enforces FIPS 140-3 compliant cryptographic standards for all certificate handling. Only RSA and ECDSA public key algorithms are accepted; Ed25519 is rejected because it is not FIPS 140-3 compliant. Custom trusted certificates and certificates used during database upgrades must comply with FIPS requirements. DSM validates certificates during upload to reject unsupported algorithms or parameters.

Version 9.1 also introduces explicit KeyUsage enforcement at certificate upload. DSM validates KeyUsage requirements during upload and rejects server certificates that lack proper KeyUsage configuration. Server certificates must carry the digitalSignature bit in the KeyUsage extension. DSM accepts certificates that omit the KeyUsage extension entirely, treating the absence as permitting all usages per RFC 5280.

Certificate lifecycle management is supported through integration with Kubernetes cert-manager. Organizations can use cert-manager to automate certificate issuance, renewal, and rotation; cert-manager outputs the result as a Kubernetes TLS Secret in the required format, which DSM then references. Administrators are responsible for renewing Provider VM certificates before expiration and updating the corresponding Kubernetes TLS Secret.

Trust store management is an explicit operational step in DSM deployments. To establish secure connections to database clusters or the DSM Provider VM, clients must add the CA to the operating system trust store. DSM exposes the CA certificate through its API, allowing clients to retrieve it for both the provider VM and individual database clusters.

Password and credential protection in DSM operates at multiple layers. DSM stores local user passwords using PGCrypto symmetric encryption with a dedicated encryption key. PostgreSQL databases on DSM support the pgcrypto extension, which provides cryptographic functions for securing data at the database level. PostgreSQL instances enforce a minimum TLS protocol version for client connections via the `ssl_min_protocol_version` setting. MySQL databases on DSM have FIPS mode active on the client side via the `ssl_fips_mode` setting. SQL Server clusters in DSM require TLS for the DSM operator to manage them and support configurable TLS protocol lists via the `network.tlsProtocols` engine setting.

DSM contributes to this control's requirements by providing structured certificate and key storage, FIPS 140-3 aligned cryptographic enforcement, KeyUsage validation at upload, integration with cert-manager for lifecycle automation, and API-driven CA certificate retrieval. Full satisfaction of the control requires organizational processes to govern the external certificate authority, define certificate rotation schedules, establish key escrow procedures, and address key availability through backup and recovery planning. The underlying Kubernetes platform's configuration for Kubernetes Secret encryption at rest also affects the overall key management posture.

## VMware Avi Load Balancer (Contributes)

VMware Avi Load Balancer provides several mechanisms that support cryptographic key management. The primary mechanism is Hardware Security Module (HSM) integration: the Avi Controller integrates with Thales Luna (formerly SafeNet Luna) HSM for cryptographic key storage and management. Both the Controller and Service Engines support dedicated HSM interfaces for certificate and key operations through HSM Groups configured at Templates > Security > HSM Groups, enabling organizations to store and manage private keys in tamper-resistant hardware rather than in software on the host. The Controller supports both RSA and EC key and certificate creation directly on the HSM.

For cryptographic algorithm governance, Avi's SSL/TLS Profile Templates define the accepted protocol versions and cipher suites applied to virtual services, configured under Templates > Security > SSL/TLS Profile in the Avi Controller UI. Cipher suites can be restricted or prioritized to match organizational key management policies. Key exchange algorithms for SSH management channels can be restricted on both the Controller and Service Engine Groups through the `kex_algorithm_exclude` field in the CLI, allowing exclusion of weak cryptographic key exchange methods such as `diffie-hellman-group14-sha1`.

When FIPS mode is enabled on the Avi Controller cluster, the Controller restricts all cryptographic operations to FIPS-compliant algorithms. The Controller uses the OpenSSL FIPS Provider, which has been validated at FIPS 140-3 Security Level 1 and awarded Certificate #4985 by CMVP, enabled at the Controller cluster level under Administration settings. This mode provides an enforcement boundary for organizations that require alignment with government-recognized cryptographic module standards.

For cloud deployments, Avi integrates with platform-native key management services. In AWS environments, the Controller can list and use AWS KMS keys, including customer-managed keys (CMKs) with configurable key policies and IAM-governed access, supporting SNS/SQS, S3, and EBS encryption. KMS key access permissions can be scoped to a specific key ID or defined directly in the KMS key policy for tighter access control. In Google Cloud Platform deployments, the Controller supports separate KMS keys for Service Engine disk encryption, GCS bucket encryption, Service Engine image encryption, and GCS object encryption through Cloud KMS keys managed with appropriately scoped IAM roles.

For Kubernetes deployments, the Avi Kubernetes Operator (AKO) supports configuring both RSA and ECC signed certificates simultaneously on a single virtual service through the `alt.crt` and `alt.key` fields in ingress configurations and TLS default secrets. AKO also supports PKIProfile custom resources for certificate-based authentication and encryption with backend services. These capabilities extend Avi's key and certificate management to containerized workloads.

Data-at-rest encryption key management for the Avi Controller and Service Engine virtual machine disks is provided by VCF through VM Encryption and vSAN Encryption with Native Key Provider or external Key Management Server integration.

## Control CR 1.9(e)

> Sub-item (e) of CR 1.9 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 1.9 requirement in the standard.

**SCF Controls:** CRY-09

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides cryptographic key management controls that protect the confidentiality, integrity, and availability of keys through multiple integrated mechanisms spanning key storage, access control, lifecycle management, and availability safeguards.

At the core of VCF's key management architecture is the CryptoManager managed object hierarchy in the vSphere API. The CryptoManager and its child object CryptoManagerKmp provide programmatic interfaces for managing cryptographic keys, integrating with external key management servers, and performing cryptographic operations. The API exposes methods for adding, listing, and removing keys, giving administrators and automation tools full control over key lifecycle operations.

VCF supports two models for key storage. Organizations can integrate with external Key Management Servers that support the Key Management Interoperability Protocol (KMIP) 1.1 standard, allowing interoperability with a wide range of third-party KMS products. Alternatively, the vSphere Native Key Provider (NKP) is included in all vSphere editions and does not require an external key server. NKP can be added, updated, and removed through the Providers interface in the vSphere API, and its configuration must be exported at least once before it can be used. For organizations that choose an external KMS, administrators establish a trust relationship by uploading a certificate and key file to the KMS vendor, and then select a KMS cluster during encryption configuration. The KMS must appear in the VMware Compatibility Guide for Key Management Servers and be KMIP 1.1 compliant; that guide provides the authoritative list of tested and supported third-party KMS products. Some KMS vendors allow further isolation of encryption keys by specifying dedicated credentials during configuration. Administrators can also configure separate key providers for different users or departments, providing more granular access control over encryption keys by organizational unit. VCF establishes a cryptographic domain of trust between key providers, VMware vCenter, and VMware ESX hosts so that keys are accessible only to authorized components.

For organizations consuming VCF services through the multi-tenant consumption layer, VCF Encryption Management extends key management to organization administrators. This feature allows organization administrators to use encryption keys from their own key providers for encryption of virtual machines and disks within their VCF environment, enabling tenant-controlled key custody independent of the platform provider's key management infrastructure.

VCF uses a layered key hierarchy for encryption. Key Encryption Keys (KEK) wrap Disk Encryption Keys (DEK), with optional wrapping keys protecting KEKs. Disk encryption keys use XTS-AES-256. This layered approach means that rotating or revoking a higher-level key protects all keys beneath it without requiring re-encryption of the underlying data. ESX hosts use the KEK to encrypt their internal keys and store only the encrypted internal keys on disk, not the KEK itself.

For virtual machines, the vSphere API supports two recrypt operations: shallow recrypt replaces only the KEK and can be performed while the virtual machine is powered on, while deep recrypt replaces both KEKs and DEKs but requires the virtual machine to be powered off. For vSAN data-at-rest encryption, administrators can generate new KEKs or new DEKs with the option to re-encrypt all data on the storage using new keys. vSAN encryption supports configurable rekey intervals that can be set to default or custom values. vSAN data-in-transit encryption also supports a configurable rekey interval, defaulting to 1,440 minutes (24 hours) with a range from 30 minutes to 7 days. An Allow Reduced Redundancy option is available for clusters with limited resources during key rotation operations.

Workloads running in VMware Kubernetes Service (VKS) clusters on Supervisor benefit from an additional layer of key management integrated with the same external KMS infrastructure. VKS uses the Kubernetes KMS provider plugin interface to protect data at rest in etcd. The EncryptionConfiguration resource configures a provider of type kms in the providers array on the API server; the plugin communicates with an external Key Management Service via a Unix domain socket endpoint, keeping all key material outside the cluster. The current stable implementation is KMS v2 (available from Kubernetes v1.29),

which uses an envelope encryption scheme where each resource is encrypted under a Data Encryption Key (DEK) that is itself wrapped by a Key Encryption Key (KEK) held in the remote KMS. KMS v2 employs a key derivation function to generate single-use DEKs from a secret seed combined with random data, reducing the frequency of external KMS calls while maintaining per-encryption key uniqueness. KMS plugins are required to verify that a `key_id` presented for decryption was encrypted by them, providing a chain-of-custody check that restricts decryption to the originating plugin. The KMS provider must be listed first in the `EncryptionConfiguration` file to take precedence over any local encryption providers. VKS administrators manage sensitive material such as passwords, tokens, and SSH keys through Kubernetes Secret objects, which are protected at rest by the cluster's encryption-at-rest configuration. For environments requiring centralized secrets management beyond what Kubernetes natively provides, Supervisor and VKS clusters support integration with external vault systems. In CI/CD architectures built on Supervisor through integrations such as Harness or Argo CD, HashiCorp Vault provides dynamic secret generation, access policies, and a centralized store for API keys, certificates, and credentials; dynamic secrets reduce exposure from long-lived credentials by generating short-lived, automatically expiring credentials per service. Secrets are managed outside of source control, with access controls enforced across all components.

VKS node pools can be provisioned using Kubernetes releases that include FIPS cryptographic module support. The `tkr.reference.name` field in the `TanzuKubernetesCluster` specification selects a FIPS-enabled release so that cryptographic operations within the cluster use validated modules. For Windows node pools using Group Managed Service Accounts, a KDS Root Encryption Key must be provisioned on each domain controller by an Enterprise Administrator before Group Managed Service Account credentials can be derived; this key is managed in Active Directory and forms the cryptographic foundation for credential derivation. Certificate lifecycle management for VKS clusters is provided by Cert Manager, which handles issuance and renewal of certificates across cluster workloads. VKS clusters include automatic certificate rotation enabled by default, with renewal initiated 90 days before certificate expiry; this interval is configurable per cluster using the `renewalDaysBeforeExpiry` parameter in the ClusterClass `certificateRotation` variable. Administrators can also rotate certificates manually using the `kubeadm certs renew` command, and the `kubeadm certs check-expiration` command reports expiration dates and residual time for all control plane certificates, including the API server, etcd client, and controller-manager certificates. The Supervisor Kubernetes control plane supports replacing its default certificate with one signed by an external certificate authority, and `CertificateSigningRequest` resources from both the native `certificates.k8s.io` API group and the `cert-manager.io` API group are supported for certificate provisioning and rotation.

Access to key management functions is controlled through vCenter's privilege system. The Cryptographic Operations privilege set provides fine-grained control over encryption tasks, including separate privileges for individual operations. The `Cryptographer.ManageKeyServers` permission is specifically required to manage key servers. A dedicated No Cryptography Administrator system role includes all Administrator privileges except Cryptographic Operations privileges, enforcing separation of duties between key managers and system administrators. Administrators can clone the No Cryptography Administrator role to create custom roles with only selected Cryptographic Operations privileges, such as allowing encryption but not decryption of virtual machines.

For key availability, VCF supports several protective measures. Administrators should deploy highly available KMS clusters when configuring a standard key provider. KMS servers should not be deployed on the same vSAN datastore or cluster that is being encrypted, to avoid circular dependencies that could make keys inaccessible. VCF validates that the KMS is healthy before encryption can be enabled. On ESX hosts, the `esxcli system security keypersistence enable` command allows encryption key persistence, so that hosts can access encrypted workloads after a reboot without requiring immediate KMS connectivity.

Key lifecycle management extends to individual virtual machines and their disks, where different keys may be in use simultaneously. Administrators should take care when managing the lifecycle of these keys to avoid accidental data loss. VCF Operations also provides Keystore Workflows through the orchestrator that help manage certificates and keys within keystores. The vSAN Security Configuration Guide provides additional guidance on selecting appropriate encryption algorithms and managing KMS integration.

When an external key management system is used, responsibility for the lifecycle of keys held in that system, including generation, rotation, escrow, and destruction, rests with the organization operating the KMS; the platform integrates with the KMS through the standard key provider interface but does not manage the external key lifecycle on the organization's behalf.

### **VMware vDefend (Contributes)**

VMware vDefend contributes to cryptographic key management through certificate lifecycle management within the Security Services Platform (SSP) and key pair controls for the Malware Prevention service.

The Security Services Platform manages certificates that identify client and host machines, establish trust between components, secure inter-component communication channels, and control access levels. SSP provides certificate management capabilities including export, certificate signing request (CSR) generation with configurable cryptographic algorithm selection and key size, and replacement with CA-signed certificates. Certificate initialization is handled by CertWatcher, which loads and validates root CA certificates and keys during SSP startup. Administrators should confirm certificate consistency between backup and restored SSP environments to avoid service disruptions during recovery scenarios.

SSP's cryptographic operations use FIPS 140-2 and FIPS 140-3 validated modules: the VMware BouncyCastle Module (Certificate #4943) and the VMware OpenSSL FIPS provider Object Module (Certificate #4861). These validated modules underpin certificate operations and inter-component communications for SSP and its embedded services, including the NDR Sensor for vDefend. Administrators should be aware that TLS protocol version and cipher suite configuration on SSP Installer and SSP cannot be modified; these are fixed by the platform at deployment time. When deploying SSP on vSAN, administrators should activate the data-at-rest encryption option in the vSAN storage policy to provide encryption for SSP data at rest.

For Distributed Malware Prevention, vDefend manages public-private key pairs that secure communication between the malware analysis infrastructure and the hosts it protects. Administrators can configure whether a new key pair is generated per cluster or shared across all service deployments. Private keys must be stored securely, and the deployment strategy gives organizations control over the scope of any single key exposure.

vDefend TLS Inspection includes certificate management as part of its traffic inspection infrastructure. Administrators can manage certificate revocation within the TLS inspection path, and TLS Inspection policies include a Crypto Enforcement option that allows administrators to define the acceptable range of cryptographic strength for inspected connections.

These capabilities address certificate and key material that vDefend generates, consumes, and rotates as part of its security service infrastructure. Broader enterprise key management functions, including external key management server integration, key hierarchy management, and configurable rotation intervals, are provided by VCF rather than by vDefend.

### **VCF Protection and Recovery (Meets)**

VCF Protection and Recovery (PNR) enforces cryptographic key management requirements across both replication and recovery operations, making key availability a technical prerequisite for protecting and recovering encrypted virtual machines.

For encrypted virtual machine protection, PNR requires that the recovery and protected sites share a common Key Management Server (KMS) or that the KMS clusters at both sites use common encryption keys. The KMS cluster must be registered with the same name on both the protected and recovery sites. PNR maintains a trusted connection between VMware vCenter and the KMS, through which vCenter retrieves encryption keys as needed for encrypted virtual machine disks and other files. If the KMS is not available at the recovery site, encrypted virtual machine recovery fails at the power-on step, making key availability a hard technical requirement for successful failover.

vSphere Replication imposes the same requirements when replicating encrypted virtual machines: the KMS server must be registered with the same name at both source and target sites, and common encryption keys must be present at both locations. Encrypted virtual machines can also be protected using array-based replication protection groups within PNR.

PNR supports the vSphere Native Key Provider as an alternative to external KMS infrastructure. The Native Key Provider enables encryption-related functionality for virtual machines and virtual disks without requiring an external key server, and



must be manually configured on both the local and remote sites. The vSphere Native Key Provider requires vSphere Enterprise+ edition.

For its own communications, PNR enforces certificate-based controls. Server SSL/TLS certificates must be x509v3 server certificates with Extended Key Usage attributes specifying both TLS Web Server Authentication (serverAuth) and client authentication (clientAuth). PNR supports custom certificates signed by a certificate authority as an alternative to self-signed certificates. PNR defaults to a strong TLS protocol version for all connections, with the minimum acceptable version configurable through the `envoy-proxy-config.json` file using the `tls_minimum_protocol_version` parameter.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides several mechanisms that support cryptographic key management for database infrastructure. These center on certificate and private key handling for the DSM Provider VM and managed database clusters.

Certificate storage in DSM follows the Kubernetes TLS Secret model. Custom certificates for both the DSM Provider VM and individual database clusters are stored as Kubernetes TLS Secrets in the appropriate namespaces. For database clusters, the TLS Secret must reside in the same namespace as the cluster and include a Base64-encoded PEM signed certificate chain, a Base64-encoded PEM private key, and a Base64-encoded PEM CA certificate. DSM stores CA certificates as PEM-encoded data in Kubernetes Secrets within the cluster's namespace, making them available for client trust store configuration.

DSM supports configurable key algorithms for certificate generation. Administrators can specify RSA or ECDSA as the private key algorithm, with supported key sizes of 256 and 384 bits. Starting with version 9.1, DSM enforces FIPS 140-3 compliant cryptographic standards for all certificate handling. Only RSA and ECDSA public key algorithms are accepted; Ed25519 is rejected because it is not FIPS 140-3 compliant. Custom trusted certificates and certificates used during database upgrades must comply with FIPS requirements. DSM validates certificates during upload to reject unsupported algorithms or parameters.

Version 9.1 also introduces explicit KeyUsage enforcement at certificate upload. DSM validates KeyUsage requirements during upload and rejects server certificates that lack proper KeyUsage configuration. Server certificates must carry the digitalSignature bit in the KeyUsage extension. DSM accepts certificates that omit the KeyUsage extension entirely, treating the absence as permitting all usages per RFC 5280.

Certificate lifecycle management is supported through integration with Kubernetes cert-manager. Organizations can use cert-manager to automate certificate issuance, renewal, and rotation; cert-manager outputs the result as a Kubernetes TLS Secret in the required format, which DSM then references. Administrators are responsible for renewing Provider VM certificates before expiration and updating the corresponding Kubernetes TLS Secret.

Trust store management is an explicit operational step in DSM deployments. To establish secure connections to database clusters or the DSM Provider VM, clients must add the CA to the operating system trust store. DSM exposes the CA certificate through its API, allowing clients to retrieve it for both the provider VM and individual database clusters.

Password and credential protection in DSM operates at multiple layers. DSM stores local user passwords using PGCrypto symmetric encryption with a dedicated encryption key. PostgreSQL databases on DSM support the pgcrypto extension, which provides cryptographic functions for securing data at the database level. PostgreSQL instances enforce a minimum TLS protocol version for client connections via the `ssl_min_protocol_version` setting. MySQL databases on DSM have FIPS mode active on the client side via the `ssl_fips_mode` setting. SQL Server clusters in DSM require TLS for the DSM operator to manage them and support configurable TLS protocol lists via the `network.tlsProtocols` engine setting.

DSM contributes to this control's requirements by providing structured certificate and key storage, FIPS 140-3 aligned cryptographic enforcement, KeyUsage validation at upload, integration with cert-manager for lifecycle automation, and API-driven CA certificate retrieval. Full satisfaction of the control requires organizational processes to govern the external certificate authority, define certificate rotation schedules, establish key escrow procedures, and address key availability through backup and recovery planning. The underlying Kubernetes platform's configuration for Kubernetes Secret encryption at rest also affects the overall key management posture.



## VMware Avi Load Balancer (Contributes)

VMware Avi Load Balancer provides several mechanisms that support cryptographic key management. The primary mechanism is Hardware Security Module (HSM) integration: the Avi Controller integrates with Thales Luna (formerly SafeNet Luna) HSM for cryptographic key storage and management. Both the Controller and Service Engines support dedicated HSM interfaces for certificate and key operations through HSM Groups configured at Templates > Security > HSM Groups, enabling organizations to store and manage private keys in tamper-resistant hardware rather than in software on the host. The Controller supports both RSA and EC key and certificate creation directly on the HSM.

For cryptographic algorithm governance, Avi's SSL/TLS Profile Templates define the accepted protocol versions and cipher suites applied to virtual services, configured under Templates > Security > SSL/TLS Profile in the Avi Controller UI. Cipher suites can be restricted or prioritized to match organizational key management policies. Key exchange algorithms for SSH management channels can be restricted on both the Controller and Service Engine Groups through the `kex_algorithm_exclude` field in the CLI, allowing exclusion of weak cryptographic key exchange methods such as `diffie-hellman-group14-sha1`.

When FIPS mode is enabled on the Avi Controller cluster, the Controller restricts all cryptographic operations to FIPS-compliant algorithms. The Controller uses the OpenSSL FIPS Provider, which has been validated at FIPS 140-3 Security Level 1 and awarded Certificate #4985 by CMVP, enabled at the Controller cluster level under Administration settings. This mode provides an enforcement boundary for organizations that require alignment with government-recognized cryptographic module standards.

For cloud deployments, Avi integrates with platform-native key management services. In AWS environments, the Controller can list and use AWS KMS keys, including customer-managed keys (CMKs) with configurable key policies and IAM-governed access, supporting SNS/SQS, S3, and EBS encryption. KMS key access permissions can be scoped to a specific key ID or defined directly in the KMS key policy for tighter access control. In Google Cloud Platform deployments, the Controller supports separate KMS keys for Service Engine disk encryption, GCS bucket encryption, Service Engine image encryption, and GCS object encryption through Cloud KMS keys managed with appropriately scoped IAM roles.

For Kubernetes deployments, the Avi Kubernetes Operator (AKO) supports configuring both RSA and ECC signed certificates simultaneously on a single virtual service through the `alt.crt` and `alt.key` fields in ingress configurations and TLS default secrets. AKO also supports PKIProfile custom resources for certificate-based authentication and encryption with backend services. These capabilities extend Avi's key and certificate management to containerized workloads.

Data-at-rest encryption key management for the Avi Controller and Service Engine virtual machine disks is provided by VCF through VM Encryption and vSAN Encryption with Native Key Provider or external Key Management Server integration.

## Control CR 1.9(f)

> Sub-item (f) of CR 1.9 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 1.9 requirement in the standard.

**SCF Controls:** CRY-09

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides cryptographic key management controls that protect the confidentiality, integrity, and availability of keys through multiple integrated mechanisms spanning key storage, access control, lifecycle management, and availability safeguards.

At the core of VCF's key management architecture is the CryptoManager managed object hierarchy in the vSphere API. The CryptoManager and its child object CryptoManagerKmp provide programmatic interfaces for managing cryptographic keys, integrating with external key management servers, and performing cryptographic operations. The API exposes methods for adding, listing, and removing keys, giving administrators and automation tools full control over key lifecycle operations.

VCF supports two models for key storage. Organizations can integrate with external Key Management Servers that support the Key Management Interoperability Protocol (KMIP) 1.1 standard, allowing interoperability with a wide range of third-party KMS products. Alternatively, the vSphere Native Key Provider (NKP) is included in all vSphere editions and does not require an external key server. NKP can be added, updated, and removed through the Providers interface in the vSphere API, and its configuration must be exported at least once before it can be used. For organizations that choose an external KMS, administrators establish a trust relationship by uploading a certificate and key file to the KMS vendor, and then select a KMS cluster during encryption configuration. The KMS must appear in the VMware Compatibility Guide for Key Management Servers and be KMIP 1.1 compliant; that guide provides the authoritative list of tested and supported third-party KMS products. Some KMS vendors allow further isolation of encryption keys by specifying dedicated credentials during configuration. Administrators can also configure separate key providers for different users or departments, providing more granular access control over encryption keys by organizational unit. VCF establishes a cryptographic domain of trust between key providers, VMware vCenter, and VMware ESX hosts so that keys are accessible only to authorized components.

For organizations consuming VCF services through the multi-tenant consumption layer, VCF Encryption Management extends key management to organization administrators. This feature allows organization administrators to use encryption keys from their own key providers for encryption of virtual machines and disks within their VCF environment, enabling tenant-controlled key custody independent of the platform provider's key management infrastructure.

VCF uses a layered key hierarchy for encryption. Key Encryption Keys (KEK) wrap Disk Encryption Keys (DEK), with optional wrapping keys protecting KEKs. Disk encryption keys use XTS-AES-256. This layered approach means that rotating or revoking a higher-level key protects all keys beneath it without requiring re-encryption of the underlying data. ESX hosts use the KEK to encrypt their internal keys and store only the encrypted internal keys on disk, not the KEK itself.

For virtual machines, the vSphere API supports two recrypt operations: shallow recrypt replaces only the KEK and can be performed while the virtual machine is powered on, while deep recrypt replaces both KEKs and DEKs but requires the virtual machine to be powered off. For vSAN data-at-rest encryption, administrators can generate new KEKs or new DEKs with the option to re-encrypt all data on the storage using new keys. vSAN encryption supports configurable rekey intervals that can be set to default or custom values. vSAN data-in-transit encryption also supports a configurable rekey interval, defaulting to 1,440 minutes (24 hours) with a range from 30 minutes to 7 days. An Allow Reduced Redundancy option is available for clusters with limited resources during key rotation operations.

Workloads running in VMware Kubernetes Service (VKS) clusters on Supervisor benefit from an additional layer of key management integrated with the same external KMS infrastructure. VKS uses the Kubernetes KMS provider plugin interface to protect data at rest in etcd. The EncryptionConfiguration resource configures a provider of type kms in the providers array on the API server; the plugin communicates with an external Key Management Service via a Unix domain socket endpoint, keeping all key material outside the cluster. The current stable implementation is KMS v2 (available from Kubernetes v1.29),

which uses an envelope encryption scheme where each resource is encrypted under a Data Encryption Key (DEK) that is itself wrapped by a Key Encryption Key (KEK) held in the remote KMS. KMS v2 employs a key derivation function to generate single-use DEKs from a secret seed combined with random data, reducing the frequency of external KMS calls while maintaining per-encryption key uniqueness. KMS plugins are required to verify that a `key_id` presented for decryption was encrypted by them, providing a chain-of-custody check that restricts decryption to the originating plugin. The KMS provider must be listed first in the `EncryptionConfiguration` file to take precedence over any local encryption providers. VKS administrators manage sensitive material such as passwords, tokens, and SSH keys through Kubernetes Secret objects, which are protected at rest by the cluster's encryption-at-rest configuration. For environments requiring centralized secrets management beyond what Kubernetes natively provides, Supervisor and VKS clusters support integration with external vault systems. In CI/CD architectures built on Supervisor through integrations such as Harness or Argo CD, HashiCorp Vault provides dynamic secret generation, access policies, and a centralized store for API keys, certificates, and credentials; dynamic secrets reduce exposure from long-lived credentials by generating short-lived, automatically expiring credentials per service. Secrets are managed outside of source control, with access controls enforced across all components.

VKS node pools can be provisioned using Kubernetes releases that include FIPS cryptographic module support. The `tkr.reference.name` field in the `TanzuKubernetesCluster` specification selects a FIPS-enabled release so that cryptographic operations within the cluster use validated modules. For Windows node pools using Group Managed Service Accounts, a KDS Root Encryption Key must be provisioned on each domain controller by an Enterprise Administrator before Group Managed Service Account credentials can be derived; this key is managed in Active Directory and forms the cryptographic foundation for credential derivation. Certificate lifecycle management for VKS clusters is provided by Cert Manager, which handles issuance and renewal of certificates across cluster workloads. VKS clusters include automatic certificate rotation enabled by default, with renewal initiated 90 days before certificate expiry; this interval is configurable per cluster using the `renewalDaysBeforeExpiry` parameter in the ClusterClass `certificateRotation` variable. Administrators can also rotate certificates manually using the `kubeadm certs renew` command, and the `kubeadm certs check-expiration` command reports expiration dates and residual time for all control plane certificates, including the API server, etcd client, and controller-manager certificates. The Supervisor Kubernetes control plane supports replacing its default certificate with one signed by an external certificate authority, and `CertificateSigningRequest` resources from both the native `certificates.k8s.io` API group and the `cert-manager.io` API group are supported for certificate provisioning and rotation.

Access to key management functions is controlled through vCenter's privilege system. The Cryptographic Operations privilege set provides fine-grained control over encryption tasks, including separate privileges for individual operations. The `Cryptographer.ManageKeyServers` permission is specifically required to manage key servers. A dedicated No Cryptography Administrator system role includes all Administrator privileges except Cryptographic Operations privileges, enforcing separation of duties between key managers and system administrators. Administrators can clone the No Cryptography Administrator role to create custom roles with only selected Cryptographic Operations privileges, such as allowing encryption but not decryption of virtual machines.

For key availability, VCF supports several protective measures. Administrators should deploy highly available KMS clusters when configuring a standard key provider. KMS servers should not be deployed on the same vSAN datastore or cluster that is being encrypted, to avoid circular dependencies that could make keys inaccessible. VCF validates that the KMS is healthy before encryption can be enabled. On ESX hosts, the `esxcli system security keypersistence enable` command allows encryption key persistence, so that hosts can access encrypted workloads after a reboot without requiring immediate KMS connectivity.

Key lifecycle management extends to individual virtual machines and their disks, where different keys may be in use simultaneously. Administrators should take care when managing the lifecycle of these keys to avoid accidental data loss. VCF Operations also provides Keystore Workflows through the orchestrator that help manage certificates and keys within keystores. The vSAN Security Configuration Guide provides additional guidance on selecting appropriate encryption algorithms and managing KMS integration.

When an external key management system is used, responsibility for the lifecycle of keys held in that system, including generation, rotation, escrow, and destruction, rests with the organization operating the KMS; the platform integrates with the KMS through the standard key provider interface but does not manage the external key lifecycle on the organization's behalf.

### **VMware vDefend (Contributes)**

VMware vDefend contributes to cryptographic key management through certificate lifecycle management within the Security Services Platform (SSP) and key pair controls for the Malware Prevention service.

The Security Services Platform manages certificates that identify client and host machines, establish trust between components, secure inter-component communication channels, and control access levels. SSP provides certificate management capabilities including export, certificate signing request (CSR) generation with configurable cryptographic algorithm selection and key size, and replacement with CA-signed certificates. Certificate initialization is handled by CertWatcher, which loads and validates root CA certificates and keys during SSP startup. Administrators should confirm certificate consistency between backup and restored SSP environments to avoid service disruptions during recovery scenarios.

SSP's cryptographic operations use FIPS 140-2 and FIPS 140-3 validated modules: the VMware BouncyCastle Module (Certificate #4943) and the VMware OpenSSL FIPS provider Object Module (Certificate #4861). These validated modules underpin certificate operations and inter-component communications for SSP and its embedded services, including the NDR Sensor for vDefend. Administrators should be aware that TLS protocol version and cipher suite configuration on SSP Installer and SSP cannot be modified; these are fixed by the platform at deployment time. When deploying SSP on vSAN, administrators should activate the data-at-rest encryption option in the vSAN storage policy to provide encryption for SSP data at rest.

For Distributed Malware Prevention, vDefend manages public-private key pairs that secure communication between the malware analysis infrastructure and the hosts it protects. Administrators can configure whether a new key pair is generated per cluster or shared across all service deployments. Private keys must be stored securely, and the deployment strategy gives organizations control over the scope of any single key exposure.

vDefend TLS Inspection includes certificate management as part of its traffic inspection infrastructure. Administrators can manage certificate revocation within the TLS inspection path, and TLS Inspection policies include a Crypto Enforcement option that allows administrators to define the acceptable range of cryptographic strength for inspected connections.

These capabilities address certificate and key material that vDefend generates, consumes, and rotates as part of its security service infrastructure. Broader enterprise key management functions, including external key management server integration, key hierarchy management, and configurable rotation intervals, are provided by VCF rather than by vDefend.

### **VCF Protection and Recovery (Meets)**

VCF Protection and Recovery (PNR) enforces cryptographic key management requirements across both replication and recovery operations, making key availability a technical prerequisite for protecting and recovering encrypted virtual machines.

For encrypted virtual machine protection, PNR requires that the recovery and protected sites share a common Key Management Server (KMS) or that the KMS clusters at both sites use common encryption keys. The KMS cluster must be registered with the same name on both the protected and recovery sites. PNR maintains a trusted connection between VMware vCenter and the KMS, through which vCenter retrieves encryption keys as needed for encrypted virtual machine disks and other files. If the KMS is not available at the recovery site, encrypted virtual machine recovery fails at the power-on step, making key availability a hard technical requirement for successful failover.

vSphere Replication imposes the same requirements when replicating encrypted virtual machines: the KMS server must be registered with the same name at both source and target sites, and common encryption keys must be present at both locations. Encrypted virtual machines can also be protected using array-based replication protection groups within PNR.

PNR supports the vSphere Native Key Provider as an alternative to external KMS infrastructure. The Native Key Provider enables encryption-related functionality for virtual machines and virtual disks without requiring an external key server, and

must be manually configured on both the local and remote sites. The vSphere Native Key Provider requires vSphere Enterprise+ edition.

For its own communications, PNR enforces certificate-based controls. Server SSL/TLS certificates must be x509v3 server certificates with Extended Key Usage attributes specifying both TLS Web Server Authentication (serverAuth) and client authentication (clientAuth). PNR supports custom certificates signed by a certificate authority as an alternative to self-signed certificates. PNR defaults to a strong TLS protocol version for all connections, with the minimum acceptable version configurable through the `envoy-proxy-config.json` file using the `tls_minimum_protocol_version` parameter.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides several mechanisms that support cryptographic key management for database infrastructure. These center on certificate and private key handling for the DSM Provider VM and managed database clusters.

Certificate storage in DSM follows the Kubernetes TLS Secret model. Custom certificates for both the DSM Provider VM and individual database clusters are stored as Kubernetes TLS Secrets in the appropriate namespaces. For database clusters, the TLS Secret must reside in the same namespace as the cluster and include a Base64-encoded PEM signed certificate chain, a Base64-encoded PEM private key, and a Base64-encoded PEM CA certificate. DSM stores CA certificates as PEM-encoded data in Kubernetes Secrets within the cluster's namespace, making them available for client trust store configuration.

DSM supports configurable key algorithms for certificate generation. Administrators can specify RSA or ECDSA as the private key algorithm, with supported key sizes of 256 and 384 bits. Starting with version 9.1, DSM enforces FIPS 140-3 compliant cryptographic standards for all certificate handling. Only RSA and ECDSA public key algorithms are accepted; Ed25519 is rejected because it is not FIPS 140-3 compliant. Custom trusted certificates and certificates used during database upgrades must comply with FIPS requirements. DSM validates certificates during upload to reject unsupported algorithms or parameters.

Version 9.1 also introduces explicit KeyUsage enforcement at certificate upload. DSM validates KeyUsage requirements during upload and rejects server certificates that lack proper KeyUsage configuration. Server certificates must carry the digitalSignature bit in the KeyUsage extension. DSM accepts certificates that omit the KeyUsage extension entirely, treating the absence as permitting all usages per RFC 5280.

Certificate lifecycle management is supported through integration with Kubernetes cert-manager. Organizations can use cert-manager to automate certificate issuance, renewal, and rotation; cert-manager outputs the result as a Kubernetes TLS Secret in the required format, which DSM then references. Administrators are responsible for renewing Provider VM certificates before expiration and updating the corresponding Kubernetes TLS Secret.

Trust store management is an explicit operational step in DSM deployments. To establish secure connections to database clusters or the DSM Provider VM, clients must add the CA to the operating system trust store. DSM exposes the CA certificate through its API, allowing clients to retrieve it for both the provider VM and individual database clusters.

Password and credential protection in DSM operates at multiple layers. DSM stores local user passwords using PGCrypto symmetric encryption with a dedicated encryption key. PostgreSQL databases on DSM support the pgcrypto extension, which provides cryptographic functions for securing data at the database level. PostgreSQL instances enforce a minimum TLS protocol version for client connections via the `ssl_min_protocol_version` setting. MySQL databases on DSM have FIPS mode active on the client side via the `ssl_fips_mode` setting. SQL Server clusters in DSM require TLS for the DSM operator to manage them and support configurable TLS protocol lists via the `network.tlsProtocols` engine setting.

DSM contributes to this control's requirements by providing structured certificate and key storage, FIPS 140-3 aligned cryptographic enforcement, KeyUsage validation at upload, integration with cert-manager for lifecycle automation, and API-driven CA certificate retrieval. Full satisfaction of the control requires organizational processes to govern the external certificate authority, define certificate rotation schedules, establish key escrow procedures, and address key availability through backup and recovery planning. The underlying Kubernetes platform's configuration for Kubernetes Secret encryption at rest also affects the overall key management posture.

## VMware Avi Load Balancer (Contributes)

VMware Avi Load Balancer provides several mechanisms that support cryptographic key management. The primary mechanism is Hardware Security Module (HSM) integration: the Avi Controller integrates with Thales Luna (formerly SafeNet Luna) HSM for cryptographic key storage and management. Both the Controller and Service Engines support dedicated HSM interfaces for certificate and key operations through HSM Groups configured at Templates > Security > HSM Groups, enabling organizations to store and manage private keys in tamper-resistant hardware rather than in software on the host. The Controller supports both RSA and EC key and certificate creation directly on the HSM.

For cryptographic algorithm governance, Avi's SSL/TLS Profile Templates define the accepted protocol versions and cipher suites applied to virtual services, configured under Templates > Security > SSL/TLS Profile in the Avi Controller UI. Cipher suites can be restricted or prioritized to match organizational key management policies. Key exchange algorithms for SSH management channels can be restricted on both the Controller and Service Engine Groups through the `kex_algorithm_exclude` field in the CLI, allowing exclusion of weak cryptographic key exchange methods such as `diffie-hellman-group14-sha1`.

When FIPS mode is enabled on the Avi Controller cluster, the Controller restricts all cryptographic operations to FIPS-compliant algorithms. The Controller uses the OpenSSL FIPS Provider, which has been validated at FIPS 140-3 Security Level 1 and awarded Certificate #4985 by CMVP, enabled at the Controller cluster level under Administration settings. This mode provides an enforcement boundary for organizations that require alignment with government-recognized cryptographic module standards.

For cloud deployments, Avi integrates with platform-native key management services. In AWS environments, the Controller can list and use AWS KMS keys, including customer-managed keys (CMKs) with configurable key policies and IAM-governed access, supporting SNS/SQS, S3, and EBS encryption. KMS key access permissions can be scoped to a specific key ID or defined directly in the KMS key policy for tighter access control. In Google Cloud Platform deployments, the Controller supports separate KMS keys for Service Engine disk encryption, GCS bucket encryption, Service Engine image encryption, and GCS object encryption through Cloud KMS keys managed with appropriately scoped IAM roles.

For Kubernetes deployments, the Avi Kubernetes Operator (AKO) supports configuring both RSA and ECC signed certificates simultaneously on a single virtual service through the `alt.crt` and `alt.key` fields in ingress configurations and TLS default secrets. AKO also supports PKIProfile custom resources for certificate-based authentication and encryption with backend services. These capabilities extend Avi's key and certificate management to containerized workloads.

Data-at-rest encryption key management for the Avi Controller and Service Engine virtual machine disks is provided by VCF through VM Encryption and vSAN Encryption with Native Key Provider or external Key Management Server integration.



## Control CR 1.10

> Authenticator feedback: Obscure feedback of authentication information (e.g., asterisks) during entry.

**SCF Controls:** IAC-11

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF obscures authentication feedback across all management interfaces and programmatic endpoints through standard credential masking, a token-based authentication architecture that minimizes direct credential exposure, and encryption of all authentication exchanges in transit.

All VCF web-based management interfaces, including the VCF Operations UI, VMware vCenter Client, and NSX Manager console, render password input fields using standard masked controls that replace typed characters with dot or asterisk symbols. This applies to every login screen and credential entry dialog across the platform. Console-based access to VMware ESX hosts similarly masks password input at the Direct Console User Interface (DCUI) and SSH login prompts through standard operating system conventions that suppress terminal echo during credential entry. Command-line tools that accept passwords follow the same echo suppression behavior.

Beyond input masking, VCF's authentication architecture reduces the frequency and scope of password exposure. Identity provider federation uses token-based authentication and minimizes credential interception risk by exchanging security tokens rather than transmitting usernames and passwords directly between components. When configured with an external identity provider, users authenticate at the identity provider's login page, and VCF receives a security assertion or token rather than the raw credentials. Security Assertion Markup Language (SAML) is used by vCenter for exchanging authentication and authorization data between parties, and vCenter LoginByToken uses SAML 2.0 assertion tokens for authentication. The VCF Identity Broker integrates with selected identity providers to enable user authentication for VCF SSO, further reducing the number of times users must enter credentials across the platform.

OAuth 2.0 support allows information exchanges among VCF web services without exposing sensitive data such as usernames and passwords. The platform documentation recommends that the Password grant type be avoided whenever possible, especially for third-party application authentication, in favor of token-based flows that do not require credentials to traverse the network in cleartext form. API clients can configure token-based authentication to communicate with the server, and the LoginByToken method authenticates clients using holder-of-key security tokens that embed security artifacts for authentication without exposing credentials. Both approaches avoid password transmission after initial token issuance.

Session identifiers are used to maintain authenticated state after initial login, so credentials are not re-transmitted with each request. Valid session identifiers establish trust with API endpoints without requiring repeated credential entry. TLS encryption protects all authentication exchanges in transit, so even the masked credential data transmitted during initial login is encrypted on the wire.

At the workload consumption layer, VMware Kubernetes Service (VKS) and vSphere Supervisor extend these protections into the Kubernetes platform. The Kubernetes API server uses JWT and OIDC-based authentication flows in which the actual credential exchange occurs at the identity provider, such as VCF SSO or an external IdP, and the API server receives and validates signed tokens rather than handling plaintext passwords directly. JWTAuthenticator configuration specifies a unique issuer URL per authenticator and applies claim validation rules to incoming tokens; at most one JWT authenticator cryptographically validates a token per request, so authentication decisions are made on signed assertions without requiring credentials to traverse the API layer. Audience-aware token authenticators validate that a token was intended for the specific server it was presented to by verifying it against the server's audience identifier, further scoping token validity without additional password transmission.

Kubernetes Secrets provide the primary mechanism for storing authentication credentials within VKS clusters, including registry passwords, service account tokens, and other sensitive values. Secret data fields are stored in base64-encoded form and are not returned as human-readable text by default in API responses; when etcd encryption at rest is configured, Secret values are encrypted in the datastore. The Kubernetes platform requires that applications protect confidential values after



reading them from environment variables or volumes by not logging secret data in cleartext and not transmitting it to untrusted parties. The Harbor registry Supervisor Service stores its authentication credentials, including admin passwords, secret keys, and job service secrets, as Kubernetes Secrets rather than in plaintext configuration files, keeping sensitive material out of environment variables visible to other processes. Registry secrets in Kubernetes store authentication credentials for private container image registries within the Secrets API rather than in plaintext manifests.

The Supervisor Secret Store service enforces role-based access control for administrators, DevOps engineers, and application tenants to manage and distribute sensitive authentication credentials through a controlled interface. For CI/CD pipelines on VKS, Argo CD's OIDC integration supports Proof Key for Code Exchange (PKCE) authentication through the `enablePKCEAuthentication` field, reducing the risk of authorization code interception in OIDC flows by requiring a verifier that cannot be replayed. HashiCorp Vault can generate dynamic secrets with limited lifespans for services, reducing the risk of long-lived credentials being exposed through logs or process inspection. The Kubernetes hardening guidance recommends configuring short token lifespans via the `TokenRequest` API to limit the exposure window if tokens are captured.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) contributes to authenticator feedback protection through several mechanisms that operate at different layers of the authentication stack.

The DSM administrative and user interface presents a web-based login screen where users supply an email address and password. Standard web browser behavior renders password input fields with character masking, so entered characters are not displayed in plaintext. DSM authenticates local users by matching the email address and validating the submitted password before any directory service processing occurs. For environments using LDAPS, DSM matches the email domain against the registered LDAP server and validates credentials over an encrypted channel, so the password is not transmitted or displayed in cleartext during the authentication exchange. DSM can be configured to use an LDAP server for both authentication and authorization and falls back to local user authentication when directory service credentials are non-functional.

At the database layer, DSM supports host-based authentication (HBA) configuration for database instances. Custom HBA entries support the `scram-sha-256`, `reject`, and `cert` authentication methods. The `scram-sha-256` method is a challenge-response protocol that does not transmit the password itself over the connection, limiting exposure of credential material during the authentication handshake. DSM also supports enabling or disabling directory service authentication per database instance through the `Enable Directory Service Authentication` toggle, allowing administrators to enforce LDAPS-backed authentication for database access rather than relying on simpler mechanisms.

In 9.1, DSM extends LDAPS-encrypted authentication to MySQL databases. MySQL database users can be configured with LDAPS authentication, including Active Directory Secure/SSL Global Catalog configurations for multi-domain environments, carrying credentials over an encrypted channel. SQL Server databases support Windows Principal authentication through Kerberos-based Active Directory domain resources, which is a challenge-response mechanism that does not transmit passwords in plaintext. SQL Server data service policies also support SQL User authentication for standard SQL Server logins.

DSM's LDAPS integration uses certificate-based validation of directory service server connections, supporting protection of the credential exchange channel. What DSM does not provide is explicit product-level configuration for UI feedback behavior, such as a setting that controls whether password fields mask characters. That behavior is inherited from the client browser and web framework conventions rather than being a DSM-configurable control. Organizations should also account for any programmatic or API-based access patterns where credential handling falls outside the DSM UI and must be governed by application-level controls.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides authenticator feedback protection through log-level credential masking for application proxy authentication flows and through its administrative authentication interface.

For application-layer authentication, the `sensitive_log_profile` can be configured to mask sensitive fields, including Basic Authorization credentials and OAuth tokens, from virtual service application logs. This setting is available in the OAuth and

OIDC client and resource server configuration. Organizations deploying Avi as a proxy for OAuth-authenticated applications should enable this profile to restrict credential material from appearing in operational log output. The masking applies to credentials that Avi processes at the proxy boundary, not to credentials transmitted between clients and backend servers outside Avi's inspection scope.

Avi's log protection capabilities extend masking to additional credential-bearing HTTP content. HTTP headers containing sensitive user-related information, including cookies and authorization headers, can be masked or removed from application logs. A sensitive log policy can be applied to mask cookies in both WAF logs and application log headers, which keeps session and credential tokens out of operational log storage.

For LDAP-based basic authentication exposed through a virtual service, Avi supports configurable realm indication to clients during authentication requests. This identifies the authentication realm to the client without exposing credential content in the response.

For administrative access, the Avi Controller web administration interface is the credential entry point for all administrator logins, including local accounts and accounts authenticated through federated sources such as LDAP, TACACS+, or SAML identity providers. When Avi acts as a proxy for federated authentication flows in which end users authenticate interactively against an external identity provider, such as a SAML identity provider, an LDAP directory, or an OAuth/OIDC authorization server, the credential entry experience and feedback masking for those user-facing flows are rendered and governed by the identity provider. Avi validates the resulting token or session assertion at the virtual service boundary but does not control what the end user sees during the interactive login step with the external provider.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 1.11

> Unsuccessful login attempts: Enforce a limit of consecutive invalid login attempts before logout or delay.

**SCF Controls:** IAC-22

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides account lockout mechanisms across its management components, each with configurable thresholds, lockout durations, and automatic unlock behavior. These capabilities allow an organization to enforce limits on consecutive invalid login attempts in line with its defined security policies.

NSX Manager enforces account lockout on both its API and CLI interfaces. The API authentication policy defaults to a maximum of 5 authentication failures before lockout, with a lockout period of 900 seconds (15 minutes). The CLI interface similarly defaults to 5 maximum authentication failures before lockout, with a `cli_failed_auth_lockout_period` of 900 seconds. These values are configurable through the NSX authentication policy endpoint (`/api/v1/node/aaa/auth-policy`) and through the "set auth-policy cli" command, which exposes parameters for `lockout-period` and `max-auth-failures`. Administrators can also disable account lockout on the CLI by setting the maximum failed attempts to 0, though this is not recommended in production environments.

VMware ESX hosts implement account lockout through two advanced system settings: `Security.AccountLockFailures` specifies the maximum number of failed login attempts before an account is locked (default 5 within a 3-minute window), and `Security.AccountUnlockTime` specifies the number of seconds the account remains locked (default 900 seconds, or 15 minutes). Both settings are configurable through the VMware Host Client under `Manage > System Settings` or through the ESX advanced system settings interface. Setting `Security.AccountLockFailures` to 0 deactivates account locking entirely, which is not recommended in production environments. Administrators should note that the Direct Console User Interface (DCUI) and the ESX Shell do not support account lockout, so access through those interfaces is not subject to the same enforcement.

VCF SSO implements a configurable Lockout Policy that specifies the maximum number of failed login attempts, the time interval in which failures must occur to trigger a lockout, and the duration of the lockout. By default, VCF SSO users are locked out after five consecutive failed attempts within three minutes, and locked accounts are automatically unlocked after five minutes. Setting the unlock time to 0 disables automatic unlock and requires an administrator to manually release the account.

VCF Operations enforces account lockout with a default threshold of seven failed login attempts, or if a user enters an incorrect password three times within five minutes. The account lockout policy is activated by default for super administrator users, and this policy cannot be deactivated. Super administrator accounts are locked for approximately one hour after lockout is triggered and then automatically unlocked. VCF Operations exposes a `Locked` attribute on user accounts that indicates whether the account has been locked, allowing administrators to monitor lockout status.

VCF Operations for Networks implements a progressive lockout model. After five consecutive failed login attempts, the account is temporarily locked for 15 minutes. Each subsequent failed login attempt after the initial lockout adds an additional 15 minutes to the lockout duration. An account can remain locked for a maximum of one day if login attempts continue after the initial lockout.

VCF Automation supports account lockout through its password policy system settings. Accounts are locked after a configurable number of invalid login attempts, and an account lockout interval specifies the duration before an account is automatically unlocked. Provider administrators can also unlock a user account manually without waiting for the configured lockout interval to expire.

VCF Operations HCX locks user accounts after three failed login attempts. If an account becomes locked, it can be reset by accessing the HCX Manager through the Web Console in VMware vCenter and running the `faillock` command with the `--reset` option.

To fully satisfy this control, the organization must define its own time periods and attempt thresholds for account lockout, then configure each VCF component accordingly. The default values vary across components (3, 5, or 7 attempts depending on the

component), so a consistent lockout policy requires reviewing and aligning the settings across all management interfaces. The organization should also establish procedures for monitoring lockout events and handling account unlock requests.

#### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) maintains account lockout state for local user accounts through dedicated tracking fields. DSM records `failed_login_count`, `last_failed_login_time`, `locked_state_start_time`, and `force_password_change` on each local user account, providing DSM administrators with visibility into failed login activity, account lock status, and pending password-change requirements. These fields are accessible through the DSM administration interface in the "Managing LDAP Users in Data Services Manager" section of the administration guide.

When a local user account becomes locked, DSM provides two recovery paths. Administrators can manually reset a locked account through the DSM administration console. DSM also exposes a "Forgot Password" link on the login screen that allows users to initiate a password reset via email; reset links are active for 20 minutes. This self-service path is available only if an SMTP server has been configured in the DSM settings. If no SMTP server is configured, self-service password recovery is unavailable, and all locked accounts require direct administrator intervention.

For users who authenticate through LDAPS, DSM delegates authentication and lockout enforcement to the external directory service. The lockout threshold, time window, and automatic unlock behavior for LDAPS-authenticated users are governed by the organization's LDAP or Active Directory configuration, not by DSM. LDAPS users who need to reset a password must contact their LDAPS administrator rather than using the local password reset mechanism.

Organizations deploying DSM should configure lockout policies in their directory services for all LDAPS-authenticated users, configure SMTP to support self-service account recovery for local users, and establish administrator procedures for unlocking local accounts when self-service recovery is not available.

#### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer includes a native account lockout mechanism for the Avi Controller administrative identity plane, configured through User Account Profiles. Each profile defines three lockout settings: Max Authentication Failures (the maximum number of consecutive failed login attempts before the account is locked), the Evaluation Period (the time window, in seconds, within which failures are counted toward lockout), and the lockout duration (the period the account remains locked after reaching the threshold). These settings are accessible through the Controller UI under Administration > User Account Security and through the Avi REST API.

The Default-User-Account-Profile ships with `max_login_failure_count` set to 20 and an `account_lock_timeout` of 30 minutes. Organizations can modify the failure threshold and lockout duration by editing the default profile or by creating a custom User Account Profile and assigning it to specific users. A built-in No-Lockout-User-Account-Profile sets Max Authentication Failures to 0, which disables lockout for any account assigned to it; when this setting is 0, the Evaluation Period and Lockout Period values have no effect. Auditors should verify that accounts assigned to the No-Lockout-User-Account-Profile carry an explicit authorization exception, as this profile bypasses the lockout control.

The Avi GSLB administrative interface has a parallel lockout configuration, where a specified number of authentication failures triggers an account lockout for a configured duration. This is controlled by the `login-failure-lockout-duration` setting in the GSLB configuration; the default lockout duration is 30 minutes.

Organizations using remote authentication sources such as LDAP, TACACS+, SAML, or RADIUS for Controller access will rely on the lockout policies of those external directories rather than on Avi's own profile settings. The organization is responsible for configuring Max Authentication Failures and the Evaluation Period to reflect its defined policy thresholds and time periods.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 1.11(a)

> Sub-item (a) of CR 1.11 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 1.11 requirement in the standard.

**SCF Controls:** IAC-22

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides account lockout mechanisms across its management components, each with configurable thresholds, lockout durations, and automatic unlock behavior. These capabilities allow an organization to enforce limits on consecutive invalid login attempts in line with its defined security policies.

NSX Manager enforces account lockout on both its API and CLI interfaces. The API authentication policy defaults to a maximum of 5 authentication failures before lockout, with a lockout period of 900 seconds (15 minutes). The CLI interface similarly defaults to 5 maximum authentication failures before lockout, with a `cli_failed_auth_lockout_period` of 900 seconds. These values are configurable through the NSX authentication policy endpoint (`/api/v1/node/aaa/auth-policy`) and through the "set auth-policy cli" command, which exposes parameters for lockout-period and max-auth-failures. Administrators can also disable account lockout on the CLI by setting the maximum failed attempts to 0, though this is not recommended in production environments.

VMware ESX hosts implement account lockout through two advanced system settings: `Security.AccountLockFailures` specifies the maximum number of failed login attempts before an account is locked (default 5 within a 3-minute window), and `Security.AccountUnlockTime` specifies the number of seconds the account remains locked (default 900 seconds, or 15 minutes). Both settings are configurable through the VMware Host Client under Manage > System Settings or through the ESX advanced system settings interface. Setting `Security.AccountLockFailures` to 0 deactivates account locking entirely, which is not recommended in production environments. Administrators should note that the Direct Console User Interface (DCUI) and the ESX Shell do not support account lockout, so access through those interfaces is not subject to the same enforcement.

VCF SSO implements a configurable Lockout Policy that specifies the maximum number of failed login attempts, the time interval in which failures must occur to trigger a lockout, and the duration of the lockout. By default, VCF SSO users are locked out after five consecutive failed attempts within three minutes, and locked accounts are automatically unlocked after five minutes. Setting the unlock time to 0 disables automatic unlock and requires an administrator to manually release the account.

VCF Operations enforces account lockout with a default threshold of seven failed login attempts, or if a user enters an incorrect password three times within five minutes. The account lockout policy is activated by default for super administrator users, and this policy cannot be deactivated. Super administrator accounts are locked for approximately one hour after lockout is triggered and then automatically unlocked. VCF Operations exposes a `Locked` attribute on user accounts that indicates whether the account has been locked, allowing administrators to monitor lockout status.

VCF Operations for Networks implements a progressive lockout model. After five consecutive failed login attempts, the account is temporarily locked for 15 minutes. Each subsequent failed login attempt after the initial lockout adds an additional 15 minutes to the lockout duration. An account can remain locked for a maximum of one day if login attempts continue after the initial lockout.

VCF Automation supports account lockout through its password policy system settings. Accounts are locked after a configurable number of invalid login attempts, and an account lockout interval specifies the duration before an account is automatically unlocked. Provider administrators can also unlock a user account manually without waiting for the configured lockout interval to expire.

VCF Operations HCX locks user accounts after three failed login attempts. If an account becomes locked, it can be reset by accessing the HCX Manager through the Web Console in VMware vCenter and running the `faillock` command with the `--reset` option.

To fully satisfy this control, the organization must define its own time periods and attempt thresholds for account lockout, then configure each VCF component accordingly. The default values vary across components (3, 5, or 7 attempts depending on the component), so a consistent lockout policy requires reviewing and aligning the settings across all management interfaces. The organization should also establish procedures for monitoring lockout events and handling account unlock requests.

#### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) maintains account lockout state for local user accounts through dedicated tracking fields. DSM records `failed_login_count`, `last_failed_login_time`, `locked_state_start_time`, and `force_password_change` on each local user account, providing DSM administrators with visibility into failed login activity, account lock status, and pending password-change requirements. These fields are accessible through the DSM administration interface in the "Managing LDAP Users in Data Services Manager" section of the administration guide.

When a local user account becomes locked, DSM provides two recovery paths. Administrators can manually reset a locked account through the DSM administration console. DSM also exposes a "Forgot Password" link on the login screen that allows users to initiate a password reset via email; reset links are active for 20 minutes. This self-service path is available only if an SMTP server has been configured in the DSM settings. If no SMTP server is configured, self-service password recovery is unavailable, and all locked accounts require direct administrator intervention.

For users who authenticate through LDAPS, DSM delegates authentication and lockout enforcement to the external directory service. The lockout threshold, time window, and automatic unlock behavior for LDAPS-authenticated users are governed by the organization's LDAP or Active Directory configuration, not by DSM. LDAPS users who need to reset a password must contact their LDAPS administrator rather than using the local password reset mechanism.

Organizations deploying DSM should configure lockout policies in their directory services for all LDAPS-authenticated users, configure SMTP to support self-service account recovery for local users, and establish administrator procedures for unlocking local accounts when self-service recovery is not available.

#### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer includes a native account lockout mechanism for the Avi Controller administrative identity plane, configured through User Account Profiles. Each profile defines three lockout settings: Max Authentication Failures (the maximum number of consecutive failed login attempts before the account is locked), the Evaluation Period (the time window, in seconds, within which failures are counted toward lockout), and the lockout duration (the period the account remains locked after reaching the threshold). These settings are accessible through the Controller UI under Administration > User Account Security and through the Avi REST API.

The Default-User-Account-Profile ships with `max_login_failure_count` set to 20 and an `account_lock_timeout` of 30 minutes. Organizations can modify the failure threshold and lockout duration by editing the default profile or by creating a custom User Account Profile and assigning it to specific users. A built-in No-Lockout-User-Account-Profile sets Max Authentication Failures to 0, which disables lockout for any account assigned to it; when this setting is 0, the Evaluation Period and Lockout Period values have no effect. Auditors should verify that accounts assigned to the No-Lockout-User-Account-Profile carry an explicit authorization exception, as this profile bypasses the lockout control.

The Avi GSLB administrative interface has a parallel lockout configuration, where a specified number of authentication failures triggers an account lockout for a configured duration. This is controlled by the `login-failure-lockout-duration` setting in the GSLB configuration; the default lockout duration is 30 minutes.

Organizations using remote authentication sources such as LDAP, TACACS+, SAML, or RADIUS for Controller access will rely on the lockout policies of those external directories rather than on Avi's own profile settings. The organization is responsible for configuring Max Authentication Failures and the Evaluation Period to reflect its defined policy thresholds and time periods.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.



## Control CR 1.11(b)

> Sub-item (b) of CR 1.11 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 1.11 requirement in the standard.

**SCF Controls:** IAC-22

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides account lockout mechanisms across its management components, each with configurable thresholds, lockout durations, and automatic unlock behavior. These capabilities allow an organization to enforce limits on consecutive invalid login attempts in line with its defined security policies.

NSX Manager enforces account lockout on both its API and CLI interfaces. The API authentication policy defaults to a maximum of 5 authentication failures before lockout, with a lockout period of 900 seconds (15 minutes). The CLI interface similarly defaults to 5 maximum authentication failures before lockout, with a `cli_failed_auth_lockout_period` of 900 seconds. These values are configurable through the NSX authentication policy endpoint (`/api/v1/node/aaa/auth-policy`) and through the "set auth-policy cli" command, which exposes parameters for lockout-period and max-auth-failures. Administrators can also disable account lockout on the CLI by setting the maximum failed attempts to 0, though this is not recommended in production environments.

VMware ESX hosts implement account lockout through two advanced system settings: `Security.AccountLockFailures` specifies the maximum number of failed login attempts before an account is locked (default 5 within a 3-minute window), and `Security.AccountUnlockTime` specifies the number of seconds the account remains locked (default 900 seconds, or 15 minutes). Both settings are configurable through the VMware Host Client under Manage > System Settings or through the ESX advanced system settings interface. Setting `Security.AccountLockFailures` to 0 deactivates account locking entirely, which is not recommended in production environments. Administrators should note that the Direct Console User Interface (DCUI) and the ESX Shell do not support account lockout, so access through those interfaces is not subject to the same enforcement.

VCF SSO implements a configurable Lockout Policy that specifies the maximum number of failed login attempts, the time interval in which failures must occur to trigger a lockout, and the duration of the lockout. By default, VCF SSO users are locked out after five consecutive failed attempts within three minutes, and locked accounts are automatically unlocked after five minutes. Setting the unlock time to 0 disables automatic unlock and requires an administrator to manually release the account.

VCF Operations enforces account lockout with a default threshold of seven failed login attempts, or if a user enters an incorrect password three times within five minutes. The account lockout policy is activated by default for super administrator users, and this policy cannot be deactivated. Super administrator accounts are locked for approximately one hour after lockout is triggered and then automatically unlocked. VCF Operations exposes a `Locked` attribute on user accounts that indicates whether the account has been locked, allowing administrators to monitor lockout status.

VCF Operations for Networks implements a progressive lockout model. After five consecutive failed login attempts, the account is temporarily locked for 15 minutes. Each subsequent failed login attempt after the initial lockout adds an additional 15 minutes to the lockout duration. An account can remain locked for a maximum of one day if login attempts continue after the initial lockout.

VCF Automation supports account lockout through its password policy system settings. Accounts are locked after a configurable number of invalid login attempts, and an account lockout interval specifies the duration before an account is automatically unlocked. Provider administrators can also unlock a user account manually without waiting for the configured lockout interval to expire.

VCF Operations HCX locks user accounts after three failed login attempts. If an account becomes locked, it can be reset by accessing the HCX Manager through the Web Console in VMware vCenter and running the `faillock` command with the `--reset` option.



To fully satisfy this control, the organization must define its own time periods and attempt thresholds for account lockout, then configure each VCF component accordingly. The default values vary across components (3, 5, or 7 attempts depending on the component), so a consistent lockout policy requires reviewing and aligning the settings across all management interfaces. The organization should also establish procedures for monitoring lockout events and handling account unlock requests.

#### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) maintains account lockout state for local user accounts through dedicated tracking fields. DSM records `failed_login_count`, `last_failed_login_time`, `locked_state_start_time`, and `force_password_change` on each local user account, providing DSM administrators with visibility into failed login activity, account lock status, and pending password-change requirements. These fields are accessible through the DSM administration interface in the "Managing LDAP Users in Data Services Manager" section of the administration guide.

When a local user account becomes locked, DSM provides two recovery paths. Administrators can manually reset a locked account through the DSM administration console. DSM also exposes a "Forgot Password" link on the login screen that allows users to initiate a password reset via email; reset links are active for 20 minutes. This self-service path is available only if an SMTP server has been configured in the DSM settings. If no SMTP server is configured, self-service password recovery is unavailable, and all locked accounts require direct administrator intervention.

For users who authenticate through LDAPS, DSM delegates authentication and lockout enforcement to the external directory service. The lockout threshold, time window, and automatic unlock behavior for LDAPS-authenticated users are governed by the organization's LDAP or Active Directory configuration, not by DSM. LDAPS users who need to reset a password must contact their LDAPS administrator rather than using the local password reset mechanism.

Organizations deploying DSM should configure lockout policies in their directory services for all LDAPS-authenticated users, configure SMTP to support self-service account recovery for local users, and establish administrator procedures for unlocking local accounts when self-service recovery is not available.

#### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer includes a native account lockout mechanism for the Avi Controller administrative identity plane, configured through User Account Profiles. Each profile defines three lockout settings: Max Authentication Failures (the maximum number of consecutive failed login attempts before the account is locked), the Evaluation Period (the time window, in seconds, within which failures are counted toward lockout), and the lockout duration (the period the account remains locked after reaching the threshold). These settings are accessible through the Controller UI under Administration > User Account Security and through the Avi REST API.

The Default-User-Account-Profile ships with `max_login_failure_count` set to 20 and an `account_lock_timeout` of 30 minutes. Organizations can modify the failure threshold and lockout duration by editing the default profile or by creating a custom User Account Profile and assigning it to specific users. A built-in No-Lockout-User-Account-Profile sets Max Authentication Failures to 0, which disables lockout for any account assigned to it; when this setting is 0, the Evaluation Period and Lockout Period values have no effect. Auditors should verify that accounts assigned to the No-Lockout-User-Account-Profile carry an explicit authorization exception, as this profile bypasses the lockout control.

The Avi GSLB administrative interface has a parallel lockout configuration, where a specified number of authentication failures triggers an account lockout for a configured duration. This is controlled by the `login-failure-lockout-duration` setting in the GSLB configuration; the default lockout duration is 30 minutes.

Organizations using remote authentication sources such as LDAP, TACACS+, SAML, or RADIUS for Controller access will rely on the lockout policies of those external directories rather than on Avi's own profile settings. The organization is responsible for configuring Max Authentication Failures and the Evaluation Period to reflect its defined policy thresholds and time periods.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 1.12

> System use notification: Display a notification message before establishing user identification.

**SCF Controls:** SEA-18, SEA-18.1, SEA-18.2

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides configurable logon banner and system use notification capabilities across its management interfaces, allowing organizations to display approved notification messages before granting access.

NSX Manager supports configuring a security banner or message of the day. A separate login user agreement banner can be activated to display at every login. When enabled, only users with Enterprise Admin roles can modify the banner content, restricting unauthorized changes to the notification text. This banner can be configured with organization-specific language covering acceptable use policies, monitoring notices, and legal disclaimers. NSX login user agreement banner configuration is recommended for organizations with security compliance requirements. In federated environments, each Global Manager and Local Manager maintains its own login user agreement banner configuration, allowing site-specific messaging.

VCF Operations for Networks provides configurable login message banners through both the web UI and the CLI. Administrators can configure custom message text through the System Configuration settings, where login banners can be enabled or deactivated via a toggle button on the Settings page. When login banners are enabled, VCF Operations for Networks supports requiring users to click an explicit consent check box to indicate acceptance of terms and conditions before proceeding, providing a documented acknowledgment mechanism. A separate user consent form can also be configured with a title and content message that requires users to agree before logging in. The set-banner CLI command configures both the SSH pre-login banner for the support user and the post-login banner for the console user, with support for quoted text and newline characters for multi-line messages.

VCF Operations (the monitoring and analytics platform) can display a login message when users access the login page, with the option to require explicit user consent for terms, conditions, or mandatory notices before proceeding.

VCF Automation for VM Apps supports enabling a Notice and Consent Banner through administrator configuration in Workspace ONE Access. This is configured through the Workspace ONE Access administrative console by setting the CertificateAuthAdapter with an Enable Consent Form before Authentication setting. The banner informs users of monitoring and data inspection policies before they access the automation interface.

VMware ESX supports configurable SSH connection banner messages that display when users connect via SSH. A separate welcome message can also be configured for display to users accessing ESX hosts. These settings are exposed as host system properties that can be monitored and enforced through VCF Operations.

vSphere Supervisor cluster configuration supports a login banner message displayed to users upon authentication, configurable through the cluster setup parameters.

These banner capabilities span VCF management components at the network, compute, operations, automation, and Kubernetes layers. Administrators should configure banners on each management interface with the organization's approved notification text and enable consent requirements where available.

### VMware Avi Load Balancer (Meets)

VMware Avi Load Balancer provides configurable Login Banner and Message of the Day mechanisms that apply to all administrative access paths: the Controller web UI, CLI, and SSH. The Login Banner is displayed before authentication completes, meaning it is presented to all users attempting access regardless of whether their credentials succeed. The Message of the Day is displayed after a successful login through any of these interfaces. Both messages are configured in Administration > System Settings and take effect on the next login session.

The Login Banner text is fully customizable and can contain any organizational-approved system use notification, including statements about authorized use, monitoring, and consent. Together, these two mechanisms satisfy the requirement for a pre-

login system use notification across all supported administrative interfaces of Avi Load Balancer, with no dependency on the underlying VCF platform beyond network access to the Controller.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 1.14

> Strength of symmetric key-based authentication: Capability to manage symmetric keys securely. Symmetric key protection: Hardware-based protection for symmetric keys.

**SCF Controls:** CRY-09.1

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides integrated symmetric key management capabilities through its Key Management Server (KMS) infrastructure and the VMware vCenter CryptoManager interface. VCF requires that the KMS support the Key Management Interoperability Protocol (KMIP) 1.1 standard and that it be capable of acting as a symmetric key foundry and server. The KMS must also appear in the VMware Compatibility Guide for Key Management Servers before it can be used in a VCF environment. Alternatively, the vSphere Native Key Provider can manage symmetric keys without requiring an external KMS and can coexist with existing key server infrastructure. This architecture centralizes symmetric key production and lifecycle management through a standards-based protocol, with vCenter as the orchestration layer between VCF components and the key provider.

VCF uses a key hierarchy for encryption operations. Key Encryption Keys (KEK) wrap Disk Encryption Keys (DEK), with DEKs using XTS-AES-256. VMware ESX hosts use the KEK to encrypt their internal keys and store only the encrypted internal keys on disk; the KEK itself is not persisted locally. Internal encryption keys are kept in memory only during operation, with only the encrypted data stored on disk. This separation of key tiers limits the exposure of key material at rest.

vSAN data-at-rest encryption relies on KMS integration for all symmetric key operations. Enabling vSAN encryption requires selecting a KMS cluster during configuration, establishing a trust relationship with the KMS by uploading certificate and key files, and confirming that the KMS is healthy before encryption can proceed. Administrators should deploy KMS servers on infrastructure separate from the vSAN datastore being encrypted and should use highly available KMS clusters to distribute the keys that protect vSAN datastores. Some KMS vendors support credential-based isolation of encryption keys, allowing administrators to specify a username and password during vSAN encryption configuration for key partitioning. An optional proxy port configuration is also available for KMS connectivity. vSAN also supports the vSphere Native Key Provider for encrypting virtual machines in vSAN clusters.

vSAN data-in-transit encryption uses symmetric keys that are generated dynamically and shared between hosts to encrypt inter-host traffic. The rekey interval for data-in-transit encryption is configurable, with a default of 1,440 minutes (24 hours) and a supported range of 30 minutes to 10,080 minutes (7 days). For data-at-rest encryption, vSAN supports generating new Key Encryption Keys (KEK) and new Data Encryption Keys (DEK), with the capability to re-encrypt all stored data using new keys when existing keys expire or become compromised. Both shallow rekeying (replacing only KEKs via `CryptoSpecShallowRekey`) and deep rekeying (replacing all KEKs and DEKs via `CryptoSpecDeepRekey`) are available through the API. vSAN OSA and vSAN ESA both support these key rotation options using AES-256 bit encryption. AES-NI hardware acceleration significantly improves encryption performance, and administrators should enable AES-NI in the host BIOS.

The vCenter CryptoManager managed object provides programmatic control over cryptographic key operations in vSphere, including integration with the KMS for key creation, retrieval, and lifecycle management. This API supports both vSAN encryption and virtual machine encryption workflows, where symmetric keys are managed through the same KMS infrastructure. The vSphere Web Services API also provides `registerKmpServer()` to register a KMIP key management server with vCenter's cryptography manager.

VMware Kubernetes Service (VKS) clusters running on the vSphere Supervisor extend symmetric key management to the Kubernetes control plane through the Kubernetes KMS provider framework. The `kube-apiserver` supports `etcd` encryption-at-rest through an `EncryptionConfiguration` resource that specifies a KMS provider type with a Unix domain socket endpoint connecting to an external key management system. This framework uses an envelope encryption scheme: data stored in `etcd` is encrypted with a DEK, which is itself encrypted by a KEK held in the external KMS, keeping the root of trust outside the cluster boundary. Kubernetes recommends the KMS provider over storing raw encryption keys inline in the

EncryptionConfiguration file for this reason. KMS v2, stable from Kubernetes 1.29, uses the KmsV2Kdf feature gate to generate single-use DEKs from a secret seed combined with random data via a key derivation function, reducing call volume to the external KMS while maintaining per-object cryptographic isolation. KMS plugin implementations are required to verify the key\_id of any wrapped key before attempting decryption, which supports key versioning and rotation workflows. Key rotation requires an administrator to explicitly trigger re-encryption of stored secrets after a KEK rotation; VKS provides the technical hooks for this process. For environments requiring FIPS-validated cryptographic operations at the Kubernetes layer, VKS supports TanzuKubernetesRelease (TKR) images with FIPS cryptographic module support, such as the vmware.3-fips.1-tkg.1 release variants, which configure Kubernetes node binaries to use a FIPS-validated module for symmetric operations within the cluster. The external KMS must independently be a FIPS-validated product or hardware security module.

VCF components use FIPS-validated cryptographic modules across the stack. VCF supports FIPS 140-2 requirements for cryptographic modules. On the vCenter appliance, VMware's Boring Crypto Module version 6.0 provides FIPS-validated support for AES, ECDSA, HMAC, RSA, and SHA2 variants; the VMware OpenSSL FIPS Object Module (versions 2.0.20-vmw and 3.0) also covers DRBG, DSA, SHS, and Triple-DES algorithms. NSX is configured to use FIPS 140-3 validated cryptographic modules by default, with validation performed through the Cryptographic Module Validation Program (CMVP). NSX Manager uses FIPS 140 approved algorithms for authentication to cryptographic modules and generates unique session identifiers using a FIPS 140 approved random number generator. VCF Automation uses the BouncyCastle FIPS 2.0.0 cryptographic module, which is compliant with FIPS 140-3 (Certificate #4743). VCF Operations for Networks uses the BC-FJA (Bouncy Castle FIPS Java API) version 2.0.0 as its validated cryptographic module and can restrict cryptographic operations to FIPS-validated modules by enabling FIPS mode for external connections. VCF Operations for Networks also uses FIPS-compliant cryptographic algorithms for internal connections. VCF Operations orchestrator uses a FIPS-compliant AES with a 256-bit cipher key for encryption of strings.

VCF Operations supports a FIPS Security Mode that, when enabled, applies additional algorithm and cipher prerequisites across the platform. This mode restricts the cryptographic algorithms and ciphers available to those that meet FIPS validation requirements, including restricting key exchange algorithms to FIPS-approved options. This provides a platform-wide enforcement mechanism for FIPS-compliant symmetric key usage.

Two limitations apply in FIPS-sensitive environments. Self-signed certificates and private keys generated using OpenSSL are not FIPS-compatible; administrators should use FIPS-compatible key generation methods such as pyOpenSSL for PKCS#8 format. While NSX supports additional elliptic curve key types for certificates, non-FIPS-approved key types should not be used in environments requiring only FIPS-approved cryptographic keys.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) contributes to symmetric cryptographic key management through its requirements for consistent key management infrastructure across protected and recovery sites.

PNR and vSphere Replication require that either a KMIP-compatible Key Management Server (KMS) or the vSphere Native Key Provider be available at both the protected and recovery sites to protect and recover encrypted virtual machines. The KMS cluster must be registered with the same name on both sites. When using the vSphere Native Key Provider, it must be manually configured on both the local and remote sites. The vSphere Native Key Provider enables encryption-related functionality for virtual machines and virtual disks without requiring an external KMS.

PNR establishes a trusted connection between VMware vCenter and the KMS from which vCenter retrieves keys as needed to protect virtual machine disks and other files. This cross-site KMS consistency is a prerequisite for encrypted VM failover and failback through recovery plans.

The generation and lifecycle management of symmetric keys, including the key hierarchy, FIPS validation of cryptographic modules, key rotation scheduling, and TPM-based key persistence, are capabilities of the VCF platform. PNR's contribution is the enforcement of consistent KMS deployment across recovery topologies, which is a necessary condition for encrypted workload protection and recovery to function.

### **VMware Data Services Manager (Contributes)**

This control requires mechanisms for producing and managing symmetric cryptographic keys using FIPS-compliant key management technology and processes. VMware Data Services Manager (DSM) applies symmetric encryption across several operational contexts and exposes FIPS-related configuration options, but relies on the broader VCF platform and external tooling to satisfy the full scope of this requirement.

In DSM, the product enforces FIPS 140-3 compliant cryptographic standards and validates certificates during upload, rejecting unsupported algorithms or parameters. DSM specifically rejects the Ed25519 elliptic curve algorithm because it is not FIPS 140-3 compliant. Accepted key algorithms for custom certificates include ECDSA P-256, ECDSA P-384, and RSA keys with a minimum size of 2048 bits. This enforcement is documented in "Using Trusted Certificates in Data Services Manager > Configuring Custom Certificates for Data Services Manager > Supported Cryptographic Algorithms."

For MySQL databases managed by DSM, the `ssl_fips_mode` setting activates FIPS mode on the client side, restricting cipher negotiation to FIPS-approved algorithms. This setting is documented in "Databases in VMware Data Services Manager > Using MySQL in Data Services Manager > Configuration."

DSM also applies symmetric encryption in its user management subsystem. User passwords are stored encrypted using PGCrypto symmetric encryption with dedicated encryption keys, as documented in "Administering Data Services Manager > Managing LDAP Users in Data Services Manager." PostgreSQL databases managed by DSM include the pgcrypto extension version 1.3, which provides cryptographic functions available to database workloads.

The backup subsystem uses AES-256 to encrypt backup data written to S3-compatible storage, consistent with FIPS-approved algorithm requirements.

What DSM does not provide is a standalone FIPS-validated key management module or integration with an external key management service (KMS). The production, lifecycle management, rotation, and revocation of symmetric keys at the organizational level requires an external FIPS-validated KMS or hardware security module (HSM) operating at the infrastructure layer. DSM consumes keys and applies FIPS-approved symmetric algorithms, but the key generation and management controls for those keys are the responsibility of the organization and the underlying platform. Auditors should verify that the VCF infrastructure supporting DSM is configured with an appropriate FIPS-validated key provider, and that the organization has key management processes that cover the DSM-managed symmetric keys used for backups and internal encryption.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) supports a FIPS 140-3 compliance mode that uses OpenSSL FIPS Provider version 3.1.2 for all cryptographic operations. The OpenSSL FIPS Object Module v3.1.2 has been validated at FIPS 140-3 Security Level 1 and awarded Certificate #4985 by the Cryptographic Module Validation Program (CMVP). When FIPS mode is enabled on the Avi Controller and Service Engines, only cryptographic algorithms approved under FIPS 140-3 are used. FIPS 140-3 requires stricter handling of keys, authentication data, and random number generation than its predecessor, and aligns cryptographic module standards with international standards ISO/IEC 19790:2012 and ISO/IEC 24759:2017. This covers the symmetric encryption operations involved in TLS termination, re-encryption between Service Engines and backend servers, and internal control-plane communication. FIPS mode is configured at deployment time for Controller cluster deployments.

SSL/TLS Profile Templates in Avi give administrators control over the cipher suites and protocol versions used by virtual services. These profiles specify a prioritized cipher list and accepted protocol versions per virtual service and are managed under Templates > Security > SSL/TLS Profile in the Avi Controller UI. Configuring a FIPS-aligned SSL/TLS Profile and applying it to virtual services restricts symmetric key negotiation and cipher use across all load-balanced application traffic to FIPS-approved algorithms.

Post-quantum cryptography (PQC)-based Key Encapsulation Mechanisms and Signature Algorithms are not supported in Avi deployments running in FIPS mode.

For organizations deploying Avi in public cloud environments, Avi integrates with cloud-native key management services. In AWS, Avi's Cloud Connector integrates with AWS KMS for Service Engine disk and SQS queue encryption, supporting customer-managed keys with configurable IAM and key policies; the Avi Controller can list KMS encryption keys in the Avi UI

and supports S3 and EBS encryption. In Google Cloud Platform, Avi supports Google Cloud KMS for Service Engine disk and image encryption, GCS bucket encryption, and GCS object encryption, using separate KMS keys for each resource type.

The Avi Kubernetes Operator (AKO) 2.1 adds support for configuring both RSA and ECC signed certificates simultaneously on a single virtual service through the alt.crt and alt.key fields in the default TLS secret, enabling certificate algorithm flexibility for Kubernetes ingress workloads.

Not applicable for this control: VMware vDefend. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.



## Control CR 1.14(1)

> Strength of symmetric key-based authentication: Capability to manage symmetric keys securely. Symmetric key protection: Hardware-based protection for symmetric keys.

**SCF Controls:** IAC-12.1

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides several integration points for Hardware Security Modules (HSMs) and hardware-rooted trust mechanisms that support the protection of authenticators and cryptographic material, though VCF itself does not include a built-in HSM. The platform is designed to work with external HSM devices and hardware security components to strengthen key management, entropy sourcing, and boot integrity.

VCF integrates with external Key Management Servers (KMS) through the KMIP 1.1 protocol, providing the mechanism through which organizations can connect HSM-backed key management infrastructure to the platform's encryption services. The external KMS is the component that would be backed by an HSM to protect the root encryption keys on which VCF relies for vSAN data-at-rest encryption, VM Encryption, and other cryptographic operations. The KMS must support KMIP 1.1 and be capable of acting as a symmetric key foundry and server. VCF supports Standard Key Providers that connect to external KMIP-compliant KMS systems, where the KMS can be backed by HSMs for hardware-protected key storage. VMware vCenter establishes a trusted connection with the KMS by uploading the KMS server certificate and private key, and provisions encryption keys from the key provider. Each VMware ESX host uses the Key Encryption Key (KEK) to encrypt its Data Encryption Keys (DEKs) and stores the encrypted DEKs on disk, but does not store the KEK on disk. When a host reboots, it requests the KEK with the corresponding ID from the KMS and decrypts its DEKs as needed. The vSAN KMS configuration also supports an optional proxy address for environments that require it. When a user performs an encryption task such as creating an encrypted virtual machine, vCenter requests a new key from the default KMS to use as the KEK.

ESX supports the use of external HSM devices that are FIPS 140-3 and EAL4 certified as sources of external entropy data. Starting with vSphere 8.0 Update 1, administrators can add external entropy sources to an ESX host to feed high-quality, hardware-generated randomness into the host's cryptographic subsystem. This capability directly strengthens the generation of keys and other authenticators used across the platform by supplementing the host's own entropy sources.

ESX host hardening guidance includes enabling the Trusted Platform Module (TPM), a hardware-based security component embedded in the physical server. TPM provides a hardware root of trust for secure boot attestation and can store cryptographic keys and measurements in tamper-resistant hardware. When TPM is enabled and configured as part of the ESX hardening baseline, it provides hardware-backed protection for host-level authenticators and boot integrity verification. The ESX hardening guidance covers configuring authentication and authorization settings, enabling Secure Boot, disabling unnecessary services, enabling TPM, and implementing network security measures as part of a layered security posture. TPM modules must run the latest 2.0 firmware when in use, or be disabled in the BIOS if not in use. The physical TPM on ESX hosts also supports sealing policies based on Platform Configuration Register (PCR) measurements, which restrict access to sensitive data stored in the TPM to configurations that match a known-good platform state. Beyond host-level TPM, vSphere supports Virtual Trusted Platform Modules (vTPM) for virtual machines, allowing guest operating systems to use hardware-rooted cryptographic security without requiring a dedicated physical TPM per VM; vTPM keys are not exposed to the guest operating system itself, which reduces the attack surface at the VM level.

VCF also supports hardware memory encryption through confidential vSphere Pods. Confidential vSphere Pods use hardware technology such as AMD SEV-ES to keep guest OS memory encrypted, protecting it against access from the hypervisor. This capability addresses workload confidentiality rather than authenticator storage, but is part of the platform's hardware-rooted security mechanisms.

Organizations deploying VCF in environments that require HSM-protected authenticators should procure and integrate external HSM appliances, configure ESX hosts to use them for entropy and key management, enable TPM on all hosts, and establish trusted connections between vCenter and the KMS infrastructure. The platform provides the integration interfaces,

protocol support, and configuration options, but the HSM hardware itself, its lifecycle management, and the policies governing its use are organizational responsibilities.

#### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer supports Hardware Security Module (HSM) integration as a native capability for protecting the cryptographic keys that Avi's TLS termination and certificate operations depend on. The Avi Controller integrates with Thales Luna (formerly SafeNet Luna) HSM appliances, enabling hardware-backed storage and management of SSL certificates and private keys rather than software-based key storage.

The primary configuration mechanism is the HSM Group, configured in the Service Engine Group settings under the Advanced Settings tab in the Avi Controller UI or through the API. An HSM Group defines how Avi Controller and Service Engine nodes reach and authenticate with the external HSM device, including registering the IP addresses of all Controllers and Service Engines authorized to interact with it. The HSM Group is assigned per Service Engine Group and acts as a template of HSM settings applied uniformly to every Service Engine in that group. Mutual certificate authentication is established between the Avi Controller and the HSM during initial registration, so that only authorized nodes can request cryptographic operations from the device.

Avi Service Engines support dedicated network interfaces for HSM communication, separate from their standard management and data-plane interfaces. This interface isolation keeps HSM traffic off the general management network. Avi Controller cluster nodes similarly use dedicated network interfaces labeled HSM for hardware security module communication, and the HSM interface can be moved to a secondary interface if needed. High availability across multiple HSM devices is supported, allowing organizations to add redundant HSM nodes to an HSM Group to reduce single points of failure in key availability.

Avi Load Balancer's FIPS 140-3 compliance posture also incorporates dedicated HSM interfaces on Service Engines for secure cryptographic operations. Supported HSM clients include Thales Luna HSM client 7.3 and Universal Client v 10.4.1 for key management integration.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 1.14(a)

> Sub-item (a) of CR 1.14 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 1.14 requirement in the standard.

**SCF Controls:** CRY-09.1

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides integrated symmetric key management capabilities through its Key Management Server (KMS) infrastructure and the VMware vCenter CryptoManager interface. VCF requires that the KMS support the Key Management Interoperability Protocol (KMIP) 1.1 standard and that it be capable of acting as a symmetric key foundry and server. The KMS must also appear in the VMware Compatibility Guide for Key Management Servers before it can be used in a VCF environment. Alternatively, the vSphere Native Key Provider can manage symmetric keys without requiring an external KMS and can coexist with existing key server infrastructure. This architecture centralizes symmetric key production and lifecycle management through a standards-based protocol, with vCenter as the orchestration layer between VCF components and the key provider.

VCF uses a key hierarchy for encryption operations. Key Encryption Keys (KEK) wrap Disk Encryption Keys (DEK), with DEKs using XTS-AES-256. VMware ESX hosts use the KEK to encrypt their internal keys and store only the encrypted internal keys on disk; the KEK itself is not persisted locally. Internal encryption keys are kept in memory only during operation, with only the encrypted data stored on disk. This separation of key tiers limits the exposure of key material at rest.

vSAN data-at-rest encryption relies on KMS integration for all symmetric key operations. Enabling vSAN encryption requires selecting a KMS cluster during configuration, establishing a trust relationship with the KMS by uploading certificate and key files, and confirming that the KMS is healthy before encryption can proceed. Administrators should deploy KMS servers on infrastructure separate from the vSAN datastore being encrypted and should use highly available KMS clusters to distribute the keys that protect vSAN datastores. Some KMS vendors support credential-based isolation of encryption keys, allowing administrators to specify a username and password during vSAN encryption configuration for key partitioning. An optional proxy port configuration is also available for KMS connectivity. vSAN also supports the vSphere Native Key Provider for encrypting virtual machines in vSAN clusters.

vSAN data-in-transit encryption uses symmetric keys that are generated dynamically and shared between hosts to encrypt inter-host traffic. The rekey interval for data-in-transit encryption is configurable, with a default of 1,440 minutes (24 hours) and a supported range of 30 minutes to 10,080 minutes (7 days). For data-at-rest encryption, vSAN supports generating new Key Encryption Keys (KEK) and new Data Encryption Keys (DEK), with the capability to re-encrypt all stored data using new keys when existing keys expire or become compromised. Both shallow rekeying (replacing only KEKs via `CryptoSpecShallowRekey`) and deep rekeying (replacing all KEKs and DEKs via `CryptoSpecDeepRekey`) are available through the API. vSAN OSA and vSAN ESA both support these key rotation options using AES-256 bit encryption. AES-NI hardware acceleration significantly improves encryption performance, and administrators should enable AES-NI in the host BIOS.

The vCenter CryptoManager managed object provides programmatic control over cryptographic key operations in vSphere, including integration with the KMS for key creation, retrieval, and lifecycle management. This API supports both vSAN encryption and virtual machine encryption workflows, where symmetric keys are managed through the same KMS infrastructure. The vSphere Web Services API also provides `registerKmpServer()` to register a KMIP key management server with vCenter's cryptography manager.

VMware Kubernetes Service (VKS) clusters running on the vSphere Supervisor extend symmetric key management to the Kubernetes control plane through the Kubernetes KMS provider framework. The `kube-apiserver` supports `etcd` encryption-at-rest through an `EncryptionConfiguration` resource that specifies a KMS provider type with a Unix domain socket endpoint connecting to an external key management system. This framework uses an envelope encryption scheme: data stored in `etcd` is encrypted with a DEK, which is itself encrypted by a KEK held in the external KMS, keeping the root of trust outside the cluster boundary. Kubernetes recommends the KMS provider over storing raw encryption keys inline in the

EncryptionConfiguration file for this reason. KMS v2, stable from Kubernetes 1.29, uses the KmsV2Kdf feature gate to generate single-use DEKs from a secret seed combined with random data via a key derivation function, reducing call volume to the external KMS while maintaining per-object cryptographic isolation. KMS plugin implementations are required to verify the key\_id of any wrapped key before attempting decryption, which supports key versioning and rotation workflows. Key rotation requires an administrator to explicitly trigger re-encryption of stored secrets after a KEK rotation; VKS provides the technical hooks for this process. For environments requiring FIPS-validated cryptographic operations at the Kubernetes layer, VKS supports TanzuKubernetesRelease (TKR) images with FIPS cryptographic module support, such as the vmware.3-fips.1-tkg.1 release variants, which configure Kubernetes node binaries to use a FIPS-validated module for symmetric operations within the cluster. The external KMS must independently be a FIPS-validated product or hardware security module.

VCF components use FIPS-validated cryptographic modules across the stack. VCF supports FIPS 140-2 requirements for cryptographic modules. On the vCenter appliance, VMware's Boring Crypto Module version 6.0 provides FIPS-validated support for AES, ECDSA, HMAC, RSA, and SHA2 variants; the VMware OpenSSL FIPS Object Module (versions 2.0.20-vmw and 3.0) also covers DRBG, DSA, SHS, and Triple-DES algorithms. NSX is configured to use FIPS 140-3 validated cryptographic modules by default, with validation performed through the Cryptographic Module Validation Program (CMVP). NSX Manager uses FIPS 140 approved algorithms for authentication to cryptographic modules and generates unique session identifiers using a FIPS 140 approved random number generator. VCF Automation uses the BouncyCastle FIPS 2.0.0 cryptographic module, which is compliant with FIPS 140-3 (Certificate #4743). VCF Operations for Networks uses the BC-FJA (Bouncy Castle FIPS Java API) version 2.0.0 as its validated cryptographic module and can restrict cryptographic operations to FIPS-validated modules by enabling FIPS mode for external connections. VCF Operations for Networks also uses FIPS-compliant cryptographic algorithms for internal connections. VCF Operations orchestrator uses a FIPS-compliant AES with a 256-bit cipher key for encryption of strings.

VCF Operations supports a FIPS Security Mode that, when enabled, applies additional algorithm and cipher prerequisites across the platform. This mode restricts the cryptographic algorithms and ciphers available to those that meet FIPS validation requirements, including restricting key exchange algorithms to FIPS-approved options. This provides a platform-wide enforcement mechanism for FIPS-compliant symmetric key usage.

Two limitations apply in FIPS-sensitive environments. Self-signed certificates and private keys generated using OpenSSL are not FIPS-compatible; administrators should use FIPS-compatible key generation methods such as pyOpenSSL for PKCS#8 format. While NSX supports additional elliptic curve key types for certificates, non-FIPS-approved key types should not be used in environments requiring only FIPS-approved cryptographic keys.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) contributes to symmetric cryptographic key management through its requirements for consistent key management infrastructure across protected and recovery sites.

PNR and vSphere Replication require that either a KMIP-compatible Key Management Server (KMS) or the vSphere Native Key Provider be available at both the protected and recovery sites to protect and recover encrypted virtual machines. The KMS cluster must be registered with the same name on both sites. When using the vSphere Native Key Provider, it must be manually configured on both the local and remote sites. The vSphere Native Key Provider enables encryption-related functionality for virtual machines and virtual disks without requiring an external KMS.

PNR establishes a trusted connection between VMware vCenter and the KMS from which vCenter retrieves keys as needed to protect virtual machine disks and other files. This cross-site KMS consistency is a prerequisite for encrypted VM failover and failback through recovery plans.

The generation and lifecycle management of symmetric keys, including the key hierarchy, FIPS validation of cryptographic modules, key rotation scheduling, and TPM-based key persistence, are capabilities of the VCF platform. PNR's contribution is the enforcement of consistent KMS deployment across recovery topologies, which is a necessary condition for encrypted workload protection and recovery to function.

### **VMware Data Services Manager (Contributes)**

This control requires mechanisms for producing and managing symmetric cryptographic keys using FIPS-compliant key management technology and processes. VMware Data Services Manager (DSM) applies symmetric encryption across several operational contexts and exposes FIPS-related configuration options, but relies on the broader VCF platform and external tooling to satisfy the full scope of this requirement.

In DSM, the product enforces FIPS 140-3 compliant cryptographic standards and validates certificates during upload, rejecting unsupported algorithms or parameters. DSM specifically rejects the Ed25519 elliptic curve algorithm because it is not FIPS 140-3 compliant. Accepted key algorithms for custom certificates include ECDSA P-256, ECDSA P-384, and RSA keys with a minimum size of 2048 bits. This enforcement is documented in "Using Trusted Certificates in Data Services Manager > Configuring Custom Certificates for Data Services Manager > Supported Cryptographic Algorithms."

For MySQL databases managed by DSM, the `ssl_fips_mode` setting activates FIPS mode on the client side, restricting cipher negotiation to FIPS-approved algorithms. This setting is documented in "Databases in VMware Data Services Manager > Using MySQL in Data Services Manager > Configuration."

DSM also applies symmetric encryption in its user management subsystem. User passwords are stored encrypted using PGCrypto symmetric encryption with dedicated encryption keys, as documented in "Administering Data Services Manager > Managing LDAP Users in Data Services Manager." PostgreSQL databases managed by DSM include the pgcrypto extension version 1.3, which provides cryptographic functions available to database workloads.

The backup subsystem uses AES-256 to encrypt backup data written to S3-compatible storage, consistent with FIPS-approved algorithm requirements.

What DSM does not provide is a standalone FIPS-validated key management module or integration with an external key management service (KMS). The production, lifecycle management, rotation, and revocation of symmetric keys at the organizational level requires an external FIPS-validated KMS or hardware security module (HSM) operating at the infrastructure layer. DSM consumes keys and applies FIPS-approved symmetric algorithms, but the key generation and management controls for those keys are the responsibility of the organization and the underlying platform. Auditors should verify that the VCF infrastructure supporting DSM is configured with an appropriate FIPS-validated key provider, and that the organization has key management processes that cover the DSM-managed symmetric keys used for backups and internal encryption.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) supports a FIPS 140-3 compliance mode that uses OpenSSL FIPS Provider version 3.1.2 for all cryptographic operations. The OpenSSL FIPS Object Module v3.1.2 has been validated at FIPS 140-3 Security Level 1 and awarded Certificate #4985 by the Cryptographic Module Validation Program (CMVP). When FIPS mode is enabled on the Avi Controller and Service Engines, only cryptographic algorithms approved under FIPS 140-3 are used. FIPS 140-3 requires stricter handling of keys, authentication data, and random number generation than its predecessor, and aligns cryptographic module standards with international standards ISO/IEC 19790:2012 and ISO/IEC 24759:2017. This covers the symmetric encryption operations involved in TLS termination, re-encryption between Service Engines and backend servers, and internal control-plane communication. FIPS mode is configured at deployment time for Controller cluster deployments.

SSL/TLS Profile Templates in Avi give administrators control over the cipher suites and protocol versions used by virtual services. These profiles specify a prioritized cipher list and accepted protocol versions per virtual service and are managed under Templates > Security > SSL/TLS Profile in the Avi Controller UI. Configuring a FIPS-aligned SSL/TLS Profile and applying it to virtual services restricts symmetric key negotiation and cipher use across all load-balanced application traffic to FIPS-approved algorithms.

Post-quantum cryptography (PQC)-based Key Encapsulation Mechanisms and Signature Algorithms are not supported in Avi deployments running in FIPS mode.

For organizations deploying Avi in public cloud environments, Avi integrates with cloud-native key management services. In AWS, Avi's Cloud Connector integrates with AWS KMS for Service Engine disk and SQS queue encryption, supporting customer-managed keys with configurable IAM and key policies; the Avi Controller can list KMS encryption keys in the Avi UI

and supports S3 and EBS encryption. In Google Cloud Platform, Avi supports Google Cloud KMS for Service Engine disk and image encryption, GCS bucket encryption, and GCS object encryption, using separate KMS keys for each resource type.

The Avi Kubernetes Operator (AKO) 2.1 adds support for configuring both RSA and ECC signed certificates simultaneously on a single virtual service through the alt.crt and alt.key fields in the default TLS secret, enabling certificate algorithm flexibility for Kubernetes ingress workloads.

Not applicable for this control: VMware vDefend. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 1.14(b)

> Sub-item (b) of CR 1.14 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 1.14 requirement in the standard.

**SCF Controls:** CRY-09.1

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides integrated symmetric key management capabilities through its Key Management Server (KMS) infrastructure and the VMware vCenter CryptoManager interface. VCF requires that the KMS support the Key Management Interoperability Protocol (KMIP) 1.1 standard and that it be capable of acting as a symmetric key foundry and server. The KMS must also appear in the VMware Compatibility Guide for Key Management Servers before it can be used in a VCF environment. Alternatively, the vSphere Native Key Provider can manage symmetric keys without requiring an external KMS and can coexist with existing key server infrastructure. This architecture centralizes symmetric key production and lifecycle management through a standards-based protocol, with vCenter as the orchestration layer between VCF components and the key provider.

VCF uses a key hierarchy for encryption operations. Key Encryption Keys (KEK) wrap Disk Encryption Keys (DEK), with DEKs using XTS-AES-256. VMware ESX hosts use the KEK to encrypt their internal keys and store only the encrypted internal keys on disk; the KEK itself is not persisted locally. Internal encryption keys are kept in memory only during operation, with only the encrypted data stored on disk. This separation of key tiers limits the exposure of key material at rest.

vSAN data-at-rest encryption relies on KMS integration for all symmetric key operations. Enabling vSAN encryption requires selecting a KMS cluster during configuration, establishing a trust relationship with the KMS by uploading certificate and key files, and confirming that the KMS is healthy before encryption can proceed. Administrators should deploy KMS servers on infrastructure separate from the vSAN datastore being encrypted and should use highly available KMS clusters to distribute the keys that protect vSAN datastores. Some KMS vendors support credential-based isolation of encryption keys, allowing administrators to specify a username and password during vSAN encryption configuration for key partitioning. An optional proxy port configuration is also available for KMS connectivity. vSAN also supports the vSphere Native Key Provider for encrypting virtual machines in vSAN clusters.

vSAN data-in-transit encryption uses symmetric keys that are generated dynamically and shared between hosts to encrypt inter-host traffic. The rekey interval for data-in-transit encryption is configurable, with a default of 1,440 minutes (24 hours) and a supported range of 30 minutes to 10,080 minutes (7 days). For data-at-rest encryption, vSAN supports generating new Key Encryption Keys (KEK) and new Data Encryption Keys (DEK), with the capability to re-encrypt all stored data using new keys when existing keys expire or become compromised. Both shallow rekeying (replacing only KEKs via `CryptoSpecShallowRekey`) and deep rekeying (replacing all KEKs and DEKs via `CryptoSpecDeepRekey`) are available through the API. vSAN OSA and vSAN ESA both support these key rotation options using AES-256 bit encryption. AES-NI hardware acceleration significantly improves encryption performance, and administrators should enable AES-NI in the host BIOS.

The vCenter CryptoManager managed object provides programmatic control over cryptographic key operations in vSphere, including integration with the KMS for key creation, retrieval, and lifecycle management. This API supports both vSAN encryption and virtual machine encryption workflows, where symmetric keys are managed through the same KMS infrastructure. The vSphere Web Services API also provides `registerKmpServer()` to register a KMIP key management server with vCenter's cryptography manager.

VMware Kubernetes Service (VKS) clusters running on the vSphere Supervisor extend symmetric key management to the Kubernetes control plane through the Kubernetes KMS provider framework. The `kube-apiserver` supports `etcd` encryption-at-rest through an `EncryptionConfiguration` resource that specifies a KMS provider type with a Unix domain socket endpoint connecting to an external key management system. This framework uses an envelope encryption scheme: data stored in `etcd` is encrypted with a DEK, which is itself encrypted by a KEK held in the external KMS, keeping the root of trust outside the cluster boundary. Kubernetes recommends the KMS provider over storing raw encryption keys inline in the



EncryptionConfiguration file for this reason. KMS v2, stable from Kubernetes 1.29, uses the KmsV2Kdf feature gate to generate single-use DEKs from a secret seed combined with random data via a key derivation function, reducing call volume to the external KMS while maintaining per-object cryptographic isolation. KMS plugin implementations are required to verify the key\_id of any wrapped key before attempting decryption, which supports key versioning and rotation workflows. Key rotation requires an administrator to explicitly trigger re-encryption of stored secrets after a KEK rotation; VKS provides the technical hooks for this process. For environments requiring FIPS-validated cryptographic operations at the Kubernetes layer, VKS supports TanzuKubernetesRelease (TKR) images with FIPS cryptographic module support, such as the vmware.3-fips.1-tkg.1 release variants, which configure Kubernetes node binaries to use a FIPS-validated module for symmetric operations within the cluster. The external KMS must independently be a FIPS-validated product or hardware security module.

VCF components use FIPS-validated cryptographic modules across the stack. VCF supports FIPS 140-2 requirements for cryptographic modules. On the vCenter appliance, VMware's Boring Crypto Module version 6.0 provides FIPS-validated support for AES, ECDSA, HMAC, RSA, and SHA2 variants; the VMware OpenSSL FIPS Object Module (versions 2.0.20-vmw and 3.0) also covers DRBG, DSA, SHS, and Triple-DES algorithms. NSX is configured to use FIPS 140-3 validated cryptographic modules by default, with validation performed through the Cryptographic Module Validation Program (CMVP). NSX Manager uses FIPS 140 approved algorithms for authentication to cryptographic modules and generates unique session identifiers using a FIPS 140 approved random number generator. VCF Automation uses the BouncyCastle FIPS 2.0.0 cryptographic module, which is compliant with FIPS 140-3 (Certificate #4743). VCF Operations for Networks uses the BC-FJA (Bouncy Castle FIPS Java API) version 2.0.0 as its validated cryptographic module and can restrict cryptographic operations to FIPS-validated modules by enabling FIPS mode for external connections. VCF Operations for Networks also uses FIPS-compliant cryptographic algorithms for internal connections. VCF Operations orchestrator uses a FIPS-compliant AES with a 256-bit cipher key for encryption of strings.

VCF Operations supports a FIPS Security Mode that, when enabled, applies additional algorithm and cipher prerequisites across the platform. This mode restricts the cryptographic algorithms and ciphers available to those that meet FIPS validation requirements, including restricting key exchange algorithms to FIPS-approved options. This provides a platform-wide enforcement mechanism for FIPS-compliant symmetric key usage.

Two limitations apply in FIPS-sensitive environments. Self-signed certificates and private keys generated using OpenSSL are not FIPS-compatible; administrators should use FIPS-compatible key generation methods such as pyOpenSSL for PKCS#8 format. While NSX supports additional elliptic curve key types for certificates, non-FIPS-approved key types should not be used in environments requiring only FIPS-approved cryptographic keys.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) contributes to symmetric cryptographic key management through its requirements for consistent key management infrastructure across protected and recovery sites.

PNR and vSphere Replication require that either a KMIP-compatible Key Management Server (KMS) or the vSphere Native Key Provider be available at both the protected and recovery sites to protect and recover encrypted virtual machines. The KMS cluster must be registered with the same name on both sites. When using the vSphere Native Key Provider, it must be manually configured on both the local and remote sites. The vSphere Native Key Provider enables encryption-related functionality for virtual machines and virtual disks without requiring an external KMS.

PNR establishes a trusted connection between VMware vCenter and the KMS from which vCenter retrieves keys as needed to protect virtual machine disks and other files. This cross-site KMS consistency is a prerequisite for encrypted VM failover and failback through recovery plans.

The generation and lifecycle management of symmetric keys, including the key hierarchy, FIPS validation of cryptographic modules, key rotation scheduling, and TPM-based key persistence, are capabilities of the VCF platform. PNR's contribution is the enforcement of consistent KMS deployment across recovery topologies, which is a necessary condition for encrypted workload protection and recovery to function.

### **VMware Data Services Manager (Contributes)**

This control requires mechanisms for producing and managing symmetric cryptographic keys using FIPS-compliant key management technology and processes. VMware Data Services Manager (DSM) applies symmetric encryption across several operational contexts and exposes FIPS-related configuration options, but relies on the broader VCF platform and external tooling to satisfy the full scope of this requirement.

In DSM, the product enforces FIPS 140-3 compliant cryptographic standards and validates certificates during upload, rejecting unsupported algorithms or parameters. DSM specifically rejects the Ed25519 elliptic curve algorithm because it is not FIPS 140-3 compliant. Accepted key algorithms for custom certificates include ECDSA P-256, ECDSA P-384, and RSA keys with a minimum size of 2048 bits. This enforcement is documented in "Using Trusted Certificates in Data Services Manager > Configuring Custom Certificates for Data Services Manager > Supported Cryptographic Algorithms."

For MySQL databases managed by DSM, the `ssl_fips_mode` setting activates FIPS mode on the client side, restricting cipher negotiation to FIPS-approved algorithms. This setting is documented in "Databases in VMware Data Services Manager > Using MySQL in Data Services Manager > Configuration."

DSM also applies symmetric encryption in its user management subsystem. User passwords are stored encrypted using PGCrypto symmetric encryption with dedicated encryption keys, as documented in "Administering Data Services Manager > Managing LDAP Users in Data Services Manager." PostgreSQL databases managed by DSM include the pgcrypto extension version 1.3, which provides cryptographic functions available to database workloads.

The backup subsystem uses AES-256 to encrypt backup data written to S3-compatible storage, consistent with FIPS-approved algorithm requirements.

What DSM does not provide is a standalone FIPS-validated key management module or integration with an external key management service (KMS). The production, lifecycle management, rotation, and revocation of symmetric keys at the organizational level requires an external FIPS-validated KMS or hardware security module (HSM) operating at the infrastructure layer. DSM consumes keys and applies FIPS-approved symmetric algorithms, but the key generation and management controls for those keys are the responsibility of the organization and the underlying platform. Auditors should verify that the VCF infrastructure supporting DSM is configured with an appropriate FIPS-validated key provider, and that the organization has key management processes that cover the DSM-managed symmetric keys used for backups and internal encryption.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) supports a FIPS 140-3 compliance mode that uses OpenSSL FIPS Provider version 3.1.2 for all cryptographic operations. The OpenSSL FIPS Object Module v3.1.2 has been validated at FIPS 140-3 Security Level 1 and awarded Certificate #4985 by the Cryptographic Module Validation Program (CMVP). When FIPS mode is enabled on the Avi Controller and Service Engines, only cryptographic algorithms approved under FIPS 140-3 are used. FIPS 140-3 requires stricter handling of keys, authentication data, and random number generation than its predecessor, and aligns cryptographic module standards with international standards ISO/IEC 19790:2012 and ISO/IEC 24759:2017. This covers the symmetric encryption operations involved in TLS termination, re-encryption between Service Engines and backend servers, and internal control-plane communication. FIPS mode is configured at deployment time for Controller cluster deployments.

SSL/TLS Profile Templates in Avi give administrators control over the cipher suites and protocol versions used by virtual services. These profiles specify a prioritized cipher list and accepted protocol versions per virtual service and are managed under Templates > Security > SSL/TLS Profile in the Avi Controller UI. Configuring a FIPS-aligned SSL/TLS Profile and applying it to virtual services restricts symmetric key negotiation and cipher use across all load-balanced application traffic to FIPS-approved algorithms.

Post-quantum cryptography (PQC)-based Key Encapsulation Mechanisms and Signature Algorithms are not supported in Avi deployments running in FIPS mode.

For organizations deploying Avi in public cloud environments, Avi integrates with cloud-native key management services. In AWS, Avi's Cloud Connector integrates with AWS KMS for Service Engine disk and SQS queue encryption, supporting customer-managed keys with configurable IAM and key policies; the Avi Controller can list KMS encryption keys in the Avi UI

and supports S3 and EBS encryption. In Google Cloud Platform, Avi supports Google Cloud KMS for Service Engine disk and image encryption, GCS bucket encryption, and GCS object encryption, using separate KMS keys for each resource type.

The Avi Kubernetes Operator (AKO) 2.1 adds support for configuring both RSA and ECC signed certificates simultaneously on a single virtual service through the `alt.crt` and `alt.key` fields in the default TLS secret, enabling certificate algorithm flexibility for Kubernetes ingress workloads.

Not applicable for this control: VMware vDefend. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 1.14(c)

> Sub-item (c) of CR 1.14 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 1.14 requirement in the standard.

**SCF Controls:** CRY-09.1

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides integrated symmetric key management capabilities through its Key Management Server (KMS) infrastructure and the VMware vCenter CryptoManager interface. VCF requires that the KMS support the Key Management Interoperability Protocol (KMIP) 1.1 standard and that it be capable of acting as a symmetric key foundry and server. The KMS must also appear in the VMware Compatibility Guide for Key Management Servers before it can be used in a VCF environment. Alternatively, the vSphere Native Key Provider can manage symmetric keys without requiring an external KMS and can coexist with existing key server infrastructure. This architecture centralizes symmetric key production and lifecycle management through a standards-based protocol, with vCenter as the orchestration layer between VCF components and the key provider.

VCF uses a key hierarchy for encryption operations. Key Encryption Keys (KEK) wrap Disk Encryption Keys (DEK), with DEKs using XTS-AES-256. VMware ESX hosts use the KEK to encrypt their internal keys and store only the encrypted internal keys on disk; the KEK itself is not persisted locally. Internal encryption keys are kept in memory only during operation, with only the encrypted data stored on disk. This separation of key tiers limits the exposure of key material at rest.

vSAN data-at-rest encryption relies on KMS integration for all symmetric key operations. Enabling vSAN encryption requires selecting a KMS cluster during configuration, establishing a trust relationship with the KMS by uploading certificate and key files, and confirming that the KMS is healthy before encryption can proceed. Administrators should deploy KMS servers on infrastructure separate from the vSAN datastore being encrypted and should use highly available KMS clusters to distribute the keys that protect vSAN datastores. Some KMS vendors support credential-based isolation of encryption keys, allowing administrators to specify a username and password during vSAN encryption configuration for key partitioning. An optional proxy port configuration is also available for KMS connectivity. vSAN also supports the vSphere Native Key Provider for encrypting virtual machines in vSAN clusters.

vSAN data-in-transit encryption uses symmetric keys that are generated dynamically and shared between hosts to encrypt inter-host traffic. The rekey interval for data-in-transit encryption is configurable, with a default of 1,440 minutes (24 hours) and a supported range of 30 minutes to 10,080 minutes (7 days). For data-at-rest encryption, vSAN supports generating new Key Encryption Keys (KEK) and new Data Encryption Keys (DEK), with the capability to re-encrypt all stored data using new keys when existing keys expire or become compromised. Both shallow reencryption (replacing only KEKs via `CryptoSpecShallowReencrypt`) and deep reencryption (replacing all KEKs and DEKs via `CryptoSpecDeepReencrypt`) are available through the API. vSAN OSA and vSAN ESA both support these key rotation options using AES-256 bit encryption. AES-NI hardware acceleration significantly improves encryption performance, and administrators should enable AES-NI in the host BIOS.

The vCenter CryptoManager managed object provides programmatic control over cryptographic key operations in vSphere, including integration with the KMS for key creation, retrieval, and lifecycle management. This API supports both vSAN encryption and virtual machine encryption workflows, where symmetric keys are managed through the same KMS infrastructure. The vSphere Web Services API also provides `registerKmipServer()` to register a KMIP key management server with vCenter's cryptography manager.

VMware Kubernetes Service (VKS) clusters running on the vSphere Supervisor extend symmetric key management to the Kubernetes control plane through the Kubernetes KMS provider framework. The `kube-apiserver` supports `etcd` encryption-at-rest through an `EncryptionConfiguration` resource that specifies a KMS provider type with a Unix domain socket endpoint connecting to an external key management system. This framework uses an envelope encryption scheme: data stored in `etcd` is encrypted with a DEK, which is itself encrypted by a KEK held in the external KMS, keeping the root of trust outside the cluster boundary. Kubernetes recommends the KMS provider over storing raw encryption keys inline in the

EncryptionConfiguration file for this reason. KMS v2, stable from Kubernetes 1.29, uses the KmsV2Kdf feature gate to generate single-use DEKs from a secret seed combined with random data via a key derivation function, reducing call volume to the external KMS while maintaining per-object cryptographic isolation. KMS plugin implementations are required to verify the key\_id of any wrapped key before attempting decryption, which supports key versioning and rotation workflows. Key rotation requires an administrator to explicitly trigger re-encryption of stored secrets after a KEK rotation; VKS provides the technical hooks for this process. For environments requiring FIPS-validated cryptographic operations at the Kubernetes layer, VKS supports TanzuKubernetesRelease (TKR) images with FIPS cryptographic module support, such as the vmware.3-fips.1-tkg.1 release variants, which configure Kubernetes node binaries to use a FIPS-validated module for symmetric operations within the cluster. The external KMS must independently be a FIPS-validated product or hardware security module.

VCF components use FIPS-validated cryptographic modules across the stack. VCF supports FIPS 140-2 requirements for cryptographic modules. On the vCenter appliance, VMware's Boring Crypto Module version 6.0 provides FIPS-validated support for AES, ECDSA, HMAC, RSA, and SHA2 variants; the VMware OpenSSL FIPS Object Module (versions 2.0.20-vmw and 3.0) also covers DRBG, DSA, SHS, and Triple-DES algorithms. NSX is configured to use FIPS 140-3 validated cryptographic modules by default, with validation performed through the Cryptographic Module Validation Program (CMVP). NSX Manager uses FIPS 140 approved algorithms for authentication to cryptographic modules and generates unique session identifiers using a FIPS 140 approved random number generator. VCF Automation uses the BouncyCastle FIPS 2.0.0 cryptographic module, which is compliant with FIPS 140-3 (Certificate #4743). VCF Operations for Networks uses the BC-FJA (Bouncy Castle FIPS Java API) version 2.0.0 as its validated cryptographic module and can restrict cryptographic operations to FIPS-validated modules by enabling FIPS mode for external connections. VCF Operations for Networks also uses FIPS-compliant cryptographic algorithms for internal connections. VCF Operations orchestrator uses a FIPS-compliant AES with a 256-bit cipher key for encryption of strings.

VCF Operations supports a FIPS Security Mode that, when enabled, applies additional algorithm and cipher prerequisites across the platform. This mode restricts the cryptographic algorithms and ciphers available to those that meet FIPS validation requirements, including restricting key exchange algorithms to FIPS-approved options. This provides a platform-wide enforcement mechanism for FIPS-compliant symmetric key usage.

Two limitations apply in FIPS-sensitive environments. Self-signed certificates and private keys generated using OpenSSL are not FIPS-compatible; administrators should use FIPS-compatible key generation methods such as pyOpenSSL for PKCS#8 format. While NSX supports additional elliptic curve key types for certificates, non-FIPS-approved key types should not be used in environments requiring only FIPS-approved cryptographic keys.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) contributes to symmetric cryptographic key management through its requirements for consistent key management infrastructure across protected and recovery sites.

PNR and vSphere Replication require that either a KMIP-compatible Key Management Server (KMS) or the vSphere Native Key Provider be available at both the protected and recovery sites to protect and recover encrypted virtual machines. The KMS cluster must be registered with the same name on both sites. When using the vSphere Native Key Provider, it must be manually configured on both the local and remote sites. The vSphere Native Key Provider enables encryption-related functionality for virtual machines and virtual disks without requiring an external KMS.

PNR establishes a trusted connection between VMware vCenter and the KMS from which vCenter retrieves keys as needed to protect virtual machine disks and other files. This cross-site KMS consistency is a prerequisite for encrypted VM failover and failback through recovery plans.

The generation and lifecycle management of symmetric keys, including the key hierarchy, FIPS validation of cryptographic modules, key rotation scheduling, and TPM-based key persistence, are capabilities of the VCF platform. PNR's contribution is the enforcement of consistent KMS deployment across recovery topologies, which is a necessary condition for encrypted workload protection and recovery to function.

### **VMware Data Services Manager (Contributes)**

This control requires mechanisms for producing and managing symmetric cryptographic keys using FIPS-compliant key management technology and processes. VMware Data Services Manager (DSM) applies symmetric encryption across several operational contexts and exposes FIPS-related configuration options, but relies on the broader VCF platform and external tooling to satisfy the full scope of this requirement.

In DSM, the product enforces FIPS 140-3 compliant cryptographic standards and validates certificates during upload, rejecting unsupported algorithms or parameters. DSM specifically rejects the Ed25519 elliptic curve algorithm because it is not FIPS 140-3 compliant. Accepted key algorithms for custom certificates include ECDSA P-256, ECDSA P-384, and RSA keys with a minimum size of 2048 bits. This enforcement is documented in "Using Trusted Certificates in Data Services Manager > Configuring Custom Certificates for Data Services Manager > Supported Cryptographic Algorithms."

For MySQL databases managed by DSM, the `ssl_fips_mode` setting activates FIPS mode on the client side, restricting cipher negotiation to FIPS-approved algorithms. This setting is documented in "Databases in VMware Data Services Manager > Using MySQL in Data Services Manager > Configuration."

DSM also applies symmetric encryption in its user management subsystem. User passwords are stored encrypted using PGCrypto symmetric encryption with dedicated encryption keys, as documented in "Administering Data Services Manager > Managing LDAP Users in Data Services Manager." PostgreSQL databases managed by DSM include the pgcrypto extension version 1.3, which provides cryptographic functions available to database workloads.

The backup subsystem uses AES-256 to encrypt backup data written to S3-compatible storage, consistent with FIPS-approved algorithm requirements.

What DSM does not provide is a standalone FIPS-validated key management module or integration with an external key management service (KMS). The production, lifecycle management, rotation, and revocation of symmetric keys at the organizational level requires an external FIPS-validated KMS or hardware security module (HSM) operating at the infrastructure layer. DSM consumes keys and applies FIPS-approved symmetric algorithms, but the key generation and management controls for those keys are the responsibility of the organization and the underlying platform. Auditors should verify that the VCF infrastructure supporting DSM is configured with an appropriate FIPS-validated key provider, and that the organization has key management processes that cover the DSM-managed symmetric keys used for backups and internal encryption.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) supports a FIPS 140-3 compliance mode that uses OpenSSL FIPS Provider version 3.1.2 for all cryptographic operations. The OpenSSL FIPS Object Module v3.1.2 has been validated at FIPS 140-3 Security Level 1 and awarded Certificate #4985 by the Cryptographic Module Validation Program (CMVP). When FIPS mode is enabled on the Avi Controller and Service Engines, only cryptographic algorithms approved under FIPS 140-3 are used. FIPS 140-3 requires stricter handling of keys, authentication data, and random number generation than its predecessor, and aligns cryptographic module standards with international standards ISO/IEC 19790:2012 and ISO/IEC 24759:2017. This covers the symmetric encryption operations involved in TLS termination, re-encryption between Service Engines and backend servers, and internal control-plane communication. FIPS mode is configured at deployment time for Controller cluster deployments.

SSL/TLS Profile Templates in Avi give administrators control over the cipher suites and protocol versions used by virtual services. These profiles specify a prioritized cipher list and accepted protocol versions per virtual service and are managed under Templates > Security > SSL/TLS Profile in the Avi Controller UI. Configuring a FIPS-aligned SSL/TLS Profile and applying it to virtual services restricts symmetric key negotiation and cipher use across all load-balanced application traffic to FIPS-approved algorithms.

Post-quantum cryptography (PQC)-based Key Encapsulation Mechanisms and Signature Algorithms are not supported in Avi deployments running in FIPS mode.

For organizations deploying Avi in public cloud environments, Avi integrates with cloud-native key management services. In AWS, Avi's Cloud Connector integrates with AWS KMS for Service Engine disk and SQS queue encryption, supporting customer-managed keys with configurable IAM and key policies; the Avi Controller can list KMS encryption keys in the Avi UI

and supports S3 and EBS encryption. In Google Cloud Platform, Avi supports Google Cloud KMS for Service Engine disk and image encryption, GCS bucket encryption, and GCS object encryption, using separate KMS keys for each resource type.

The Avi Kubernetes Operator (AKO) 2.1 adds support for configuring both RSA and ECC signed certificates simultaneously on a single virtual service through the `alt.crt` and `alt.key` fields in the default TLS secret, enabling certificate algorithm flexibility for Kubernetes ingress workloads.

Not applicable for this control: VMware vDefend. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.



## Control CR 1.14(d)

> Sub-item (d) of CR 1.14 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 1.14 requirement in the standard.

**SCF Controls:** CRY-09.1

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides integrated symmetric key management capabilities through its Key Management Server (KMS) infrastructure and the VMware vCenter CryptoManager interface. VCF requires that the KMS support the Key Management Interoperability Protocol (KMIP) 1.1 standard and that it be capable of acting as a symmetric key foundry and server. The KMS must also appear in the VMware Compatibility Guide for Key Management Servers before it can be used in a VCF environment. Alternatively, the vSphere Native Key Provider can manage symmetric keys without requiring an external KMS and can coexist with existing key server infrastructure. This architecture centralizes symmetric key production and lifecycle management through a standards-based protocol, with vCenter as the orchestration layer between VCF components and the key provider.

VCF uses a key hierarchy for encryption operations. Key Encryption Keys (KEK) wrap Disk Encryption Keys (DEK), with DEKs using XTS-AES-256. VMware ESX hosts use the KEK to encrypt their internal keys and store only the encrypted internal keys on disk; the KEK itself is not persisted locally. Internal encryption keys are kept in memory only during operation, with only the encrypted data stored on disk. This separation of key tiers limits the exposure of key material at rest.

vSAN data-at-rest encryption relies on KMS integration for all symmetric key operations. Enabling vSAN encryption requires selecting a KMS cluster during configuration, establishing a trust relationship with the KMS by uploading certificate and key files, and confirming that the KMS is healthy before encryption can proceed. Administrators should deploy KMS servers on infrastructure separate from the vSAN datastore being encrypted and should use highly available KMS clusters to distribute the keys that protect vSAN datastores. Some KMS vendors support credential-based isolation of encryption keys, allowing administrators to specify a username and password during vSAN encryption configuration for key partitioning. An optional proxy port configuration is also available for KMS connectivity. vSAN also supports the vSphere Native Key Provider for encrypting virtual machines in vSAN clusters.

vSAN data-in-transit encryption uses symmetric keys that are generated dynamically and shared between hosts to encrypt inter-host traffic. The rekey interval for data-in-transit encryption is configurable, with a default of 1,440 minutes (24 hours) and a supported range of 30 minutes to 10,080 minutes (7 days). For data-at-rest encryption, vSAN supports generating new Key Encryption Keys (KEK) and new Data Encryption Keys (DEK), with the capability to re-encrypt all stored data using new keys when existing keys expire or become compromised. Both shallow rekeying (replacing only KEKs via `CryptoSpecShallowRekey`) and deep rekeying (replacing all KEKs and DEKs via `CryptoSpecDeepRekey`) are available through the API. vSAN OSA and vSAN ESA both support these key rotation options using AES-256 bit encryption. AES-NI hardware acceleration significantly improves encryption performance, and administrators should enable AES-NI in the host BIOS.

The vCenter CryptoManager managed object provides programmatic control over cryptographic key operations in vSphere, including integration with the KMS for key creation, retrieval, and lifecycle management. This API supports both vSAN encryption and virtual machine encryption workflows, where symmetric keys are managed through the same KMS infrastructure. The vSphere Web Services API also provides `registerKmpServer()` to register a KMIP key management server with vCenter's cryptography manager.

VMware Kubernetes Service (VKS) clusters running on the vSphere Supervisor extend symmetric key management to the Kubernetes control plane through the Kubernetes KMS provider framework. The kube-apiserver supports etcd encryption-at-rest through an `EncryptionConfiguration` resource that specifies a KMS provider type with a Unix domain socket endpoint connecting to an external key management system. This framework uses an envelope encryption scheme: data stored in etcd is encrypted with a DEK, which is itself encrypted by a KEK held in the external KMS, keeping the root of trust outside the cluster boundary. Kubernetes recommends the KMS provider over storing raw encryption keys inline in the

EncryptionConfiguration file for this reason. KMS v2, stable from Kubernetes 1.29, uses the KmsV2Kdf feature gate to generate single-use DEKs from a secret seed combined with random data via a key derivation function, reducing call volume to the external KMS while maintaining per-object cryptographic isolation. KMS plugin implementations are required to verify the key\_id of any wrapped key before attempting decryption, which supports key versioning and rotation workflows. Key rotation requires an administrator to explicitly trigger re-encryption of stored secrets after a KEK rotation; VKS provides the technical hooks for this process. For environments requiring FIPS-validated cryptographic operations at the Kubernetes layer, VKS supports TanzuKubernetesRelease (TKR) images with FIPS cryptographic module support, such as the vmware.3-fips.1-tkg.1 release variants, which configure Kubernetes node binaries to use a FIPS-validated module for symmetric operations within the cluster. The external KMS must independently be a FIPS-validated product or hardware security module.

VCF components use FIPS-validated cryptographic modules across the stack. VCF supports FIPS 140-2 requirements for cryptographic modules. On the vCenter appliance, VMware's Boring Crypto Module version 6.0 provides FIPS-validated support for AES, ECDSA, HMAC, RSA, and SHA2 variants; the VMware OpenSSL FIPS Object Module (versions 2.0.20-vmw and 3.0) also covers DRBG, DSA, SHS, and Triple-DES algorithms. NSX is configured to use FIPS 140-3 validated cryptographic modules by default, with validation performed through the Cryptographic Module Validation Program (CMVP). NSX Manager uses FIPS 140 approved algorithms for authentication to cryptographic modules and generates unique session identifiers using a FIPS 140 approved random number generator. VCF Automation uses the BouncyCastle FIPS 2.0.0 cryptographic module, which is compliant with FIPS 140-3 (Certificate #4743). VCF Operations for Networks uses the BC-FJA (Bouncy Castle FIPS Java API) version 2.0.0 as its validated cryptographic module and can restrict cryptographic operations to FIPS-validated modules by enabling FIPS mode for external connections. VCF Operations for Networks also uses FIPS-compliant cryptographic algorithms for internal connections. VCF Operations orchestrator uses a FIPS-compliant AES with a 256-bit cipher key for encryption of strings.

VCF Operations supports a FIPS Security Mode that, when enabled, applies additional algorithm and cipher prerequisites across the platform. This mode restricts the cryptographic algorithms and ciphers available to those that meet FIPS validation requirements, including restricting key exchange algorithms to FIPS-approved options. This provides a platform-wide enforcement mechanism for FIPS-compliant symmetric key usage.

Two limitations apply in FIPS-sensitive environments. Self-signed certificates and private keys generated using OpenSSL are not FIPS-compatible; administrators should use FIPS-compatible key generation methods such as pyOpenSSL for PKCS#8 format. While NSX supports additional elliptic curve key types for certificates, non-FIPS-approved key types should not be used in environments requiring only FIPS-approved cryptographic keys.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) contributes to symmetric cryptographic key management through its requirements for consistent key management infrastructure across protected and recovery sites.

PNR and vSphere Replication require that either a KMIP-compatible Key Management Server (KMS) or the vSphere Native Key Provider be available at both the protected and recovery sites to protect and recover encrypted virtual machines. The KMS cluster must be registered with the same name on both sites. When using the vSphere Native Key Provider, it must be manually configured on both the local and remote sites. The vSphere Native Key Provider enables encryption-related functionality for virtual machines and virtual disks without requiring an external KMS.

PNR establishes a trusted connection between VMware vCenter and the KMS from which vCenter retrieves keys as needed to protect virtual machine disks and other files. This cross-site KMS consistency is a prerequisite for encrypted VM failover and failback through recovery plans.

The generation and lifecycle management of symmetric keys, including the key hierarchy, FIPS validation of cryptographic modules, key rotation scheduling, and TPM-based key persistence, are capabilities of the VCF platform. PNR's contribution is the enforcement of consistent KMS deployment across recovery topologies, which is a necessary condition for encrypted workload protection and recovery to function.

### **VMware Data Services Manager (Contributes)**

This control requires mechanisms for producing and managing symmetric cryptographic keys using FIPS-compliant key management technology and processes. VMware Data Services Manager (DSM) applies symmetric encryption across several operational contexts and exposes FIPS-related configuration options, but relies on the broader VCF platform and external tooling to satisfy the full scope of this requirement.

In DSM, the product enforces FIPS 140-3 compliant cryptographic standards and validates certificates during upload, rejecting unsupported algorithms or parameters. DSM specifically rejects the Ed25519 elliptic curve algorithm because it is not FIPS 140-3 compliant. Accepted key algorithms for custom certificates include ECDSA P-256, ECDSA P-384, and RSA keys with a minimum size of 2048 bits. This enforcement is documented in "Using Trusted Certificates in Data Services Manager > Configuring Custom Certificates for Data Services Manager > Supported Cryptographic Algorithms."

For MySQL databases managed by DSM, the `ssl_fips_mode` setting activates FIPS mode on the client side, restricting cipher negotiation to FIPS-approved algorithms. This setting is documented in "Databases in VMware Data Services Manager > Using MySQL in Data Services Manager > Configuration."

DSM also applies symmetric encryption in its user management subsystem. User passwords are stored encrypted using PGCrypto symmetric encryption with dedicated encryption keys, as documented in "Administering Data Services Manager > Managing LDAP Users in Data Services Manager." PostgreSQL databases managed by DSM include the pgcrypto extension version 1.3, which provides cryptographic functions available to database workloads.

The backup subsystem uses AES-256 to encrypt backup data written to S3-compatible storage, consistent with FIPS-approved algorithm requirements.

What DSM does not provide is a standalone FIPS-validated key management module or integration with an external key management service (KMS). The production, lifecycle management, rotation, and revocation of symmetric keys at the organizational level requires an external FIPS-validated KMS or hardware security module (HSM) operating at the infrastructure layer. DSM consumes keys and applies FIPS-approved symmetric algorithms, but the key generation and management controls for those keys are the responsibility of the organization and the underlying platform. Auditors should verify that the VCF infrastructure supporting DSM is configured with an appropriate FIPS-validated key provider, and that the organization has key management processes that cover the DSM-managed symmetric keys used for backups and internal encryption.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) supports a FIPS 140-3 compliance mode that uses OpenSSL FIPS Provider version 3.1.2 for all cryptographic operations. The OpenSSL FIPS Object Module v3.1.2 has been validated at FIPS 140-3 Security Level 1 and awarded Certificate #4985 by the Cryptographic Module Validation Program (CMVP). When FIPS mode is enabled on the Avi Controller and Service Engines, only cryptographic algorithms approved under FIPS 140-3 are used. FIPS 140-3 requires stricter handling of keys, authentication data, and random number generation than its predecessor, and aligns cryptographic module standards with international standards ISO/IEC 19790:2012 and ISO/IEC 24759:2017. This covers the symmetric encryption operations involved in TLS termination, re-encryption between Service Engines and backend servers, and internal control-plane communication. FIPS mode is configured at deployment time for Controller cluster deployments.

SSL/TLS Profile Templates in Avi give administrators control over the cipher suites and protocol versions used by virtual services. These profiles specify a prioritized cipher list and accepted protocol versions per virtual service and are managed under Templates > Security > SSL/TLS Profile in the Avi Controller UI. Configuring a FIPS-aligned SSL/TLS Profile and applying it to virtual services restricts symmetric key negotiation and cipher use across all load-balanced application traffic to FIPS-approved algorithms.

Post-quantum cryptography (PQC)-based Key Encapsulation Mechanisms and Signature Algorithms are not supported in Avi deployments running in FIPS mode.

For organizations deploying Avi in public cloud environments, Avi integrates with cloud-native key management services. In AWS, Avi's Cloud Connector integrates with AWS KMS for Service Engine disk and SQS queue encryption, supporting customer-managed keys with configurable IAM and key policies; the Avi Controller can list KMS encryption keys in the Avi UI

and supports S3 and EBS encryption. In Google Cloud Platform, Avi supports Google Cloud KMS for Service Engine disk and image encryption, GCS bucket encryption, and GCS object encryption, using separate KMS keys for each resource type.

The Avi Kubernetes Operator (AKO) 2.1 adds support for configuring both RSA and ECC signed certificates simultaneously on a single virtual service through the alt.crt and alt.key fields in the default TLS secret, enabling certificate algorithm flexibility for Kubernetes ingress workloads.

Not applicable for this control: VMware vDefend. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 2.1

> Authorization enforcement: Enforce assigned privileges to all users and software processes. Limit use of privileged accounts: Restrict the use of privileged accounts to only necessary tasks. Mapping of users to roles: Support the mapping of users to specific authorized roles. Supervisor override: Capability for a supervisor to override certain locks/blocks. Dual control: Require two authorized individuals to perform certain sensitive actions. Response to audit processing failures: Capability to protect audit record generation in case of failure. Timestamps: Capability to include timestamps in audit records. Time synchronization: Capability to synchronize the component clock with an NTP source. Time synchronization protection: Protect the time synchronization process from tampering. Non-repudiation: Capability to provide evidence that a user performed a specific action. Non-repudiation for digital signatures: Use of digital signatures for non-repudiation of origin/receipt. Physical tamper resistance and detection: Capability to detect physical tampering of the component.

**SCF Controls:** IAC-08

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF enforces role-based access control across all major platform components, restricting access to technology assets, applications, services, and data based on assigned roles with defined permissions. RBAC is applied consistently throughout the stack, covering compute management, networking, storage, operations management, automation services, and Kubernetes workloads.

VMware vCenter provides fine-grained authorization through a permissions model that associates roles with users or groups on objects in the vCenter inventory hierarchy. Roles are sets of privileges, and vCenter defines three authorization levels: operator (read-only access to configuration, the most restricted level), administrator (read and write access to configuration but cannot manage user accounts, intended for users who alter vCenter configuration or exercise control functions), and super administrator (full control). When a predefined role is modified, the updated privilege set applies immediately to all users assigned that role. New roles are created with three default system-defined privileges (System.Anonymous, System.Read, System.View), and administrators add specific privileges to match the intended access level. vCenter also supports creation of custom roles with granular privilege assignment and permission propagation to vSphere objects in the inventory hierarchy.

NSX implements RBAC by assigning users predefined roles with specific permissions. The platform includes 15 built-in roles: Enterprise Admin, Admin, Network Admin, Network Operator, Security Admin, Security Operator, Cloud Admin, Cloud Operator, Cloud Project Admin, Load Balancer Admin, Load Balancer Operator, VPN Admin, GI Platform Admin, NSX Platform Admin, and Support Bundle Collector. These roles follow the principle of least privilege, with each role carrying only the permissions needed for its functional area. NSX RBAC applies across multiple operational domains including inventory, plan and troubleshoot, and system management functions. NSX also supports project-scoped RBAC, where Network Admin, Network Operator, Security Admin, and Security Operator roles assigned within a project have permissions limited to that project's scope rather than the entire NSX system. This project-scoped model extends to VPCs and subnets within the NSX networking hierarchy. For organizations that need role definitions beyond the built-in set, NSX allows creation of custom roles tailored to specific operational requirements. Only an Enterprise Administrator can assign the role management permission to a custom role. NSX Federation extends RBAC to multi-site deployments, applying the same access control model across federated NSX instances. Principal identities, which are used for service-to-service authentication local to NSX, also support RBAC role assignment, so automated integrations operate under the same access control model as human users.

VCF Operations enforces RBAC through the assignment of service roles to users and groups. Administrators configure role-based permissions through the Roles tab under Infrastructure Operations, where individual permissions can be selected or deselected to define what each role is authorized to do. The System Administrator role includes rights covering approvals, audits, certificates, datastores, edge clusters, integrations, LDAP settings, namespaces, proxy rules, system settings, tokens, and truststore operations. Administrative operations such as password management require authentication with a user assigned the ADMIN role. VCF Operations also defines its own roles within vCenter, with privileges named using a convention that appends the role to the product name (such as VCF Operations ContentAdmin Role and VCF Operations PowerUser Role). The PowerUser role has privileges to perform the actions of the Administrator role except for user management and

cluster management. A further restricted PowerUserMinusRemediation role excludes remediation actions in addition to those limitations. A vCenter administrator assigns these roles to users, and VCF Operations maps vCenter users to the appropriate role.

vSAN supports RBAC and permissions management for authentication and authorization of storage operations. The vSAN Security Configuration Guide provides guidance on configuring these role-based controls for storage infrastructure.

VCF Automation governs access through multiple RBAC mechanisms at different scopes. At the organization level, rights bundles assign specific API permissions to roles including Organization Administrator, Organization Auditor, and Organization User. Group membership and role assignment are managed through the Access Control interface, and organization administrators can edit role assignments for groups at any time. At the project level, VCF Automation defines three distinct roles: administrators, members, and viewers, providing tiered access to project resources and operations. VCF Automation also governs blueprints, catalogs, content libraries, and custom entities through its role model. The Assembler Administrator role has read and write access to the entire user interface and API resources, while other roles have progressively narrower permissions.

For Kubernetes workloads running on VMware Kubernetes Service (VKS), VCF enforces RBAC through two complementary control layers. At the vSphere layer, vSphere Namespaces assign three permission roles: Owner (full control including namespace deletion), Can Edit (provisioning and operating workloads), and Can View (read-only access). At the Kubernetes layer, the Kubernetes RBAC authorization framework governs access to all API server requests through four core objects: Roles define permissions scoped to a single namespace, ClusterRoles define permissions applying cluster-wide, and RoleBindings and ClusterRoleBindings associate those roles with authenticated subjects such as users, groups, or ServiceAccounts. Permissions are purely additive with no deny rules, and administrators can restrict access to individual resource instances by name using the resourceNames list in role rules.

VKS clusters integrate with VCF SSO so that Kubernetes RoleBinding objects grant RBAC permissions directly to VCF SSO users and groups. This means the same identity provider that governs VMware vCenter access also governs Kubernetes cluster access, providing a consistent identity model across both the platform and workload layers. Cluster operators use kubectl or the Kubernetes API to assign role permissions to developers and other personas with defined responsibilities.

Kubernetes RBAC also governs ServiceAccounts, which provide discrete identities for pods and automation processes. This separation allows human access and workload access to be governed independently, with each scoped to the minimum permissions required for its function. ServiceAccounts follow the principle of least privilege by receiving only the minimum set of permissions required to function correctly, and RBAC should be applied to Secret objects to enforce least-privilege access to sensitive data stored within namespaces.

The Kubernetes RBAC API includes built-in controls against privilege escalation: the API prevents users from editing roles or role bindings to gain permissions beyond those they already hold, and this enforcement applies at the API level regardless of the authorization mode in use. Role-binding authority can be explicitly delegated by granting a user permission to perform the bind verb on a specific Role or ClusterRole, limiting the roles that identity can delegate to others.

In multi-tenant VKS deployments, RBAC is the primary mechanism for isolating tenants within a shared cluster. Administrators use Role and RoleBinding objects at the namespace level to restrict each tenant's access to its assigned namespaces, and ClusterRole and ClusterRoleBinding objects to control access to cluster-level resources, blocking unprivileged users from crossing tenant boundaries.

Supervisor services deployed on VKS clusters extend RBAC to their own access models. The Secret Store Supervisor Service enforces role-based access control across administrator, DevOps engineer, and application tenant roles, controlling who can manage and retrieve sensitive data. The Argo CD Supervisor Service implements RBAC through a configurable policy field using Casbin syntax, with the spec.rbac.policy setting defining role assignments and permission rules; policy rules (p rules) control account permissions including the ability to grant admin roles to specific local accounts, and group mappings (g rules) assign accounts to role groups.



VMware Private AI Services (PAIS) provides role-based access control scoped to its AI services and AI workload delivery components. The central mechanism is OIDC group-based authorization for PAIS. When activating PAIS in an organization namespace, administrators populate the `authorizedGroups` setting with one or more OIDC group identifiers so that only users who belong to those groups can access the service. The OIDC configuration supports a configurable groups claim name through the `oidc-groups-claim` setting, with the groups claim carried in the Access Token and evaluated at access time. PAIS requires an OIDC provider configured as a public client application with Authorization Code and Refresh Token grant types, and it can be configured with one OIDC provider. Additional OAuth 2.0 client identifiers can be accepted by configuring Extra Audiences through the `oidc-extra-audiences` setting when the OIDC provider issues tokens for multiple audiences.

PAIS extends the same identity model to the Grafana observability platform used for PAIS metrics. Grafana role assignment is configured through the `grafana-oidc-role-mapping` property using an expression such as `contains(groups['*'], 'pais-access-group') && 'Admin' || 'Viewer'`, so OIDC group membership maps automatically to Grafana Admin or Viewer roles. PAIS uses the same OIDC provider for Grafana authentication that it uses for service access.

For machine learning model storage, the Model Gallery uses Harbor project access capabilities to restrict access to the sensitive data used to train and tune models. PAIS integrates with Harbor registries as the model storage backend and requires authentication and certificate-based trust for model retrieval. PAIS is compatible with OCI-compliant registries from other vendors when deployed in a disconnected or air-gapped environment, and the project-scoped access controls apply to the chosen registry.

Access to PAIS catalog items for GPU-accelerated VKS clusters and other AI workloads is governed through project-based permissions in VCF Automation. PAIS custom forms in blueprints can be modified to control user inputs at request time, scoping which configuration choices a requester can make when provisioning AI workloads. Knowledge Base data sources used by PAIS retrieval augmented generation require authenticated access to external repositories, including user name and password authentication for SharePoint On-Premises data sources and access keys with list and read permissions on the target bucket for Amazon S3 compatible stores. PAIS uses a CA trust bundle for secure communication with external services and databases.

These PAIS-specific controls operate at the AI services and workload layer. Access to the underlying compute, storage, and networking infrastructure is governed by platform-level RBAC in VCF, including VCF SSO and VMware vCenter role assignments, and database-level access for vector and operational data is governed by Data Services Manager. PAIS adds service-level and workload-level RBAC for AI-specific assets on top of the VCF platform foundation rather than replacing those platform controls.

### **VMware vDefend (Contributes)**

VMware vDefend (vDefend) enforces role-based access control across its security management interfaces to restrict access to firewall policy administration, threat analysis functions, and monitoring capabilities based on assigned user roles.

The vDefend distributed firewall (DFW) implements RBAC to control who can create or modify firewall rules, limiting policy changes to authorized personnel. Access to DFW monitoring features, including Firewall Insights, requires one of three designated roles: Enterprise Admin, Security Admin, or Security Operator. Rule Analysis operations within the platform are also role-gated across the five defined role types.

Security Services Platform (SSP) implements RBAC with five defined roles governing access to platform operations: Enterprise Admin, Auditor, Security Engineer, Security Operator, and Support Bundle Collector. Each role carries differentiated permissions; Full Access grants create, read, update, and delete capabilities, while Read-only access is available for monitoring-oriented roles such as Auditor and Support Bundle Collector. Role assignments can be applied to individual remote users or to remote user groups managed through a directory service, allowing administrators to align vDefend access with existing organizational structures. Multiple roles can be assigned to a single user or group to meet varying operational requirements. SSP's RBAC model also applies to License Hub operations, restricting licensing management actions to authorized roles.



Security Intelligence RBAC in 9.1 uses the same five-role model as SSP (Enterprise Admin, Auditor, Security Engineer, Security Operator, and Support Bundle Collector), aligning Security Intelligence access governance with SSP's permission model and providing consistent role boundaries across both the detection and platform layers.

For organizations using VCF Automation for vDefend firewall management, VCF Automation provides a dedicated Firewall category under RBAC rights, controlling access to the management and viewing of DFW rules, Tenant Gateway Firewall policy, VPC Gateway Firewall policy, and VPC Security Profiles. This allows organizations to scope firewall administration rights independently of other VCF Automation capabilities, supporting separation of duties between network automation and security policy management.

vDefend's RBAC controls complement the broader VCF RBAC model by restricting access to network security policy management independently of compute, storage, or platform configuration access. An administrator with VMware vCenter privileges does not automatically gain the ability to modify firewall rules or security analytics configurations unless explicitly granted the appropriate vDefend roles. This separation of duties between infrastructure administration and security policy management supports distinct team structures for network security management.

### **VCF Protection and Recovery (Contributes)**

Protection and Recovery (PNR) implements role-based access control through purpose-built roles registered in VMware vCenter and through a multi-tenant administrative model for Protection and Recovery for VCF Automation. The SRM Administrator role allows users or user groups to perform all PNR configuration and administration operations. The Protection and Recovery Administrator role can be assigned to user groups to manage permissions. Default roles provide the privileges users require to manage protection groups and perform recoveries, while restricting access to resources belonging to other users.

For automatic protection of virtual machines within a protection group, administrators assign either the SrmAdministrator or SrmProtectionGroupsAdministrator role to the automatic protection user or user group. This scopes the protection actions performed on the user's behalf to the privileges of the assigned role.

Each role is a collection of privileges that defines the actions a user can perform on the protection and recovery site. A user can have at most one role on an object, but roles combine when the user belongs to multiple groups that all have roles on the object. PNR supports propagating roles to child objects in the inventory hierarchy, so administrators can extend privileges from parent objects such as folders to all virtual machines within them.

Protection and Recovery for VCF Automation introduces a multi-tenant role model with Provider Administrator, Organization Administrator, Project Administrator, and Project Advanced User. Each role has specific authorization levels for disaster recovery operations. Provider Administrators select which replication classes each Organization can use, restricting organizational access to specific recovery configurations. Organization Administrators manage the pairing of namespaces and assignment of replication classes within their tenancy.

Underlying directory services, authentication, and enterprise identity governance are provided by VCF SSO and the broader VCF platform. PNR consumes those identities and layers its protection and recovery roles on top of them.

### **VMware Data Services Manager (Meets)**

VMware Data Services Manager (DSM) implements role-based access control natively, with two defined role types that govern access to all data service resources. The DSM Admin role is a super-user role with full access to create, monitor, and manage all data services across the environment. The DSM User role is a restricted role that limits a user to viewing and managing only the data service deployments they own. This separation is enforced at the console level: the views and available actions in the DSM interface differ based on the assigned role, so users with the DSM User role cannot see or act on resources outside their own scope.

Access control is further refined through namespace scoping. DSM User role assignments can be restricted to one or more specific namespaces, limiting access to data services within those assigned namespaces only. Administrators can modify a user's assigned role and namespace access after initial provisioning. DSM namespaces enforce role-based access control that

allows multiple users to work on data services within a shared namespace while maintaining appropriate permission boundaries between them.

DSM supports both local user accounts and directory service integration for identity management. Local users are created through the DSM UI, where administrators specify the user's role, email address, and password. For enterprise directory integration, DSM supports LDAPS-based authentication with Directory Service Group mapping, allowing administrators to assign users to either the DSM Admin or DSM User role based on their LDAP group membership. This group-based mapping is configured through the vSphere Client during DSM setup or through the DSM UI after deployment.

Access control extends to storage and database resources as well. DSM IAM policies for S3-compatible storage support granular access control through policy statements that allow specific S3 actions, including PutObject, GetObject, DeleteObject, and ListMultipartUploadParts. At the database level, PostgreSQL instances support the CREATE ROLE command to create database roles for LDAPS users with configurable login capabilities, and administrators can configure Host-Based Authentication (HBA) rules to restrict which clients and authentication methods may connect to a PostgreSQL instance. For SQL Server instances, DSM applies least-privilege access by granting the cluster administrator login restricted read-only privileges and specific ALTER privileges for server management and auditing, rather than full administrative access. Data service policies also support allowlist configuration to restrict users to selecting only from a specific set of approved parameters when provisioning databases.

The DSM Permissions view provides a centralized interface for administrators to create, monitor, and manage configured users. This view supports ongoing governance of who has access to what within the DSM environment, making it straightforward to audit and adjust role assignments as business needs change.

### **VMware Avi Load Balancer (Meets)**

VMware Avi Load Balancer provides a native, multi-tiered Role-Based Access Control (RBAC) framework that governs all administrative access to the Avi Controller. At the base level, Avi roles assign one of four privilege types to each resource category: Write (full create, read, modify, and delete access), Read (view-only access to configuration, health, and analytics data without modification capability), Assorted (a mixed access level combining permissions across specific operations), and No Access (no access to the resource at all, including the inability to read or list it). Roles are combined with tenants to define the complete authorization settings for each user, scoping what resources a user can interact with and what actions they may take.

Beyond base privilege levels, the Extended Granular RBAC feature refines access using markers, which are key-value label pairs assigned to Avi objects. A role definition specifies that a user may access only objects bearing labels that match the role's marker set. The `allow_unlabelled_access` parameter controls whether a user can view objects that lack matching labels, allowing administrators to restrict visibility to labeled objects only. Label groups can be associated with tenants to enforce label-based access policies across the tenant boundary. Multiple label values can be mapped to a single key, enabling flexible classification schemes. This mechanism enables resource-level scoping across tenants and teams: for example, restricting a group of operators to only the virtual services labeled with their team's identifier, without granting visibility into other teams' objects.

At the finest granularity, Avi implements sub-resource (field-level) RBAC, which scopes which specific fields within a configuration object a given user is authorized to modify. Sub-resource definitions can be combined so that a user can configure multiple fields or features within a single object while remaining restricted from updating others. Label-based permissions for cloud objects extend this field-level control to infrastructure resources managed by the Avi Controller. Virtual service configuration exposes RBAC directly: the RBAC tab in the Virtual Service advanced setup provides label-based access control for health monitors and associated objects, extending role enforcement down to individual service configuration items.

For GSLB deployments, the Avi Controller enforces RBAC through role references such as `Gslb_Group_Enabled` and `Gslb_Health_Monitor` that can be assigned to user accounts with tenant-level scope. These role references apply to follower sites configured with federated JWT profiles, scoping which accounts can modify GSLB configuration.

The Avi Controller applies RBAC within the License Hub, limiting which users can perform specific actions based on their assigned role.

For deployments using the Avi Kubernetes Operator (AKO), the Avi Controller implements RBAC by assigning tenants and roles to user accounts at the Kubernetes or OpenShift cluster level. AKO Custom Resource Definitions (CRDs) can leverage Kubernetes RBAC policies to restrict which user groups can configure routing rules, extending the Controller's authorization model into the Kubernetes control plane.

For deployments on Microsoft Azure, Avi integrates with Azure IAM to control which Azure identities have access to Avi Load Balancer resources hosted in Azure. Administrators can use built-in Azure roles such as Reader (read-only access) or Owner (full access), or define custom roles for finer control over specific resources.

RBAC configuration is managed through the Users section of the Avi Controller UI. Roles are defined per resource type and then assigned to users within a tenant. Extended Granular RBAC markers are attached to individual Avi objects (virtual services, pools, health monitors, and others) and referenced in the role definition. Sub-resource RBAC is configured as part of the Extended Granular RBAC object, combining field-level restrictions as needed.

## Control CR 2.1(1)

> Authorization enforcement: Enforce assigned privileges to all users and software processes. Limit use of privileged accounts: Restrict the use of privileged accounts to only necessary tasks. Mapping of users to roles: Support the mapping of users to specific authorized roles. Supervisor override: Capability for a supervisor to override certain locks/blocks. Dual control: Require two authorized individuals to perform certain sensitive actions. Response to audit processing failures: Capability to protect audit record generation in case of failure. Timestamps: Capability to include timestamps in audit records. Time synchronization: Capability to synchronize the component clock with an NTP source. Time synchronization protection: Protect the time synchronization process from tampering. Non-repudiation: Capability to provide evidence that a user performed a specific action. Non-repudiation for digital signatures: Use of digital signatures for non-repudiation of origin/receipt. Physical tamper resistance and detection: Capability to detect physical tampering of the component.

**SCF Controls:** IAC-08

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF enforces role-based access control across all major platform components, restricting access to technology assets, applications, services, and data based on assigned roles with defined permissions. RBAC is applied consistently throughout the stack, covering compute management, networking, storage, operations management, automation services, and Kubernetes workloads.

VMware vCenter provides fine-grained authorization through a permissions model that associates roles with users or groups on objects in the vCenter inventory hierarchy. Roles are sets of privileges, and vCenter defines three authorization levels: operator (read-only access to configuration, the most restricted level), administrator (read and write access to configuration but cannot manage user accounts, intended for users who alter vCenter configuration or exercise control functions), and super administrator (full control). When a predefined role is modified, the updated privilege set applies immediately to all users assigned that role. New roles are created with three default system-defined privileges (System.Anonymous, System.Read, System.View), and administrators add specific privileges to match the intended access level. vCenter also supports creation of custom roles with granular privilege assignment and permission propagation to vSphere objects in the inventory hierarchy.

NSX implements RBAC by assigning users predefined roles with specific permissions. The platform includes 15 built-in roles: Enterprise Admin, Admin, Network Admin, Network Operator, Security Admin, Security Operator, Cloud Admin, Cloud Operator, Cloud Project Admin, Load Balancer Admin, Load Balancer Operator, VPN Admin, GI Platform Admin, NSX Platform Admin, and Support Bundle Collector. These roles follow the principle of least privilege, with each role carrying only the permissions needed for its functional area. NSX RBAC applies across multiple operational domains including inventory, plan and troubleshoot, and system management functions. NSX also supports project-scoped RBAC, where Network Admin, Network Operator, Security Admin, and Security Operator roles assigned within a project have permissions limited to that project's scope rather than the entire NSX system. This project-scoped model extends to VPCs and subnets within the NSX networking hierarchy. For organizations that need role definitions beyond the built-in set, NSX allows creation of custom roles tailored to specific operational requirements. Only an Enterprise Administrator can assign the role management permission to a custom role. NSX Federation extends RBAC to multi-site deployments, applying the same access control model across federated NSX instances. Principal identities, which are used for service-to-service authentication local to NSX, also support RBAC role assignment, so automated integrations operate under the same access control model as human users.

VCF Operations enforces RBAC through the assignment of service roles to users and groups. Administrators configure role-based permissions through the Roles tab under Infrastructure Operations, where individual permissions can be selected or deselected to define what each role is authorized to do. The System Administrator role includes rights covering approvals, audits, certificates, datastores, edge clusters, integrations, LDAP settings, namespaces, proxy rules, system settings, tokens, and truststore operations. Administrative operations such as password management require authentication with a user assigned the ADMIN role. VCF Operations also defines its own roles within vCenter, with privileges named using a convention that appends the role to the product name (such as VCF Operations ContentAdmin Role and VCF Operations PowerUser Role). The PowerUser role has privileges to perform the actions of the Administrator role except for user management and

cluster management. A further restricted PowerUserMinusRemediation role excludes remediation actions in addition to those limitations. A vCenter administrator assigns these roles to users, and VCF Operations maps vCenter users to the appropriate role.

vSAN supports RBAC and permissions management for authentication and authorization of storage operations. The vSAN Security Configuration Guide provides guidance on configuring these role-based controls for storage infrastructure.

VCF Automation governs access through multiple RBAC mechanisms at different scopes. At the organization level, rights bundles assign specific API permissions to roles including Organization Administrator, Organization Auditor, and Organization User. Group membership and role assignment are managed through the Access Control interface, and organization administrators can edit role assignments for groups at any time. At the project level, VCF Automation defines three distinct roles: administrators, members, and viewers, providing tiered access to project resources and operations. VCF Automation also governs blueprints, catalogs, content libraries, and custom entities through its role model. The Assembler Administrator role has read and write access to the entire user interface and API resources, while other roles have progressively narrower permissions.

For Kubernetes workloads running on VMware Kubernetes Service (VKS), VCF enforces RBAC through two complementary control layers. At the vSphere layer, vSphere Namespaces assign three permission roles: Owner (full control including namespace deletion), Can Edit (provisioning and operating workloads), and Can View (read-only access). At the Kubernetes layer, the Kubernetes RBAC authorization framework governs access to all API server requests through four core objects: Roles define permissions scoped to a single namespace, ClusterRoles define permissions applying cluster-wide, and RoleBindings and ClusterRoleBindings associate those roles with authenticated subjects such as users, groups, or ServiceAccounts. Permissions are purely additive with no deny rules, and administrators can restrict access to individual resource instances by name using the resourceNames list in role rules.

VKS clusters integrate with VCF SSO so that Kubernetes RoleBinding objects grant RBAC permissions directly to VCF SSO users and groups. This means the same identity provider that governs VMware vCenter access also governs Kubernetes cluster access, providing a consistent identity model across both the platform and workload layers. Cluster operators use kubectl or the Kubernetes API to assign role permissions to developers and other personas with defined responsibilities.

Kubernetes RBAC also governs ServiceAccounts, which provide discrete identities for pods and automation processes. This separation allows human access and workload access to be governed independently, with each scoped to the minimum permissions required for its function. ServiceAccounts follow the principle of least privilege by receiving only the minimum set of permissions required to function correctly, and RBAC should be applied to Secret objects to enforce least-privilege access to sensitive data stored within namespaces.

The Kubernetes RBAC API includes built-in controls against privilege escalation: the API prevents users from editing roles or role bindings to gain permissions beyond those they already hold, and this enforcement applies at the API level regardless of the authorization mode in use. Role-binding authority can be explicitly delegated by granting a user permission to perform the bind verb on a specific Role or ClusterRole, limiting the roles that identity can delegate to others.

In multi-tenant VKS deployments, RBAC is the primary mechanism for isolating tenants within a shared cluster. Administrators use Role and RoleBinding objects at the namespace level to restrict each tenant's access to its assigned namespaces, and ClusterRole and ClusterRoleBinding objects to control access to cluster-level resources, blocking unprivileged users from crossing tenant boundaries.

Supervisor services deployed on VKS clusters extend RBAC to their own access models. The Secret Store Supervisor Service enforces role-based access control across administrator, DevOps engineer, and application tenant roles, controlling who can manage and retrieve sensitive data. The Argo CD Supervisor Service implements RBAC through a configurable policy field using Casbin syntax, with the spec.rbac.policy setting defining role assignments and permission rules; policy rules (p rules) control account permissions including the ability to grant admin roles to specific local accounts, and group mappings (g rules) assign accounts to role groups.

VMware Private AI Services (PAIS) provides role-based access control scoped to its AI services and AI workload delivery components. The central mechanism is OIDC group-based authorization for PAIS. When activating PAIS in an organization namespace, administrators populate the `authorizedGroups` setting with one or more OIDC group identifiers so that only users who belong to those groups can access the service. The OIDC configuration supports a configurable groups claim name through the `oidc-groups-claim` setting, with the groups claim carried in the Access Token and evaluated at access time. PAIS requires an OIDC provider configured as a public client application with Authorization Code and Refresh Token grant types, and it can be configured with one OIDC provider. Additional OAuth 2.0 client identifiers can be accepted by configuring Extra Audiences through the `oidc-extra-audiences` setting when the OIDC provider issues tokens for multiple audiences.

PAIS extends the same identity model to the Grafana observability platform used for PAIS metrics. Grafana role assignment is configured through the `grafana-oidc-role-mapping` property using an expression such as `contains(groups['*'], 'pais-access-group') && 'Admin' || 'Viewer'`, so OIDC group membership maps automatically to Grafana Admin or Viewer roles. PAIS uses the same OIDC provider for Grafana authentication that it uses for service access.

For machine learning model storage, the Model Gallery uses Harbor project access capabilities to restrict access to the sensitive data used to train and tune models. PAIS integrates with Harbor registries as the model storage backend and requires authentication and certificate-based trust for model retrieval. PAIS is compatible with OCI-compliant registries from other vendors when deployed in a disconnected or air-gapped environment, and the project-scoped access controls apply to the chosen registry.

Access to PAIS catalog items for GPU-accelerated VKS clusters and other AI workloads is governed through project-based permissions in VCF Automation. PAIS custom forms in blueprints can be modified to control user inputs at request time, scoping which configuration choices a requester can make when provisioning AI workloads. Knowledge Base data sources used by PAIS retrieval augmented generation require authenticated access to external repositories, including user name and password authentication for SharePoint On-Premises data sources and access keys with list and read permissions on the target bucket for Amazon S3 compatible stores. PAIS uses a CA trust bundle for secure communication with external services and databases.

These PAIS-specific controls operate at the AI services and workload layer. Access to the underlying compute, storage, and networking infrastructure is governed by platform-level RBAC in VCF, including VCF SSO and VMware vCenter role assignments, and database-level access for vector and operational data is governed by Data Services Manager. PAIS adds service-level and workload-level RBAC for AI-specific assets on top of the VCF platform foundation rather than replacing those platform controls.

### **VMware vDefend (Contributes)**

VMware vDefend (vDefend) enforces role-based access control across its security management interfaces to restrict access to firewall policy administration, threat analysis functions, and monitoring capabilities based on assigned user roles.

The vDefend distributed firewall (DFW) implements RBAC to control who can create or modify firewall rules, limiting policy changes to authorized personnel. Access to DFW monitoring features, including Firewall Insights, requires one of three designated roles: Enterprise Admin, Security Admin, or Security Operator. Rule Analysis operations within the platform are also role-gated across the five defined role types.

Security Services Platform (SSP) implements RBAC with five defined roles governing access to platform operations: Enterprise Admin, Auditor, Security Engineer, Security Operator, and Support Bundle Collector. Each role carries differentiated permissions; Full Access grants create, read, update, and delete capabilities, while Read-only access is available for monitoring-oriented roles such as Auditor and Support Bundle Collector. Role assignments can be applied to individual remote users or to remote user groups managed through a directory service, allowing administrators to align vDefend access with existing organizational structures. Multiple roles can be assigned to a single user or group to meet varying operational requirements. SSP's RBAC model also applies to License Hub operations, restricting licensing management actions to authorized roles.



Security Intelligence RBAC in 9.1 uses the same five-role model as SSP (Enterprise Admin, Auditor, Security Engineer, Security Operator, and Support Bundle Collector), aligning Security Intelligence access governance with SSP's permission model and providing consistent role boundaries across both the detection and platform layers.

For organizations using VCF Automation for vDefend firewall management, VCF Automation provides a dedicated Firewall category under RBAC rights, controlling access to the management and viewing of DFW rules, Tenant Gateway Firewall policy, VPC Gateway Firewall policy, and VPC Security Profiles. This allows organizations to scope firewall administration rights independently of other VCF Automation capabilities, supporting separation of duties between network automation and security policy management.

vDefend's RBAC controls complement the broader VCF RBAC model by restricting access to network security policy management independently of compute, storage, or platform configuration access. An administrator with VMware vCenter privileges does not automatically gain the ability to modify firewall rules or security analytics configurations unless explicitly granted the appropriate vDefend roles. This separation of duties between infrastructure administration and security policy management supports distinct team structures for network security management.

### **VCF Protection and Recovery (Contributes)**

Protection and Recovery (PNR) implements role-based access control through purpose-built roles registered in VMware vCenter and through a multi-tenant administrative model for Protection and Recovery for VCF Automation. The SRM Administrator role allows users or user groups to perform all PNR configuration and administration operations. The Protection and Recovery Administrator role can be assigned to user groups to manage permissions. Default roles provide the privileges users require to manage protection groups and perform recoveries, while restricting access to resources belonging to other users.

For automatic protection of virtual machines within a protection group, administrators assign either the SrmAdministrator or SrmProtectionGroupsAdministrator role to the automatic protection user or user group. This scopes the protection actions performed on the user's behalf to the privileges of the assigned role.

Each role is a collection of privileges that defines the actions a user can perform on the protection and recovery site. A user can have at most one role on an object, but roles combine when the user belongs to multiple groups that all have roles on the object. PNR supports propagating roles to child objects in the inventory hierarchy, so administrators can extend privileges from parent objects such as folders to all virtual machines within them.

Protection and Recovery for VCF Automation introduces a multi-tenant role model with Provider Administrator, Organization Administrator, Project Administrator, and Project Advanced User. Each role has specific authorization levels for disaster recovery operations. Provider Administrators select which replication classes each Organization can use, restricting organizational access to specific recovery configurations. Organization Administrators manage the pairing of namespaces and assignment of replication classes within their tenancy.

Underlying directory services, authentication, and enterprise identity governance are provided by VCF SSO and the broader VCF platform. PNR consumes those identities and layers its protection and recovery roles on top of them.

### **VMware Data Services Manager (Meets)**

VMware Data Services Manager (DSM) implements role-based access control natively, with two defined role types that govern access to all data service resources. The DSM Admin role is a super-user role with full access to create, monitor, and manage all data services across the environment. The DSM User role is a restricted role that limits a user to viewing and managing only the data service deployments they own. This separation is enforced at the console level: the views and available actions in the DSM interface differ based on the assigned role, so users with the DSM User role cannot see or act on resources outside their own scope.

Access control is further refined through namespace scoping. DSM User role assignments can be restricted to one or more specific namespaces, limiting access to data services within those assigned namespaces only. Administrators can modify a user's assigned role and namespace access after initial provisioning. DSM namespaces enforce role-based access control that



allows multiple users to work on data services within a shared namespace while maintaining appropriate permission boundaries between them.

DSM supports both local user accounts and directory service integration for identity management. Local users are created through the DSM UI, where administrators specify the user's role, email address, and password. For enterprise directory integration, DSM supports LDAPS-based authentication with Directory Service Group mapping, allowing administrators to assign users to either the DSM Admin or DSM User role based on their LDAP group membership. This group-based mapping is configured through the vSphere Client during DSM setup or through the DSM UI after deployment.

Access control extends to storage and database resources as well. DSM IAM policies for S3-compatible storage support granular access control through policy statements that allow specific S3 actions, including PutObject, GetObject, DeleteObject, and ListMultipartUploadParts. At the database level, PostgreSQL instances support the CREATE ROLE command to create database roles for LDAPS users with configurable login capabilities, and administrators can configure Host-Based Authentication (HBA) rules to restrict which clients and authentication methods may connect to a PostgreSQL instance. For SQL Server instances, DSM applies least-privilege access by granting the cluster administrator login restricted read-only privileges and specific ALTER privileges for server management and auditing, rather than full administrative access. Data service policies also support allowlist configuration to restrict users to selecting only from a specific set of approved parameters when provisioning databases.

The DSM Permissions view provides a centralized interface for administrators to create, monitor, and manage configured users. This view supports ongoing governance of who has access to what within the DSM environment, making it straightforward to audit and adjust role assignments as business needs change.

### **VMware Avi Load Balancer (Meets)**

VMware Avi Load Balancer provides a native, multi-tiered Role-Based Access Control (RBAC) framework that governs all administrative access to the Avi Controller. At the base level, Avi roles assign one of four privilege types to each resource category: Write (full create, read, modify, and delete access), Read (view-only access to configuration, health, and analytics data without modification capability), Assorted (a mixed access level combining permissions across specific operations), and No Access (no access to the resource at all, including the inability to read or list it). Roles are combined with tenants to define the complete authorization settings for each user, scoping what resources a user can interact with and what actions they may take.

Beyond base privilege levels, the Extended Granular RBAC feature refines access using markers, which are key-value label pairs assigned to Avi objects. A role definition specifies that a user may access only objects bearing labels that match the role's marker set. The `allow_unlabelled_access` parameter controls whether a user can view objects that lack matching labels, allowing administrators to restrict visibility to labeled objects only. Label groups can be associated with tenants to enforce label-based access policies across the tenant boundary. Multiple label values can be mapped to a single key, enabling flexible classification schemes. This mechanism enables resource-level scoping across tenants and teams: for example, restricting a group of operators to only the virtual services labeled with their team's identifier, without granting visibility into other teams' objects.

At the finest granularity, Avi implements sub-resource (field-level) RBAC, which scopes which specific fields within a configuration object a given user is authorized to modify. Sub-resource definitions can be combined so that a user can configure multiple fields or features within a single object while remaining restricted from updating others. Label-based permissions for cloud objects extend this field-level control to infrastructure resources managed by the Avi Controller. Virtual service configuration exposes RBAC directly: the RBAC tab in the Virtual Service advanced setup provides label-based access control for health monitors and associated objects, extending role enforcement down to individual service configuration items.

For GSLB deployments, the Avi Controller enforces RBAC through role references such as `Gslb_Group_Enabled` and `Gslb_Health_Monitor` that can be assigned to user accounts with tenant-level scope. These role references apply to follower sites configured with federated JWT profiles, scoping which accounts can modify GSLB configuration.

The Avi Controller applies RBAC within the License Hub, limiting which users can perform specific actions based on their assigned role.

For deployments using the Avi Kubernetes Operator (AKO), the Avi Controller implements RBAC by assigning tenants and roles to user accounts at the Kubernetes or OpenShift cluster level. AKO Custom Resource Definitions (CRDs) can leverage Kubernetes RBAC policies to restrict which user groups can configure routing rules, extending the Controller's authorization model into the Kubernetes control plane.

For deployments on Microsoft Azure, Avi integrates with Azure IAM to control which Azure identities have access to Avi Load Balancer resources hosted in Azure. Administrators can use built-in Azure roles such as Reader (read-only access) or Owner (full access), or define custom roles for finer control over specific resources.

RBAC configuration is managed through the Users section of the Avi Controller UI. Roles are defined per resource type and then assigned to users within a tenant. Extended Granular RBAC markers are attached to individual Avi objects (virtual services, pools, health monitors, and others) and referenced in the role definition. Sub-resource RBAC is configured as part of the Extended Granular RBAC object, combining field-level restrictions as needed.

## Control CR 2.1(2)

> Authorization enforcement: Enforce assigned privileges to all users and software processes. Limit use of privileged accounts: Restrict the use of privileged accounts to only necessary tasks. Mapping of users to roles: Support the mapping of users to specific authorized roles. Supervisor override: Capability for a supervisor to override certain locks/blocks. Dual control: Require two authorized individuals to perform certain sensitive actions. Response to audit processing failures: Capability to protect audit record generation in case of failure. Timestamps: Capability to include timestamps in audit records. Time synchronization: Capability to synchronize the component clock with an NTP source. Time synchronization protection: Protect the time synchronization process from tampering. Non-repudiation: Capability to provide evidence that a user performed a specific action. Non-repudiation for digital signatures: Use of digital signatures for non-repudiation of origin/receipt. Physical tamper resistance and detection: Capability to detect physical tampering of the component.

**SCF Controls:** IAC-08

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF enforces role-based access control across all major platform components, restricting access to technology assets, applications, services, and data based on assigned roles with defined permissions. RBAC is applied consistently throughout the stack, covering compute management, networking, storage, operations management, automation services, and Kubernetes workloads.

VMware vCenter provides fine-grained authorization through a permissions model that associates roles with users or groups on objects in the vCenter inventory hierarchy. Roles are sets of privileges, and vCenter defines three authorization levels: operator (read-only access to configuration, the most restricted level), administrator (read and write access to configuration but cannot manage user accounts, intended for users who alter vCenter configuration or exercise control functions), and super administrator (full control). When a predefined role is modified, the updated privilege set applies immediately to all users assigned that role. New roles are created with three default system-defined privileges (System.Anonymous, System.Read, System.View), and administrators add specific privileges to match the intended access level. vCenter also supports creation of custom roles with granular privilege assignment and permission propagation to vSphere objects in the inventory hierarchy.

NSX implements RBAC by assigning users predefined roles with specific permissions. The platform includes 15 built-in roles: Enterprise Admin, Admin, Network Admin, Network Operator, Security Admin, Security Operator, Cloud Admin, Cloud Operator, Cloud Project Admin, Load Balancer Admin, Load Balancer Operator, VPN Admin, GI Platform Admin, NSX Platform Admin, and Support Bundle Collector. These roles follow the principle of least privilege, with each role carrying only the permissions needed for its functional area. NSX RBAC applies across multiple operational domains including inventory, plan and troubleshoot, and system management functions. NSX also supports project-scoped RBAC, where Network Admin, Network Operator, Security Admin, and Security Operator roles assigned within a project have permissions limited to that project's scope rather than the entire NSX system. This project-scoped model extends to VPCs and subnets within the NSX networking hierarchy. For organizations that need role definitions beyond the built-in set, NSX allows creation of custom roles tailored to specific operational requirements. Only an Enterprise Administrator can assign the role management permission to a custom role. NSX Federation extends RBAC to multi-site deployments, applying the same access control model across federated NSX instances. Principal identities, which are used for service-to-service authentication local to NSX, also support RBAC role assignment, so automated integrations operate under the same access control model as human users.

VCF Operations enforces RBAC through the assignment of service roles to users and groups. Administrators configure role-based permissions through the Roles tab under Infrastructure Operations, where individual permissions can be selected or deselected to define what each role is authorized to do. The System Administrator role includes rights covering approvals, audits, certificates, datastores, edge clusters, integrations, LDAP settings, namespaces, proxy rules, system settings, tokens, and truststore operations. Administrative operations such as password management require authentication with a user assigned the ADMIN role. VCF Operations also defines its own roles within vCenter, with privileges named using a convention that appends the role to the product name (such as VCF Operations ContentAdmin Role and VCF Operations PowerUser Role). The PowerUser role has privileges to perform the actions of the Administrator role except for user management and

cluster management. A further restricted PowerUserMinusRemediation role excludes remediation actions in addition to those limitations. A vCenter administrator assigns these roles to users, and VCF Operations maps vCenter users to the appropriate role.

vSAN supports RBAC and permissions management for authentication and authorization of storage operations. The vSAN Security Configuration Guide provides guidance on configuring these role-based controls for storage infrastructure.

VCF Automation governs access through multiple RBAC mechanisms at different scopes. At the organization level, rights bundles assign specific API permissions to roles including Organization Administrator, Organization Auditor, and Organization User. Group membership and role assignment are managed through the Access Control interface, and organization administrators can edit role assignments for groups at any time. At the project level, VCF Automation defines three distinct roles: administrators, members, and viewers, providing tiered access to project resources and operations. VCF Automation also governs blueprints, catalogs, content libraries, and custom entities through its role model. The Assembler Administrator role has read and write access to the entire user interface and API resources, while other roles have progressively narrower permissions.

For Kubernetes workloads running on VMware Kubernetes Service (VKS), VCF enforces RBAC through two complementary control layers. At the vSphere layer, vSphere Namespaces assign three permission roles: Owner (full control including namespace deletion), Can Edit (provisioning and operating workloads), and Can View (read-only access). At the Kubernetes layer, the Kubernetes RBAC authorization framework governs access to all API server requests through four core objects: Roles define permissions scoped to a single namespace, ClusterRoles define permissions applying cluster-wide, and RoleBindings and ClusterRoleBindings associate those roles with authenticated subjects such as users, groups, or ServiceAccounts. Permissions are purely additive with no deny rules, and administrators can restrict access to individual resource instances by name using the resourceNames list in role rules.

VKS clusters integrate with VCF SSO so that Kubernetes RoleBinding objects grant RBAC permissions directly to VCF SSO users and groups. This means the same identity provider that governs VMware vCenter access also governs Kubernetes cluster access, providing a consistent identity model across both the platform and workload layers. Cluster operators use kubectl or the Kubernetes API to assign role permissions to developers and other personas with defined responsibilities.

Kubernetes RBAC also governs ServiceAccounts, which provide discrete identities for pods and automation processes. This separation allows human access and workload access to be governed independently, with each scoped to the minimum permissions required for its function. ServiceAccounts follow the principle of least privilege by receiving only the minimum set of permissions required to function correctly, and RBAC should be applied to Secret objects to enforce least-privilege access to sensitive data stored within namespaces.

The Kubernetes RBAC API includes built-in controls against privilege escalation: the API prevents users from editing roles or role bindings to gain permissions beyond those they already hold, and this enforcement applies at the API level regardless of the authorization mode in use. Role-binding authority can be explicitly delegated by granting a user permission to perform the bind verb on a specific Role or ClusterRole, limiting the roles that identity can delegate to others.

In multi-tenant VKS deployments, RBAC is the primary mechanism for isolating tenants within a shared cluster. Administrators use Role and RoleBinding objects at the namespace level to restrict each tenant's access to its assigned namespaces, and ClusterRole and ClusterRoleBinding objects to control access to cluster-level resources, blocking unprivileged users from crossing tenant boundaries.

Supervisor services deployed on VKS clusters extend RBAC to their own access models. The Secret Store Supervisor Service enforces role-based access control across administrator, DevOps engineer, and application tenant roles, controlling who can manage and retrieve sensitive data. The Argo CD Supervisor Service implements RBAC through a configurable policy field using Casbin syntax, with the spec.rbac.policy setting defining role assignments and permission rules; policy rules (p rules) control account permissions including the ability to grant admin roles to specific local accounts, and group mappings (g rules) assign accounts to role groups.

VMware Private AI Services (PAIS) provides role-based access control scoped to its AI services and AI workload delivery components. The central mechanism is OIDC group-based authorization for PAIS. When activating PAIS in an organization namespace, administrators populate the `authorizedGroups` setting with one or more OIDC group identifiers so that only users who belong to those groups can access the service. The OIDC configuration supports a configurable groups claim name through the `oidc-groups-claim` setting, with the groups claim carried in the Access Token and evaluated at access time. PAIS requires an OIDC provider configured as a public client application with Authorization Code and Refresh Token grant types, and it can be configured with one OIDC provider. Additional OAuth 2.0 client identifiers can be accepted by configuring Extra Audiences through the `oidc-extra-audiences` setting when the OIDC provider issues tokens for multiple audiences.

PAIS extends the same identity model to the Grafana observability platform used for PAIS metrics. Grafana role assignment is configured through the `grafana-oidc-role-mapping` property using an expression such as `contains(groups['*'], 'pais-access-group') && 'Admin' || 'Viewer'`, so OIDC group membership maps automatically to Grafana Admin or Viewer roles. PAIS uses the same OIDC provider for Grafana authentication that it uses for service access.

For machine learning model storage, the Model Gallery uses Harbor project access capabilities to restrict access to the sensitive data used to train and tune models. PAIS integrates with Harbor registries as the model storage backend and requires authentication and certificate-based trust for model retrieval. PAIS is compatible with OCI-compliant registries from other vendors when deployed in a disconnected or air-gapped environment, and the project-scoped access controls apply to the chosen registry.

Access to PAIS catalog items for GPU-accelerated VKS clusters and other AI workloads is governed through project-based permissions in VCF Automation. PAIS custom forms in blueprints can be modified to control user inputs at request time, scoping which configuration choices a requester can make when provisioning AI workloads. Knowledge Base data sources used by PAIS retrieval augmented generation require authenticated access to external repositories, including user name and password authentication for SharePoint On-Premises data sources and access keys with list and read permissions on the target bucket for Amazon S3 compatible stores. PAIS uses a CA trust bundle for secure communication with external services and databases.

These PAIS-specific controls operate at the AI services and workload layer. Access to the underlying compute, storage, and networking infrastructure is governed by platform-level RBAC in VCF, including VCF SSO and VMware vCenter role assignments, and database-level access for vector and operational data is governed by Data Services Manager. PAIS adds service-level and workload-level RBAC for AI-specific assets on top of the VCF platform foundation rather than replacing those platform controls.

### **VMware vDefend (Contributes)**

VMware vDefend (vDefend) enforces role-based access control across its security management interfaces to restrict access to firewall policy administration, threat analysis functions, and monitoring capabilities based on assigned user roles.

The vDefend distributed firewall (DFW) implements RBAC to control who can create or modify firewall rules, limiting policy changes to authorized personnel. Access to DFW monitoring features, including Firewall Insights, requires one of three designated roles: Enterprise Admin, Security Admin, or Security Operator. Rule Analysis operations within the platform are also role-gated across the five defined role types.

Security Services Platform (SSP) implements RBAC with five defined roles governing access to platform operations: Enterprise Admin, Auditor, Security Engineer, Security Operator, and Support Bundle Collector. Each role carries differentiated permissions; Full Access grants create, read, update, and delete capabilities, while Read-only access is available for monitoring-oriented roles such as Auditor and Support Bundle Collector. Role assignments can be applied to individual remote users or to remote user groups managed through a directory service, allowing administrators to align vDefend access with existing organizational structures. Multiple roles can be assigned to a single user or group to meet varying operational requirements. SSP's RBAC model also applies to License Hub operations, restricting licensing management actions to authorized roles.

Security Intelligence RBAC in 9.1 uses the same five-role model as SSP (Enterprise Admin, Auditor, Security Engineer, Security Operator, and Support Bundle Collector), aligning Security Intelligence access governance with SSP's permission model and providing consistent role boundaries across both the detection and platform layers.

For organizations using VCF Automation for vDefend firewall management, VCF Automation provides a dedicated Firewall category under RBAC rights, controlling access to the management and viewing of DFW rules, Tenant Gateway Firewall policy, VPC Gateway Firewall policy, and VPC Security Profiles. This allows organizations to scope firewall administration rights independently of other VCF Automation capabilities, supporting separation of duties between network automation and security policy management.

vDefend's RBAC controls complement the broader VCF RBAC model by restricting access to network security policy management independently of compute, storage, or platform configuration access. An administrator with VMware vCenter privileges does not automatically gain the ability to modify firewall rules or security analytics configurations unless explicitly granted the appropriate vDefend roles. This separation of duties between infrastructure administration and security policy management supports distinct team structures for network security management.

### **VCF Protection and Recovery (Contributes)**

Protection and Recovery (PNR) implements role-based access control through purpose-built roles registered in VMware vCenter and through a multi-tenant administrative model for Protection and Recovery for VCF Automation. The SRM Administrator role allows users or user groups to perform all PNR configuration and administration operations. The Protection and Recovery Administrator role can be assigned to user groups to manage permissions. Default roles provide the privileges users require to manage protection groups and perform recoveries, while restricting access to resources belonging to other users.

For automatic protection of virtual machines within a protection group, administrators assign either the SrmAdministrator or SrmProtectionGroupsAdministrator role to the automatic protection user or user group. This scopes the protection actions performed on the user's behalf to the privileges of the assigned role.

Each role is a collection of privileges that defines the actions a user can perform on the protection and recovery site. A user can have at most one role on an object, but roles combine when the user belongs to multiple groups that all have roles on the object. PNR supports propagating roles to child objects in the inventory hierarchy, so administrators can extend privileges from parent objects such as folders to all virtual machines within them.

Protection and Recovery for VCF Automation introduces a multi-tenant role model with Provider Administrator, Organization Administrator, Project Administrator, and Project Advanced User. Each role has specific authorization levels for disaster recovery operations. Provider Administrators select which replication classes each Organization can use, restricting organizational access to specific recovery configurations. Organization Administrators manage the pairing of namespaces and assignment of replication classes within their tenancy.

Underlying directory services, authentication, and enterprise identity governance are provided by VCF SSO and the broader VCF platform. PNR consumes those identities and layers its protection and recovery roles on top of them.

### **VMware Data Services Manager (Meets)**

VMware Data Services Manager (DSM) implements role-based access control natively, with two defined role types that govern access to all data service resources. The DSM Admin role is a super-user role with full access to create, monitor, and manage all data services across the environment. The DSM User role is a restricted role that limits a user to viewing and managing only the data service deployments they own. This separation is enforced at the console level: the views and available actions in the DSM interface differ based on the assigned role, so users with the DSM User role cannot see or act on resources outside their own scope.

Access control is further refined through namespace scoping. DSM User role assignments can be restricted to one or more specific namespaces, limiting access to data services within those assigned namespaces only. Administrators can modify a user's assigned role and namespace access after initial provisioning. DSM namespaces enforce role-based access control that



allows multiple users to work on data services within a shared namespace while maintaining appropriate permission boundaries between them.

DSM supports both local user accounts and directory service integration for identity management. Local users are created through the DSM UI, where administrators specify the user's role, email address, and password. For enterprise directory integration, DSM supports LDAPS-based authentication with Directory Service Group mapping, allowing administrators to assign users to either the DSM Admin or DSM User role based on their LDAP group membership. This group-based mapping is configured through the vSphere Client during DSM setup or through the DSM UI after deployment.

Access control extends to storage and database resources as well. DSM IAM policies for S3-compatible storage support granular access control through policy statements that allow specific S3 actions, including PutObject, GetObject, DeleteObject, and ListMultipartUploadParts. At the database level, PostgreSQL instances support the CREATE ROLE command to create database roles for LDAPS users with configurable login capabilities, and administrators can configure Host-Based Authentication (HBA) rules to restrict which clients and authentication methods may connect to a PostgreSQL instance. For SQL Server instances, DSM applies least-privilege access by granting the cluster administrator login restricted read-only privileges and specific ALTER privileges for server management and auditing, rather than full administrative access. Data service policies also support allowlist configuration to restrict users to selecting only from a specific set of approved parameters when provisioning databases.

The DSM Permissions view provides a centralized interface for administrators to create, monitor, and manage configured users. This view supports ongoing governance of who has access to what within the DSM environment, making it straightforward to audit and adjust role assignments as business needs change.

### **VMware Avi Load Balancer (Meets)**

VMware Avi Load Balancer provides a native, multi-tiered Role-Based Access Control (RBAC) framework that governs all administrative access to the Avi Controller. At the base level, Avi roles assign one of four privilege types to each resource category: Write (full create, read, modify, and delete access), Read (view-only access to configuration, health, and analytics data without modification capability), Assorted (a mixed access level combining permissions across specific operations), and No Access (no access to the resource at all, including the inability to read or list it). Roles are combined with tenants to define the complete authorization settings for each user, scoping what resources a user can interact with and what actions they may take.

Beyond base privilege levels, the Extended Granular RBAC feature refines access using markers, which are key-value label pairs assigned to Avi objects. A role definition specifies that a user may access only objects bearing labels that match the role's marker set. The `allow_unlabelled_access` parameter controls whether a user can view objects that lack matching labels, allowing administrators to restrict visibility to labeled objects only. Label groups can be associated with tenants to enforce label-based access policies across the tenant boundary. Multiple label values can be mapped to a single key, enabling flexible classification schemes. This mechanism enables resource-level scoping across tenants and teams: for example, restricting a group of operators to only the virtual services labeled with their team's identifier, without granting visibility into other teams' objects.

At the finest granularity, Avi implements sub-resource (field-level) RBAC, which scopes which specific fields within a configuration object a given user is authorized to modify. Sub-resource definitions can be combined so that a user can configure multiple fields or features within a single object while remaining restricted from updating others. Label-based permissions for cloud objects extend this field-level control to infrastructure resources managed by the Avi Controller. Virtual service configuration exposes RBAC directly: the RBAC tab in the Virtual Service advanced setup provides label-based access control for health monitors and associated objects, extending role enforcement down to individual service configuration items.

For GSLB deployments, the Avi Controller enforces RBAC through role references such as `Gslb_Group_Enabled` and `Gslb_Health_Monitor` that can be assigned to user accounts with tenant-level scope. These role references apply to follower sites configured with federated JWT profiles, scoping which accounts can modify GSLB configuration.

The Avi Controller applies RBAC within the License Hub, limiting which users can perform specific actions based on their assigned role.



For deployments using the Avi Kubernetes Operator (AKO), the Avi Controller implements RBAC by assigning tenants and roles to user accounts at the Kubernetes or OpenShift cluster level. AKO Custom Resource Definitions (CRDs) can leverage Kubernetes RBAC policies to restrict which user groups can configure routing rules, extending the Controller's authorization model into the Kubernetes control plane.

For deployments on Microsoft Azure, Avi integrates with Azure IAM to control which Azure identities have access to Avi Load Balancer resources hosted in Azure. Administrators can use built-in Azure roles such as Reader (read-only access) or Owner (full access), or define custom roles for finer control over specific resources.

RBAC configuration is managed through the Users section of the Avi Controller UI. Roles are defined per resource type and then assigned to users within a tenant. Extended Granular RBAC markers are attached to individual Avi objects (virtual services, pools, health monitors, and others) and referenced in the role definition. Sub-resource RBAC is configured as part of the Extended Granular RBAC object, combining field-level restrictions as needed.

### Control CR 2.1(3)

> Authorization enforcement: Enforce assigned privileges to all users and software processes. Limit use of privileged accounts: Restrict the use of privileged accounts to only necessary tasks. Mapping of users to roles: Support the mapping of users to specific authorized roles. Supervisor override: Capability for a supervisor to override certain locks/blocks. Dual control: Require two authorized individuals to perform certain sensitive actions. Response to audit processing failures: Capability to protect audit record generation in case of failure. Timestamps: Capability to include timestamps in audit records. Time synchronization: Capability to synchronize the component clock with an NTP source. Time synchronization protection: Protect the time synchronization process from tampering. Non-repudiation: Capability to provide evidence that a user performed a specific action. Non-repudiation for digital signatures: Use of digital signatures for non-repudiation of origin/receipt. Physical tamper resistance and detection: Capability to detect physical tampering of the component.

**SCF Controls:** IAC-16.5

**Aggregate Coverage:** Not Supported

Not supported directly for this control: VCF. This product does not currently provide the capability this control requires.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 2.1(4)

> Authorization enforcement: Enforce assigned privileges to all users and software processes. Limit use of privileged accounts: Restrict the use of privileged accounts to only necessary tasks. Mapping of users to roles: Support the mapping of users to specific authorized roles. Supervisor override: Capability for a supervisor to override certain locks/blocks. Dual control: Require two authorized individuals to perform certain sensitive actions. Response to audit processing failures: Capability to protect audit record generation in case of failure. Timestamps: Capability to include timestamps in audit records. Time synchronization: Capability to synchronize the component clock with an NTP source. Time synchronization protection: Protect the time synchronization process from tampering. Non-repudiation: Capability to provide evidence that a user performed a specific action. Non-repudiation for digital signatures: Use of digital signatures for non-repudiation of origin/receipt. Physical tamper resistance and detection: Capability to detect physical tampering of the component.

**SCF Controls:** CHG-08

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides technical mechanisms that support dual approval requirements for changes to critical infrastructure. Determining which environments qualify as "high-impact" and enforcing dual approval across all change types require organizational processes layered on top of the platform.

VCF Automation approval policies are the primary dual-approval mechanism. Policies support a HARD enforcement type that mandates approval before specified actions can proceed, and the approvalMode can be set to ALL\_OF to require approval from all designated approvers before an action is authorized. Multiple approval policies can be enforceable simultaneously, and when the approvers differ across policies, all approvers are added to the approval chain. Approvers can include individual users, user groups, and role-based approvers such as Project Administrators or Project Supervisors. Approvers receive email notifications when a request is pending, and if any approver rejects the request, it is denied and the requester is notified.

Approval policies can be scoped by criteria such as resource type, resource size, catalog item, or the specific action being requested. This scoping is directly relevant to this control because it allows organizations to target dual approval at their most critical or high-impact change types rather than applying it uniformly. For example, deployments that consume significant cloud resources can be configured to require multiple managers to review the request before provisioning.

Day 2 action policies extend the approval model to operations on existing deployments. These policies create a list of permitted actions, restricting which users can run specific operations (such as power operations, scaling, lease changes, or deployment modifications). Approval policies can be applied to these day 2 actions so that high-impact operations on production resources require the same dual-approval gate as the original provisioning request.

Policy as Code in VCF Automation enables Organization Administrators to enforce instance-centric policies for IaaS resources from a central location, reducing the risk of unauthorized or unilateral modifications to the policy framework itself. Custom user roles can be defined to give specific individuals the ability to review and respond to approval requests, establishing a reviewer role distinct from the role that initiates changes.

For network and security policy changes, NSX supports role-based policy authoring with separate categories: a security administrator can create policies in the emergency category, an application developer can create policies in the application category, and a network administrator can create policies in the infrastructure category. This category-based separation means that changes to security policies and network infrastructure policies are authored by different roles, reinforcing separation of responsibilities at the network layer.

VCF Automation integrates with ServiceNow through the ServiceNow ITSM Plug-in, which imports VCF Automation catalog items into ServiceNow and supports approval groups, email notifications, and request status tracking. Organizations that require dual approval can route high-impact change requests through their existing ServiceNow change workflows, where approval chains and emergency change procedures may already be defined.

VCF's role-based access control model across vCenter, NSX, and VCF Operations allows organizations to separate the privileges needed to propose, approve, and implement changes across different user accounts. This separation of duties provides the foundation for dual approval workflows, so that no single account holds both the authority to request a change and the authority to execute it.

Direct changes made through the vCenter, ESX, or NSX management interfaces are governed by role-based access control but do not pass through VCF Automation's approval workflow. Extending dual approval requirements to all change types, including those outside VCF Automation, requires organizational change management processes that prescribe which changes require dual approval and route them through appropriate review before execution.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 2.2

> Wireless use control: Capability to authorize and restrict the use of wireless communications.

**SCF Controls:** CFG-02, CFG-03

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides secure baseline configurations for all major platform components, consistent with industry-accepted hardening standards. The VCF Security Baseline is a purpose-built document that builds on VCF security fundamentals to address frequently requested compliance standards, regulations, and frameworks. It is designed and validated to tailor security configurations without affecting VCF's ability to meet its design objectives. Organizations can use the VCF Security Baseline to evaluate both default and non-default configurations and to secure their environments in a compliance context.

VCF Operations includes a consolidated out-of-the-box benchmark called the VCF 9 Security Baseline, which provides a ready-to-use compliance profile for evaluating the security posture of a VCF environment. This benchmark enables automated assessment of configurations against the documented baseline, helping maintain a consistent configuration across VCF components. ISO Security Standards compliance benchmarks are also available from the VCF Operations repository as a native pack, giving organizations the ability to measure their environment against internationally recognized hardening standards. Broadcom also provides DISA Security Technical Implementation Guides (STIGs) for VCF components, including vSphere 7.0 and vSphere 8.0 STIGs, which provide detailed instructions for securing and configuring systems to meet specific security requirements.

Individual VCF components have their own security configuration guides that document hardening recommendations. The vSphere Security Configuration Guide provides best practices for securing VMware vCenter, including securing the installation, configuring access and authentication, defining roles and permissions, and enabling auditing and logging. VMware ESX host hardening covers authentication and authorization settings, disabling unnecessary services, enabling security features such as Secure Boot and Trusted Platform Module (TPM), and implementing network security measures. The vSphere Security Configuration Guide documents specific STIG-aligned values for the `Security.PasswordQualityControlPwquality` setting on ESX hosts, including parameters such as `minlen=15`, `maxrepeat=3`, `maxclassrepeat=4`, `difok=4`, and `enforce_for_root`. The NSX Security Configuration Guide provides recommendations for securing NSX components, including NSX nodes, NSX Controllers, and NSX Edge nodes, by configuring strong authentication, implementing role-based access control, and securing administrative access.

VCF Operations also provides a Configure Host Security Config Data workflow that allows selective enforcement of host security hardening rules. Beyond security settings, vSphere Lifecycle Manager maintains a desired-state cluster image that defines the ESX base build, vendor add-ons, and firmware for every host in a cluster, so the software baseline stays consistent across hosts, and VCF Operations runs scheduled drift detection that flags configuration deviations from the desired state for remediation. Administrators can configure individual configuration elements under Enforce Options, enabling granular control over which baseline settings are applied and enforced across the environment. This supports reducing organizational and business risk by maintaining configuration consistency. vSphere Host Profiles provide an additional mechanism for automated and centrally managed host configuration baseline enforcement, capturing a reference host's configuration and applying it consistently across ESX hosts, covering memory, storage, networking, and other configuration aspects.

For containerized workloads, VMware Kubernetes Service (VKS) includes preconfigured security policy templates. The Baseline security template provides a set of constraints that restrict known privilege escalations while maintaining compatibility with common containerized workloads.

DISA STIGs for Supervisor and VKRs are available to guide hardening of the Supervisor control plane and individual VKS cluster releases, providing baseline security guidance at the same standard as the DISA STIGs applied to VCF infrastructure components. The VKS cluster management interface provides out-of-the-box security policy templates that mirror industry-standard Pod Security Standards profiles, including both Baseline and Restricted configurations. Custom VKS Cluster

Management Security Policy templates are also available to fine-tune individual security context parameters and support governance controls tailored to specific organizational requirements.

Pod Security Admission (PSA), enabled in VKr 1.25 and later, enforces pod security standards at the namespace level. Three policy profiles are defined: Privileged (unrestricted, for system-level workloads), Baseline (helps prevent known privilege escalations while allowing the default minimally specified pod configuration), and Restricted (heavily restricted, following pod hardening best practices). Three enforcement modes are available: enforce, warn, and audit. The `podSecurityStandard` field in the ClusterClass API encodes the intended security posture in cluster definitions, accepting these modes with specific policy versions and namespace exemptions so that baseline security configurations are applied consistently across VKS deployments. System namespaces including `kube-system`, `tkg-system`, and `vmware-system-cloudprovider` are excluded from pod security enforcement. The PSA admission controller can also be statically configured with default enforce, audit, and warn levels for unlabeled namespaces, so namespaces created without explicit labels still receive a defined security baseline. To support gradual policy adoption, audit and warn modes can be set to a stricter level such as restricted while enforce remains at baseline, giving administrators visibility into policy violations before full enforcement is applied. VCF Automation IaaS Resource Policies provide an additional enforcement point at the management layer, supporting requirements that Kubernetes clusters operate at a minimum baseline pod security level.

Kubernetes-native kernel security mechanisms provide workload-level baseline hardening options. `seccomp` restricts the Linux kernel system call surface available inside containers using BPF-based profiles, and the `RuntimeDefault` `seccomp` profile provides a strong set of security defaults while preserving workload functionality. AppArmor and SELinux provide further kernel-level restrictions through default configurations that offer foundational protections. Kubernetes default RBAC role bindings should be reviewed as part of establishing a security baseline, as defaults may grant broader access than a hardened configuration requires. Isolating each workload in its own Kubernetes namespace allows tailored security policies to be applied per workload and restricts access to namespace-scoped resources such as ConfigMaps and Secrets.

Harbor Registry, available as a standard package for VKS clusters, supports a controlled baseline for container artifacts entering the environment. Harbor provides vulnerability scanning, content trust, and policy-based replication, along with role-based access control and artifact security policies, supporting controlled management of the container image supply chain. For stand-alone virtual machines deployed in vSphere Supervisor environments, Content Libraries can distribute VM images created with current patches and required security settings that follow corporate security policies.

VMware Private AI Services (PAIS) provides PAIS-specific declarative configuration mechanisms that organizations can use to define and apply baseline settings across the AI infrastructure stack on VCF.

The primary mechanism is the `PAISConfiguration` custom resource. Private AI Services settings are declared in a YAML file compliant with the `PAISConfiguration` schema and applied with `kubectl`, giving administrators a single resource that captures observability, identity, and trust configuration for an organization namespace. Observability includes the `spec.observability.prometheusRuntime` section for Prometheus metrics collection with a configurable storage class and metrics retention period, and an `llmTraces` section for routing large language model traces to an OpenTelemetry Collector. The `PAISConfiguration` custom resource can be patched with `kubectl patch` and verified with `kubectl get paisconfiguration`, supporting baseline change control and drift detection at the AI service layer.

Certificate trust is handled through the `spec.clientTls.caBundleRefs` section, which references ConfigMaps containing CA trust bundles. Private AI Services requires HTTPS trust bundles for the OIDC provider, the Harbor registry, content management systems hosting knowledge database data, and MCP servers that provide tools to models. Trust bundles can be organized per application, with periodic updates required before certificate expiration. Access scope is restricted through the `authorizedGroups` setting, which limits Private AI Services access to specified OIDC groups.

Harbor registry distribution carries baseline controls for model and image artifacts. Replication rules support configurable filters, target namespace, and trigger mode, including a manual trigger mode, so administrators can align artifact replication with organizational policy for connected and disconnected sites. The Model Gallery uses Harbor project access capabilities to restrict access to sensitive training and tuning data. Air-gapped deployments capture the self-hosted registry URL and registry login credentials as part of the activation flow.

Workload resource baselines are defined through `SupervisorNamespaceClassConfig`, which specifies per-zone CPU limits, CPU reservations, memory limits, and memory reservations for resource allocation across zones. GPU-capable VM classes can carry GPU reservations and be confined to a single zone or supervisor namespace, supporting isolation of GPU consumption to a specific tenant. A default storage policy selection in the Private AI Services quickstart defines storage placement and resources for the service database.

Deep Learning VM (DL VM) baselines are captured through VCF Automation self-service catalog items that parameterize VM class, data disk size, user password, and optional SSH public key authentication for remote access. The DL VM OVF property `config-json` accepts a base64-encoded configuration file that controls DCGM Exporter metrics visibility through `export_dcgml_to_public` and JupyterLab authentication through `enable_jupyter_auth`, giving operators a consistent surface to define DL VM hardening parameters.

Credentials supplied during Private AI Services activation, including the private container registry password and proxy password, are stored as secrets in VCF Automation rather than as plaintext configuration values, supporting credential hygiene as part of the deployment baseline.

### VMware vDefend (Meets)

VMware vDefend contributes to secure baseline configurations within its network security domain by providing structured configuration management capabilities, predefined security profiles and strategies, and configuration integrity controls that align with industry-accepted hardening practices.

vDefend supports draft configuration management with configuration locking to prevent multiple users from simultaneously opening and editing a manual draft, helping maintain configuration integrity and reducing the risk of conflicting changes to firewall policy. Role-Based Access Control (RBAC) in vDefend governs who can create or modify firewall policies, providing control over which administrators can alter the security baseline.

In vDefend, five system-defined VPC Security Strategies are available for establishing security baselines across Virtual Private Clouds (VPCs): VPC Isolation, VPC Isolation with Essential Services, VPC External Connectivity, VPC Secure Connection, and NONE. These strategies are consumed through Security Profiles in VCF Automation, which are predefined configuration templates that define the default security posture for VPCs. VPC Security Strategies provide a declarative, consistent way to apply DFW security baselines across multiple VPCs within a project, and Project Admins are guided to select a pre-defined high-level security posture that enforces Secure by Default principles rather than manually configuring individual firewall rules. Security profiles can be applied to one or more VPCs simultaneously and updated as requirements evolve.

vDefend provides predefined TLS Decryption Action Profiles with four settings: Balanced (default), High Fidelity, High Security, and Custom. These profiles are vendor-defined baseline configurations for TLS inspection behavior, giving organizations a documented starting point aligned with different security postures. TLS Inspection policies also support an Advanced Configuration option to lock policies, preventing multiple users from making concurrent changes to TLS inspection policy baseline settings.

Security Services Platform validates configuration changes before implementation through a deployment wizard that conducts thorough validations to confirm configurations are correct and compatible. The platform enforces initial security policies that cannot be modified through the user interface after they have been published, providing an immutability mechanism for foundational policy baseline elements. Security Services Platform supports backup and restore of configuration data, and certificate consistency between backup and restored environments should be verified to maintain NDR sensor integrity.

Security Services Platform's Bare Metal Security feature extends vDefend policy enforcement to physical servers, applying L3/4 security policies to bare metal infrastructure using the same policy model and management plane as virtualized workloads. Two default DFW rules are created when Bare Metal servers are onboarded to Security Services Platform, providing an immediate baseline posture for physical infrastructure. Security policies are defined, propagated, and enforced across Bare Metal servers with consistent application and monitoring.

vDefend's configuration management capabilities operate within the network security layer. The broader secure baseline configuration program for VCF, including the VCF Security Baseline document, component hardening guides, DISA STIGs,



and compliance scanning through VCF Operations, is managed at the platform level. vDefend contributes by maintaining its own secure defaults and configuration controls within the network security domain.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) provides configurable security baseline mechanisms for its replication and recovery infrastructure. The PNR appliance includes a security reference section that identifies configuration files containing settings that affect environment security, and PNR documentation establishes that secure operation depends on proper configuration and maintenance of the PNR server. The security reference also documents network port requirements, providing the technical specifications needed to develop and maintain secure baseline configurations for disaster recovery infrastructure.

PNR includes a Configuration Import/Export Tool that captures the complete set of protection and recovery configuration settings, including server version, build number, local and remote site names, inventory mappings, placeholder datastores, advanced settings, and array managers. The tool supports both UI-based and scripted exports and can transfer configuration data without requiring credential entry when using the bundled generated script. This gives organizations a mechanism to document, back up, and restore known-good baseline configurations for their disaster recovery infrastructure. The underlying system configuration is stored in the `va-configurator.xml` file and is accessible through Advanced Settings on the Site Pair tab in the management interface, providing a defined location for baseline review and modification.

The PNR appliance management interface allows administrators to configure appliance security settings after deployment. The Advanced Settings system provides privileged users with the ability to adjust default values governing protection and recovery features. When modifying the minimum protocol version for TLS, PNR requires that the change be applied consistently across all configuration occurrences in the system, supporting systematic baseline management for transport security settings.

For cyber recovery deployments, PNR documentation recommends using standard VCF topology with separate workload and management domains to achieve better isolation and align with security best practices at the recovery site.

Full compliance with this control requires an organizational baseline configuration management program, documentation practices, version control, and hardening tooling extending beyond what PNR provides directly.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) contributes to secure baseline configuration management through its infrastructure policy framework, data service policy enforcement, and built-in secure defaults for authentication and certificate management.

Infrastructure policies in DSM define the compute, storage, and networking resources for each database deployment, including the list of allowed VM classes and compatible storage policies. These policies are configured by DSM administrators and act as standardized, repeatable deployment baselines that database users must follow. When administrators configure infrastructure policies, they constrain the quality and quantity of resources that users can consume from vSphere clusters, establishing guardrails that support consistent deployments across the environment. Database cluster creation applies both infrastructure policies and storage policies that enforce isolation and resource constraints, reducing the risk of misconfiguration by individual users.

Data service policies extend this baseline enforcement across multi-tenant deployments. DSM administrators can define and configure data service policies for various database services and enforce them across different tenants to maintain a consistent security and compliance posture. Data service policies constrain which infrastructure options can be applied to database instances. When DSM is integrated with VCF Automation, data service policies can be applied consistently across organizational units.

DSM also ships with predefined secure defaults for database authentication. PostgreSQL databases on DSM use `scram-sha-256` as the default password-based authentication method, as configured in the default `pg_hba.conf` entries stored at `/pgsql/data/pg_hba.conf`. Custom `pg_hba.conf` entries support `scram-sha-256`, `reject`, and `cert` authentication methods. TLS is enforced by default for database client connections, with an administrator-controlled option to relax enforcement via the `postgrescluster-allow-non-ssl-system-users` setting in the advanced-system-config ConfigMap within the `dsm-system` namespace.

Certificate management is integrated into the database creation workflow and validates certificate chains for connections to external systems, including VMware vCenter. DSM introduces a stricter default certificate validation mode, the "latest" mode, that enforces cryptographic and KeyUsage validations for all DSM endpoints, including the Provider VM and data service clusters. Certificates uploaded to DSM are validated against FIPS 140-3 cryptographic standards, and uploads containing unsupported algorithms or parameters are rejected. Custom trusted root certificates must comply with FIPS requirements. DSM also verifies the SHA256 thumbprint of vCenter during initial configuration to confirm the management connection targets the expected endpoint. Custom certificate support allows organizations to replace the default DSM certificate with organization-issued certificates for both the Provider VM and database clusters. For self-service deployments using the DSM Consumption Operator, signature verification confirms package integrity before installation.

DSM validates certificate chains using configured root CA certificates stored in ConfigMaps and supports private container registry configuration with CA certificate validation for air-gapped environments.

Addressing this control fully requires organizational processes beyond what DSM provides directly. DSM does not include tooling to document baseline configurations in a compliance-ready format, map configurations to specific industry hardening standards, or audit deviations from defined baselines over time. Organizations should supplement DSM's policy and defaults framework with configuration management documentation, change tracking processes, and integration with compliance scanning tools to satisfy the full requirements of this control.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides several built-in mechanisms that support the development and application of secure baseline configurations for the application delivery layer.

The SSL/TLS Profile system gives administrators a documented starting point for cryptographic configuration. Profile Templates define accepted SSL and TLS versions and ciphers in prioritized order, and can be associated with virtual services or the Controller itself. The Standard SSL/TLS Profile is suitable for typical deployments, while the System-Standard profile provides modern cipher suites for backend server re-encryption in Kubernetes environments managed through the Avi Kubernetes Operator (AKO). Avi includes a curated list of cipher suites considered reasonably strong for SSL/TLS configuration, and the documentation notes the trade-offs between security, compatibility, and computational expense when selecting profiles. The Service Engine disables unsafe legacy renegotiation by default, providing a built-in protocol hardening baseline that reduces the risk of protocol downgrade attacks.

The System-Secure-HTTP application profile is a pre-configured baseline for securing HTTP virtual services. It enforces HTTPS, protects cookies, manages client IP handling, and controls protocol behavior, giving administrators a hardened starting point for web application delivery. Administrators can supplement these defaults with more granular settings through policies or DataScripts.

For deployments requiring validated cryptographic modules, Avi Load Balancer implements FIPS 140-3 compliance, providing a cryptographic baseline aligned with the U.S. and Canadian government standard for cryptographic modules.

The Avi Web Application Firewall (WAF) Positive Security model supports application-layer baseline configuration through rules that define allowed application behavior. Each rule supports Enable/Disable control, mode selection between Detection and Enforcement, and configurable Paranoia levels, allowing organizations to tune an application security baseline to their risk posture. The Positive Security Rule Updates feature can be enabled to activate additional configuration options for application security policies. The Cloud Console Application Rules service extends this with rules specifically designed to address known application vulnerabilities, including those with CVEs, and updates them automatically.

For Kubernetes deployments, AKO uses Custom Resource Definitions (CRDs) to enable syntactical validation of configuration objects at creation time, which helps prevent invalid configurations from being deployed. The AKO L4 CRD Rule supports SSL profiles and PKI profiles for securing pool connections, allowing Kubernetes-native expression of Avi's cryptographic baseline settings.

Certificate lifecycle management supports maintaining cryptographic baselines over time. Certificate Management Profiles support automated certificate renewal workflows, and the Trust Protection Platform integration can automatically handle certificate and chain certificate renewals by learning the mapping of virtual service names to their certificates.

The organizational process of developing, documenting, and maintaining baseline configurations across all in-scope technology assets, including selecting appropriate profile settings, applying them consistently, and updating them as software versions and security guidance evolve, remains outside what Avi performs on the organization's behalf.

## Control CR 2.5

> Session lock: Inhibit further access to the component after a period of inactivity.

**SCF Controls:** IAC-24

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF components provide session timeout and automatic logout capabilities rather than true session lock functionality. Session lock, which freezes the current session display and requires re-authentication to resume, is not natively supported by VCF management interfaces. VCF's session management controls terminate idle sessions and require users to re-authenticate.

NSX Manager explicitly does not support session lock. The documented guidance is to use the workstation operating system's screen lock to initiate a session lock when accessing NSX Manager. NSX Manager does support session timeout and automatic user logout, and idle sessions time out automatically. Administrators can also delete idle sessions through the API. These timeout mechanisms terminate inactive sessions, which forces re-authentication but does not preserve the session state in a locked condition.

ESX hosts provide two configurable shell-level session timeout settings. The ``UserVars.ESXiShellInteractiveTimeOut`` setting logs out users from idle interactive ESX Shell sessions after a configurable period of inactivity. The ``UserVars.ESXiShellTimeOut`` setting controls shell availability and disables the ESX Shell service entirely after a specified period, preventing new logins until the service is re-enabled. Both settings can be configured through the vSphere Client, VMware Host Client, or DCUI.

VMware vCenter provides session management through the SessionManager managed object, which controls user access through methods for logging in, obtaining a session, and logging out. Each session is associated with the authenticated user account for the duration of the session. The SessionManager uses a session ID as a temporary substitute for username and password, limiting exposure of the principal's credentials during active sessions. SessionManager also defines the visibility of many objects, and session-specific objects are not visible outside the session in which they were created. While vCenter manages session lifecycle, the session lock behavior depends on the client interface and the workstation operating system.

VCF Operations for Networks provides a configurable user session timeout value that administrators can adjust according to organizational requirements, with a range from 15 minutes to 24 hours and a default of 15 minutes. Defining a Release Session Request is recommended when using session authentication to log out sessions, reduce session leaks, and improve security. These capabilities allow organizations to enforce inactivity-based session termination aligned with their defined time periods.

For command-line access through VCF PowerCLI, the `WebOperationTimeoutSeconds` setting can be configured at session scope using the `Set-PowerCLIConfiguration` cmdlet. This allows administrators to control how long PowerCLI sessions remain active before timing out, and the setting can be reverted to its initial value.

For API-based access, service accounts authenticate using API access tokens. The VCF Automation API refresh token is valid for 90 days and can be used to generate new access tokens when the access token expires. If rotation is activated, each refresh token grant call returns both a new access token and a new refresh token, invalidating the original refresh token. Users can also revoke API tokens through the User preferences page. This token lifecycle provides time-bounded session control for programmatic access.

Organizations should configure workstation-level screen lock policies through their endpoint management tools (such as Group Policy for Windows systems) to provide the session lock behavior that VCF's web-based management interfaces do not natively implement. VCF's session timeout settings should be configured to align with the organization's defined inactivity period, and workstation screen lock timeouts should be set to equal or shorter intervals.

### VMware Avi Load Balancer (Contributes)

VMware Avi Load Balancer provides session timeout and termination mechanisms at two layers: the administrative control plane (the Avi Controller) and the application data plane (virtual services and Kubernetes ingress).

For administrative access to the Avi Controller GUI and REST API, the User Account Security settings include a configurable idle timeout, accessible under User Authentication and Authorization > User Account Security > User Account Self-Service > Configuring Idle Timeout for Admin Access to GUI and API. This setting terminates sessions that have been inactive for an administrator-defined period, after which the administrator must re-authenticate before regaining access. Administrators can also explicitly invalidate all active sessions using the shell `--clear-sessions` command. The Maximum Concurrent Login Sessions feature, configured through the `max-concurrent-login-sessions` setting in the `UserAccountProfile` object (manageable via CLI or REST API), limits the number of simultaneous active sessions per user account. The default value of 0 disables concurrent session limiting; organizations should set this to a non-zero value appropriate for their environment.

For application sessions managed through Avi virtual services, the Auth Credentials Cache Expiration setting controls the maximum time a client's authentication credentials remain cached. This bounds the window during which a previously authenticated client can access resources without re-presenting credentials, supporting time-bounded session validity for application traffic. Organizations should configure the HTTP Cookie Persistence timeout to address cases where a client browser does not clear session cookies when a session ends or the browser is closed.

For application workloads using OAuth or OIDC-based authentication, Avi's RP-Initiated Logout support allows a logout event to clear the user's session details from the session database. Subsequent access attempts initiate the OAuth login flow, requiring users to provide credentials again before accessing protected resources.

For Kubernetes workloads managed through the Avi Kubernetes Operator (AKO), the `SSORule` custom resource definition (CRD) provides session timeout controls for virtual services handling OAuth/OIDC and SAML authentication. The `cookieTimeout` parameter specifies the authorized session cookie expiration period, controlling how long an OAuth or OIDC session remains valid before the user must re-authenticate. The `useIdpSessionTimeout` property allows the Identity Provider to control the Service Provider session duration through the `SessionNotOnOrAfter` field in the SAML `AuthNStatement`, enabling organizations to delegate session lifetime enforcement to their external IdP.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 2.5(a)

> Sub-item (a) of CR 2.5 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 2.5 requirement in the standard.

**SCF Controls:** IAC-24

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF components provide session timeout and automatic logout capabilities rather than true session lock functionality. Session lock, which freezes the current session display and requires re-authentication to resume, is not natively supported by VCF management interfaces. VCF's session management controls terminate idle sessions and require users to re-authenticate.

NSX Manager explicitly does not support session lock. The documented guidance is to use the workstation operating system's screen lock to initiate a session lock when accessing NSX Manager. NSX Manager does support session timeout and automatic user logout, and idle sessions time out automatically. Administrators can also delete idle sessions through the API. These timeout mechanisms terminate inactive sessions, which forces re-authentication but does not preserve the session state in a locked condition.

ESX hosts provide two configurable shell-level session timeout settings. The ``UserVars.ESXiShellInteractiveTimeOut`` setting logs out users from idle interactive ESX Shell sessions after a configurable period of inactivity. The ``UserVars.ESXiShellTimeOut`` setting controls shell availability and disables the ESX Shell service entirely after a specified period, preventing new logins until the service is re-enabled. Both settings can be configured through the vSphere Client, VMware Host Client, or DCUI.

VMware vCenter provides session management through the SessionManager managed object, which controls user access through methods for logging in, obtaining a session, and logging out. Each session is associated with the authenticated user account for the duration of the session. The SessionManager uses a session ID as a temporary substitute for username and password, limiting exposure of the principal's credentials during active sessions. SessionManager also defines the visibility of many objects, and session-specific objects are not visible outside the session in which they were created. While vCenter manages session lifecycle, the session lock behavior depends on the client interface and the workstation operating system.

VCF Operations for Networks provides a configurable user session timeout value that administrators can adjust according to organizational requirements, with a range from 15 minutes to 24 hours and a default of 15 minutes. Defining a Release Session Request is recommended when using session authentication to log out sessions, reduce session leaks, and improve security. These capabilities allow organizations to enforce inactivity-based session termination aligned with their defined time periods.

For command-line access through VCF PowerCLI, the WebOperationTimeoutSeconds setting can be configured at session scope using the Set-PowerCLIConfiguration cmdlet. This allows administrators to control how long PowerCLI sessions remain active before timing out, and the setting can be reverted to its initial value.

For API-based access, service accounts authenticate using API access tokens. The VCF Automation API refresh token is valid for 90 days and can be used to generate new access tokens when the access token expires. If rotation is activated, each refresh token grant call returns both a new access token and a new refresh token, invalidating the original refresh token. Users can also revoke API tokens through the User preferences page. This token lifecycle provides time-bounded session control for programmatic access.

Organizations should configure workstation-level screen lock policies through their endpoint management tools (such as Group Policy for Windows systems) to provide the session lock behavior that VCF's web-based management interfaces do not natively implement. VCF's session timeout settings should be configured to align with the organization's defined inactivity period, and workstation screen lock timeouts should be set to equal or shorter intervals.

### VMware Avi Load Balancer (Contributes)

VMware Avi Load Balancer provides session timeout and termination mechanisms at two layers: the administrative control plane (the Avi Controller) and the application data plane (virtual services and Kubernetes ingress).

For administrative access to the Avi Controller GUI and REST API, the User Account Security settings include a configurable idle timeout, accessible under User Authentication and Authorization > User Account Security > User Account Self-Service > Configuring Idle Timeout for Admin Access to GUI and API. This setting terminates sessions that have been inactive for an administrator-defined period, after which the administrator must re-authenticate before regaining access. Administrators can also explicitly invalidate all active sessions using the shell `--clear-sessions` command. The Maximum Concurrent Login Sessions feature, configured through the `max-concurrent-login-sessions` setting in the `UserAccountProfile` object (manageable via CLI or REST API), limits the number of simultaneous active sessions per user account. The default value of 0 disables concurrent session limiting; organizations should set this to a non-zero value appropriate for their environment.

For application sessions managed through Avi virtual services, the Auth Credentials Cache Expiration setting controls the maximum time a client's authentication credentials remain cached. This bounds the window during which a previously authenticated client can access resources without re-presenting credentials, supporting time-bounded session validity for application traffic. Organizations should configure the HTTP Cookie Persistence timeout to address cases where a client browser does not clear session cookies when a session ends or the browser is closed.

For application workloads using OAuth or OIDC-based authentication, Avi's RP-Initiated Logout support allows a logout event to clear the user's session details from the session database. Subsequent access attempts initiate the OAuth login flow, requiring users to provide credentials again before accessing protected resources.

For Kubernetes workloads managed through the Avi Kubernetes Operator (AKO), the `SSORule` custom resource definition (CRD) provides session timeout controls for virtual services handling OAuth/OIDC and SAML authentication. The `cookieTimeout` parameter specifies the authorized session cookie expiration period, controlling how long an OAuth or OIDC session remains valid before the user must re-authenticate. The `useIdpSessionTimeout` property allows the Identity Provider to control the Service Provider session duration through the `SessionNotOnOrAfter` field in the SAML `AuthNStatement`, enabling organizations to delegate session lifetime enforcement to their external IdP.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.



## Control CR 2.5(b)

> Sub-item (b) of CR 2.5 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 2.5 requirement in the standard.

**SCF Controls:** IAC-24

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF components provide session timeout and automatic logout capabilities rather than true session lock functionality. Session lock, which freezes the current session display and requires re-authentication to resume, is not natively supported by VCF management interfaces. VCF's session management controls terminate idle sessions and require users to re-authenticate.

NSX Manager explicitly does not support session lock. The documented guidance is to use the workstation operating system's screen lock to initiate a session lock when accessing NSX Manager. NSX Manager does support session timeout and automatic user logout, and idle sessions time out automatically. Administrators can also delete idle sessions through the API. These timeout mechanisms terminate inactive sessions, which forces re-authentication but does not preserve the session state in a locked condition.

ESX hosts provide two configurable shell-level session timeout settings. The ``UserVars.ESXiShellInteractiveTimeOut`` setting logs out users from idle interactive ESX Shell sessions after a configurable period of inactivity. The ``UserVars.ESXiShellTimeOut`` setting controls shell availability and disables the ESX Shell service entirely after a specified period, preventing new logins until the service is re-enabled. Both settings can be configured through the vSphere Client, VMware Host Client, or DCUI.

VMware vCenter provides session management through the SessionManager managed object, which controls user access through methods for logging in, obtaining a session, and logging out. Each session is associated with the authenticated user account for the duration of the session. The SessionManager uses a session ID as a temporary substitute for username and password, limiting exposure of the principal's credentials during active sessions. SessionManager also defines the visibility of many objects, and session-specific objects are not visible outside the session in which they were created. While vCenter manages session lifecycle, the session lock behavior depends on the client interface and the workstation operating system.

VCF Operations for Networks provides a configurable user session timeout value that administrators can adjust according to organizational requirements, with a range from 15 minutes to 24 hours and a default of 15 minutes. Defining a Release Session Request is recommended when using session authentication to log out sessions, reduce session leaks, and improve security. These capabilities allow organizations to enforce inactivity-based session termination aligned with their defined time periods.

For command-line access through VCF PowerCLI, the WebOperationTimeoutSeconds setting can be configured at session scope using the Set-PowerCLIConfiguration cmdlet. This allows administrators to control how long PowerCLI sessions remain active before timing out, and the setting can be reverted to its initial value.

For API-based access, service accounts authenticate using API access tokens. The VCF Automation API refresh token is valid for 90 days and can be used to generate new access tokens when the access token expires. If rotation is activated, each refresh token grant call returns both a new access token and a new refresh token, invalidating the original refresh token. Users can also revoke API tokens through the User preferences page. This token lifecycle provides time-bounded session control for programmatic access.

Organizations should configure workstation-level screen lock policies through their endpoint management tools (such as Group Policy for Windows systems) to provide the session lock behavior that VCF's web-based management interfaces do not natively implement. VCF's session timeout settings should be configured to align with the organization's defined inactivity period, and workstation screen lock timeouts should be set to equal or shorter intervals.

### VMware Avi Load Balancer (Contributes)

VMware Avi Load Balancer provides session timeout and termination mechanisms at two layers: the administrative control plane (the Avi Controller) and the application data plane (virtual services and Kubernetes ingress).

For administrative access to the Avi Controller GUI and REST API, the User Account Security settings include a configurable idle timeout, accessible under User Authentication and Authorization > User Account Security > User Account Self-Service > Configuring Idle Timeout for Admin Access to GUI and API. This setting terminates sessions that have been inactive for an administrator-defined period, after which the administrator must re-authenticate before regaining access. Administrators can also explicitly invalidate all active sessions using the shell `--clear-sessions` command. The Maximum Concurrent Login Sessions feature, configured through the `max-concurrent-login-sessions` setting in the `UserAccountProfile` object (manageable via CLI or REST API), limits the number of simultaneous active sessions per user account. The default value of 0 disables concurrent session limiting; organizations should set this to a non-zero value appropriate for their environment.

For application sessions managed through Avi virtual services, the Auth Credentials Cache Expiration setting controls the maximum time a client's authentication credentials remain cached. This bounds the window during which a previously authenticated client can access resources without re-presenting credentials, supporting time-bounded session validity for application traffic. Organizations should configure the HTTP Cookie Persistence timeout to address cases where a client browser does not clear session cookies when a session ends or the browser is closed.

For application workloads using OAuth or OIDC-based authentication, Avi's RP-Initiated Logout support allows a logout event to clear the user's session details from the session database. Subsequent access attempts initiate the OAuth login flow, requiring users to provide credentials again before accessing protected resources.

For Kubernetes workloads managed through the Avi Kubernetes Operator (AKO), the `SSORule` custom resource definition (CRD) provides session timeout controls for virtual services handling OAuth/OIDC and SAML authentication. The `cookieTimeout` parameter specifies the authorized session cookie expiration period, controlling how long an OAuth or OIDC session remains valid before the user must re-authenticate. The `useIdpSessionTimeout` property allows the Identity Provider to control the Service Provider session duration through the `SessionNotOnOrAfter` field in the SAML `AuthNStatement`, enabling organizations to delegate session lifetime enforcement to their external IdP.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 2.6

> Remote session termination: Capability to terminate a remote session by an authorized user.

**SCF Controls:** NET-07

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides configurable session timeout and connection termination mechanisms across its management interfaces, hypervisor layer, and programmatic access points. Sessions terminate on explicit logout and after configurable inactivity periods, with the specific timeout values set by administrators to match organizational policy.

NSX Manager sessions expire after 1800 seconds (30 minutes) of inactivity by default. Administrators can adjust this value using the CLI command "set service http session-timeout" with a value in seconds, followed by a UI service restart. The connection\_timeout API parameter is also configurable for API service connections. Idle sessions are terminated automatically, and NSX Manager immediately destroys the session identifier when any user logs out, preventing session identifier reuse. The system recognizes only system-generated session identifiers and invalidates them upon administrator logout or other session termination. Administrators can also explicitly delete active sessions through the /api/session/destroy API endpoint.

On the hypervisor layer, VMware ESX provides a Host Client Session Timeout setting that controls the duration before an inactive Host Client session is terminated, giving administrators direct control over idle session lifetimes on individual hosts. This setting maps to the UserVars.HostClientSessionTimeout advanced system setting, which specifies idle time in seconds. ESX also provides shell-layer timeout controls: the ESX-Shell-idle-timeout setting terminates idle interactive shell sessions after a configurable period, and the ESX-Shell-availability-timeout setting deactivates the ESX Shell service itself after a specified period of inactivity, preventing further logins until re-enabled.

VCF Operations for Networks enforces a User Session Timeout that defaults to 15 minutes and can be configured from 15 minutes to 24 hours, allowing organizations to align session duration with their security policies. The VCF Operations orchestrator also supports configuring timeout periods for user interactions through the timeout.date variable. VCF Operations for Networks exposes a Session Count metric that tracks the total number of allowed sessions at configurable intervals, providing visibility into active session counts for monitoring purposes.

For programmatic access, the vSphere Web Services API provides a logout() method that clients use to close authenticated session connections. The documentation states that API clients must always close server connections by calling the logout method. The vSphere Client enforces a separate inactivity timeout that defaults to 120 minutes and is configurable via the session.timeout parameter, specified in minutes. The VCF Operations PowerShell plug-in provides a closeSession method that can terminate sessions by session ID, enabling scripted or automated session cleanup.

The VMware Kubernetes Service (VKS) and vSphere Supervisor layer within VCF extends connection termination controls to containerized workloads. At the node level, the kubelet streamingConnectionIdleTimeout setting governs the maximum time that streaming connections (exec, attach, and port-forward operations) can remain idle before automatic closure; in VKS clusters using the built-in ClusterClass at version 3.5.0 and later, this defaults to 10 minutes and can be overridden through the kubeletConfiguration section of ClusterClass variables. kube-proxy controls idle TCP connection state through the tcpEstablishedTimeout parameter in KubeProxyConfiguration, allowing administrators to define how long idle TCP sessions persist before termination. The VKS API server enforces a configurable timeout for all non-long-running requests, defaulting to 1 minute, after which the server terminates the connection if the request has not completed. Kubernetes Service objects using ClientIP session affinity bound the sticky session duration through sessionAffinityConfig.clientIP.timeoutSeconds, which defaults to 10800 seconds (3 hours) and is configurable from 1 to 86400 seconds. The Contour ingress controller, available as a standard package for VKS clusters, provides granular HTTP timeout controls: timeouts.connection-idle-timeout terminates connections that have been idle, defaulting to 60 seconds; timeouts.stream-idle-timeout terminates streams or requests with no activity; and timeouts.max-connection-duration bounds absolute connection lifetime independent of activity. Contour Envoy pods use a configurable termination grace period, defaulting to 300 seconds via

`envoy.terminationGracePeriodSeconds`. When the Istio service mesh standard package is deployed on VKS clusters, `meshConfig.connectTimeout` controls the connection establishment timeout across the mesh, defaulting to 10 seconds. At the workload level, each Pod specification accepts a `terminationGracePeriodSeconds` field that defines the grace period for open connections to complete before the platform forcibly halts the process, defaulting to 30 seconds. Kubernetes surfaces an `endpoints.conditions.terminating` state in `EndpointSlice` resources so that load balancers and service proxies can implement connection draining during rolling updates and scale-down events.

These mechanisms cover end-of-session and inactivity-based termination across all VCF management, programmatic access, and containerized workload paths. Timeout values are configurable at every layer so administrators can align session and connection duration with organizational security policy.

### **VMware vDefend (Contributes)**

VMware vDefend provides configurable session timers and connection state management that terminate network connections after inactivity, directly addressing the workload traffic dimension of this control.

The vDefend distributed firewall (DFW) maintains session state for all connections traversing the firewall and applies configurable session timers that determine how long a session remains active after inactivity. These timers run on every VMware ESX host at the vNIC level, covering workload-to-workload connections regardless of physical network topology. When a session timer expires, the firewall removes the connection entry from its state table, terminating the session. Non-TCP connections (UDP, ICMP, and other protocols) are terminated with a `TIMEOUT` reason, and TCP connections record the specific termination reason in the firewall logs.

For TCP connections, vDefend provides granular timer controls across the connection lifecycle. The Established timer governs active TCP sessions with a default of 43,200 seconds. The Closing timer applies after the first FIN has been sent, defaulting to 120 seconds. The TCP Closed timer takes effect after one endpoint sends a RST, defaulting to 20 seconds. The TCP Opening timer governs sessions where the handshake is in progress, defaulting to 30 seconds after the second packet has been transferred. When both endpoints have sent packets, the multiple-connection timer defaults to 60 seconds. These values are configurable through session timer profiles, which can be applied to user-defined workload groups or to Tier-0 and Tier-1 gateways, allowing administrators to align timeout behavior with organizational security policy. Shorter timeout values can be applied to specific groups, such as externally facing workloads, while longer timeouts accommodate batch processing or database connections.

The Gateway Firewall provides comparable controls for north-south traffic, with configurable session timers spanning TCP connection states. Where Gateway Firewall TLS Inspection is enabled, a separate idle connection timeout governs how long a server can remain idle after a TCP connection is established, defaulting to 5,400 seconds and configurable through the Decryption Action Profile.

vDefend's firewall logging captures connection termination events, including the reason for termination. Distributed Firewall packet logs indicate the actual reason a TCP connection was terminated and generate a termination log entry when a TCP session has ended. These logs record the protocol (TCP, UDP, ICMP, or protocol number) for each packet and include termination reasons for TCP connections. Gateway Firewall packet logs record termination events including source IP, destination IP, ports, protocol, and direction, extending this audit capability to north-south traffic paths.

While vDefend addresses connection termination for workload network traffic at the data plane, management session timeouts (such as NSX Manager session expiry and VMware vCenter session management) are handled by other VCF components. vDefend contributes the workload-level network connection termination capability that complements these management-plane controls.

### **VMware Avi Load Balancer (Meets)**

VMware Avi Load Balancer (Avi) operates as a full-proxy application delivery controller, managing the complete connection lifecycle on both the client-facing and server-facing sides of every proxied session. This architecture gives Avi direct control over when connections are established and terminated, without relying on underlying infrastructure components.

For TCP connections, the TCP Proxy profile in Custom Mode offers two mechanisms for handling idle connections: TCP keepalive, which periodically sends a keepalive packet to the client and resets the idle timer on successful acknowledgment, and Age Out Idle Connections, which terminates idle connections that have not received a keepalive signal after a configurable period. The Idle Duration setting accepts values between 5 and 14,400 seconds, or 0 for an infinite timeout. Administrators configure this in the TCP/UDP Profile settings in the Avi Controller UI under Templates. The Service Engine enforces this setting on back-end connections as well, closing server-side connections once a server has been idle for the configured Idle Duration.

For UDP traffic, the System-UDP-Proxy network profile includes a `session_idle_timeout` parameter with a default value of 60 seconds. This parameter can be adjusted to match organizational requirements for session termination.

Pool-level connection termination is controlled through the Graceful Deactivate Timeout parameter. When set to a value between 1 and 432,000 seconds, Avi stops accepting new sessions to a deactivating server while allowing existing sessions to complete within the specified timeout, after which remaining connections are terminated. Setting the parameter to 0 causes immediate termination of all client sessions to deactivated servers; setting it to -1 allows existing sessions to remain open indefinitely. When a back-end server is deactivated, it continues to process persistent and active connections; new connections are accepted only if they belong to an existing persistent session until persistence times out. Deactivating a virtual service terminates all existing connections immediately.

During Service Engine scale-in operations, the `vs_scalein_timeout` CLI parameter controls the timeout period for secondary Service Engines to gracefully close active connections before scale-in completes.

For HTTP multiplexing, the connection pool Life Timeout parameter specifies the maximum duration a connection can exist in the pool from creation, after which the connection is removed and closed following completion of outstanding requests.

Connection termination events are recorded in Avi's application logs. Significant events logged include connections closed due to the TCP profile idle timeout, connections closed abnormally due to TCP RST from clients or servers, and connections prematurely closed by the client. These records are accessible through the Virtual Service Application Logs in the Avi Controller interface.

Timeout values are configured through the network profile and pool settings in the Avi Controller UI or API. No external platform dependency is required; all mechanisms operate within Avi's own data plane on the Service Engines.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 2.7

> Session termination: Automatically terminate local/remote sessions after defined conditions.

**SCF Controls:** IAC-23

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides several configurable session controls across its component products that support concurrent session limiting, though centralized enforcement of per-account session limits requires additional organizational policy and monitoring.

VMware vCenter includes a configurable maximum session count through the `config.vmacore.soap.maxSessionCount` property, which can be set in `vpix.conf` or through the vSphere Client UI. This setting controls the maximum number of inflight sessions on the vCenter. The vCenter SessionManager managed object controls user access through methods for logging in, obtaining sessions, and logging out. Each user session consumes system resources and creates locks on the server side, so administrators should monitor session activity to identify excessive concurrent sessions from individual accounts. The CIS Session service in vCenter provides additional session token management capabilities, including creation, deletion, and session information retrieval, which can be used programmatically to enforce session policies. The VCF Operations PowerShell plug-in includes a `closeSession` method that allows administrators to terminate individual sessions by session ID, enabling manual enforcement of session limits when needed. The VMware Remote Console Proxy on vCenter also supports a configurable cap on the maximum number of concurrent console connections, set through the `config.mksdevproxy.connLimit` parameter. At the individual virtual machine level, the maximum number of simultaneous remote console sessions can be configured using the `maximum-remote-console-sessions` setting; in highly secure environments, this limit should be set to one concurrent connection.

VCF Operations allows administrators to configure the number of concurrent UI login sessions permitted per user. Changes to this setting take effect for subsequent login sessions. When using Session per user mode, administrators should account for increased CPU, memory, and bandwidth consumption that this mode can introduce.

VCF Operations for Networks enforces a hard limit of 100 unique active API sessions, returning an HTTP 412 response code when the limit is reached. This built-in cap restricts the total number of concurrent API sessions across all users of the Operations for Networks interface.

NSX Manager controls API session concurrency with a per-client limit of 40 concurrent connections (`client_api_concurrency_limit`) and a global limit of 199 concurrent connections. The per-client concurrency limit can be adjusted from its default value through the API or CLI.

For workloads running on VMware Kubernetes Service (VKS) clusters provisioned through vSphere Supervisor, the Kubernetes API server implements API Priority and Fairness (APF) to bound concurrent API requests by identity. APF uses FlowSchema objects to classify incoming requests by authenticated subject (individual users or service accounts) and route each flow to a PriorityLevelConfiguration that specifies an explicit concurrency limit in execution seats. The total concurrency budget is the sum of the `--max-requests-inflight` server parameter (default 400 in VKS clusters) and `--max-mutating-requests-inflight` (default 200); these values are then distributed proportionally among PriorityLevelConfigurations according to their configured shares. Setting a PriorityLevelConfiguration's concurrency limit to zero creates an isolation queue for flows assigned to that level, effectively preventing those requests from executing while they remain queued. Dynamic lending between priority levels allows under-utilized levels to temporarily contribute concurrency budget to heavily-utilized levels within configured upper and lower bounds, observable through the `apiserver_flowcontrol_upper_limit_seats` and `apiserver_flowcontrol_lower_limit_seats` metrics. Cluster administrators can monitor current execution seat allocation per priority level using the `apiserver_flowcontrol_request_concurrency_limit` metric and track seats occupied by executing requests through `apiserver_flowcontrol_request_concurrency_in_use`. The Kubernetes EventRateLimit admission controller provides a complementary mechanism, allowing rate limits to be placed on API events per user, per namespace, per source-and-object, or server-wide; User-type limits are expressed as queries-per-second with a burst capacity (for example, qps: 10, burst: 50). The Istio standard package, available for VKS clusters, supplies a pluggable policy layer with access controls, rate

limits, and quotas at the service mesh level, supporting per-service or per-identity concurrency controls for workloads using HTTP-based session patterns. These concurrency controls operate at the API request layer; interactive administrative console sessions in Kubernetes environments are governed by the underlying VCF SSO and identity provider configuration.

These capabilities provide the technical foundation for concurrent session control across the VCF platform. The per-user session limit in VCF Operations applies caps at the user account level. The per-client and global limits in NSX and vSphere provide connection-level bounding. APF FlowSchemas in VKS clusters support per-identity API request concurrency limits. The controls are distributed across individual product interfaces rather than managed through a single centralized policy. Organizations should establish session limit standards, configure the available settings in each component to align with those standards, define APF FlowSchemas in Kubernetes environments to scope per-identity API concurrency, and monitor both session counts and API request metrics to detect when accounts exceed defined thresholds.

#### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides per-user concurrent session limits through the ``max_concurrent_sessions`` field in the `UserAccountProfile` object on the Avi Controller. This setting defines the maximum number of simultaneous administrative sessions permitted per user account and is configurable through the Controller UI, CLI, or REST API.

The Default-User-Account-Profile ships with ``max_concurrent_sessions`` set to 0, which allows unlimited concurrent sessions. To satisfy this control, administrators must create or modify a `UserAccountProfile` with a nonzero limit and assign it to the relevant user accounts. The No-Lockout-User-Account-Profile also retains a value of 0 and should not be assigned to accounts subject to session limits. When the configured session maximum is reached, additional logins are blocked; administrators can use the ``shell --clear-sessions`` command to invalidate existing sessions and restore access when needed.

The setting is accessible in the Controller at User Authentication and Authorization > User Accounts > User Profile.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.



## Control CR 2.8

> Auditable events: Capability to generate audit records for security-relevant events.

**SCF Controls:** MON-01.4, MON-03

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF produces structured event logs across its component stack that contain all six content elements specified by this control: event type, timestamp, location, source, outcome, and user identity. The platform's Consumption layer, which includes VMware Kubernetes Service (VKS), vSphere Supervisor, and vSphere namespaces, extends this coverage to Kubernetes workloads using the kube-apiserver audit mechanism and component logging infrastructure.

#### Event type identification

VCF components classify events by category and severity. VCF Operations generates audit events organized into categories including Access, Access\_control, Account\_management, Configuration, Data\_access, Network, Notification, Permissions, Session, System, Policy, and Firewall. VCF Operations log entries include a logging level (info, warning, error) and a component class name identifying the subsystem that generated the event, along with a descriptive message. VCF Automation records events for catalog deployments, cloud template operations, Day 2 operations, and resource actions, and its audit logs display a status value or message for each reportable action type. VCF Automation issues specific events such as "Deployment requested" and "Deployment completed" around catalog and cloud template operations. NSX Manager audit logs record administrative changes with structured metadata including request ID, component name, subcomponent, and update status, and include a description of the event for actions such as logon, logoff, and resource access. The VMware vCenter EventEx interface constructs extended events with an explicit eventType and severity parameter, providing machine-readable event classification at the API level. For workloads running on VKS clusters, the kube-apiserver records every API request using the audit.k8s.io/v1 Event schema, where the verb field captures the operation performed (create, get, update, delete, list, watch, patch) and the objectRef field identifies the resource type and API group involved. The audit Event record also includes an auditID that uniquely identifies each event for correlation across log entries, and a stage field indicating which stage of request processing produced the entry (RequestReceived, ResponseStarted, ResponseComplete, or Panic).

#### Timestamps

All logging subsystems across the VCF stack include date and time information. VCF Operations log events contain a timestamp as the first of eight common fields. VCF Operations Infrastructure Operations records the date and time when events and alerts are generated in the Created On field. NSX Manager audit logs record timestamps in ISO 8601 format for create and modify operations, and each NSX audit log entry includes a timestamp. VCF Automation audit logs record a timestamp for every reportable action. Kubernetes audit events include requestReceivedTimestamp and stageTimestamp fields, recording when the request was received by the API server and when each processing stage completed. Kubernetes system component logs in klog format carry a timestamp in every log entry alongside severity level, source file, and line number.

#### Event location and source

VCF Operations log entries include the source IP address and host IP of the system that generated the event, along with a source field indicating where the event originated, such as a VMware ESX host or syslog aggregation forwarder. Each event forwarded to VCF Operations contains source information and name-value pair fields extracted from the raw event data. NSX audit logs record the source of each event. VCF Log Management allows operators to search and filter collected log events by source. In VKS clusters, the Kubernetes audit event objectRef field records the namespace, resource type, resource name, API group, and API version; the requestURI field records the full API path; and the sourceIPs field records the originating IP address of the request, with the userAgent field providing additional context for service-to-service calls. Syslog log messages from vSphere Pods include namespace name, pod UID, pod name, container name, and ESX host name as metadata fields when the LogAgentConfiguration Custom Resource is applied, providing workload-level source attribution alongside

infrastructure-level source data. The Supervisor can also be configured to preserve the source IP addresses of authentication requests and Kubernetes APIServer requests, making those addresses available for inspection in audit logs.

## Outcome

VCF Operations log events can be filtered by logging level, and the Analyze page allows searching and filtering based on timestamp, text, source, and extracted fields. NSX audit logs include a result field that records the success or failure of the logged action. NSX Manager audit logging records operation status with detailed change tracking for administrative operations, including old and new values. VCF Automation audit logs contain a status value for each action indicating the outcome. Audit events across the platform capture success and failure states for operations. Kubernetes audit events record outcome in the responseStatus field, which contains the HTTP response status code indicating success or failure; at the RequestResponse audit level, response bodies are also captured, including error messages for failed operations. VM Operator logs record VM creation failures and reconciliation events for VirtualMachineService resources in VKS clusters, capturing failure details such as OVF deployment errors caused by insufficient ESX host resources.

## User identity

VCF Operations audit logs can be filtered by User ID, User Name, Auth Source, Session, Client IP, Category, and Message. User permission events are sourced from VMware vCenter logs and can be filtered by event, status, updated by, object name, and object type. VCF Automation events record the user who triggered each event in the Owner field, and event logs record API access token usage by including the UUID of the token that performed the action. VCF Automation audit logs also include the user name and project name associated with each action, along with a trace ID for correlation. NSX Manager generates audit logs that record UserName, ModuleName, Operation, and Operation status with timestamps and request identifiers. The vCenter EventEx interface accepts a userName parameter when constructing event records, associating user identity with the logged action at the API level. VCF SSO audit event records include the timestamp, the user who performed the action, an event description, severity, and the IP address of the client connection to vCenter. Kubernetes audit events capture identity in the user field, which records the authenticated username, UID, group memberships, and extra attributes of the requester; for requests made through service accounts, the service account name and namespace are recorded as part of the user context.

## Log detail and centralized management

ESX hosts support configurable syslog log levels that can be adjusted using esxcli system syslog commands, and the host agent log level can be set to verbose or other levels to control the detail captured. ESX syslog messages conform with RFC 3164, which defines structured message formatting. ESX audit records conform to RFC 5424 and contain event time, status, description, and user information for security-relevant activity tracked on the host. The Syslog.global.auditRecord.remote parameter enables forwarding ESX audit records to the remote syslog collector specified by Syslog.global.logHost. VCF Operations log events contain eight common parts: timestamp, thread name, host IP, logging level, class name, message text, token number, and duration. For centralized log management, VCF Log Management can be configured to pull tasks, events, and alerts from vCenter instances and receive syslog feeds from ESX hosts. vCenter can also stream events directly to a remote syslog server by enabling the vpxd.event.syslog.enabled option in the vCenter Management Interface; forwarded events carry the fields eventId, createTime, eventType, severity, user, target, chainId, and desc. The VCF Operations audit log is integrated with syslog, allowing collection of all audit log data through the syslog collector. The VCF Log Management Analyze page provides a centralized interface for searching and correlating log events across all sources. NSX audit log messages are stored in /var/log/audit/audit.log. For deployments evaluated against global compliance frameworks, all auditable events defined by the applicable security requirements are logged by the platform.

For the Consumption layer, Kubernetes audit policy supports four logging levels (None, Metadata, Request, and RequestResponse), giving administrators fine-grained control over the verbosity and scope of audit records for VKS clusters. Audit events are written in JSONlines format to a log file via the kube-apiserver Log backend, configured through the --audit-log-path, --audit-log-maxage, --audit-log-maxbackup, and --audit-log-maxsize flags; audit events can also be forwarded in real time to an external system using the Webhook backend, which delivers batched events as HTTP POST requests to a configured endpoint. Kubernetes audit records are collected by node log agents and forwarded to a central log store as part of the cluster observability pipeline. Fluent Bit logging agents deployed in the vmware-system-logging namespace collect and

forward logs from nodes across the cluster; on VKS clusters, Fluent Bit collects Kubernetes container logs, Kubernetes API audit logs, Linux daemon logs, Linux authentication logs, and container runtime interface logs through configurable parsers and filters, and supports forwarding to Syslog, HTTP, Elasticsearch, Kafka, and Splunk. Workload logs from vSphere Pods and vSphere Namespaces are streamed to a remote Syslog server by applying a LogAgentConfiguration Custom Resource; each syslog message carries namespace name, pod UID, pod name, container name, and ESX host name as metadata fields. Kubernetes seccomp supports the SCMP\_ACT\_LOG action, which logs syscall execution to syslog or auditd without restricting the syscall, and the audit.json seccomp profile logs all syscalls of a process for detailed syscall-level auditing. For Windows workloads on VKS, the support bundle collects Windows Event Logs covering System, Security, Application, Windows Firewall, and networking event sources. Pod-level events accessible through kubectl describe pod record each event with a From field indicating the component source, a Reason field indicating why the event occurred, and a Message field providing details, all with associated timestamps.

VMware Private AI Services (PAIS) includes several event logging and observability mechanisms across its AI-specific components that contribute to the requirements of this control.

PAIS observability is configured through the PAISConfiguration custom resource. When activated, PAIS collects metrics in an integrated Prometheus server and supports forwarding LLM traces to an OpenTelemetry Collector over http/protobuf or gRPC. The spec.observability.prometheusRuntime section enables Prometheus with configurable storage class and metrics retention period, and the spec.observability.llmTraces section activates trace forwarding. These traces capture timing, event type, and source component information for model inference, agent execution flow, and knowledge base indexing. MLOps engineers and application developers use observability platforms such as Grafana to monitor model health, quality, and behavior of AI applications. PAIS Controller metrics are exposed at a Prometheus metrics endpoint on the pais-controller-manager service and can also stream to VCF Operations through Telegraf with namepass filters that capture controller runtime worker counts, work queue depth, and reconcile concurrency.

Model endpoints built on the vLLM inference server support the VLLM\_LOGGING\_LEVEL environment variable to control inference logging verbosity, with DEBUG available as a supported value. Operators can retrieve logs for a specific model endpoint pod in the pais namespace by running kubectl logs against the pais-kubeconfig. Knowledge base operations record timing metrics for file downloads, embedding generation, and index operations, along with size metrics including total megabytes downloaded, number of new chunks indexed, and final index size. These records identify the type and duration of each operation.

Deep Learning VMs write cloud-init deployment logs to /var/log/dl.log and vGPU or NVIDIA data center GPU driver installation logs to /var/log/gpu-install.log. Deep Learning VMs also expose vGPU performance metrics through Prometheus for collection and visualization. These logs and metrics capture deployment progress, driver installation outcomes, and runtime performance.

Configuring all Technology Assets, Applications, and Services with sufficient event log content, including complete user identity trails and system-level API audit records across the full stack, requires configuration at the underlying VCF platform layer, including Kubernetes API server audit logging through VMware Kubernetes Service (VKS), VMware vCenter event logs, and VMware ESX host logs.

### **VMware vDefend (Contributes)**

VMware vDefend (vDefend) produces event logs across its components that contain the information elements specified by this control, covering the network security layer of the VCF infrastructure.

Event type identification: vDefend components categorize events by type across multiple security functions. The Distributed Firewall (DFW) logs firewall rule hits with the action taken (allow, drop, or reject) and the rule ID that matched. IDS/IPS generates detection events categorized by signature severity level and threat category. Security Intelligence provides a View Full Event History capability that displays all events associated with a signature ID and the intrusion severity. Malware Prevention Service logs file analysis events categorized by detection status, verdict (BENIGN, malicious, or suspicious), and malware family. Network Detection and Response (NDR) generates detection events forwarded to SIEM endpoints with structured metadata identifying the type of network threat detected.

**Timestamps:** All vDefend event logs include timestamps. Malware Prevention Service event logs record an inspection timestamp and event timestamps for each file analysis operation. Security Services Platform (SSP) logs certificate loading events with timestamp and component name information. Guest Introspection Platform Host Agent (MUX) messages written to `/var/log/syslog.log` include a timestamp in each log entry.

**Event source:** vDefend event logs identify the source of each event. DFW logs identify the specific vNIC, virtual machine, and security policy involved in each rule evaluation. IDS/IPS logs identify the affected workload and the network segment where the detection occurred. Malware Prevention Service logs include the client VM ID, submission source, server information, and gateway identification for each file analysis event, enabling analysts to trace events back to the originating workload. NDR events identify the network flows and endpoints involved in detected anomalies.

**Outcome:** vDefend logs capture the outcome of security operations. DFW logs record whether traffic was allowed, dropped, or rejected based on rule evaluation. IDS/IPS events record whether a signature match triggered an alert or a block action. Malware Prevention Service logs record the verdict (BENIGN, malicious, or other classification) and a numeric risk assessment score alongside the analysis completion status for each inspected file. Malware Prevention Service event logs also capture file hashes (SHA-256, SHA-1, MD5), file metadata including file name, file type, and file size, as well as HTTP domain, HTTP method, user agent, and file upload status for web-based file transfers. NDR detection events capture the severity and confidence level of detected threats.

**User identity:** vDefend captures user identity through multiple mechanisms. The Identity Firewall component collects Windows event logs from configured Event Log Servers to associate network traffic with Active Directory users and groups, enabling user-attributed firewall rule enforcement. Event Log Scraping is configurable on a per-domain basis from Servers & Site Mapping; in federated environments, Event Log Scraping and processing occur only at the Local Manager site. Event Log Sources are configured in Security > General Settings > Identity Firewall Event Log Sources, with the option to forward user identity events to VCF Log Management. NSX Manager audit logs capture the identity of administrators making configuration changes to firewall rules, security policies, and other objects through administrator identity fields, providing accountability for policy creation and modification.

**Centralized log collection:** VCF Log Management provides a centralized log management framework for vDefend. Unified Security Logs in VCF Log Management contain log messages for DFW, Gateway Firewall, Identity Firewall, FQDN Analysis, TLS Inspection, Intrusion Detection/Prevention System (IDPS), and URL Filtering, enabling correlation across vDefend security functions from a single interface. Logs can be forwarded to VCF Log Management by configuring it as a syslog server.

**SIEM integration:** NDR forwards event logs to SIEM endpoints supporting Splunk (on-premises or cloud-hosted) and custom endpoints with configurable HTTP headers for authentication. NDR supports SSL verification for event logs sent over HTTPS and integrates with VCF Log Management as a SIEM server. Security Services Platform sends campaign and event logs to the connected SIEM as they occur. IDS/IPS events can be exported to external syslog consumers by enabling the `ids_events_to_syslog` setting (disabled by default); this requires remote syslog to be configured on the ESX host. vDefend syslog messages conform to RFC 5424 on vDefend appliances and RFC 3164 on ESX hosts. Guest Introspection Platform Host Agent (MUX) logs are written to `/var/log/syslog.log` on ESX hosts in a structured format that includes a timestamp, ContextMUX identifier, and thread ID.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) produces structured event logs that capture the information elements required for audit trail analysis.

PNR events are identified by named event types that indicate what occurred. vSphere Replication generates typed audit events when virtual machines and hosts are configured for or removed from replication, with event types including `com.vmware.vcHms.replicationConfiguredEvent`, `com.vmware.vcHms.replicationUnconfiguredEvent`, `com.vmware.vcHms.hostConfiguredForHbrEvent`, and `com.vmware.vcHms.hostUnconfiguredForHbrEvent`. Protection group and consistency group lifecycle changes are recorded with assigned severity levels: adding or removing a consistency group from a protection group records an Info-level event, and consistency group errors or warnings record an Error-level event.

Authorization errors are logged with timestamps and component information for security tracking. Each event type has a corresponding alarm that PNR can trigger if the event occurs.

Events include timestamps for time-based correlation. Login events record timestamp, log level, component, operation ID, and session details including `userName` and `loginTime`. Logout events record timestamp, user identity, source IP address, and operation ID. The Clean Room Orchestrator event details panel displays event severity, timestamp, target system information for replication or failover tasks, and event description.

Events are associated with specific objects, including protection groups, consistency groups, virtual machines, datastores, and sites, establishing where the event occurred and what was affected. The PNR global console displays all events occurring in the system, including both system-generated events and user actions. vSphere Replication maintains an audit log at `/opt/vmware/support/logs/hms/hms-audit.log` on the appliance, with broker logs at `/opt/vmware/support/logs/hbr` on the appliance and at `/var/run/log` on VMware ESX hosts.

Login events include the `userName` and `loginTime` fields, establishing the identity and session context of the user. Logout events record user identity and source IP address. The `SrmDpxAgentUser` role enables the DPX agent service to log events to VMware vCenter Event Manager.

Error-level events distinguish failed or abnormal conditions from informational state changes. Consistency group errors and warnings generate Error-level events; normal lifecycle transitions such as configuring replication or adding datastores record at the Info level. Snapshot task completion is logged and accessible through the Snapshot log interface in the Clean Room event details panel.

The PNR Appliance supports forwarding log files to a remote syslog server. Event forwarding to VCF Log Management version 9.0 or later is supported for centralized collection and retention. Clean Room Orchestrator events can be forwarded for external analysis, with event payloads that include timestamp, priority, message, component name, severity, configuration ID, component ID, and event type. Documented event forwarding destinations for Clean Room events include VCF Log Management, Microsoft Sentinel, and Splunk Cloud.

### VMware Data Services Manager (Meets)

VMware Data Services Manager (DSM) provides a structured audit logging framework that captures event data across multiple dimensions required by this control. The gateway audit logs and System Audit Events view expose six core fields for each recorded event: Component (identifying which DSM component performed the operation), Event Type (classifying the action taken), Event Details (providing operation-specific context), Event Time in UTC (recording when the event occurred), Status (recording the outcome as success or failure), and Source (recording the identity of the user or system that initiated the action, using the user's email address for authenticated user actions or "System" for internally initiated operations). These fields collectively address all six event content requirements: event type, timestamp, location, source, outcome, and user identity.

The System Audit Events view in the DSM Provider interface is the designated location for reviewing non-database-related operations and tasks. DSM generates system audit events that track user actions performed through the DSM UI and API, which can be examined using the System Audit Events menu. Administrators can access this view through the System Audit section of the UI, where events are presented with all six columns visible. Audit event data can be exported in CSV or XLSX format using the ACTIONS dropdown menu, and the export covers all events visible to the authenticated user regardless of pagination.

Gateway audit log files are stored locally on the Provider VM at `/var/log/tdm/provider/audit/` with filenames in the format `gateway-audit-timestamp.log`. These files are retained for the last five days on the Provider VM, including the file containing the most recent events. The gateway logs track user read and write operations to DSM data services or infrastructure objects performed through the DSM UI and through the DSM Gateway Kubernetes interface. When a default S3 provider log repository is configured, DSM archives gateway audit log files periodically to an S3 subpath named "gateway audit."

Recovery and operational events, such as Provider Restore Success, are recorded in the System Audit Events table with Event Type and Event Time fields, allowing administrators to correlate recovery operations with specific time windows.

DSM's audit logging covers DSM control-plane events directly. However, database engines provisioned by DSM, including PostgreSQL, MySQL, and MinIO instances, may require separate database-level audit logging configuration to capture events at the application layer with the same level of field detail. Organizations should review the audit logging capabilities of each provisioned database engine and configure them to meet the full scope of this requirement across all managed assets.

### **VMware Avi Load Balancer (Meets)**

VMware Avi Load Balancer (Avi) includes a native event logging system across both the Avi Controller and Service Engines that records structured event data covering all six fields this control specifies.

Avi system events capture a defined set of fields per entry: Timestamp (the date and time the event occurred), Resource Name and Resource Type (identifying the specific object and category of resource involved, establishing where the event occurred), Event Code (a named classification such as CONFIG\_CREATE or CONFIG\_UPDATE that identifies the type of event), and Description (a brief account of the event). Event logs track who made configuration changes and when, and can be filtered by event type using search parameters such as event\_id=CONFIG\_CREATE or event\_id=CONFIG\_UPDATE. For operational events such as configuration restores, the event record includes task name, status, node IP address, and success or failure messages, capturing the outcome of the operation.

Service Engine application logs apply the same structured fields at the data-plane layer, with each entry recording Timestamp, Resource Name, Resource Type, Event Code, and Description for virtual service and pool events. This extends structured logging beyond administrative actions on the Controller to application-layer events processed by the Service Engines.

The Controller's global event log, accessible via Operations > All Events, displays events across all objects including deleted objects and sub-objects such as profiles and certificates. Object-specific event logs are also available from the Events tab on detail pages for Virtual Services, Pools, Service Engines, and Controllers, automatically filtered to that object. The analytics API endpoint /api/analytics/logs retrieves event logs with configurable type, duration (up to one year), page\_size, and end parameters to support queries across specific time periods.

Avi Controller and Service Engines log audit records for trusted channel initiation, termination, and function failures to Events, Syslog, and Splunk. The Avi Web Application Firewall (WAF) logs matched application-layer events including matched variable name and matched variable content, with audit log parts configurable to include full event data. Avi Controller also logs denial of service attack events with event\_id, attack\_count, threshold values, and alert level.

The Controller's Syslog-Audit-Persistence alert action template supports forwarding events to external syslog servers for audit compliance. Note that syslog notifications push alerts but do not automatically export virtual service logs; virtual service logs must be pulled from an external logging system through the API or scripted to push from the Controller to a remote destination.



## Control CR 2.9(1)

> Audit storage capacity: Capability to define behavior when audit storage is full. Audit storage alert: Capability to provide a warning when audit storage is near capacity.

**SCF Controls:** MON-05.2

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides automated alerting mechanisms that notify personnel when log and disk storage usage approaches capacity limits, with configurable thresholds that map directly to organization-defined percentages.

VCF Operations includes a Capacity Remaining policy element in the Policy Workspace that administrators configure to set the percentage threshold at which capacity remaining alerts fire. This applies across monitored storage resources, including datastores used for log retention. Capacity settings can be turned on and configured per policy, allowing administrators to override default thresholds for specific environments. Administrators can also configure different capacity calculations for production environments versus testing environments using custom groups.

VCF Operations supports capacity-specific alert types, with Alert Type and Subtype options that can be configured to select Application: Capacity. This allows capacity-related conditions, including log storage utilization, to be surfaced as distinct alert categories. VCF Operations also supports log-based alerts configured through the Symptom Definitions card in Infrastructure Operations, enabling administrators to define custom symptoms tied to log storage conditions and trigger alerts when those conditions are met.

VCF Log Management ships with pre-built content pack alerts for storage conditions. The SysDiskUsage alert triggers when disk usage for one or more partitions exceeds 89% for more than 10 minutes, providing out-of-the-box detection of log storage exhaustion on the logging infrastructure itself without requiring custom configuration. The SysMemUsage alert similarly triggers when memory usage exceeds 95% for more than 10 minutes, covering resource exhaustion conditions that could affect log processing. VCF Log Management can also be configured to pull tasks, events, and alerts from VMware vCenter instances and receive syslog feeds from VMware ESX hosts through Log Collections configuration. This centralized collection means that log storage capacity can be monitored from a single pane rather than requiring per-host monitoring.

For workloads running on VMware Kubernetes Service (VKS) clusters in the VCF consumption layer, Prometheus and Alertmanager are available as standard packages installable on VKS clusters. The kubelet exposes kubelet\_volume\_stats\_capacity\_bytes and kubelet\_volume\_stats\_used\_bytes metrics labeled by namespace and PersistentVolumeClaim, enabling calculation of storage fill percentage per PVC. Prometheus scrapes these metrics and evaluates organization-defined alerting rules that fire when any volume approaches its configured capacity threshold. The Kubernetes API server exposes the apiserver\_storage\_db\_total\_size\_in\_bytes metric, which reports the physical size of the etcd storage database, allowing organizations to incorporate audit log backend storage into the same alerting framework. Alertmanager is deployed as a StatefulSet with three replicas in the tanzu-system-monitoring namespace, backed by a 2 GiB persistent volume claim with ReadWriteOnce access, and routes alert notifications to configured receivers including email, Slack, and PagerDuty. Organizations must define the threshold percentages, write the Prometheus alerting rules, and configure Alertmanager routing and receivers; the platform provides the instrumentation and delivery infrastructure but does not ship pre-configured rules for event log storage thresholds.

Workload logs from vSphere Namespaces can be streamed to a remote Syslog server by applying the LogAgentConfiguration custom resource to a namespace; each log message includes metadata fields such as namespace name, pod UID, pod name, container name, and VMware ESX host name, preserving context for capacity monitoring at the receiving system. Fluent Bit, available as a VKS standard package, can forward logs from cluster workloads to Syslog, HTTP, Elasticsearch, Kafka, and Splunk destinations, enabling integration with external log storage and monitoring systems that perform their own capacity alerting. When configured for audit log collection, Fluent Bit reads kube-apiserver audit logs in JSON format from /var/log/kubernetes/kube-apiserver.log, with a configurable memory buffer limit of 50 MB per instance. The Kubernetes audit log backend supports configurable truncation through the audit-log-truncate-enabled flag and the audit-log-truncate-max-



event-size parameter, which limit individual event sizes and batch sizes sent to the underlying backend and allow administrators to control the rate of audit log storage consumption. Audit event truncation is disabled by default and must be explicitly enabled via API server configuration before the size parameters take effect.

VCF Operations for Networks includes disk utilization alerts viewable on the platform dashboard, the collector dashboard, or in the Infrastructure and Support > Infrastructure and Updates page. These alerts surface storage pressure on the network monitoring components that collect and store network flow data and logs.

VCF Operations also publishes Capacity Analytics metrics for disk space, calculating capacity remaining in GB as the maximum point between usable capacity and projected utilization between now and three days into the future. This forward-looking analysis gives administrators advance warning before storage is actually exhausted, rather than only alerting after the fact. Datastore capacity metric trends can be checked to identify spikes or abnormal growth in storage usage. VCF Automation provides an Alerts page that surfaces capacity, performance, and availability issues, enabling cloud administrators to react proactively before resources are exhausted.

When capacity thresholds are reached and alerts fire, VCF provides multiple automated notification channels to reach appropriate personnel. vCenter can send SNMP traps and email notifications when alarms are triggered, with configurable recipient addresses. vCenter's SNMP agent supports v1 and v2c traps as well as v3 traps with all security levels. The vCenter SNMP agent also supports notification filtering, allowing administrators to suppress specific notifications that should not reach their SNMP management software. VCF PowerCLI provides the New-AlarmAction command to create SNMP alarm actions for alarm definitions programmatically, enabling infrastructure-as-code approaches to alarm notification configuration. VCF Operations can send alert notifications via SNMP trap to up to four SNMP trap destinations, with filtering available when defining notifications. These notification mechanisms allow organizations to route capacity alerts to the appropriate teams through their existing monitoring and incident management infrastructure.

#### **VMware vDefend (Contributes)**

VMware vDefend's Security Services Platform (SSP) generates alarms when allocated resources exceed predetermined threshold values, alerting administrators to capacity conditions that may affect analytics and detection services. A separate alarm mechanism in the Distributed Firewall Security Overview Dashboard triggers and remains active when dropped events are detected, notifying administrators of data loss conditions at the event collection layer.

For organizations implementing threshold-based log storage capacity alerting at organization-defined percentages, vDefend provides multiple log forwarding integration points. VMware vDefend Distributed IDS/IPS can be configured to send IDS events to external syslog consumers via the `ids_events_to_syslog` setting (disabled by default). vDefend appliances emit RFC 5424-compliant syslog messages, and VMware ESX hosts emit RFC 3164-compliant messages, making vDefend log streams compatible with standard syslog infrastructure and SIEM platforms. SSP SIEM integration sends Network Detection and Response (NDR) campaign and event logs to the connected SIEM as they occur, including logs generated when new detections are created, when existing detections are updated, and when new campaigns are identified. Identity Firewall Event Log Sources can be integrated with VCF Log Management for centralized log collection and monitoring.

Native configurable alerting to designated personnel at an organization-defined percentage of log storage capacity is not a documented vDefend feature; this function is provided by an external monitoring platform or SIEM that ingests vDefend log streams and implements capacity-based threshold alerting.

#### **VCF Protection and Recovery (Contributes)**

Protection and Recovery (PNR) monitors free disk space on the appliance host and raises a Warning event, logged as `LowDiskSpaceEvent`, when disk space falls below a configured threshold. PNR alarms can be configured to send email notifications, SNMP traps, or run scripts on the VMware vCenter host when threshold conditions are met. Configuring PNR alarms to deliver email notifications requires that Mail settings be configured in vCenter. PNR appliance registration requires that an administrator email address be provided, establishing the notification path for disk space and other system events.

PNR log storage is managed through a configurable file count limit. The `maxFileNum` parameter controls the maximum number of log files retained by the PNR Server, with a default of 20 files. When log files reach their configured size, older files are compressed to keep storage consumption bounded.

The PNR Appliance supports forwarding log files to a remote syslog server for centralized log analysis. PNR also supports forwarding events to VCF Log Management version 9.0 and above. In Live Cyber Recovery (LCR) deployments, the Clean Room Orchestrator can forward events to a Microsoft Azure Log Analytics Workspace using Data Collection Rules and Data Collection Endpoints, extending event visibility to external log management systems.

Percentage-based log storage capacity alerting with organization-configurable thresholds depends on the broader VCF platform monitoring stack. VCF Operations provides an advanced alert system for monitored objects that includes health alerts for protection groups, recovery plans, cloud file systems, recovery SDDCs, and protected sites.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) monitors storage capacity on the data disk and generates global alerts when usage reaches threshold values. A WARNING alert fires when storage on the data disk reaches 70% usage, and a CRITICAL alert fires when it reaches 90%. In DSM, both thresholds are exposed as named settings (`data-disk-warning-threshold` and `data-disk-critical-threshold`) in the Configure the Storage on a Data Disk section, giving administrators a documented location to review and adjust these values when sizing storage. The alerts are designed to notify administrators of capacity conditions that may affect backup and WAL archival operations, both of which depend on available storage to function correctly. The alert system operates automatically without administrator intervention and is visible through the Global Alerts Reference in the DSM interface.

DSM uses a three-level severity model for alerts: OK, WARNING, and CRITICAL. The `DATA_DISK_HEALTH_ALERT` type is associated with disk usage thresholds and is surfaced in the database monitoring view under Database Alerts and Metrics. When this alert fires, administrators should increase the data disk size to prevent data service outage or read-only mode conditions. DSM also documents the ability to change the storage amount directly when disk warning or critical alerts are observed, allowing remediation within DSM itself without external tooling.

For alert delivery, DSM supports webhook-based notifications that can route alert events to email recipients or to external applications such as Slack or ServiceNow. This allows organizations to connect DSM storage capacity alerts to their existing incident management or on-call workflows. Configuring these notification targets is an administrative step performed by the DSM Provider administrator.

The organization is responsible for defining which personnel receive these alerts and for configuring the notification delivery targets. Organizations that require alerting at thresholds other than DSM's defaults should verify whether the named threshold settings meet their requirements, or supplement DSM's built-in alerts with external monitoring tools that observe DSM disk metrics.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) includes an automated disk utilization alerting mechanism through its configurable metric-based alert framework. The Alert feature processes system events and metrics against organization-defined threshold rules and generates alerts that initiate configured notification actions. When disk usage on a Service Engine exceeds a configured threshold percentage, the Controller generates an alert and delivers it to appropriate personnel. The threshold percentage is organization-defined, allowing teams to set the level at which alerting fires.

The Avi Controller supports multiple alert notification destinations: email, syslog, SNMP v2c and SNMPv3 traps, ControlScript automation, and Splunk. These destinations are configured in the Operations > Notifications page. The Syslog-Audit-Persistence template supports streaming audit events to external syslog servers, integrating Avi alerts into centralized log management systems. SNMP notifications support both v2c and v3 protocols, with optional TLS encryption and strict certificate verification for secure delivery to SNMP-based monitoring infrastructure. Multiple SNMP servers can be configured simultaneously, and the Avi Controller cluster leader issues SNMP trap notifications based on system events.

Service Engine log storage is configurable through SE runtime properties. The debug-logs-capacity is calculated as (SE Total Disk minus Maximum Storage not allocated for logs on SE) divided by 100. Client log capacity is derived from total SE disk space minus the debug log allocation, with further adjustments based on configured minimum storage and SE RAM. Increasing disk resources on Controllers and Service Engines extends the volume of logs that can be stored and the overall retention duration.

For deployments using the Avi Kubernetes Operator for VCF clusters, the AKO `persistentVolumeClaimSpec.storage` parameter allocates a default of 2Gi of storage for the log Persistent Volume Claim, which can be adjusted at deployment time to accommodate larger log volumes.

To activate disk utilization alerting, administrators must configure a metric-based alert with an appropriate threshold and assign at least one notification destination with the relevant recipient addresses, trap server endpoints, or syslog server details in the Operations > Notifications page. Avi system component debug logs and Event Logs are not forwarded to remote syslog servers through OS-level syslog configuration; they require Avi-specific alert action configuration to reach external logging systems. These organizational configuration steps are required before automated alerting is active.

## Control CR 2.9(a)

> Sub-item (a) of CR 2.9 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 2.9 requirement in the standard.

**SCF Controls:** MON-04

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides multiple layers of log storage capacity management across its component products, covering local log retention controls, centralized log aggregation with scalable architecture, proactive capacity monitoring with forward-looking projections, and Kubernetes-native container log bounding for workloads running on VMware Kubernetes Service (VKS).

At the VMware ESX host level, administrators can configure log file retention and rotation policies. The `log.keepOld` parameter controls how many log files are retained, and when that limit is reached, the oldest file is automatically deleted. VMware recommends a log file size of 500 KB per file for host logs. The `syslog` directory location is configurable through the `Syslog.global.logDir` parameter, allowing administrators to direct log output to a storage volume with sufficient capacity. These same retention controls apply to virtual machine log files, where the number of retained log files can be limited through the `log.keepOld` setting. Per-folder log size limits prevent any single log source from consuming disproportionate storage, and logging levels can be adjusted per log folder to balance detail against storage consumption. ESX also exposes dedicated audit record storage parameters: the `Syslog.global.auditRecord.storageCapacity` parameter defines the size of the local audit record storage directory in MiB, and this capacity can be set or adjusted at any time using the `esxcli system auditrecords local set --size` command. VMware vCenter provides equivalent log rotation controls, with the ability to limit the number of retained log files through the same `log.keepOld` parameter.

For VKS clusters, the kubelet exposes two key parameters for container log management that can be configured through ClusterClass variables during cluster provisioning. The `containerLogMaxSizeMiB` setting defines the maximum size of a container log file before rotation, with a default of 10 MiB. The `containerLogMaxFiles` setting controls how many rotated log files are retained per container, with a valid range of 1 to 100 and a default of 5; the oldest files are removed automatically when the limit is exceeded during rotation. Together these parameters establish a predictable ceiling on per-node storage consumption from container logs. The Kubernetes API server's audit log backend provides complementary controls: the `audit-log-truncate-enabled` flag activates event and batch truncation, and the `audit-log-truncate-max-event-size` parameter (default 10,240 bytes) limits the size of each individual audit event sent to the backend. The `--audit-log-batch-max-size` flag bounds the number of events per batch. For environments where audit completeness is a compliance requirement, administrators should size the `--audit-log-batch-buffer-size` appropriately, as events are dropped rather than queued indefinitely if the buffer overflows. The audit policy configured for the API server directly affects overall storage consumption: selecting appropriate verbosity levels (None, Metadata, Request, or RequestResponse) for different resource types controls the rate at which audit records accumulate, and restricting verbose logging to high-value operations such as secret access or privileged resource modifications reduces the overall storage footprint. The Kubernetes `apiserver_audit_event_total` Prometheus metric tracks the total number of audit events exported, providing a mechanism to monitor audit throughput and detect developing capacity pressure before events begin to drop. A companion metric, `apiserver_audit_level_total`, provides a per-request counter partitioned by the audit level assigned by the active policy, giving administrators visibility into audit volume by verbosity tier to inform storage sizing and policy tuning decisions.

Namespace-level storage governance for VKS workloads is available through `LimitRange` and `ResourceQuota` objects. `LimitRange` allows cluster administrators to set per-`PersistentVolumeClaim` storage request ceilings, while `ResourceQuota` caps total storage consumption per namespace, enabling cluster-wide storage budget planning and preventing any single project from exhausting its allotment. For persistent log-related storage in standard packages, the Harbor container registry job log PVC defaults to 1 GB (configured via `persistence.persistentVolumeClaim.jobLog.size`) and can be expanded to 5 GB or higher after verifying available disk space; the Harbor main registry storage PVC defaults to 10 GB and can be increased to 20 GB or higher. The Alertmanager standard package PVC defaults to 2 GiB.

For centralized log management, VCF Log Management is the primary log aggregation platform. It collects syslog feeds from ESX hosts and pulls tasks, events, and alerts from vCenter instances. The platform supports a High Availability deployment model with multiple instances behind an internal load balancer, and it provides both scale-up and scale-out capabilities to accommodate growing log volumes. The deployment model supports a configurable retention notification threshold and the ability to add disk nodes, giving administrators direct control over when they are alerted about storage consumption and the ability to expand capacity without service interruption. Administrators can configure log retention policies by log type and source, controlling both log file size and the number of retained log files.

VCF Operations provides configurable data retention settings that allow administrators to tune how long historical data is kept, directly managing the storage space consumed by the system. Support log properties can be edited to limit retained log size and set logging levels for individual log file folders, providing granular control over which logs consume the most storage. The orchestrator scripting log size can also be defined in megabytes through a dedicated configuration parameter. VCF Operations retains recent task data for actions for 14 days, after which the data is purged from the system.

For proactive capacity monitoring, VCF Operations collects disk log metrics including used, total, and available space for node logs. The platform publishes a Capacity Analytics metric for disk space usage that provides the remaining capacity in GB, calculated as the maximum point between usable capacity and projected utilization over the next three days. This forward-looking projection allows administrators to identify and address storage shortfalls before they affect log collection. Capacity metrics can also be calculated for datastore objects, providing visibility into the underlying storage that hosts log data. VCF Operations generates disk capacity alerts when utilization reaches 80% (Critical) or 90% (Immediate) severity levels, providing automated notification when log storage is approaching exhaustion. VCF Operations also includes a Kubernetes Capacity dashboard that displays Kubernetes workload resource usage against configured requests and limits, providing unified visibility into K8s storage consumption for capacity planning.

Syslog forwarding to external targets provides an additional safeguard for log storage capacity. ESX hosts can be configured to forward syslog messages to a remote log server, offloading log storage from local host volumes to dedicated infrastructure with managed capacity. vCenter supports forwarding to up to three remote syslog servers using TCP, UDP, TLS, or RELP protocols. For VKS clusters, the Fluent Bit standard package can forward container and system logs to Syslog, HTTP, Elasticsearch, Kafka, or Splunk targets, offloading log storage from cluster node volumes to dedicated log infrastructure. When Fluent Bit is configured to collect kube-apiserver audit logs from VKS cluster nodes, the audit log stream operates with a 50 MB in-memory buffer limit, bounding the local storage footprint for audit events that are pending forwarding. On the vSphere Supervisor control plane, kube-apiserver audit logs are collected by Fluent Bit from `/var/log/vmware/audit/kube-apiserver.log` and stored locally at `/var/log/vmware/fluentbit/flb_audit_apiserver` before any external forwarding is applied; directing these logs to an external target reduces the storage load on the Supervisor control plane nodes. Workloads running in vSphere Namespaces can stream their logs to a remote Syslog server by applying a `LogAgentConfiguration` Custom Resource to the namespace; log messages are enriched with namespace name, pod UID, pod name, container name, and ESX host name for traceability. By forwarding logs to external SIEM or log management systems, organizations maintain a secondary copy of log data that is independent of local storage constraints, so that even if local storage is exhausted, the forwarded copies remain intact. Administrators should note that ESX hosts deployed with Auto Deploy store logs on a RAM disk, which provides limited local log capacity; for those deployments, configuring remote syslog forwarding is particularly important.

VMware Private AI Services (PAIS) includes several mechanisms for managing the storage capacity used by its observability and logging components. The most direct is the `PAISConfiguration` custom resource (CR), which governs PAIS observability. Within the CR, the `metrics-retention` property sets the data retention period for Prometheus metrics, and the Prometheus metrics database is backed by a Kubernetes Persistent Volume whose size is inferred from this retention setting. Operators configure this alongside a storage class via `observability.prometheusRuntime` when activating PAIS observability, giving them direct control over how much storage is allocated to the metrics backend and how long data is retained before being pruned. The metrics retention period has a default of 14 days, which establishes a predictable upper bound on the metrics volume an operator must size storage for.

For LLM request tracing, PAIS supports forwarding LLM traces to an OpenTelemetry Collector by uncommenting the `spec.observability.llmTraces` section of the PAISConfiguration CR. This externalizes trace storage to whichever observability platform the organization operates, so trace volume does not accumulate within the PAIS cluster itself. PAIS Controller metrics are also exposed at a Prometheus endpoint on the `pais-controller-manager` service for collection by external monitoring systems, which keeps long-term metric history out of the PAIS namespace.

Deep Learning VMs write logs to well-known paths: JupyterLab authentication events and cloud-init provisioning output go to `/var/log/dl.log`, and vGPU and NVIDIA data center GPU driver installation events go to `/var/log/gpu-install.log`. The Triton Inference Server on Deep Learning VMs generates inference request logs and performance metrics that can be collected for audit and compliance review. The vLLM inference server on model endpoints supports a `VLLM_LOGGING_LEVEL` environment variable that controls logging verbosity, allowing operators to reduce log volume by selecting an appropriate level for production workloads. Model endpoints also have a configurable `tempoMountSize` on the `/tmp` mount point that defaults to 1Gi, which bounds ephemeral working storage on each endpoint.

Capacity for the supporting Kubernetes storage on VKS worker nodes is sized through PAIS-provided defaults. VKS worker node pools provisioned for PAIS use `kubelet-dir` and `containerd-dir` storage volumes with a default capacity of 100Gi each, mounted to vSAN storage policies. Data Indexing and Retrieval worker nodes use a storage policy configured through the `spec.worker.storageClassName` section of the PAISConfiguration YAML. For environments that host a private Harbor registry for AI container images, PAIS guidance recommends allocating registry storage space to accommodate at least three versions of each container image planned for deployment, so that registry growth is anticipated rather than reactive.

The foundational storage layer, including vSAN storage policies, namespace storage limits, and platform-level capacity monitoring, is managed through VCF rather than PAIS itself.

### VMware vDefend (Contributes)

VMware vDefend (vDefend) provides several log storage capacity management mechanisms for its security event data.

At the Distributed Firewall (DFW) layer, packet logging uses bounded log rotation: the `DFWPKTLOGS.log` file rotates at a 50 MB per-file limit, and the system retains a maximum of 10 log files at a time. This bounds DFW packet log storage on transport nodes while retaining sufficient history for troubleshooting and forensic analysis.

vDefend IDS/IPS includes an automated protective mechanism that handles high-volume event scenarios. When incoming IDS/IPS events and PCAP data would otherwise exhaust memory, the mechanism selectively drops non-critical events and PCAPs to maintain system stability. Administrators can observe event volumes through the IDS/IPS Security Dashboard and Security Overview Dashboard.

IDS/IPS event data can be forwarded to external syslog consumers by enabling the `ids_events_to_syslog` setting under IDS/IPS Settings. Exporting events externally offloads local storage consumption so that log accumulation on transport nodes is bounded by the rotation policy rather than event volume. Remote syslog must be configured on the ESX host to receive IDS/IPS events exported directly from ESX hosts. vDefend components write logs to `/var/log`, with syslog messages conforming to RFC 5424 on vDefend appliances and RFC 3164 on ESX hosts.

Guest Introspection Host Agent (MUX) logs are written to `/var/log/syslog.log` on ESX hosts, with each log message including a timestamp, ContextMUX identifier, and thread ID.

The Security Services Platform (SSP), which hosts Security Intelligence and Network Detection and Response analytics, has documented storage sizing baselines for capacity planning. Each SSP controller node requires 225 GB of storage and each worker node also requires 225 GB. Storage requirements scale with worker node count, adding approximately 155 GB per additional worker node. For a maximum configuration of 10 worker nodes (14 total nodes), the recommended persistent volume claim is 3,545 GB. These sizing baselines enable administrators to allocate storage capacity appropriately at deployment time.

vDefend logs can be forwarded to VCF Log Management by configuring it as a syslog server. VCF Log Management provides a centralized log management framework and supports querying log data by time range through Interactive Analytics.



These mechanisms address event log storage capacity for vDefend's security event data. Broader log storage governance across the VCF platform, including infrastructure-wide capacity monitoring and centralized retention policies, is handled by VCF Log Management and VCF Operations.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) provides configurable log storage capacity management through file size limits, file count limits, and per-component logging level controls that help prevent log storage exhaustion.

PNR Server log files are managed through the `vmware-dr.xml` configuration file. The `<maxFileSize>` parameter controls the maximum size of individual log files, with a default of 10,485,760 bytes (10 MB). The `<maxFileNum>` parameter controls the maximum number of log files retained, with a default of 20 files. PNR also raises a low disk space event when available disk space on the server falls below a configurable threshold, controlled by the `localSiteStatus.minDiskSpace` parameter with a default of 100 MB. This event can trigger alarms that alert administrators before log storage is exhausted.

Configurable logging levels per component allow administrators to reduce log verbosity when log files are growing too quickly, or increase it temporarily for troubleshooting. Named per-component settings configurable through the Reconfigure Protection and Recovery Settings interface include `logManager.Storage`, `logManager.StorageProvider`, `logManager.Licensing`, and `logManager.VvolProvider`, each with a default value of `verbose`. This allows log output from storage, storage provider, licensing, and Virtual Volumes subsystems to be tuned independently.

The PNR Appliance supports forwarding log files to a remote syslog server, allowing log volume to be offloaded to centralized log management infrastructure. In environments using the Live Cyber Recovery (LCR) clean room, PNR can forward events from the clean room to VCF Log Management (version 9.0 or above), Microsoft Sentinel, or Splunk Cloud.

vSphere Replication, which is part of PNR, supports event logging and allows definition of alarms for each event. vSphere Replication maintains dedicated audit logs at `/opt/vmware/support/logs/hms/hms-audit.log`, with broker logs at `/opt/vmware/support/logs/hbr` on the Appliance and at `/var/run/log` on ESX hosts.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides configurable log storage settings and two-tier capacity alerting that support proactive management of event log storage.

DSM Provider logging includes configurable log storage settings accessible through the Provider interface under Storage Settings. Administrators configure the Provider Log Repo URL to direct logs to an S3-compatible object storage repository and bucket; the Bucket Name and optional Region settings complete the S3 object store configuration. Gateway audit log files are stored locally on the Provider VM at `/var/log/tdm/provider/audit/` using filenames in the format ``gateway-audit-timestamp.log``, and are archived periodically to an S3 subpath named ``gateway audit`` when a provider log repository is configured. The Provider VM ships with a default storage configuration of three disks totaling 595 GB (80 GB, 15 GB, and 500 GB), which can be sized for the anticipated log and backup volume at deployment time.

DSM includes two-tier capacity alerting to notify administrators before storage is exhausted. The ``data-disk-warning-threshold`` setting governs a WARNING global alert when data disk usage reaches 70%; the ``data-disk-critical-threshold`` setting governs a CRITICAL global alert at 90%. When either threshold is breached, administrators can increase the storage allocation using the ``storage-amount`` setting. These alerts are visible to DSM administrators and prompt timely remediation before log capacity is exceeded.

For SQL Server databases, DSM exposes the ``errorLog.numErrorLogs`` custom engine setting, which controls how many error log files are retained before recycling. Configuring this setting allows administrators to manage database-engine-level log storage consumption directly.

S3-compatible storage sizing for DSM depends on the volume of data backups and the configured retention policy. Organizations must assess their log generation rates, retention requirements, and backup volumes to right-size the S3 repository. DSM does not automatically provision or expand storage capacity, so organizational processes for periodic capacity review and storage expansion remain necessary to fully satisfy this control.



### VMware Avi Load Balancer (Contributes)

VMware Avi Load Balancer provides configurable log storage capacity at both the Controller and Service Engine layers, giving organizations the mechanisms to allocate and proactively manage event log storage for application delivery traffic.

At the Service Engine layer, disk capacity for logs is calculated dynamically based on total disk size, CPU cores, RAM, and the configurable maximum-storage-not-allocated-for-logs setting, adjustable through SE runtime properties. These values drive two derived pools: client-logs-capacity (total disk minus debug logs capacity, with adjustments based on configured minimum storage allocated for logs and SE RAM) and debug-logs-capacity (calculated as the SE total disk minus the maximum storage not allocated for logs, divided by 100). A minimally sized Service Engine with two vCPUs, 2 GB of RAM, and 10 GB of storage can hold approximately 8 million log entries. When storage is exhausted, the oldest Significant Log entries are phased out on a per-Service Engine basis and their disk space freed automatically, keeping the Service Engine operational without requiring operator intervention.

Increasing disk capacity on a Service Engine directly extends both the volume of logs retained and the total retention duration. Avi documentation recommends provisioning Controllers and Service Engines with larger storage allocations in environments that require extended log retention. Adding CPU, memory, and disk resources to Service Engines and Controllers scales log throughput and retention proportionally.

For deployments using the Avi Kubernetes Operator (AKO) within VCF, the AKO `persistentVolumeClaimSpec.storage` parameter allocates 2 GiB of storage for the log Persistent Volume Claim by default, providing a configurable log storage allocation for Kubernetes-based workloads.

For log retention beyond what local disk supports, the Avi Controller's Syslog-Audit-Persistence feature streams alerts and audit events to external syslog servers. The Controller also supports sending audit records for trusted channel initiation, termination, and failures to Events, Syslog, and Splunk destinations. Offloading to an external syslog or Splunk infrastructure decouples event log retention from local Service Engine disk allocation and supports longer retention windows managed at the logging tier. Note that Syslog notification actions push alert events to syslog servers but do not directly export virtual service logs; virtual service logs must be pulled from an external logging system through the API or scripted to be pushed from the Controller.

## Control CR 2.9(b)

> Sub-item (b) of CR 2.9 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 2.9 requirement in the standard.

**SCF Controls:** MON-04

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides multiple layers of log storage capacity management across its component products, covering local log retention controls, centralized log aggregation with scalable architecture, proactive capacity monitoring with forward-looking projections, and Kubernetes-native container log bounding for workloads running on VMware Kubernetes Service (VKS).

At the VMware ESX host level, administrators can configure log file retention and rotation policies. The `log.keepOld` parameter controls how many log files are retained, and when that limit is reached, the oldest file is automatically deleted. VMware recommends a log file size of 500 KB per file for host logs. The `syslog` directory location is configurable through the `Syslog.global.logDir` parameter, allowing administrators to direct log output to a storage volume with sufficient capacity. These same retention controls apply to virtual machine log files, where the number of retained log files can be limited through the `log.keepOld` setting. Per-folder log size limits prevent any single log source from consuming disproportionate storage, and logging levels can be adjusted per log folder to balance detail against storage consumption. ESX also exposes dedicated audit record storage parameters: the `Syslog.global.auditRecord.storageCapacity` parameter defines the size of the local audit record storage directory in MiB, and this capacity can be set or adjusted at any time using the `esxcli system auditrecords local set --size` command. VMware vCenter provides equivalent log rotation controls, with the ability to limit the number of retained log files through the same `log.keepOld` parameter.

For VKS clusters, the kubelet exposes two key parameters for container log management that can be configured through ClusterClass variables during cluster provisioning. The `containerLogMaxSizeMiB` setting defines the maximum size of a container log file before rotation, with a default of 10 MiB. The `containerLogMaxFiles` setting controls how many rotated log files are retained per container, with a valid range of 1 to 100 and a default of 5; the oldest files are removed automatically when the limit is exceeded during rotation. Together these parameters establish a predictable ceiling on per-node storage consumption from container logs. The Kubernetes API server's audit log backend provides complementary controls: the `audit-log-truncate-enabled` flag activates event and batch truncation, and the `audit-log-truncate-max-event-size` parameter (default 10,240 bytes) limits the size of each individual audit event sent to the backend. The `--audit-log-batch-max-size` flag bounds the number of events per batch. For environments where audit completeness is a compliance requirement, administrators should size the `--audit-log-batch-buffer-size` appropriately, as events are dropped rather than queued indefinitely if the buffer overflows. The audit policy configured for the API server directly affects overall storage consumption: selecting appropriate verbosity levels (None, Metadata, Request, or RequestResponse) for different resource types controls the rate at which audit records accumulate, and restricting verbose logging to high-value operations such as secret access or privileged resource modifications reduces the overall storage footprint. The Kubernetes `apiserver_audit_event_total` Prometheus metric tracks the total number of audit events exported, providing a mechanism to monitor audit throughput and detect developing capacity pressure before events begin to drop. A companion metric, `apiserver_audit_level_total`, provides a per-request counter partitioned by the audit level assigned by the active policy, giving administrators visibility into audit volume by verbosity tier to inform storage sizing and policy tuning decisions.

Namespace-level storage governance for VKS workloads is available through LimitRange and ResourceQuota objects. LimitRange allows cluster administrators to set per-PersistentVolumeClaim storage request ceilings, while ResourceQuota caps total storage consumption per namespace, enabling cluster-wide storage budget planning and preventing any single project from exhausting its allotment. For persistent log-related storage in standard packages, the Harbor container registry job log PVC defaults to 1 GB (configured via `persistence.persistentVolumeClaim.jobLog.size`) and can be expanded to 5 GB or higher after verifying available disk space; the Harbor main registry storage PVC defaults to 10 GB and can be increased to 20 GB or higher. The Alertmanager standard package PVC defaults to 2 GiB.

For centralized log management, VCF Log Management is the primary log aggregation platform. It collects syslog feeds from ESX hosts and pulls tasks, events, and alerts from vCenter instances. The platform supports a High Availability deployment model with multiple instances behind an internal load balancer, and it provides both scale-up and scale-out capabilities to accommodate growing log volumes. The deployment model supports a configurable retention notification threshold and the ability to add disk nodes, giving administrators direct control over when they are alerted about storage consumption and the ability to expand capacity without service interruption. Administrators can configure log retention policies by log type and source, controlling both log file size and the number of retained log files.

VCF Operations provides configurable data retention settings that allow administrators to tune how long historical data is kept, directly managing the storage space consumed by the system. Support log properties can be edited to limit retained log size and set logging levels for individual log file folders, providing granular control over which logs consume the most storage. The orchestrator scripting log size can also be defined in megabytes through a dedicated configuration parameter. VCF Operations retains recent task data for actions for 14 days, after which the data is purged from the system.

For proactive capacity monitoring, VCF Operations collects disk log metrics including used, total, and available space for node logs. The platform publishes a Capacity Analytics metric for disk space usage that provides the remaining capacity in GB, calculated as the maximum point between usable capacity and projected utilization over the next three days. This forward-looking projection allows administrators to identify and address storage shortfalls before they affect log collection. Capacity metrics can also be calculated for datastore objects, providing visibility into the underlying storage that hosts log data. VCF Operations generates disk capacity alerts when utilization reaches 80% (Critical) or 90% (Immediate) severity levels, providing automated notification when log storage is approaching exhaustion. VCF Operations also includes a Kubernetes Capacity dashboard that displays Kubernetes workload resource usage against configured requests and limits, providing unified visibility into K8s storage consumption for capacity planning.

Syslog forwarding to external targets provides an additional safeguard for log storage capacity. ESX hosts can be configured to forward syslog messages to a remote log server, offloading log storage from local host volumes to dedicated infrastructure with managed capacity. vCenter supports forwarding to up to three remote syslog servers using TCP, UDP, TLS, or RELP protocols. For VKS clusters, the Fluent Bit standard package can forward container and system logs to Syslog, HTTP, Elasticsearch, Kafka, or Splunk targets, offloading log storage from cluster node volumes to dedicated log infrastructure. When Fluent Bit is configured to collect kube-apiserver audit logs from VKS cluster nodes, the audit log stream operates with a 50 MB in-memory buffer limit, bounding the local storage footprint for audit events that are pending forwarding. On the vSphere Supervisor control plane, kube-apiserver audit logs are collected by Fluent Bit from `/var/log/vmware/audit/kube-apiserver.log` and stored locally at `/var/log/vmware/fluentbit/flb_audit_apiserver` before any external forwarding is applied; directing these logs to an external target reduces the storage load on the Supervisor control plane nodes. Workloads running in vSphere Namespaces can stream their logs to a remote Syslog server by applying a `LogAgentConfiguration` Custom Resource to the namespace; log messages are enriched with namespace name, pod UID, pod name, container name, and ESX host name for traceability. By forwarding logs to external SIEM or log management systems, organizations maintain a secondary copy of log data that is independent of local storage constraints, so that even if local storage is exhausted, the forwarded copies remain intact. Administrators should note that ESX hosts deployed with Auto Deploy store logs on a RAM disk, which provides limited local log capacity; for those deployments, configuring remote syslog forwarding is particularly important.

VMware Private AI Services (PAIS) includes several mechanisms for managing the storage capacity used by its observability and logging components. The most direct is the `PAISConfiguration` custom resource (CR), which governs PAIS observability. Within the CR, the `metrics-retention` property sets the data retention period for Prometheus metrics, and the Prometheus metrics database is backed by a Kubernetes Persistent Volume whose size is inferred from this retention setting. Operators configure this alongside a storage class via `observability.prometheusRuntime` when activating PAIS observability, giving them direct control over how much storage is allocated to the metrics backend and how long data is retained before being pruned. The metrics retention period has a default of 14 days, which establishes a predictable upper bound on the metrics volume an operator must size storage for.

For LLM request tracing, PAIS supports forwarding LLM traces to an OpenTelemetry Collector by uncommenting the `spec.observability.llmTraces` section of the `PAISConfiguration` CR. This externalizes trace storage to whichever observability platform the organization operates, so trace volume does not accumulate within the PAIS cluster itself. PAIS Controller metrics are also exposed at a Prometheus endpoint on the `pais-controller-manager` service for collection by external monitoring systems, which keeps long-term metric history out of the PAIS namespace.

Deep Learning VMs write logs to well-known paths: JupyterLab authentication events and cloud-init provisioning output go to `/var/log/dl.log`, and vGPU and NVIDIA data center GPU driver installation events go to `/var/log/gpu-install.log`. The Triton Inference Server on Deep Learning VMs generates inference request logs and performance metrics that can be collected for audit and compliance review. The vLLM inference server on model endpoints supports a `VLLM_LOGGING_LEVEL` environment variable that controls logging verbosity, allowing operators to reduce log volume by selecting an appropriate level for production workloads. Model endpoints also have a configurable `tempoMountSize` on the `/tmp` mount point that defaults to 1Gi, which bounds ephemeral working storage on each endpoint.

Capacity for the supporting Kubernetes storage on VKS worker nodes is sized through PAIS-provided defaults. VKS worker node pools provisioned for PAIS use `kubelet-dir` and `containerd-dir` storage volumes with a default capacity of 100Gi each, mounted to vSAN storage policies. Data Indexing and Retrieval worker nodes use a storage policy configured through the `spec.worker.storageClassName` section of the `PAISConfiguration` YAML. For environments that host a private Harbor registry for AI container images, PAIS guidance recommends allocating registry storage space to accommodate at least three versions of each container image planned for deployment, so that registry growth is anticipated rather than reactive.

The foundational storage layer, including vSAN storage policies, namespace storage limits, and platform-level capacity monitoring, is managed through VCF rather than PAIS itself.

### VMware vDefend (Contributes)

VMware vDefend (vDefend) provides several log storage capacity management mechanisms for its security event data.

At the Distributed Firewall (DFW) layer, packet logging uses bounded log rotation: the `DFWPKTLOGS.log` file rotates at a 50 MB per-file limit, and the system retains a maximum of 10 log files at a time. This bounds DFW packet log storage on transport nodes while retaining sufficient history for troubleshooting and forensic analysis.

vDefend IDS/IPS includes an automated protective mechanism that handles high-volume event scenarios. When incoming IDS/IPS events and PCAP data would otherwise exhaust memory, the mechanism selectively drops non-critical events and PCAPs to maintain system stability. Administrators can observe event volumes through the IDS/IPS Security Dashboard and Security Overview Dashboard.

IDS/IPS event data can be forwarded to external syslog consumers by enabling the `ids_events_to_syslog` setting under IDS/IPS Settings. Exporting events externally offloads local storage consumption so that log accumulation on transport nodes is bounded by the rotation policy rather than event volume. Remote syslog must be configured on the ESX host to receive IDS/IPS events exported directly from ESX hosts. vDefend components write logs to `/var/log`, with syslog messages conforming to RFC 5424 on vDefend appliances and RFC 3164 on ESX hosts.

Guest Introspection Host Agent (MUX) logs are written to `/var/log/syslog.log` on ESX hosts, with each log message including a timestamp, ContextMUX identifier, and thread ID.

The Security Services Platform (SSP), which hosts Security Intelligence and Network Detection and Response analytics, has documented storage sizing baselines for capacity planning. Each SSP controller node requires 225 GB of storage and each worker node also requires 225 GB. Storage requirements scale with worker node count, adding approximately 155 GB per additional worker node. For a maximum configuration of 10 worker nodes (14 total nodes), the recommended persistent volume claim is 3,545 GB. These sizing baselines enable administrators to allocate storage capacity appropriately at deployment time.

vDefend logs can be forwarded to VCF Log Management by configuring it as a syslog server. VCF Log Management provides a centralized log management framework and supports querying log data by time range through Interactive Analytics.

These mechanisms address event log storage capacity for vDefend's security event data. Broader log storage governance across the VCF platform, including infrastructure-wide capacity monitoring and centralized retention policies, is handled by VCF Log Management and VCF Operations.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) provides configurable log storage capacity management through file size limits, file count limits, and per-component logging level controls that help prevent log storage exhaustion.

PNR Server log files are managed through the `vmware-dr.xml` configuration file. The `<maxFileSize>` parameter controls the maximum size of individual log files, with a default of 10,485,760 bytes (10 MB). The `<maxFileNum>` parameter controls the maximum number of log files retained, with a default of 20 files. PNR also raises a low disk space event when available disk space on the server falls below a configurable threshold, controlled by the `localSiteStatus.minDiskSpace` parameter with a default of 100 MB. This event can trigger alarms that alert administrators before log storage is exhausted.

Configurable logging levels per component allow administrators to reduce log verbosity when log files are growing too quickly, or increase it temporarily for troubleshooting. Named per-component settings configurable through the Reconfigure Protection and Recovery Settings interface include `logManager.Storage`, `logManager.StorageProvider`, `logManager.Licensing`, and `logManager.VvolProvider`, each with a default value of `verbose`. This allows log output from storage, storage provider, licensing, and Virtual Volumes subsystems to be tuned independently.

The PNR Appliance supports forwarding log files to a remote syslog server, allowing log volume to be offloaded to centralized log management infrastructure. In environments using the Live Cyber Recovery (LCR) clean room, PNR can forward events from the clean room to VCF Log Management (version 9.0 or above), Microsoft Sentinel, or Splunk Cloud.

vSphere Replication, which is part of PNR, supports event logging and allows definition of alarms for each event. vSphere Replication maintains dedicated audit logs at `/opt/vmware/support/logs/hms/hms-audit.log`, with broker logs at `/opt/vmware/support/logs/hbr` on the Appliance and at `/var/run/log` on ESX hosts.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides configurable log storage settings and two-tier capacity alerting that support proactive management of event log storage.

DSM Provider logging includes configurable log storage settings accessible through the Provider interface under Storage Settings. Administrators configure the Provider Log Repo URL to direct logs to an S3-compatible object storage repository and bucket; the Bucket Name and optional Region settings complete the S3 object store configuration. Gateway audit log files are stored locally on the Provider VM at `/var/log/tdm/provider/audit/` using filenames in the format ``gateway-audit-timestamp.log``, and are archived periodically to an S3 subpath named ``gateway audit`` when a provider log repository is configured. The Provider VM ships with a default storage configuration of three disks totaling 595 GB (80 GB, 15 GB, and 500 GB), which can be sized for the anticipated log and backup volume at deployment time.

DSM includes two-tier capacity alerting to notify administrators before storage is exhausted. The ``data-disk-warning-threshold`` setting governs a WARNING global alert when data disk usage reaches 70%; the ``data-disk-critical-threshold`` setting governs a CRITICAL global alert at 90%. When either threshold is breached, administrators can increase the storage allocation using the ``storage-amount`` setting. These alerts are visible to DSM administrators and prompt timely remediation before log capacity is exceeded.

For SQL Server databases, DSM exposes the ``errorLog.numErrorLogs`` custom engine setting, which controls how many error log files are retained before recycling. Configuring this setting allows administrators to manage database-engine-level log storage consumption directly.

S3-compatible storage sizing for DSM depends on the volume of data backups and the configured retention policy. Organizations must assess their log generation rates, retention requirements, and backup volumes to right-size the S3 repository. DSM does not automatically provision or expand storage capacity, so organizational processes for periodic capacity review and storage expansion remain necessary to fully satisfy this control.

### VMware Avi Load Balancer (Contributes)

VMware Avi Load Balancer provides configurable log storage capacity at both the Controller and Service Engine layers, giving organizations the mechanisms to allocate and proactively manage event log storage for application delivery traffic.

At the Service Engine layer, disk capacity for logs is calculated dynamically based on total disk size, CPU cores, RAM, and the configurable maximum-storage-not-allocated-for-logs setting, adjustable through SE runtime properties. These values drive two derived pools: client-logs-capacity (total disk minus debug logs capacity, with adjustments based on configured minimum storage allocated for logs and SE RAM) and debug-logs-capacity (calculated as the SE total disk minus the maximum storage not allocated for logs, divided by 100). A minimally sized Service Engine with two vCPUs, 2 GB of RAM, and 10 GB of storage can hold approximately 8 million log entries. When storage is exhausted, the oldest Significant Log entries are phased out on a per-Service Engine basis and their disk space freed automatically, keeping the Service Engine operational without requiring operator intervention.

Increasing disk capacity on a Service Engine directly extends both the volume of logs retained and the total retention duration. Avi documentation recommends provisioning Controllers and Service Engines with larger storage allocations in environments that require extended log retention. Adding CPU, memory, and disk resources to Service Engines and Controllers scales log throughput and retention proportionally.

For deployments using the Avi Kubernetes Operator (AKO) within VCF, the AKO `persistentVolumeClaimSpec.storage` parameter allocates 2 GiB of storage for the log Persistent Volume Claim by default, providing a configurable log storage allocation for Kubernetes-based workloads.

For log retention beyond what local disk supports, the Avi Controller's Syslog-Audit-Persistence feature streams alerts and audit events to external syslog servers. The Controller also supports sending audit records for trusted channel initiation, termination, and failures to Events, Syslog, and Splunk destinations. Offloading to an external syslog or Splunk infrastructure decouples event log retention from local Service Engine disk allocation and supports longer retention windows managed at the logging tier. Note that Syslog notification actions push alert events to syslog servers but do not directly export virtual service logs; virtual service logs must be pulled from an external logging system through the API or scripted to be pushed from the Controller.



## Control CR 2.10(a)

> Sub-item (a) of CR 2.10 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 2.10 requirement in the standard.

**SCF Controls:** BCD-01

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides a broad set of infrastructure-level capabilities that support business continuity and disaster recovery planning. While VCF does not itself constitute a Business Continuity Management System, it supplies the technical mechanisms that underpin resilient deployments and contingency operations.

vSAN stretched clusters allow organizations to extend a single vSAN datastore across two geographically separated sites with an independent witness host. When configured with site mirroring, a stretched cluster can tolerate the complete loss of one site while maintaining data availability. vSphere HA, when enabled on a stretched cluster, automatically restarts virtual machines on the surviving site during a site failure. To support this, vSphere HA admission control should be configured to reserve 50% of the cluster's aggregate CPU and memory resources for failover, and site-specific isolation response addresses should be defined so that vSphere HA can validate host isolation even when network connectivity between sites is lost. The admission control setting for host failures the cluster tolerates should also be updated after cluster expansion to reflect the correct host count per availability zone. When stretching a workload domain cluster, the default management vSphere cluster must already be stretched. vSphere HA uses network heartbeats to validate the state of VMware ESX hosts, and can also use heartbeat datastores mounted to more than one host (excluding vSAN datastores) for cluster membership monitoring.

vSphere HA also operates at the individual cluster level independent of stretched clusters. The HA configuration remediation mode can be set to quarantine mode, maintenance mode, or mixed mode to handle host failures in a controlled manner, helping maintain workload availability during hardware events. vSphere Fault Tolerance provides a higher tier of availability for individual virtual machines by maintaining a secondary VM in continuous lockstep with the primary, enabling instantaneous failover with no data loss if the primary host fails.

vSAN Data Protection provides snapshot-based protection through the vSAN Snapshot Service. Protection groups support configurable snapshot scheduling with adjustable intervals and retention policies that define retention duration and time units. This allows organizations to define data protection policies aligned with their recovery point objectives. The Snapshot Service API also supports operations such as restoring deleted virtual machines from snapshots and creating linked clones from snapshots.

vSphere Replication provides asynchronous replication of virtual machines with a configurable Recovery Point Objective (RPO), allowing organizations to replicate workloads to a secondary site and define acceptable data loss windows as part of their contingency planning.

VCF requires all management components, including VCF Operations, VMware vCenter, NSX, and VCF Automation, to be configured with a regular backup schedule and retention policy. VCF Operations configures NSX Manager backups during the initial deployment, with a recommended backup frequency of hourly and 7-day retention. Backup jobs for VCF Operations and all vCenter instances should start within the same 5-minute window to capture a consistent state. The VMware Infrastructure Health adapter within VCF Operations monitors backup job retention for management components, including NSX Manager. Changed Block Tracking (CBT) tracks modified disk sectors on virtual machines to support incremental backups, reducing backup windows and storage requirements for VM-level backup operations. vCenter itself also supports a High Availability configuration with Active and Passive nodes whose configuration files are continuously replicated, providing management-plane resilience independent of the backup schedule.

VMware HCX provides workload mobility capabilities for migrating and replicating VMs between sites. HCX supports multiple migration types through its Mobility Agent service, including vMotion, Cold Migration, Bulk Migration, and Replication Assisted vMotion (RAV), each addressing different scenarios from large-scale parallel migrations to zero-downtime live migrations.



HCX also includes appliance-level backup and restore capabilities for the HCX Manager, including certificate backup and restore, supporting planned migrations and recovery scenarios as part of broader BC/DR operations.

VCF Operations monitors infrastructure health and surfaces alerts when conditions threaten availability, giving operations teams early warning of potential failures. The Platform Health Alerts page provides a centralized view of alerts across data sources and infrastructure nodes. VCF Automation provides alerts through the associated VCF Operations application for capacity, performance, and availability monitoring. VCF Operations also includes infrastructure planning and what-if analysis capabilities that help organizations forecast capacity requirements and plan for contingency scenarios. Committed scenarios reserve capacity by setting aside resources for new, upcoming, or planned workloads, helping stakeholders understand capacity requirements across the environment. Integration with IT Service Management and Configuration Management Database platforms is supported through VCF Automation, enabling organizations to tie their infrastructure management into broader service continuity workflows.

Lifecycle management tools allow administrators to plan updates and schedule maintenance windows to make changes with minimal disruption to services, supporting the operational continuity aspect of business continuity planning.

While VCF provides these technical capabilities, a complete BCMS requires organizational elements that fall outside the platform: documented continuity of operations plans, recovery time and recovery point objectives defined by business stakeholders, regular testing and exercise of recovery procedures, communication plans, and management oversight of the continuity program. VCF supplies the infrastructure resilience and recovery mechanisms that such plans depend on, but the planning, governance, and procedural components must be established and maintained by the organization.

#### **VMware vDefend (Contributes)**

VMware vDefend contributes to business continuity by providing security-layer continuity mechanisms that remain active during disaster recovery events and workload migrations. NSX Federation Security maintains security posture across data centers during disaster recovery and workload migration, preserving distributed firewall and gateway firewall policies so that enforcement continues through failover.

The Security Services Platform, which provides the underlying infrastructure for vDefend's threat detection and inspection services, supports vSphere High Availability configurations. When a VMware ESX host becomes unhealthy, vSphere HA migrates the affected Security Services Platform virtual machines to healthy hosts in the cluster, restoring threat detection capacity without manual intervention. VMware recommends enabling vSphere HA for host-level failure recovery and configuring a storage policy with RAID for datastore-level failure tolerance in Security Services Platform deployments.

Security Services Platform can also be deployed on a vSAN stretched cluster configuration, which replicates storage across two physically distinct sites connected by a high-bandwidth or low-latency inter-site link. This configuration requires a minimum of two sites with their own hosts and a witness host to handle potential conflicts arising from network partitions, supporting site-level failure scenarios.

When vSphere HA restarts workload VMs on a new recovery host after a failover, the vDefend distributed firewall automatically reinspects all traffic from those VMs on the receiving host, because the Flow Table contents cannot be moved between hosts during host recovery. Security policy enforcement resumes on the new host without manual reconfiguration.

The broader BCMS framework, including Continuity of Operations Plan and BC/DR playbooks, impact analyses, and organizational governance processes, is outside vDefend's scope and must be established through dedicated business continuity management programs.

#### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) provides the technical execution layer for contingency planning in virtualized infrastructure. Recovery plans define the sequence, priority, and configuration for failover operations, acting as executable BC/DR playbooks. Protection groups organize virtual machines into logical units aligned with application tiers or business functions, establishing the scope of each contingency plan.

Recovery plans support non-disruptive testing before use in planned migration or disaster recovery, allowing organizations to validate their contingency procedures without affecting production workloads. Recovery plan configuration differs between emergency scenarios at the protected site and planned migration of services from one site to another, with PNR supporting both modes. A forced recovery function is available during disaster recovery plan execution for scenarios requiring immediate failover.

vSphere Replication provides host-based replication with configurable Recovery Point Objectives (RPO), allowing organizations to set per-workload data loss tolerances. vSphere Replication supports up to 200 point-in-time recovery instances per virtual machine using the Latest Instances option, enabling recovery to specific earlier states. This capability is relevant to contingency scenarios involving data corruption or ransomware events. When the source site is accessible at the time of recovery, vSphere Replication can synchronize recent changes from the source site to the target site before completing the operation, reducing data loss.

In 9.1, PNR extends to multi-tenant disaster recovery through integration with VCF Automation. The product implements role-based access control with Organization Administrator, Project Administrator, and Project Advanced User roles, each with defined authorization levels for disaster recovery operations. Role assignments must be configured on both the primary and secondary PNR sites to maintain consistent access control across the DR environment. A multi-region deployment topology places separate management domains in each region to contain disaster impact to one region, while enabling service recovery in an alternate management domain when VCF Automation and VCF Operations are affected. The VCF Operations Orchestrator Plug-In for PNR provides Configuration, Inventory Mappings, Protection Groups, and Storage predefined workflow categories for workflow-driven recovery operations.

Cyber recovery deployments support a standard VCF topology where the workload domain is isolated from the management domain, providing separation between management and user workloads in clean-room recovery scenarios.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer provides application-layer resilience mechanisms at both the control plane and the data plane that organizations can incorporate into business continuity plans for application delivery services.

At the control plane, the Avi Controller is deployed as a cluster with continuous configuration replication across all nodes, providing management-plane redundancy so that a single Controller node failure does not interrupt application delivery operations. Avi supports deployment of the Controller cluster across two availability zones, with vSphere HA recommended to support recovery of any node that fails in the secondary zone. For single-node deployments, enabling vSphere HA on the Controller VM is recommended for node recovery following a host failure. The Avi Controller provides configuration backup and restore capability that supports disaster recovery and configuration migration across Controller instances; organizations should capture and retain Controller backups as part of their continuity documentation.

Availability Zone awareness in Avi allows multiple availability zones to be configured, each defining a subset of vCenter instances, hosts, and datastores that share a fault domain. Configuring availability zones during Service Engine Group setup distributes data-plane capacity across isolated failure domains, supporting continuity designs where application delivery capacity remains available if one fault domain becomes unavailable.

At the data plane, Avi's Elastic High Availability mode provides fast recovery for individual virtual services following Service Engine failure. In vSphere DRS-enabled clusters, virtual service recovery occurs before vSphere HA brings up a replacement Service Engine when virtual service slots are available in other Service Engines, giving the application delivery layer its own recovery path distinct from the infrastructure restart cycle. If vSphere HA does not recover the failed Service Engine within a configured timeout period, the Avi Controller resumes recovery independently. Service Engine VMs should be deployed on vSphere clusters configured with both HA and DRS in fully automated mode; when Avi is deployed with VMware Kubernetes Service (VKS) on VCF, DRS and HA are required on every vSphere cluster used with Avi.

For Kubernetes environments, the Avi Multi-Cluster Kubernetes Operator (AMKO) provides federation architecture that supports disaster recovery across Kubernetes clusters. AMKO designates a leader cluster and multiple member clusters, coordinating application delivery failover as part of a multi-cluster DR strategy.

Avi ControlScripts, used in conjunction with the alert framework, can perform automated failover actions such as adding or removing routes based on VIP attachment and detachment events, allowing organizations to define automated remediation workflows that respond to system events without manual administrator intervention.

In 9.1 deployments using VCF Automation, Avi's Central Licensing maintains a global capacity pool that removes the requirement for duplicate licenses in DR scenarios, supporting license continuity across failover sites.

Organizations should document these mechanisms, including the Elastic HA configuration, Controller cluster topology and backup schedule, availability zone placement, and AMKO federation design, in their Business Continuity and Disaster Recovery playbooks. The technical recovery capabilities are built into the product; the planning artifacts, testing procedures, and escalation workflows are organizational responsibilities that extend beyond what Avi provides.

Not applicable for this control: VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 2.10(b)

> Sub-item (b) of CR 2.10 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 2.10 requirement in the standard.

**SCF Controls:** BCD-12.2

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides multiple layers of real-time and near-real-time failover capability across compute, storage, networking, and management components, directly addressing this control's requirements for maintaining availability of critical technology assets, applications, services, and data.

At the compute layer, vSphere High Availability (HA) monitors host health and automatically restarts virtual machines on surviving hosts when a failure is detected. Automatic failover shifts resources to healthy nodes in the cluster without manual intervention. For workloads requiring zero downtime, vSphere Fault Tolerance maintains a live shadow instance of a protected virtual machine, so that any hardware failure results in no data loss and no service interruption. Proactive HA integrates with hardware partner monitoring through vSAN Proactive Hardware Management (PHM), which detects degrading components via predictive failure events generated by the OEM vendor. When a predictive failure is identified, Proactive HA places the affected host into maintenance mode, evacuating VMs before a failure occurs. A supported Hardware Support Manager (HSM) must be configured and registered to VMware vCenter for this capability.

vSphere HA admission control can be configured to reserve a percentage of the cluster's aggregate CPU and memory resources specifically for failover capacity. Three admission control policy types are available: cluster resource percentage, slot policy, and dedicated failover hosts. The "Host failure cluster tolerates" admission control setting defines how many simultaneous host failures the cluster can absorb while still restarting all affected workloads. Remediation mode can be set to Quarantine mode, Maintenance mode, or Mixed mode to control how host failures are handled. For vSAN stretched clusters, admission control should reserve 50% of the cluster's aggregate CPU and memory resources so that all virtual machines from one site can be restarted on the other. vSphere HA can also use DRS to bring hosts out of standby mode and migrate virtual machines to defragment cluster resources, so that failover capacity remains available when aggregate resources are fragmented across the cluster.

vMotion enables live migration of running virtual machines to a new host without interruption in availability. VCF Operations HCX extends this mobility to remote sites for cross-site workload movement. HCX Network Extension High Availability provides failover protection for extended networks. Network Extension appliances operate in active-standby pairs through role negotiation, and during a failover event the standby pair takes over the Network Extension service from the active pair. After the failed appliance recovers, the pair renegotiates roles and returns to a healthy state, providing self-healing behavior.

For site-level failover, vSAN stretched clusters span two geographically separated sites with a witness host, enabling VCF to tolerate an entire site failure. When one site becomes unavailable, vSphere HA restarts affected virtual machines on the remaining active site. The storage policy "Site disaster tolerance" rule, when configured for site mirroring, allows the stretched cluster to tolerate a witness host failure even when one site is already unavailable. Site-specific isolation response addresses should be configured so that vSphere HA can validate host isolation even during a network failure between sites. Both availability zones must contain an equal number of hosts to provide failover capacity when either zone goes down.

At the storage layer, vSAN provides storage-level resilience through multiple mechanisms. vSAN fault domains enable the environment to tolerate entire rack failures in addition to single host, storage disk, or network failures, by organizing hosts into separate fault domains so that vSAN distributes object replicas across different physical racks. Configuring fault domains of smaller sizes reduces the impact of a potential rack failure. vSAN Express Storage Architecture (ESA) supports disaggregated storage clusters that pool storage devices across dedicated hosts, separating storage resources from compute hosts. Storage multipathing provides path selection policies with automatic failover to maintain storage access during path failures, and NFS 4.1 datastores support multipathing for redundant storage connectivity. Path policies must be consistent within a cluster. vSAN network implementations should typically use four physical uplinks for full redundancy, with additional failover NICs

provisioned for the vSAN network. The teaming and failover policy configured on the backing virtual switch provides network redundancy for vSAN traffic.

At the networking layer, multi-NIC teaming, Link Aggregation Control Protocol (LACP), and failover policies handle network-level failover. Advanced NIC teaming can implement an air gap so that a failure on one network path does not affect the other. NSX tier-0 gateways support active-standby high availability mode with configurable failover behavior (preemptive or nonpreemptive). NSX Edge nodes perform graceful failover to a peer Edge when a failure or maintenance event occurs. For multi-site NSX deployments, the NSX Global Manager can be deployed in a highly available cluster model with a standby cluster ready for manual failover. In unplanned switchover scenarios where a primary site becomes unavailable, a forced failover can be initiated to restore networking services at the surviving site.

VCF also provides failover capabilities for its management and operations components. vCenter supports native high availability through vCenter High Availability, which deploys Active, Passive, and Witness nodes; when the Active node becomes unavailable, the Passive node automatically assumes the active role, preserving management plane operations without manual intervention. The Continuous Availability VCF Operations Model deploys a highly available cluster with data redundancy across different availability zones, maintaining service during a single availability zone failure. VCF Operations failover server sets establish server connections in a defined order and can link separate server sets, providing high availability for monitoring infrastructure in complex environments. Application monitoring high availability with activated collector groups in failover mode can continue monitoring applications without losing data or metrics even when a cloud proxy is down or its components fail. Cloud proxy itself supports high availability. The High Availability VCF Automation Model supports use of an external load balancer for management service resilience.

The VCF platform's Kubernetes consumption layer, provided by vSphere Supervisor and VMware Kubernetes Service (VKS), extends these failover mechanisms to containerized workloads. Supervisor requires vSphere HA and fully-automated DRS on the underlying vSphere cluster, so VKS node virtual machines benefit from the same automatic restart and workload balancing that protects traditional VMs. In a zoned Supervisor deployment, Supervisor stretches across three vSphere clusters, with each cluster assigned to a vSphere Zone that represents a distinct failure domain; a failure in one zone does not automatically affect workloads running in the other zones. Starting with VCF and VKS 3.6, Node Auto Placement allows the VKS placement engine to automatically manage resource distribution across failure domains, spreading node pools across zones using failure domain specifications in the cluster topology. Before any zone is taken out of service, control plane nodes must be relocated to a different zone to maintain cluster availability, supporting planned maintenance without workload interruption. Kubernetes provides self-healing behavior at the workload layer: the platform automatically replaces failed containers, reschedules workloads when nodes become unavailable, and continuously reconciles the system toward its desired state. VKS cluster version updates use a rolling model in which control plane nodes are updated first, followed by worker nodes one at a time per node pool starting with the Zone-A pool, keeping the cluster operational throughout planned maintenance. For stateful workloads, VKS clusters use the VMware vSphere Container Storage Interface (CSI) driver to provision persistent volumes backed by vSAN datastores, anchoring data durability in the same vSAN fault-tolerance mechanisms described above. High-availability applications on VKS must be deployed as replicated stateless or stateful workloads to take advantage of these self-healing mechanisms at the application layer.

For planned migrations and disaster recovery workflows, vSphere Replication supports configurable Recovery Point Objectives (RPO) for near-real-time data replication. Planned failover on a replication group allows recovery of workloads on a target site and then reverses the direction of replication, making the target site the new source. The recommended approach is to combine real-time availability through HA with site-wide disaster recovery for maximum resilience.

#### **VMware vDefend (Contributes)**

VMware vDefend (vDefend) implements failover capabilities for its own security infrastructure to maintain continuous network security enforcement during component failures.

The vDefend gateway firewall operates in primary-passive high availability mode with non-preemptive failover. When all uplink routing links go down on the active Edge node, failover is triggered and the standby node assumes the active role. Firewall flow state is synchronized between Edge nodes, so existing sessions are preserved across failover events without

requiring re-establishment. Sessions resume after a brief convergence period during Edge node failover. This flow synchronization allows security policy enforcement to continue through a Gateway Firewall failover without leaving traffic unprotected during the transition.

The Security Services Platform (SSP), which hosts vDefend's advanced threat detection functions including IDS/IPS, Malware Prevention, and Network Detection and Response, has built-in high availability by default. Enabling vSphere HA is recommended for the vSphere cluster hosting the Security Services Platform. When a host becomes unhealthy, vSphere HA migrates the affected Security Services Platform virtual machines to healthy hosts in the cluster. Admission Control should be configured with sufficient resource reservations so that recovery capacity is available. Configuring a storage policy with RAID provides datastore-level failure tolerance alongside vSphere HA. In environments using vSAN stretched clusters, the Security Services Platform benefits from storage replication across two physically distinct sites, providing site-level resilience and protection against multiple host failures within a single site. The License Hub component also supports vSphere HA for host-level failure recovery.

The Distributed Firewall operates at the hypervisor kernel level on every host, providing inherent distribution with no single point of failure for workload security policy enforcement. One operational characteristic to note: when vSphere HA is enabled and workload VMs restart on a new recovery host after a failure, all flows from those HA-impacted VMs are reinspected by the Distributed Firewall because vDefend cannot migrate Flow Table contents during host recovery. Policy enforcement applies to these VMs immediately upon restart on the recovery host.

These failover capabilities help vDefend's security enforcement layer remain operational during infrastructure failures. Network security policy enforcement, threat detection, and microsegmentation continue to function as the underlying compute and storage infrastructure undergoes failover events.

### **VCF Protection and Recovery (Meets)**

VCF Protection and Recovery (PNR) implements site-level failover capabilities to maintain availability of critical virtual machine workloads after a disruption.

Recovery plans automate the failover process between protected and recovery sites. During disaster recovery failover, if an error occurs on the protected site, the failover continues without interruption, prioritizing workload recovery. Recovery plans support both planned migration for orderly, scheduled failover and disaster recovery for unplanned site failures. PNR provides an API workflow through the VCF Operations Orchestrator Plug-In for PNR to initiate failover programmatically, enabling integration with automated orchestration frameworks.

PNR supports multiple replication methods that underpin the failover capability. vSphere Replication provides host-based replication with configurable recovery point objectives. A recovery option allows vSphere Replication to synchronize recent changes from the source site to the target before recovering virtual machines, reducing data loss during recovery operations. vSphere Replication supports up to 200 recovery points when using the Latest instances option, enabling selection of a specific point in time for recovery. Replication continues at the defined recovery point objective after vSphere High Availability restarts a virtual machine. Array-based replication leverages storage vendor capabilities for datastore-level failover, with a configurable option to wait for device rediscovery after failover. Automated failback is supported only for array-based replication. Virtual Volumes policy-driven replication provides a third replication method.

Failover and failback can be performed independently on sites, allowing isolated recovery operations without affecting other recovery sites. For stretched storage environments, PNR requires that the failover command isolate devices at the recovery site. IP customization is applied during the failover process, supporting DHCP, static IPv4, and IPv6 addressing so that recovered virtual machines are reachable on the recovery site network. Inventory mappings translate protected site resources to their recovery site equivalents.

PNR supports failover even when operating in degraded mode, though test failover and reprotect operations are not available in that state. Test failover allows organizations to validate their failover procedures without disrupting production replication. After a successful failover, the reprotect process reverses the direction of protection, making the recovery site the new protected site and restoring replication to prepare for failback.



PNR supports failover for Windows Server Failover Clustering (WSFC) workloads. The Cyber Recovery capability enables scheduled snapshot management through vSAN Data Protection groups for VM-level protection that integrates with the broader failover workflow.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides built-in high availability and failover mechanisms for PostgreSQL, MySQL, and SQL Server database workloads through two HA topologies that organizations can select based on their availability requirements.

The Single vSphere Cluster HA topology co-locates all database nodes within one vSphere cluster. For PostgreSQL, DSM deploys a primary instance, one database-level synchronous replica, and a dedicated monitor node. The monitor node orchestrates automatic database-level failover: when the primary fails, the monitor detects the failure and promotes the synchronous streaming replica to the primary role without requiring manual intervention. MySQL under this topology uses a three-node configuration with a single primary and two database-level synchronous replicas sharing the same cluster. SQL Server uses a three-node configuration with Microsoft Always On availability groups, two synchronous replicas, and one configuration-only replica. After a SQL Server failover, the new primary maintains the security and management configurations from the previous primary instance. In all cases, DSM database nodes also rely on the underlying VCF platform for VM-level recovery through vSphere HA and Distributed Resource Scheduler, and on platform storage redundancy.

The Cross vSphere Cluster HA topology distributes database nodes across three separate vSphere clusters, reducing the blast radius of a single cluster failure. For PostgreSQL, this topology places a primary, a database-level replica, and a monitor on distinct clusters. For MySQL, a primary, a replica, and a monitor are distributed across three clusters. For SQL Server, Always On availability groups are configured with two synchronous replicas and one configuration-only replica, with each replica node in a separate vSphere cluster. Cross vSphere Cluster HA provides the highest availability and resilience for environments that require high availability across multiple failure domains. When configuring Cross-Cluster HA in an infrastructure policy, three clusters must be selected from the same datacenter and must share the same storage policies and distributed port group. The Enable-Cross-Cluster-HA infrastructure policy setting activates this topology. DSM raises a compliance alert if vSphere HA is not active on a cluster referenced by an infrastructure policy.

The DSM Provider Appliance management plane also relies on vSphere High Availability so the control plane can recover from a host failure without manual intervention.

For cross-site disaster recovery, DSM supports a multi-site deployment model for PostgreSQL that replicates data to a secondary site. When a failover to the secondary site is needed, DSM provides a switchover workflow to promote the secondary instance to the primary role. After the original site recovers, a fallback switchover reverses the roles to restore the original topology. DSM also provides the ability to copy replication user connection strings from a primary instance so that secondary instances can be redirected to the new primary during failover operations. One documented limitation applies: PostgreSQL logical replication requires manual intervention to resume replication after the source cluster experiences a failover. When a MySQL HA cluster is used as the replication source, automatic source failover for read replicas requires the allocation of additional IP addresses from the IP pool for each HA node, and read replicas reconnect to another cluster member after several failed attempts, which can take a few minutes before the switch completes. For SQL Server, during a failover new writes are not protected with a second copy until the failed node rejoins the cluster.

DSM HA clusters require automated backups to be enabled in order to provision databases. DSM provides real-time alerts for resource and connectivity issues together with continuous performance monitoring, which supports operators in detecting conditions that may require a failover response. Webhook notifications can also be configured so that external systems are alerted on data service operation failures.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides multiple layers of real-time and near-real-time failover mechanisms for application delivery. These operate at the Service Engine data plane, at the load balancing pool level, and at the Controller management plane, working in conjunction with the availability capabilities of the VCF platform.



At the data plane, Avi's Elastic High Availability (Elastic HA) mode provides fast recovery for individual virtual services following Service Engine failure. Elastic HA supports both Active-Active and N+M configurations, giving administrators a choice of redundancy model suited to their requirements. When a virtual service is already scaled across multiple Service Engines and one fails, the virtual service switches over to the remaining available Service Engines; this recovery occurs before vSphere HA brings up a replacement Service Engine when virtual service slots are available in other Service Engines. The switchover time for Service Engines in vSphere HA failover scenarios is configurable per virtual service requirements.

For deployments requiring active/standby behavior, Avi supports Legacy Active/Standby High Availability mode, where a pair of Service Engines share a floating interface IP address for transparent failover. By default, the standby Service Engine sends health checks to pool members so that server health state is already known at the time a failover occurs, reducing recovery latency. For Layer 4 connections, enabling connection mirroring on Avi provides seamless failover and avoids disruption of existing connections during failover events.

The Autoreistribute Load option on Service Engine groups can be enabled to make fail-back automatic when load is concentrated on the failover Service Engine after a recovery event.

At the pool level, Avi implements priority-based pool failover logic. The `deactivate_primary_pool_on_down` setting enables automatic deactivation of the primary pool when it becomes unavailable, triggering automatic failover to a designated secondary pool. The Avi Controller ControlScript can also execute failover operations on pool groups triggered by Pool Up and Pool Down events.

For cross-fault-domain resilience, Availability Zone support enables spreading Service Engines across transport nodes that do not share common failure points, such as shared power supplies or shared networking devices. Defining Availability Zones based on fault domains helps keep virtual services available across fault conditions that affect a single zone. At the Controller plane, Controller cluster deployments can span two availability zones with defined failure scenarios and recovery actions for each configuration.

For global application availability across data centers, Avi GSLB supports Active-Active configuration where applications are deployed in multiple data centers and instances in remaining data centers can take over user traffic if a data center becomes unavailable.

Service Engine VM-level recovery in host failure scenarios depends on vSphere HA from the underlying VCF platform. Avi documentation recommends enabling vSphere HA on the vSphere clusters where Service Engine VMs are deployed. In VCF-integrated deployments, vSphere DRS and HA must be enabled on each vSphere cluster used with Avi, with vSphere DRS configured in Fully Automate mode. VMware HA provides failure protection for a single-node Avi Controller by dynamically migrating and restarting the virtual machine on another server in the event of server failure; enabling vSphere HA is similarly recommended for the vSphere cluster where Avi License Hub is deployed. Administrators should note that if all Service Engines are placed on a single host and that host fails, there is no high availability or immediate failover, making host distribution a deployment planning consideration. Administrators must configure both Avi HA policies and the vSphere cluster settings to fully address this control.

## Control CR 2.11

> Timestamps: Capability to include timestamps in audit records. Time synchronization: Capability to synchronize the component clock with an NTP source. Time synchronization protection: Protect the time synchronization process from tampering.

**SCF Controls:** MON-07

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides Network Time Protocol (NTP) and Precision Time Protocol (PTP) configuration across all infrastructure components, allowing administrators to designate authoritative time sources that generate consistent timestamps for event logs throughout the environment.

At the host layer, VMware ESX hosts support both NTP and PTP through the `HostDateTimeSystem` managed object. PTP provides sub-microsecond synchronization accuracy for environments that require high-precision time coordination. PTP supports both software- and hardware-based timestamping on ESX hosts, with hardware timestamping available for deployments where network adapters provide that capability. During deployment, ESX hosts must have the NTP client enabled with the service startup policy set to start and stop with the host. The `HostServiceSystem` API allows administrators to start and stop the NTP daemon and retrieve its status, while the `UpdateDateTimeConfig` API method updates the date and time configuration including NTP servers, PTP configuration, and time zone. Each ESX host generates syslog messages that conform with RFC 3164, which includes structured timestamp fields in log output. ESX system log file timestamps use UTC/GMT timezone with millisecond-or-finer granularity in ISO 8601 format (FULL-DATE T FULL-TIME Z), and syslog message transmissions include a `TIMESTAMP` field following the same ISO 8601 standard with full date and UTC time.

VCF Operations is the central point for time synchronization across VCF core components. When an administrator updates the NTP server configuration through VCF Operations, it propagates the change to VCF Operations itself, all VMware vCenter instances, ESX hosts, NSX Manager nodes, and NSX Edge nodes. If an NTP configuration update fails on a component, VCF Operations rolls back the NTP settings for that failed component. Time skew between new NTP servers must be less than five minutes for the update to proceed. VCF management components can also be configured to use ESX NTP servers as their time source, establishing a consistent time hierarchy across the platform.

NSX Manager uses its internal system clock to generate timestamps for audit records, and each syslog and audit log message contains a timestamp generated by an NTP server when one is configured, or by the system clock otherwise. NSX documentation states that NTP must be configured correctly across all devices that use log scraping. The recommended method to configure an NTP server for all NSX appliances is through a node profile, which allows centralized NTP management across NSX transport nodes and manager clusters. NSX Manager should have an NTP server configured to support accurate time synchronization for logging and system operations.

The vSAN Witness Host is also configured with NTP servers to synchronize time, keeping storage-layer events aligned with the rest of the infrastructure. For vSphere Supervisor deployments, the same NTP servers must be configured for the vCenter instance, all hosts in the cluster, NSX, and the Supervisor itself.

VCF Operations monitors NTP health by supporting NTPD as a monitored application service through its product-managed Telegraf agent. VCF Operations for Networks supports UTC format in audit logs. VCF Operations also allows administrators to synchronize the displayed time using either browser time or host machine time for dates and times shown in the user interface. CLI commands are available for NTP management in VCF Operations: the `ntp show` command displays the current NTP configuration and status, the `ntp set` command configures NTP servers with support for secure NTP, and the `ntp sync` command forces synchronization of NTP settings.

Timestamp traceability extends into authentication and API operations. The VCF SSO server validates timestamps to authenticate token issue requests, and security policies require message timestamps in SOAP security headers. The VCF SSO client can add a `TimeStampHandler` to include timestamp information in security token requests. The ESX Agent Manager API

includes a time attribute of type `dateTime` in its message elements to record when each message was generated, with messages also carrying a level attribute for info, warning, or alert classification.

The Consumption layer of VCF extends authoritative time synchronization and structured timestamp generation to workloads running on VMware Kubernetes Service (VKS) and vSphere Supervisor. Supervisor nodes run as virtual machines on vSphere infrastructure, and VMware Tools keeps guest VM clocks aligned with their ESX hosts, which synchronize with the platform NTP source. This mechanism propagates NTP-synchronized time to all Kubernetes control plane and worker node processes, so timestamps recorded in audit logs and event logs reflect the same authoritative time source as the underlying infrastructure. Within VKS clusters, the Kubernetes API server audit system records every API request using the `audit.k8s.io/v1` Event schema; at the Metadata level and above, each audit event carries a timestamp along with the requesting user, affected resource, and a unique audit ID. Audit events are written in JSONLines format to the file backend configured via the `--audit-log-path` flag. The Kubernetes Event API (`events.k8s.io/v1`) records cluster events with an `eventTime` field at microsecond-level precision; for repeated events, the `series.lastObservedTime` field records the most recent observation as a `MicroTime` value. Kubernetes records pod scheduling, image pull operations, and container creation events with timestamps and source information in the event log, and CronJob-controlled Jobs carry a `batch.kubernetes.io/cronjob-scheduled-timestamp` annotation in RFC 3339 format recording the original expected creation time.

Log forwarding from VKS clusters uses Fluent Bit, available as a standard package, which applies `Time_Key` and `Time_Format` parameters to format all forwarded events as ISO 8601 strings with timezone offset. Fluent Bit can deliver to Syslog, Elasticsearch, Kafka, and Splunk destinations, ensuring consistent timestamp formatting across log aggregation systems. For workloads running in vSphere Namespaces, the LogAgentConfiguration Custom Resource streams syslog messages to a remote syslog server; each forwarded message includes structured metadata fields for namespace name, pod UID, pod name, container name, and ESX host name, providing full context alongside the timestamp for each event.

To satisfy this control, administrators should configure NTP servers on all ESX hosts, NSX Manager nodes, vCenter instances, and VCF Operations using a stratum-appropriate authoritative time source designated by the organization. VCF Operations provides the centralized interface for propagating NTP configuration to all VCF core components. At the Consumption layer, Fluent Bit on VKS clusters should be configured with appropriate `Time_Key` and `Time_Format` values to ensure consistently formatted timestamps in forwarded logs, and LogAgentConfiguration Custom Resources should be applied to vSphere Namespaces to enable structured, timestamped workload log streaming to a central syslog destination.

### VMware vDefend (Contributes)

VMware vDefend and the Security Services Platform require accurate time synchronization across all components and treat NTP configuration as a prerequisite for several features. The Security Services Platform installer requires accurate time synchronization for certificate validation, log correlation, and secure communications to function correctly.

The Identity Firewall feature has a specific NTP dependency for event log scraping: NTP must be correctly configured across all devices participating in the identity-based policy workflow so that Active Directory event logs carry accurate timestamps. Without proper time synchronization, identity-based firewall rules cannot reliably correlate user login events with network traffic.

VMware vDefend supports time-based policies that activate or deactivate based on time of day or day of week, specified in 24-hour format. Correct operation of these policies depends on consistent time across all participating components.

Security Services Platform logs record timestamps in UTC format throughout operation, covering container initialization events, service status transitions, and certificate loading events. Usage data files use the ISO 8601 standard as the default timestamp format. The platform's Metrics API supports retrieval of time-series data with configurable granularity parameters. The Suspicious Traffic Summary panel in the Security Intelligence dashboard allows administrators to filter displayed events by configurable time ranges, and Network Traffic Analysis for Suspicious Traffic retains network telemetry for 30 days, supporting historical visibility and correlation. The Bare Metal server extension should be configured to use the same NTP server as the Security Services Platform to maintain consistent time synchronization across all components.

VMware vDefend components write syslog messages conforming to RFC 5424 on vDefend appliances and RFC 3164 on ESX hosts, providing standardized timestamp formatting that can be consumed and correlated by downstream monitoring and SIEM systems.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) generates timestamped event logs and audit records across its components for replication events, configuration changes, and recovery operations. vSphere Replication maintains a dedicated security audit log at `/opt/vmware/support/logs/hms/hms-audit.log` and records authorization error messages with timestamps and component information. vSphere Replication generates typed audit events for replication configuration changes, such as VM and host replication configuration and deconfiguration events, which are logged to vCenter Event Manager with typed event identifiers.

The PNR Appliance supports configurable time synchronization through three modes: Disabled (uses system time zone settings without active synchronization), Host (uses VMware Tools to synchronize time with the ESX host), and NTP (synchronizes directly with one or more NTP servers). The NTP server address or FQDN is configured on the Appliance via the time synchronization settings, and the PNR Proxy component separately requires NTP server configuration when deployed. The PNR Appliance also provides REST API endpoints to retrieve and update current time settings, allowing administrators to confirm and adjust time configuration programmatically.

When vCenter and the PNR Appliance run as virtual machines, PNR documentation recommends configuring NTP time synchronization on the ESX host using a persistent synchronization agent such as NTPD, W32Time, or VMware Tools time synchronization to avoid time management conflicts across PNR systems.

The PNR Appliance supports forwarding log files to a remote syslog server for centralized log analysis and monitoring. For Live Cyber Recovery deployments, the Clean Room Orchestrator forwards events to a Microsoft Azure Log Analytics workspace using a Data Collection Rule that includes a TimeGenerated column by default, providing timestamped event records in the isolated recovery environment.

The authoritative time source for the underlying VCF infrastructure, including ESX hosts, is configured at the VCF platform level. PNR's time configuration applies to the PNR Appliance and PNR Proxy components, which can be set to inherit host time or synchronize with a dedicated NTP server independently of the platform configuration.

### **VMware Data Services Manager (Meets)**

VMware Data Services Manager (DSM) supports timestamp accuracy through NTP integration and standardized UTC-based event logging. DSM lists NTP service as a hardware and software requirement: the system must be configured with a reliable time source to synchronize the clocks of all deployed VMs. The System Settings tab in the DSM UI exposes an NTP servers field where administrators specify the authoritative time sources the platform references. The System Config also includes NTP settings that apply to managed database instances, helping align clock sources across the DSM provider and the databases it provisions.

To assist administrators in detecting synchronization failures, DSM generates an NTP\_NOT\_APPLIED alert when NTP settings from the system configuration do not apply to a database, indicating that the database is not refreshing or synchronizing with system time. This alert appears in the Database Alerts and Metrics view and gives administrators a signal to investigate and correct the database's time configuration.

All DSM audit logs record timestamps in Coordinated Universal Time (UTC). The System Audit Events view displays Component name, Event Type, Event Details, Event Time, and Status for each audited operation; it is the only location in the DSM UI where non-database-related operations and tasks are tracked. When administrators export audit events using the ACTIONS drop-down menu, the export in CSV or XLSX format includes Event Time (UTC) alongside Component, Event Type, Event Details, Status, and Source, making the time reference explicit and consistent. Support bundle timestamps are displayed in UTC. Gateway audit log files are stored on the Provider VM in `/var/log/tdm/provider/audit/` with filenames in the format `gateway-audit-timestamp.log` and are archived periodically to an S3 subpath named 'gateway audit' when a default S3 provider log repository is configured.

For webhook-based alerting integrations, DSM includes multiple timestamp fields in its notification payloads: a Timestamp parameter in yyyy-MM-ddTHH:mm:ss format, a TriggeredAt field in yyyy-MM-dd-HH:mm format, and separate TriggeredDate and TriggeredTime parameters. These allow downstream systems to correlate DSM alerts with other event sources using consistent time values. The Maintenance Policy for software updates also requires a start time expressed in UTC only, so scheduled operations reference the same time base as DSM audit events.

DSM provides the NTP configuration surface and enforces UTC-based timestamps natively in its logging, export, and alerting outputs. Organizations designate the authoritative external NTP source that DSM references.

### **VMware Avi Load Balancer (Meets)**

VMware Avi Load Balancer provides native NTP configuration for both the Controller (control plane) and Service Engines (data plane), allowing administrators to designate authoritative time sources across all components independently.

The Avi Controller is configured with NTP servers through the `ntp_servers` setting under Controller Settings > DNS/NTP Settings. Service Engines synchronize time with the Controller by default, or administrators can configure NTP directly at the cloud level so that Service Engines sync with a designated NTP server independently of the Controller. On bare-metal (LSC) deployments, Service Engines support both `ntpd` (Network Time Protocol daemon) and `chronyd` as NTP daemon implementations. Service Engines synchronize time with configured NTP servers at startup and periodically monitor synchronization status thereafter.

All log entries carry timestamps derived from the NTP-synchronized system clock. Service Engine application logs include a Timestamp field recording the date and time of each event. The System Events Overlay in the Avi UI records a timestamp for each system event. The Controller Alert Table includes a timestamp field recording when each alert was triggered. The analytics logs API supports ISO 8601 time format (YYYY-MM-DDTHH:MM:SS in 24-hour format), with UTC as the default timezone unless a timezone is explicitly provided. The analytics metrics API supports querying metrics since a specific timestamp, enabling time-range queries from a defined starting point. Audit records for trusted channel initiation, termination, and failures are logged to Events, Syslog, and Splunk.

The Avi Controller provides a Syslog-Audit-Persistence template for streaming audit compliance events to external syslog servers. When deploying Avi Load Balancer in a VCF environment, NTP configuration is recommended for the Controller to support accurate logging and analytics.

Users can configure a time zone preference (UTC or local time) for display purposes through User Account Self-Service settings. The underlying time source remains the NTP-synchronized system clock, so timestamp presentation in the UI is consistent regardless of display preference.

## Control CR 2.11(1)

> Timestamps: Capability to include timestamps in audit records. Time synchronization: Capability to synchronize the component clock with an NTP source. Time synchronization protection: Protect the time synchronization process from tampering.

**SCF Controls:** MON-07.1

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides built-in Network Time Protocol (NTP) configuration across all components in the stack, enabling synchronization of internal system clocks with an authoritative time source. VCF Operations is the central point for NTP management: it uses NTP servers to synchronize time between VCF core components, and when an administrator updates the NTP server configuration, VCF Operations propagates those settings to vCenter instances, ESX hosts, NSX Managers, and NSX Edge nodes. If an NTP configuration update fails on any component, VCF Operations rolls back the NTP settings for that failed component. VCF Operations must be able to reach all VCF components over SSH to perform NTP configuration updates. It is recommended that all VCF systems receive their time from the same NTP source, even when configured with different time zones, to maintain consistency across the environment.

At the infrastructure layer, ESX hosts support NTP server configuration directly, and VCF management components can be configured to use the ESX host NTP server as their time source. vCenter time synchronization can be configured to use either an NTP server or VMware Tools as the synchronization method; native NTP is preferred because it is typically more accurate than VMware Tools periodic time synchronization. The vSAN Witness Host is also configured with NTP servers to synchronize time. NSX Manager supports NTP server configuration through both the management UI and the API (PUT <https://<nsx-mgr>/api/v1/node/services/ntp>), and accurate time synchronization on NSX Manager is an operational necessity because it uses its internal system clock to generate timestamps for audit records. NSX Node Profiles also support configuration of time zone and NTP servers, and some NSX features require that NTP be configured on all components in the NSX environment. Only one NTP service can be running at a time on NSX appliances and transport nodes. NSX transport nodes use the default time service provided by their underlying operating system: chronyd on RHEL/CentOS 7.6 and 7.7, systemd-timesyncd on Ubuntu (including edge nodes), and systemd-timedated on SLES 12 SP4.

VCF Operations supports two NTP synchronization modes. In self-managed mode, the primary node is the NTP server by default, and the time of all nodes is synchronized to the primary node, with the primary and replica nodes managing synchronization across the cluster. Alternatively, VCF Operations can synchronize with an external NTP server by entering a URL or IP address. VCF Operations for Networks provides CLI commands for NTP management: "ntp show" displays the current configuration and status, "ntp set" resets NTP servers with support for IP/FQDN and secure flags, and "ntp sync" forces synchronization of NTP settings. Time synchronization between endpoint VMs, vCenter, ESX hosts, cloud proxy, and VCF Operations must be maintained using NTP, as time synchronization is a prerequisite for TLS and SSO communication between client and server. Administrators should configure NTP to synchronize the server system time of the VCF SSO source and VCF Operations.

HCX also requires a valid NTP server with synchronized time for integrated systems operations. Administrators should schedule NTP server configuration updates at a time that minimizes impact to system users, as updates can take significant time depending on the size of the environment. VCF Operations restore procedures also require NTP server details to be configured for the appliance.

Accurate and consistent time synchronization is important for several operational functions across VCF. Kerberos authentication is sensitive to clock skew, and clock skew between systems can cause Kerberos authentication failures. Log correlation across components depends on consistent timestamps, and certificate validation relies on accurate system time. Time synchronization across ESX host environments should be verified as part of operational best practices.

At the Consumption layer, VMware Kubernetes Service (VKS) cluster nodes support NTP server configuration through the Cluster API. Administrators configure NTP servers for VKS cluster nodes by setting ``osConfiguration.ntp.servers`` in the cluster



definition, available in ClusterClass versions v3.2.0 and later. In earlier releases, the ``ntp`` cluster variable serves the same purpose. When ``osConfiguration.ntp.servers`` is not set, VKS cluster nodes inherit NTP configuration from the Supervisor automatically, so clusters receive authoritative time settings from the underlying platform without additional configuration. Consistent time synchronization across VKS cluster nodes is a requirement for correct Kubernetes operation: certificate validity checks, service account token expiration, audit log timestamp accuracy, and workload scheduling behaviors all depend on coordinated clocks. Clock skew between control plane and worker nodes can cause operational issues, including DaemonSet controllers misreading rollout progress when ``spec.minReadySeconds`` is configured. Auditors evaluating a VKS environment should verify that ``osConfiguration.ntp.servers`` is configured in cluster definitions, or confirm that the Supervisor is configured with authoritative NTP sources that VKS clusters inherit by default.

For workloads requiring higher precision than NTP provides, VCF supports the Precision Time Protocol (PTP) at the hypervisor level. PTP is a high-precision time protocol for synchronization used in measurement and control systems on a local area network, with accuracy in the sub-microsecond range achievable with low-cost implementations. The HostDateTimeSystem supports both NTP and PTP configuration. A precision clock device can be added to a virtual machine class to provide VMs with access to the system time of the primary ESX host, enabling PTP time passthrough to guest workloads. The Precision Clock device supports time synchronization protocols including Any, NTP, and PTP, giving administrators flexibility to match the synchronization method to guest workload requirements.

### **VCF Protection and Recovery (Meets)**

VCF Protection and Recovery (PNR) provides native time synchronization mechanisms within the Protection and Recovery Appliance. The appliance supports three configurable time synchronization modes: NTP, which enables direct synchronization against administrator-specified NTP server addresses; Host, which synchronizes with the underlying ESX host time via VMware Tools; and Disabled. In NTP mode, administrators configure the appliance to synchronize directly with the organization's authoritative time source, independent of the underlying VCF platform's host-level NTP configuration.

PNR enforces clock alignment across its server components through the maximum-clock-skew setting, which defines the maximum allowed time difference between server clocks (default: 20 seconds). Time zone and time synchronization settings are editable through the appliance configuration interface when network time settings change.

For VCF Automation environments, the Protection and Recovery Proxy also requires explicit NTP server configuration, specified as an IP address or FQDN. PNR deployment guidance recommends using a persistent synchronization agent such as network time protocol daemon (NTPD), W32Time, or VMware Tools time synchronization to maintain consistent time across recovery infrastructure and avoid conflicts in time management across systems.

### **VMware Data Services Manager (Meets)**

VMware Data Services Manager (DSM) requires configuration of a Network Time Protocol (NTP) service with a reliable time source to synchronize the clocks of all deployed VMs. This requirement is documented as a deployment prerequisite in the DSM installation guide. The Provider Appliance and all managed database instances must use the same NTP server, enforcing clock uniformity across the entire DSM environment.

NTP server configuration is managed through the DSM System Settings interface. The Provider VM's System Settings panel includes an NTP Servers field where administrators specify one or more NTP servers as a list of IP addresses or fully qualified domain names. DSM applies NTP settings from the system configuration to synchronize database time with system time, helping maintain consistent timekeeping across the Provider Appliance and all managed database VMs.

DSM monitors time synchronization health through its global alerts system. When Provider NTP synchronization fails, DSM generates an `NTP_SYNC_FAILED` alert. At the database level, DSM generates an `NTP_NOT_APPLIED` alert when NTP settings from the system configuration have not been applied to a database instance, indicating that the system is not synchronizing database time. NTP synchronization failures can result in authentication failures, backup and restore corruptions, and certificate errors, which is why DSM enforces NTP synchronization as a deployment prerequisite rather than a recommendation. Administrators can configure webhooks to receive notification of these alerts. When `NTP_SYNC_FAILED` is triggered, DSM documentation directs administrators to verify and correct any invalid entries in the NTP Servers field in System Settings.



DSM uses Coordinated Universal Time (UTC) for system-generated timestamps, including timestamps on downloaded support bundles. Maintenance Policy schedules for software updates also require start time specification in UTC, reinforcing consistent time reference across operational activities.

DSM does not provision or manage the authoritative time source itself, which is standard for any product that synchronizes to an external NTP server. Organizations establish and maintain the NTP infrastructure that DSM references.

#### **VMware Avi Load Balancer (Meets)**

VMware Avi Load Balancer (Avi) provides built-in NTP synchronization for both the Controller and Service Engines, covering the full application delivery stack without requiring external time-management tooling.

The Avi Controller is configured with a list of NTP servers through the DNS/NTP Settings section of Controller Settings, using the `ntp_configuration.ntp_servers` parameter. Multiple servers can be specified to provide redundancy. Service Engines synchronize time with the Avi Controller as part of normal operation, so time consistency flows from the authoritative NTP source through the Controller to every Service Engine in the deployment. Service Engines can also synchronize directly with an NTP server configured at the cloud level, rather than through the Controller hierarchy.

For deployments where Service Engines need direct access to NTP infrastructure, Avi supports configuring NTP servers directly on Service Engines for both virtual machine and LSC-based (bare metal) deployments via the `ntp-servers` setting. Service Engines synchronize time with configured NTP servers at startup and periodically monitor time synchronization status thereafter. The platform supports `ntpd` (Network Time Protocol daemon) and `chronyd` as NTP daemon implementations on Service Engines.

Avi provides operational visibility into time synchronization health. Service Engines periodically verify that configured NTP daemons can acquire and synchronize time with the configured servers, and raise an event if synchronization fails. This built-in monitoring means time sync failures surface in the Avi event log rather than going undetected. Accurate time synchronization is also required for correct operation of Avi's logging and analytics features.

In Avi for VCF deployments, NTP server addresses can be configured for the environment when deploying a vSphere Supervisor with Avi, covering the integrated VCF deployment pattern.

Not applicable for this control: VMware vDefend. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 2.11(2)

> Timestamps: Capability to include timestamps in audit records. Time synchronization: Capability to synchronize the component clock with an NTP source. Time synchronization protection: Protect the time synchronization process from tampering.

**SCF Controls:** SEA-20

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides time synchronization across all platform components using Network Time Protocol (NTP), with Precision Time Protocol (PTP) available for workloads requiring sub-microsecond accuracy.

At the foundation layer, NTP service must be configured on all VMware ESX hosts. This is configured through the VMware Host Client by navigating to Manage > System tab > Time & date > Edit NTP Settings. The NTP client must be enabled by selecting the "Use Network Time Protocol" radio button, and the NTP service startup policy should be set to "Start and stop with host" so that time synchronization is maintained across host reboots. Using at least three NTP servers is a best practice for ESX host time synchronization, providing resilience against any single time source becoming unavailable. VMware Knowledge Base article 317537 provides detailed guidance on ESX host NTP configuration, and article 318888 provides procedures for verifying time synchronization across ESX host environments. ESX also supports PTP configuration through the HostDateTimeSystem, giving administrators a choice between NTP and PTP depending on the precision requirements of the environment.

VMware vCenter must also be configured with NTP to maintain clock synchronization with managed ESX hosts. Time synchronization between vCenter and ESX hosts is a stated requirement for VMware Infrastructure operation. Consistent time across these components is critical for accurate logging, troubleshooting, and event correlation. The NTP daemon should be activated on ESX hosts for this reason. For environments running vSphere Supervisor, the same NTP servers must be configured for the vCenter instance, all hosts in the cluster, NSX, and the Supervisor.

VMware Kubernetes Service (VKS) clusters running on vSphere Supervisor extend NTP synchronization to Kubernetes control-plane and worker nodes through the ClusterClass variable framework. When provisioning VKS clusters using the Cluster v1beta1/v1beta2 API, administrators specify NTP servers in the `osConfiguration.ntp.servers` parameter (VKS 3.2.0 and later) or in the `ntp cluster` variable (VKS 3.1.0 and earlier); both parameters accept an array of NTP server hostnames or IP addresses. If left unset, VKS nodes inherit default NTP server addresses from the Supervisor, which in turn derives its time configuration from the underlying infrastructure. Post-creation changes to NTP server configuration require manually updating the ClusterClass cluster variable, as NTP changes are not automatically propagated to running clusters; administrators should account for this in change management processes. Clock accuracy within VKS clusters is operationally significant: clock skew between control-plane and worker nodes can cause DaemonSet rollout progress to be reported incorrectly when `spec.minReadySeconds` is in use, and correct timestamps are required for Kubernetes audit log reliability, certificate validity checks, and bootstrap token expiry enforcement.

VCF Operations is the central point for NTP configuration across VCF core components. When you update the NTP server configuration through VCF Operations, it propagates the change to VCF Operations itself, all vCenter instances, ESX hosts, NSX Manager nodes, and NSX Edge nodes. If an NTP update fails for any component, VCF Operations rolls back the NTP settings for the failed component. NTP configuration changes go through a precheck validation step before deployment. VCF management components can also be configured to use the ESX NTP server for time synchronization through the `time-sync-mode` setting, allowing centralized time management where management components inherit their time from the underlying ESX infrastructure.

VCF Operations supports two NTP synchronization modes: self-managed synchronization, where all nodes synchronize with the primary and replica node, or external NTP server synchronization using a URL or IP address. The NTP setting synchronizes time across VCF Operations cluster nodes. When configuring VCF SSO integration, NTP should be configured to synchronize

the server system time of the identity source and VCF Operations. The `ntp sync` command is available to force synchronization of NTP settings within VCF Operations.

NSX Manager appliances require NTP server configuration. NSX recommends using a node profile to configure NTP for all appliances rather than configuring each one individually. If NTP is configured individually on an NSX appliance, the same NTP server must be used on all appliances to maintain consistency. NTP is recommended for all NSX environment components regardless of which NSX features are in use.

VCF Operations for Networks depends on clock synchronization with NTP for service health; if the system clock drifts out of sync, some services may become unhealthy or stop working. HCX similarly requires a valid NTP server with synchronized time for integrated systems operations.

VCF provides built-in tools for validating time synchronization status. The SoS (Suite of Services) Utility offers an `--ntp-health` option that verifies whether the time on VCF components is synchronized with the NTP server configured in the VCF Installer appliance, allowing administrators to validate synchronization without relying on external tools. VCF Health Diagnostics also monitors NTP configuration data across VCF components and provides per-component NTP status information.

Accurate time synchronization across the platform is a prerequisite for TLS and VCF SSO communication, Kerberos authentication (which fails on clock skew), certificate validation, and consistent log timestamps for auditing and troubleshooting.

For workloads requiring higher precision than NTP provides, vSphere supports a precision clock device that gives a virtual machine access to the system time of the primary ESX host. PTP is also available for environments requiring sub-microsecond accuracy on a local area network. When an ESX host is configured for PTP, the NTP service must not be running concurrently; NTP and PTP are mutually exclusive at the host level.

#### **VMware vDefend (Contributes)**

VMware vDefend both depends on and validates time synchronization across its enforcement and management components. Accurate clock synchronization is a prerequisite for time-based firewall policy enforcement, log correlation, certificate validation, and secure communications throughout the vDefend stack.

vDefend (DFW) and vDefend (GFW) support time-based firewall policies that allow administrators to define rules that activate or deactivate based on configurable time windows. Network Time Protocol (NTP) service must be running on each transport node when using time-based rule publishing in either the Distributed Firewall or the Gateway Firewall. Time windows for both can be configured to use either Coordinated Universal Time (UTC) or the local time of the transport node. Gateway Firewall time windows support either a weekly or one-time frequency. When local time zones are used for DFW policies, the entire time window must fall within the same UTC calendar day. For Edge transport nodes, if the time zone is changed after the node is deployed, the Edge node must be reloaded or the data plane service restarted for the new time zone to take effect.

Identity Firewall, which enforces access policies based on Active Directory user identity, has additional time synchronization requirements. When using Identity Firewall event log scraping to track user login events, NTP must be correctly configured across all participating devices. Administrators should schedule Active Directory full synchronization during off-peak or non-business hours, as a full sync can take considerable time.

Security Services Platform (SSP), the management and orchestration layer for vDefend, requires accurate time synchronization for certificate validation, log correlation, and secure communications to function correctly. The SSP installer validates time synchronization as part of its deployment prerequisites and requires configuration of one or more NTP servers, which can be updated if they become unreachable. For bare metal deployments, the bare metal server should be configured to use the same NTP server as SSP to maintain consistent time synchronization across all components.

Network Detection and Response (NDR) sensor appliances require NTP to be configured and synchronizing before they can successfully register with SSP. Administrators should verify NTP synchronization status using the `get clock` command on the NDR sensor CLI before initiating registration. The NDR sensor monitoring view within SSP displays the NTP servers configured for each sensor, providing visibility into time synchronization configuration across deployed sensors.

SSP Segmentation Monitoring can monitor NTP infrastructure services on UDP/123, providing visibility into time synchronization infrastructure health alongside other monitored services. SSP Inventory tracks environment synchronization status and identifies environments that are timed out or out of sync, supporting troubleshooting of time-related connectivity issues.

Accurate timestamps across all vDefend components are necessary for correlating security events across the Distributed Firewall, IDS/IPS, malware prevention, and NDR capabilities. When investigating a security incident, analysts rely on consistent timestamps to reconstruct the sequence of events across multiple enforcement points.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) integrates NTP synchronization as a core operational requirement across all of its managed components. DSM requires configuration of a Network Time Protocol (NTP) service with a reliable time source to synchronize the clocks of all deployed VMs. Administrators configure NTP servers through the System Settings interface in the DSM UI, and DSM mandates that the Provider Appliance and all managed database instances use the same NTP server. Both the Provider Appliance and database cluster nodes communicate with NTP servers over UDP on port 123.

DSM enforces UTC as the standard time representation for maintenance policies and support bundle timestamps, reducing ambiguity in scheduled operations and log interpretation. DSM MySQL instances support configurable time zone settings, with UTC as the default value.

DSM monitors synchronization health at two levels. When NTP synchronization fails on the Provider Appliance, DSM generates an `NTP_SYNC_FAILED` global alert; administrators can resolve this by verifying and correcting the NTP servers field in System Settings. DSM also generates an `NTP_NOT_APPLIED` alert when NTP settings from the system configuration do not apply to a database instance, indicating that the database time is not being refreshed or synchronized. Together, these alerts surface clock synchronization failures within DSM's scope and make them actionable rather than silent.

DSM does not control clock synchronization for the broader infrastructure on which it runs, including ESX hosts, VMware vCenter, and network devices. Organizations must configure and maintain NTP synchronization for those layers independently through vCenter and ESX host configuration.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) maintains synchronized clocks across its Controller and Service Engine components through a configurable NTP stack. Administrators specify NTP servers in the Controller system configuration via the `ntp_configuration.ntp_servers` setting, accessible through the Controller UI, API, or CLI. The Controller synchronizes its own clock against the configured NTP servers, and Avi Service Engines in turn synchronize their clocks with the Controller by default, or directly with an NTP server configured at the cloud level. In Avi for VCF deployments, public NTP servers are used by default, making explicit NTP configuration optional when public NTP access is available; in most enterprise environments, administrators should configure the organization's approved NTP servers explicitly.

For VM-deployed Service Engines, NTP servers can also be configured directly at the Service Engine level via the `ntp-servers` setting, enabling them to synchronize independently with NTP infrastructure where needed. For bare metal (LSC) deployments, Service Engine time synchronization supports `ntpd` (Network Time Protocol daemon) and `chronyd` as NTP daemon implementations, selectable via the `ntp_daemon` setting. NTP traffic uses UDP port 123 on the management interface.

Avi monitors time synchronization health continuously. Service Engines synchronize with configured NTP servers at startup and periodically monitor synchronization status. When synchronization fails, Avi raises an event until synchronization is restored, providing administrators with ongoing visibility into the time synchronization status of the load balancing infrastructure.

One configuration requirement applies to VM deployments: when Avi Controller and Service Engine VMs run on VMware ESX hosts, ESX host time synchronization must be disabled for those VMs. Allowing both the hypervisor VMware Tools time sync mechanism and the in-guest NTP daemon to operate simultaneously causes clock conflicts. The Avi documentation requires

that ESX time synchronization be disabled on all Avi Controller and Service Engine virtual machines so that NTP is the sole time authority for those VMs.

In Avi for VCF deployments, NTP server addresses can be configured when deploying a vSphere Supervisor with the Avi Load Balancer, supporting consistent time synchronization across the VCF environment.

Administrators responsible for this control should configure the `ntp_configuration.ntp_servers` field in Avi Controller system settings with the organization's approved NTP servers, verify that Service Engines are synchronized, and confirm that ESX time synchronization (VMware Tools) is disabled for the relevant VMs.

Not applicable for this control: VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 2.12

> Non-repudiation: Capability to provide evidence that a user performed a specific action. Non-repudiation for digital signatures: Use of digital signatures for non-repudiation of origin/receipt.

**SCF Controls:** MON-09

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides non-repudiation through audit logging that ties every administrative action to an authenticated identity, combined with role-based access controls and public key infrastructure (PKI) that verify and constrain user actions across the platform.

All management operations across VCF components require authenticated sessions with appropriate privileges. VMware vCenter enforces privilege-based access for sensitive operations, including cryptographic tasks, where users who perform encryption-related actions must hold the specific privileges required for those operations. Administrators can create custom roles that grant only a subset of cryptographic operations privileges, such as allowing encryption but not decryption, so that each action is attributable to a user with a narrowly scoped set of permissions. This privilege model extends across the platform: VCF Automation, NSX, and VCF Operations all enforce authenticated, privilege-checked access for every management action.

VCF Operations records audit events for platform interactions including logins, logouts, capability checks, and configuration modifications. Each event is attributed to a specific user, and the user activity audit reports when users logged in, what actions they took on clusters and nodes, what changes they made to system passwords, when they activated certificates, and when they logged out. Audit logs can be filtered by User ID, User Name, Auth Source, Session, Client IP, Category, and Message, providing multiple dimensions for tracing actions back to individuals. VCF Operations also collects and displays vCenter authentication logs, including successful and failed logins and user authentication details over time.

VCF Operations demonstrates fine-grained action ownership tracking through tiered permissions. Only admin users can delete incidents created by other users, while member users can delete only the incidents they themselves created. Audit users cannot delete or create any incidents. This permission model ties record lifecycle actions directly to the identity of the user who performed them, providing an accountability chain for operational records.

NSX Manager generates audit logs for all API calls, CLI commands, and UI access, with audit logging enabled by default and not configurable to be disabled. Each audit log entry includes metadata fields such as audit flag, component name, entity ID, request ID, log level, subcomponent, update flag, and username, which together link every network management action to the user who performed it. NSX also records user operations including UserName, ModuleName, Operation, and Operation status with timestamps and request identifiers. Role binding changes for remote user assignments are logged with timestamp, component identifier, audit flag, user identity, module name, operation name, operation status, and role binding details. NSX audit logs can be accessed programmatically through the /api/v1/administration/audit-logs API endpoint, and an audit log associated with an API call includes a request ID parameter (req-id) to identify a specific API call.

VCF Automation creates audit logs for reportable actions such as deploying cloud templates or catalog items, performing Day 2 actions on resources and deployments, and creating, deleting, or updating property groups and policies (including Day 2 actions, lease, content sharing, approval, deployment limit, and resource quota policies). Each log entry includes a timestamp, message, user name, project name, and trace ID value. Organization Administrator and Organization Auditor roles can view these audit logs.

VMware Kubernetes Service (VKS) and vSphere Supervisor extend non-repudiation to the workload consumption layer. ValidatingAdmissionPolicyBinding resources can be configured with a validationActions value of Audit, which causes the Kubernetes API server to record an audit event for every policy evaluation attempt, capturing the requesting principal's identity, the resource targeted, and the policy outcome. When the Deny action is combined with Audit, both the enforcement decision and the corresponding audit record are generated for the same request, creating a linked account of what was attempted, by whom, and what the result was. Operators configure these bindings to control which resource operations and

policy evaluations generate audit records, giving platform administrators precise control over the scope of the audit trail. VKS Cluster Management Security Policies extend this pattern to managed clusters: the default Policy enforcement action is Deny, which blocks admission requests with violations; a Dry run option generates policy evaluation records without blocking, supporting audit-only observation of policy compliance. Pod Security Admission (PSA) in Supervisor provides an additional audit channel for workload operations. PSA supports enforce, audit, and warn modes per namespace; in audit mode, security policy violations are recorded as audit annotations without blocking the workload, generating a record of every pod that violated the applicable security profile and the identity of the principal that submitted it. The default VKS cluster configuration sets the PodSecurityStandard audit level to privileged and the enforce level to restricted, so security profile violations are always captured in the audit record even when the pod is admitted. For high-assurance deployments, Kubernetes guidance recommends applying cryptographic protections to audit logs to make them tamper-proof and confidential; organizations requiring non-repudiation as a defensible claim should forward VKS and Supervisor audit logs to an external, append-only system.

The platform's PKI infrastructure strengthens non-repudiation through certificate-based authentication. VCF supports Online Certificate Status Protocol (OCSP) responders that verify a user's private key has not been compromised or revoked, and maintains Certificate Revocation Lists (CRLs) to identify certificates that should no longer be trusted. VCF can detect when a certificate has been revoked by the issuing certificate authority. These mechanisms help confirm that a presented identity was valid at the time an action occurred, making it harder for an individual to claim their credentials were invalid or compromised after the fact.

Audit logs from VCF Operations can be sent to an external syslog server to meet security auditing requirements, with configurable syslog server IP address or hostname and port. VCF Operations for Networks integrates its audit log with syslog, and the syslog collector can be configured to collect all audit logs. Each syslog and audit log message contains a timestamp generated by an NTP server when configured. vCenter stores events in its internal database for a configurable retention period, after which they are deleted; auditors should confirm that external log forwarding is configured before this retention window closes to preserve the complete audit trail. This forwarding capability supports long-term retention and tamper-evident storage on external log management platforms, further strengthening non-repudiation by preserving an independent record of actions outside the platform itself.

VMware Private AI Services (PAIS) contributes to non-repudiation through identity binding, audit-relevant observability, and integrity-anchored model storage that together create records associating AI workload operations with the identities that performed them.

PAIS access is bound to authenticated user identities through OIDC group-based authorization. The `authorizedGroups` field in the `PAISConfiguration` custom resource restricts service access to named OIDC groups, so AI service operations are attributable to authenticated users rather than anonymous service callers. HTTPS-based communication with the OIDC provider is required, and PAIS uses a configurable CA trust bundle for secure communication with external services and databases, helping protect the authentication exchange against tampering or impersonation.

For AI operation tracing and metrics, PAIS supports sending traces to an OpenTelemetry collector over HTTP/protobuf or gRPC, configured through the observability parameters in the `PAISConfiguration` custom resource. The `observability.prometheusRuntime` setting activates a Prometheus runtime with a configurable storage class and a metrics retention period that defaults to 14 days. These records associate inference and LLM operations with the identities that initiated them and provide an audit-relevant trail at the AI application layer.

For models themselves, the PAIS Model Gallery anchors non-repudiation of ML model provenance. Model revisions are stored as immutable OCI artifacts identified by content digest, with deduplication so that identical content pushed multiple times resolves to a single revision. Model Gallery assigns a unique digest to each revision when the same model is pushed multiple times, and PAIS Deep Learning VMs perform model validation through hash code checking, malware scanning, serialization-attack scanning, and inference testing with Triton Inference Server. These mechanisms produce a tamper-evident record of what model content was uploaded and validated, supporting the ability to attribute model changes to a specific revision.



These PAIS-specific mechanisms supplement the broader audit logging, log forwarding, identity, and platform audit records provided by VCF, VCF SSO, and DSM. The organization must enable observability components and integrate the resulting traces and metrics with a centralized log management system to address the full intent of the control.

#### **VMware vDefend (Contributes)**

VMware vDefend contributes to non-repudiation by generating audit records for network security enforcement actions and by enabling controls that attribute those actions to authenticated identities.

The VMware vDefend distributed firewall (DFW) supports per-rule logging in multiple modes, including Application Connectivity Strategy configurations where logging is enabled for all matched traffic. When the DFW operates with a Deny List strategy and logging enabled, traffic matching the policy produces a log entry recording the enforcement action. The DFW also supports a REJECT action, producing a distinct enforcement record for blocked connections. When Identity Firewall is enabled, DFW rules can be scoped to specific user groups and identities. Identity Firewall rules support a Reject action targeting specified user groups, services, and destinations, meaning access-denial events can be attributed to authenticated user identities rather than source IP addresses alone, which strengthens attribution at the network layer.

The VMware vDefend gateway firewall similarly supports per-rule logging of matched rules, creating a parallel record of north-south enforcement actions on NSX edge nodes.

Security Intelligence, a capability within the Security Services Platform (SSP), enables administrators to review analyzed traffic flows and transition security posture from observation to enforcement by modifying allow policies to deny after confirming no additional unprotected flows exist. These policy enforcement decisions create a record of when and by whom the enforcement posture changed. SSP alarms can be acknowledged or suppressed by users; those actions are attributed to the acting user, creating an operational decision audit trail.

vDefend's Malware Prevention capability and Network Detection and Response (NDR) generate security event records and can integrate with Security Information and Event Management (SIEM) systems, providing additional channels for exporting security events into centralized logging infrastructure.

The broader VCF platform handles administrative non-repudiation for management plane operations, such as who created or modified a firewall rule. vDefend provides the corresponding data plane records: what the firewall enforced, what threats were detected, and how analysts responded. Coverage is Contributes rather than Meets because the control's full scope spans organizational processes and management plane accountability that depend on VCF infrastructure beyond vDefend's data plane enforcement.

#### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) contributes to non-repudiation through audit logging and access controls that tie recovery administrative actions to authenticated identities. All PNR operations require authentication through VCF SSO. PNR records operational information in log files that exclude sensitive data such as private keys and passwords, and generates events covering protection group operations, recovery plan execution, and licensing status changes.

PNR includes a defined set of roles with specific privileges that control what each user can perform. This role-based access model links actions to identity, supporting the attribution chain that non-repudiation requires. For high-impact operations, PNR requires explicit user confirmation before proceeding: the reprotect operation requires the user to confirm the action is irreversible before PNR proceeds, creating a documented acknowledgment tied to the authenticated user identity.

These audit events flow through the VMware vCenter event system and can be forwarded via syslog to external log management platforms for long-term retention, supporting the non-repudiation chain by preserving evidence of who performed which recovery actions and when. Organizations must configure syslog forwarding and manage log retention outside PNR to complete the chain.

#### **VMware Data Services Manager (Contributes)**

Non-repudiation requires technical mechanisms that create records linking specific actions to specific individuals, making it difficult for a user to credibly deny having performed a given operation. VMware Data Services Manager (DSM) contributes to

this capability through audit logging for managed databases and role-based access controls that tie actions to authenticated accounts.

PostgreSQL databases managed by DSM include the `pgaudit` extension, which provides detailed audit logging for database-level operations. The `pgaudit` extension records SQL statements, the roles involved, and object-level actions, creating a log trail that associates database activity with the authenticated session that initiated it. The extension is available across all supported PostgreSQL versions. Auditors should verify that `pgaudit` is enabled and that log output is captured and retained in a centralized log store.

The role-based access model in DSM supports the attribution side of non-repudiation for administrative actions. The DSM console restricts the views and actions available to each user based on their assigned role, so actions performed through the DSM interface can be associated with specific authenticated accounts. Administrative operations such as database provisioning, policy changes, and backup configuration are performed by users authenticated to DSM and are tied to those sessions.

Strong authentication reinforces the link between actions and identities. DSM supports configurable `pg_hba.conf` entries for PostgreSQL database connections, including methods such as SCRAM-SHA-256, which help establish that the account performing a given action is the legitimate account holder.

DSM does not delete audit logs stored in the S3 repository, retaining the audit trail in a form that is not subject to in-platform deletion. This supports chain-of-custody requirements by keeping historical records accessible for review and dispute resolution.

For SQL Server databases, the `dsm_db_user` role denies the `IMPERSONATE` privilege on the database owner account, which restricts users from assuming the identity and full permissions of the Database Owner and keeps actions attributed to the account that executed them.

Non-repudiation as an organizational capability requires more than technical logging within a single platform. Organizations must integrate DSM's audit logs with a centralized log management or SIEM system to provide tamper-evident storage, long-term retention, and chain-of-custody controls. DSM supports log forwarding to syslog or VCF Log Management to facilitate this integration. Policies governing log access, retention periods, and dispute resolution procedures are organizational responsibilities that lie outside the platform itself.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides mechanisms that support non-repudiation of administrative and application-layer actions.

At the administrative plane, Avi's role-based access architecture ties actions to specific authenticated identities. Configuring purpose-scoped roles, such as a dedicated SSL administration role with write access limited to SSL/TLS certificate management and a corresponding dedicated admin account, reduces the permissible action surface for each identity and strengthens audit log attribution. This practice is documented in Avi's 9.1 security guidance for App Transport Security.

At the application traffic layer, Avi's DataScript capability extends the audit record beyond administrative actions. DataScript can enforce blacklist-based access control against a named IP group, terminating connections from blocked clients and logging each violation with client IP attribution. This creates application-layer records that establish which client addresses attempted or completed access and what outcome was applied.

Avi supports syslog forwarding to connect its audit streams to external log management or SIEM platforms. Non-repudiation as a complete organizational capability requires that Avi's logs be forwarded to and preserved in a centralized, tamper-evident log management system operated by the organization. The organization is responsible for configuring the forwarding destination, setting retention periods, and protecting the integrity of stored records.

One important caveat for 9.1 deployments: Avi Pulse does not log audit records for trusted channel initiation and termination, trusted channel function failures, unsuccessful certificate validation attempts, client authentication failures, or SSH session

establishment. Organizations relying on Avi Pulse for audit data should supplement it with direct Controller log collection via syslog to capture these security-relevant events and maintain a complete non-repudiation record.

## Control CR 2.12(1)

> Non-repudiation: Capability to provide evidence that a user performed a specific action. Non-repudiation for digital signatures: Use of digital signatures for non-repudiation of origin/receipt.

**SCF Controls:** MON-09

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides non-repudiation through audit logging that ties every administrative action to an authenticated identity, combined with role-based access controls and public key infrastructure (PKI) that verify and constrain user actions across the platform.

All management operations across VCF components require authenticated sessions with appropriate privileges. VMware vCenter enforces privilege-based access for sensitive operations, including cryptographic tasks, where users who perform encryption-related actions must hold the specific privileges required for those operations. Administrators can create custom roles that grant only a subset of cryptographic operations privileges, such as allowing encryption but not decryption, so that each action is attributable to a user with a narrowly scoped set of permissions. This privilege model extends across the platform: VCF Automation, NSX, and VCF Operations all enforce authenticated, privilege-checked access for every management action.

VCF Operations records audit events for platform interactions including logins, logouts, capability checks, and configuration modifications. Each event is attributed to a specific user, and the user activity audit reports when users logged in, what actions they took on clusters and nodes, what changes they made to system passwords, when they activated certificates, and when they logged out. Audit logs can be filtered by User ID, User Name, Auth Source, Session, Client IP, Category, and Message, providing multiple dimensions for tracing actions back to individuals. VCF Operations also collects and displays vCenter authentication logs, including successful and failed logins and user authentication details over time.

VCF Operations demonstrates fine-grained action ownership tracking through tiered permissions. Only admin users can delete incidents created by other users, while member users can delete only the incidents they themselves created. Audit users cannot delete or create any incidents. This permission model ties record lifecycle actions directly to the identity of the user who performed them, providing an accountability chain for operational records.

NSX Manager generates audit logs for all API calls, CLI commands, and UI access, with audit logging enabled by default and not configurable to be disabled. Each audit log entry includes metadata fields such as audit flag, component name, entity ID, request ID, log level, subcomponent, update flag, and username, which together link every network management action to the user who performed it. NSX also records user operations including UserName, ModuleName, Operation, and Operation status with timestamps and request identifiers. Role binding changes for remote user assignments are logged with timestamp, component identifier, audit flag, user identity, module name, operation name, operation status, and role binding details. NSX audit logs can be accessed programmatically through the /api/v1/administration/audit-logs API endpoint, and an audit log associated with an API call includes a request ID parameter (req-id) to identify a specific API call.

VCF Automation creates audit logs for reportable actions such as deploying cloud templates or catalog items, performing Day 2 actions on resources and deployments, and creating, deleting, or updating property groups and policies (including Day 2 actions, lease, content sharing, approval, deployment limit, and resource quota policies). Each log entry includes a timestamp, message, user name, project name, and trace ID value. Organization Administrator and Organization Auditor roles can view these audit logs.

VMware Kubernetes Service (VKS) and vSphere Supervisor extend non-repudiation to the workload consumption layer. ValidatingAdmissionPolicyBinding resources can be configured with a validationActions value of Audit, which causes the Kubernetes API server to record an audit event for every policy evaluation attempt, capturing the requesting principal's identity, the resource targeted, and the policy outcome. When the Deny action is combined with Audit, both the enforcement decision and the corresponding audit record are generated for the same request, creating a linked account of what was attempted, by whom, and what the result was. Operators configure these bindings to control which resource operations and

policy evaluations generate audit records, giving platform administrators precise control over the scope of the audit trail. VKS Cluster Management Security Policies extend this pattern to managed clusters: the default Policy enforcement action is Deny, which blocks admission requests with violations; a Dry run option generates policy evaluation records without blocking, supporting audit-only observation of policy compliance. Pod Security Admission (PSA) in Supervisor provides an additional audit channel for workload operations. PSA supports enforce, audit, and warn modes per namespace; in audit mode, security policy violations are recorded as audit annotations without blocking the workload, generating a record of every pod that violated the applicable security profile and the identity of the principal that submitted it. The default VKS cluster configuration sets the PodSecurityStandard audit level to privileged and the enforce level to restricted, so security profile violations are always captured in the audit record even when the pod is admitted. For high-assurance deployments, Kubernetes guidance recommends applying cryptographic protections to audit logs to make them tamper-proof and confidential; organizations requiring non-repudiation as a defensible claim should forward VKS and Supervisor audit logs to an external, append-only system.

The platform's PKI infrastructure strengthens non-repudiation through certificate-based authentication. VCF supports Online Certificate Status Protocol (OCSP) responders that verify a user's private key has not been compromised or revoked, and maintains Certificate Revocation Lists (CRLs) to identify certificates that should no longer be trusted. VCF can detect when a certificate has been revoked by the issuing certificate authority. These mechanisms help confirm that a presented identity was valid at the time an action occurred, making it harder for an individual to claim their credentials were invalid or compromised after the fact.

Audit logs from VCF Operations can be sent to an external syslog server to meet security auditing requirements, with configurable syslog server IP address or hostname and port. VCF Operations for Networks integrates its audit log with syslog, and the syslog collector can be configured to collect all audit logs. Each syslog and audit log message contains a timestamp generated by an NTP server when configured. vCenter stores events in its internal database for a configurable retention period, after which they are deleted; auditors should confirm that external log forwarding is configured before this retention window closes to preserve the complete audit trail. This forwarding capability supports long-term retention and tamper-evident storage on external log management platforms, further strengthening non-repudiation by preserving an independent record of actions outside the platform itself.

VMware Private AI Services (PAIS) contributes to non-repudiation through identity binding, audit-relevant observability, and integrity-anchored model storage that together create records associating AI workload operations with the identities that performed them.

PAIS access is bound to authenticated user identities through OIDC group-based authorization. The `authorizedGroups` field in the `PAISConfiguration` custom resource restricts service access to named OIDC groups, so AI service operations are attributable to authenticated users rather than anonymous service callers. HTTPS-based communication with the OIDC provider is required, and PAIS uses a configurable CA trust bundle for secure communication with external services and databases, helping protect the authentication exchange against tampering or impersonation.

For AI operation tracing and metrics, PAIS supports sending traces to an OpenTelemetry collector over HTTP/protobuf or gRPC, configured through the observability parameters in the `PAISConfiguration` custom resource. The `observability.prometheusRuntime` setting activates a Prometheus runtime with a configurable storage class and a metrics retention period that defaults to 14 days. These records associate inference and LLM operations with the identities that initiated them and provide an audit-relevant trail at the AI application layer.

For models themselves, the PAIS Model Gallery anchors non-repudiation of ML model provenance. Model revisions are stored as immutable OCI artifacts identified by content digest, with deduplication so that identical content pushed multiple times resolves to a single revision. Model Gallery assigns a unique digest to each revision when the same model is pushed multiple times, and PAIS Deep Learning VMs perform model validation through hash code checking, malware scanning, serialization-attack scanning, and inference testing with Triton Inference Server. These mechanisms produce a tamper-evident record of what model content was uploaded and validated, supporting the ability to attribute model changes to a specific revision.

These PAIS-specific mechanisms supplement the broader audit logging, log forwarding, identity, and platform audit records provided by VCF, VCF SSO, and DSM. The organization must enable observability components and integrate the resulting traces and metrics with a centralized log management system to address the full intent of the control.

#### **VMware vDefend (Contributes)**

VMware vDefend contributes to non-repudiation by generating audit records for network security enforcement actions and by enabling controls that attribute those actions to authenticated identities.

The VMware vDefend distributed firewall (DFW) supports per-rule logging in multiple modes, including Application Connectivity Strategy configurations where logging is enabled for all matched traffic. When the DFW operates with a Deny List strategy and logging enabled, traffic matching the policy produces a log entry recording the enforcement action. The DFW also supports a REJECT action, producing a distinct enforcement record for blocked connections. When Identity Firewall is enabled, DFW rules can be scoped to specific user groups and identities. Identity Firewall rules support a Reject action targeting specified user groups, services, and destinations, meaning access-denial events can be attributed to authenticated user identities rather than source IP addresses alone, which strengthens attribution at the network layer.

The VMware vDefend gateway firewall similarly supports per-rule logging of matched rules, creating a parallel record of north-south enforcement actions on NSX edge nodes.

Security Intelligence, a capability within the Security Services Platform (SSP), enables administrators to review analyzed traffic flows and transition security posture from observation to enforcement by modifying allow policies to deny after confirming no additional unprotected flows exist. These policy enforcement decisions create a record of when and by whom the enforcement posture changed. SSP alarms can be acknowledged or suppressed by users; those actions are attributed to the acting user, creating an operational decision audit trail.

vDefend's Malware Prevention capability and Network Detection and Response (NDR) generate security event records and can integrate with Security Information and Event Management (SIEM) systems, providing additional channels for exporting security events into centralized logging infrastructure.

The broader VCF platform handles administrative non-repudiation for management plane operations, such as who created or modified a firewall rule. vDefend provides the corresponding data plane records: what the firewall enforced, what threats were detected, and how analysts responded. Coverage is Contributes rather than Meets because the control's full scope spans organizational processes and management plane accountability that depend on VCF infrastructure beyond vDefend's data plane enforcement.

#### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) contributes to non-repudiation through audit logging and access controls that tie recovery administrative actions to authenticated identities. All PNR operations require authentication through VCF SSO. PNR records operational information in log files that exclude sensitive data such as private keys and passwords, and generates events covering protection group operations, recovery plan execution, and licensing status changes.

PNR includes a defined set of roles with specific privileges that control what each user can perform. This role-based access model links actions to identity, supporting the attribution chain that non-repudiation requires. For high-impact operations, PNR requires explicit user confirmation before proceeding: the reprotect operation requires the user to confirm the action is irreversible before PNR proceeds, creating a documented acknowledgment tied to the authenticated user identity.

These audit events flow through the VMware vCenter event system and can be forwarded via syslog to external log management platforms for long-term retention, supporting the non-repudiation chain by preserving evidence of who performed which recovery actions and when. Organizations must configure syslog forwarding and manage log retention outside PNR to complete the chain.

#### **VMware Data Services Manager (Contributes)**

Non-repudiation requires technical mechanisms that create records linking specific actions to specific individuals, making it difficult for a user to credibly deny having performed a given operation. VMware Data Services Manager (DSM) contributes to

this capability through audit logging for managed databases and role-based access controls that tie actions to authenticated accounts.

PostgreSQL databases managed by DSM include the pgaudit extension, which provides detailed audit logging for database-level operations. The pgaudit extension records SQL statements, the roles involved, and object-level actions, creating a log trail that associates database activity with the authenticated session that initiated it. The extension is available across all supported PostgreSQL versions. Auditors should verify that pgaudit is enabled and that log output is captured and retained in a centralized log store.

The role-based access model in DSM supports the attribution side of non-repudiation for administrative actions. The DSM console restricts the views and actions available to each user based on their assigned role, so actions performed through the DSM interface can be associated with specific authenticated accounts. Administrative operations such as database provisioning, policy changes, and backup configuration are performed by users authenticated to DSM and are tied to those sessions.

Strong authentication reinforces the link between actions and identities. DSM supports configurable `pg_hba.conf` entries for PostgreSQL database connections, including methods such as SCRAM-SHA-256, which help establish that the account performing a given action is the legitimate account holder.

DSM does not delete audit logs stored in the S3 repository, retaining the audit trail in a form that is not subject to in-platform deletion. This supports chain-of-custody requirements by keeping historical records accessible for review and dispute resolution.

For SQL Server databases, the `dsm_db_user` role denies the `IMPERSONATE` privilege on the database owner account, which restricts users from assuming the identity and full permissions of the Database Owner and keeps actions attributed to the account that executed them.

Non-repudiation as an organizational capability requires more than technical logging within a single platform. Organizations must integrate DSM's audit logs with a centralized log management or SIEM system to provide tamper-evident storage, long-term retention, and chain-of-custody controls. DSM supports log forwarding to syslog or VCF Log Management to facilitate this integration. Policies governing log access, retention periods, and dispute resolution procedures are organizational responsibilities that lie outside the platform itself.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides mechanisms that support non-repudiation of administrative and application-layer actions.

At the administrative plane, Avi's role-based access architecture ties actions to specific authenticated identities. Configuring purpose-scoped roles, such as a dedicated SSL administration role with write access limited to SSL/TLS certificate management and a corresponding dedicated admin account, reduces the permissible action surface for each identity and strengthens audit log attribution. This practice is documented in Avi's 9.1 security guidance for App Transport Security.

At the application traffic layer, Avi's DataScript capability extends the audit record beyond administrative actions. DataScript can enforce blacklist-based access control against a named IP group, terminating connections from blocked clients and logging each violation with client IP attribution. This creates application-layer records that establish which client addresses attempted or completed access and what outcome was applied.

Avi supports syslog forwarding to connect its audit streams to external log management or SIEM platforms. Non-repudiation as a complete organizational capability requires that Avi's logs be forwarded to and preserved in a centralized, tamper-evident log management system operated by the organization. The organization is responsible for configuring the forwarding destination, setting retention periods, and protecting the integrity of stored records.

One important caveat for 9.1 deployments: Avi Pulse does not log audit records for trusted channel initiation and termination, trusted channel function failures, unsuccessful certificate validation attempts, client authentication failures, or SSH session



establishment. Organizations relying on Avi Pulse for audit data should supplement it with direct Controller log collection via syslog to capture these security-relevant events and maintain a complete non-repudiation record.

## Control CR 3.1

> Communication integrity: Components shall provide the capability to protect the integrity of data in transit. Communication integrity enhancement: Use of cryptographic mechanisms to protect integrity. Support for updates: Provide a mechanism to verify and apply security updates. Physical tamper resistance/detection: Capability to detect or resist physical tampering. Provisioning Product: Ensure secure provisioning of the product during manufacturing. Integrity of the boot process: Verify the integrity of the boot process (Secure Boot). Integrity of data at rest: Protect the integrity of sensitive data stored on the component.

**SCF Controls:** CRY-04

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides cryptographic mechanisms across its component stack to protect the integrity of data in transit. These mechanisms operate across storage cluster traffic, management plane communications, and network overlay tunnels, and rely on validated cryptographic modules throughout.

All VCF management communications use TLS, which provides integrity verification through message authentication codes (MACs) on every transmitted record. NSX Manager has modern TLS protocol versions enabled by default and older versions disabled. The Telegraf monitoring agent is configured with a minimum TLS version for its communication channels. VCF Operations supports a FIPS Security Mode that, when enabled, imposes additional algorithm and cipher prerequisites on management plane communications.

NSX, which provides the networking infrastructure for VCF, is configured to use FIPS 140-3 validated cryptographic modules by default. These modules are validated through the NIST Cryptographic Module Validation Program (CMVP). NSX Manager uses FIPS 140 approved algorithms for authentication to its cryptographic module. NSX Manager also supports activation or deactivation of cipher suites through API configuration to align with specific security requirements. For tunnel-based connectivity, the NSX L2 VPN FIPS compliance suite uses FIPS-validated authenticated encryption with Perfect Forward Secrecy enabled, protecting both confidentiality and integrity of tunneled traffic. NSX supports the Wrapped Encapsulating Security Payload (WESP) protocol, which builds on Encapsulating Security Payload (ESP) and provides integrity and authentication services for network traffic while allowing intermediate devices to determine whether data confidentiality is in use.

VCF Operations for Networks uses FIPS-validated cryptographic modules for both internal and external connections. The component uses BC-FJA (Bouncy Castle FIPS Java API) version 2.0.0 as its validated cryptographic module. FIPS mode can be enabled specifically for external connections, restricting all external communication to FIPS-validated modules. VCF Operations for Networks supports several encryption algorithms and ciphers for its data sources, allowing administrators to align cryptographic settings with organizational requirements. VCF Automation also uses the BouncyCastle FIPS 2.0.0 cryptographic module, which is compliant with FIPS 140-3.

The VMware vCenter appliance uses VMware's Boring Crypto Module version 6.0, which provides FIPS-validated AES, ECDSA, HMAC, RSA, and SHA2 algorithms, and the OpenSSL FIPS Object Module (versions 2.0.20-vmw and 3.0), which extends coverage to additional algorithms including SHA-3 and PBKDF. Starting in vSphere 9.0, all TLS profiles across the vSphere platform are FIPS compliant.

vSAN data-in-transit (DIT) encryption is a cluster-wide setting that, when enabled, encrypts all data and metadata traffic as it transits across hosts in the cluster. This includes fault tolerance traffic between cluster nodes. The setting is configurable during encryption configuration and applies uniformly to all inter-host vSAN communication. vSAN Data Services include both data-at-rest encryption and data-in-transit encryption as integrated capabilities. The Reliable Data Transport (RDT) protocol handles communication between hosts over the vSAN VMkernel ports, and enabling DIT encryption adds cryptographic integrity protection to that transport layer.

vSAN uses end-to-end checksums to verify the integrity of stored and transmitted data, confirming that each copy of a file matches the source. The system checks data validity during read and write operations, and if a checksum mismatch is detected, vSAN automatically repairs the data by overwriting the incorrect copy with correct data.

Encrypted vSphere vMotion protects the integrity and confidentiality of data transferred during VM migrations. vSphere vMotion always uses encryption when migrating encrypted virtual machines. For virtual machines that are not encrypted, the encrypted vMotion setting can be configured as Disabled, Opportunistic (use encrypted vMotion if supported, which is the default), or Required (do not migrate if encryption is unsupported). Fault Tolerance log traffic, which replicates running VM state to a secondary host, can be encrypted separately to protect that synchronization channel. Auditors should note that storage vMotion operations performed entirely within a single host, where only the datastore changes without a host transfer, are not covered by the vMotion encryption setting.

For storage protocols, vSAN File Service supports NFS4 Kerberos authentication with security options krb5, krb5i, and krb5p. The krb5i level provides authentication plus integrity checking on all NFS operations without the overhead of full data encryption. The krb5p level adds encryption on top of integrity checking. Both Kerberos-authenticated and system-authenticated NFS4 mounts are supported on the same share, giving administrators flexibility to apply integrity verification where needed. For iSCSI storage connections, VMware ESX supports configurable header and data digests that verify data integrity at the iSCSI layer, beyond the checks that TCP and Ethernet already provide.

VCF Operations monitors transmission integrity through metrics that track frames transmitted or received with bad checksums (CRC values), which indicate data corrupted during transmission. Frames with bad checksums are dropped and not propagated to other ports, which limits the spread of corrupted data on the network.

VCF Operations orchestrator signs packages and encrypts them for data protection, providing integrity assurance for software distribution across the environment. Package signing certificates provide digital identification for encrypted communication and sign packages. Packages exported from a VCF Operations orchestrator server are digitally signed, and the recommended practice is to use certificates signed by a Certificate Authority rather than self-signed certificates.

ESX hosts generate high-quality entropy that supports security-related operations such as generating encryption keys for secure communication over the network. This entropy source underpins the strength of cryptographic keys used across the platform for transmission protection.

The VMware Kubernetes Service (VKS) clusters managed by Supervisor extend these protections to the workload consumption layer. Kubernetes enforces TLS across all control plane communications as a structural requirement of its architecture: connections between the API server and etcd, between the API server and kubelets, and between the API server and cluster clients are TLS-protected by default. Cipher suite selection and minimum protocol version are configurable at multiple points in the control plane. The kube-apiserver accepts a `--tls-cipher-suites` parameter that restricts the server to a specified list of cipher suites, and a `--tls-min-version` parameter that sets the minimum acceptable protocol version. The kubelet exposes equivalent settings through `tlsCipherSuites`, which controls which cipher suites are available for HTTPS connections, and `tlsMinVersion`, which sets the minimum acceptable version. For the kube-scheduler, the `--tls-cipher-suites` parameter restricts cipher negotiation to approved suites. Antrea, the CNI plugin in Supervisor environments, also supports configurable TLS cipher suites for its inter-component communications. The vSphere Cloud Provider Interface (CPI), which handles the integration between VKS clusters and the underlying vSphere infrastructure, supports a configurable set of TLS cipher suites for its API communications with vSphere. VKS on Supervisor ships with managed TLS certificates that underpin the cryptographic infrastructure for these connections.

At the workload ingress layer, the Contour ingress controller package exposes `tls.minimum-protocol-version` and `tls.maximum-protocol-version` parameters in its configuration file, giving administrators control over the protocol floor for inbound traffic to applications running on VKS clusters. For service-to-service communication within a cluster, the Istio service mesh standard package operates in mutual TLS (mTLS) mode, requiring both parties to authenticate and encrypting inter-service traffic. The minimum protocol version for Istio mTLS is configurable via `meshConfig.meshMTLS.minProtocolVersion` and defaults to TLS 1.2, with TLS 1.3 also available as an option. Environments with elevated cryptographic requirements can activate FIPS mode at the cluster level through the `ClusterClass` variable `fips.enabled`, which restricts the cryptographic

algorithms available to those meeting Federal Information Processing Standards; the Istio standard package also provides FIPS compliance as part of its standard configuration. Harbor, the container image registry available as a Supervisor service, secures registry traffic through TLS certificate configuration via the `tlsCertificate.tls.crt` and `tlsCertificate.tls.key` fields. The Supervisor secret injection agent automatically configures mutual TLS for communication with the vault-agent-injector service using auto-generated certificates.

Defining the integrity-protection policy for customer workload traffic, validating that customer application transmissions use authenticated encryption, and setting the integrity policy for workload-to-workload traffic running in VKS pods are organizational data-policy decisions.

VMware Private AI Services (PAIS) applies TLS to its AI service communication paths through CA trust bundle management and client TLS configuration in the PAISConfiguration custom resource. The `spec.clientTls.caBundleRefs` setting references ConfigMap resources that hold CA trust bundles for outbound TLS to the OIDC provider and Harbor container registry, with separate bundles recommended per application and periodic updates required before certificate expiration. PAIS endpoints use self-signed certificate authentication; the CA certificate is stored in the `pais-ingress-default` Kubernetes Secret in the Private AI Services namespace, and administrators retrieve it with `kubectl get secrets/pais-ingress-default -o yaml` to distribute to client services.

PAIS requires HTTPS trust bundles for every external integration point: OIDC providers, Harbor registries, content management systems used as Knowledge Base data sources, MCP servers that supply tools to models, and the pgvector database connection through Data Services Manager (DSM). CA trust bundles are imported as certificate files or ConfigMap references, and dedicated settings exist for each integration class, including `ca-trust-bundle-for-dsm` for the pgvector database path. When connecting an MCP server, the issuer certificate must be added as a CA trust bundle before the HTTPS connection can be established. The PAIS Supervisor endpoint communicates over HTTPS on port 6443 with certificate-based authentication, and the Prometheus metrics endpoint uses TLS with the same self-signed certificate model retrieved from the `pais-ingress-default` Secret.

GPU-accelerated VMware Kubernetes Service (VKS) clusters provisioned through PAIS run a FIPS-enabled VKR build (v1.33.1-vmware.1-fips-vkr.2 on Ubuntu 24.04), which provides FIPS-compliant cryptographic modules for control plane and worker node communication. JWT-based authentication for VCF Automation integration uses the `TM_CERT_BASE64` credential captured by running `openssl s_client` against the VCF Automation FQDN on port 443 and encoding the certificate in base64. The VCF Consumption CLI supports optional CA certificate validation through the `--ca-certificate` option when establishing context connections to VCF Automation endpoints. The Model Gallery assigns a unique digest to each model revision when the same model is pushed multiple times, which helps administrators detect tampering or unintended overwrites during artifact retrieval from Harbor.

Administrators should note that TLS client authentication and certificate validation on the Prometheus endpoint are turned off by default and must be explicitly enabled through the `tls-client-authentication` setting in PAISConfiguration to activate mutual certificate validation.

### VMware vDefend (Contributes)

VMware vDefend contributes to transmission integrity monitoring through several complementary mechanism clusters. The IDS/IPS engine, available in both Distributed and Gateway profiles, includes signature rules that match on TCP protocol fields, including flags, acknowledgement numbers, sequence numbers, and window sizes, to detect protocol anomalies and tampering indicators in network traffic. Custom IDS/IPS signature rules also support TLS-specific matching and can store TLS/SSL certificate artifacts to disk for forensic analysis and threat investigation. The IDS/IPS engine detects obsolete or integrity-deficient protocols by combining port verification with application identification, flagging traffic that uses older SSL and TLS versions that predate current security standards, early SMB versions that lack message signing, and cleartext protocols such as telnet.

The Security Services Platform's Network Detection and Response capability supports SSL verification for event logs sent over HTTPS, protecting the integrity of telemetry data in transit between NDR sensors and the platform. The Security Services

Platform's security segmentation analysis identifies risky application protocols that transport data without adequate integrity protection and provides guidance on restricting or blocking them through vDefend policy.

The Gateway Firewall's TLS Inspection feature operates as a TLS proxy to transparently intercept and inspect encrypted traffic for threats while retaining end-to-end encryption for compliant sessions. Administrators can configure Crypto Enforcement on TLS Inspection profiles to require that client and server connections meet defined minimum cipher suite requirements, providing a policy-enforcement mechanism over the cryptographic strength of inspected flows.

The Security Services Platform establishes external communications using TLS handshake negotiation and rejects connections that use deprecated cryptographic algorithms, weak key lengths, or older protocol versions. The SSP Installer and NDR Sensor components use FIPS 140-2 and FIPS 140-3 validated cryptographic modules for their own internal communications, including the VMware BouncyCastle Module and the VMware OpenSSL FIPS provider Object Module.

vDefend does not itself provide cryptographic integrity mechanisms such as message authentication codes or digital signatures for data in transit. Those protections are the responsibility of the applications and transport protocols in use and fall outside vDefend's scope.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) applies cryptographic mechanisms to protect the integrity of data in transit across management communications and replication data flows.

All management and API communications to the PNR server use TLS, which provides integrity verification through message authentication on each transmitted record. PNR requires an SSL/TLS endpoint certificate for every TLS connection to the PNR server; certificates must be server certificates with x509v3 Extended Key Usage indicating TLS Web Server Authentication. PNR uses TLS certificates and private keys to protect network communication and establish authentication with connected services. Administrators can configure the minimum TLS protocol version through envoy proxy settings by editing the `tls_minimum_protocol_version` parameter in the `envoy-proxy-config.json` file, allowing organizations to enforce a higher minimum where policy requires. When the minimum version is changed, PNR requires the setting to be updated consistently across all system components.

Replication traffic between sites supports optional network encryption. vSphere Replication supports enabling data encryption for replication traffic on a per-replication basis. When replicating encrypted virtual machines using array-based or vSphere Replication protection groups, PNR requires that the recovery and protected sites share a common Key Management Server (KMS) or that KMS clusters at both sites use common encryption keys, so that replicated data maintains its encryption across sites.

VCF adds platform-layer integrity protections, including vSAN data-in-transit encryption for inter-host storage traffic.

Selecting which data flows receive network encryption and managing the certificate and key management infrastructure are organizational data-policy decisions.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) applies cryptographic protection to data in transit across its major communication paths, using TLS to protect both the confidentiality and integrity of transmitted data.

DSM enforces FIPS 140-3 compliant cryptographic standards for certificate handling. The platform validates certificate algorithms at upload time, rejecting keys or signatures that do not conform to FIPS 140-3 requirements. These requirements apply to trusted root certificates and to custom certificates used during database version upgrades.

Database connections from clients to DSM-managed PostgreSQL, MySQL, and SQL Server clusters use TLS. PostgreSQL instances enforce a minimum TLS protocol version through the `'ssl_min_protocol_version'` setting. PostgreSQL supports TLS-encrypted connections with certificate-based authentication and CA certificate verification. MySQL enforces TLS on client connections, and FIPS mode on the MySQL client side can be activated through the `'ssl_fips_mode'` setting. DSM also manages SQL Server clusters using TLS for client connections; the `'network.tlsProtocols'` setting defines the allowed protocol versions for those connections. DSM validates SSL trust during MySQL replication operations.

DSM's certificate management supports custom certificates for database clusters. Administrators can configure database cluster certificates through the DSM UI or via Kubernetes cert-manager, which outputs certificates as Kubernetes TLS Secrets. DSM validates certificate algorithms at upload time and during version upgrades to confirm FIPS 140-3 compliance. The DSM API provides endpoints for retrieving the certificate authority needed to establish secure connections to individual database clusters.

Communication between DSM components is also secured with TLS. The Consumption Operator requires a TLS certificate for communication with the DSM provider. DSM log forwarding supports Auto TLS configuration to secure connections to the SMTP server.

For PostgreSQL data migration, DSM replication connections support SSL/TLS with a configurable SSL mode parameter to secure data in transit between source and target databases.

To fully satisfy this control, organizations should verify that clients connecting to DSM databases are configured to use the TLS connection path and that CA certificates are properly deployed to client trust stores. DSM's internal and external communication paths rely on this client-side configuration for the full TLS chain to be effective.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides TLS termination and re-encryption as a core function of its Service Engine data plane, giving administrators direct control over the cryptographic mechanisms applied to application traffic in transit. The SSL/TLS Profile, configured under Templates > Security > SSL/TLS Profile, defines the accepted protocol versions and the prioritized list of cipher suites for each virtual service. Built-in profile templates provide starting configurations, and administrators can create custom profiles that restrict accepted cipher suites to those using authenticated encryption. The Service Engine completes the TLS handshake for both client-to-Avi and Avi-to-backend sessions, so traffic passes through a cryptographically protected channel on each leg. The SSL Profile also supports post-quantum cryptography cipher suites, providing an option for deployments that need to adopt algorithms addressing emerging cryptographic requirements.

For deployments requiring validated cryptographic modules, Avi supports a FIPS mode in which only FIPS-compliant algorithms are used for all cryptographic operations. When FIPS mode is enabled on a Controller cluster, the Controller and Service Engines use the OpenSSL FIPS Provider, which has been validated at FIPS 140-3 Security Level 1 and awarded CMVP Certificate #4985, for both data-plane TLS sessions and management-plane communications between Avi components.

Cryptographic key material backing TLS certificates can be stored in Thales Luna Hardware Security Modules (HSMs) integrated with both the Avi Controller and Service Engines. Service Engine HSM groups, defined under Templates > Security > HSM Groups, allow Service Engines to authenticate with HSMs for secure storage of SSL certificate private keys. TLS session persistence data shared between Service Engines in a cluster is transmitted in encrypted form, maintaining the integrity of session routing state across the data plane. Configuration backup archives transferred to remote destinations use Secure Copy Protocol (SCP) or Secure File Transfer Protocol (SFTP). Intel QuickAssist Technology (QAT) acceleration is available on Service Engines for bulk cryptographic operations following TLS handshake completion.

For Kubernetes workloads, the Avi Kubernetes Operator (AKO) supports PKIProfile custom resources, which enable certificate-based authentication and encryption for backend service communication. The System-Standard SSL Profile applied through AKO provides TLS re-encryption for traffic to backend servers.

Defining which application traffic requires integrity protection and configuring the virtual service profiles accordingly are organizational data-policy decisions.

## Control CR 3.1(1)

> Communication integrity: Components shall provide the capability to protect the integrity of data in transit. Communication integrity enhancement: Use of cryptographic mechanisms to protect integrity. Support for updates: Provide a mechanism to verify and apply security updates. Physical tamper resistance/detection: Capability to detect or resist physical tampering. Provisioning Product: Ensure secure provisioning of the product during manufacturing. Integrity of the boot process: Verify the integrity of the boot process (Secure Boot). Integrity of data at rest: Protect the integrity of sensitive data stored on the component.

**SCF Controls:** CRY-04

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides cryptographic mechanisms across its component stack to protect the integrity of data in transit. These mechanisms operate across storage cluster traffic, management plane communications, and network overlay tunnels, and rely on validated cryptographic modules throughout.

All VCF management communications use TLS, which provides integrity verification through message authentication codes (MACs) on every transmitted record. NSX Manager has modern TLS protocol versions enabled by default and older versions disabled. The Telegraf monitoring agent is configured with a minimum TLS version for its communication channels. VCF Operations supports a FIPS Security Mode that, when enabled, imposes additional algorithm and cipher prerequisites on management plane communications.

NSX, which provides the networking infrastructure for VCF, is configured to use FIPS 140-3 validated cryptographic modules by default. These modules are validated through the NIST Cryptographic Module Validation Program (CMVP). NSX Manager uses FIPS 140 approved algorithms for authentication to its cryptographic module. NSX Manager also supports activation or deactivation of cipher suites through API configuration to align with specific security requirements. For tunnel-based connectivity, the NSX L2 VPN FIPS compliance suite uses FIPS-validated authenticated encryption with Perfect Forward Secrecy enabled, protecting both confidentiality and integrity of tunneled traffic. NSX supports the Wrapped Encapsulating Security Payload (WESP) protocol, which builds on Encapsulating Security Payload (ESP) and provides integrity and authentication services for network traffic while allowing intermediate devices to determine whether data confidentiality is in use.

VCF Operations for Networks uses FIPS-validated cryptographic modules for both internal and external connections. The component uses BC-FJA (Bouncy Castle FIPS Java API) version 2.0.0 as its validated cryptographic module. FIPS mode can be enabled specifically for external connections, restricting all external communication to FIPS-validated modules. VCF Operations for Networks supports several encryption algorithms and ciphers for its data sources, allowing administrators to align cryptographic settings with organizational requirements. VCF Automation also uses the BouncyCastle FIPS 2.0.0 cryptographic module, which is compliant with FIPS 140-3.

The VMware vCenter appliance uses VMware's Boring Crypto Module version 6.0, which provides FIPS-validated AES, ECDSA, HMAC, RSA, and SHA2 algorithms, and the OpenSSL FIPS Object Module (versions 2.0.20-vmw and 3.0), which extends coverage to additional algorithms including SHA-3 and PBKDF. Starting in vSphere 9.0, all TLS profiles across the vSphere platform are FIPS compliant.

vSAN data-in-transit (DIT) encryption is a cluster-wide setting that, when enabled, encrypts all data and metadata traffic as it transits across hosts in the cluster. This includes fault tolerance traffic between cluster nodes. The setting is configurable during encryption configuration and applies uniformly to all inter-host vSAN communication. vSAN Data Services include both data-at-rest encryption and data-in-transit encryption as integrated capabilities. The Reliable Data Transport (RDT) protocol handles communication between hosts over the vSAN VMkernel ports, and enabling DIT encryption adds cryptographic integrity protection to that transport layer.



vSAN uses end-to-end checksums to verify the integrity of stored and transmitted data, confirming that each copy of a file matches the source. The system checks data validity during read and write operations, and if a checksum mismatch is detected, vSAN automatically repairs the data by overwriting the incorrect copy with correct data.

Encrypted vSphere vMotion protects the integrity and confidentiality of data transferred during VM migrations. vSphere vMotion always uses encryption when migrating encrypted virtual machines. For virtual machines that are not encrypted, the encrypted vMotion setting can be configured as Disabled, Opportunistic (use encrypted vMotion if supported, which is the default), or Required (do not migrate if encryption is unsupported). Fault Tolerance log traffic, which replicates running VM state to a secondary host, can be encrypted separately to protect that synchronization channel. Auditors should note that storage vMotion operations performed entirely within a single host, where only the datastore changes without a host transfer, are not covered by the vMotion encryption setting.

For storage protocols, vSAN File Service supports NFS4 Kerberos authentication with security options krb5, krb5i, and krb5p. The krb5i level provides authentication plus integrity checking on all NFS operations without the overhead of full data encryption. The krb5p level adds encryption on top of integrity checking. Both Kerberos-authenticated and system-authenticated NFS4 mounts are supported on the same share, giving administrators flexibility to apply integrity verification where needed. For iSCSI storage connections, VMware ESX supports configurable header and data digests that verify data integrity at the iSCSI layer, beyond the checks that TCP and Ethernet already provide.

VCF Operations monitors transmission integrity through metrics that track frames transmitted or received with bad checksums (CRC values), which indicate data corrupted during transmission. Frames with bad checksums are dropped and not propagated to other ports, which limits the spread of corrupted data on the network.

VCF Operations orchestrator signs packages and encrypts them for data protection, providing integrity assurance for software distribution across the environment. Package signing certificates provide digital identification for encrypted communication and sign packages. Packages exported from a VCF Operations orchestrator server are digitally signed, and the recommended practice is to use certificates signed by a Certificate Authority rather than self-signed certificates.

ESX hosts generate high-quality entropy that supports security-related operations such as generating encryption keys for secure communication over the network. This entropy source underpins the strength of cryptographic keys used across the platform for transmission protection.

The VMware Kubernetes Service (VKS) clusters managed by Supervisor extend these protections to the workload consumption layer. Kubernetes enforces TLS across all control plane communications as a structural requirement of its architecture: connections between the API server and etcd, between the API server and kubelets, and between the API server and cluster clients are TLS-protected by default. Cipher suite selection and minimum protocol version are configurable at multiple points in the control plane. The kube-apiserver accepts a `--tls-cipher-suites` parameter that restricts the server to a specified list of cipher suites, and a `--tls-min-version` parameter that sets the minimum acceptable protocol version. The kubelet exposes equivalent settings through `tlsCipherSuites`, which controls which cipher suites are available for HTTPS connections, and `tlsMinVersion`, which sets the minimum acceptable version. For the kube-scheduler, the `--tls-cipher-suites` parameter restricts cipher negotiation to approved suites. Antrea, the CNI plugin in Supervisor environments, also supports configurable TLS cipher suites for its inter-component communications. The vSphere Cloud Provider Interface (CPI), which handles the integration between VKS clusters and the underlying vSphere infrastructure, supports a configurable set of TLS cipher suites for its API communications with vSphere. VKS on Supervisor ships with managed TLS certificates that underpin the cryptographic infrastructure for these connections.

At the workload ingress layer, the Contour ingress controller package exposes `tls.minimum-protocol-version` and `tls.maximum-protocol-version` parameters in its configuration file, giving administrators control over the protocol floor for inbound traffic to applications running on VKS clusters. For service-to-service communication within a cluster, the Istio service mesh standard package operates in mutual TLS (mTLS) mode, requiring both parties to authenticate and encrypting inter-service traffic. The minimum protocol version for Istio mTLS is configurable via `meshConfig.meshMTLS.minProtocolVersion` and defaults to TLS 1.2, with TLS 1.3 also available as an option. Environments with elevated cryptographic requirements can activate FIPS mode at the cluster level through the `ClusterClass` variable `fips.enabled`, which restricts the cryptographic

algorithms available to those meeting Federal Information Processing Standards; the Istio standard package also provides FIPS compliance as part of its standard configuration. Harbor, the container image registry available as a Supervisor service, secures registry traffic through TLS certificate configuration via the `tlsCertificate.tls.crt` and `tlsCertificate.tls.key` fields. The Supervisor secret injection agent automatically configures mutual TLS for communication with the vault-agent-injector service using auto-generated certificates.

Defining the integrity-protection policy for customer workload traffic, validating that customer application transmissions use authenticated encryption, and setting the integrity policy for workload-to-workload traffic running in VKS pods are organizational data-policy decisions.

VMware Private AI Services (PAIS) applies TLS to its AI service communication paths through CA trust bundle management and client TLS configuration in the PAISConfiguration custom resource. The `spec.clientTls.caBundleRefs` setting references ConfigMap resources that hold CA trust bundles for outbound TLS to the OIDC provider and Harbor container registry, with separate bundles recommended per application and periodic updates required before certificate expiration. PAIS endpoints use self-signed certificate authentication; the CA certificate is stored in the `pais-ingress-default` Kubernetes Secret in the Private AI Services namespace, and administrators retrieve it with `kubectl get secrets/pais-ingress-default -o yaml` to distribute to client services.

PAIS requires HTTPS trust bundles for every external integration point: OIDC providers, Harbor registries, content management systems used as Knowledge Base data sources, MCP servers that supply tools to models, and the pgvector database connection through Data Services Manager (DSM). CA trust bundles are imported as certificate files or ConfigMap references, and dedicated settings exist for each integration class, including `ca-trust-bundle-for-dsm` for the pgvector database path. When connecting an MCP server, the issuer certificate must be added as a CA trust bundle before the HTTPS connection can be established. The PAIS Supervisor endpoint communicates over HTTPS on port 6443 with certificate-based authentication, and the Prometheus metrics endpoint uses TLS with the same self-signed certificate model retrieved from the `pais-ingress-default` Secret.

GPU-accelerated VMware Kubernetes Service (VKS) clusters provisioned through PAIS run a FIPS-enabled VKR build (v1.33.1-vmware.1-fips-vkr.2 on Ubuntu 24.04), which provides FIPS-compliant cryptographic modules for control plane and worker node communication. JWT-based authentication for VCF Automation integration uses the `TM_CERT_BASE64` credential captured by running `openssl s_client` against the VCF Automation FQDN on port 443 and encoding the certificate in base64. The VCF Consumption CLI supports optional CA certificate validation through the `--ca-certificate` option when establishing context connections to VCF Automation endpoints. The Model Gallery assigns a unique digest to each model revision when the same model is pushed multiple times, which helps administrators detect tampering or unintended overwrites during artifact retrieval from Harbor.

Administrators should note that TLS client authentication and certificate validation on the Prometheus endpoint are turned off by default and must be explicitly enabled through the `tls-client-authentication` setting in PAISConfiguration to activate mutual certificate validation.

### VMware vDefend (Contributes)

VMware vDefend contributes to transmission integrity monitoring through several complementary mechanism clusters. The IDS/IPS engine, available in both Distributed and Gateway profiles, includes signature rules that match on TCP protocol fields, including flags, acknowledgement numbers, sequence numbers, and window sizes, to detect protocol anomalies and tampering indicators in network traffic. Custom IDS/IPS signature rules also support TLS-specific matching and can store TLS/SSL certificate artifacts to disk for forensic analysis and threat investigation. The IDS/IPS engine detects obsolete or integrity-deficient protocols by combining port verification with application identification, flagging traffic that uses older SSL and TLS versions that predate current security standards, early SMB versions that lack message signing, and cleartext protocols such as telnet.

The Security Services Platform's Network Detection and Response capability supports SSL verification for event logs sent over HTTPS, protecting the integrity of telemetry data in transit between NDR sensors and the platform. The Security Services

Platform's security segmentation analysis identifies risky application protocols that transport data without adequate integrity protection and provides guidance on restricting or blocking them through vDefend policy.

The Gateway Firewall's TLS Inspection feature operates as a TLS proxy to transparently intercept and inspect encrypted traffic for threats while retaining end-to-end encryption for compliant sessions. Administrators can configure Crypto Enforcement on TLS Inspection profiles to require that client and server connections meet defined minimum cipher suite requirements, providing a policy-enforcement mechanism over the cryptographic strength of inspected flows.

The Security Services Platform establishes external communications using TLS handshake negotiation and rejects connections that use deprecated cryptographic algorithms, weak key lengths, or older protocol versions. The SSP Installer and NDR Sensor components use FIPS 140-2 and FIPS 140-3 validated cryptographic modules for their own internal communications, including the VMware BouncyCastle Module and the VMware OpenSSL FIPS provider Object Module.

vDefend does not itself provide cryptographic integrity mechanisms such as message authentication codes or digital signatures for data in transit. Those protections are the responsibility of the applications and transport protocols in use and fall outside vDefend's scope.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) applies cryptographic mechanisms to protect the integrity of data in transit across management communications and replication data flows.

All management and API communications to the PNR server use TLS, which provides integrity verification through message authentication on each transmitted record. PNR requires an SSL/TLS endpoint certificate for every TLS connection to the PNR server; certificates must be server certificates with x509v3 Extended Key Usage indicating TLS Web Server Authentication. PNR uses TLS certificates and private keys to protect network communication and establish authentication with connected services. Administrators can configure the minimum TLS protocol version through envoy proxy settings by editing the `tls_minimum_protocol_version` parameter in the `envoy-proxy-config.json` file, allowing organizations to enforce a higher minimum where policy requires. When the minimum version is changed, PNR requires the setting to be updated consistently across all system components.

Replication traffic between sites supports optional network encryption. vSphere Replication supports enabling data encryption for replication traffic on a per-replication basis. When replicating encrypted virtual machines using array-based or vSphere Replication protection groups, PNR requires that the recovery and protected sites share a common Key Management Server (KMS) or that KMS clusters at both sites use common encryption keys, so that replicated data maintains its encryption across sites.

VCF adds platform-layer integrity protections, including vSAN data-in-transit encryption for inter-host storage traffic.

Selecting which data flows receive network encryption and managing the certificate and key management infrastructure are organizational data-policy decisions.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) applies cryptographic protection to data in transit across its major communication paths, using TLS to protect both the confidentiality and integrity of transmitted data.

DSM enforces FIPS 140-3 compliant cryptographic standards for certificate handling. The platform validates certificate algorithms at upload time, rejecting keys or signatures that do not conform to FIPS 140-3 requirements. These requirements apply to trusted root certificates and to custom certificates used during database version upgrades.

Database connections from clients to DSM-managed PostgreSQL, MySQL, and SQL Server clusters use TLS. PostgreSQL instances enforce a minimum TLS protocol version through the `'ssl_min_protocol_version'` setting. PostgreSQL supports TLS-encrypted connections with certificate-based authentication and CA certificate verification. MySQL enforces TLS on client connections, and FIPS mode on the MySQL client side can be activated through the `'ssl_fips_mode'` setting. DSM also manages SQL Server clusters using TLS for client connections; the `'network.tlsProtocols'` setting defines the allowed protocol versions for those connections. DSM validates SSL trust during MySQL replication operations.

DSM's certificate management supports custom certificates for database clusters. Administrators can configure database cluster certificates through the DSM UI or via Kubernetes cert-manager, which outputs certificates as Kubernetes TLS Secrets. DSM validates certificate algorithms at upload time and during version upgrades to confirm FIPS 140-3 compliance. The DSM API provides endpoints for retrieving the certificate authority needed to establish secure connections to individual database clusters.

Communication between DSM components is also secured with TLS. The Consumption Operator requires a TLS certificate for communication with the DSM provider. DSM log forwarding supports Auto TLS configuration to secure connections to the SMTP server.

For PostgreSQL data migration, DSM replication connections support SSL/TLS with a configurable SSL mode parameter to secure data in transit between source and target databases.

To fully satisfy this control, organizations should verify that clients connecting to DSM databases are configured to use the TLS connection path and that CA certificates are properly deployed to client trust stores. DSM's internal and external communication paths rely on this client-side configuration for the full TLS chain to be effective.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides TLS termination and re-encryption as a core function of its Service Engine data plane, giving administrators direct control over the cryptographic mechanisms applied to application traffic in transit. The SSL/TLS Profile, configured under Templates > Security > SSL/TLS Profile, defines the accepted protocol versions and the prioritized list of cipher suites for each virtual service. Built-in profile templates provide starting configurations, and administrators can create custom profiles that restrict accepted cipher suites to those using authenticated encryption. The Service Engine completes the TLS handshake for both client-to-Avi and Avi-to-backend sessions, so traffic passes through a cryptographically protected channel on each leg. The SSL Profile also supports post-quantum cryptography cipher suites, providing an option for deployments that need to adopt algorithms addressing emerging cryptographic requirements.

For deployments requiring validated cryptographic modules, Avi supports a FIPS mode in which only FIPS-compliant algorithms are used for all cryptographic operations. When FIPS mode is enabled on a Controller cluster, the Controller and Service Engines use the OpenSSL FIPS Provider, which has been validated at FIPS 140-3 Security Level 1 and awarded CMVP Certificate #4985, for both data-plane TLS sessions and management-plane communications between Avi components.

Cryptographic key material backing TLS certificates can be stored in Thales Luna Hardware Security Modules (HSMs) integrated with both the Avi Controller and Service Engines. Service Engine HSM groups, defined under Templates > Security > HSM Groups, allow Service Engines to authenticate with HSMs for secure storage of SSL certificate private keys. TLS session persistence data shared between Service Engines in a cluster is transmitted in encrypted form, maintaining the integrity of session routing state across the data plane. Configuration backup archives transferred to remote destinations use Secure Copy Protocol (SCP) or Secure File Transfer Protocol (SFTP). Intel QuickAssist Technology (QAT) acceleration is available on Service Engines for bulk cryptographic operations following TLS handshake completion.

For Kubernetes workloads, the Avi Kubernetes Operator (AKO) supports PKIProfile custom resources, which enable certificate-based authentication and encryption for backend service communication. The System-Standard SSL Profile applied through AKO provides TLS re-encryption for traffic to backend servers.

Defining which application traffic requires integrity protection and configuring the virtual service profiles accordingly are organizational data-policy decisions.

### Control CR 3.3

> Security functionality verification: Provide the capability to verify the correct operation of security functions. Verification during boot: Verify security functions during the boot process.

**SCF Controls:** CHG-06

**Aggregate Coverage:** Contributes

#### VCF (Contributes)

VCF provides several technical mechanisms that support verification of cybersecurity and data protection control functionality following changes, though the organizational process of systematically defining when verification is required and documenting results remains an additional responsibility.

VCF Operations includes a Configuration Drifts capability that monitors vCenter instances and vSphere clusters for deviations from assigned configuration templates. When a change is made, scheduled drift detection compares current configuration values against the desired state defined in those templates. Any deviation is flagged, and administrators can view and download drift status reports from the Configuration Drifts dashboard. Configuration templates can be integrated with a source control repository for versioning and change control, supporting automated testing and validation of configuration updates after changes are applied.

At the host and cluster level, vSphere Configuration Profiles manage the configuration of all hosts in a cluster collectively, ensuring consistency and detecting when individual host configurations deviate from the cluster-level desired state. vSphere Lifecycle Manager extends this to software specifications, offering compliance check operations that return the compliance status of each host in a cluster against the committed desired state. Together, these mechanisms allow administrators to confirm that host-level security controls remain intact after patching, upgrades, or configuration changes.

VCF Operations continuously monitors infrastructure objects against configured compliance benchmarks and identifies non-compliant objects along with their specific violations. When non-compliance is detected, VCF Operations provides actionable remediation recommendations. Automated remediation actions can also be configured to bring objects back into compliance based on severity. Separately, VCF includes automated compliance assessment capabilities for vSphere security controls based on the VMware vSphere Security Configuration Guide. These assessments can be run after changes to confirm that host and virtual machine configurations remain aligned with the established security baseline, where drift from the baseline after a change would indicate that a control is no longer functioning as intended.

NSX Manager maintains detailed audit logs that capture both old and new values for modified security policies. When a security policy is changed, the audit log records the user who performed the operation, the security policy identifier, rule configurations, action types, and sequence numbers. This level of detail allows administrators to review exactly what changed in a security policy and compare the before-and-after state, which is a foundational step in verifying that security controls continue to operate as designed after modifications.

Lifecycle management operations include built-in precheck validation. VCF Operations requires a precheck validation step before management component upgrades can proceed, and NSX Manager's upgrade coordinator includes pre-checks to validate upgrade readiness. These prechecks help confirm that the environment is in a known-good state before changes are applied.

VCF Automation provides a Content and Policies interface that identifies custom forms containing actions that cannot be validated. The Form Actions tab in the custom form designer lists all actions used in a form, allowing administrators to review non-validatable actions and verify their input validation code. This provides a mechanism for verifying that automation-driven controls continue to enforce expected behavior after form or policy modifications. VCF Automation day 2 action policies control which users can run selected actions on deployed resources, with only users matching the policy criteria permitted to execute those actions. After changes to these policies, administrators can review the policy configuration to confirm that the intended access restrictions remain in place.

At the Kubernetes consumption layer, VMware Kubernetes Service (VKS) and vSphere Supervisor provide additional mechanisms for verifying security control behavior after changes. Pod Security Admission supports three enforcement modes that can be set independently per namespace: enforce, audit, and warn. The enforce mode blocks pods that do not comply with the configured Pod Security Standard; the audit mode logs non-compliant pod creation attempts without blocking; and the warn mode returns a warning to the API client without blocking. VKS cluster management provides out-of-the-box security policy templates, including baseline and restricted profiles that mirror Pod Security Standard definitions, and custom policies can be created in Dry run mode to observe their impact before enabling full enforcement. After a change to Pod Security Admission configuration, operators can apply the restricted profile in audit or warn mode to observe which workloads would be affected, with the audit log capturing the results for review.

Kubernetes ValidatingAdmissionPolicy bindings support validationActions of Deny, Warn, and Audit to control how validation failures are handled during admission. When set to Audit, policy violations are recorded in the Kubernetes audit log without blocking traffic, allowing operators to observe policy match behavior after a policy change without disrupting production workloads. The FailurePolicy field governs behavior when a policy expression fails to evaluate, and bindings are scoped to namespaces so that namespace-level labels determine enforcement scope. After deploying or modifying a ValidatingAdmissionPolicy, operators can submit representative test resources to verify that validation rules fire as intended.

Kubernetes RBAC provides a SelfSubjectAccessReview capability that allows operators to query whether a given subject holds a specific permission on a given resource, enabling direct confirmation that role bindings are applying as intended after changes to Roles, ClusterRoles, RoleBindings, or ClusterRoleBindings. Pod security contexts can also be reviewed after policy changes to verify that individual workloads are operating within the intended privilege boundaries.

At the Supervisor infrastructure level, core cluster component pods such as etcd and kube-apiserver record restart events and uptime duration. After configuration changes that affect the cluster control plane, operators can examine these health indicators to confirm that cluster infrastructure components remain continuously operational and that changes have not disrupted core control functionality.

For resilience control verification, VKS cluster management supports backup and restore of entire clusters, selected namespaces, or specific resources identified by a label, managed through the VCF Automation Tenant Portal under Manage & Govern > VCF Services > Kubernetes Management. The Velero Plugin for vSphere supports both stateless and stateful applications and allows restore operations to be initiated from the Data Protection tab, including restoring a backup from a source cluster to a different target cluster. VKS cluster node VMs are backed up and restored through Supervisor state backup using the vCenter backup feature. After changes to backup or storage configuration, performing a test restore from a recent backup confirms that the resilience control is functioning correctly.

Security Technical Implementation Guides (STIGs) for Supervisor and VMware Kubernetes Service release nodes (VKRs) are available to guide hardening verification for the Kubernetes platform layer, providing a reference baseline against which post-change configuration compliance can be assessed.

These capabilities provide the technical building blocks for post-change control verification across the full VCF platform: configuration drift detection, compliance benchmarking, audit logging of security policy changes, upgrade prechecks, automation policy review, Pod Security Admission audit modes, admission policy verification, Kubernetes RBAC inspection, and VKS backup restore testing. However, they do not by themselves constitute a complete verification program. Organizations must establish procedures that define when functional verification is required, what controls to test after specific types of changes, how to perform functional testing of whether security controls are actively blocking, detecting, or enforcing as intended, and how to document verification results.

### **VMware vDefend (Contributes)**

VMware vDefend and the Security Services Platform (SSP) provide several mechanisms that help organizations verify that network security controls continue to operate as intended after changes to firewall policies, segmentation rules, and threat protection configurations.

The Security Services Platform's Security Intelligence feature includes a Monitoring capability that checks for changes in the scope of input entities every hour when enabled. When a change is detected, affected policy recommendations are flagged



with a "change-detected" status indicator. Administrators can review the flagged recommendations through the Policy Recommendations interface, accessible via Monitor & Plan, and rerun the analysis to confirm that segmentation policies and firewall rules remain consistent with the intended security posture after an infrastructure or policy change.

The Rule Analysis feature on the Security Services Platform provides a structured mechanism for verifying the efficiency and correctness of Distributed Firewall rules. When activated, Rule Analysis evaluates rule configurations and generates reports identifying redundant, shadowed, or misconfigured rules. After changes are made to firewall policies, running Rule Analysis helps confirm that the updated rule set operates as intended and that no unintended effects have been introduced. The feature's operational status is visible in the Security Services Platform UI at System > Platform & Features by checking the Rule Analysis feature card Status field.

The Security Services Platform extracts all IDS policies, rules, and security profiles to support multiple remediations, clean up stale policies, and suggest policies that do not conflict with one another. This supports a verification workflow where policy consistency across IDS and threat prevention configurations can be reviewed following changes. Before republishing policies, administrators can also review infrastructure and environment rules through the Review & Add Rules interface for the SSP-Infrastructure policy, confirming that rules are free of error conditions before propagating changes to the enforcement layer.

The Security Services Platform System > Overview page displays the current platform status, allowing administrators to assess stability before and after configuration changes. Feature-specific status indicators are available at System > Platform & Features, where the Rule Analysis and Malware Prevention feature cards each display a current operational status that administrators can check following updates or changes. The platform's deployment wizard also conducts validation checks before implementing configuration changes, reducing the risk that a change introduces a misconfiguration that could silently degrade security control effectiveness.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) provides several mechanisms that help verify data protection controls remain functional after changes to the environment.

Test recovery operations are the primary verification mechanism. Administrators can run test recoveries at any time to confirm that recovery plans execute correctly without affecting production workloads. Each test exercises the full recovery workflow, including virtual machine startup, network reconfiguration, and any custom steps defined in the plan. Recovery plans transition through defined states during testing, moving to a Test Complete state when the test finishes or a Test Interrupted state when a failure occurs during execution. This state tracking provides immediate visibility into whether data protection controls are operating as designed following any change.

Recovery plan history reports capture the result and timestamps for each plan execution and for individual steps within the plan. These records can be exported for documentation and compliance purposes, giving change management teams evidence that data protection controls were exercised and verified after changes. PNR explicitly requires that protection groups and recovery plans be verified following upgrades to the Protection and Recovery Appliance, and that connections between protected and recovery sites be validated for correctness after any upgrade.

Protection group state monitoring surfaces configuration problems that result from environmental changes. Protection and Recovery protection groups monitor the status of virtual machines and determine which operations are allowed in each protection state. Groups transition through well-defined states including Configuring Protection, Removing Protection, Restoring Placeholders, and Operations in Progress. When protection is disrupted by a change, the affected group transitions to a state that restricts test recovery and planned migration operations, surfacing the problem before a recovery attempt is made.

Alarms provide proactive notification when changes affect data protection controls. Alarms are generated when connectivity problems occur or when protection groups and recovery plans enter an error state, allowing administrators to detect and remediate issues promptly.

PNR introduces an Advanced Cyber Compliance (ACC) subscription that extends verification capabilities. With ACC enabled, Cyber Recovery can install security sensors on virtual machines during the validation phase to perform vulnerability analysis,



behavioral analysis, and malware signature scanning. This adds an active security verification layer alongside the existing snapshot integrity and replication status checks. The ACC subscription also enables use of the Latest instances option in recovery plans, providing additional recovery point selection capabilities for verification testing.

#### **VMware Avi Load Balancer (Contributes)**

Avi Load Balancer provides several mechanisms that support verification of security control configurations following changes. The Avi Controller CLI includes the ``show networksecuritypolicy`` command, which allows administrators to inspect the current state of Network Security Policies and confirm that changes have taken effect as intended. At the virtual service level, Avi WAF Policies attach a Positive Security group that applies learned application data and defines hit and miss actions, giving administrators a direct view of the rules actively enforced after a WAF policy modification. The ``learned-data-promotion-location`` setting controls which learned application data is promoted to Positive Security rules, allowing administrators to validate that the correct data has been incorporated following a change.

For correlating configuration changes with observed effects, the Service Engine Analytics Config Events overlay lets administrators identify whether a shift in traffic behavior was caused by a specific configuration modification, such as an update to virtual service security settings. This provides a post-change verification path that links configuration actions to traffic outcomes in a single view. Individual Layer 4 Policy Set rules provide a mechanism to control the behavior of Layer 4 virtual services by evaluating Layer 3 and Layer 4 attributes, allowing targeted policy modifications to be assessed in isolation.

ControlScripts are Python-based scripts that execute on Avi Load Balancer Controllers and are initiated by Alert Actions triggered by system events. They can alter Avi Load Balancer configuration or communicate with remote systems such as email, syslog, and SNMP servers, enabling automated post-change health checks and notification workflows. Alert Actions can also trigger ControlScripts to automatically add offending client IP addresses to a blocklist attached to a Network Security Policy, providing a feedback loop between detection and enforced policy state. The Application Security Report generates a downloadable PDF documenting security findings and pre-check results, supporting structured post-change documentation.

For deployments integrated with VCF, the Avi Load Balancer for VCF documentation specifies that Avi Management Plane alarms, alerts, and dashboards must be operational as part of post-upgrade validation, establishing explicit criteria that administrators can check following software changes.

The control's question asks whether the organization verifies control functionality as a deliberate process. Avi's observability features, CLI inspection tools, Config Events overlay, and ControlScript automation support that activity by making control state visible and queryable after a change. The organization must define acceptance criteria, expected baselines, and approval workflows. Avi does not automate the organizational verification decision itself.

Not applicable for this control: VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 3.3(1)

> Security functionality verification: Provide the capability to verify the correct operation of security functions. Verification during boot: Verify security functions during the boot process.

**SCF Controls:** CHG-06

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides several technical mechanisms that support verification of cybersecurity and data protection control functionality following changes, though the organizational process of systematically defining when verification is required and documenting results remains an additional responsibility.

VCF Operations includes a Configuration Drifts capability that monitors vCenter instances and vSphere clusters for deviations from assigned configuration templates. When a change is made, scheduled drift detection compares current configuration values against the desired state defined in those templates. Any deviation is flagged, and administrators can view and download drift status reports from the Configuration Drifts dashboard. Configuration templates can be integrated with a source control repository for versioning and change control, supporting automated testing and validation of configuration updates after changes are applied.

At the host and cluster level, vSphere Configuration Profiles manage the configuration of all hosts in a cluster collectively, ensuring consistency and detecting when individual host configurations deviate from the cluster-level desired state. vSphere Lifecycle Manager extends this to software specifications, offering compliance check operations that return the compliance status of each host in a cluster against the committed desired state. Together, these mechanisms allow administrators to confirm that host-level security controls remain intact after patching, upgrades, or configuration changes.

VCF Operations continuously monitors infrastructure objects against configured compliance benchmarks and identifies non-compliant objects along with their specific violations. When non-compliance is detected, VCF Operations provides actionable remediation recommendations. Automated remediation actions can also be configured to bring objects back into compliance based on severity. Separately, VCF includes automated compliance assessment capabilities for vSphere security controls based on the VMware vSphere Security Configuration Guide. These assessments can be run after changes to confirm that host and virtual machine configurations remain aligned with the established security baseline, where drift from the baseline after a change would indicate that a control is no longer functioning as intended.

NSX Manager maintains detailed audit logs that capture both old and new values for modified security policies. When a security policy is changed, the audit log records the user who performed the operation, the security policy identifier, rule configurations, action types, and sequence numbers. This level of detail allows administrators to review exactly what changed in a security policy and compare the before-and-after state, which is a foundational step in verifying that security controls continue to operate as designed after modifications.

Lifecycle management operations include built-in precheck validation. VCF Operations requires a precheck validation step before management component upgrades can proceed, and NSX Manager's upgrade coordinator includes pre-checks to validate upgrade readiness. These prechecks help confirm that the environment is in a known-good state before changes are applied.

VCF Automation provides a Content and Policies interface that identifies custom forms containing actions that cannot be validated. The Form Actions tab in the custom form designer lists all actions used in a form, allowing administrators to review non-validatable actions and verify their input validation code. This provides a mechanism for verifying that automation-driven controls continue to enforce expected behavior after form or policy modifications. VCF Automation day 2 action policies control which users can run selected actions on deployed resources, with only users matching the policy criteria permitted to execute those actions. After changes to these policies, administrators can review the policy configuration to confirm that the intended access restrictions remain in place.

At the Kubernetes consumption layer, VMware Kubernetes Service (VKS) and vSphere Supervisor provide additional mechanisms for verifying security control behavior after changes. Pod Security Admission supports three enforcement modes that can be set independently per namespace: enforce, audit, and warn. The enforce mode blocks pods that do not comply with the configured Pod Security Standard; the audit mode logs non-compliant pod creation attempts without blocking; and the warn mode returns a warning to the API client without blocking. VKS cluster management provides out-of-the-box security policy templates, including baseline and restricted profiles that mirror Pod Security Standard definitions, and custom policies can be created in Dry run mode to observe their impact before enabling full enforcement. After a change to Pod Security Admission configuration, operators can apply the restricted profile in audit or warn mode to observe which workloads would be affected, with the audit log capturing the results for review.

Kubernetes ValidatingAdmissionPolicy bindings support validationActions of Deny, Warn, and Audit to control how validation failures are handled during admission. When set to Audit, policy violations are recorded in the Kubernetes audit log without blocking traffic, allowing operators to observe policy match behavior after a policy change without disrupting production workloads. The FailurePolicy field governs behavior when a policy expression fails to evaluate, and bindings are scoped to namespaces so that namespace-level labels determine enforcement scope. After deploying or modifying a ValidatingAdmissionPolicy, operators can submit representative test resources to verify that validation rules fire as intended.

Kubernetes RBAC provides a SelfSubjectAccessReview capability that allows operators to query whether a given subject holds a specific permission on a given resource, enabling direct confirmation that role bindings are applying as intended after changes to Roles, ClusterRoles, RoleBindings, or ClusterRoleBindings. Pod security contexts can also be reviewed after policy changes to verify that individual workloads are operating within the intended privilege boundaries.

At the Supervisor infrastructure level, core cluster component pods such as etcd and kube-apiserver record restart events and uptime duration. After configuration changes that affect the cluster control plane, operators can examine these health indicators to confirm that cluster infrastructure components remain continuously operational and that changes have not disrupted core control functionality.

For resilience control verification, VKS cluster management supports backup and restore of entire clusters, selected namespaces, or specific resources identified by a label, managed through the VCF Automation Tenant Portal under Manage & Govern > VCF Services > Kubernetes Management. The Velero Plugin for vSphere supports both stateless and stateful applications and allows restore operations to be initiated from the Data Protection tab, including restoring a backup from a source cluster to a different target cluster. VKS cluster node VMs are backed up and restored through Supervisor state backup using the vCenter backup feature. After changes to backup or storage configuration, performing a test restore from a recent backup confirms that the resilience control is functioning correctly.

Security Technical Implementation Guides (STIGs) for Supervisor and VMware Kubernetes Service release nodes (VKRs) are available to guide hardening verification for the Kubernetes platform layer, providing a reference baseline against which post-change configuration compliance can be assessed.

These capabilities provide the technical building blocks for post-change control verification across the full VCF platform: configuration drift detection, compliance benchmarking, audit logging of security policy changes, upgrade prechecks, automation policy review, Pod Security Admission audit modes, admission policy verification, Kubernetes RBAC inspection, and VKS backup restore testing. However, they do not by themselves constitute a complete verification program. Organizations must establish procedures that define when functional verification is required, what controls to test after specific types of changes, how to perform functional testing of whether security controls are actively blocking, detecting, or enforcing as intended, and how to document verification results.

#### **VMware vDefend (Contributes)**

VMware vDefend and the Security Services Platform (SSP) provide several mechanisms that help organizations verify that network security controls continue to operate as intended after changes to firewall policies, segmentation rules, and threat protection configurations.

The Security Services Platform's Security Intelligence feature includes a Monitoring capability that checks for changes in the scope of input entities every hour when enabled. When a change is detected, affected policy recommendations are flagged

with a "change-detected" status indicator. Administrators can review the flagged recommendations through the Policy Recommendations interface, accessible via Monitor & Plan, and rerun the analysis to confirm that segmentation policies and firewall rules remain consistent with the intended security posture after an infrastructure or policy change.

The Rule Analysis feature on the Security Services Platform provides a structured mechanism for verifying the efficiency and correctness of Distributed Firewall rules. When activated, Rule Analysis evaluates rule configurations and generates reports identifying redundant, shadowed, or misconfigured rules. After changes are made to firewall policies, running Rule Analysis helps confirm that the updated rule set operates as intended and that no unintended effects have been introduced. The feature's operational status is visible in the Security Services Platform UI at System > Platform & Features by checking the Rule Analysis feature card Status field.

The Security Services Platform extracts all IDS policies, rules, and security profiles to support multiple remediations, clean up stale policies, and suggest policies that do not conflict with one another. This supports a verification workflow where policy consistency across IDS and threat prevention configurations can be reviewed following changes. Before republishing policies, administrators can also review infrastructure and environment rules through the Review & Add Rules interface for the SSP-Infrastructure policy, confirming that rules are free of error conditions before propagating changes to the enforcement layer.

The Security Services Platform System > Overview page displays the current platform status, allowing administrators to assess stability before and after configuration changes. Feature-specific status indicators are available at System > Platform & Features, where the Rule Analysis and Malware Prevention feature cards each display a current operational status that administrators can check following updates or changes. The platform's deployment wizard also conducts validation checks before implementing configuration changes, reducing the risk that a change introduces a misconfiguration that could silently degrade security control effectiveness.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) provides several mechanisms that help verify data protection controls remain functional after changes to the environment.

Test recovery operations are the primary verification mechanism. Administrators can run test recoveries at any time to confirm that recovery plans execute correctly without affecting production workloads. Each test exercises the full recovery workflow, including virtual machine startup, network reconfiguration, and any custom steps defined in the plan. Recovery plans transition through defined states during testing, moving to a Test Complete state when the test finishes or a Test Interrupted state when a failure occurs during execution. This state tracking provides immediate visibility into whether data protection controls are operating as designed following any change.

Recovery plan history reports capture the result and timestamps for each plan execution and for individual steps within the plan. These records can be exported for documentation and compliance purposes, giving change management teams evidence that data protection controls were exercised and verified after changes. PNR explicitly requires that protection groups and recovery plans be verified following upgrades to the Protection and Recovery Appliance, and that connections between protected and recovery sites be validated for correctness after any upgrade.

Protection group state monitoring surfaces configuration problems that result from environmental changes. Protection and Recovery protection groups monitor the status of virtual machines and determine which operations are allowed in each protection state. Groups transition through well-defined states including Configuring Protection, Removing Protection, Restoring Placeholders, and Operations in Progress. When protection is disrupted by a change, the affected group transitions to a state that restricts test recovery and planned migration operations, surfacing the problem before a recovery attempt is made.

Alarms provide proactive notification when changes affect data protection controls. Alarms are generated when connectivity problems occur or when protection groups and recovery plans enter an error state, allowing administrators to detect and remediate issues promptly.

PNR introduces an Advanced Cyber Compliance (ACC) subscription that extends verification capabilities. With ACC enabled, Cyber Recovery can install security sensors on virtual machines during the validation phase to perform vulnerability analysis,

behavioral analysis, and malware signature scanning. This adds an active security verification layer alongside the existing snapshot integrity and replication status checks. The ACC subscription also enables use of the Latest instances option in recovery plans, providing additional recovery point selection capabilities for verification testing.

#### **VMware Avi Load Balancer (Contributes)**

Avi Load Balancer provides several mechanisms that support verification of security control configurations following changes. The Avi Controller CLI includes the ``show networksecuritypolicy`` command, which allows administrators to inspect the current state of Network Security Policies and confirm that changes have taken effect as intended. At the virtual service level, Avi WAF Policies attach a Positive Security group that applies learned application data and defines hit and miss actions, giving administrators a direct view of the rules actively enforced after a WAF policy modification. The ``learned-data-promotion-location`` setting controls which learned application data is promoted to Positive Security rules, allowing administrators to validate that the correct data has been incorporated following a change.

For correlating configuration changes with observed effects, the Service Engine Analytics Config Events overlay lets administrators identify whether a shift in traffic behavior was caused by a specific configuration modification, such as an update to virtual service security settings. This provides a post-change verification path that links configuration actions to traffic outcomes in a single view. Individual Layer 4 Policy Set rules provide a mechanism to control the behavior of Layer 4 virtual services by evaluating Layer 3 and Layer 4 attributes, allowing targeted policy modifications to be assessed in isolation.

ControlScripts are Python-based scripts that execute on Avi Load Balancer Controllers and are initiated by Alert Actions triggered by system events. They can alter Avi Load Balancer configuration or communicate with remote systems such as email, syslog, and SNMP servers, enabling automated post-change health checks and notification workflows. Alert Actions can also trigger ControlScripts to automatically add offending client IP addresses to a blocklist attached to a Network Security Policy, providing a feedback loop between detection and enforced policy state. The Application Security Report generates a downloadable PDF documenting security findings and pre-check results, supporting structured post-change documentation.

For deployments integrated with VCF, the Avi Load Balancer for VCF documentation specifies that Avi Management Plane alarms, alerts, and dashboards must be operational as part of post-upgrade validation, establishing explicit criteria that administrators can check following software changes.

The control's question asks whether the organization verifies control functionality as a deliberate process. Avi's observability features, CLI inspection tools, Config Events overlay, and ControlScript automation support that activity by making control state visible and queryable after a change. The organization must define acceptance criteria, expected baselines, and approval workflows. Avi does not automate the organizational verification decision itself.

Not applicable for this control: VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 3.4

> Software and information integrity: Provide the capability to verify the integrity of software, firmware, and data at rest.

Authenticity of software/firmware: Capability to verify the authenticity of software and firmware before installation.

Automated notification of integrity violations: Provide automated notification if integrity violations are detected.

**SCF Controls:** END-06

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF contributes to file integrity monitoring through configuration drift detection, desired-state management, and configuration change auditing at the infrastructure layer, though it does not provide traditional file-level FIM for guest operating systems or endpoint file systems.

vSphere Configuration Profiles define the desired configuration state for all VMware ESX hosts in a cluster and detect when hosts drift from that baseline. vSphere Configuration Profiles must be enabled on vSphere clusters to view drift status in VCF Operations. When drift is detected, vSphere Lifecycle Manager (vLCM) can remediate the host back to the expected configuration using its desired-state API. Administrators can check the compliance of a cluster or host against the desired state and determine whether additional remediation steps are required. vSphere also supports configuring alarms that activate in response to configuration drift events and notify administrators via email or SNMP traps, enabling proactive reporting of unauthorized or unintended configuration changes.

VCF Operations extends host-level drift detection with its Configuration Drift feature, which monitors VMware vCenter instances and vSphere clusters for configuration changes and compares them against assigned desired values. The Configuration Drift dashboard provides visibility into which settings have changed across managed infrastructure. Drift detection runs on a configurable collection cycle (every 8 hours by default) and flags deviations from the established template, giving operations teams a mechanism to detect unauthorized or unintended configuration changes. Configuration drift templates can be integrated with a source control repository, enabling versioning, auditing, and management of configuration baselines outside the VCF Operations console.

VCF Operations also maintains an audit trail for its own configuration files. The Configuration Files page displays Last Modified and Modified By fields that track when each configuration file was last updated and which user modified it. These fields are available for management packs and other configuration files, and can be filtered by name, description, last modified date, and modifying user. This provides traceability for changes made to VCF Operations configuration settings.

At the activity level, VCF Operations audit logging tracks each configuration change to the authenticated user who initiated the change or scheduled the job that performed the change. Data center administrators can determine the principal identifier of the authenticated user who initiated a change to an object at a specific time, supporting forensic investigation of unauthorized modifications.

The vSphere Daily Check Dashboard provides an additional layer of configuration verification. It covers configuration changes and verifies that settings match authorized changes executed during approved change windows, giving administrators a recurring check against unauthorized modifications.

Within the storage layer, vSAN Distributed File System Snapshots provide information about files that have been changed and the changes made to each file, offering a point-in-time comparison capability at the storage level. These snapshots can be used by third-party backup vendors to copy changed files incrementally, though they are designed for data protection rather than real-time integrity monitoring.

For file-level integrity monitoring within guest operating systems or for monitoring arbitrary files on endpoints in real time, organizations need to deploy a dedicated FIM solution. ESX Agent Manager can facilitate the deployment and lifecycle management of third-party security agents, including FIM tools, within the virtualized environment by managing ESX agent virtual machine deployment across hosts in a cluster.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.



## Control CR 3.4(1)

> Software and information integrity: Provide the capability to verify the integrity of software, firmware, and data at rest.

Authenticity of software/firmware: Capability to verify the authenticity of software and firmware before installation.

Automated notification of integrity violations: Provide automated notification if integrity violations are detected.

**SCF Controls:** CHG-04.2

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF enforces digital signature verification for software and firmware components at multiple layers of the stack, from hypervisor boot through application deployment.

During VCF deployment, each component is assigned a certificate from a default signing certificate authority. The VMware Certificate Authority (VMCA) is an internal CA that comes integral with vSphere deployment and has been purpose-built to serve the certificate needs of the vSphere environment. From initial installation onward, all VCF components operate within a certificate-based trust framework rooted in a known signing authority. Administrators can configure VCF to use certificates from VMCA, Microsoft CA, OpenSSL CA, or external third-party certificate authorities, giving organizations flexibility to integrate their approved CA hierarchy into the VCF signing and verification workflow. All communication between the VCF Installer and ESX hosts is performed over HTTPS with certificate identity validation, and the installer validates the identity of each host by comparing the certificate common name against the provided FQDN.

At the hypervisor level, VMware ESX uses UEFI Secure Boot to verify that the boot loader and kernel modules are digitally signed before execution. The host acceptance level setting controls which vSphere Installation Bundles (VIBs) are permitted: it can be configured to accept only VMware-certified VIBs, restricting installation to components signed by VMware's certificate authority. The vSphere Security Configuration Guide includes recommendations for enabling Secure Boot and configuring TPM on ESX hosts. When UEFI Secure Boot is enabled on an ESX host, the host performs full signature verification of each package installed on the host at every boot, not only at initial installation. vSphere also supports UEFI Secure Boot for individual virtual machines, where the bootloader, operating system kernel, and operating system drivers must each carry a valid signature before the virtual machine is permitted to boot.

vSphere Lifecycle Manager (vLCM) provides desired-state image management for ESX hosts. Administrators define a target image specification for each cluster, and vLCM checks host compliance against that image. When hosts deviate from the desired image, vLCM can remediate them back to the approved configuration. This approach restricts ad-hoc installation of unsigned or unapproved software components on managed hosts.

VCF Operations enforces digital signing for its management packs and orchestrator packages. Management pack PAK files are verified against a digital signature provided by VMware. If a PAK file lacks a valid digital signature, the system displays a warning indicating the file cannot be verified as authentic, and the digital signature indicates the original developer or publisher. For orchestrator packages, all packages exported from a VCF Operations orchestrator server are digitally signed, and the orchestrator's certificate infrastructure allows signing of all elements created within the system. When a client receives an element from the orchestrator server, the client verifies the server identity and decides whether to trust the signature. Package signing certificates are a form of digital identification used to support encrypted communication and provide a signature for orchestrator packages. For production environments, VCF recommends using a certificate signed by a certificate authority rather than a self-signed certificate, allowing organizations to align package signing with their approved CA infrastructure.

For container workloads, VMware Kubernetes Service (VKS) validates TLS certificates when fetching container images from registries and requires that registry certificates be signed by a trusted certificate authority. The platform requires that a certificate signing request be generated, signed by a certificate authority, and applied to the Workload Platform Management layer to secure the Kubernetes control plane. TLS certificates for the Supervisor layer are derived from the VMCA certificate, anchoring the Kubernetes control plane trust chain in the same root authority as the broader VCF infrastructure. The VCF CLI `imgpkg` tool supports CA certificate validation for registry connections through its `--registry-ca-cert-path` flag, which can be

specified multiple times to add CA certificates for registry API communication. VKS clusters can also be configured to trust any number of CA certificates for private registry endpoints, directing image pulls to controlled, organization-approved sources.

Harbor is available as a standard package for VKS clusters and provides enterprise container registry capabilities including content trust and image signing. Organizations can configure Harbor to enforce signature requirements at image push or pull time, restricting deployments to images signed under an organization-approved certificate authority. Admission controllers are the primary enforcement mechanism for image signing policies within VKS clusters. The ImagePolicyWebhook admission controller allows administrators to configure an external image verification backend that is consulted before any pod is scheduled; if the backend determines that an image lacks a valid signature from an approved authority, the pod is rejected before it runs. ValidatingWebhookConfiguration objects can integrate third-party image signing solutions with the Kubernetes API server, creating an enforcement layer that intercepts pod creation requests and validates image signatures as part of the admission control chain.

The Kubernetes control plane components used by VKS are themselves distributed as signed artifacts. Kubernetes release artifacts, including standalone binaries and Software Bill of Materials files, are signed using cosign keyless signing, and control plane component images can be verified using cosign with certificate-based identity and OIDC issuer validation. Kubernetes CertificateSigningRequest signers can deny requests that do not meet approval conditions, and CertificateSigningRequests with unknown signerName values are never auto-approved, restricting unauthorized certificate issuance within the cluster. For workload certificate lifecycle management, the cert-manager standard package creates TLS certificates for workloads and automatically renews them before expiration, with VKS cluster control plane certificate rotation configured through the ClusterClass API and defaulting to renewal 90 days before expiry. The renewal window is configurable through the renewalDaysBeforeExpiry parameter in the ClusterClass specification, allowing organizations to set this period to match their certificate management policies.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 3.4(2)

> Software and information integrity: Provide the capability to verify the integrity of software, firmware, and data at rest.

Authenticity of software/firmware: Capability to verify the authenticity of software and firmware before installation.

Automated notification of integrity violations: Provide automated notification if integrity violations are detected.

**SCF Controls:** MON-01.7

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF does not include a dedicated File Integrity Monitoring (FIM) agent that watches for unauthorized changes to individual files on disk. However, several platform components provide change-detection, integrity-verification, and alerting capabilities that contribute to an organization's FIM strategy.

VCF Operations delivers a configurable alerting framework built on symptom definitions and alert definitions that can detect changes across the managed infrastructure. Administrators can create, edit, clone, and import symptom definitions to define the conditions that signal a potential unauthorized modification, and then associate those symptoms with alert definitions that trigger notifications. VCF Operations supports the comparison of changes to an object with symptoms and other operational data, helping administrators correlate a detected modification with its potential cause and determine whether the change was authorized. Custom compliance benchmarks can also be defined within VCF Operations, with editable alert definitions and policies that can be tailored to flag deviations from an approved baseline configuration.

VCF Operations Configuration Drifts continuously monitors VMware vCenter and vSphere cluster settings against defined configuration templates and notifies administrators when values deviate from the desired state. The Detect Drift function detects configuration drift across all configuration templates and displays results in a Drifts table. Drift detection can be scheduled, drift reports can be downloaded for review, and configuration templates can be integrated with source control for versioning and audit. The Configuration Drift dashboard allows administrators to view and compare drifts and identify settings that have changed. This addresses unauthorized configuration changes at the platform level, though it operates on vCenter and cluster settings rather than on arbitrary files within guest operating systems or appliances.

VCF Operations for Networks generates alert notifications whenever it detects changes in security certificates from identity providers such as vIDM or LDAP, providing automated change detection for a category of critical configuration artifact. VCF Operations also raises alerts when any property of a Security Tag or Security Group is changed, giving visibility into modifications to network security policy objects.

Alert notifications from VCF Operations can be forwarded to external monitoring systems through outbound notification plugins, including SNMP trap destinations (up to four), standard email via SMTP, and log file output. This allows integration with a centralized security information and event management (SIEM) platform for correlation with other FIM data sources. vCenter's built-in SNMP agent supports SNMPv1, v2c, and v3 traps, with v3 providing authentication and privacy options across all security levels, so alert traffic can be forwarded to a SIEM over an authenticated and encrypted transport channel.

At the host level, vSphere Lifecycle Manager (vLCM) performs compliance checks of VMware ESX hosts against a specified image or desired-state configuration. Image Manager supports validation of the desired state and detection of drifts from that state. When the current image on a host differs from the desired specification, vLCM can remediate the host back to the known-good image. This functions as a form of change detection and correction for the hypervisor software stack itself.

For boot-time integrity, ESX hosts support Secure Boot and Trusted Platform Module (TPM) attestation. The TPM event log records measurements taken during the boot process, and vCenter can retrieve these logs through the trusted infrastructure API for validation. Administrators can unseal a secret bound to an endorsement key to verify reported measurements from the TPM event log. This helps detect unauthorized modifications to firmware and boot components.

At the storage layer, vSAN uses end-to-end checksums to verify the integrity of data by confirming that each copy of a file matches the source. While this mechanism is oriented toward data corruption detection rather than unauthorized file modification, it provides a layer of integrity assurance for objects stored on vSAN datastores.

To satisfy the full intent of this control, organizations should deploy a dedicated FIM solution on VCF hosts and management components. VCF's alerting, drift detection, compliance checking, and integrity-verification capabilities provide supporting telemetry and integration points, but they do not replace a purpose-built FIM tool that monitors file system changes, computes cryptographic hashes, and generates alerts on unauthorized modifications to operating system files, binaries, and configuration files.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 3.5

> Input validation: Provide the capability to validate the syntax and content of all inputs.

**SCF Controls:** TDA-18

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides built-in input data validation mechanisms across its automation, orchestration, and management components. These capabilities allow administrators and developers to enforce data quality at the point of entry, whether through interactive forms or programmatic interfaces.

The VCF Automation custom form designer supports data validation by adding constraints to form fields. Administrators can configure field-level validation using regular expressions (which must follow both Java and ECMAScript syntax), minimum and maximum value constraints, maximum character length limits, and required field checks. For example, the deploymentName field supports a maximum character length constraint of 900 characters. VCF Automation input properties also support maxLength and minLength attributes to constrain the number of characters allowed for string inputs. Validation rules can be set as constant values or as conditional values determined by expressions, allowing dynamic validation logic that adapts based on other field selections or inputs. Numeric fields support minimum value validation with configurable error messages displayed to the user when input falls outside acceptable ranges. Regular expression validation supports custom error messages following JavaScript syntax, so users receive clear feedback when their input does not match the expected pattern.

VCF Automation also supports external validation through VCF Operations orchestrator action scripts. These external validations run when a user submits a catalog request form from either the UI or the API. The validation action scripts evaluate input parameter values and return a string describing the error when invalid data is detected. When input is valid, the external validation returns nothing, allowing the request to proceed. The names of validation action input parameters must match the names of the workflow input parameters being validated, enforcing a structured approach to validation design. VCF Automation provides a Content and Policies interface to identify which custom forms contain actions that cannot be validated, giving administrators visibility into validation coverage across the environment.

The VCF Operations orchestrator Input Form Designer provides its own input validation capabilities, including regular expression-based validation, minimum and maximum value or length constraints for input parameters, and external actions to validate the inputs of custom workflows. This enables validation at the orchestration layer for automated processes and workflows that accept user or system-provided input.

At the infrastructure layer, VMware vCenter includes a configuration validation process that examines configuration files for possible errors and conflicts and returns output to help administrators avoid configuration issues or loading incorrect configuration files.

At the Consumption layer, VMware Kubernetes Service (VKS), the Supervisor-based Kubernetes environment embedded in VCF, brings input validation to the Kubernetes control plane. The Kubernetes API server embedded in VKS enforces schema-based field validation on all incoming API requests; when configured in strict mode via the ServerSideFieldValidation feature gate, the API server validates each request against the resource's OpenAPI schema and rejects submissions that contain unknown or incorrectly typed fields, whether submitted through kubectl, the API server, or automation tooling.

VKS supports ValidatingAdmissionPolicy, a declarative mechanism for policy-based validation of API objects before they are persisted. Policies are expressed in Common Expression Language (CEL), which supports comparisons, map key existence checks, string pattern matching (including prefix checks via methods such as startsWith()), date calculations, and list membership validation. The Kubernetes CEL format library extends this with a Format.validate(string) function that validates a string against a specified format and returns a list of error strings when invalid. The spec.validations.message field carries human-readable error text to the caller on failure, while spec.validations.reason provides a machine-readable failure code from values including Unauthorized, Forbidden, Invalid, and RequestEntityTooLarge. A spec.variables field supports defining named CEL expressions that can be composed and reused within the same policy, reducing duplication and computational expense. Policies are bound to resources through ValidatingAdmissionPolicyBinding objects, which scope enforcement to

specific namespaces via label selectors; when multiple parameter objects match the label selector, policy rules are evaluated for each and the results are ANDed together. For validation logic that cannot be expressed declaratively in CEL, validating admission webhooks provide an extension point for integration with external validation services. The `ValidatingAdmissionPolicy` `status.conditions` array records the latest system observations of a policy's current state, and `status.typeChecking` records type-checking results for each CEL expression to detect missing fields and type confusion.

For user attribute validation, `UserValidationRule` applies a required CEL expression evaluated against `UserInfo` objects to enforce attribute-level constraints on who may interact with specific API resources; the `message` field customizes the error returned when a rule returns false. The Kubernetes declarative validation system supports tags including `+k8s:required` (indicating a field must be specified by clients), `+k8s:minimum` (a numeric minimum value constraint), and `+k8s:neq` (disallowing a specific value, with type inferred from the disallowed value). Custom resource definitions support `FieldValueForbidden` validation, which reports values that would be acceptable under some conditions but are prohibited by current security policy. Organizations are responsible for defining and maintaining the admission policies appropriate to their security requirements; VKS provides the control plane infrastructure on which those policies operate.

For integrations with external systems, the ServiceNow ITSM Plug-in for VCF Automation supports required field validation on data source choices, though regular expression and min/max value constraints are not available for all data types such as dropdown and checkbox fields.

Input validation for customer applications running on VCF is the responsibility of those applications and their developers. VCF provides infrastructure on which workloads run; the application-level input validation required by web applications, APIs, microservices, and data pipelines is implemented in the application code or in security overlays such as VMware vDefend (which provides Web Application Firewall capabilities through Avi Load Balancer) and dedicated DLP, WAF, or API security tooling deployed alongside the platform.

VMware Private AI Services (PAIS) provides a structured ML model validation workflow that checks the validity of model inputs before they are admitted to production use. Before a model can be uploaded to a Model Gallery for use by AI application developers, it must pass a mandatory validation sequence performed in a Deep Learning VM acting as a sandbox environment.

The validation sequence includes integrity verification by hash-code comparison, malware scanning to detect supply chain contamination, scanning for serialization attacks (a class of vulnerability specific to ML model files that can execute arbitrary code on deserialization), and inference behavior testing using Triton Inference Server. Models must be downloaded into the Deep Learning VM from trusted sources such as the NVIDIA NGC catalog, Hugging Face, or another model hub before validation. The Model Gallery preserves the integrity of stored models by assigning a unique digest to each revision when the same model is pushed multiple times, and PAIS uses a Harbor registry in the Supervisor (or an OCI-compliant registry from another vendor) as the model storage backend, requiring authentication and certificate-based trust for model retrieval.

Beyond technical file-level checks, MLOps engineers validate ML models against the security, privacy, and technical requirements of the organization before upload. This includes examining inference requests for malicious behavior and performing hands-on functional tests to confirm the model performs as expected. PAIS guidance is that models be sourced and validated internally rather than consumed directly from the Internet, where they could contain malicious code or be tuned for malicious behavior.

For agent and retrieval-augmented generation workflows, PAIS extends input validation to two additional surfaces. Knowledge Bases that supply context to model responses can be linked to a bounded number of data sources, and PAIS requires examination and approval of MCP server tools and capabilities before they are used in agents, helping prevent unsafe or untrusted external capabilities from being introduced into AI applications.

#### **VMware vDefend (Contributes)**

VMware vDefend checks the validity of information inputs across several of its network security subsystems, contributing to the organization's broader input validation posture.

Security Intelligence parses and validates the content format and inventory asset data in administrator-provided CSV files used during segmentation planning imports. When data mapping fails due to file errors, an error notification appears with a link to an annotated CSV file that highlights the specific errors for correction. Administrators can correct and re-upload the file, or allow analysis to continue with incorrect entries excluded from processing. Administrators are advised to validate inventory asset data for completeness before importing CSV files into the Security Services Platform.

Security Intelligence stores system-inferred infrastructure classifications in a "Not reviewed" state until an administrator explicitly accepts or modifies them. This classification review mechanism applies to compute entity classifications used in segmentation planning and policy generation, requiring human acceptance before automated classifications can influence policy decisions.

Security Intelligence Policy Recommendations includes a Monitoring capability that checks for changes in the scope of input entities every hour when enabled. When the monitoring feature detects a change in input scope, administrators are prompted to review the detected changes and rerun recommendation analysis to regenerate policy based on the updated inputs.

Security Intelligence enforces input limits on policy recommendation generation: a maximum of 100 VM entities can be selected as direct input, and the total number of effective VMs when groups are included cannot exceed 500. These constraints are enforced before analysis begins, validating that inputs fall within the range where analysis results are reliable.

The vDefend distributed firewall includes an auditing and remediation workflow that validates rule components, requiring rules to use IPv4 addresses for successful validation through the security gate before deployment.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) validates inputs across its recovery-plan, replication, and management surfaces. Configuration data exported in XML format can be validated against an XSD schema, providing structural validation of imported and exported configuration data (p.288). After PNR upgrades, administrators must re-verify that protection groups and recovery plans remain valid before they are placed back into operational use (p.108).

For recovery-plan IP customization, the DR IP Customizer tool uses a structured comma-separated value file with a header row defining column meanings and one or more rows for each placeholder virtual machine in a recovery plan (p.379, p.382). The `dr-ip-customizer` command supports a `--verbose` option that enables additional diagnostic logging for validation of customizer inputs (p.376).

PNR's Cyber Recovery workflow includes explicit validation states for VMs prior to recovery. Cyber Recovery VMs transition through a Preparing validation state and then an In validation state when an administrator selects a snapshot and initiates validation (p.575). Cyber Recovery supports on-demand vulnerability analysis through the Analysis tab to validate workload integrity before completing recovery validation (p.574).

Integrated Security and Vulnerability Analysis with CrowdStrike examines VMs in the clean room using behavioral analysis and machine learning during validation, with continuous analysis of running software and processes on the guest operating system to detect suspicious behavior (p.580). The integration performs vulnerability analysis combining Common Vulnerability Scoring System (CVSS) information with proprietary threat data and advanced modeling from Kenna Security, and identifies vulnerabilities by their CVE designation (p.582). Operators use these signals to select a clean snapshot for recovery rather than restoring a compromised one.

PNR logs certificate verification error messages in security-related log files for audit trail purposes, so that input-validation failures on certificate fields are captured (p.133). For operational health checks, PNR can adjust the number of status check attempts before declaring a check failed using the `connections.hmsPingFailedDelay` parameter (p.440). Trusted environment setup for the PNR Appliance requires text values for organization and organization unit during configuration (p.112). The Clean Room Orchestrator categorizes validation events using Info and Warning severities so that operators can act on input or state conditions surfaced during recovery operations (p.562).

Input validation for customer applications running on workloads protected by PNR is the responsibility of those applications and their developers. PNR provides input validation for its own recovery-plan, configuration import/export, customizer, API,



and orchestration surfaces, and provides Cyber Recovery validation states and CrowdStrike-driven analysis to validate workload integrity before recovered VMs return to production.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) contributes to input data validation through mechanisms built into its configuration, policy management, and integration interfaces.

At the policy layer, DSM data service policies support allowlist creation, restricting users to selecting only from administrator-defined sets of approved parameters when provisioning or configuring databases. In 9.1, SQL Server data service policies include an "Allow with a List" option that creates a specific allowlist of approved parameter values users can select from. Policies also enforce required-field validation: administrators must supply a name, description, and target data service before a policy can be saved. When applying a policy to existing databases, DSM displays a preview of any policy violations that would result, allowing review and correction before the policy is created or modified.

DSM validates configuration inputs for backup storage at multiple points. During monitoring, DSM checks the connection to the backup storage repository, verifies the correctness of the bucket name, confirms certificate expiration dates, validates credential currency, and verifies that sufficient available space exists in the repository. Administrators can view these validation results through the database monitoring interface.

In System Settings, DSM validates SMTP configuration inputs when settings are saved and returns an error if validation fails. DSM also requires validation of the SMTP certificate as part of SMTP configuration. NTP server entries are validated and flagged for correction if invalid values are present.

For webhook integrations, DSM provides a TEST function that sends the configured payload to the endpoint URL before the webhook is created, allowing administrators to verify that the endpoint is reachable and that the payload is accepted before deployment.

DSM introduces a "latest" certificate validation mode as the default for version 9.1 and later. This mode enforces strict cryptographic and KeyUsage validations for all DSM endpoints, including the Provider VM and data service clusters. The validation mode is configurable through the advanced-system-config ConfigMap.

For PostgreSQL databases, DSM includes the amcheck extension, which provides functions for verifying relation integrity at the database engine level.

For local user accounts, DSM enforces password complexity requirements: passwords must be between 8 and 20 characters and include at least one uppercase letter, one lowercase letter, one number, and at least one special character. This validates credential inputs at the authentication boundary. DSM does not send a verification message to confirm the validity of local user email addresses.

DSM's input validation mechanisms cover its own administrative and configuration interfaces. They do not extend to application-layer validation for data flowing through the PostgreSQL, MySQL, or MinIO databases it manages. Input validation for workloads connected to DSM-managed databases is an organizational and application development responsibility.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer provides application-layer input validation through three complementary mechanisms: WAF signatures and the Positive Security Model, DataScript programmable rules, and Avi Kubernetes Operator (AKO) configuration validation.

Avi WAF signatures perform input and output validation by analyzing all incoming and outgoing traffic. The WAF Positive Security Model, also called Application Specific Policy, describes expected application behavior and enforces it by setting an accepted range and length of characters for HTTP fields. Requests that do not meet those specifications are reported as policy violations before reaching backend workloads, allowing administrators to restrict inputs to known-valid character sets and field lengths on a per-application basis. The WAF also provides analytics to identify possible false positives and allows operators to add exceptions through a modal dialog that generates suggested values pre-computed from log entries and related findings, supporting ongoing policy refinement.

DataScript extends input validation with programmable, event-driven logic executed within Avi Service Engines. Using Lua-based scripting with relational, logical, and arithmetic operators, DataScript rules can inspect and validate request parameters, HTTP headers, and request body content. Specific capabilities include an external request validation API that offloads validation decisions to an external service and returns a response object containing status code, headers, and HTTP response body; XML body parsing via the Parse function, which returns a success indicator and either the parsed result or error information if parsing fails; client certificate validation via the `avi.ssl.check_client_cert_validity()` function, which returns distinct values when the certificate is not presented, is valid, or is invalid; conditional clearing of certificate verification errors based on error code evaluation; string pattern functions including `string.contains()` and `string.startswith()` for field content checks; and group-based user identity validation using `avi.stringgroup.equals`, which checks whether a user ID is in an approved list and can redirect unauthenticated users to a login page. DataScript rules should be tested and validated after modification to confirm that the intended validation logic operates correctly without affecting other virtual services.

The Avi Kubernetes Operator (AKO) performs syntactical validations at the time of Custom Resource Definition (CRD) object creation to verify configuration correctness and provides effective status messages. The AKO CRD Operator `condition-status` field indicates whether conditions are 'True' or 'False', supporting automated validation of AKO configuration inputs in Kubernetes environments.

## Control CR 3.6

> Deterministic output: Ensure that the component produces deterministic output for given inputs.

**SCF Controls:** SEA-07.3

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF implements fail-safe procedures across the compute, storage, networking, orchestration, and lifecycle management layers, providing automatic responses to failure conditions at each tier.

At the compute layer, vSphere HA defines automated restart procedures for virtual machines when host failures occur. HA detects host unavailability through network heartbeat monitoring and triggers VM restart on surviving hosts. Admission control policies can be set to Cluster Resource Percentage, Slot Policy, or Dedicated Failover Hosts, each reserving capacity so that failover procedures have the resources they need to complete. For workloads that require continuous availability, vSphere Fault Tolerance maintains a synchronized secondary VM on a separate host, providing rapid failover when the primary host experiences a failure condition. vCenter High Availability (vCenter HA) extends fail-safe protection to the VMware vCenter management layer itself, guarding against host failures, hardware failures, and vCenter application failures. Fault Tolerance failover behavior can be validated before production reliance using the Test Restart Secondary feature, which induces a secondary VM failure to confirm that the protection is functioning correctly. vMotion enables live migration of running virtual machines during planned maintenance, allowing administrators to evacuate workloads before performing host-level changes. vSphere Supervisor zones further strengthen failure isolation by treating each zone as an independent failure domain, so failures within one zone do not affect others.

At the storage layer, vSAN incorporates several fail-safe mechanisms. Fault domains distribute data replicas across physically separate groups of hosts, so that data remains available even when failures occur within a domain. The failures to tolerate (FTT) storage policy controls how many simultaneous failures vSAN can absorb while maintaining data availability. When a component failure occurs, vSAN automatically redirects writes from the failed component to a durability component, maintaining data integrity without manual intervention. On permanent failure, vSAN rebuilds affected components on other hosts in the cluster to restore the policy-defined level of protection. Administrators can configure failback behavior to suit their environment; for example, setting failback to "No" in environments using Spanning Tree Protocol avoids problems caused by automatic failback when a physical switch port recovers from failure and takes several seconds to begin forwarding traffic. Proactive Hardware Management (PHM) monitors hardware health and informs administrators of degrading devices based on predictive failure events from the OEM, enabling action before data loss occurs. VMware recommends planning and designing vSAN upgrades to be fail-safe as well.

For broader disruption scenarios, Application Disaster Recovery enables failover to a secondary site to maintain operations during major disruptions. At the networking layer, NSX provides documented recovery procedures for Global Manager failures, including scenarios where both active and standby instances fail. In failure scenarios involving risk of data loss or catastrophic failure, documented procedures direct administrators to contact Broadcom Support to review the recovery plan before taking remediation steps.

VCF Operations extends fail-safe behavior into the orchestration and automation layer. Workflows support structured exception handling, where exception handlers catch errors and react according to the specific errors received. Elements within a workflow can set exception bindings that define how the workflow behaves when it encounters an error in that element. If the built-in exception handlers cannot handle a particular error, the element's exception output parameter can be bound to an Exception element, which ends the workflow run in a controlled failed state rather than an undefined condition. For user interaction elements, defining exception behavior is a recommended practice; if a user does not provide input parameters within the timeout period, the interaction returns an exception, and administrators should define the exception behavior in a scripted function to avoid uncontrolled workflow failure.

VCF Operations also provides alerting mechanisms that detect and surface failure conditions. Policy-based alert definitions use combinations of symptoms and recommendations to identify conditions classified as problems, such as failures or high

stress. Risk and warning alerts indicate conditions requiring near-term investigation. VCF Operations for Networks raises alerts when a network configuration deviates from its defined intent, marking the intent status as failed and triggering investigation workflows.

Lifecycle management operations include built-in fail-safe steps. Upgrade and patching workflows run precheck validation that must complete before changes are applied, catching potential issues before they affect production systems. The vCenter upgrade validation operation runs upgrade prechecks and allows administrators to review errors, warnings, and status data before proceeding. VCF Operations provides a separate precheck capability that validates readiness before an upgrade proceeds. After any recovery operation, VCF requires verification of the recovered system's health and remediation of any issues found.

### **VMware vDefend (Contributes)**

VMware vDefend includes several fail-safe mechanisms within its network security platform, primarily through the Security Services Platform (SSP).

The SSP enforces healthy-state prerequisites before executing backup and restore operations, blocking recovery procedures when the platform is degraded. SSP backup or restore operations enter a failed state after 30 minutes if internal services are restarted manually or due to failure during the operation. SSP must also be in a healthy state before the Malware Prevention Service can be activated, and Network Detection and Response (NDR) activation fails if the SSP cluster is in a degraded or unstable state. These gating behaviors help prevent security service activations and recovery procedures from proceeding from a compromised baseline.

The SSP monitors current deployment status and raises platform-related alarms to highlight errors requiring attention. When a scheduled backup fails, SSP triggers a Scheduled Backup Error alarm to notify administrators. SSP generates an alarm when Bare Metal servers fail to update their certificates, and displays publish status with error indicators in the Status column to identify publication failures requiring troubleshooting. Certificate manager and security intelligence services can be verified for operational status to diagnose service availability issues. Security Intelligence suspicious traffic detectors report execution status after each run, with possible values of NOT\_STARTED, SUCCESS, NOT\_ENOUGH\_BASELINE, or FAILURE, giving operators visibility into detection system health.

For SSP deployments, vSphere HA (a VCF platform capability) is recommended for host-level failure recovery, with Admission Control configured with sufficient resources for recovery. Storage policies with RAID configuration support datastore-level failure tolerance. For deployments on vSAN stretched configurations, resilience protects SSP data from multiple host failures within a single site.

Broader organizational fail-safe procedures covering the infrastructure as a whole, including the processes that define how and when these mechanisms are invoked, remain outside vDefend's scope.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) provides fail-safe behavior through controlled recovery operations and configurable safety thresholds. During disaster recovery, if errors occur on the protected site, PNR continues the recovery operation rather than aborting, so a failure at the protected site does not block workload recovery at the recovery site. If the protected site goes offline unexpectedly during recovery, the failure is reported but otherwise ignored, allowing recovery to proceed without both sites being available. Custom command steps within a recovery plan have type-based failure handling so an operator can specify how the plan responds when a step does not succeed.

By contrast, planned migration is treated as a controlled operation rather than an emergency. If errors occur on the protected site during a planned migration failover, the operation stops and requires manual error resolution and plan rerun before proceeding. This distinguishes routine migration from disaster recovery semantics and preserves operator control during non-emergency moves.

Site-down detection thresholds are configurable to reduce the risk of premature failover declarations from transient connectivity issues. The `connections.drPingFailedDelay` parameter controls how many consecutive status check failures must

occur before PNR declares a disaster recovery connection failure, and `connections.hmsPingFailedDelay` controls the equivalent threshold for health management checks.

PNR surfaces operational health through VCF Operations dashboards and the PNR Global Console. The Configuration Summary Dashboard generates critical warnings when a recovery plan is in an error state, when a plan ran while two sites were disconnected and a split-brain condition resulted, and when reprotect fails for a plan. PNR also alerts users to minor errors including expired snapshot schedules, replication errors such as missing VMs, connectivity issues with protected or destination sites, and site misconfigurations that require investigation. The Clean Room Orchestrator raises warning-status alerts for the same conditions in cyber recovery workflows. Recovery plans should be tested before use for planned migration or disaster recovery to validate functionality before production execution.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer provides application-layer fail-safe mechanisms at multiple levels, enabling organizations to define predictable, controlled behavior when failure conditions occur at the load balancing and application delivery layer.

At the server pool level, Avi supports configurable fail action behavior. When all pool servers become unavailable, the Pool Fail Action setting determines how Avi responds. The HTTP Local Response option serves a custom error page to clients, presenting a controlled failure state rather than an unhandled connection timeout. DataScript and policy-driven `POOL_DOWN` actions provide an additional method for generating custom error responses when backend pools are unavailable.

Health monitors control failure detection sensitivity through the Failed Checks parameter, which specifies the number of consecutive failed checks required before marking a server as down. In the Avi Kubernetes Operator (AKO), the HealthMonitor custom resource definition exposes the same `failed_checks` parameter, with a valid range of 1 to 50 and a default value of 2, extending configurable fail-safe behavior to Kubernetes-native workloads. The corresponding `successful_checks` parameter governs recovery, requiring a configurable number of consecutive successful checks before marking a backend as UP again.

At the Service Engine level, Avi's failure detection algorithm declares a Service Engine suspect down after four consecutive echo messages fail over an eight-second window under standard settings. An aggressive failure detection mode is available for environments requiring faster detection but is recommended only for bare-metal deployments; on virtualized environments, this mode can produce false positives.

Avi's Internet Content Adaptation Protocol (ICAP) Profile supports configurable failure action handling when ICAP servers encounter errors. Administrators can select Fail Closed, which returns an HTTP 503 response, or Fail Open, which allows traffic to pass through when the ICAP service is unavailable. This gives organizations direct control over the security-versus-availability trade-off for content inspection failures.

Avi's Global Server Load Balancing (GSLB) uses both control-plane and data-plane health monitors to detect site failures. When monitors at both planes mark members at a failed site as Down, GSLB redirects application traffic to surviving sites. In partial-site failure scenarios, the leader and peer Controllers detect the failure using control-plane monitors, but data-plane health monitors at surviving sites may continue to perceive members at the failed site as UP. Administrators should be aware that automatic GSLB leader failover will not occur if both the leader and the designated backup leader sites are simultaneously unavailable. Custom scripts used in external GSLB health monitors should be thoroughly validated for long-term stability before deployment against production servers.

Avi supports Availability Zones based on fault domains associated with shared power supplies, shared networking devices, or other common infrastructure. Configuring Service Engines across these zones can keep virtual services continuously available across fault conditions. For Linux Server Cloud deployments, traffic directed on a failing link within a port channel interface is automatically redirected to other links, providing link-level fault tolerance.

Avi ControlScript alert actions execute in response to detected events such as Pool Up and Pool Down and can programmatically invoke failover operations on pool groups through the Avi Controller API. Automation workflows using

Python or Ansible modules support error handling and retry logic to handle exceptions gracefully and reduce the risk of partial configurations or failed deployments.

Not applicable for this control: VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 3.7

> Error handling: Ensure that the component does not provide information in error messages that could be used by an attacker.

**SCF Controls:** TDA-19

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF contributes to error handling by providing structured mechanisms for identifying security-relevant error conditions, centralizing error and log data, and restricting access to error details through role-based access control.

VCF Operations defines alert policies that use combinations of symptom definitions and recommendations to identify problems such as failures and high-stress conditions. Symptom definitions can evaluate message events against specific object types, using operators to compare incoming event messages against configured text strings. Event categories include System Degradation, Change, Environment, Notification, Data Availability, Collector Down, and Object Error. Administrators can create alert definitions with tighter symptom parameters to track problems and identify issues before users encounter further disruption. Risk alerts flag problems that require investigation, and alert definitions can include incremental severity levels, such as Warning for conditions approaching a threshold and Critical for conditions that have reached it. When workflows execute in VCF Operations orchestrator, failed workflows generate notifications in the alerts window with details about the resources for which the workflow failed.

VCF Operations orchestrator includes structured exception handling within workflows. Workflow elements that encounter errors send error signals to exception handlers, which react according to the specific error received. Exception bindings on individual elements define how the workflow behaves when that element encounters an error. A default error handler catches errors not addressed by custom handlers. If no handler can process a particular error, the element's exception output parameter can be bound to an Exception element, which ends the workflow run in a failed state with explicit error information. Defining exception behavior for user interaction elements is a recommended workflow development practice. Exception handling variables capture error codes and messages, providing structured diagnostic information without exposing raw system internals.

VCF Log Management is the central point for log collection, aggregation, and analysis across all VCF instances within a fleet. VCF Operations and VCF Log Management must be integrated to implement the centralized log collection architecture. This centralization gives administrators the ability to analyze error conditions in context, correlating log entries across components. When errors occur during deployments, administrators can use log files and the Supportability and Serviceability (SoS) Utility for troubleshooting. Management Pack logs record errors related to user permissions or session errors that affect data collection, helping isolate permission-related failures from application-level errors. VMware ESX system logging supports eight severity levels as defined in RFC 5424: Emergency, Alert, Critical, Error, Warning, Notice, Informational, and Debug, providing a standardized hierarchy for categorizing and filtering error conditions. The ESX syslog message format includes an optional LINE-MARKER field to identify multiline log submissions and prevent log injection in syslog streams.

The Consumption layer of VCF, comprising VMware Kubernetes Service (VKS), vSphere Supervisor, and VCF Services, extends these error handling mechanisms into the Kubernetes workload tier. The Kubernetes API server within VKS Supervisor captures security-relevant error conditions including authentication failures, authorization denials, and admission control rejections in the cluster audit log. Admission webhooks log denial events with specific error messages that identify the reason for request rejection. ValidatingAdmissionPolicy definitions are parsed for syntax errors and rejected at creation or update time if errors are found, providing early detection of misconfigurations before they reach runtime. The ValidatingAdmissionPolicy failurePolicy field controls how CEL expression parse errors, type check errors, and runtime errors are handled, with allowed values of Ignore or Fail. ValidatingAdmissionPolicyBinding supports multiple validationActions, including Warn and Audit, which can be applied together: in Warn mode a client-facing warning is returned without blocking the request, while in Audit mode the violation is recorded in the audit log without notifying the caller. Each audit failure object includes the validation failure message string, the resource name of the ValidatingAdmissionPolicy, the resource name of the ValidatingAdmissionPolicyBinding, and the index of the failed validation expression. ValidatingAdmissionPolicy message



expressions, defined via the `spec.validations.message` or `messageExpression` field, allow operators to provide informative, context-specific denial reasons; when a `messageExpression` encounters a runtime error, the failure message falls back to the static message and the runtime error is logged separately rather than being surfaced to the caller.

VKS cluster status conditions use structured machine-readable representations that include a `Message` field providing human-readable information about the Reason for each condition. Machine health conditions in the `v1beta1` API classify events by Severity: Info for reconciliation in progress, Warning for conditions that may require attention, and Error for conditions that block operation. A `CSIProvisioningFailed` warning condition documents CSI addon storage provisioning failures. TLS handshake errors in Supervisor NSX networking are logged with timestamp, source IP, and port information, flagging potential routing or MTU issues in the network fabric. Supervisor Services installation distinguishes compatibility warnings, which can be acknowledged and skipped, from incompatibility errors, which must be resolved before installation continues.

Pod Security Admission in VKS Supervisor supports enforce, audit, and warn modes. In enforce mode, security violations cause the pod to be rejected. In audit mode, violations are recorded in the audit log without blocking the request. In warn mode, a user-facing warning is triggered without blocking the request. Applying the baseline Pod Security Standard in enforce mode while applying the restricted standard in warn and audit modes lets operators balance corrective feedback to users with security event capture for administrators. Contour Envoy ingress logging verbosity is configurable through the Envoy logging level setting, with supported values of trace, debug, info, warn, error, critical, and off, defaulting to info. Harbor container registry components deployed as VKS standard packages have a configurable log level that accepts values of debug, info, warning, error, or fatal, allowing operators to limit the diagnostic detail written to logs to what is operationally necessary.

Role-based access control across VCF management interfaces restricts access to logs, alerts, and error messages to authorized personnel. Users who require access to log integration must be assigned the Security Operations permission through the VCF Operations console under Administrator, Control Panel, Access Control, Roles. VCF Operations supports assigning specific roles and scope to users and groups, allowing administrators to limit who can view diagnostic information and error details. Within the Kubernetes layer, RBAC governs access to cluster and namespace logs and status conditions. Access to `kubectl` commands such as `kubectl logs`, `kubectl describe`, and `kubectl get events` requires appropriate ClusterRole or Role bindings scoped to the relevant namespace or cluster. Cluster audit logs are accessible only through cluster administrator channels and are not exposed to namespace-scoped users by default. For high-assurance environments, Kubernetes supports deploying cryptographic protections to help maintain tamper-proof and confidential log handling.

The organizational aspects of this control require processes outside VCF's scope. Defining policies for which error conditions are security-relevant, establishing standards for what information error messages may or may not reveal to different audiences, and creating procedures to review and update error handling practices over time are all activities that require organizational governance. Operators must design their `ValidatingAdmissionPolicy` message expressions to avoid exposing sensitive values such as secrets, token contents, or internal resource names in error output. RBAC configuration to restrict log access requires deliberate role design for each namespace and cluster, which is an organizational responsibility. VCF provides the technical mechanisms to identify, capture, and restrict access to error information, but the policies governing how those mechanisms are configured and what constitutes an appropriate level of detail in error messages must come from the organization.

VMware Private AI Services (PAIS) contributes to error handling requirements through several mechanisms specific to its AI workload infrastructure.

PAIS uses Kubernetes Conditions metadata for component status reporting. Component state is surfaced through typed fields including `Type` (for example, `Ready`), `Status` (`True` or `False`), and `Reason` (for example, `paisAvailable`). Administrators verifying a PAIS deployment inspect these structured conditions through `kubectl` or the VCF Automation UI, which provides diagnostic information scoped to availability status without exposing raw internal state. The `PAISConfiguration` custom resource must reach a `Ready` status with reason `paisAvailable` to indicate successful activation, which establishes a defined diagnostic checkpoint for identifying initialization failures as security-relevant error conditions at the component level.

For runtime visibility, PAIS supports Prometheus metrics collection with a configurable retention period and forwarding of LLM traces to an OpenTelemetry Collector. MLOps engineers and application developers monitor these signals through observability platforms such as Grafana to track model health, quality, and behavior of AI applications and identify anomalous conditions. NVIDIA vGPU license server communication errors, a failure mode specific to GPU-accelerated deployments, can be diagnosed by examining `/var/log/syslog` on the host for NVIDIA-related error messages.

The model validation workflow in PAIS addresses a platform-specific class of security-relevant error condition. Before a model is uploaded to a Model Gallery, the validation pipeline checks file integrity through hash codes, scans for malware, scans for serialization attacks, and runs functional inference tests through Triton Inference Server as part of a sandbox validation process on a Deep Learning VM. Models that fail these checks are contained at ingestion rather than reaching inference time. The Model Gallery further maintains revision integrity by assigning a unique digest to each pushed revision, which helps identify tampering or unexpected version drift. Harbor project access capabilities restrict who can pull models and reach the training data associated with them, which limits the audience for any diagnostic detail exposed during model retrieval failures.

The organization is responsible for implementing application-level error message generation in workloads running on PAIS, defining which application error conditions are security-relevant, and controlling access to system logs where detailed diagnostic messages may appear.

### **VMware vDefend (Contributes)**

VMware vDefend contributes to error handling by surfacing security-relevant error conditions across its management interfaces and policy enforcement components.

Security Services Platform (SSP) surfaces policy publication failures with a red exclamation icon in the Status column; authorized administrators can click the error status to view detailed error information before republishing. SSP monitors current deployment status and surfaces platform-related alarms that highlight errors requiring attention. NSX Manager-related alarms displayed in the SSP System Dashboard indicate errors that may need attention. When CSV data mapping fails during segmentation planning, an error notification appears with a link to an annotated CSV file that highlights the specific errors for correction before re-upload, providing corrective guidance without exposing raw configuration data. Rule Analysis failures are captured in a HAR file along with NSX and SSP support bundles when filing a support ticket, providing a structured process for error diagnosis.

SSP can be configured to forward log messages to a remote syslog server, supporting centralized collection and analysis of error events by authorized operations personnel.

The IDS/IPS signature bundle management Errors Tab in vDefend Advanced Threat Protection provides a dedicated troubleshooting view that isolates failed updates and displays specific error messages and codes to help identify why a signature bundle failed to download or apply to a particular node. A Failed/Down status filter allows administrators to view only nodes that encountered errors or are currently unreachable.

Malware Prevention Service event logs record an `analysisStatus` field indicating completion state along with `errorCode` and `errorMessage` fields for troubleshooting analysis failures. The Network Detection and Response (NDR) Sensor provides a CLI command "get sensor details" to view registration details and display error messages if sensor registration fails. When reviewing migration alerts in the migration plan, administrators can identify specific violation messages to determine which rules require remediation.

Bare Metal Agent Health Status monitoring in SSP displays Upgrade Failed and Offboard Failed counts along with Last Operation status for each Bare Metal server, giving administrators a targeted view of operational errors.

At the host level, the Guest Introspection Platform Host Agent (MUX) supports configurable log levels and classifies messages as [ERROR], [DEBUG], or [INFO]; the EPSeclib component uses the same classification with an (EPSEC) designation for Endpoint Security-specific messages, enabling targeted review of error-severity events during troubleshooting.

vDefend does not fully address all three requirements of this control. Systematic restriction of error message visibility to authorized personnel and comprehensive sanitization across all platform components depend on the broader platform access control and logging infrastructure rather than vDefend alone.

### VMware Data Services Manager (Contributes)

VMware Data Services Manager (DSM) generates structured alerts and log records that identify error conditions relevant to database operations, authentication, and platform connectivity. The Global Alerts Reference documents categories of security-relevant errors that DSM surfaces, including authentication failures, backup and restore corruptions, and certificate errors, each mapping to conditions where administrative action may be required. DSM also tracks infrastructure policy status and surfaces policy violations before a data service policy is applied to databases, allowing review before changes take effect. A compliance report for each data service policy displays policy violations, noncompliant databases, data services, and namespaces, giving administrators a structured view of out-of-compliance objects.

DSM provides multiple mechanisms to expose error information in actionable form. The database Status indicator reflects availability, identifies in-progress operations, and flags critical operations that have failed. The Database Alerts panel surfaces configurable threshold-based alerts for CPU Health, Data Disk Health, System Disk Health, and Max Connections metrics, with separate warning and critical thresholds that administrators and users can adjust. Mitigation suggestions accompany database alerts to guide resolution. The monitoring framework supports metrics-based and status-condition-based alerts that identify critical failure conditions, including database engine degradation, non-operational status, and automated or transaction log backup failures.

DSM logs provide structured error records for administrative review. For PostgreSQL, DSM logs logical replication errors with timestamps, error codes, constraint violations, and transaction details in the log bundle; replication worker failures, including permission denied errors and replication slot operations, are recorded separately. For SQL Server, DSM supports the `errorLog.numErrorLogs` custom engine setting to control how many error log files are retained before recycling. When troubleshooting database provisioning failures, DSM documentation directs administrators to review VMware vCenter logs for API errors. The Consumption Operator Pod logs can be inspected directly from the running container or via command-line interface to diagnose error status conditions in VCF Automation deployments.

When deeper investigation is needed, DSM supports generation of database support bundles that capture database activity and diagnostic information, and a system-level support bundle function for broader troubleshooting of system configuration failures. Alerts can be activated or deactivated for data service operation failures, and webhook integrations forward operation failure notifications to external systems with alert level information indicating whether an alert is CRITICAL or WARNING; a TEST function is available to validate webhook payloads before deployment.

Access to error information and alert detail is governed by DSM's role-based access model. DSM distinguishes between Admin and User roles with role-based UI view restrictions that scope what each user can see and act on, limiting exposure of operational error detail to personnel with appropriate access. The broader requirement to formally identify all security-relevant error conditions, define policies for what information error messages may contain, and enforce organization-wide controls on message disclosure requires organizational policies and procedures beyond what DSM itself provides.

### VMware Avi Load Balancer (Contributes)

VMware Avi Load Balancer provides mechanisms that address all three dimensions of this control at the application delivery layer.

For identifying security-relevant error conditions, the Avi Controller and Service Engines log audit record failures covering a defined set of security-significant events: initiation and termination of trusted channels, trusted channel function failures, unsuccessful certificate validation attempts, client authentication failures, and SSH session establishment failures. These records include initiator and target identification, making them useful for detecting and investigating security incidents. Audit record failures are forwarded to Events, Syslog, and Splunk. Virtual service logs record request policy rules, network security rules, request outcomes, and error conditions including abnormal request termination and HTTP response codes. The analytics profile attached to a virtual service defines what constitutes an error for the purposes of significant log classification, giving administrators control over which conditions are surfaced as noteworthy events.

For controlling what information is returned to clients in error responses, Avi supports Error Page Profiles that let operators define the content returned to clients for each HTTP status code. Administrators configure Error Page Profile templates to associate custom error page bodies with specific HTTP status codes, which are then applied per virtual service. DataScripts

can return custom error messages to the client by detecting the RESP\_FAILED event, which is triggered when Avi Load Balancer cannot process a transaction. Custom error responses can also be generated from policy or POOL\_DOWN actions. When a DataScript encounters an execution failure, the default behavior returns a generic HTTP 500 Internal Server Error to the client, limiting exposure of internal application details in client-facing messages. For ICAP-integrated virtual services, the ICAP Profile supports configurable failure action handling, with options for Fail Closed (returns HTTP 503 on error) or Fail Open (allows traffic to pass), giving administrators explicit control over error behavior when content inspection is unavailable.

Access to detailed diagnostic logs and audit records is governed by Avi's role-based access control (RBAC) system. Administrative interfaces and detailed event and audit log views are available only to authenticated users with appropriate role assignments. The Alert feature processes system events and metrics against user-defined rules and generates alerts that initiate configured actions such as sending email, syslog messages, or SNMP traps, routing operational notifications to the authorized administrative audience.

The organizational dimension of this control requires policy decisions that Avi supports through its configuration capabilities but does not make automatically: deciding which conditions are security-relevant, specifying what information is sensitive in error output, and defining which personnel are authorized to view detailed logs.

Not applicable for this control: VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 3.8

> Requirement CR 3.8 in IEC 62443-4-2. The licensed standard text is not reproduced here; refer to the standard.

**SCF Controls:** NET-09

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides multiple mechanisms to protect the authenticity and integrity of communications sessions across its management interfaces, API endpoints, and infrastructure components.

Session identity and token management

VCF's VMware vCenter includes a SessionManager service that provides the interface to the authentication infrastructure on the target server system. Each session is associated with a specific user account for its full duration, and valid session identifiers are used to establish and verify the identity of principals and to establish trust with API endpoints. A session ID acquired from SessionManager is a temporary substitute for username and password, limiting risk to the principal's credentials. The Common Infrastructure Services (CIS) Session service provides additional token management capabilities, including creation, deletion, and retrieval of session information. The client API supports session maintenance through HTTP cookies via the SESSION\_MAINTAIN\_PROPERTY setting, and a session preservator capability saves user sessions and acquires persistent tokens to preserve identity across reconnections. Administrators should define Release Session Requests when using session authentication to properly log out sessions and reduce the risk of session leaks. Concurrent session monitoring is recommended to detect excessive session creation that could indicate unauthorized activity or resource exhaustion.

At the Consumption layer, the Kubernetes Secrets API provides a mechanism for storing and distributing session tokens and other short-lived credentials to workloads without exposing them in pod specifications. The operator pattern allows a controller to fetch short-lived session tokens from an external service and create Secrets that pods consume, keeping token issuance and refresh logic separate from pod implementations. Using short-lived Secrets limits the exposure window when credential data is inadvertently disclosed. For workloads running as VMs through VM Service, secrets can be injected via a disk-based sidecar mechanism, where a trusted pod retrieves secrets and makes them available to the VM without direct credential exposure in pod specifications.

Transport-layer encryption for session communications

All management plane communications across VCF are encrypted using TLS. NSX Manager session communications are encrypted to restrict session hijacking, and NSX authentication mechanisms are resistant to replay attacks. NSX uses mutual TLS authentication based on self-signed or CA certificates between transport nodes (hosts) and the Central Control Plane, providing bidirectional identity verification for infrastructure-level sessions. vCenter API authentication transports credentials over HTTPS with TLS encryption. VCF Operations supports enforcement of TLS for secure communications through the ServerCertificateValidationCallback configuration. VCF Operations for Networks also supports TLS for its communications.

NSX BGP neighbors can be configured with message authentication to apply integrity checks to routing protocol messages, protecting the integrity of routing sessions between network components. NSX remote logging can be configured to use TLS for secure log transport. VCF supports SSH fingerprint verification for remote server connections, allowing administrators to confirm they are connecting to the correct server and providing protection against interception attacks. Metadata exchanged between service providers and identity providers includes information required for establishing trust and facilitating secure communication, supporting session authenticity at the federation layer.

At the Consumption layer, the Kubernetes API server in VMware Kubernetes Service (VKS) clusters on vSphere Supervisor operates exclusively over TLS, establishing authenticated and encrypted sessions between clients and the API server. Secure communications between all Kubernetes control plane services are implemented using certificates. For service-to-service session integrity within a cluster, Istio is available as a standard package on VKS clusters and provides mutual TLS encryption between services with identity-based authentication and authorization, so that each service authenticates the other before

establishing a session. Organizations whose workload sensitivity warrants it should deploy a service mesh to encrypt all intra-cluster communications.

#### API session authentication

NSX Manager supports session-cookie-based API authentication over TLS. Cookie-based authentication is enabled by default, and subsequent API calls use the session cookie and XSRF token for request validation rather than transmitting credentials with each request. VCF Automation uses OAuth access tokens for session management. Device authorization tokens issued by VCF Automation expire in 3600 seconds by default, and API access tokens have their own configurable expiration periods. The access token returned upon successful authentication through VCF Automation's identity provider proxy is valid for 5 minutes, requiring periodic refresh.

At the Consumption layer, the Kubernetes API server in VKS supports client authentication via service account tokens, X.509 client certificates, OIDC tokens, and webhook tokens, depending on the deployment configuration. Audience-aware token authentication validates that a token was specifically issued for the target API server by verifying the token against the server's expected audience identifier, which restricts token reuse across different services. For container registry operations, Harbor Registry (available as a standard package on VKS clusters) uses shared secrets for inter-component communication among its core server, job service, and registry components.

#### Session lifetime controls

Session lifetimes are bounded by configurable timeouts to limit exposure from idle or abandoned sessions. NSX Manager session expiry defaults to 1800 seconds (30 minutes) and can be adjusted using the CLI or API. VCF Operations session inactivity timeout defaults to 30 minutes. VCF Operations for Networks supports user session timeout configuration with values ranging from 15 minutes to 24 hours, defaulting to 15 minutes. The VMware ESX Host Client session timeout controls the duration before an inactive host management session is terminated. The vCenter session timeout for unified VIM/vAPI sessions can be configured through the vpx.conf file or the vSphere Client UI.

At the Consumption layer, Kubernetes guidance recommends setting short lifetimes on certificates and automating their rotation to reduce the window of opportunity for credential exploitation. Using short-lived Kubernetes Secrets for session tokens further limits the impact of credential exposure.

#### PKI infrastructure and cryptographic foundation

The PKI infrastructure that underpins session security offers multiple certificate management options. VCF supports fully automated certificate issuance within the platform, replacement of default certificates with enterprise CA certificates, and integration with Microsoft Active Directory Certificate Services using a restricted service account for least-privilege access. NSX cryptographic modules are validated to FIPS 140-3 standards by the NIST Cryptographic Module Validation Program (CMVP), and NSX is configured to use FIPS 140-3 validated cryptographic modules by default. VCF Operations for Networks uses FIPS-validated cryptographic modules matching those specified by the FIPS 140-2 standard.

VKS clusters on vSphere Supervisor include a set of TLS certificates that support the cryptographic infrastructure for secure communications among cluster components; these certificates can be manually rotated through the VKS certificate management interface when needed. For ingress traffic, the Contour ingress controller is available as a standard package on VKS clusters and uses internally issued certificates with configurable validity periods to secure communications between the Contour controller and its Envoy proxies. Leaf certificates default to 720 hours and the CA certificate defaults to 8,760 hours; both durations are configurable.

#### Data plane session integrity

For data plane sessions, encrypted vSphere vMotion secures the confidentiality, integrity, and authenticity of VM migration traffic. Encrypted vMotion can be set to Opportunistic (use encryption if supported) or Required (do not migrate if unsupported), and vSphere always attempts encryption when migrating encrypted virtual machines. Fault Tolerance (FT) logging traffic between primary and secondary VMs, which replicates guest network and storage I/O data as well as memory contents, is unencrypted by default and should be isolated on a dedicated network segment; vSphere supports optional



encryption of FT log traffic to protect the integrity and confidentiality of this inter-host session channel. vSAN data-in-transit encryption is a cluster-wide setting that, when enabled, encrypts all data and metadata traffic as it transits across hosts and file service inter-host connections. NSX IPSec VPN supports certificate-based authentication for site-to-site VPN sessions, where the site certificate configured on the local endpoint is used for mutual authentication.

### **VMware vDefend (Contributes)**

VMware vDefend (DFW) and Gateway Firewall provide stateful session tracking that monitors entire conversations between endpoints, enforcing session boundaries and detecting anomalous session behavior. DFW rules can be configured with TCP strict mode (tcp-strict-mode), which enforces connection state validation and drops packets that do not conform to expected TCP state transitions. Session Timers define how long a session is maintained on the firewall after inactivity, with configurable timeouts for TCP, UDP, and ICMP sessions that can be applied to user-defined groups or Tier-0 and Tier-1 gateways.

Security Intelligence analyzes flow data to identify unexpected communications between workloads, enabling exception policy creation to protect required inter-workload sessions. Security Intelligence also detects suspicious behavior associated with remote connection protocols such as telnet, SSH, and VNC.

vDefend TLS Inspection extends session integrity enforcement to encrypted traffic by detecting advanced threats transmitted over encrypted channels. TLS Inspection includes certificate revocation list validation to restrict sessions using revoked certificates. vDefend IDS/IPS detects sessions using obsolete transport protocols by combining port verification with application identification, covering clear-text protocols and outdated transport protocol versions.

Security Services Platform (SSP) establishes its own external communications using TLS handshake negotiation and rejects any certificate or TLS configuration that uses deprecated cryptographic algorithms or older protocol versions. Network Detection and Response (NDR) uses SSL verification for event logs sent over HTTPS.

vDefend does not provide cryptographic end-to-end session authentication or confidentiality between communicating endpoints; those protections require application-layer or infrastructure-layer controls outside vDefend's scope.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) protects the authenticity and integrity of management and replication sessions through several mechanisms. PNR uses TLS certificates and private keys to protect network communication and to authenticate with remote servers. Replication traffic between protected and recovery sites supports optional network encryption.

For service-to-service authentication, the PNR server acquires a holder-of-key SAML token from VCF SSO and sends the token in a cryptographically signed message to remote services. Remote services validate the signature to confirm the identity of the calling service before accepting requests, which helps protect against token reuse across paired sites.

The PNR REST API Gateway enforces explicit session lifecycle controls. Callers authenticate through the Authentication API POST endpoint, which returns a session identifier that must be included as the x-dr-session header on subsequent requests. A DELETE endpoint terminates an authenticated session, and a GET endpoint returns information about the current session if one exists. The Site Pairing REST API uses a POST endpoint to create an authenticated session to a remote PNR server, and a Validate Connection POST operation tests connectivity to a remote appliance before pairing.

PNR records session-related activity in security log files, including login events with timestamps, user identity, session keys, and locale information. Certificate verification errors and connection failures during authentication attempts are also recorded in the log files and in the user interface. The logManager.SessionManager setting controls the verbosity of session management logging, with a default value of verbose. PNR monitors active site-to-site connections by sending periodic ping requests and logs connection check events, so broken connections are detected operationally.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides session integrity mechanisms for database and management-plane communications through certificate-based encryption and strong authentication. Database cluster connections use TLS certificates for client connectivity and monitoring. DSM stores PEM-encoded certificate authorities in the ca.crt property of Kubernetes Secrets for database cluster authentication, and administrators can configure custom CA certificates per database



cluster through the DSM UI. The DSM Provider Appliance supports secure TLS connections validated against a trusted CA. DSM also establishes secure connections to VMware vCenter using certificate validation. This certificate validation chain supports session authenticity by confirming that connections reach the intended endpoint before data exchange begins. DSM requires that certificate expiration dates are verified to maintain secure operations, with the alerting system tracking certificate validity as part of its monitoring coverage.

For PostgreSQL databases, DSM uses host-based authentication (HBA) configured through `pg_hba.conf` rules. These rules apply ordered matching based on connection type, client address, requested database name, and username, and the first matching record determines the authentication outcome. DSM supports `scram-sha-256`, `reject`, and `cert` as configurable authentication methods for custom HBA entries, giving administrators control over which connection patterns are accepted or denied. The `scram-sha-256` method is the default for standard user accounts and internal accounts including `pgadmin` and `postgres_exporter`. Administrators can configure additional HBA entries to include LDAP-backed authentication where directory service integration is desired. DSM also supports enabling or disabling directory service authentication on a per-database-instance basis through the database settings interface.

PostgreSQL replication session integrity is maintained through a trust bundle and dedicated replication credentials that authenticate connections between primary and secondary instances. Replication connections support SSL/TLS encryption with a configurable `sslmode` parameter to scope encryption to approved connection modes. For remote replication, users with the DSM user role must manually establish trust between primary and secondary instances before replication begins. For MySQL, DSM verifies network access to the source database and validates SSL trust during replication operations, and controls TLS protocols permitted during group replication recovery connections through the `group_replication_recovery_tls_version` setting.

For SQL Server databases, DSM uses TLS to encrypt communication between clients and SQL Server clusters. An internal management login (`dsm_mgmt`) restricts access to system-level objects, scoping administrative operations within a secure context.

The DSM Consumption Operator requires a TLS certificate for secure communication between the operator and the DSM provider, obtained and saved by the Cloud Administrator as part of operator installation. Metrics forwarding from DSM to VCF Operations supports Mutual TLS (mTLS) authentication, enabled by configuring the Client Certificate Authentication toggle along with the client certificate and private key, extending mutual authentication to management-plane communications. Connection activity logging is available for PostgreSQL through the `log_connections` parameter, which records connection attempts and the successful completion of client authentication, supporting session visibility alongside the integrity controls.

DSM session integrity mechanisms apply to DSM-managed database connections and management-plane communications. Session integrity for non-database services, user interfaces outside DSM, and broader organizational network communications relies on platform-level and organizational controls. Operators are responsible for managing CA certificate trust stores, configuring HBA rules suited to their environment, and applying network-layer session protections beyond what DSM configures directly.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides multiple application-layer mechanisms that contribute to protecting the authenticity and integrity of communications sessions.

TLS termination is the primary session-integrity mechanism for application traffic. Avi Service Engines terminate SSL/TLS sessions on behalf of backend applications, with SSL profiles controlling the allowable cipher suites for each virtual service. The SSL Cipher Strength setting governs the cipher strength used to authenticate applications to clients. During SSL handshake processing, Avi negotiates protocol version and cipher suites to establish authenticated, encrypted sessions. Avi also supports TLS session persistence, which maintains session continuity using TLS session tickets with automated mirroring across Service Engines, without adding processing overhead.

The System-Secure-HTTP application profile secures HTTP traffic by enforcing HTTPS, protecting cookies, managing client IPs, and controlling protocol behavior for virtual services. This provides a consistent session-security baseline for HTTP application workloads and can be assigned to virtual services during advanced setup.

For HTTP application workloads, Avi provides Cross-Site Request Forgery (CSRF) protection through session token management. When a session ID is invalid or absent, Avi creates a new session for the request and restarts the CSRF protection flow from token creation, helping maintain the integrity of application sessions against forged requests.

For internal control-plane communications, Avi establishes a secure TLS session between an Envoy proxy on each Service Engine and the Avi Controller. Secure Channel Certificates for Controller-to-Service Engine communication are configured in the System Settings administration panel under Administration > System Settings.

At the management plane, Avi's User Profile includes a Maximum Concurrent Login Sessions feature that restricts simultaneous active sessions per administrator account to a configurable threshold. When the threshold is reached, existing sessions can be invalidated using the `--clear-sessions` shell command.

For BGP routing sessions, Avi supports MD5 authentication. When BGP authentication is enabled, each TCP segment exchanged between BGP peers is verified and accepted only when authentication succeeds, blocking unauthorized peer session establishment.

In Kubernetes deployments, the Avi Kubernetes Operator (AKO) supports session affinity through the HTTPRule Custom Resource Definition (CRD), which provides cookie-based session persistence for application workloads managed by Avi ingress controllers. Avi also supports MCP load balancing with session affinity based on unique MCP Session IDs, maintaining connection persistence between clients and servers for AI model serving workloads.

In distributed deployments, GSLB Site Persistence directs all requests in a long-lived transaction from a client to the same server, maintaining application session integrity across sites.

Avi requires that all external communication-related authentication failures be logged and persisted for auditing purposes, covering session establishment and authentication events across the platform.

Administrators should configure SSL profiles, cipher suites, CSRF policies, session limits, and the System-Secure-HTTP application profile for each virtual service and user profile based on the sensitivity of the applications and the organization's session security requirements.

## Control CR 3.8(a)

> Session integrity: Protect the integrity of communication sessions at the application layer. Invalidate session identifiers: Capability to invalidate session identifiers upon logout or timeout. Unique session identifiers: Generate unique session identifiers.

**SCF Controls:** NET-09

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides multiple mechanisms to protect the authenticity and integrity of communications sessions across its management interfaces, API endpoints, and infrastructure components.

#### Session identity and token management

VCF's VMware vCenter includes a SessionManager service that provides the interface to the authentication infrastructure on the target server system. Each session is associated with a specific user account for its full duration, and valid session identifiers are used to establish and verify the identity of principals and to establish trust with API endpoints. A session ID acquired from SessionManager is a temporary substitute for username and password, limiting risk to the principal's credentials. The Common Infrastructure Services (CIS) Session service provides additional token management capabilities, including creation, deletion, and retrieval of session information. The client API supports session maintenance through HTTP cookies via the SESSION\_MAINTAIN\_PROPERTY setting, and a session preservator capability saves user sessions and acquires persistent tokens to preserve identity across reconnections. Administrators should define Release Session Requests when using session authentication to properly log out sessions and reduce the risk of session leaks. Concurrent session monitoring is recommended to detect excessive session creation that could indicate unauthorized activity or resource exhaustion.

At the Consumption layer, the Kubernetes Secrets API provides a mechanism for storing and distributing session tokens and other short-lived credentials to workloads without exposing them in pod specifications. The operator pattern allows a controller to fetch short-lived session tokens from an external service and create Secrets that pods consume, keeping token issuance and refresh logic separate from pod implementations. Using short-lived Secrets limits the exposure window when credential data is inadvertently disclosed. For workloads running as VMs through VM Service, secrets can be injected via a disk-based sidecar mechanism, where a trusted pod retrieves secrets and makes them available to the VM without direct credential exposure in pod specifications.

#### Transport-layer encryption for session communications

All management plane communications across VCF are encrypted using TLS. NSX Manager session communications are encrypted to restrict session hijacking, and NSX authentication mechanisms are resistant to replay attacks. NSX uses mutual TLS authentication based on self-signed or CA certificates between transport nodes (hosts) and the Central Control Plane, providing bidirectional identity verification for infrastructure-level sessions. vCenter API authentication transports credentials over HTTPS with TLS encryption. VCF Operations supports enforcement of TLS for secure communications through the ServerCertificateValidationCallback configuration. VCF Operations for Networks also supports TLS for its communications.

NSX BGP neighbors can be configured with message authentication to apply integrity checks to routing protocol messages, protecting the integrity of routing sessions between network components. NSX remote logging can be configured to use TLS for secure log transport. VCF supports SSH fingerprint verification for remote server connections, allowing administrators to confirm they are connecting to the correct server and providing protection against interception attacks. Metadata exchanged between service providers and identity providers includes information required for establishing trust and facilitating secure communication, supporting session authenticity at the federation layer.

At the Consumption layer, the Kubernetes API server in VMware Kubernetes Service (VKS) clusters on vSphere Supervisor operates exclusively over TLS, establishing authenticated and encrypted sessions between clients and the API server. Secure communications between all Kubernetes control plane services are implemented using certificates. For service-to-service

session integrity within a cluster, Istio is available as a standard package on VKS clusters and provides mutual TLS encryption between services with identity-based authentication and authorization, so that each service authenticates the other before establishing a session. Organizations whose workload sensitivity warrants it should deploy a service mesh to encrypt all intra-cluster communications.

#### API session authentication

NSX Manager supports session-cookie-based API authentication over TLS. Cookie-based authentication is enabled by default, and subsequent API calls use the session cookie and XSRF token for request validation rather than transmitting credentials with each request. VCF Automation uses OAuth access tokens for session management. Device authorization tokens issued by VCF Automation expire in 3600 seconds by default, and API access tokens have their own configurable expiration periods. The access token returned upon successful authentication through VCF Automation's identity provider proxy is valid for 5 minutes, requiring periodic refresh.

At the Consumption layer, the Kubernetes API server in VKS supports client authentication via service account tokens, X.509 client certificates, OIDC tokens, and webhook tokens, depending on the deployment configuration. Audience-aware token authentication validates that a token was specifically issued for the target API server by verifying the token against the server's expected audience identifier, which restricts token reuse across different services. For container registry operations, Harbor Registry (available as a standard package on VKS clusters) uses shared secrets for inter-component communication among its core server, job service, and registry components.

#### Session lifetime controls

Session lifetimes are bounded by configurable timeouts to limit exposure from idle or abandoned sessions. NSX Manager session expiry defaults to 1800 seconds (30 minutes) and can be adjusted using the CLI or API. VCF Operations session inactivity timeout defaults to 30 minutes. VCF Operations for Networks supports user session timeout configuration with values ranging from 15 minutes to 24 hours, defaulting to 15 minutes. The VMware ESX Host Client session timeout controls the duration before an inactive host management session is terminated. The vCenter session timeout for unified VIM/vAPI sessions can be configured through the vpx.conf file or the vSphere Client UI.

At the Consumption layer, Kubernetes guidance recommends setting short lifetimes on certificates and automating their rotation to reduce the window of opportunity for credential exploitation. Using short-lived Kubernetes Secrets for session tokens further limits the impact of credential exposure.

#### PKI infrastructure and cryptographic foundation

The PKI infrastructure that underpins session security offers multiple certificate management options. VCF supports fully automated certificate issuance within the platform, replacement of default certificates with enterprise CA certificates, and integration with Microsoft Active Directory Certificate Services using a restricted service account for least-privilege access. NSX cryptographic modules are validated to FIPS 140-3 standards by the NIST Cryptographic Module Validation Program (CMVP), and NSX is configured to use FIPS 140-3 validated cryptographic modules by default. VCF Operations for Networks uses FIPS-validated cryptographic modules matching those specified by the FIPS 140-2 standard.

VKS clusters on vSphere Supervisor include a set of TLS certificates that support the cryptographic infrastructure for secure communications among cluster components; these certificates can be manually rotated through the VKS certificate management interface when needed. For ingress traffic, the Contour ingress controller is available as a standard package on VKS clusters and uses internally issued certificates with configurable validity periods to secure communications between the Contour controller and its Envoy proxies. Leaf certificates default to 720 hours and the CA certificate defaults to 8,760 hours; both durations are configurable.

#### Data plane session integrity

For data plane sessions, encrypted vSphere vMotion secures the confidentiality, integrity, and authenticity of VM migration traffic. Encrypted vMotion can be set to Opportunistic (use encryption if supported) or Required (do not migrate if unsupported), and vSphere always attempts encryption when migrating encrypted virtual machines. Fault Tolerance (FT)

logging traffic between primary and secondary VMs, which replicates guest network and storage I/O data as well as memory contents, is unencrypted by default and should be isolated on a dedicated network segment; vSphere supports optional encryption of FT log traffic to protect the integrity and confidentiality of this inter-host session channel. vSAN data-in-transit encryption is a cluster-wide setting that, when enabled, encrypts all data and metadata traffic as it transits across hosts and file service inter-host connections. NSX IPSec VPN supports certificate-based authentication for site-to-site VPN sessions, where the site certificate configured on the local endpoint is used for mutual authentication.

#### **VMware vDefend (Contributes)**

VMware vDefend (DFW) and Gateway Firewall provide stateful session tracking that monitors entire conversations between endpoints, enforcing session boundaries and detecting anomalous session behavior. DFW rules can be configured with TCP strict mode (tcp-strict-mode), which enforces connection state validation and drops packets that do not conform to expected TCP state transitions. Session Timers define how long a session is maintained on the firewall after inactivity, with configurable timeouts for TCP, UDP, and ICMP sessions that can be applied to user-defined groups or Tier-0 and Tier-1 gateways.

Security Intelligence analyzes flow data to identify unexpected communications between workloads, enabling exception policy creation to protect required inter-workload sessions. Security Intelligence also detects suspicious behavior associated with remote connection protocols such as telnet, SSH, and VNC.

vDefend TLS Inspection extends session integrity enforcement to encrypted traffic by detecting advanced threats transmitted over encrypted channels. TLS Inspection includes certificate revocation list validation to restrict sessions using revoked certificates. vDefend IDS/IPS detects sessions using obsolete transport protocols by combining port verification with application identification, covering clear-text protocols and outdated transport protocol versions.

Security Services Platform (SSP) establishes its own external communications using TLS handshake negotiation and rejects any certificate or TLS configuration that uses deprecated cryptographic algorithms or older protocol versions. Network Detection and Response (NDR) uses SSL verification for event logs sent over HTTPS.

vDefend does not provide cryptographic end-to-end session authentication or confidentiality between communicating endpoints; those protections require application-layer or infrastructure-layer controls outside vDefend's scope.

#### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) protects the authenticity and integrity of management and replication sessions through several mechanisms. PNR uses TLS certificates and private keys to protect network communication and to authenticate with remote servers. Replication traffic between protected and recovery sites supports optional network encryption.

For service-to-service authentication, the PNR server acquires a holder-of-key SAML token from VCF SSO and sends the token in a cryptographically signed message to remote services. Remote services validate the signature to confirm the identity of the calling service before accepting requests, which helps protect against token reuse across paired sites.

The PNR REST API Gateway enforces explicit session lifecycle controls. Callers authenticate through the Authentication API POST endpoint, which returns a session identifier that must be included as the x-dr-session header on subsequent requests. A DELETE endpoint terminates an authenticated session, and a GET endpoint returns information about the current session if one exists. The Site Pairing REST API uses a POST endpoint to create an authenticated session to a remote PNR server, and a Validate Connection POST operation tests connectivity to a remote appliance before pairing.

PNR records session-related activity in security log files, including login events with timestamps, user identity, session keys, and locale information. Certificate verification errors and connection failures during authentication attempts are also recorded in the log files and in the user interface. The logManager.SessionManager setting controls the verbosity of session management logging, with a default value of verbose. PNR monitors active site-to-site connections by sending periodic ping requests and logs connection check events, so broken connections are detected operationally.

#### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides session integrity mechanisms for database and management-plane communications through certificate-based encryption and strong authentication. Database cluster connections use TLS

certificates for client connectivity and monitoring. DSM stores PEM-encoded certificate authorities in the `ca.crt` property of Kubernetes Secrets for database cluster authentication, and administrators can configure custom CA certificates per database cluster through the DSM UI. The DSM Provider Appliance supports secure TLS connections validated against a trusted CA. DSM also establishes secure connections to VMware vCenter using certificate validation. This certificate validation chain supports session authenticity by confirming that connections reach the intended endpoint before data exchange begins. DSM requires that certificate expiration dates are verified to maintain secure operations, with the alerting system tracking certificate validity as part of its monitoring coverage.

For PostgreSQL databases, DSM uses host-based authentication (HBA) configured through `pg_hba.conf` rules. These rules apply ordered matching based on connection type, client address, requested database name, and username, and the first matching record determines the authentication outcome. DSM supports `scram-sha-256`, `reject`, and `cert` as configurable authentication methods for custom HBA entries, giving administrators control over which connection patterns are accepted or denied. The `scram-sha-256` method is the default for standard user accounts and internal accounts including `pgadmin` and `postgres_exporter`. Administrators can configure additional HBA entries to include LDAP-backed authentication where directory service integration is desired. DSM also supports enabling or disabling directory service authentication on a per-database-instance basis through the database settings interface.

PostgreSQL replication session integrity is maintained through a trust bundle and dedicated replication credentials that authenticate connections between primary and secondary instances. Replication connections support SSL/TLS encryption with a configurable `sslmode` parameter to scope encryption to approved connection modes. For remote replication, users with the DSM user role must manually establish trust between primary and secondary instances before replication begins. For MySQL, DSM verifies network access to the source database and validates SSL trust during replication operations, and controls TLS protocols permitted during group replication recovery connections through the `group_replication_recovery_tls_version` setting.

For SQL Server databases, DSM uses TLS to encrypt communication between clients and SQL Server clusters. An internal management login (`dsm_mgmt`) restricts access to system-level objects, scoping administrative operations within a secure context.

The DSM Consumption Operator requires a TLS certificate for secure communication between the operator and the DSM provider, obtained and saved by the Cloud Administrator as part of operator installation. Metrics forwarding from DSM to VCF Operations supports Mutual TLS (mTLS) authentication, enabled by configuring the Client Certificate Authentication toggle along with the client certificate and private key, extending mutual authentication to management-plane communications. Connection activity logging is available for PostgreSQL through the `log_connections` parameter, which records connection attempts and the successful completion of client authentication, supporting session visibility alongside the integrity controls.

DSM session integrity mechanisms apply to DSM-managed database connections and management-plane communications. Session integrity for non-database services, user interfaces outside DSM, and broader organizational network communications relies on platform-level and organizational controls. Operators are responsible for managing CA certificate trust stores, configuring HBA rules suited to their environment, and applying network-layer session protections beyond what DSM configures directly.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides multiple application-layer mechanisms that contribute to protecting the authenticity and integrity of communications sessions.

TLS termination is the primary session-integrity mechanism for application traffic. Avi Service Engines terminate SSL/TLS sessions on behalf of backend applications, with SSL profiles controlling the allowable cipher suites for each virtual service. The SSL Cipher Strength setting governs the cipher strength used to authenticate applications to clients. During SSL handshake processing, Avi negotiates protocol version and cipher suites to establish authenticated, encrypted sessions. Avi also supports TLS session persistence, which maintains session continuity using TLS session tickets with automated mirroring across Service Engines, without adding processing overhead.

The System-Secure-HTTP application profile secures HTTP traffic by enforcing HTTPS, protecting cookies, managing client IPs, and controlling protocol behavior for virtual services. This provides a consistent session-security baseline for HTTP application workloads and can be assigned to virtual services during advanced setup.

For HTTP application workloads, Avi provides Cross-Site Request Forgery (CSRF) protection through session token management. When a session ID is invalid or absent, Avi creates a new session for the request and restarts the CSRF protection flow from token creation, helping maintain the integrity of application sessions against forged requests.

For internal control-plane communications, Avi establishes a secure TLS session between an Envoy proxy on each Service Engine and the Avi Controller. Secure Channel Certificates for Controller-to-Service Engine communication are configured in the System Settings administration panel under Administration > System Settings.

At the management plane, Avi's User Profile includes a Maximum Concurrent Login Sessions feature that restricts simultaneous active sessions per administrator account to a configurable threshold. When the threshold is reached, existing sessions can be invalidated using the `--clear-sessions` shell command.

For BGP routing sessions, Avi supports MD5 authentication. When BGP authentication is enabled, each TCP segment exchanged between BGP peers is verified and accepted only when authentication succeeds, blocking unauthorized peer session establishment.

In Kubernetes deployments, the Avi Kubernetes Operator (AKO) supports session affinity through the HTTPRule Custom Resource Definition (CRD), which provides cookie-based session persistence for application workloads managed by Avi ingress controllers. Avi also supports MCP load balancing with session affinity based on unique MCP Session IDs, maintaining connection persistence between clients and servers for AI model serving workloads.

In distributed deployments, GSLB Site Persistence directs all requests in a long-lived transaction from a client to the same server, maintaining application session integrity across sites.

Avi requires that all external communication-related authentication failures be logged and persisted for auditing purposes, covering session establishment and authentication events across the platform.

Administrators should configure SSL profiles, cipher suites, CSRF policies, session limits, and the System-Secure-HTTP application profile for each virtual service and user profile based on the sensitivity of the applications and the organization's session security requirements.



## Control CR 3.8(b)

> Session integrity: Protect the integrity of communication sessions at the application layer. Invalidate session identifiers: Capability to invalidate session identifiers upon logout or timeout. Unique session identifiers: Generate unique session identifiers.

**SCF Controls:** NET-09

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides multiple mechanisms to protect the authenticity and integrity of communications sessions across its management interfaces, API endpoints, and infrastructure components.

#### Session identity and token management

VCF's VMware vCenter includes a SessionManager service that provides the interface to the authentication infrastructure on the target server system. Each session is associated with a specific user account for its full duration, and valid session identifiers are used to establish and verify the identity of principals and to establish trust with API endpoints. A session ID acquired from SessionManager is a temporary substitute for username and password, limiting risk to the principal's credentials. The Common Infrastructure Services (CIS) Session service provides additional token management capabilities, including creation, deletion, and retrieval of session information. The client API supports session maintenance through HTTP cookies via the SESSION\_MAINTAIN\_PROPERTY setting, and a session preservator capability saves user sessions and acquires persistent tokens to preserve identity across reconnections. Administrators should define Release Session Requests when using session authentication to properly log out sessions and reduce the risk of session leaks. Concurrent session monitoring is recommended to detect excessive session creation that could indicate unauthorized activity or resource exhaustion.

At the Consumption layer, the Kubernetes Secrets API provides a mechanism for storing and distributing session tokens and other short-lived credentials to workloads without exposing them in pod specifications. The operator pattern allows a controller to fetch short-lived session tokens from an external service and create Secrets that pods consume, keeping token issuance and refresh logic separate from pod implementations. Using short-lived Secrets limits the exposure window when credential data is inadvertently disclosed. For workloads running as VMs through VM Service, secrets can be injected via a disk-based sidecar mechanism, where a trusted pod retrieves secrets and makes them available to the VM without direct credential exposure in pod specifications.

#### Transport-layer encryption for session communications

All management plane communications across VCF are encrypted using TLS. NSX Manager session communications are encrypted to restrict session hijacking, and NSX authentication mechanisms are resistant to replay attacks. NSX uses mutual TLS authentication based on self-signed or CA certificates between transport nodes (hosts) and the Central Control Plane, providing bidirectional identity verification for infrastructure-level sessions. vCenter API authentication transports credentials over HTTPS with TLS encryption. VCF Operations supports enforcement of TLS for secure communications through the ServerCertificateValidationCallback configuration. VCF Operations for Networks also supports TLS for its communications.

NSX BGP neighbors can be configured with message authentication to apply integrity checks to routing protocol messages, protecting the integrity of routing sessions between network components. NSX remote logging can be configured to use TLS for secure log transport. VCF supports SSH fingerprint verification for remote server connections, allowing administrators to confirm they are connecting to the correct server and providing protection against interception attacks. Metadata exchanged between service providers and identity providers includes information required for establishing trust and facilitating secure communication, supporting session authenticity at the federation layer.

At the Consumption layer, the Kubernetes API server in VMware Kubernetes Service (VKS) clusters on vSphere Supervisor operates exclusively over TLS, establishing authenticated and encrypted sessions between clients and the API server. Secure communications between all Kubernetes control plane services are implemented using certificates. For service-to-service

session integrity within a cluster, Istio is available as a standard package on VKS clusters and provides mutual TLS encryption between services with identity-based authentication and authorization, so that each service authenticates the other before establishing a session. Organizations whose workload sensitivity warrants it should deploy a service mesh to encrypt all intra-cluster communications.

#### API session authentication

NSX Manager supports session-cookie-based API authentication over TLS. Cookie-based authentication is enabled by default, and subsequent API calls use the session cookie and XSRF token for request validation rather than transmitting credentials with each request. VCF Automation uses OAuth access tokens for session management. Device authorization tokens issued by VCF Automation expire in 3600 seconds by default, and API access tokens have their own configurable expiration periods. The access token returned upon successful authentication through VCF Automation's identity provider proxy is valid for 5 minutes, requiring periodic refresh.

At the Consumption layer, the Kubernetes API server in VKS supports client authentication via service account tokens, X.509 client certificates, OIDC tokens, and webhook tokens, depending on the deployment configuration. Audience-aware token authentication validates that a token was specifically issued for the target API server by verifying the token against the server's expected audience identifier, which restricts token reuse across different services. For container registry operations, Harbor Registry (available as a standard package on VKS clusters) uses shared secrets for inter-component communication among its core server, job service, and registry components.

#### Session lifetime controls

Session lifetimes are bounded by configurable timeouts to limit exposure from idle or abandoned sessions. NSX Manager session expiry defaults to 1800 seconds (30 minutes) and can be adjusted using the CLI or API. VCF Operations session inactivity timeout defaults to 30 minutes. VCF Operations for Networks supports user session timeout configuration with values ranging from 15 minutes to 24 hours, defaulting to 15 minutes. The VMware ESX Host Client session timeout controls the duration before an inactive host management session is terminated. The vCenter session timeout for unified VIM/vAPI sessions can be configured through the vpx.conf file or the vSphere Client UI.

At the Consumption layer, Kubernetes guidance recommends setting short lifetimes on certificates and automating their rotation to reduce the window of opportunity for credential exploitation. Using short-lived Kubernetes Secrets for session tokens further limits the impact of credential exposure.

#### PKI infrastructure and cryptographic foundation

The PKI infrastructure that underpins session security offers multiple certificate management options. VCF supports fully automated certificate issuance within the platform, replacement of default certificates with enterprise CA certificates, and integration with Microsoft Active Directory Certificate Services using a restricted service account for least-privilege access. NSX cryptographic modules are validated to FIPS 140-3 standards by the NIST Cryptographic Module Validation Program (CMVP), and NSX is configured to use FIPS 140-3 validated cryptographic modules by default. VCF Operations for Networks uses FIPS-validated cryptographic modules matching those specified by the FIPS 140-2 standard.

VKS clusters on vSphere Supervisor include a set of TLS certificates that support the cryptographic infrastructure for secure communications among cluster components; these certificates can be manually rotated through the VKS certificate management interface when needed. For ingress traffic, the Contour ingress controller is available as a standard package on VKS clusters and uses internally issued certificates with configurable validity periods to secure communications between the Contour controller and its Envoy proxies. Leaf certificates default to 720 hours and the CA certificate defaults to 8,760 hours; both durations are configurable.

#### Data plane session integrity

For data plane sessions, encrypted vSphere vMotion secures the confidentiality, integrity, and authenticity of VM migration traffic. Encrypted vMotion can be set to Opportunistic (use encryption if supported) or Required (do not migrate if unsupported), and vSphere always attempts encryption when migrating encrypted virtual machines. Fault Tolerance (FT)

logging traffic between primary and secondary VMs, which replicates guest network and storage I/O data as well as memory contents, is unencrypted by default and should be isolated on a dedicated network segment; vSphere supports optional encryption of FT log traffic to protect the integrity and confidentiality of this inter-host session channel. vSAN data-in-transit encryption is a cluster-wide setting that, when enabled, encrypts all data and metadata traffic as it transits across hosts and file service inter-host connections. NSX IPSec VPN supports certificate-based authentication for site-to-site VPN sessions, where the site certificate configured on the local endpoint is used for mutual authentication.

#### **VMware vDefend (Contributes)**

VMware vDefend (DFW) and Gateway Firewall provide stateful session tracking that monitors entire conversations between endpoints, enforcing session boundaries and detecting anomalous session behavior. DFW rules can be configured with TCP strict mode (tcp-strict-mode), which enforces connection state validation and drops packets that do not conform to expected TCP state transitions. Session Timers define how long a session is maintained on the firewall after inactivity, with configurable timeouts for TCP, UDP, and ICMP sessions that can be applied to user-defined groups or Tier-0 and Tier-1 gateways.

Security Intelligence analyzes flow data to identify unexpected communications between workloads, enabling exception policy creation to protect required inter-workload sessions. Security Intelligence also detects suspicious behavior associated with remote connection protocols such as telnet, SSH, and VNC.

vDefend TLS Inspection extends session integrity enforcement to encrypted traffic by detecting advanced threats transmitted over encrypted channels. TLS Inspection includes certificate revocation list validation to restrict sessions using revoked certificates. vDefend IDS/IPS detects sessions using obsolete transport protocols by combining port verification with application identification, covering clear-text protocols and outdated transport protocol versions.

Security Services Platform (SSP) establishes its own external communications using TLS handshake negotiation and rejects any certificate or TLS configuration that uses deprecated cryptographic algorithms or older protocol versions. Network Detection and Response (NDR) uses SSL verification for event logs sent over HTTPS.

vDefend does not provide cryptographic end-to-end session authentication or confidentiality between communicating endpoints; those protections require application-layer or infrastructure-layer controls outside vDefend's scope.

#### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) protects the authenticity and integrity of management and replication sessions through several mechanisms. PNR uses TLS certificates and private keys to protect network communication and to authenticate with remote servers. Replication traffic between protected and recovery sites supports optional network encryption.

For service-to-service authentication, the PNR server acquires a holder-of-key SAML token from VCF SSO and sends the token in a cryptographically signed message to remote services. Remote services validate the signature to confirm the identity of the calling service before accepting requests, which helps protect against token reuse across paired sites.

The PNR REST API Gateway enforces explicit session lifecycle controls. Callers authenticate through the Authentication API POST endpoint, which returns a session identifier that must be included as the x-dr-session header on subsequent requests. A DELETE endpoint terminates an authenticated session, and a GET endpoint returns information about the current session if one exists. The Site Pairing REST API uses a POST endpoint to create an authenticated session to a remote PNR server, and a Validate Connection POST operation tests connectivity to a remote appliance before pairing.

PNR records session-related activity in security log files, including login events with timestamps, user identity, session keys, and locale information. Certificate verification errors and connection failures during authentication attempts are also recorded in the log files and in the user interface. The logManager.SessionManager setting controls the verbosity of session management logging, with a default value of verbose. PNR monitors active site-to-site connections by sending periodic ping requests and logs connection check events, so broken connections are detected operationally.

#### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides session integrity mechanisms for database and management-plane communications through certificate-based encryption and strong authentication. Database cluster connections use TLS

certificates for client connectivity and monitoring. DSM stores PEM-encoded certificate authorities in the `ca.crt` property of Kubernetes Secrets for database cluster authentication, and administrators can configure custom CA certificates per database cluster through the DSM UI. The DSM Provider Appliance supports secure TLS connections validated against a trusted CA. DSM also establishes secure connections to VMware vCenter using certificate validation. This certificate validation chain supports session authenticity by confirming that connections reach the intended endpoint before data exchange begins. DSM requires that certificate expiration dates are verified to maintain secure operations, with the alerting system tracking certificate validity as part of its monitoring coverage.

For PostgreSQL databases, DSM uses host-based authentication (HBA) configured through `pg_hba.conf` rules. These rules apply ordered matching based on connection type, client address, requested database name, and username, and the first matching record determines the authentication outcome. DSM supports `scram-sha-256`, `reject`, and `cert` as configurable authentication methods for custom HBA entries, giving administrators control over which connection patterns are accepted or denied. The `scram-sha-256` method is the default for standard user accounts and internal accounts including `pgadmin` and `postgres_exporter`. Administrators can configure additional HBA entries to include LDAP-backed authentication where directory service integration is desired. DSM also supports enabling or disabling directory service authentication on a per-database-instance basis through the database settings interface.

PostgreSQL replication session integrity is maintained through a trust bundle and dedicated replication credentials that authenticate connections between primary and secondary instances. Replication connections support SSL/TLS encryption with a configurable `sslmode` parameter to scope encryption to approved connection modes. For remote replication, users with the DSM user role must manually establish trust between primary and secondary instances before replication begins. For MySQL, DSM verifies network access to the source database and validates SSL trust during replication operations, and controls TLS protocols permitted during group replication recovery connections through the `group_replication_recovery_tls_version` setting.

For SQL Server databases, DSM uses TLS to encrypt communication between clients and SQL Server clusters. An internal management login (`dsm_mgmt`) restricts access to system-level objects, scoping administrative operations within a secure context.

The DSM Consumption Operator requires a TLS certificate for secure communication between the operator and the DSM provider, obtained and saved by the Cloud Administrator as part of operator installation. Metrics forwarding from DSM to VCF Operations supports Mutual TLS (mTLS) authentication, enabled by configuring the Client Certificate Authentication toggle along with the client certificate and private key, extending mutual authentication to management-plane communications. Connection activity logging is available for PostgreSQL through the `log_connections` parameter, which records connection attempts and the successful completion of client authentication, supporting session visibility alongside the integrity controls.

DSM session integrity mechanisms apply to DSM-managed database connections and management-plane communications. Session integrity for non-database services, user interfaces outside DSM, and broader organizational network communications relies on platform-level and organizational controls. Operators are responsible for managing CA certificate trust stores, configuring HBA rules suited to their environment, and applying network-layer session protections beyond what DSM configures directly.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides multiple application-layer mechanisms that contribute to protecting the authenticity and integrity of communications sessions.

TLS termination is the primary session-integrity mechanism for application traffic. Avi Service Engines terminate SSL/TLS sessions on behalf of backend applications, with SSL profiles controlling the allowable cipher suites for each virtual service. The SSL Cipher Strength setting governs the cipher strength used to authenticate applications to clients. During SSL handshake processing, Avi negotiates protocol version and cipher suites to establish authenticated, encrypted sessions. Avi also supports TLS session persistence, which maintains session continuity using TLS session tickets with automated mirroring across Service Engines, without adding processing overhead.

The System-Secure-HTTP application profile secures HTTP traffic by enforcing HTTPS, protecting cookies, managing client IPs, and controlling protocol behavior for virtual services. This provides a consistent session-security baseline for HTTP application workloads and can be assigned to virtual services during advanced setup.

For HTTP application workloads, Avi provides Cross-Site Request Forgery (CSRF) protection through session token management. When a session ID is invalid or absent, Avi creates a new session for the request and restarts the CSRF protection flow from token creation, helping maintain the integrity of application sessions against forged requests.

For internal control-plane communications, Avi establishes a secure TLS session between an Envoy proxy on each Service Engine and the Avi Controller. Secure Channel Certificates for Controller-to-Service Engine communication are configured in the System Settings administration panel under Administration > System Settings.

At the management plane, Avi's User Profile includes a Maximum Concurrent Login Sessions feature that restricts simultaneous active sessions per administrator account to a configurable threshold. When the threshold is reached, existing sessions can be invalidated using the `--clear-sessions` shell command.

For BGP routing sessions, Avi supports MD5 authentication. When BGP authentication is enabled, each TCP segment exchanged between BGP peers is verified and accepted only when authentication succeeds, blocking unauthorized peer session establishment.

In Kubernetes deployments, the Avi Kubernetes Operator (AKO) supports session affinity through the HTTPRule Custom Resource Definition (CRD), which provides cookie-based session persistence for application workloads managed by Avi ingress controllers. Avi also supports MCP load balancing with session affinity based on unique MCP Session IDs, maintaining connection persistence between clients and servers for AI model serving workloads.

In distributed deployments, GSLB Site Persistence directs all requests in a long-lived transaction from a client to the same server, maintaining application session integrity across sites.

Avi requires that all external communication-related authentication failures be logged and persisted for auditing purposes, covering session establishment and authentication events across the platform.

Administrators should configure SSL profiles, cipher suites, CSRF policies, session limits, and the System-Secure-HTTP application profile for each virtual service and user profile based on the sensitivity of the applications and the organization's session security requirements.

## Control CR 3.8(c)

> Session integrity: Protect the integrity of communication sessions at the application layer. Invalidate session identifiers: Capability to invalidate session identifiers upon logout or timeout. Unique session identifiers: Generate unique session identifiers.

**SCF Controls:** NET-09

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides multiple mechanisms to protect the authenticity and integrity of communications sessions across its management interfaces, API endpoints, and infrastructure components.

#### Session identity and token management

VCF's VMware vCenter includes a SessionManager service that provides the interface to the authentication infrastructure on the target server system. Each session is associated with a specific user account for its full duration, and valid session identifiers are used to establish and verify the identity of principals and to establish trust with API endpoints. A session ID acquired from SessionManager is a temporary substitute for username and password, limiting risk to the principal's credentials. The Common Infrastructure Services (CIS) Session service provides additional token management capabilities, including creation, deletion, and retrieval of session information. The client API supports session maintenance through HTTP cookies via the SESSION\_MAINTAIN\_PROPERTY setting, and a session preservator capability saves user sessions and acquires persistent tokens to preserve identity across reconnections. Administrators should define Release Session Requests when using session authentication to properly log out sessions and reduce the risk of session leaks. Concurrent session monitoring is recommended to detect excessive session creation that could indicate unauthorized activity or resource exhaustion.

At the Consumption layer, the Kubernetes Secrets API provides a mechanism for storing and distributing session tokens and other short-lived credentials to workloads without exposing them in pod specifications. The operator pattern allows a controller to fetch short-lived session tokens from an external service and create Secrets that pods consume, keeping token issuance and refresh logic separate from pod implementations. Using short-lived Secrets limits the exposure window when credential data is inadvertently disclosed. For workloads running as VMs through VM Service, secrets can be injected via a disk-based sidecar mechanism, where a trusted pod retrieves secrets and makes them available to the VM without direct credential exposure in pod specifications.

#### Transport-layer encryption for session communications

All management plane communications across VCF are encrypted using TLS. NSX Manager session communications are encrypted to restrict session hijacking, and NSX authentication mechanisms are resistant to replay attacks. NSX uses mutual TLS authentication based on self-signed or CA certificates between transport nodes (hosts) and the Central Control Plane, providing bidirectional identity verification for infrastructure-level sessions. vCenter API authentication transports credentials over HTTPS with TLS encryption. VCF Operations supports enforcement of TLS for secure communications through the ServerCertificateValidationCallback configuration. VCF Operations for Networks also supports TLS for its communications.

NSX BGP neighbors can be configured with message authentication to apply integrity checks to routing protocol messages, protecting the integrity of routing sessions between network components. NSX remote logging can be configured to use TLS for secure log transport. VCF supports SSH fingerprint verification for remote server connections, allowing administrators to confirm they are connecting to the correct server and providing protection against interception attacks. Metadata exchanged between service providers and identity providers includes information required for establishing trust and facilitating secure communication, supporting session authenticity at the federation layer.

At the Consumption layer, the Kubernetes API server in VMware Kubernetes Service (VKS) clusters on vSphere Supervisor operates exclusively over TLS, establishing authenticated and encrypted sessions between clients and the API server. Secure communications between all Kubernetes control plane services are implemented using certificates. For service-to-service



session integrity within a cluster, Istio is available as a standard package on VKS clusters and provides mutual TLS encryption between services with identity-based authentication and authorization, so that each service authenticates the other before establishing a session. Organizations whose workload sensitivity warrants it should deploy a service mesh to encrypt all intra-cluster communications.

#### API session authentication

NSX Manager supports session-cookie-based API authentication over TLS. Cookie-based authentication is enabled by default, and subsequent API calls use the session cookie and XSRF token for request validation rather than transmitting credentials with each request. VCF Automation uses OAuth access tokens for session management. Device authorization tokens issued by VCF Automation expire in 3600 seconds by default, and API access tokens have their own configurable expiration periods. The access token returned upon successful authentication through VCF Automation's identity provider proxy is valid for 5 minutes, requiring periodic refresh.

At the Consumption layer, the Kubernetes API server in VKS supports client authentication via service account tokens, X.509 client certificates, OIDC tokens, and webhook tokens, depending on the deployment configuration. Audience-aware token authentication validates that a token was specifically issued for the target API server by verifying the token against the server's expected audience identifier, which restricts token reuse across different services. For container registry operations, Harbor Registry (available as a standard package on VKS clusters) uses shared secrets for inter-component communication among its core server, job service, and registry components.

#### Session lifetime controls

Session lifetimes are bounded by configurable timeouts to limit exposure from idle or abandoned sessions. NSX Manager session expiry defaults to 1800 seconds (30 minutes) and can be adjusted using the CLI or API. VCF Operations session inactivity timeout defaults to 30 minutes. VCF Operations for Networks supports user session timeout configuration with values ranging from 15 minutes to 24 hours, defaulting to 15 minutes. The VMware ESX Host Client session timeout controls the duration before an inactive host management session is terminated. The vCenter session timeout for unified VIM/vAPI sessions can be configured through the vpx.conf file or the vSphere Client UI.

At the Consumption layer, Kubernetes guidance recommends setting short lifetimes on certificates and automating their rotation to reduce the window of opportunity for credential exploitation. Using short-lived Kubernetes Secrets for session tokens further limits the impact of credential exposure.

#### PKI infrastructure and cryptographic foundation

The PKI infrastructure that underpins session security offers multiple certificate management options. VCF supports fully automated certificate issuance within the platform, replacement of default certificates with enterprise CA certificates, and integration with Microsoft Active Directory Certificate Services using a restricted service account for least-privilege access. NSX cryptographic modules are validated to FIPS 140-3 standards by the NIST Cryptographic Module Validation Program (CMVP), and NSX is configured to use FIPS 140-3 validated cryptographic modules by default. VCF Operations for Networks uses FIPS-validated cryptographic modules matching those specified by the FIPS 140-2 standard.

VKS clusters on vSphere Supervisor include a set of TLS certificates that support the cryptographic infrastructure for secure communications among cluster components; these certificates can be manually rotated through the VKS certificate management interface when needed. For ingress traffic, the Contour ingress controller is available as a standard package on VKS clusters and uses internally issued certificates with configurable validity periods to secure communications between the Contour controller and its Envoy proxies. Leaf certificates default to 720 hours and the CA certificate defaults to 8,760 hours; both durations are configurable.

#### Data plane session integrity

For data plane sessions, encrypted vSphere vMotion secures the confidentiality, integrity, and authenticity of VM migration traffic. Encrypted vMotion can be set to Opportunistic (use encryption if supported) or Required (do not migrate if unsupported), and vSphere always attempts encryption when migrating encrypted virtual machines. Fault Tolerance (FT)



logging traffic between primary and secondary VMs, which replicates guest network and storage I/O data as well as memory contents, is unencrypted by default and should be isolated on a dedicated network segment; vSphere supports optional encryption of FT log traffic to protect the integrity and confidentiality of this inter-host session channel. vSAN data-in-transit encryption is a cluster-wide setting that, when enabled, encrypts all data and metadata traffic as it transits across hosts and file service inter-host connections. NSX IPSec VPN supports certificate-based authentication for site-to-site VPN sessions, where the site certificate configured on the local endpoint is used for mutual authentication.

#### **VMware vDefend (Contributes)**

VMware vDefend (DFW) and Gateway Firewall provide stateful session tracking that monitors entire conversations between endpoints, enforcing session boundaries and detecting anomalous session behavior. DFW rules can be configured with TCP strict mode (tcp-strict-mode), which enforces connection state validation and drops packets that do not conform to expected TCP state transitions. Session Timers define how long a session is maintained on the firewall after inactivity, with configurable timeouts for TCP, UDP, and ICMP sessions that can be applied to user-defined groups or Tier-0 and Tier-1 gateways.

Security Intelligence analyzes flow data to identify unexpected communications between workloads, enabling exception policy creation to protect required inter-workload sessions. Security Intelligence also detects suspicious behavior associated with remote connection protocols such as telnet, SSH, and VNC.

vDefend TLS Inspection extends session integrity enforcement to encrypted traffic by detecting advanced threats transmitted over encrypted channels. TLS Inspection includes certificate revocation list validation to restrict sessions using revoked certificates. vDefend IDS/IPS detects sessions using obsolete transport protocols by combining port verification with application identification, covering clear-text protocols and outdated transport protocol versions.

Security Services Platform (SSP) establishes its own external communications using TLS handshake negotiation and rejects any certificate or TLS configuration that uses deprecated cryptographic algorithms or older protocol versions. Network Detection and Response (NDR) uses SSL verification for event logs sent over HTTPS.

vDefend does not provide cryptographic end-to-end session authentication or confidentiality between communicating endpoints; those protections require application-layer or infrastructure-layer controls outside vDefend's scope.

#### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) protects the authenticity and integrity of management and replication sessions through several mechanisms. PNR uses TLS certificates and private keys to protect network communication and to authenticate with remote servers. Replication traffic between protected and recovery sites supports optional network encryption.

For service-to-service authentication, the PNR server acquires a holder-of-key SAML token from VCF SSO and sends the token in a cryptographically signed message to remote services. Remote services validate the signature to confirm the identity of the calling service before accepting requests, which helps protect against token reuse across paired sites.

The PNR REST API Gateway enforces explicit session lifecycle controls. Callers authenticate through the Authentication API POST endpoint, which returns a session identifier that must be included as the x-dr-session header on subsequent requests. A DELETE endpoint terminates an authenticated session, and a GET endpoint returns information about the current session if one exists. The Site Pairing REST API uses a POST endpoint to create an authenticated session to a remote PNR server, and a Validate Connection POST operation tests connectivity to a remote appliance before pairing.

PNR records session-related activity in security log files, including login events with timestamps, user identity, session keys, and locale information. Certificate verification errors and connection failures during authentication attempts are also recorded in the log files and in the user interface. The logManager.SessionManager setting controls the verbosity of session management logging, with a default value of verbose. PNR monitors active site-to-site connections by sending periodic ping requests and logs connection check events, so broken connections are detected operationally.

#### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides session integrity mechanisms for database and management-plane communications through certificate-based encryption and strong authentication. Database cluster connections use TLS

certificates for client connectivity and monitoring. DSM stores PEM-encoded certificate authorities in the `ca.crt` property of Kubernetes Secrets for database cluster authentication, and administrators can configure custom CA certificates per database cluster through the DSM UI. The DSM Provider Appliance supports secure TLS connections validated against a trusted CA. DSM also establishes secure connections to VMware vCenter using certificate validation. This certificate validation chain supports session authenticity by confirming that connections reach the intended endpoint before data exchange begins. DSM requires that certificate expiration dates are verified to maintain secure operations, with the alerting system tracking certificate validity as part of its monitoring coverage.

For PostgreSQL databases, DSM uses host-based authentication (HBA) configured through `pg_hba.conf` rules. These rules apply ordered matching based on connection type, client address, requested database name, and username, and the first matching record determines the authentication outcome. DSM supports `scram-sha-256`, `reject`, and `cert` as configurable authentication methods for custom HBA entries, giving administrators control over which connection patterns are accepted or denied. The `scram-sha-256` method is the default for standard user accounts and internal accounts including `pgadmin` and `postgres_exporter`. Administrators can configure additional HBA entries to include LDAP-backed authentication where directory service integration is desired. DSM also supports enabling or disabling directory service authentication on a per-database-instance basis through the database settings interface.

PostgreSQL replication session integrity is maintained through a trust bundle and dedicated replication credentials that authenticate connections between primary and secondary instances. Replication connections support SSL/TLS encryption with a configurable `sslmode` parameter to scope encryption to approved connection modes. For remote replication, users with the DSM user role must manually establish trust between primary and secondary instances before replication begins. For MySQL, DSM verifies network access to the source database and validates SSL trust during replication operations, and controls TLS protocols permitted during group replication recovery connections through the `group_replication_recovery_tls_version` setting.

For SQL Server databases, DSM uses TLS to encrypt communication between clients and SQL Server clusters. An internal management login (`dsm_mgmt`) restricts access to system-level objects, scoping administrative operations within a secure context.

The DSM Consumption Operator requires a TLS certificate for secure communication between the operator and the DSM provider, obtained and saved by the Cloud Administrator as part of operator installation. Metrics forwarding from DSM to VCF Operations supports Mutual TLS (mTLS) authentication, enabled by configuring the Client Certificate Authentication toggle along with the client certificate and private key, extending mutual authentication to management-plane communications. Connection activity logging is available for PostgreSQL through the `log_connections` parameter, which records connection attempts and the successful completion of client authentication, supporting session visibility alongside the integrity controls.

DSM session integrity mechanisms apply to DSM-managed database connections and management-plane communications. Session integrity for non-database services, user interfaces outside DSM, and broader organizational network communications relies on platform-level and organizational controls. Operators are responsible for managing CA certificate trust stores, configuring HBA rules suited to their environment, and applying network-layer session protections beyond what DSM configures directly.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides multiple application-layer mechanisms that contribute to protecting the authenticity and integrity of communications sessions.

TLS termination is the primary session-integrity mechanism for application traffic. Avi Service Engines terminate SSL/TLS sessions on behalf of backend applications, with SSL profiles controlling the allowable cipher suites for each virtual service. The SSL Cipher Strength setting governs the cipher strength used to authenticate applications to clients. During SSL handshake processing, Avi negotiates protocol version and cipher suites to establish authenticated, encrypted sessions. Avi also supports TLS session persistence, which maintains session continuity using TLS session tickets with automated mirroring across Service Engines, without adding processing overhead.

The System-Secure-HTTP application profile secures HTTP traffic by enforcing HTTPS, protecting cookies, managing client IPs, and controlling protocol behavior for virtual services. This provides a consistent session-security baseline for HTTP application workloads and can be assigned to virtual services during advanced setup.

For HTTP application workloads, Avi provides Cross-Site Request Forgery (CSRF) protection through session token management. When a session ID is invalid or absent, Avi creates a new session for the request and restarts the CSRF protection flow from token creation, helping maintain the integrity of application sessions against forged requests.

For internal control-plane communications, Avi establishes a secure TLS session between an Envoy proxy on each Service Engine and the Avi Controller. Secure Channel Certificates for Controller-to-Service Engine communication are configured in the System Settings administration panel under Administration > System Settings.

At the management plane, Avi's User Profile includes a Maximum Concurrent Login Sessions feature that restricts simultaneous active sessions per administrator account to a configurable threshold. When the threshold is reached, existing sessions can be invalidated using the `--clear-sessions` shell command.

For BGP routing sessions, Avi supports MD5 authentication. When BGP authentication is enabled, each TCP segment exchanged between BGP peers is verified and accepted only when authentication succeeds, blocking unauthorized peer session establishment.

In Kubernetes deployments, the Avi Kubernetes Operator (AKO) supports session affinity through the HTTPRule Custom Resource Definition (CRD), which provides cookie-based session persistence for application workloads managed by Avi ingress controllers. Avi also supports MCP load balancing with session affinity based on unique MCP Session IDs, maintaining connection persistence between clients and servers for AI model serving workloads.

In distributed deployments, GSLB Site Persistence directs all requests in a long-lived transaction from a client to the same server, maintaining application session integrity across sites.

Avi requires that all external communication-related authentication failures be logged and persisted for auditing purposes, covering session establishment and authentication events across the platform.

Administrators should configure SSL profiles, cipher suites, CSRF policies, session limits, and the System-Secure-HTTP application profile for each virtual service and user profile based on the sensitivity of the applications and the organization's session security requirements.

## Control CR 3.9

> Audit storage capacity: Ensure sufficient storage for audit records and define behavior when full. Warning on capacity: Provide a warning when audit storage reaches a defined threshold.

**SCF Controls:** MON-08

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides several technical mechanisms that help protect event logs from unauthorized access, modification, and deletion, though full satisfaction of this control requires organizational policies and additional tooling beyond the platform itself.

Remote syslog forwarding is the primary mechanism for protecting log integrity across the VCF stack. VMware ESX hosts can be configured to send syslog messages to a remote log server using the `esxcli system syslog config set` command, specifying the target server IP and port. The `Syslog.global.auditRecord.remote` parameter specifically controls whether ESX audit records are forwarded to the remote hosts defined by `Syslog.global.logHost`, providing a dedicated configuration knob for audit record remote transmission separate from general message forwarding. The NSX networking components send audit logs directly to the configured remote syslog server, separating them from the local syslog file and storing them in `/var/log/audit/audit.log` on the local host. NSX API audit logging is always enabled and cannot be disabled, providing a persistent record of all API-driven administrative actions. VCF Log Management can be configured to pull tasks, events, and alerts from VMware vCenter instances and receive syslog feeds from ESX hosts, acting as a centralized log aggregation point within the VCF environment. VCF Operations itself integrates its audit log with syslog and supports forwarding user activity audit logs to an external syslog server. VCF Operations HCX similarly supports configuring a remote syslog server so that logs generated on both the HCX manager and appliance virtual machines are forwarded off-host.

Log forwarding supports encrypted transport to protect log data in transit. NSX Manager syslog supports the TLS protocol and requires port 6514 to be open on both the NSX Manager and the remote log server for TLS-encrypted log transmission. The vCenter syslog forwarding API supports TCP, UDP, TLS, and RELP protocols, and Foundation Load Balancer syslog communication requires a certificate for secure log forwarding. vCenter event streaming to a remote syslog server is separately enabled through the `vpzd.event.syslog.enabled` option in the vCenter Management Interface, providing a dedicated control for event export independent of the syslog API.

By forwarding logs to remote collection infrastructure, VCF reduces the risk that a compromised host or component could modify or delete its own log records. The local copies remain on each component, but the remote copies provide an independent record that local administrators cannot alter.

Access controls on log data are handled through the role-based access model within each VCF component. Access to VCF Log Management is governed by role-based access controls, restricting who can view and manage log data. Within VCF Operations for Networks, only users with the Admin role can view audit logs and configure whether personally identifiable information is included in those logs. For Windows-based domain controllers in the environment, read-only security log access can be enabled, and membership in the Event Log Readers group governs which accounts have permission to read event logs. The NSX documentation notes that core files and audit logs may contain sensitive information such as passwords or encryption keys.

VCF Operations provides an Audit Events page accessible through the Security menu, where administrators can review audit records that capture interactions within the platform including searches, logins, logouts, capability checks, and configuration modifications. Audit logs across VCF Operations capture actions from the API, user interface, and CLI, including regular create, read, update, and delete operations as well as login and logout events. VCF Operations audit data can be filtered by user ID, user name, authentication source, session, client IP, category, and message, supporting targeted review of specific activities. VCF Operations for Networks audit logs similarly capture API, UI, and CLI actions. These audit records include structured metadata such as timestamps, thread identifiers, source IP addresses, log levels, and component class names.

NSX audit logs use a structured syslog format with an audit flag (`audit="true"`) in the structured-data field to distinguish security-relevant events from operational messages. Dedicated log files separate audit records: `nsx-audit.log` contains all

messages with the audit flag, while `nsx-audit-write.log` contains messages flagged as both audit and update operations. NSX audit log messages include metadata fields such as component identifier, log level, request ID, split ID, and subcomponent classification, and each API-associated audit log entry includes a request ID parameter (`req-id`) to identify a specific API call. NSX logs all auditable events to support compliance requirements.

The Consumption layer of VCF, comprising VMware Kubernetes Service (VKS), vSphere Supervisor, and vSphere namespaces, adds its own audit logging mechanisms that extend event log protection to the Kubernetes control plane. The Kubernetes API server generates a security-relevant, chronological audit log documenting all actions in the cluster, including activities from users, applications using the Kubernetes API, and the control plane itself. Each audit event in the `audit.k8s.io` API group carries structured fields including `apiVersion`, `kind`, `level`, `auditID`, and `stage`, along with an `AuthenticationMetadata` block that records impersonation constraints when constrained impersonation was used to authorize a request. Audit event delivery is configured through the `kube-apiserver --audit-log-path` and `--audit-log-mode` parameters; the delivery mode defaults to blocking, which writes each event synchronously before the API server returns a response, reducing the risk of event loss under load. An audit policy file, mounted at `/etc/kubernetes/audit-policy.yaml`, governs the scope of the audit stream by defining which event types are recorded at which detail level; if the audit logging flag is omitted from the policy, no events are logged. Audit event truncation is disabled by default and must be explicitly enabled, so audit records are written in their entirety unless an administrator opts into size-based truncation. Backends include local log files written to `/var/log/kubernetes/audit/` and webhook endpoints for forwarding to external systems. Pod Security Standards (PSA) audit mode, when enabled on a VKS cluster, records security policy violations as audit annotations in the audit log stream, extending audit coverage to workload-level security events. Kubernetes seccomp profiles using the `SCMP_ACT_LOG` action allow individual syscalls of a process to be logged to syslog or auditd, providing fine-grained syscall-level audit recording for sensitive workloads.

Fluent Bit agents deployed by VCF in the `vmware-system-logging` namespace on VKS clusters collect and forward logs to external destinations, with access to that namespace restricted to cluster administrators. Fluent Bit processes Kubernetes API server audit logs separately from container logs and Linux daemon logs when forwarding to VCF Log Management, maintaining the distinction of audit records in the forwarded stream. On vSphere Supervisor control plane nodes, Fluent Bit collects `kube-apiserver` audit logs from `/var/log/vmware/audit/kube-apiserver.log` using the tag `audit.apiserver.*` and stores them in a local database at `/var/log/vmware/fluentbit/flb_audit_apiserver` before forwarding; on VKS worker cluster nodes, audit logs are collected in JSON format from `/var/log/kubernetes/kube-apiserver.log` with a 50 MB memory buffer limit. Fluent Bit also collects Linux authentication logs and container runtime interface logs from VKS cluster nodes, extending the set of log types forwarded to external destinations. The Supervisor can be configured to preserve the source IP addresses of authentication and Kubernetes API server requests for later inspection in audit logs, supporting attribution of actions to specific network origins. Supported forwarding targets include VCF Log Management, Syslog servers, Elasticsearch, Kafka, and Splunk. For workloads running in vSphere Namespaces as vSphere Pods, the `LogAgentConfiguration` custom resource can be applied to a namespace to stream syslog messages to a remote Syslog server; each forwarded message includes metadata fields for namespace name, pod UID, pod name, container name, and ESX host name. By routing audit records to an external log management system, the platform creates a copy of the audit trail outside the cluster, so that a compromise or administrative action affecting the cluster does not also destroy audit evidence.

To fully satisfy this control, organizations deploying VCF should implement log forwarding to a dedicated, hardened log management or SIEM platform where retention, integrity verification, and access controls can be managed independently of the VCF infrastructure. VCF does not provide immutable or tamper-evident log storage on its own; protection against log modification or deletion at the storage layer requires integration with a write-once-read-many (WORM) log repository or equivalent storage. For high-assurance environments, Kubernetes guidance recommends deploying cryptographic protections to produce tamper-evident and confidential logs. Organizations should also establish policies governing who may access log data, how long logs are retained, and how log integrity is validated over time.

VMware Private AI Services (PAIS) generates audit-relevant logs across its AI infrastructure components and exposes an observability subsystem with configurable retention and external forwarding that contributes to event log protection.

Deep Learning VMs record cloud-init script logs in `/var/log/dl.log` and GPU driver installation logs for the vGPU guest driver or NVIDIA data center GPU driver in `/var/log/gpu-install.log`. JupyterLab instances on Deep Learning VMs surface logs that can be examined to verify component installation and operation. Model endpoints support configurable log verbosity through the `VLLM_LOGGING_LEVEL` environment variable, letting operators control what the vLLM inference server records.

PAIS observability is activated by updating the `PAISConfiguration` custom resource. Operators set a Prometheus storage class policy through `observability.prometheusRuntime.storageClassName` and a retention period through `observability.prometheusRuntime.metricsRetention.period`, which define where collected metrics are stored and how long they are preserved. LLM traces can be forwarded to an external OpenTelemetry Collector over HTTP/protobuf or gRPC by uncommenting the `spec.observability.llmTraces` section of the `PAISConfiguration` custom resource, externalizing trace data away from the system where it was generated. PAIS Controller metrics are exposed at the Prometheus metrics endpoint on the `pais-controller-manager` service and can be streamed from the Supervisor cluster to VCF Operations for consolidated observability. PAIS integrates with Grafana for metrics visualization using OIDC provider authentication, restricting who can view the collected observability data.

Access to the underlying log files on Deep Learning VM filesystems and the persistent volumes backing Prometheus storage is governed by the Kubernetes RBAC and persistent volume access controls provided by VCF. Organizations should configure the Prometheus storage class and retention parameters to align with their log protection requirements and route LLM traces and observability data to a protected external platform such as Grafana, VCF Operations, or another OpenTelemetry-compatible backend.

### **VMware vDefend (Contributes)**

VMware vDefend generates structured security event logs across its Distributed Firewall, Gateway Firewall, IDS/IPS, and Malware Prevention components, with several mechanisms that support protection of those logs from suppression or loss.

The Security Services Platform audit log is enabled by default and cannot be disabled through configuration. All access events to the Security Services Platform, including access to the License Hub, are recorded in these audit logs, preventing log suppression at the platform level.

The Distributed IDS/IPS can be configured to forward events directly to external syslog consumers via the `ids_events_to_syslog` setting; this forwarding is disabled by default and requires explicit configuration. When remote syslog is enabled on ESX hosts, IDS/IPS events are exported directly from those hosts to external receivers. vDefend components write log files to `/var/log`, with syslog messages conforming to RFC 5424 on vDefend appliances and RFC 3164 on ESX hosts. Gateway Firewall isolation rules log blocked traffic attempts to the `unifiedsyslog.log` file on Edge Nodes.

Malware Prevention produces structured event logs in JSON format that include standard attributes (date, log level, module) along with detailed file analysis metadata: file hashes (SHA256, SHA1, and MD5), file type, file size, inspection timestamp, client and server VM identifiers, submission source, detection status, verdict, analysis status, malware classification, malware family, HTTP domain, HTTP method, and user agent. These records capture the full analysis lifecycle, including file intercepted, verdict cache hit, local static analysis submission, cloud dynamic analysis submission, verdict obtained, and policy enforced events. In 9.1, the Malware Prevention Service can be configured to forward logs to a remote logging server using TLS encryption with mutual certificate authentication (requiring server CA certificate, client CA certificate, client certificate, and client key), providing an off-appliance copy of event records with in-transit protection.

Identity Firewall event log sources can be integrated with VCF Log Management for centralized log collection and monitoring, providing an additional path to off-appliance log preservation.

Protection of forwarded or archived logs from unauthorized access and modification at rest is outside vDefend's scope and depends on the receiving log management or SIEM platform's controls.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) contributes to this control through role-based log access controls, syslog forwarding to external platforms, and audit event generation that captures user identity and action details.



PNR generates audit events across a broad range of recovery and replication operations, including configuration changes, recovery plan executions, and protection group modifications. Each audit event record includes the event name, user identity, and source IP address from which the action was initiated. Login events include timestamps, session keys, and user identity information in a structured format. Events covering protection group and consistency group changes are logged at appropriate severity levels (Info or Error), and corresponding alarms can be configured to trigger notification when specific events occur.

PNR log files are designed to exclude sensitive data such as private keys and passwords. The vSphere Replication component maintains security-relevant audit logs at defined paths on the appliance, including authorization error messages, login events, and certificate verification errors, each recorded with timestamps and component information to support forensic review.

To move logs off the local appliance, PNR supports syslog forwarding from the PNR Appliance to a remote syslog server. The Clean Room Orchestrator supports forwarding events to VCF Log Management, Microsoft Sentinel, or Splunk Cloud, enabling integration with enterprise log management and security monitoring platforms. Event forwarding to VCF Log Management requires version 9.0 or later. Centralizing logs in an external platform removes them from local storage where an attacker with appliance access could otherwise modify or delete them.

PNR does not provide write-once or tamper-evident local log storage. Organizations should forward PNR logs to a centralized SIEM or write-once-read-many log repository to protect against modification or deletion at the storage layer.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) produces structured audit logs that record platform-level and gateway-level operations, and provides several mechanisms that contribute to protecting those logs from unauthorized access and loss.

At the platform level, DSM maintains gateway audit log files on the Provider VM at `/var/log/tdm/provider/audit/`, with individual files named using a timestamp format (`gateway-audit-timestamp.log`). Local retention covers the last five days of log files; once files are successfully uploaded to a configured S3 repository, DSM removes them from the local filesystem after five days. Each log entry captures structured fields including Component, Event Type, Event Details, Event Time (UTC), Status, and Source. The Source field records the email identifier of the user who initiated the operation, or "System" for events initiated by DSM without user intervention. The System Audit Events view, accessible through the Provider interface, is the designated location for reviewing non-database-related platform operations. Access to the Provider interface is scoped to the administrative role, restricting which users can read or interact with the system-level audit record.

DSM supports offloading audit logs to external storage as the primary mechanism for durable protection. When a default S3 provider log repository is configured, DSM periodically archives gateway audit log files to an S3 subpath named "gateway audit." DSM does not delete audit logs stored in the S3 repository, which means the archived log record persists beyond the five-day local retention window. Externalizing audit logs to S3 separates the log record from the Provider VM that generated it, reducing the risk that a compromise or failure of the Provider VM affects the integrity or availability of the audit trail. Organizations apply bucket-level access controls and retention policies on the S3 repository to restrict who can read, modify, or delete the archived logs.

DSM supports log forwarding, allowing DSM logs to be sent to a centralized log management or security information and event management (SIEM) platform via the System Logs capability. Forwarding logs to an external system provides a second copy of the audit record under access controls and integrity protections that are independent of the DSM platform itself.

For SQL Server databases provisioned through DSM, the SQL Server data service uses server-level DDL triggers to restrict the file paths available for audit logs, limiting where audit log files can be written. DSM supports enabling audit logging for SQL Server with configurable levels of auditing for system logins and a configurable log retention period. DSM can automatically create a simple audit for login events based on the Login Auditing option selected during cluster creation. For custom audit requirements, the standard T-SQL `CREATE AUDIT` and `CREATE AUDIT SPECIFICATION` DDL statements are available.

DSM can generate and download log bundles that aggregate relevant logs from database VMs, supporting diagnosis and out-of-band preservation of the operational log record.



Audit events can be exported from the System Audit Events view in CSV or XLSX format using the ACTIONS menu. Exports include all events visible to the authenticated user regardless of pagination, supporting external archival and out-of-band preservation of the audit record.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer supports the protection of event logs and audit tools through real-time log forwarding to external systems, structured audit record content, and access controls on the Controller's audit interfaces. The 9.1 release adds License Hub audit logging capabilities that are always active and optionally forwarded to external syslog.

The primary off-system protection mechanism is the Syslog-Audit-Persistence feature, configured under Alert Actions in the Avi Controller. When enabled, audit events stream to one or more external syslog servers in real time; once forwarded, the records reside outside the Controller where local administrative access cannot modify or remove them. The Controller supports syslog delivery over TCP in addition to UDP, and syslog server targets are specified in Controller Settings. Any compatible syslog receiver, including VCF Log Management configured as a log forwarder, can serve as a destination.

An important architectural boundary applies: Avi Load Balancer debug logs and Event Logs are not forwarded to remote syslog servers through the OS-level syslog configuration. Debug logs are accessible only through Tech Support generation. Event Logs require a separate API pull or a scripted push from the Controller to a remote logging system, which means organizations should configure explicit export or retrieval processes for off-system preservation.

The Avi Controller's Event Logs provide detailed records of system activities, configuration changes, and operational events, including an audit trail that identifies who made each change and when. Object-specific event logs are accessible via the Events tab on detail pages for Virtual Services, Pools, Service Engines, and Controllers, automatically filtered to the relevant object. The API supports querying events over extended periods, including a one-year lookback using the CONFIG\_DELETE endpoint with the appropriate duration parameter.

Avi WAF generates audit log records for matched security events, such as cross-site scripting detection, capturing the matched variable name and content when audit log parts are configured to include event data (ctl:auditLogParts=+E). This extends event log coverage to L7 application protection events beyond configuration and access activity.

In 9.1, the License Hub provides dedicated audit logging that records all access events and cannot be disabled. License Hub can be configured to transmit log messages to a remote syslog server. A built-in Audit local user account with the auditor role provides dedicated access to APIs and services for system auditing, separate from the administrative account.

Both the Controller and Service Engines log delivery failures when transmitting to Events, Syslog, and Splunk targets, including failed trusted channel initiations and function failures. Disruptions to the log forwarding pipeline are captured as auditable events in their own right.

### Control CR 3.9(1)

> Audit storage capacity: Ensure sufficient storage for audit records and define behavior when full. Warning on capacity: Provide a warning when audit storage reaches a defined threshold.

**SCF Controls:** MON-19

**Aggregate Coverage:** Not Supported

Not supported directly for this control: VCF. This product does not currently provide the capability this control requires.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

### Control CR 3.10

> Support for updates: Provide a mechanism to verify and apply security updates.

**SCF Controls:** TDA-01.1

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 3.11

> Physical tamper resistance/detection: Capability to detect or resist physical tampering.

**SCF Controls:** AST-08

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides monitoring, alerting, compliance assessment, and forensic capabilities that support periodic inspection of systems and system components for indicators of compromise (IoC). While VCF does not include a dedicated IoC detection engine or endpoint detection and response (EDR) tool, its operational monitoring and change-detection features contribute to an organization's ability to identify anomalous conditions that may signal compromise.

VCF Operations continuously monitors the managed environment by evaluating collected data against defined policies and assigning a compliance score to each object or group based on adherence to those policies. This compliance monitoring enables discovery of non-compliant objects and associated compliance violations, which can surface configuration drift or unauthorized changes that may indicate tampering or compromise. The Anomalies widget displays metric deviations from both dynamic and static thresholds, and the Potential Evidence tab surfaces events, property changes, and anomalous metrics that may be relevant to a given alert. Risk alerts flag conditions that are not immediate threats but warrant prompt investigation.

Configuration drift detection in VCF Operations compares current settings against assigned configuration template values and notifies administrators of any deviations. The Configuration Drift dashboard allows administrators to view and compare drifts and identify settings that have changed, while the Detect Drift function detects drift across all configuration templates and displays results in the Drifts table. This change-detection capability supports identification of unauthorized modifications to managed infrastructure.

VCF Operations provides a layered alerting framework. Log-based alerts can be configured through the Symptom Definitions card in Infrastructure Operations, and alert definitions can be created to monitor infrastructure events and conditions. The system correlates a subset of events published by monitored systems and delivers them as faults, while fault symptoms can be defined based on those events and added to alert definitions. This allows administrators to build detection logic around events that may indicate compromise.

VCF Operations collects security properties for virtual machines, including the connection state of CD-ROM, USB controller, parallel port, and serial port devices, which supports detection of unauthorized device connections to virtual infrastructure. Administrators can filter virtual machines by encryption status, vCenter name, host name, and VM name to assess compliance posture across the environment.

Audit event reporting in VCF Operations records configuration changes, user actions, and authentication events across the managed infrastructure. These audit events can be filtered by authentication status, user, vCenter instance, object type, and event description, giving investigators a detailed view of activity across the environment for conducting security forensics.

VCF Operations includes a Forensics widget that allows administrators to compare data across two time periods, search for and filter objects within the environment, and display widget data based on filtered object sets. This capability supports forensic comparison and analysis when investigating potential indicators of compromise. Guided Network Troubleshooting highlights anomalous entities and metrics within the network path, aiding root-cause analysis during incident investigation.

OS and Application Monitoring Properties are collected for operating systems, application services, remote checks, Linux processes, and Windows services. These monitoring data points can be used to create reports, views, and dashboards for ongoing system inspection, providing visibility into the operational state of managed components and flagging deviations from expected behavior.

VMware ESX hosts generate detailed audit logs, and multiple VCF components support syslog forwarding to external log aggregation and SIEM systems. VMware vCenter provides both API and UI methods to configure syslog forwarding. vSphere Supervisor can forward control plane and Kubernetes API server audit logs to external syslog servers via Fluent Bit in RFC

5424 format. These log forwarding capabilities allow VCF telemetry to feed into external IoC detection and correlation tools. ESX also enforces integrity checks on installed VIBs through the Host Image Profile Acceptance Level, governing what software is permitted to run on the host and supporting detection of unauthorized additions to the hypervisor. vSphere provides preconfigured alarms that monitor the status of host hardware system boards, allowing notification when hardware-level anomalies are detected.

The NSX networking layer includes monitoring dashboards showing details about system health, networking, security, and compliance reporting. NSX supports logging and auditing capabilities that can be integrated with external logging and monitoring solutions to monitor for security events and anomalies. The System Health Agent on NSX components monitors component statistics, analyzes them, alerts on abnormalities, and can trigger remediation actions when specific conditions are met. VCF Operations for Networks monitors network flows and can detect outlier VMs based on flow metrics including bytes, packets, sessions, and traffic rate, supporting detection of anomalous traffic patterns within the environment. At the host layer, ESX measures boot components against a Trusted Platform Module (TPM) 2.0 hardware root of trust, and VMware vCenter exposes the resulting boot measurements and TPM event logs through its trusted-infrastructure API, so administrators can confirm a host booted in a known-good state and detect firmware or boot-level tampering. Secure Boot independently validates the cryptographic signatures of the ESX kernel and modules at startup, so unsigned or altered components are surfaced as an integrity failure rather than loaded silently.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer contributes to this control through its application-layer threat detection and event logging capabilities, which provide the data security teams need during periodic Indicators of Compromise (IoC) inspections.

Avi's Web Application Firewall (WAF) analyzes all incoming and outgoing application traffic using signature-based detection, a positive security model, client reputation scoring, machine learning, and outlier analysis. WAF signatures include detection for OWASP Top 10 attack vectors and other threat categories. The WAF operates in either Detection mode or Enforcement mode; Detection mode allows new rules to be evaluated without blocking legitimate traffic, making it suitable for baseline establishment and traffic anomaly review during periodic inspections. Partial detection allows individual rules to be placed in Enforcement mode while the policy as a whole remains in Detection mode, supporting incremental adoption.

The Avi Controller generates structured security events that auditors and security teams can review as part of periodic IoC inspections. The Controller tracks denial-of-service attack events with attack count metrics and generates alerts at configurable severity levels. The scan detection system dynamically classifies client IPs into Good, Bad, and Unknown groups based on observed traffic patterns, adjusting automatically as patterns change. The Controller also logs the initiator and target of failed trusted channel establishment attempts, including cases where an SSL certificate is absent, providing visibility into authentication anomalies.

Avi events serve as an audit trail for user logins, configuration changes, and runtime state changes, and can be forwarded to Splunk for centralized audit trail analysis. Client request logs can be collected automatically based on categories of significance for application-level review. System events provide a historical record of faults within the system. DataScripts support custom identification of suspicious traffic and execution of appropriate mitigation actions, extending detection coverage beyond built-in signatures.

Avi provides several tools suited for periodic review workflows. The application security report can be generated on demand through the Controller UI or CLI, with an optional duration parameter to define the report period, producing a point-in-time record of detected threats suitable for scheduled IoC review cycles. The Config Audit Trail, accessible at Operations > Events > Config Audit Trail, presents a filtered view of user activity and configuration change events over a selected period, supporting review of whether configuration changes preceded observed anomalies. Alert events can be streamed to external syslog servers using Syslog Audit Persistence, enabling integration with SIEM platforms that support automated IoC correlation. The Controller also generates system reports for upgrade operations containing pre-check results and operational status, which support verification of system state before and after changes.

The organization is responsible for defining the inspection cadence, determining what constitutes a compromise indicator in context, and performing the host-level and physical inspection that Avi does not address as an application delivery controller. Platform-level system integrity inspection relies on the underlying VCF platform.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 3.12

> Provisioning Product: Ensure secure provisioning of the product during manufacturing.

**SCF Controls:** SEA-08.1

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides multiple mechanisms to verify that software and data used for system component and service refreshes are obtained from trusted, verified sources, covering both connected and air-gapped deployment scenarios.

vSphere Lifecycle Manager (vLCM) is the primary subsystem for managing software updates across the VCF stack. Software depots within vLCM contain the actual payloads and metadata of software updates and are the authoritative source for all host and component refreshes. Administrators can create schedules to synchronize software update metadata stored in vCenter with the metadata available in accessible depots, keeping the local inventory current with vendor-published content. Update policies allow automation of checks for new updates and staging, reducing manual intervention while maintaining control over when updates are applied. vCenter also supports automated installation of software updates to keep systems stable and current. vLCM enforces a desired state model: administrators define a software specification for each cluster, validate the compliance of hosts against that specification, and then remediate any drift. This keeps hosts running the expected software baseline from a known-good source.

VCF Operations orchestrates the lifecycle of all VCF components through a centralized update process, simplifying provisioning, certificate installation, and patching across the infrastructure. Before applying updates, VCF Operations validates host compliance against the desired state software specification. VCF also requires verification of the server fingerprint for each ESX host during commissioning before activation, confirming host identity before the host joins the managed environment.

For environments operating in secure or air-gapped networks without internet access, VCF provides the Update Manager Download Service (UMDS). UMDS allows administrators to download updates to the vLCM depot from a connected system, then transfer them to the isolated environment. Administrators can configure a dedicated patch store location for storing updates after download. A re-download capability allows administrators to re-download existing updates that may have become corrupted while also fetching new updates, with an option to download metadata only when full payloads are not needed. This allows even disconnected environments to receive verified software from a controlled, trusted pipeline.

ESX supports Secure Boot and Trusted Platform Module (TPM), which validate the boot chain to confirm that only trusted code executes during startup. ESX hosts with TPM record and securely store measurements of the software modules booted on the system; vCenter remotely verifies these measurements, attesting the integrity of each host before it is trusted to run workloads. Software package acceptance levels can be set when cloning an ESX image profile (for example, to VMware Certified), restricting which packages are permitted on hosts based on their trust level. The vSphere Security Configuration Guide provides recommendations for enabling these features as part of host hardening.

VCF extends trust verification to extensibility components as well. VCF Operations management pack installation requires verification of the source trustworthiness before proceeding with installation of PAK files from untrusted sources. This gate helps prevent the introduction of unvetted third-party code into the management plane.

Certificate refresh operations reinforce the trust chain for cryptographic material. The `CertificateManager.refreshCACertificatesAndCRLs` operation fetches certificates and certificate revocation lists from trusted certificate authorities and pushes them to hosts, keeping the trust anchors current. vCenter maintains the `TRUSTED_ROOTS` store within the VMware Endpoint Certificate Store (VECS), and refresh operations push all entries from that store to managed hosts so that host trust anchors match the vCenter trust store. `CertificateManager.refreshCertificates` obtains CSRs and certificates signed by the VMware Certificate Authority (VMCA) and distributes them to hosts. Administrators can add the root certificate of an external CA to the trusted root store of the vCenter system to establish trust with external certificate authorities. VCF PowerCLI maintains its own Trusted Certificate Store where trusted certificates are saved and used to connect to server systems. VCF documentation explicitly warns that a trust-all trust manager, which returns true for all



certificate checks, is appropriate only for development environments and should not be used in production, reinforcing the expectation that certificate validation against trusted sources is mandatory in operational deployments.

For containerized workloads running on VMware Kubernetes Service (VKS) and vSphere Supervisor, VCF provides a layered container supply chain trust model. Platform administrators must explicitly allow each container registry before the platform considers it trusted; registries not on the approved list are not available to workloads. When Harbor is deployed as a Supervisor service or as the private registry for VKS clusters, it provides vulnerability scanning, content trust, and policy-based replication, giving organizations a technical mechanism to verify that container images are sourced from authorized registries and have not been tampered with before they are used in a cluster refresh or workload update. VCF supports private registry authentication through imagePullSecrets so that authenticated pulls can be enforced against an approved private registry. In air-gapped environments, customized images from a private registry are required since public registries are not accessible, keeping the container supply chain within organizational control. Admission controllers and ValidatingAdmissionPolicy objects can be configured within VKS clusters to restrict which container image sources are permitted, blocking images that do not originate from approved registries. VKS clusters support declaring trusted certificate authorities at the cluster level through the `osConfiguration.trust.additionalTrustedCAs` parameter in the Cluster v1beta1/v1beta2 API and the `network.trust.additionalTrustedCAs` parameter in the TanzuKubernetesCluster v1alpha3 API; these CA configurations are propagated to all cluster nodes during rolling updates, maintaining consistent trust across the cluster throughout the refresh process. The vCenter root CA certificate establishes trust for vSphere Supervisor connections, forming the PKI foundation for administrative access to the Consumption layer. VKS cluster updates include pre-checks that enforce compatibility requirements before updates proceed, supporting an ordered and validated update process. When stand-alone virtual machines are deployed through vSphere Supervisor, guidance directs administrators to create VM images that include the latest patches and required security settings aligned to corporate security policies. Kubernetes configuration files (kubeconfig files) sourced from outside the organization should be reviewed carefully before use, treating them with the same scrutiny as untrusted executable scripts. Cluster dependencies should be updated when new versions are available, with priority given to updates that address security vulnerabilities, maintaining supply chain integrity throughout the VKS cluster lifecycle.

Content libraries in vSphere support security policies that govern how content is validated when published or subscribed. Administrators can set, update, or remove security policies on local and subscribed content libraries using the LibraryModel update specification. If a content library is protected by a security policy, all library items must be compliant. Subscribed libraries support authentication mechanisms to authenticate to published libraries, controlling the integrity of VM templates and other content distributed through this channel.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 3.13

> Integrity of the boot process: Verify the integrity of the boot process (Secure Boot).

**SCF Controls:** SEA-08.1

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides multiple mechanisms to verify that software and data used for system component and service refreshes are obtained from trusted, verified sources, covering both connected and air-gapped deployment scenarios.

vSphere Lifecycle Manager (vLCM) is the primary subsystem for managing software updates across the VCF stack. Software depots within vLCM contain the actual payloads and metadata of software updates and are the authoritative source for all host and component refreshes. Administrators can create schedules to synchronize software update metadata stored in vCenter with the metadata available in accessible depots, keeping the local inventory current with vendor-published content. Update policies allow automation of checks for new updates and staging, reducing manual intervention while maintaining control over when updates are applied. vCenter also supports automated installation of software updates to keep systems stable and current. vLCM enforces a desired state model: administrators define a software specification for each cluster, validate the compliance of hosts against that specification, and then remediate any drift. This keeps hosts running the expected software baseline from a known-good source.

VCF Operations orchestrates the lifecycle of all VCF components through a centralized update process, simplifying provisioning, certificate installation, and patching across the infrastructure. Before applying updates, VCF Operations validates host compliance against the desired state software specification. VCF also requires verification of the server fingerprint for each ESX host during commissioning before activation, confirming host identity before the host joins the managed environment.

For environments operating in secure or air-gapped networks without internet access, VCF provides the Update Manager Download Service (UMDS). UMDS allows administrators to download updates to the vLCM depot from a connected system, then transfer them to the isolated environment. Administrators can configure a dedicated patch store location for storing updates after download. A re-download capability allows administrators to re-download existing updates that may have become corrupted while also fetching new updates, with an option to download metadata only when full payloads are not needed. This allows even disconnected environments to receive verified software from a controlled, trusted pipeline.

ESX supports Secure Boot and Trusted Platform Module (TPM), which validate the boot chain to confirm that only trusted code executes during startup. ESX hosts with TPM record and securely store measurements of the software modules booted on the system; vCenter remotely verifies these measurements, attesting the integrity of each host before it is trusted to run workloads. Software package acceptance levels can be set when cloning an ESX image profile (for example, to VMware Certified), restricting which packages are permitted on hosts based on their trust level. The vSphere Security Configuration Guide provides recommendations for enabling these features as part of host hardening.

VCF extends trust verification to extensibility components as well. VCF Operations management pack installation requires verification of the source trustworthiness before proceeding with installation of PAK files from untrusted sources. This gate helps prevent the introduction of unvetted third-party code into the management plane.

Certificate refresh operations reinforce the trust chain for cryptographic material. The `CertificateManager.refreshCACertificatesAndCRLs` operation fetches certificates and certificate revocation lists from trusted certificate authorities and pushes them to hosts, keeping the trust anchors current. vCenter maintains the TRUSTED\_ROOTS store within the VMware Endpoint Certificate Store (VECS), and refresh operations push all entries from that store to managed hosts so that host trust anchors match the vCenter trust store. `CertificateManager.refreshCertificates` obtains CSRs and certificates signed by the VMware Certificate Authority (VMCA) and distributes them to hosts. Administrators can add the root certificate of an external CA to the trusted root store of the vCenter system to establish trust with external certificate authorities. VCF PowerCLI maintains its own Trusted Certificate Store where trusted certificates are saved and used to connect to server systems. VCF documentation explicitly warns that a trust-all trust manager, which returns true for all

certificate checks, is appropriate only for development environments and should not be used in production, reinforcing the expectation that certificate validation against trusted sources is mandatory in operational deployments.

For containerized workloads running on VMware Kubernetes Service (VKS) and vSphere Supervisor, VCF provides a layered container supply chain trust model. Platform administrators must explicitly allow each container registry before the platform considers it trusted; registries not on the approved list are not available to workloads. When Harbor is deployed as a Supervisor service or as the private registry for VKS clusters, it provides vulnerability scanning, content trust, and policy-based replication, giving organizations a technical mechanism to verify that container images are sourced from authorized registries and have not been tampered with before they are used in a cluster refresh or workload update. VCF supports private registry authentication through imagePullSecrets so that authenticated pulls can be enforced against an approved private registry. In air-gapped environments, customized images from a private registry are required since public registries are not accessible, keeping the container supply chain within organizational control. Admission controllers and ValidatingAdmissionPolicy objects can be configured within VKS clusters to restrict which container image sources are permitted, blocking images that do not originate from approved registries. VKS clusters support declaring trusted certificate authorities at the cluster level through the `osConfiguration.trust.additionalTrustedCAs` parameter in the Cluster v1beta1/v1beta2 API and the `network.trust.additionalTrustedCAs` parameter in the TanzuKubernetesCluster v1alpha3 API; these CA configurations are propagated to all cluster nodes during rolling updates, maintaining consistent trust across the cluster throughout the refresh process. The vCenter root CA certificate establishes trust for vSphere Supervisor connections, forming the PKI foundation for administrative access to the Consumption layer. VKS cluster updates include pre-checks that enforce compatibility requirements before updates proceed, supporting an ordered and validated update process. When stand-alone virtual machines are deployed through vSphere Supervisor, guidance directs administrators to create VM images that include the latest patches and required security settings aligned to corporate security policies. Kubernetes configuration files (kubeconfig files) sourced from outside the organization should be reviewed carefully before use, treating them with the same scrutiny as untrusted executable scripts. Cluster dependencies should be updated when new versions are available, with priority given to updates that address security vulnerabilities, maintaining supply chain integrity throughout the VKS cluster lifecycle.

Content libraries in vSphere support security policies that govern how content is validated when published or subscribed. Administrators can set, update, or remove security policies on local and subscribed content libraries using the LibraryModel update specification. If a content library is protected by a security policy, all library items must be compliant. Subscribed libraries support authentication mechanisms to authenticate to published libraries, controlling the integrity of VM templates and other content distributed through this channel.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 3.14

> Integrity of data at rest: Protect the integrity of sensitive data stored on the component.

**SCF Controls:** END-06.5

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides automated mechanisms to verify the integrity of the boot process for both physical hosts and virtual machines, spanning hardware-rooted trust, software specification validation, and post-boot health verification.

ESX hosts support UEFI Secure Boot, which uses cryptographic signature verification to validate each component loaded during the boot sequence. When Secure Boot is enabled, the host firmware checks the digital signature of the bootloader, and each subsequent stage verifies the next, creating a chain of trust from firmware through the hypervisor kernel. The vSphere Security Configuration Guide recommends enabling Secure Boot on all ESX hosts as part of standard hardening. ESX also supports an `exclInstalledOnly` enforcement mode, built on top of UEFI Secure Boot, which restricts execution to software installed through the managed package process and prevents unauthorized binaries from running on the host.

VCF also supports Trusted Platform Module (TPM) integration on ESX hosts. TPM devices perform measured boot, recording cryptographic hashes of each boot component into platform configuration registers. VMware vCenter provides programmatic access to TPM device information through the trusted infrastructure API, including retrieval of TPM event logs that record boot-time measurements. These event logs can be used with an attestation service to validate that hosts booted with known-good firmware and software configurations. The Tpm interface from the trusted infrastructure package allows administrators to list TPM devices on a host and retrieve detailed information including manufacturer, model, and firmware version. vCenter also supports unsealing secrets bound to an endorsement key to verify reported measurements from the TPM event log, providing a second verification path for boot integrity. ESX host TPM modules must run the latest 2.0 firmware when in use, and must be disabled in the BIOS if not in use, keeping the hardware trust baseline consistent and verifiable.

Remote attestation capabilities allow vCenter to verify host boot integrity by comparing TPM measurements against expected values, providing an automated mechanism to detect tampering or unexpected changes in the boot process of managed hosts.

For virtual machines, VCF supports UEFI Secure Boot at the VM level, allowing guest operating systems to verify boot component integrity through the same cryptographic chain-of-trust mechanism used on physical hardware. VM boot firmware can be configured to EFI type using the vSphere Automation SDK `Boot.UpdateSpec()` method, and boot options including EFI firmware type and boot behavior settings are configurable through both the vSphere Client and the vSphere Automation API. VM-level Secure Boot is supported starting with Hardware Version 13. For Windows virtual machines, vSphere also supports Configurable Code Integrity under Virtualization-Based Security (VBS), which enforces that only trusted, signed code runs from the boot loader onwards within the guest.

At the software specification level, VCF's lifecycle management includes automated validation before any software is applied to hosts. The Drafts service offers a `validate_Task` method that checks whether a draft software specification is complete and identifies any conflicts between draft components or unresolved dependencies. Before saving a draft and turning it into the desired state for a cluster or standalone host, the specification should be validated to confirm it is complete and valid. This automated pre-deployment validation helps ensure that hosts receive verified, conflict-free software configurations, reducing the risk of boot failures or integrity issues caused by incompatible components. VCF also supports the creation of exported ISO images from desired software specifications for use in clean installs and bootstrapping, allowing administrators to provision systems from a known, validated software baseline.

For post-boot verification, VCF provides automated checking mechanisms across its management components. The Supportability and Serviceability tool (SoS) on VCF Operations includes a health check function that verifies system state after operations such as a vCenter instance restore, confirming that all components have started correctly and are operating as expected. VCF Operations Cloud Proxy implements automated first boot verification by checking for the presence of a

"Succeeded" file in /var/log/firstboot, providing a programmatic way to confirm that the boot process completed without error.

Auditors evaluating a VCF deployment for this control should verify that UEFI Secure Boot is enabled on all ESX hosts, that TPM 2.0 modules are active and running current firmware, that lifecycle management workflows use the software specification validation step before applying updates, that SoS health checks are run after restore or maintenance operations, and that boot verification logs are monitored for component deployments.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 4.1(a)

> Data in transit confidentiality: Protect the confidentiality of sensitive data in transit.

**SCF Controls:** CRY-05

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides multiple cryptographic mechanisms for protecting data at rest across its infrastructure stack, primarily through vSAN data-at-rest encryption and vSphere Virtual Machine Encryption.

vSAN data-at-rest encryption is the primary mechanism for protecting stored data within VCF. When enabled on a vSAN cluster, vSAN encrypts everything in the vSAN datastore, including all files, virtual machines, and their corresponding data. This protection extends to scenarios where a physical storage device is removed from the cluster, as the encrypted data remains unreadable without the proper keys. vSAN supports both Original Storage Architecture (OSA) and Express Storage Architecture (ESA) deployments, with encryption behavior varying by architecture. For OSA, data-at-rest encryption occurs at the Local Object Manager (LSOM) layer on the target host, and each disk has a different randomly generated Data Encryption Key. For ESA in hyperconverged infrastructure mode, data is encrypted at the VM host before network transmission, and all disks in the cluster use the same DEK to encrypt object data. In OSA, encryption occurs after other data processing operations such as deduplication and compression, preserving storage efficiency. vSAN encryption is the recommended approach for securing data on vSAN datastores because it is designed to be compatible with vSAN deduplication and compression, whereas virtual machine encryption causes these storage efficiency features to become less effective.

vSphere Virtual Machine Encryption provides an alternative or complementary data-at-rest encryption layer that works across all datastore types, including vSAN, iSCSI, NFS, and Fibre Channel. VM Encryption occurs before data reaches the storage layer, protecting virtual machine files and disks regardless of the underlying storage technology. This can be applied on a per-VM basis and operates independently of the underlying storage layer, making it applicable to environments where granular, VM-level encryption control is required. Most virtual machine files, in particular guest data not stored in the VMDK file, are encrypted. When combined with storage-level encryption, administrators should evaluate the performance impact of multiple encryption layers on latency and throughput, as well as the potential loss of storage efficiency features like deduplication and compression.

For organizations consuming VCF through the VCF Automation layer, VCF Encryption Management extends these VM and disk encryption capabilities to the namespace level. This service allows organization administrators to use encryption keys from their own third-party key providers to encrypt VMs and disks within a VCF Automation namespace, and also supports encrypting VMs with a Virtual Trusted Platform Module (vTPM). Provider administrators must register a default standard key provider in VMware vCenter before organization administrators can configure namespace-level encryption; the vSphere Native Key Provider cannot be used in this role, and key providers must be listed in the VMware Compatibility Guide for Key Management Servers. Organization administrators configure encryption by setting up a key provider and creating a namespace encryption class. Configuration of key providers requires either the Encryption Management Admin role or a custom role that grants the right to configure key providers.

Both encryption mechanisms require a key management infrastructure. VCF supports integration with KMIP 1.1-compatible external Key Management Servers (KMS) or the built-in vSphere Native Key Provider (NKP). VCF establishes a cryptographic domain of trust between key providers, VMware vCenter, and VMware ESX hosts. When using a Standard Key Provider, the KMS sends keys to vCenter upon request, which are used as the Key Encryption Key (KEK) and are AES-256 keys. Standard Key Providers support key wrapping, creating a three-tier key hierarchy: Data Encryption Keys (DEK) encrypt object data, Key Encryption Keys (KEK) protect the DEK, and a wrapping key stored in the KMS protects the KEK. This approach reduces the number of keys stored in external KMS systems while maintaining cryptographic separation. When using an external KMS, the recommendation is to deploy highly available KMS clusters to distribute the keys that encrypt the vSAN datastore, avoiding a single point of failure in the encryption key chain. The vSphere Native Key Provider offers a simpler deployment model for environments that do not require an external KMS. Native Key Provider equips each participating host with decryption keys so hosts can operate independently of network-accessible key servers.

vSAN supports key rotation for both Key Encryption Keys (KEK) and Data Encryption Keys (DEK), with the capability to re-encrypt all data on the storage using new keys. vSAN encryption supports configurable rekey intervals that can be set to default or custom values. Key wrapping supports automatic key rotation with configurable intervals. This allows organizations to meet key lifecycle management requirements without disrupting operations. Encryption can also be enforced through vSAN storage policies, where the data-at-rest encryption option can be specified as a policy attribute, allowing administrators to define which datastores require encryption when provisioning virtual machines.

VCF supports virtual Trusted Platform Module (vTPM) 2.0 for virtual machines, enabling in-guest encryption capabilities such as Windows BitLocker and Linux disk encryption. Virtual machines can include various forms of encryption: VMs using a vTPM device, VMs with disks encrypted, and VMs with encryption enabled without disks encrypted. These guest-level encryption features work independently of or in combination with VMware's infrastructure-level encryption, supporting defense-in-depth strategies.

ESX hosts store their configuration data in encrypted form. When the host has a physical TPM 2.0 enabled, the configuration encryption key is stored in the TPM. Core dumps on an ESX host that has encryption mode enabled are always encrypted. ESX also supports key persistence as a cache for encryption keys in environments where a KMS may be unavailable. With key persistence enabled, ESX hosts can persist the encryption keys in the Trusted Platform Module (TPM) across reboots, allowing encryption operations to continue when the key server is unavailable. Each ESX host obtains the encryption keys initially and retains them in its key cache; if the ESX host has a TPM, the encryption keys are persisted in the TPM across reboots. Key persistence requires that the ESX host has a TPM, and can be enabled using the `esxcli system security keypersistence enable` command.

Memory Tiering encryption protects tiered memory without requiring external key management. VCF generates random 256-bit AES-XTS keys at the kernel level during VM power-on, unique to each VM instance. During vMotion, tiered memory pages are decrypted and transmitted over the encrypted vMotion channel, with new keys generated at the destination.

For access control over cryptographic operations, VCF provides the No Cryptography Administrator role, which supports all Administrator privileges except for the Cryptographic Operations privileges. This allows standard VM and infrastructure management while restricting unauthorized encryption changes. The No Cryptography Administrator role can be cloned to create custom roles with only specific Cryptographic Operations privileges, providing fine-grained control over encryption-related tasks.

The cryptographic modules used across VCF components are validated against industry-standard best practices. NSX is configured to use FIPS 140-3 validated cryptographic modules by default, with validation performed through the NIST Cryptographic Module Validation Program (CMVP). VCF Automation uses the BouncyCastle FIPS 2.0.0 cryptographic module compliant with FIPS 140-3 (Certificate #4743). VCF Operations for Networks uses BC-FJA (Bouncy Castle FIPS Java API) version 2.0.0 as a validated cryptographic module. The vCenter appliance uses VMware's Boring Crypto Module version 6.0, which provides FIPS-validated cryptographic algorithms including AES, ECDSA, HMAC, RSA, and SHA2 variants, as well as VMware OpenSSL FIPS Object Module versions 3.0 and 2.0.20-vmw, which collectively support AES, CKG, DRBG, DSA, ECDSA, HMAC, KAS-SSC, RSA, SHS, Triple-DES, SHA-3, and additional key derivation functions. ESX hosts use FIPS 140-2 validated cryptographic modules that implement security functions as a combination of hardware, software, and firmware. vSAN encryption uses AES-NI CPU offloading for efficient processing.

For workloads deployed through VMware Kubernetes Service (VKS) on vSphere Supervisor, the Kubernetes control plane supports encryption at rest for API objects stored in etcd. The kube-apiserver implements this through the EncryptionConfiguration API, specified via the `--encryption-provider-config` argument. When configured, Secrets, ConfigMaps, and other designated resource types are encrypted on write to etcd. The kube-apiserver supports an external KMS provider as well as local encryption keys; when using local keys, the aescbc provider encrypts data using Base64-encoded secret keys specified in the EncryptionConfiguration resource. Using an external KMS is the recommended approach for production environments, as it allows keys to be managed and rotated independently of the cluster configuration. Encryption at rest for the Kubernetes control plane is not enabled by default; the default identity provider stores data in plaintext, and administrators must explicitly configure EncryptionConfiguration to activate it. After enabling EncryptionConfiguration, existing objects remain unencrypted until they are read and re-written. The Kubernetes etcd



encryption mechanism does not extend to data stored in filesystems mounted into containers; for container workload data on persistent volumes, encryption depends on the underlying storage layer. vSAN File Service volumes used with vSphere Supervisor are mounted without encryption by default, so workloads relying on shared file volumes must address encryption at the storage layer or within the application. Kubernetes Secrets encrypted at rest are also protected from disclosure if an attacker gains access to etcd backup data.

Classifying which data requires encryption, selecting between vSAN encryption, VM encryption, or both for specific workload categories, choosing the Key Provider model, and operating the key lifecycle (rotation, revocation, archive, destruction) are organizational data-policy decisions.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) preserves and integrates with the data-at-rest encryption capabilities of the underlying VCF platform, so that encrypted virtual machines remain protected through replication and recovery operations. The encryption mechanisms themselves, including VM Encryption, vSphere Native Key Provider, external key management servers, and storage policies, are VCF platform capabilities; PNR's role is to maintain encryption continuity through the disaster recovery lifecycle.

vSphere Replication within PNR supports replication of encrypted virtual machines when vSphere Native Key Provider (NKP) or a common external Key Management Server (KMS) is configured at both sites. When NKP is used, it must be configured manually on both the local and remote sites, and the NKP ID assigned to the encrypted VM on the local site must match the NKP ID on the remote site. NKP enables encryption without requiring a separately deployed external KMS; vSphere Enterprise+ licensing is required for NKP. Replica disks must be located on datastores accessible through at least one host in a VMware vCenter cluster. A VM Storage Policy with VM Encryption enabled must be applied to target disks to enforce encryption continuity during replication configuration. Replication configuration using seed disks together with a vSAN target datastore and a VM Encryption Policy storage policy is not a supported combination.

Not all PNR replication methods support encrypted virtual machines. vSAN Snapshots and Replication, an alternative replication method within PNR, does not support replication of encrypted VMs. Organizations that require encrypted VM replication should configure protection groups using vSphere Replication rather than vSAN Snapshots and Replication.

vSphere Replication also supports optional network encryption of replication traffic, which can be enabled during replication configuration to protect data in transit between source and target sites.

Not applicable for this control: VMware vDefend, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 4.1(b)

> Sub-item (b) of CR 4.1 in IEC 62443-4-2. The licensed standard text is not reproduced at this granularity; refer to the CR 4.1 requirement in the standard.

**SCF Controls:** CRY-03

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides cryptographic protection for data in transit across management, compute, storage, and inter-site networks. The architecture applies encryption at multiple layers using validated cryptographic modules and current TLS protocol versions.

VCF management and monitoring communications use TLS as the transport protocol. VCF Operations for Networks supports modern TLS protocol versions, with the ability to restrict cryptographic module usage to FIPS-validated modules by enabling FIPS mode for external connections. Telegraf, the metrics collection agent, is configured to require a modern minimum TLS version for its communication channels. When using the databus for on-premises environments, HTTPS is the recommended transport to protect data during transmission and to verify subscriber authenticity. The TLS implementation includes customizable PKI options ranging from fully automated certificate management within VCF to integration with enterprise PKI infrastructure such as Microsoft Active Directory Certificate Services. VCF Operations provides centralized certificate lifecycle management across VCF components, supporting auto-renewal for VMCA, MSCA, and OpenSSL CA, handling automated renewal, monitoring, and replacement. Machine SSL certificates can be renewed through the API for a specified period up to 730 days. Starting in vSphere 9.0, all vSphere TLS profiles are FIPS compliant, and the COMPATIBLE profile supports modern TLS cipher suites and elliptic curves. The vCenter appliance uses VMware's Boring Crypto Module version 6.0, which provides FIPS-validated cryptographic operations including AES, ECDSA, HMAC, RSA, and SHA-2 variants.

NSX, which provides networking services within VCF, uses FIPS 140-3 validated cryptographic modules by default. These modules are validated through the NIST Cryptographic Module Validation Program. NSX Manager uses FIPS 140 approved algorithms for authentication to its cryptographic module. For site-to-site connectivity, the NSX L2 VPN FIPS compliance suite uses FIPS-approved encryption with Perfect Forward Secrecy enabled and standardized Diffie-Hellman key exchange, providing transmission confidentiality for tunneled traffic. VCF Operations for Networks supports modern ECDSA and RSA signature algorithms for data source connections.

VCF Automation uses the BouncyCastle FIPS 2.0.0 cryptographic module, which is compliant with FIPS 140-3 (Certificate #4743). VCF Operations for Networks uses the BC-FJA (Bouncy Castle FIPS Java API) version 2.0.0 with FIPS 140-2 Certificate #37009. These validated modules underpin the cryptographic operations used for securing communications across the automation and operations components.

At the storage layer, vSAN provides data-in-transit (DIT) encryption that protects data as it moves between hosts within the cluster. DIT encryption is a cluster-wide setting that, when enabled, encrypts all data and metadata traffic across hosts. Forward secrecy is enforced for vSAN data-in-transit encryption, meaning that compromise of a long-term key does not expose previously encrypted traffic. DIT encryption operates independently of data-at-rest encryption and does not require external key management servers. The behavior differs by storage architecture: in vSAN Original Storage Architecture (OSA), data-at-rest encryption occurs at the target host's Local Object Manager layer, so data traverses the network before encryption, making DIT encryption strongly recommended. In vSAN Express Storage Architecture (ESA) in HCI mode, data is encrypted at the VM host before network transmission, making DIT encryption optional based on compliance requirements.

VM Encryption provides confidentiality protection across all datastore types including vSAN, iSCSI, NFS, and Fibre Channel. VM encryption is implemented on the ESX host rather than on the storage array, so it protects data both at rest and in transit to the storage backend. Disk encryption uses AES-256 as the disk encryption key.

Encrypted vSphere vMotion protects VM data during live migrations. For encrypted VMs, vMotion always uses encryption and cannot be disabled. For unencrypted VMs, encrypted vMotion offers three settings: Disabled, Opportunistic (default, uses encryption when supported), and Required. AES-NI significantly improves encryption performance for these operations. Cross-vCenter vMotion supports encryption when source and destination use shared key providers; vCenter Crypto Manager

provides an `importProvider` method to import a vSphere Native Key Provider configuration between sites. vSphere Fault Tolerance log traffic, which synchronizes VM state between primary and secondary instances, can also be encrypted to protect this synchronization data in transit.

Storage protocol support varies by type. vSAN File Service supports NFS4 Kerberos authentication with security options `krb5`, `krb5i`, and `krb5p`, where `krb5p` provides authentication plus encryption of data in transit. iSCSI uses CHAP for session authentication but does not encrypt data natively, so VM Encryption or in-guest encryption is needed for confidentiality on iSCSI networks. Fibre Channel Gen 7 supports end-to-end encryption with secure Host Bus Adapters.

HCX Network Extension services support transport encryption for data between site pairs, and this encryption is enabled by default. VCF Operations HCX tunnel encryption similarly uses transport encryption by default to protect Network Extension data and migration traffic between sites. This default-on approach reduces the risk of data being transmitted in cleartext during workload migrations or network extensions.

High-quality entropy generation on ESX hosts supports the proper functioning of security-related operations such as generating encryption keys for secure network communication. This foundational capability helps maintain the strength of cryptographic keys used across all transmission encryption functions in the platform.

VMware Kubernetes Service (VKS), which runs on vSphere Supervisor, applies TLS encryption across all control-plane communication paths. The Kubernetes API server enforces encrypted transmission within the control plane and between the control plane and its clients by default, so all API server interactions travel over TLS. Cipher suites are configurable at each component layer: the `kube-apiserver` `--tls-cipher-suites` parameter accepts a comma-separated list of permitted cipher suites; the `kube-scheduler` exposes the same `tls-cipher-suites` parameter to keep its API endpoint aligned; and the `kubelet` exposes `tlsCipherSuites` and `tlsMinVersion` settings in `KubeletConfiguration` to scope the permitted cipher algorithms and minimum protocol version at the node level. Antrea, the CNI plugin included with Supervisor, supports configurable TLS cipher suites for its own secure communication channels through the `tlsCipherSuites` setting, and the vSphere Cloud Provider Interface (CPI) exposes the same capability through a configurable cipher suite list.

For environments with strict cryptographic requirements, VKS supports FIPS mode through the `ClusterClass` variable `fips.enabled`, which controls which encryption algorithms are permitted at the node level. Tanzu Kubernetes Release references carrying the `-fips.1` suffix indicate that FIPS cryptographic modules are active for that node pool. The Istio service mesh package, available as a standard package on VKS clusters, ships with FIPS compliance and supports mutual TLS (mTLS) between pods within the mesh; the `meshMTLS.minProtocolVersion` setting controls the minimum acceptable protocol version for all inter-service traffic. Ingress-layer encryption for workloads is handled by the Contour package, which exposes `tls.minimum-protocol-version` and `tls.maximum-protocol-version` settings so operators can define a bounded range of acceptable protocol versions for inbound HTTPS traffic. The Supervisor secret injection agent automatically configures mutual TLS for its communication with the vault agent injector service using auto-generated certificates. Harbor, available as a Supervisor Service, accepts custom TLS certificates through its `tlsCertificate.tls.crt` and `tlsCertificate.tls.key` configuration fields, supporting encrypted access to the container registry for image distribution.

Defining the encryption policy for customer workload traffic, selecting cipher and FIPS configurations for environments with stricter requirements, and validating that data transmitted from customer applications and containerized workloads meets confidentiality standards are organizational data-policy decisions.

VMware Private AI Services (PAIS) provisions VKS clusters using a FIPS-enabled Kubernetes build for cryptographic compliance across both control plane and worker node communications. The VKS cluster topology version is `v1.33.1--vmware.1-fips-vkr.2` with FIPS compliance enabled, running on an Ubuntu 24.04 OS image, covering the cryptographic modules that protect data in transit within the cluster fabric.

PAIS, activated through the `PAISConfiguration` custom resource on the Supervisor, requires HTTPS for all external integrations. The `spec.clientTls.caBundleRefs` section of the `PAISConfiguration` YAML references CA trust bundles supplied as `ConfigMap` resources. Operators must configure HTTPS trust bundles for the OIDC provider, the Harbor container registry, content management systems hosting Knowledge Base data sources, MCP servers providing tools to models, and the

pgvector database. Separate trust bundles are recommended per application, with periodic updates required before certificate expiration.

The PAIS Supervisor endpoint uses HTTPS on port 6443 with certificate-based authentication. The PAIS ingress endpoint is protected by a self-signed certificate stored in the `pais-ingress-default` Kubernetes Secret. Administrators retrieve the CA certificate using `kubectl get secrets/pais-ingress-default -o yaml` and distribute it to clients that connect to the service, including Grafana when configured to scrape the PAIS Prometheus endpoint. CA trust bundles for external entity connections are managed through ConfigMaps and updated as certificate content changes.

VCF Automation Provider Management for PAIS uses a base64-encoded certificate (`TM_CERT_BASE64`) obtained by running `openssl s_client` against the VCF Automation FQDN on port 443, enabling authenticated HTTPS communication between AI catalog tooling and the PAIS environment. MCP server integrations require the MCP server's issuer certificate to be added as a CA trust bundle before connecting over HTTPS using either Streamable HTTP or Server-Sent Events transport.

Telemetry sent from PAIS to the OpenTelemetry Collector is authenticated using a Kubernetes Secret containing Authorization Bearer tokens and custom authorization headers. LLM trace transport to the collector uses HTTP/protobuf or gRPC, both protecting trace traffic in transit.

One configuration detail that auditors should verify: PAIS Prometheus TLS client authentication and certificate validation are turned off by default (the `tls-client-authentication` setting on the Prometheus endpoint). Organizations should review this setting and enable mutual TLS where their policy requires it. The underlying network transport encryption between VMs and workload domains is provided by VCF and is documented in VCF Coverage.

#### **VMware vDefend (Contributes)**

VMware vDefend contributes to transmission confidentiality through inspection, visibility, and enforcement mechanisms at the network layer, while the underlying cryptographic protection of data in transit is the responsibility of endpoint applications and infrastructure components.

TLS Inspection uses a transparent TLS proxy to intercept and analyze encrypted traffic flows traversing the environment. This allows security policy to be applied to traffic that would otherwise be opaque and supports detection of threats embedded within encrypted sessions. TLS Inspection can be deployed to provide visibility into encrypted connections while preserving end-to-end encryption for flows not subject to a decryption policy, and supports both inbound and outbound inspection profiles.

Security Intelligence, integrated within the Security Services Platform (SSP), surfaces workloads communicating over application protocols that transport data without encryption. The Security Segmentation Report identifies communication chains containing at-risk unprotected members and quantifies the number of unprotected data flows hitting default-allow rules across observed traffic. Administrators can use this reporting to identify and prioritize remediation of encryption coverage gaps, including restricting or blocking cleartext protocols.

The Security Services Platform and its component services use FIPS 140-2 and FIPS 140-3 validated cryptographic modules for their own communications. Network Detection and Response (NDR) Sensors connect to the SSP using mutual TLS authentication (mTLS), with a dedicated certificate for each sensor signed by an intermediate certificate authority on the SSP. The SSP can be configured to forward log messages to a syslog server over TLS, and Malware Prevention Service remote logging likewise uses TLS to transmit log data to a remote log server. NDR also supports SSL verification for event logs sent over HTTPS. When NDR shares data to the cloud, it anonymizes sensitive customer information before transmission.

#### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) provides cryptographic protection for data transmitted during disaster recovery operations, covering management communications and replication data flows.

All communications between the PNR server and VMware vCenter (vCenter) instances take place over TLS connections. PNR requires an SSL/TLS certificate, with x509v3 Extended Key Usage indicating TLS Web Server Authentication, as the endpoint certificate for all TLS connections to the PNR server. PNR defaults to the current TLS version for all communication endpoints.

and supports administrative configuration of the minimum acceptable protocol version. Administrators configure the minimum TLS protocol version by editing the `tls_minimum_protocol_version` parameter in the `envoy-proxy-config.json` file; when changing the minimum version, the update must be applied consistently across all occurrences in the system. PNR uses TLS certificates and private keys to protect network communication and authenticate with other servers.

For replication data, vSphere Replication encrypts replication traffic in transit for both new and existing replications. Network encryption is mandatory when using vSphere Replication 9.0.3 and later and is enabled by default. PNR also supports configuring network encryption of replication traffic as an option for replication scenarios where mandatory encryption does not apply automatically. PNR supports protection and recovery of encrypted virtual machines through both array-based and vSphere Replication protection groups, allowing VM disk encryption to persist through the replication process. Recovery of encrypted virtual machines requires that the protected and recovery sites share a common Key Management Server (KMS), or that the KMS clusters at both sites use common encryption keys, to maintain key availability after failover.

Defining the encryption policy for environments with stricter requirements and validating that workload traffic running on protected virtual machines meets confidentiality standards are organizational data-policy decisions.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) applies TLS-based encryption across multiple communication paths, covering database client connections, management interfaces, service-to-service communication, and data migration traffic. Starting with DSM, the platform enforces FIPS 140-3 compliant cryptographic standards and validates certificates during upload, rejecting unsupported algorithms such as Ed25519.

For database client connections, DSM enforces a minimum TLS protocol version across all supported database engines. PostgreSQL instances enforce a configurable minimum TLS protocol version via the `ssl_min_protocol_version` setting and support certificate-based authentication with CA verification. PostgreSQL replication connections support SSL/TLS encryption with a configurable `sslmode` parameter to secure data in transit between source and target databases, and DSM validates SSL trust before establishing replication. MySQL locks TLS protocol enforcement at the platform level so users cannot weaken it. MySQL has `require_secure_transport` set to ON by default, which requires all client connections to use encrypted transport. MySQL supports FIPS mode via the `ssl_fips_mode` setting. DSM adds SQL Server as a supported database engine; SQL Server clusters use TLS to encrypt client connections, require a minimum TLS version for DSM operator management, and expose the `network.tlsProtocols` engine setting to configure the set of allowed TLS versions.

For management and service-to-service communication, the DSM Provider exposes an HTTPS endpoint. The Consumption Operator requires a TLS certificate for secure communication with the DSM provider. In network-isolated deployments, Consumption Operator image transfers to a private registry support optional TLS certificate authentication. Metrics forwarding from DSM to VCF Operations supports mutual TLS (mTLS) authentication, requiring a client certificate and private key. DSM SMTP settings include an Auto TLS option that establishes a secure connection to the SMTP server.

DSM provides certificate management capabilities to support the TLS infrastructure. Custom certificates are validated for FIPS 140-3 compliance at upload time; accepted key types are ECDSA P-256, ECDSA P-384, and RSA with a minimum key size of 2048 bits, and Ed25519 keys are rejected as non-compliant. Trusted root certificates added through the DSM UI must comply with FIPS requirements.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides native, configurable cryptographic mechanisms to protect the confidentiality of data in transit at both the application data plane and the Controller management plane.

Each virtual service in Avi is associated with an SSL/TLS Profile, which defines the accepted protocol versions, cipher suites, and their priority order. SSL/TLS Profiles are managed at Templates > Security > SSL/TLS Profile in the Controller UI. Predefined profile templates are available, including a System-Standard template, and administrators can create custom profiles to enforce the cipher suites and protocol versions appropriate for their environment. Elliptic Curve (EC) cryptography is available for virtual services and supports Perfect Forward Secrecy with less computational overhead than RSA-based key exchange. TLS persistence state exchanged between Service Engines when a client reconnects is carried in encrypted form.

At the L4 proxy level, the Default-TLS protocol parser can be associated with a virtual service to inspect TLS handshake data using DataScript, supporting selective offload and re-encryption scenarios.

In Kubernetes environments, the Avi Kubernetes Operator (AKO) supports a PKIProfile custom resource definition that enables certificate-based authentication and encryption for backend service communication. The System-Standard SSL Profile can be applied through the RouteBackendExtension CRD to enable SSL/TLS re-encryption for traffic to backend servers, providing modern cipher suites for Kubernetes ingress and routing scenarios. In VCF environments, the Avi load balancer uses secrets to handle encrypted HTTPS traffic, configured through the VCF IaaS Service Console.

When FIPS mode is enabled on the Avi Controller cluster, all cryptographic operations across the Controller and Service Engines are restricted to FIPS 140-3 approved algorithms. Avi uses the OpenSSL FIPS Provider version 3.1.2 in this mode, which has been validated at FIPS 140-3 Security Level 1 and awarded Certificate #4985 by CMVP. Enabling FIPS mode is a cluster-level configuration that applies uniformly to all cryptographic material handled by the platform. FIPS mode does not support SSL Client Hello events in Avi DataScript, and post-quantum cryptographic capabilities are not available in FIPS mode.

The Avi Controller and Service Engines use SSH for secure management communication. The SE\_SECURE\_CHANNEL designation classifies traffic between the Controller and Service Engines to maintain a dedicated secure communication channel. Avi also integrates with Let's Encrypt via the ACME protocol to automate certificate issuance and renewal for virtual services, and with hardware security modules such as the Thales Luna HSM for external key management. Remote server file transfer supports Secure Copy Protocol (SCP) and Secure File Transfer Protocol (SFTP). For deployments requiring hardware acceleration, Avi Service Engine integrates with Intel QuickAssist Technology to offload bulk cryptographic operations following SSL/TLS handshake completion.

Avi DataScript exposes cryptographic functions for application-layer use cases. The `avi.crypto.encrypt()` function encrypts data using a caller-provided key, optional initialization vector, and algorithm parameter. The HTTP Cookie Encryption Gateway uses these functions to encrypt cookie values in HTTP responses, protecting session state carried between client and server. The `avi.utils.rand_bytes` function generates cryptographically secure random bytes for token and nonce generation.

Defining which application traffic requires encryption and configuring the virtual service profiles to apply protection at the appropriate boundary are organizational data-policy decisions.



## Control CR 4.2

> Information persistence: Purge sensitive information from resources before they are reassigned. Erase volatile memory: Capability to erase volatile memory upon power-off. Erase non-volatile memory: Capability to erase non-volatile memory upon command.

**SCF Controls:** DCH-09, SEA-08

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides several technical capabilities that support media sanitization workflows for virtualized infrastructure, though the full control requires organizational procedures for physical media handling, verification, and documentation.

#### Encryption-based cryptographic erasure

VCF supports encryption of virtual machine disk (VMDK) files, where data in an encrypted VMDK is never written in cleartext to storage or transmitted in cleartext over the network. For storage environments using vSAN data-at-rest encryption or VM Encryption, cryptographic erasure is available as a sanitization method: destroying the encryption keys associated with the storage renders the encrypted data unrecoverable without requiring block-level overwriting. This approach is recognized by industry-standard best practices as an acceptable sanitization technique for encrypted media. Administrators configure VM encryption through VMware vCenter and the key management infrastructure, which supports both KMIP-compatible external Key Management Servers and vSphere Native Key Provider.

The vSphere Web Services API provides `removeKey` and `removeKeys` methods for programmatic removal of cryptographic keys by ID, each with an optional `force` parameter. The `force` option is intended for cases where a key has been compromised or expired, removing it from use across the environment. These API methods support automated key destruction workflows as part of a cryptographic erasure process. On VMware ESX hosts, calling `removeKey` without the `force` option deletes the key from the host's key cache and renders any encrypted virtual machine using that key unusable, because ESX hosts do not track which keys are in use.

The No Cryptography Administrator role separates cryptographic privileges from routine administrative operations. This role includes all Administrator privileges except Cryptographic Operations, and it can be cloned to create custom roles with only specific cryptographic privileges. This separation helps restrict who can modify encryption settings or remove keys during media lifecycle activities.

#### vSAN disk sanitization features

vSAN includes a disk wipe capability for securely wiping storage devices. Before wiping ineligible disks, administrators must check the partition table and verify the data stored on those disks. When encryption is enabled on an existing vSAN cluster, the "Erase disks before use" option wipes existing data from storage devices as they are brought into encrypted disk groups. Devices added to a vSAN disk group that contain residual data or partition information must be cleaned before they can be claimed, providing a built-in sanitization step during device onboarding. vSAN also supports re-encrypting all data using new keys through a rolling disk reformat, which generates a new Key Encryption Key and new Data Encryption Keys.

#### ESX host configuration protection

ESX stores its configuration data in encrypted form on boot devices. When a Trusted Platform Module (TPM) 2.0 is available, encryption keys can be persisted in the TPM across reboots using the `esxcli system settings encryption set` command with the `--mode=TPM` parameter. This means host decommissioning can include clearing the TPM to render the stored configuration unrecoverable.

#### Orphaned disk identification and reclamation

VCF Operations provides capabilities for identifying orphaned virtual disks, defined as VMDKs on datastores that are not connected to any registered virtual machines and have not been modified during a defined period. Administrators can configure a reporting threshold that sets the minimum number of days before unattached VMDKs are flagged as orphaned.



The Reclaim Resources feature allows execution of resource reclamation processes, and administrators should verify the accuracy of reported orphaned disks before performing reclamation. A Reclaimable Orphaned Disks metric displays potential savings from removing orphaned VMDKs across all datastores. Identifying and removing orphaned disks that may contain residual sensitive data is a relevant step in the media sanitization lifecycle.

#### Datastore removal and virtual disk controls

When a VMFS datastore is destroyed via the RemoveDatastore API, the partitions that compose the VMFS volume are removed and the datastore becomes unavailable to all ESX systems after a rescan. Datastore resignaturing is irreversible because it overwrites the original VMFS UUID. These operations provide mechanisms for decommissioning storage volumes as part of a media reuse or disposal workflow. The delete operation for a VMFS datastore permanently deletes all files associated with virtual machines on the datastore. Before initiating deletion, Storage DRS and Storage I/O Control should be disabled for the datastore to prevent automated storage management processes from interfering with the deletion workflow.

VCF also provides the `isolation.tools.diskWiper.disable` setting, which controls whether guest operating systems can issue virtual disk wiper operations. In hardened environments, this setting is typically configured to restrict disk wiper access, giving administrators centralized control over disk sanitization activities rather than allowing individual virtual machines to initiate wipe operations.

#### Protecting sensitive data in operational files

Backup files for VCF Operations, VMware vCenter, and NSX are encrypted using AES-256-CBC encryption with SHA256 message digest. After file-based restore operations, administrators are directed to securely delete decrypted backup files and control access to them during the restore process.

Core dumps on vCenter systems are not encrypted and can contain memory contents that include credentials, encryption keys, or other sensitive data. Administrators should consider disabling core dumps on vCenter systems to restrict sensitive information from persisting in diagnostic files. Managing the lifecycle of these operational files is part of a complete media sanitization program.

VCF does not include dedicated media sanitization workflow tools, inventory tracking for media disposal, or chain-of-custody documentation capabilities. These organizational processes, along with physical media destruction procedures and verification of sanitization effectiveness, must be addressed through enterprise asset management and data governance programs outside of VCF.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 4.2(1)

> Information persistence: Purge sensitive information from resources before they are reassigned. Erase volatile memory: Capability to erase volatile memory upon power-off. Erase non-volatile memory: Capability to erase non-volatile memory upon command.

**SCF Controls:** DCH-09

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides several technical capabilities that support media sanitization workflows for virtualized infrastructure, though the full control requires organizational procedures for physical media handling, verification, and documentation.

#### Encryption-based cryptographic erasure

VCF supports encryption of virtual machine disk (VMDK) files, where data in an encrypted VMDK is never written in cleartext to storage or transmitted in cleartext over the network. For storage environments using vSAN data-at-rest encryption or VM Encryption, cryptographic erasure is available as a sanitization method: destroying the encryption keys associated with the storage renders the encrypted data unrecoverable without requiring block-level overwriting. This approach is recognized by industry-standard best practices as an acceptable sanitization technique for encrypted media. Administrators configure VM encryption through VMware vCenter and the key management infrastructure, which supports both KMIP-compatible external Key Management Servers and vSphere Native Key Provider.

The vSphere Web Services API provides `removeKey` and `removeKeys` methods for programmatic removal of cryptographic keys by ID, each with an optional `force` parameter. The `force` option is intended for cases where a key has been compromised or expired, removing it from use across the environment. These API methods support automated key destruction workflows as part of a cryptographic erasure process. On VMware ESX hosts, calling `removeKey` without the `force` option deletes the key from the host's key cache and renders any encrypted virtual machine using that key unusable, because ESX hosts do not track which keys are in use.

The No Cryptography Administrator role separates cryptographic privileges from routine administrative operations. This role includes all Administrator privileges except Cryptographic Operations, and it can be cloned to create custom roles with only specific cryptographic privileges. This separation helps restrict who can modify encryption settings or remove keys during media lifecycle activities.

#### vSAN disk sanitization features

vSAN includes a disk wipe capability for securely wiping storage devices. Before wiping ineligible disks, administrators must check the partition table and verify the data stored on those disks. When encryption is enabled on an existing vSAN cluster, the "Erase disks before use" option wipes existing data from storage devices as they are brought into encrypted disk groups. Devices added to a vSAN disk group that contain residual data or partition information must be cleaned before they can be claimed, providing a built-in sanitization step during device onboarding. vSAN also supports re-encrypting all data using new keys through a rolling disk reformat, which generates a new Key Encryption Key and new Data Encryption Keys.

#### ESX host configuration protection

ESX stores its configuration data in encrypted form on boot devices. When a Trusted Platform Module (TPM) 2.0 is available, encryption keys can be persisted in the TPM across reboots using the `esxcli system settings encryption set` command with the `--mode=TPM` parameter. This means host decommissioning can include clearing the TPM to render the stored configuration unrecoverable.

#### Orphaned disk identification and reclamation

VCF Operations provides capabilities for identifying orphaned virtual disks, defined as VMDKs on datastores that are not connected to any registered virtual machines and have not been modified during a defined period. Administrators can configure a reporting threshold that sets the minimum number of days before unattached VMDKs are flagged as orphaned.

The Reclaim Resources feature allows execution of resource reclamation processes, and administrators should verify the accuracy of reported orphaned disks before performing reclamation. A Reclaimable Orphaned Disks metric displays potential savings from removing orphaned VMDKs across all datastores. Identifying and removing orphaned disks that may contain residual sensitive data is a relevant step in the media sanitization lifecycle.

#### Datastore removal and virtual disk controls

When a VMFS datastore is destroyed via the RemoveDatastore API, the partitions that compose the VMFS volume are removed and the datastore becomes unavailable to all ESX systems after a rescan. Datastore resignaturing is irreversible because it overwrites the original VMFS UUID. These operations provide mechanisms for decommissioning storage volumes as part of a media reuse or disposal workflow. The delete operation for a VMFS datastore permanently deletes all files associated with virtual machines on the datastore. Before initiating deletion, Storage DRS and Storage I/O Control should be disabled for the datastore to prevent automated storage management processes from interfering with the deletion workflow.

VCF also provides the `isolation.tools.diskWiper.disable` setting, which controls whether guest operating systems can issue virtual disk wiper operations. In hardened environments, this setting is typically configured to restrict disk wiper access, giving administrators centralized control over disk sanitization activities rather than allowing individual virtual machines to initiate wipe operations.

#### Protecting sensitive data in operational files

Backup files for VCF Operations, VMware vCenter, and NSX are encrypted using AES-256-CBC encryption with SHA256 message digest. After file-based restore operations, administrators are directed to securely delete decrypted backup files and control access to them during the restore process.

Core dumps on vCenter systems are not encrypted and can contain memory contents that include credentials, encryption keys, or other sensitive data. Administrators should consider disabling core dumps on vCenter systems to restrict sensitive information from persisting in diagnostic files. Managing the lifecycle of these operational files is part of a complete media sanitization program.

VCF does not include dedicated media sanitization workflow tools, inventory tracking for media disposal, or chain-of-custody documentation capabilities. These organizational processes, along with physical media destruction procedures and verification of sanitization effectiveness, must be addressed through enterprise asset management and data governance programs outside of VCF.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 4.2(2)

> Information persistence: Purge sensitive information from resources before they are reassigned. Erase volatile memory: Capability to erase volatile memory upon power-off. Erase non-volatile memory: Capability to erase non-volatile memory upon command.

**SCF Controls:** DCH-09

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides several technical capabilities that support media sanitization workflows for virtualized infrastructure, though the full control requires organizational procedures for physical media handling, verification, and documentation.

#### Encryption-based cryptographic erasure

VCF supports encryption of virtual machine disk (VMDK) files, where data in an encrypted VMDK is never written in cleartext to storage or transmitted in cleartext over the network. For storage environments using vSAN data-at-rest encryption or VM Encryption, cryptographic erasure is available as a sanitization method: destroying the encryption keys associated with the storage renders the encrypted data unrecoverable without requiring block-level overwriting. This approach is recognized by industry-standard best practices as an acceptable sanitization technique for encrypted media. Administrators configure VM encryption through VMware vCenter and the key management infrastructure, which supports both KMIP-compatible external Key Management Servers and vSphere Native Key Provider.

The vSphere Web Services API provides `removeKey` and `removeKeys` methods for programmatic removal of cryptographic keys by ID, each with an optional `force` parameter. The `force` option is intended for cases where a key has been compromised or expired, removing it from use across the environment. These API methods support automated key destruction workflows as part of a cryptographic erasure process. On VMware ESX hosts, calling `removeKey` without the `force` option deletes the key from the host's key cache and renders any encrypted virtual machine using that key unusable, because ESX hosts do not track which keys are in use.

The No Cryptography Administrator role separates cryptographic privileges from routine administrative operations. This role includes all Administrator privileges except Cryptographic Operations, and it can be cloned to create custom roles with only specific cryptographic privileges. This separation helps restrict who can modify encryption settings or remove keys during media lifecycle activities.

#### vSAN disk sanitization features

vSAN includes a disk wipe capability for securely wiping storage devices. Before wiping ineligible disks, administrators must check the partition table and verify the data stored on those disks. When encryption is enabled on an existing vSAN cluster, the "Erase disks before use" option wipes existing data from storage devices as they are brought into encrypted disk groups. Devices added to a vSAN disk group that contain residual data or partition information must be cleaned before they can be claimed, providing a built-in sanitization step during device onboarding. vSAN also supports re-encrypting all data using new keys through a rolling disk reformat, which generates a new Key Encryption Key and new Data Encryption Keys.

#### ESX host configuration protection

ESX stores its configuration data in encrypted form on boot devices. When a Trusted Platform Module (TPM) 2.0 is available, encryption keys can be persisted in the TPM across reboots using the `esxcli system settings encryption set` command with the `--mode=TPM` parameter. This means host decommissioning can include clearing the TPM to render the stored configuration unrecoverable.

#### Orphaned disk identification and reclamation

VCF Operations provides capabilities for identifying orphaned virtual disks, defined as VMDKs on datastores that are not connected to any registered virtual machines and have not been modified during a defined period. Administrators can configure a reporting threshold that sets the minimum number of days before unattached VMDKs are flagged as orphaned.

The Reclaim Resources feature allows execution of resource reclamation processes, and administrators should verify the accuracy of reported orphaned disks before performing reclamation. A Reclaimable Orphaned Disks metric displays potential savings from removing orphaned VMDKs across all datastores. Identifying and removing orphaned disks that may contain residual sensitive data is a relevant step in the media sanitization lifecycle.

#### Datastore removal and virtual disk controls

When a VMFS datastore is destroyed via the RemoveDatastore API, the partitions that compose the VMFS volume are removed and the datastore becomes unavailable to all ESX systems after a rescan. Datastore resignaturing is irreversible because it overwrites the original VMFS UUID. These operations provide mechanisms for decommissioning storage volumes as part of a media reuse or disposal workflow. The delete operation for a VMFS datastore permanently deletes all files associated with virtual machines on the datastore. Before initiating deletion, Storage DRS and Storage I/O Control should be disabled for the datastore to prevent automated storage management processes from interfering with the deletion workflow.

VCF also provides the `isolation.tools.diskWiper.disable` setting, which controls whether guest operating systems can issue virtual disk wiper operations. In hardened environments, this setting is typically configured to restrict disk wiper access, giving administrators centralized control over disk sanitization activities rather than allowing individual virtual machines to initiate wipe operations.

#### Protecting sensitive data in operational files

Backup files for VCF Operations, VMware vCenter, and NSX are encrypted using AES-256-CBC encryption with SHA256 message digest. After file-based restore operations, administrators are directed to securely delete decrypted backup files and control access to them during the restore process.

Core dumps on vCenter systems are not encrypted and can contain memory contents that include credentials, encryption keys, or other sensitive data. Administrators should consider disabling core dumps on vCenter systems to restrict sensitive information from persisting in diagnostic files. Managing the lifecycle of these operational files is part of a complete media sanitization program.

VCF does not include dedicated media sanitization workflow tools, inventory tracking for media disposal, or chain-of-custody documentation capabilities. These organizational processes, along with physical media destruction procedures and verification of sanitization effectiveness, must be addressed through enterprise asset management and data governance programs outside of VCF.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 4.3

> Use of cryptography: Use industry-recognized cryptographic standards and algorithms.

**SCF Controls:** CRY-01

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF implements cryptographic protections across its component stack using publicly standardized, validated cryptographic modules and algorithms.

Multiple VCF components rely on cryptographic modules validated through the NIST Cryptographic Module Validation Program (CMVP) to FIPS 140-2 and FIPS 140-3 standards. NSX is configured to use FIPS 140-3 validated cryptographic modules by default, and NSX Manager uses FIPS 140 approved algorithms for authentication to its cryptographic module. Specific NSX validated modules include IKE with Rambus Safezone 2.0 (Certificate #4898), VMware OpenSSL 3.0.9 (Certificate #4861), VMware BouncyCastle 2.0.0 (Certificate #4986), and VMware BoringCrypto 6.0 (Certificate #4694). VCF Automation uses the BouncyCastle FIPS 2.0.0 cryptographic module, which holds FIPS 140-3 Certificate #4743. VCF Operations for Networks uses BC-FJA (Bouncy Castle FIPS Java API) version 2.0.0 with FIPS 140-2 Certificate #37009 as its validated cryptographic module. This consistent use of FIPS-validated modules across the platform means that organizations deploying VCF in regulated environments can rely on cryptographic implementations that have undergone independent validation against well-known public standards. The VMware Kubernetes Service (VKS) consumption layer also supports FIPS mode: VKS ClusterClass configurations include a `fips.enabled` variable that, when set, restricts cryptographic operations on cluster nodes to approved algorithm implementations. FIPS-enabled Kubernetes release images, identified by a `-fips` suffix in the release reference name, are available for organizations that require FIPS-validated cryptography across their containerized workloads.

At the protocol and algorithm level, VCF enforces modern cryptographic standards throughout. Starting in vSphere 9.0, all TLS profiles are FIPS compliant; the COMPATIBLE profile supports modern TLS cipher suites with standardized elliptic curves. Monitoring infrastructure such as Telegraf enforces a modern TLS minimum version for secure communication. The NSX L2 VPN FIPS compliance suite uses AES-GCM encryption with Perfect Forward Secrecy and Diffie-Hellman Group 20. NSX also supports ECDSA (Elliptic Curve Digital Signature Algorithm) encryption for environments with high-assurance cryptographic requirements. VCF Operations for Networks supports several encryption algorithms and ciphers for its data sources. VCF Operations provides a FIPS Security Mode that, when enabled, applies additional algorithm and cipher prerequisites across the management domain. All system management network communications are encrypted by default using standardized cipher suites. At the VKS layer, Kubernetes control plane components expose parameters for restricting permitted TLS configurations: the `kube-apiserver --tls-cipher-suites` parameter, the `kubelet tlsCipherSuites` configuration field, and the equivalent parameter on the `kube-scheduler` each restrict the set of permitted cipher suites, allowing administrators to exclude deprecated or weak algorithms. The `kubelet tlsMinVersion` field and the `--tls-min-version` parameter on the `kube-apiserver` enforce a minimum TLS protocol version floor for inbound connections. The Antrea networking component and vSphere Cloud Provider Interface on VKS clusters also support configurable TLS cipher suites for component-to-component communication. Kubernetes hardening guidance identifies specific preferred cipher suites and calls out cipher suite configuration on the `kube-scheduler` as a required hardening step. VKS clusters support the Istio service mesh package, which provides mutual TLS between workloads within the mesh; the `meshMTLS.minProtocolVersion` setting controls the minimum protocol version for inter-service communication, defaulting to TLS 1.2 with support for configuring TLS 1.3.

Data-in-transit protections using TLS have customizable PKI keypair and certificate options. These range from fully automated within VCF to semi-automated using certificates from enterprise PKI infrastructure or fully custom certificates installed manually. Automated certificate issuance is supported with Microsoft Active Directory Certificate Services. Trust establishment uses standard PKI management with multiple certificate options including Root CA certificates, CSR-based certificates, and direct certificate uploads. VCF Operations provides centralized certificate lifecycle management for all VCF components, enabling automated certificate renewal, monitoring, and replacement across the infrastructure. Organizations should be aware that if self-signed certificates and private keys are generated using OpenSSL, those certificates and private keys are not FIPS-

compatible. Environments operating under FIPS requirements should use certificate authorities and key generation methods that produce FIPS-compliant artifacts. For FIPS environments, certificates should use key sizes of 2048 or 3072 bits; key sizes greater than 3072 bits have not been tested with FIPS. At the VKS layer, the cert-manager package manages the lifecycle of certificates issued to workloads and ingress endpoints. VKS clusters maintain TLS certificates for component-to-component communication across their control and data planes. The Contour ingress package supports minimum TLS version configuration through the `tls.minimum-protocol-version` parameter. Harbor, when deployed as a Supervisor Service, accepts custom TLS certificate configuration through the `tlsCertificate.tls.crt` and `tlsCertificate.tls.key` fields, allowing organizations to supply certificates from their enterprise certificate authority rather than relying on self-signed certificates. The Supervisor secret injection agent automatically configures mutual TLS for communication with the vault-agent-injector service using auto-generated certificates.

Data-at-rest encryption uses vSAN cluster-based encryption or VM Encryption on non-vSAN storage. vSAN encryption uses AES-NI CPU offloading for efficient processing, and VMware ESX hosts encrypt vSAN disk data using the industry standard AES-256 XTS mode. Encryption keys are managed through KMIP 1.1-compatible external Key Management Servers (KMS) or vSphere Native Key Provider (NKP), which is included in all vSphere editions and does not require an external key server. VCF implements a three-element domain of trust comprising the key provider, VMware vCenter, and vSAN hosts. Key Encryption Keys (KEK) wrap Disk Encryption Keys (DEK) for secure key management; ESX hosts use the KEK to encrypt their internal keys and store only the encrypted internal keys on disk, not the KEK itself.

VCF Standard Key Providers have key wrapping capabilities that let organizations store a single wrapping key in their KMS, protecting multiple Key Encryption Keys within the VCF environment. This creates a three-tier key hierarchy: the Data Encryption Key encrypts object data, the Key Encryption Key protects the DEK, and the wrapping key protects the KEK. This approach reduces the number of keys stored in external KMS systems while maintaining cryptographic separation. Key wrapping includes automatic key rotation with configurable intervals from 30 days to 10 years.

With key persistence enabled, ESX hosts can persist encryption keys in the Trusted Platform Module (TPM) across reboots, allowing encryption operations to continue when the key server is unavailable. Each ESX host obtains the encryption keys initially and retains them in its key cache; if the ESX host has a TPM, the encryption keys are persisted in the TPM across reboots. Key persistence can be enabled using the `esxcli system security keypersistence enable` command.

VCF supports guest-level security features including virtual Trusted Platform Module (vTPM) 2.0 for VMs. This enables in-guest encryption capabilities such as Windows BitLocker, Linux disk encryption, and other operating system-native security features. Secure Boot provides UEFI verification to help ensure only signed and trusted code runs during the VM boot process. These features work independently of or in combination with infrastructure-level encryption, allowing organizations to implement defense-in-depth strategies.

vSAN offers Data-in-Transit (DIT) encryption as a cluster-wide setting that encrypts all data and metadata traffic as it transits across hosts. For vSAN Original Storage Architecture (OSA), data-at-rest encryption occurs at the Local Object Manager (LSOM) layer on the target host, making DIT encryption strongly recommended to restrict network-level interception. For vSAN Express Storage Architecture (ESA) in HCI mode, data is encrypted at the VM host before network transmission, making DIT encryption optional based on compliance requirements and risk tolerance. DIT encryption operates without requiring external key management servers and provides independent key rotation from data-at-rest encryption.

Encrypted vSphere vMotion secures confidentiality, integrity, and authenticity of data transferred during VM migrations. For encrypted VMs, vMotion always uses encryption. For unencrypted VMs, encrypted vMotion can be set to Disabled, Opportunistic (default), or Required. Cross-vCenter vMotion is supported when using shared key providers or backup/import of Native Key Provider. VCF uses one-time keys (nonces) for each migration session so that captured traffic cannot be replayed or decrypted after migration completes.

Memory Tiering encryption provides protection for tiered memory without requiring external key management. VCF generates random 256-bit AES-XTS keys at the kernel level during VM power-on, unique to each VM instance or per host depending on configuration.



VM Encryption provides protection across all datastore types including vSAN, iSCSI, NFS, and Fibre Channel. Disk encryption uses XTS-AES-256 keys as the disk encryption key. This encryption occurs before data reaches the storage layer, providing both data-at-rest and data-in-transit protection.

Kubernetes on VKS supports encryption at rest for data stored within the control plane. Operators configure the encryption provider and manage key access for control plane secrets; when relying on a managed key provider, administrators retain responsibility for access controls over the managed key material. The VCF Automation consumption layer includes VCF Encryption Management, which grants organization administrators the ability to use encryption keys from their own key providers for encryption of virtual machines and disks in their VCF environment, extending the platform's cryptographic key management to tenant-level operations.

For storage protocols, VCF provides varying levels of native encryption support. iSCSI includes CHAP for session authentication. Fibre Channel Gen 7 supports end-to-end encryption with secure HBAs. For NFS datastores, version 4.1 supports Kerberos authentication with three security levels: krb5 (authentication only), krb5i (authentication plus integrity checking), and krb5p (authentication plus privacy with full encryption).

ESX stores configuration data in encrypted form on boot devices. When the host has a TPM 2.0 enabled and configured, the configuration encryption key is stored in the TPM. The encryption mode can be configured using the `esxcli system settings encryption set` command with the `--mode=TPM` parameter.

Cryptographic operations are controlled through fine-grained privileges in vCenter. Only users assigned the Cryptographic Operations privileges can perform cryptographic operations. A dedicated No Cryptography Administrator role enables standard VM and infrastructure management while restricting unauthorized encryption changes. This role can be cloned and customized to grant only specific Cryptographic Operations privileges, such as allowing encryption but not decryption, supporting the principle of least privilege for encryption tasks. The `CryptoManager` and `CryptoManagerKmp` managed objects provide programmatic interfaces for handling cryptographic keys and integrating with key management server infrastructure. vSAN encryption key management requires specific permissions including `Cryptographer.ManageEncryptionPolicy`, `Cryptographer.ManageKeyServers`, and `Cryptographer.ManageKeys`.

vSAN includes secure disk wipe capabilities for proper media sanitization when decommissioning storage, supporting the Erase disks before use option during encryption configuration and disk management operations.

Defining the organization's cryptographic policy (algorithm selection, key strength, rotation schedules, KMS architecture, FIPS-mode activation decisions) and the key lifecycle management standards that govern key creation, distribution, storage, archive, and destruction sit with the organization above the platform. VCF supplies the validated cryptographic modules and protocol enforcement; the organizational cryptographic governance program determines how those mechanisms apply across the environment. Security Technical Implementation Guides (STIGs) for Supervisor and VKS releases are available to guide cryptographic hardening of the consumption layer.

VMware Private AI Services (PAIS) provides cryptographic controls across its AI infrastructure components, building on the cryptographic foundation of VCF.

GPU-accelerated VMware Kubernetes Service (VKS) clusters deployed for PAIS workloads use a FIPS-enabled Kubernetes runtime. The VKS cluster topology version applied to both control plane and worker deployments uses a FIPS-enabled VKR build running on Ubuntu 24.04, so AI workloads on PAIS VKS clusters operate within a FIPS-compliant Kubernetes runtime.

PAIS endpoints use TLS for service communication. The `PAISConfiguration` custom resource supports configurable CA bundle references through `spec.clientTls.caBundleRefs` for OIDC provider and Harbor registry authentication. The Supervisor endpoint communication for PAIS uses HTTPS on port 6443 with certificate-based authentication. The PAIS Prometheus metrics endpoint uses self-signed certificate authentication with TLS, where the CA certificate is retrieved from the `pais-ingress-default Secret` in the PAIS namespace and configured in base64-decoded form. Prometheus TLS client authentication and certificate validation are off by default; auditors should verify that client authentication is enabled in production deployments where mutual certificate validation is required.

PAIS requires CA trust bundles to be configured for all external HTTPS connections, including OIDC providers, Harbor registries, content management systems used as data sources in the Data Indexing and Retrieval service (such as Confluence or SharePoint), MCP servers that provide tools to agents, and the pgvector database server used for vector storage. Trust bundle material is obtained from each external system's administrator either as a certificate file or as a ConfigMap stored in the namespace where PAIS is deployed, and PAIS recommends maintaining separate trust bundles per application with periodic updates before certificate expiration. Administrators add trusted certificates through the VCF Automation Provider Management UI under Certificate Management and Trusted Certificates. The issuer certificate of the Harbor registry must also be added to the certificate trust store on Deep Learning VMs that pull models from the registry. When connecting to an MCP server over HTTPS, the MCP server issuer certificate must be added as a CA trust bundle.

The underlying cryptographic algorithms, cipher suites, and platform-level key management are provided by VCF and VKS. Organizations are responsible for selecting and enforcing approved cryptographic standards at the VCF and infrastructure level, while PAIS provides the certificate management and trust chain configuration mechanisms specific to AI service endpoints.

### **VMware vDefend (Contributes)**

VMware vDefend contributes to the implementation of cryptographic protection controls by using known public standards and trusted cryptographic technologies across its component stack for securing inter-component communications and providing cryptographic inspection capabilities for network traffic.

Security Services Platform (SSP), the control-plane component of vDefend, secures communications between vDefend product components using certificates and current TLS protocol versions with approved cipher suites. SSP and the SSP Installer deploy with the most current TLS version by default and also support the prior major TLS version. The TLS versions and cipher suites allowed on SSP Installer and SSP are fixed and cannot be modified, which enforces a consistent cryptographic baseline across deployments.

SSP components, including the SSP Installer and the NDR Sensor for vDefend, implement FIPS 140-2 and FIPS 140-3 cryptographic module standards. SSP uses the VMware BouncyCastle Module (NIST CMVP Certificate #4943) and the VMware OpenSSL FIPS provider Object Module (NIST CMVP Certificate #4861) for its cryptographic operations. SSP actively enforces cryptographic hygiene by rejecting any certificate or TLS configuration that uses deprecated algorithms, short keys, or legacy protocol versions, returning a validation error rather than permitting degraded connections.

Certificate management within SSP provides lifecycle controls including certificate export, certificate signing request (CSR) generation, and CA-signed certificate replacement. SSP monitors and manages several certificate types used to identify client and host machines, establish trust between components, secure communications, and control access levels. Administrators can retrieve and validate LDAP certificates using standard tools to establish secure LDAPS connections for platform authentication. These capabilities allow organizations to integrate vDefend into their broader PKI infrastructure and follow standard certificate lifecycle practices.

vDefend provides TLS Inspection capabilities that allow the firewall to decrypt, inspect, and re-encrypt TLS-protected traffic. Without TLS Inspection, encrypted traffic cannot be examined or enforced against security policies even when other advanced security features are enabled. With TLS Inspection enabled, vDefend gains visibility into encrypted traffic flows, supporting more effective access control and threat detection. TLS Inspection also supports importing or generating trusted and untrusted proxy certificate authorities for external decryption profiles, giving administrators control over the CA chain used in inspection.

TLS Inspection includes a Crypto Enforcement option within decryption action profiles that allows administrators to set minimum and maximum protocol version and cipher suite constraints for both client and server connections. Profiles can be configured in Transparent or Enforce modes; the Enforce mode requires compliance with the configured cryptographic constraints.

vDefend IDS/IPS signature rules extend cryptographic visibility by supporting protocol version matching and certificate Subject field matching, enabling enforcement decisions based on certificate metadata in encrypted traffic. IDS/IPS rules also support storing TLS/SSL certificates to disk for forensic analysis and threat investigation. SSP guidance recommends that

organizations restrict or block protocols that carry data in cleartext and enforce minimum secure protocol versions as part of their network security practices.

vDefend does not itself provide the underlying key management infrastructure (KMS), platform-wide PKI lifecycle management, or the broader VCF platform's cryptographic controls; those responsibilities belong to VCF. However, vDefend's consistent use of approved cryptographic standards for its own operations and its ability to perform cryptographic inspection of network traffic contribute to an organization's overall implementation of cryptographic protection controls.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) implements cryptographic protections for disaster recovery workflows using known public standards and trusted cryptographic technologies.

PNR uses modern TLS by default for all connections to the PNR server. The minimum TLS protocol version is configurable through the `envoy-proxy-config.json` file by editing the `tls_minimum_protocol_version` parameter. When the minimum protocol version is modified, PNR requires that the change be applied consistently across all relevant configuration occurrences in the system. Administrators can verify the TLS version in use on the Site Recovery Manager component using standard OpenSSL tooling.

The PNR Appliance requires an SSL/TLS certificate as the endpoint certificate for all TLS connections to the PNR server. PNR supports custom certificates signed by a certificate authority as an alternative to self-signed certificates. Custom SSL/TLS server endpoint certificates must meet specific requirements, including x509v3 Extended Key Usage indicating TLS Web Server Authentication. The Appliance Management Interface allows administrators to import CA-signed server certificates and custom root CA certificates, and to replace the appliance certificate. PNR uses TLS certificates and private keys to protect network communication and establish authentication with other servers. Certificate verification error messages are logged in security-related log files, and operational log files do not contain sensitive information such as private keys or passwords.

For encrypted virtual machine replication, PNR requires that the recovery and protected sites use a common Key Management Server (KMS) or that the KMS clusters at both sites share common encryption keys. Replication of encrypted virtual machines also requires Secure LWD support to be available on the virtual machine. This supports data-at-rest encryption continuity for virtual machines through the recovery lifecycle.

Defining the organization's cryptographic policy, including KMS architecture decisions, minimum TLS version selection, certificate management standards, and key lifecycle practices, sits with the organization above the DR layer. PNR supplies the cryptographic mechanisms needed for DR-specific traffic and operations; the organizational cryptographic governance program determines how those mechanisms apply across the environment.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) implements cryptographic protections across multiple layers of its architecture, applying standard cryptographic technologies for data in transit, certificate trust management, and credential storage.

Starting with DSM, the platform enforces FIPS 140-3 compliant cryptographic standards and validates certificates during upload, rejecting unsupported algorithms or parameters. Only RSA and ECDSA public key algorithms are accepted for certificates. The Ed25519 elliptic curve signature algorithm is rejected because it does not comply with FIPS 140-3. Custom trusted certificates should use ECDSA P-256 or P-384 algorithms; RSA certificates must meet a minimum key size of 2048 bits per FIPS requirements. DSM's latest certificate validation mode enforces strict cryptographic and KeyUsage validations across all DSM endpoints, including the Provider VM and data service clusters.

For connections to database clusters, DSM requires TLS for all client traffic. PostgreSQL database instances enforce a minimum TLS protocol version through the `ssl_min_protocol_version` setting. MySQL enforces minimum TLS versions on both the main connection interface and the administrative interface; these settings are fixed and cannot be modified by users, which helps maintain a consistent cryptographic baseline across deployments. MySQL also exposes the `group_replication_recovery_tls_version` setting to specify the TLS protocol used during distributed recovery connections. DSM supports SQL Server clusters, which use TLS to encrypt communication between clients and the cluster; the `network.tlsProtocols` custom engine setting defines the allowed TLS versions for client connections, and a minimum TLS

version must be included for the DSM operator to manage the cluster. DSM supports TLS verification during PostgreSQL database migration, covering replication traffic between source and target clusters.

DSM's certificate management capabilities support standard PKI practices. Administrators configure custom TLS certificates for database clusters through the DSM UI or API, providing certificates in PEM format with associated private keys and CA certificates. Supported private key algorithms are RSA and ECDSA with key sizes of 256 and 384 bits. When present, the X509v3 KeyUsage extension on a custom certificate must include the DigitalSignature capability; DSM also accepts certificates that omit the KeyUsage extension entirely, treating absent KeyUsage as permitting all usages in accordance with RFC 5280. To establish secure connections to database clusters and to the DSM Provider VM, clients add the CA to the operating system trust store or the trusted CA list. DSM's API provides a mechanism to retrieve the CA certificate from a custom certificate secret. The Consumption Operator, which enables self-service database provisioning, requires a TLS certificate for its communication with the DSM provider. Certificate management can also be handled through Kubernetes cert-manager, which manages various issuer types and outputs Kubernetes TLS Secrets in the format DSM requires.

DSM supports TLS for connections to external systems. When configuring log forwarding to an SMTP server, DSM supports Auto TLS to establish a secure connection. For private container registries, DSM supports CA certificate validation for untrusted registries.

For local user credential storage, DSM encrypts passwords using PGP symmetric encryption through the PGCrypto library, with dedicated encryption keys stored using the PGCRYPTO\_PASSWORD and PGCRYPTO\_ALGORITHM keys. MySQL has FIPS mode available on the client side through the `ssl_fips_mode` setting. PostgreSQL databases include the `pgcrypto` extension, which provides cryptographic functions that applications can use directly within the database. The DSM Provider Appliance includes virtual machine encryption, which activates when a Key Management Server is configured in the environment.

### VMware Avi Load Balancer (Contributes)

VMware Avi Load Balancer provides configurable SSL/TLS profiles that control which cipher suites, protocol versions, and key exchange algorithms are permitted for application traffic termination. The SSL/TLS Profile, configured under Templates > Security > SSL/TLS Profile, contains the accepted protocol version list and a prioritized cipher suite list applied to virtual services. Administrators can create custom profiles or select from built-in templates; the documentation notes that the choice of accepted ciphers and protocol versions involves trade-offs between security, client compatibility, and computational expense. SSL profiles in 9.1 include a Post Quantum Cryptography section with support for modern cipher suites. Controller SSL/TLS certificates support elliptic curve (EC) or RSA algorithms, with EC recommended for new deployments.

For environments subject to cryptographic module validation requirements, Avi supports a FIPS mode backed by the OpenSSL FIPS Provider, which has been validated at FIPS 140-3 Security Level 1 and awarded Certificate #4985 by the Cryptographic Module Validation Program (CMVP). When FIPS mode is activated on the Controller cluster, Avi restricts cryptographic operations to FIPS-compliant algorithms only. FIPS 140-3 also requires stricter handling of sensitive security parameters including keys, authentication data, and random number generation. FIPS mode is enabled at the Controller cluster level through the Controller settings. Note that FIPS mode does not support SSL Client Hello events in Avi DataScript, which may affect certain scripted traffic inspection use cases.

Avi Controller allows administrators to restrict specific key exchange algorithms and ciphers for management session access. The Allowed Ciphers field restricts management session ciphers to a specified subset, and the `kex_algorithm_exclude` field under Controller Settings excludes specific key exchange methods for SSH-based administrative access. Internal communications between the Avi Controller and Service Engines also use SSL/TLS, and custom certificates can be selected for this Secure Channel connection via the Access tab certificate field.

For certificate management, Avi integrates with Hardware Security Modules (HSMs), including the Thales Luna (formerly SafeNet Luna) HSM, for cryptographic key storage and management. The Controller is set up as an HSM client and can create keys and certificates directly on the HSM through client libraries, supporting high-availability HSM configurations. Automated certificate lifecycle management is available through Let's Encrypt integration using the ACME protocol. Avi's App Transport

Security feature supports integration with the Venafi Trust Protection Platform for certificate administration across tenants, with the Trust Protection Platform pushing signed certificates and required chain certificates to Avi Load Balancer.

For Kubernetes deployments, the Avi Kubernetes Operator (AKO) supports PKIProfile-based certificate validation and trust management for backend services. The PKIProfile, configured through custom resources, enables secure communication with backend services through certificate-based authentication and encryption. In VCF environments, the Avi PKI Profile defines the list of trusted certificate authorities permitted to sign client certificates and can be used to implement Zero Trust security between servers and clients.

Data-at-rest encryption for Avi Controller and Service Engine VMs is provided by VCF at the platform layer, not by Avi itself. Organizations must configure Avi SSL/TLS profiles, select appropriate cipher suites, and enable FIPS mode where cryptographic module validation requirements apply to activate these controls.

## Control CR 5.1

> Network segmentation: Capability to support logical or physical network segmentation.

**SCF Controls:** NET-06

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides network segmentation as a foundational design principle, using multiple layers of isolation to separate technology assets, applications, and services from one another.

At the physical and logical network layer, VCF uses VLANs and vSphere Distributed Switches (VDS) to segment traffic by function. Management traffic is isolated on dedicated networks accessible only to administrators. vMotion traffic is separated on distinct networks using dedicated VMkernel interfaces. IP-based storage traffic, including vSAN and NFS, runs on dedicated VLANs and can use dedicated physical network adapters for additional isolation. vSAN storage clusters can separate external VM traffic from internal storage traffic by using dedicated VMkernel ports for each traffic type. Network I/O Control (NIOC) on distributed switches divides traffic into predefined resource pools, including fault tolerance, iSCSI, vMotion, management, vSphere replication, NFS, and virtual machine traffic. NIOC can reserve bandwidth for each traffic type and allocate shares so that one class of traffic cannot starve another during contention. Active/standby NIC teaming configurations are available for vSAN traffic to provide resiliency without mixing storage and compute traffic paths. Distributed switch port groups also support Private VLANs (PVLANs), which further subdivide a VLAN segment into promiscuous, community, and isolated port types, restricting intra-VLAN communication to only explicitly permitted paths.

NSX extends segmentation with software-defined networking that decouples the virtual network from the physical infrastructure. NSX supports two primary segment models. The VLAN Segment Model provides isolation from other networks and can maintain separate security domains, such as isolating the fleet-level management network using VLAN-backed distributed port groups. VLAN segments can be provisioned by specifying one or more VLAN IDs on a private NSX network type. The Overlay Segment Model similarly provides isolation but decouples the virtual network infrastructure from the physical network, allowing segments to be created dynamically without reconfiguring the underlying physical network. This dual-model approach gives administrators flexibility to implement macrosegmentation using either traditional VLAN boundaries or software-defined overlay networks depending on architectural requirements.

NSX allows administrators to create logical network topologies using the NSX UI or API, building multi-tier architectures with Tier-0 and Tier-1 gateways and segments. Segments are realized as NSX Distributed Virtual Port Groups on vSphere Distributed Switches, providing consistent policy enforcement at the hypervisor level. In topologies running multiple standard overlay VDS switches, NSX provides traffic isolation such that traffic traversing one VDS is isolated from traffic on another VDS. NSX also supports applying switch security profiles to distributed port groups, and segment security features can filter malicious traffic from VMs on the network, helping protect segment integrity. NSX Distributed Port Groups and Uplink Port Groups also support Traffic Filtering and Marking policies that can be enabled and configured at the port level, allowing administrators to qualify, filter, or remark specific traffic flows at the point of ingress or egress on each port.

For multi-tenant and multi-department environments, NSX provides two isolation mechanisms. NSX Projects enable isolation of networking and security configurations across organizational units. NSX Virtual Private Clouds (VPCs) represent an additional layer of multi-tenancy within a project, providing self-contained virtual network environments with their own permission models, topology, and IP address management. VPCs support role-based access control through user and user group role assignments for Local User, LDAP User, or vIDM User account types. These can be allocated to discrete projects or tenants, enabling each to consume networking and security services in a self-service model.

VCF Automation extends segmentation into provisioned workloads by providing security groups for network isolation of deployed resources. This allows segmentation policies to be applied consistently through automated deployment workflows, so that new workloads are placed into the correct network segments from the point of provisioning.

The Consumption layer of VCF, which encompasses VMware Kubernetes Service (VKS) and vSphere Supervisor, extends network segmentation to containerized workloads using the same NSX networking infrastructure. vSphere Namespaces serve



as logical isolation units within VCF Kubernetes environments, providing resource and network boundaries between workloads deployed across different namespaces. A dedicated virtual network is provisioned for each namespace where a cluster is deployed and for each VKS cluster itself. Each VKS cluster is backed by a set of NSX networking objects: a Tier-1 gateway (router), a segment (switch) linked to that gateway, a load balancer server, and virtual servers for the cluster's control plane endpoints. VKS cluster nodes attach to these NSX-backed segments through VirtualNetworkInterface objects as logical ports, assigning each node an IP address and scoping cluster traffic to a dedicated segment isolated from other clusters and from the management plane. The management plane, which hosts VMware vCenter and NSX Managers, is architecturally separated from the compute plane, which hosts NSX Edge Nodes and Supervisor nodes, so management traffic does not mix with workload traffic at the network layer.

Within VKS clusters, Antrea serves as the CNI plugin and supports the standard Kubernetes NetworkPolicy API, allowing administrators to declare and enforce pod-level ingress and egress rules. By default, a pod is non-isolated for egress, meaning all outbound connections are permitted; a pod becomes isolated for egress only when a NetworkPolicy selects it and includes Egress in its policyTypes, at which point only connections explicitly allowed by applicable policies are permitted. A pod becomes isolated for ingress when any NetworkPolicy selects it with an Ingress rule, restricting inbound connections to those from the pod's own node and those explicitly permitted by applicable policies. Administrators can apply a namespace-scoped NetworkPolicy that denies all ingress and egress traffic, establishing a deny-all baseline from which only necessary flows are selectively opened; for environments requiring strict multi-tenant namespace isolation, the recommended approach is a default deny-all policy paired with an explicit allow for DNS resolution. Antrea distributed firewall policies can additionally restrict traffic between Kubernetes namespaces by applying filtering rules at specific pod groups, providing an enforcement layer that complements standard Kubernetes NetworkPolicies. The Antrea-NSX Adapter integrates each VKS cluster's networking with NSX Manager, enabling NSX to manage the networking behavior of the cluster and apply consistent segmentation policies across both the Kubernetes data plane and the underlying NSX infrastructure. Antrea's NetworkPolicyStats feature gate enables collection of network policy statistics, providing visibility into enforcement activity per workload. VKS cluster administrators control pod network attachment capabilities through NetworkAttachmentDefinition resources, which define the allowed network configurations that application users may request, preventing unauthorized network connections. For multi-network configurations, VKS uses SubnetConnectionBindingMap custom resources to bind overlay subnets to node vNIC subnets, with the vlanTrafficTag field specifying a unique VLAN ID for traffic isolation between parent and child subnets at the NSX data path level. The Antrea EgressSeparateSubnet feature allows egress IP allocation from a subnet distinct from the default node subnet, supporting stricter egress traffic segmentation. For SR-IOV deployments, VKS SR-IOV network configurations block cross-namespace selection of network resources, enforcing namespace-level boundaries for high-performance network access. For multi-tenant data plane separation, pod node selectors can restrict workload placement to specific nodes, preventing tenant workloads from sharing compute resources across tenant boundaries. Service meshes deployable on VKS clusters can extend segmentation to OSI Layer 7 by enforcing policies based on workload identity and providing mutual TLS encryption between workloads, adding isolation even within a shared network segment. The Argo CD Supervisor Service implements its own internal network segmentation and firewall rules to secure communication between its components.

For ongoing management and visibility, the NSX Segment Inventory Dashboard tracks and manages network segmentation while offering insights into the dependencies between logical switches and other NSX components. The NSX Switch Dashboard displays detailed information on logical switches for efficient tracking and management. VCF Operations for Networks supports segmentation planning by analyzing traffic flows and grouping them by entities such as VLAN, security group, application tier, subnet, cluster, or individual virtual machine. The micro-segmentation planning topology shows all flows present in an environment by dividing them into segments, helping administrators understand current traffic patterns and refine segmentation boundaries over time. Distributed virtual switches support health check operations for NIC teaming and VLAN/MTU configuration, and VCF Operations provides host metrics that detect unsupported MTU configurations across the infrastructure.

The VCF documentation states that network endpoints should be segmented. Administrators should configure VLAN or overlay segments appropriate to their environment, assign workloads to the correct segments, apply switch security profiles, and use the NSX dashboards and VCF Operations tools to verify and maintain segmentation boundaries. For



microsegmentation below the network segment level, VMware vDefend provides fine-grained traffic control between individual virtual machines and containers, independent of their network placement.

For operational technology (OT) and industrial workloads, NSX Industrial vSwitch (IvS) extends segmentation to PROFINET traffic between virtual Programmable Logic Controllers (vPLCs) running on VMware ESX hosts and I/O devices located outside the vSphere cluster. The associated NSX Industrial vSwitch Segment Profile supports the Discovery and Configuration Protocol (DCP) as part of the PROFINET protocol suite, allowing discovery of IO devices, retrieval of device identity information, and configuration of device settings such as device name and IP address. This capability brings OT traffic under the same segmentation model as standard IT workloads, enabling administrators to isolate industrial traffic on dedicated segments alongside the rest of the network architecture.

### **VMware vDefend (Meets)**

VMware vDefend (vDefend) provides network segmentation capabilities at multiple layers, from macrosegmentation at the network perimeter down to microsegmentation between individual workloads. This is a core function of the product and addresses the control directly.

VMware vDefend enforces segmentation at the workload level by applying security policies at the vNIC of every virtual machine and container. Policies follow the workload regardless of its network placement or physical location, so segmentation is not dependent on VLAN boundaries or physical network topology. Administrators can define rules based on security groups, application tiers, tags, and other logical constructs to isolate technology assets, applications, and services from one another. Layer 2 through Layer 7 inspection allows rules to be written against specific applications, protocols, and user behaviors rather than just IP addresses and ports. When using NSX Projects, firewall rules are automatically scoped to VMs connected to the segments within that Project and do not affect workloads outside the Project, providing a built-in boundary for multi-tenant or departmental segmentation.

Starting with VCF, vDefend supports enforcement on both Distributed Virtual Port Groups (DVPGs) created in vSphere Distributed Switch and segments created in NSX within the same cluster. This extends DFW coverage to VMs connected to traditional vSphere DVPG networks in addition to NSX overlay segments, allowing organizations to apply consistent security policy across mixed-network environments. DFW policy members can include DVPGs, NSX Segments, or both, and grouping constructs support dynamic or static membership criteria based on VM name, tags, segment, or IP addresses.

VMware vDefend provides stateful North-South traffic inspection at the network edge, controlling traffic entering and leaving each segment. Together, the Distributed Firewall and Gateway Firewall create a layered segmentation model where both East-West and North-South traffic are subject to policy enforcement.

Security Intelligence, delivered through the vDefend Security Services Platform, provides segmentation planning and monitoring tools. The Security Segmentation Report assigns a quantitative Security Segmentation Score that measures an organization's progress toward a fully segmented network. Segmentation planning workflows help administrators discover traffic flows, identify infrastructure services, group assets by environment (such as VLAN, security group, application tier, subnet, or cluster), and apply hierarchy tags to verify that segments have appropriate controls. Infrastructure services are categorized into their own groups separate from the asset hierarchy, with dedicated groups that users can create and apply policies to. Environment segmentation divides the data center into broader security zones such as production, development, testing, or regulated environments, and Security Intelligence workflows support planning and enforcement of policies at each zone boundary. Security Intelligence generates macrosegmentation policy recommendations for environment pairs based on observed traffic flow patterns; these recommendations can be reviewed, modified, and published to enforce segmentation policies through the Distributed Firewall.

Segmentation Monitoring continuously watches traffic for supported infrastructure services and detects new servers or services that appear on the network. The feature monitors supported infrastructure service types, including DHCP, DNS, NTP, and LDAP, and identifies both newly discovered servers and servers missing recommended security rules. Administrators can configure protection policies for environment pairs to Drop or Reject unauthorized traffic, and create lockdown rules to disallow traffic to servers outside defined infrastructure asset groups. Segmentation Monitoring provides detailed investigation

capabilities to identify potential security risks in application traffic and enable corrective action. This provides ongoing assurance that the segmentation scheme remains intact as the environment evolves.

Application-level microsegmentation implements granular isolation for individual applications. Segmentation planning supports import of application context using a flat file containing key workload attributes such as IP addresses, subnets, virtual machines, logical segments, and organizational roles. The Security Segmentation Report also identifies workloads running obsolete operating systems, workloads using risky application protocols, and workloads communicating with external IP addresses, helping administrators prioritize segmentation policy work.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 5.2

> Zone boundary protection: Control data flow through zone boundaries.

**SCF Controls:** NET-03

**Aggregate Coverage:** Meets

### VCF (Contributes)

VCF provides networking infrastructure and monitoring capabilities that support boundary protection at both external network boundaries and key internal boundaries within the virtual environment.

At the external boundary, NSX Edge nodes serve as the communication gateway for inter-location and northbound traffic, establishing a defined perimeter through which external communications traverse. NSX Edge supports policy-based and route-based IPsec VPN services on Tier-0, Tier-0 VRF, and Tier-1 gateways, enabling secure connectivity between sites and to external networks. NAT rules can be configured at the VPC level to perform IP translation between internal and external networks, controlling which workloads are reachable from outside the environment. External IP configurations allow outside connectivity for virtual machines on private networks by performing address translation through 1:1 NAT, granting access to specific workloads rather than exposing entire network segments.

For internal boundary control, VCF provides several layers of network segmentation. NSX segments, VLANs, and Private VLANs (PVLANS) allow administrators to create logical network boundaries that separate traffic at Layer 2. VMware distributed virtual switches support Private VLAN and VLAN-based SPAN for virtual distributed port mirroring. ESX port groups function as communication boundaries and can be configured with security policies to segment traffic between different groups of virtual machines. vSphere Distributed Switch port groups also support a Traffic Filtering and Marking Policy, which can permit or stop traffic on a per-port-group or per-port basis; port blocking policies provide a complementary mechanism to selectively prevent individual distributed ports from sending or receiving data. vSphere Distributed Switches provide centralized management of these boundaries across all hosts in a workload domain, applying consistent VLAN configurations and traffic shaping rules.

VCF supports multiple distributed switch models that isolate traffic at the physical layer. The Storage Separation Distributed Switch model isolates storage traffic onto a separate vSphere Distributed Switch from management and virtual networking traffic. The Workload Separation Distributed Switch model isolates VM Management traffic onto a separate switch from ESX Management, vMotion, Storage, and Virtual Networking traffic. In secure environments, these separation models can provide physical air-gapped separation of network traffic.

NSX Virtual Private Clouds (VPCs) provide isolated network environments within a shared infrastructure. VPCs support on-demand logical networks that can be advertised as public subnets or exposed through networking services like External IPs or NAT, providing project-level boundary isolation with their own topology, IP address management, and permission models. VCF Automation maintains secure organizational boundaries that isolate tenants from one another, providing additional logical separation within the environment.

Network I/O Control (NIOC) determines the bandwidth that different network traffic types are given on a vSphere distributed switch. When NIOC is enabled, distributed switch traffic is divided into predefined network resource pools covering Fault Tolerance, iSCSI, vMotion, management, vSphere Replication, NFS, and virtual machine traffic. Bandwidth reservation and shares allocation enforce performance boundaries between traffic types under contention. Avi Load Balancer provides Layer 4 load balancing for services, distributing traffic across server pools with configurable load balancing algorithms and health monitoring.

VCF provides network monitoring through multiple components. NSX supports IPFIX profile configuration on segments to enable flow-based network monitoring, giving administrators visibility into traffic traversing the network. At the vSphere layer, NetFlow can be enabled on individual distributed port groups or ports through the vSphere Distributed Switch monitoring policy, providing IP packet flow data independent of NSX. The pktcap-uw command-line utility on ESX hosts supports packet capture at specific points in the network stack, including at physical adapters and at the boundary between the distributed

switch and physical adapters, enabling packet-level traffic inspection without additional tools. NSX Manager monitors control plane connections between transport nodes and manager nodes, providing awareness of the network infrastructure state.

VCF Operations extends monitoring capabilities with network metrics including inbound and outbound connection tracking (net|allInboundTotal and net|allOutboundTotal). The Management Pack for Network Devices provides alerts and notifications for key network events including throughput totals and packet counts, and monitors outgoing errors on network interfaces. VCF Operations can detect protected flows, identifying traffic that has a specific firewall rule associated with it beyond a default allow-all policy, which helps administrators understand which communications are governed by explicit security policies. Outbound flow limits serve as a protection mechanism, with TCP flows limited to 1,000 allowed and 50 denied outbound, and UDP flows limited to 500 allowed and 50 denied outbound. VCF Operations also supports firewall hardening that restricts network access to its own internal services, demonstrating boundary protection for management interfaces.

VCF Operations for Networks adds flow analysis and troubleshooting capabilities, including application discovery based on flows, network topology visualization through the Topology Graph Widget, and traffic flow visualization between applications, application tiers, shared services, and the Internet. VCF Operations for Networks provides intents for network design and verification, helping validate that the network configuration matches the intended design. Distributed virtual switches support health check operations for NIC teaming and VLAN/MTU verification, while vSAN network partition health checks detect VLAN misconfigurations and uplink losses. VCF Operations for Networks also provides visibility into vDefend distributed firewall rule activity, including identification of inactive rules that have had no matching flows.

VMware Kubernetes Service (VKS) clusters and the vSphere Supervisor control plane extend boundary protection to the workload consumption layer. Within VKS clusters, Kubernetes NetworkPolicy objects provide declarative packet-filtering rules at OSI Layer 3 and Layer 4, defining permitted ingress and egress traffic based on pod label selectors, namespace selectors, and CIDR blocks. By default, pods accept all traffic; NetworkPolicy objects restrict communication to only the flows required for each workload or namespace, establishing internal boundaries between application tiers. Antrea, the CNI plugin integrated with Supervisor in NSX VPC deployments, enforces these NetworkPolicy rules and exposes NetworkPolicyStats, giving administrators measurable visibility into how boundary controls are applied across cluster-internal traffic. At the cluster perimeter, distributed firewall isolation rules on cluster nodes block external networks from reaching VKS cluster pods by default; Supervisor control plane nodes can reach cluster pods, but traffic originating outside the Supervisor control plane is dropped. Connections from worker nodes and pods to the Kubernetes API server use the API server's secure port by default, and SSH tunnels restrict control plane-to-node communications from exposure outside the cluster network. Kubernetes Ingress resources backed by ingress controllers perform TLS termination and route external requests to backend services, establishing a defined and controlled entry point at each cluster's external-facing edge. Network segmentation and firewall rules are applied to secure inter-component communication within Supervisor Service deployments such as Argo CD.

For stateful firewall inspection, microsegmentation, IDS/IPS, and deep packet inspection at network boundaries, organizations can deploy VMware vDefend, which extends VCF's networking foundation with Layer 2 through Layer 7 firewall capabilities and advanced threat prevention. VCF's networking layer provides the segmentation, routing, and monitoring infrastructure on which boundary enforcement policies operate, but the active enforcement of traffic at those boundaries requires either vDefend or integration with external security tools. Organizational processes for defining boundary policies, reviewing traffic logs, managing firewall rule sets, and maintaining network diagrams are also required.

VMware Private AI Services (PAIS) contributes to boundary protection through AI-infrastructure-specific network isolation patterns for both connected and air-gapped deployments. The core boundary enforcement capabilities, including distributed firewall rules, IDS/IPS inspection, and traffic analysis, are provided by VCF and VMware vDefend.

PAIS setup is configured according to environment Internet connectivity, supporting either connected or disconnected air-gapped deployment models. In a disconnected air-gapped environment, files are downloaded from the Internet to a bastion host VM and then transferred to an admin host VM that has no Internet access, from which VI administrators interact with the environment. This pattern removes direct Internet exposure from the AI infrastructure components and constrains the path by which external content enters the trusted enclave.

In air-gapped deployments, PAIS configures components to pull container images from a local internal registry rather than from public sources. The NVIDIA GPU Operator deploys validator, operator, driver, manager, toolkit, and devicePlugin components as separate container images pulled from the local registry. PAIS model endpoints authenticate to private registries through harbor-pull-secret-readonly to control access to container images.

PAIS integrates with Harbor registries as a model storage backend, requiring authentication and certificate-based trust for model retrieval. The Model Gallery uses Harbor project access capabilities to restrict access to sensitive data used to train and tune models, providing an internal boundary between AI workload data stores and other platform consumers. PAIS uses a CA trust bundle for secure communication with external services and databases.

The NSX Edge cluster or Virtual Network Appliance (VNA) cluster provides two-tier north-south routing for the Supervisor instance and the workloads it runs, while the Workload Domain NSX Manager provides VPC networking services to private AI workloads. VPC networking for the VCF Automation organization is configured with a Public IP range, a transit gateway IP range, and a VPC private IP range to delineate external, transit, and internal address space for AI tenants.

Organizations should refer to the VCF and vDefend Coverages for the underlying boundary monitoring and enforcement infrastructure.

### **VMware vDefend (Meets)**

VMware vDefend provides monitoring and control of communications at both external and internal network boundaries within the virtual infrastructure through stateful firewall inspection, microsegmentation, and behavioral traffic analysis.

At the external boundary, vDefend enforces zone boundaries and provides stateful North-South traffic inspection at the network perimeter. Gateway Firewall policies apply to traffic entering and leaving edge gateways, and Gateway Firewall IDS/IPS events can be monitored on nodes, providing inspection-layer visibility at the external boundary.

At internal boundaries, vDefend enforces East-West traffic policies between virtual machines, containers, and workloads regardless of their physical location. Distributed Firewall rules operate at the vNIC level, creating per-workload enforcement points that function as internal boundary controls. Distributed Firewall manages East-West traffic control for both inter-VPC and intra-VPC communication within Organizations, and Distributed Firewall rules protect traffic between objects within a VPC, enforcing boundaries within project-scoped network environments. This microsegmentation approach allows organizations to define fine-grained internal boundaries beyond traditional VLAN-based segmentation. The policy-level Applied-To field defines a clear boundary around a set of VMs, such as an entire application, allowing targeted policies for specific zones, tenants, or applications without interfering with policies defined for other applications, tenants, or zones. Distributed Firewall policies can explicitly block unsecure protocols such as Telnet, FTP, and TFTP across the data center, supporting protocol-level boundary enforcement. Distributed Firewall also supports targeted deny rules scoped to infrastructure groups via the Applied-To field to protect critical services, facilitating phased Zero Trust adoption. Distributed Firewall governance implements least-privilege security by preventing overly permissive rules such as any-to-any-allow configurations. Distributed Firewall policies manage security for Bare Metal servers through traffic-control rules, extending boundary enforcement beyond virtual workloads. For federated environments, Distributed Firewall policies applied globally allow sources or destinations to be any object without span restrictions, supporting multi-site boundary enforcement.

Security Services Platform adds environment-level boundary enforcement through Protection Rules, which define how traffic flows between environment pairs such as production, development, and test. Protection Rule actions can be set to Drop or Reject to block all traffic between specified environment pairs, or Allow for more selective enforcement. Security Services Platform Environment Protection evaluates whether workloads are protected by environment-specific segmentation policies and measures coverage including the percentage of environment pairs in blocked mode. Application Protection in Strict Mode enforces application-level firewall rules on all application workloads except infrastructure, with scoring based on workload coverage, traffic coverage, and lock-down policies. Security Intelligence, a capability within Security Services Platform, defines private IP ranges as the boundary for internal workloads, treating traffic that crosses this boundary as external. The Secure External Communications project identifies workloads with inbound or outbound connections to external IP addresses to mitigate vulnerability to attacks. Security Intelligence recommends identifying all inter-environment flows and securing them with specific policies before enforcing strict block rules, supporting a methodical approach to boundary hardening.

Network Detection and Response (NDR) monitors traffic across boundaries and collects detection events representing security-relevant activity that has occurred in the network. NDR can be integrated with SIEMs to send security event logs about network infrastructure for centralized boundary monitoring. In sensitive or classified environments, NDR can be configured for air-gapped networks, executing all detection, correlation, and response activities entirely within closed networks using local storage and processing without requiring external connectivity. NDR remediations for events detected on the Gateway Firewall are applied on the Distributed Firewall, providing a unified enforcement response across both the internal and external boundary layers.

vDefend does not define which boundaries an organization should establish or set the policies governing boundary placement decisions. Those are organizational architecture and governance responsibilities. The underlying networking infrastructure of VCF, comprising NSX segments, VLANs, and edge gateways, provides the segmentation and routing fabric on which vDefend's boundary enforcement operates.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer provides boundary protection mechanisms at the application delivery layer for traffic it proxies. Network Security Policies in Avi evaluate incoming client traffic based on network-level parameters and determine whether a connection should be accepted, discarded, or rate-limited. These policies apply IP-based traffic filtering at the virtual service level; a deny rule stops further policy evaluation, preventing downstream HTTP Security, Request, or Response policies from executing for that traffic. Network security policies are configurable per virtual service and per Virtual IP address, giving administrators auditable and consistent access controls across the deployment.

Avi's Web Application Firewall (WAF) inspects application-layer traffic at the proxy boundary using positive and negative security models. Positive Security rules define what constitutes allowed application behavior and enforce restrictions on traffic that does not conform to that model. WAF Application Rules provide protection against known attack patterns, including rules targeting specific CVEs; rules are automatically updated through the Controller Cloud Services connection, and existing rules continue to protect applications when that connection is unavailable. WAF rule actions support pass and deny options for controlling traffic based on policy matches. Administrators can also enable the Application Rules service through the Avi Cloud Console, which delivers automatically updated CVE-targeted rules directly to Avi Controllers.

For DDoS mitigation, Avi's DDoS Policy detects and mitigates Layer 4 through Layer 7 network attacks through the Network Security policy framework. This includes protection against DNS Reflection Ingress attacks, which involve sending DNS queries with spoofed IP addresses to flood a victim with unsolicited DNS server responses.

In Kubernetes deployments, the Avi Kubernetes Operator (AKO) extends boundary controls to Kubernetes ingress traffic. The L4 Rule custom resource definition (CRD) in AKO allows administrators to express custom Security Policies applied to virtual service traffic, including configuration for DDoS attack detection and mitigation. The `'insecureEdgeTerminationPolicy'` setting in AKO controls handling of insecure traffic, with a redirect option that enforces secure traffic routing through HTTP Request policies applied per FQDN.

Avi Service Engines implement traffic isolation between the data plane and the management and control plane using a dedicated management vNIC bound to the default Linux network namespace, with health monitor traffic further isolated through Linux network namespace features in the kernel. Service Engine IP addresses egressing the Data Plane vNIC are routed through the external network and subjected to routing isolation and ACL/firewalling controls present in that external network. A dedicated management network is required for communication between Avi Controllers and Service Engines. In isolated network deployments, each Avi Controller supports a second management IP address or DNS-resolvable FQDN for Service Engines connecting from an isolated segment, configured via the `'isolated-network-management-address'` setting.

In VCF deployments using the Avi-for-VCF integration, the Management Network hosts the Avi Controller and provides connectivity to vCenter, ESX hosts, and Supervisor control plane nodes, maintaining a defined boundary between management and workload traffic.

Incoming HTTP(S) traffic can be replicated to logging and sideband servers at the protocol level via the Virtual Service Sideband Profile, enabling deep traffic inspection by external appliances such as web application firewalls and supporting compliance and auditing requirements.

Avi also supports a Firewall Sandwich Topology, where virtual services and firewall pool members are chained so that application traffic traverses configured firewall members on its path from the Internet to internal servers. In this topology, a sample of traffic is inspected by the firewall while remaining traffic considered trusted is directly routed to the internal network.

Boundary protection for segments and layers outside Avi's application delivery scope, including network-level perimeter controls, east-west enforcement between application tiers, and internal boundary protection across the broader VCF environment, requires the VCF networking infrastructure and VMware vDefend, which is a separate product.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.



## Control CR 5.3

> General purpose person-to-person communication: Restrict person-to-person communication (e.g., email) from IACS components.

**SCF Controls:** NET-04.1

**Aggregate Coverage:** Meets

### VCF (Contributes)

VCF supports deny-by-default and allow-by-exception network traffic patterns at multiple layers, including the ESX host firewall, vSphere Distributed Switch traffic filtering, and NSX routing layer. Full workload firewall deny-by-default enforcement at distributed and gateway firewall levels is provided by VMware vDefend, which is a separate product.

ESX hosts include a built-in firewall that controls incoming and outgoing connections for management services, with service groups including SSH and SNMP. Each firewall service allows access from all IP addresses by default; administrators can restrict access by specifying allowed IP addresses in CIDR notation (for example, 192.168.0.0/24) or as individual comma-separated addresses. The `esxcli network firewall ruleset set --allowed-all` command disables open access for a ruleset and enforces an explicit IP allowlist per service, implementing a deny-by-default posture at the management-service level on every ESX host.

vSphere Distributed Switches support Traffic Filtering and Marking policies at the distributed port and uplink port level. Network traffic rules can be configured with Allow or Drop actions to control traffic flowing through individual ports, and port-level override options allow policy exceptions to be applied at the individual port level without modifying the parent distributed port group policy. These rules operate at the virtual switch tier and provide a deny-by-default mechanism independent of firewall functionality.

At the routing layer, NSX Tier-0 Gateways support IP prefix lists that control route redistribution with permit and deny actions. The default action for a route in an IP prefix list is Deny, which implements a deny-by-default posture for route advertisement. Administrators must explicitly add a Permit entry for each desired route, with all others denied by default. This deny-by-default routing posture restricts which traffic flows can be advertised and reachable through the NSX infrastructure.

VCF Automation allows administrators to associate security groups with network profiles. When a network profile includes security groups, all machines provisioned with that profile inherit the associated network policies, providing a way to apply consistent deny/allow policies at provisioning time. The security group constructs integrate with vDefend firewall enforcement when vDefend is deployed, surfacing rule-based access controls through the automation layer.

VCF Operations for Networks provides a Security Planning capability through the Plan and Assess interface. This feature analyzes flows and displays traffic flow patterns and micro-segmentation data, giving administrators visibility into which traffic is currently permitted and helping identify gaps in deny-by-default coverage. Planning queries can analyze data center security by planning security policies for specified entities before they are applied.

At the workload consumption layer, VMware Kubernetes Service (VKS) clusters use Kubernetes NetworkPolicy resources as the primary mechanism for deny-by-default traffic control between pods. A NetworkPolicy selects pods by label selector and specifies which ingress and egress traffic is permitted; once at least one NetworkPolicy applies to a pod, all traffic not explicitly permitted is dropped for that pod. To establish a deny-by-default posture in a namespace, administrators create a NetworkPolicy that selects all pods but specifies empty ingress and egress rules, then layer explicit allow rules on top using pod label selectors, namespace selectors, destination ports, or IP block CIDR ranges with optional exclusions. VKS uses Antrea as its default Container Network Interface (CNI) plugin; Antrea implements NetworkPolicy enforcement using Open vSwitch, enabling per-flow filtering at the virtual switch layer. The AntreaPolicy feature gate enables Antrea-native policies, which support tiered policy ordering and cluster-scoped rules beyond what standard NetworkPolicy resources support.

Beginning with VKS 3.6, VKS clusters support configurable inbound firewall rules for cluster nodes through the `firewall` subvariable of the `ClusterClass` node variable. The `firewall.inboundRules` variable can be set on control plane nodes and as variable overrides on each worker node `machineDeployment`; each rule accepts an optional source IP address or CIDR block

and a target protocol specified as an IANA protocol number or alias string such as ``tcp`` or ``udp``. This provides a declarative API for controlling node-level inbound access, complementing the pod-level NetworkPolicy layer.

Each VKS cluster is provisioned with a built-in distributed firewall rule for kube-dns, applied to the CoreDNS pod logical port at cluster creation, restricting CoreDNS access to its designated port. Antrea NetworkPolicyStats is enabled by default on VKS clusters, providing statistics on which network policies are active and what connection volumes they have matched. VKS Cluster Management Security Policies include a Policy enforcement setting whose default action is Deny, which fully enforces the policy and denies admission requests that contain violations; a Dry run mode is available for validation before enforcement is activated. The ``DenyServiceExternalIPs`` admission controller restricts use of ``LoadBalancer`` and ``ExternalIPs`` service types, limiting which services can advertise external endpoints.

Kubernetes itself enforces deny-by-default access control at the API server level: all parts of an API request must be explicitly permitted by at least one authorization mechanism in order to proceed. One scope boundary applies at the pod networking layer: Kubernetes NetworkPolicy resources are specified to restrict TCP, UDP, and SCTP traffic; for other protocols such as ARP and ICMP, enforcement behavior depends on the CNI plugin in use, and administrators should verify handling for those protocols in their specific deployment. A namespace with no NetworkPolicy applied allows all pod traffic by default; achieving deny-by-default behavior requires deliberate creation and deployment of default-deny NetworkPolicy objects in every namespace, which organizations must establish as an operational practice.

VMware vDefend extends these capabilities with full stateful firewall enforcement at both the gateway and workload tiers. vDefend applies security policies at tier-1 gateway service interfaces, controlling traffic as it enters and exits network segments. vDefend firewall rules provide granular control through configurable source groups, destination groups, services, and direction settings (inbound, outbound, or both). Each rule supports three actions: Allow permits matching traffic, Deny actively rejects it with a notification to the client, and Drop silently discards the packet. This rule structure enables administrators to build policies that block all traffic by default and then define specific exceptions for permitted flows.

Rule ordering within vDefend firewall policies is significant for implementing deny-by-default patterns. Rules that permit specific exceptions should be placed before rules that drop traffic, so that the allowed exceptions are evaluated first and all remaining unmatched traffic falls through to the default deny rule. vDefend preserves the order in which policy rules are added, and precedence is determined at both the policy level and the individual rule level.

vDefend Layer 7 access profiles support URL category filtering with configurable Allow, Reject, or Reject with Response actions for matched traffic. Administrators can define FQDN-based firewall rules to allow or reject traffic going to specific domains, using both pre-defined and custom FQDN lists. URL categories support wildcard pattern matching for domain filtering, including single-level and multi-level wildcards and anchored patterns to match specific URL structures.

vDefend includes a Firewall Exclusion List that allows administrators to designate up to 100 groups that should bypass firewall rule evaluation. This should be used sparingly, as excluded objects are not subject to the deny-by-default policy.

vDefend extends deny-by-default enforcement to east-west traffic between virtual machines. vDefend DFW is a software-defined Layer 2-7 firewall that provides stateful firewalling with distributed firewall policies supporting Allow, Drop, and Reject actions, direction controls, and context-aware rule parameters including sources, destinations, services, and context profiles. vDefend functionality can be configured on virtual distributed switches in data centers and workloads where segmentation, visibility, or advanced security capabilities are desired.

Organizational governance of firewall rule policies, change management for rule updates, and periodic review processes to verify that deny-by-default posture remains intact across the environment must be established outside the platform.

### VMware vDefend (Meets)

VMware vDefend (VMware vDefend) implements a deny-by-default, allow-by-exception traffic policy for both East-West and North-South traffic in a VCF environment.

The vDefend distributed firewall (DFW) enforces stateful Layer 2-7 security policies at the virtual network interface level of each workload. Each firewall rule is configured with one of three actions: Allow, Drop, or Reject. DFW automatically generates a Default Drop rule that denies all traffic from any source to any destination for any service. This catch-all rule sits at the

bottom of the rule table and enforces policy for packets not matching any other rule. VMware recommends changing the DFW default rule to a block action and enforcing access control through a positive control model where only traffic explicitly defined in firewall rules is permitted. Specific rules should be placed above more general rules in the rule table so that more targeted permits take precedence. DFW supports Context Profiles for Layer 7-aware rule matching, enabling allow-by-exception rules that specify application-layer context in addition to source, destination, and port.

The Application Connectivity Strategy in DFW provides structured, API-driven approaches to configuring the default traffic posture. The Allow Intra, Deny Other Traffic strategy uses the ALLOW\_INTRA and ALLOWLIST APIs to generate an intra-group Allow rule paired with a default ANY/ANY/DROP rule, isolating the group from all other traffic. The Allow List connectivity preference configures a Drop action as the default, denying all traffic not explicitly permitted. For traffic not covered by named strategies, DFW supports custom policies that provide granular control of lateral East-West traffic.

FQDN filtering rules in DFW support Allow, Drop, or Reject actions, extending the deny-by-exception model to domain-name-based traffic control. The Identity Firewall enforces the allow-by-exception discipline for user-identity-based access: each Identity Firewall rule that permits traffic from a user group to a destination must be paired with a corresponding DFW or Gateway Firewall rule applied to the destination group. Identity Firewall rules also support a Reject action, allowing administrators to deny traffic matching specified user groups, services, and destinations.

The vDefend gateway firewall provides deny-by-default enforcement for North-South traffic at the NSX edge. Gateway Firewall rules support Allow, Drop, and Reject actions. In federated deployments, DFW and Gateway Firewall rules can be created from the Global Manager with global, regional, or local spans and are synchronized to Local Managers. The default gateway firewall rule is editable from the Global Manager and can be configured as allow-all or deny-all, giving administrators control over the baseline North-South posture. The L7 Access Profile extends allow-by-exception to application-layer traffic, with rule actions of Allow, Reject, and Reject with Response.

The Security Services Platform (SSP) and Security Intelligence support the transition to and maintenance of a deny-by-default posture. After establishing exception policies for an environment pair, administrators can change the protection rule action to Drop or Reject to block new, unknown traffic. Policy recommendations can be scoped with a Source Exclusions List, allowing fine-tuned control over which traffic sources are included in generated allowlists. The Security Overview Dashboard provides views of DFW policies, including policy actions (allow, drop, and reject) and total rule counts, giving administrators visibility into the active enforcement posture. Security Policies in vDefend DFW can be reviewed and modified through the Policies and Rules field.

The Firewall Rule Analysis Report identifies potential security misconfigurations in the distributed firewall configuration, helping administrators find shadowed or overly permissive rules that could undermine the deny-by-default posture.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides deny-by-default traffic controls at the application delivery layer through Network Security Policies, HTTP Policy Sets, and Web Application Firewall (WAF) policies in Enforcement mode.

Avi's Network Security Policy is evaluated for every client connection to a virtual service. The policy determines whether a connection is accepted, discarded, or rate-limited based on network-level parameters. Multiple ordered rules can be configured within each Network Security Policy, with explicit Allow and Deny actions available. When a rule with a Deny action matches a connection, further policy evaluation stops, so downstream HTTP Security, Request, or Response policies do not execute for that traffic. Deny rules support a log parameter that, when set to True, records all denied traffic. ControlScripts can dynamically create Network Security Policy rules that deny traffic from specific client IP addresses identified during DDoS events, using IS\_IN match criteria against a configured list of source addresses. Network Security Policy can also be configured to deny connection attempts from addresses identified as malicious through IP Reputation data, dropping TCP SYN packets before connections are established. Both Network Security and HTTP Security policies support a Blocklist mechanism, where a policy rule references a configured IP group named Blocklist to discard clients matching that group.

The HTTP Policy Set provides an additional allow-or-deny mechanism at the application layer. Match conditions classify traffic and apply explicit allow or deny actions; denied traffic receives a TCP RST unless the system is under a volumetric denial-of-service attack, in which case the connection is silently discarded.

When configured in Enforcement mode, Avi's WAF applies a positive security model to application-layer traffic. WAF rule actions support pass (allow) and deny (block) options; requests that satisfy configured rules are permitted, and those that do not are blocked by default. Administrators can author custom WAF rules using SecRule syntax with chain operators, enabling multi-condition enforcement such as allowlisting specific source IP ranges for specific URI paths. WAF Exceptions allow specific traffic to be excluded from rule enforcement based on Client IP, Path, or Match Element filter combinations. WAF Allowlist rules support a BYPASS action for trusted IP address groups, including IP Group-based allowlisting, providing an allow-by-exception mechanism alongside the default-deny posture.

Avi's traffic controls apply only to application traffic passing through its virtual services and Service Engines. Deny-by-default policy enforcement for general east-west and north-south network traffic across the VCF platform is a function of VMware vDefend, which is a separate product. An organization implementing deny-by-default network traffic controls in a VCF environment will typically use both Avi and vDefend together, with Avi addressing application-layer traffic at the load balancer boundary.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 6.1

> Audit reporting: Capability to report audit records to a centralized repository. Automated reporting: Provide automated transmission of audit records.

**SCF Controls:** MON-08, MON-08.2

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides several technical mechanisms that help protect event logs from unauthorized access, modification, and deletion, though full satisfaction of this control requires organizational policies and additional tooling beyond the platform itself.

Remote syslog forwarding is the primary mechanism for protecting log integrity across the VCF stack. VMware ESX hosts can be configured to send syslog messages to a remote log server using the `esxcli system syslog config set` command, specifying the target server IP and port. The `Syslog.global.auditRecord.remote` parameter specifically controls whether ESX audit records are forwarded to the remote hosts defined by `Syslog.global.logHost`, providing a dedicated configuration knob for audit record remote transmission separate from general message forwarding. The NSX networking components send audit logs directly to the configured remote syslog server, separating them from the local syslog file and storing them in `/var/log/audit/audit.log` on the local host. NSX API audit logging is always enabled and cannot be disabled, providing a persistent record of all API-driven administrative actions. VCF Log Management can be configured to pull tasks, events, and alerts from VMware vCenter instances and receive syslog feeds from ESX hosts, acting as a centralized log aggregation point within the VCF environment. VCF Operations itself integrates its audit log with syslog and supports forwarding user activity audit logs to an external syslog server. VCF Operations HCX similarly supports configuring a remote syslog server so that logs generated on both the HCX manager and appliance virtual machines are forwarded off-host.

Log forwarding supports encrypted transport to protect log data in transit. NSX Manager syslog supports the TLS protocol and requires port 6514 to be open on both the NSX Manager and the remote log server for TLS-encrypted log transmission. The vCenter syslog forwarding API supports TCP, UDP, TLS, and RELP protocols, and Foundation Load Balancer syslog communication requires a certificate for secure log forwarding. vCenter event streaming to a remote syslog server is separately enabled through the `vpzd.event.syslog.enabled` option in the vCenter Management Interface, providing a dedicated control for event export independent of the syslog API.

By forwarding logs to remote collection infrastructure, VCF reduces the risk that a compromised host or component could modify or delete its own log records. The local copies remain on each component, but the remote copies provide an independent record that local administrators cannot alter.

Access controls on log data are handled through the role-based access model within each VCF component. Access to VCF Log Management is governed by role-based access controls, restricting who can view and manage log data. Within VCF Operations for Networks, only users with the Admin role can view audit logs and configure whether personally identifiable information is included in those logs. For Windows-based domain controllers in the environment, read-only security log access can be enabled, and membership in the Event Log Readers group governs which accounts have permission to read event logs. The NSX documentation notes that core files and audit logs may contain sensitive information such as passwords or encryption keys.

VCF Operations provides an Audit Events page accessible through the Security menu, where administrators can review audit records that capture interactions within the platform including searches, logins, logouts, capability checks, and configuration modifications. Audit logs across VCF Operations capture actions from the API, user interface, and CLI, including regular create, read, update, and delete operations as well as login and logout events. VCF Operations audit data can be filtered by user ID, user name, authentication source, session, client IP, category, and message, supporting targeted review of specific activities. VCF Operations for Networks audit logs similarly capture API, UI, and CLI actions. These audit records include structured metadata such as timestamps, thread identifiers, source IP addresses, log levels, and component class names.

NSX audit logs use a structured syslog format with an audit flag (`audit="true"`) in the structured-data field to distinguish security-relevant events from operational messages. Dedicated log files separate audit records: `nsx-audit.log` contains all

messages with the audit flag, while `nsx-audit-write.log` contains messages flagged as both audit and update operations. NSX audit log messages include metadata fields such as component identifier, log level, request ID, split ID, and subcomponent classification, and each API-associated audit log entry includes a request ID parameter (`req-id`) to identify a specific API call. NSX logs all auditable events to support compliance requirements.

The Consumption layer of VCF, comprising VMware Kubernetes Service (VKS), vSphere Supervisor, and vSphere namespaces, adds its own audit logging mechanisms that extend event log protection to the Kubernetes control plane. The Kubernetes API server generates a security-relevant, chronological audit log documenting all actions in the cluster, including activities from users, applications using the Kubernetes API, and the control plane itself. Each audit event in the `audit.k8s.io` API group carries structured fields including `apiVersion`, `kind`, `level`, `auditID`, and `stage`, along with an `AuthenticationMetadata` block that records impersonation constraints when constrained impersonation was used to authorize a request. Audit event delivery is configured through the `kube-apiserver --audit-log-path` and `--audit-log-mode` parameters; the delivery mode defaults to blocking, which writes each event synchronously before the API server returns a response, reducing the risk of event loss under load. An audit policy file, mounted at `/etc/kubernetes/audit-policy.yaml`, governs the scope of the audit stream by defining which event types are recorded at which detail level; if the audit logging flag is omitted from the policy, no events are logged. Audit event truncation is disabled by default and must be explicitly enabled, so audit records are written in their entirety unless an administrator opts into size-based truncation. Backends include local log files written to `/var/log/kubernetes/audit/` and webhook endpoints for forwarding to external systems. Pod Security Standards (PSA) audit mode, when enabled on a VKS cluster, records security policy violations as audit annotations in the audit log stream, extending audit coverage to workload-level security events. Kubernetes seccomp profiles using the `SCMP_ACT_LOG` action allow individual syscalls of a process to be logged to syslog or auditd, providing fine-grained syscall-level audit recording for sensitive workloads.

Fluent Bit agents deployed by VCF in the `vmware-system-logging` namespace on VKS clusters collect and forward logs to external destinations, with access to that namespace restricted to cluster administrators. Fluent Bit processes Kubernetes API server audit logs separately from container logs and Linux daemon logs when forwarding to VCF Log Management, maintaining the distinction of audit records in the forwarded stream. On vSphere Supervisor control plane nodes, Fluent Bit collects `kube-apiserver` audit logs from `/var/log/vmware/audit/kube-apiserver.log` using the tag `audit.apiserver.*` and stores them in a local database at `/var/log/vmware/fluentbit/flb_audit_apiserver` before forwarding; on VKS worker cluster nodes, audit logs are collected in JSON format from `/var/log/kubernetes/kube-apiserver.log` with a 50 MB memory buffer limit. Fluent Bit also collects Linux authentication logs and container runtime interface logs from VKS cluster nodes, extending the set of log types forwarded to external destinations. The Supervisor can be configured to preserve the source IP addresses of authentication and Kubernetes API server requests for later inspection in audit logs, supporting attribution of actions to specific network origins. Supported forwarding targets include VCF Log Management, Syslog servers, Elasticsearch, Kafka, and Splunk. For workloads running in vSphere Namespaces as vSphere Pods, the `LogAgentConfiguration` custom resource can be applied to a namespace to stream syslog messages to a remote Syslog server; each forwarded message includes metadata fields for namespace name, pod UID, pod name, container name, and ESX host name. By routing audit records to an external log management system, the platform creates a copy of the audit trail outside the cluster, so that a compromise or administrative action affecting the cluster does not also destroy audit evidence.

To fully satisfy this control, organizations deploying VCF should implement log forwarding to a dedicated, hardened log management or SIEM platform where retention, integrity verification, and access controls can be managed independently of the VCF infrastructure. VCF does not provide immutable or tamper-evident log storage on its own; protection against log modification or deletion at the storage layer requires integration with a write-once-read-many (WORM) log repository or equivalent storage. For high-assurance environments, Kubernetes guidance recommends deploying cryptographic protections to produce tamper-evident and confidential logs. Organizations should also establish policies governing who may access log data, how long logs are retained, and how log integrity is validated over time.

VMware Private AI Services (PAIS) generates audit-relevant logs across its AI infrastructure components and exposes an observability subsystem with configurable retention and external forwarding that contributes to event log protection.



Deep Learning VMs record cloud-init script logs in `/var/log/dl.log` and GPU driver installation logs for the vGPU guest driver or NVIDIA data center GPU driver in `/var/log/gpu-install.log`. JupyterLab instances on Deep Learning VMs surface logs that can be examined to verify component installation and operation. Model endpoints support configurable log verbosity through the `VLLM_LOGGING_LEVEL` environment variable, letting operators control what the vLLM inference server records.

PAIS observability is activated by updating the `PAISConfiguration` custom resource. Operators set a Prometheus storage class policy through `observability.prometheusRuntime.storageClassName` and a retention period through `observability.prometheusRuntime.metricsRetention.period`, which define where collected metrics are stored and how long they are preserved. LLM traces can be forwarded to an external OpenTelemetry Collector over HTTP/protobuf or gRPC by uncommenting the `spec.observability.llmTraces` section of the `PAISConfiguration` custom resource, externalizing trace data away from the system where it was generated. PAIS Controller metrics are exposed at the Prometheus metrics endpoint on the `pais-controller-manager` service and can be streamed from the Supervisor cluster to VCF Operations for consolidated observability. PAIS integrates with Grafana for metrics visualization using OIDC provider authentication, restricting who can view the collected observability data.

Access to the underlying log files on Deep Learning VM filesystems and the persistent volumes backing Prometheus storage is governed by the Kubernetes RBAC and persistent volume access controls provided by VCF. Organizations should configure the Prometheus storage class and retention parameters to align with their log protection requirements and route LLM traces and observability data to a protected external platform such as Grafana, VCF Operations, or another OpenTelemetry-compatible backend.

### **VMware vDefend (Meets)**

VMware vDefend generates structured security event logs across its Distributed Firewall, Gateway Firewall, IDS/IPS, and Malware Prevention components, with several mechanisms that support protection of those logs from suppression or loss.

The Security Services Platform audit log is enabled by default and cannot be disabled through configuration. All access events to the Security Services Platform, including access to the License Hub, are recorded in these audit logs, preventing log suppression at the platform level.

The Distributed IDS/IPS can be configured to forward events directly to external syslog consumers via the `ids_events_to_syslog` setting; this forwarding is disabled by default and requires explicit configuration. When remote syslog is enabled on ESX hosts, IDS/IPS events are exported directly from those hosts to external receivers. vDefend components write log files to `/var/log`, with syslog messages conforming to RFC 5424 on vDefend appliances and RFC 3164 on ESX hosts. Gateway Firewall isolation rules log blocked traffic attempts to the `unifiedsyslog.log` file on Edge Nodes.

Malware Prevention produces structured event logs in JSON format that include standard attributes (date, log level, module) along with detailed file analysis metadata: file hashes (SHA256, SHA1, and MD5), file type, file size, inspection timestamp, client and server VM identifiers, submission source, detection status, verdict, analysis status, malware classification, malware family, HTTP domain, HTTP method, and user agent. These records capture the full analysis lifecycle, including file intercepted, verdict cache hit, local static analysis submission, cloud dynamic analysis submission, verdict obtained, and policy enforced events. In 9.1, the Malware Prevention Service can be configured to forward logs to a remote logging server using TLS encryption with mutual certificate authentication (requiring server CA certificate, client CA certificate, client certificate, and client key), providing an off-appliance copy of event records with in-transit protection.

Identity Firewall event log sources can be integrated with VCF Log Management for centralized log collection and monitoring, providing an additional path to off-appliance log preservation.

Protection of forwarded or archived logs from unauthorized access and modification at rest is outside vDefend's scope and depends on the receiving log management or SIEM platform's controls.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) contributes to this control through role-based log access controls, syslog forwarding to external platforms, and audit event generation that captures user identity and action details.



PNR generates audit events across a broad range of recovery and replication operations, including configuration changes, recovery plan executions, and protection group modifications. Each audit event record includes the event name, user identity, and source IP address from which the action was initiated. Login events include timestamps, session keys, and user identity information in a structured format. Events covering protection group and consistency group changes are logged at appropriate severity levels (Info or Error), and corresponding alarms can be configured to trigger notification when specific events occur.

PNR log files are designed to exclude sensitive data such as private keys and passwords. The vSphere Replication component maintains security-relevant audit logs at defined paths on the appliance, including authorization error messages, login events, and certificate verification errors, each recorded with timestamps and component information to support forensic review.

To move logs off the local appliance, PNR supports syslog forwarding from the PNR Appliance to a remote syslog server. The Clean Room Orchestrator supports forwarding events to VCF Log Management, Microsoft Sentinel, or Splunk Cloud, enabling integration with enterprise log management and security monitoring platforms. Event forwarding to VCF Log Management requires version 9.0 or later. Centralizing logs in an external platform removes them from local storage where an attacker with appliance access could otherwise modify or delete them.

PNR does not provide write-once or tamper-evident local log storage. Organizations should forward PNR logs to a centralized SIEM or write-once-read-many log repository to protect against modification or deletion at the storage layer.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) produces structured audit logs that record platform-level and gateway-level operations, and provides several mechanisms that contribute to protecting those logs from unauthorized access and loss.

At the platform level, DSM maintains gateway audit log files on the Provider VM at `/var/log/tdm/provider/audit/`, with individual files named using a timestamp format (`gateway-audit-timestamp.log`). Local retention covers the last five days of log files; once files are successfully uploaded to a configured S3 repository, DSM removes them from the local filesystem after five days. Each log entry captures structured fields including Component, Event Type, Event Details, Event Time (UTC), Status, and Source. The Source field records the email identifier of the user who initiated the operation, or "System" for events initiated by DSM without user intervention. The System Audit Events view, accessible through the Provider interface, is the designated location for reviewing non-database-related platform operations. Access to the Provider interface is scoped to the administrative role, restricting which users can read or interact with the system-level audit record.

DSM supports offloading audit logs to external storage as the primary mechanism for durable protection. When a default S3 provider log repository is configured, DSM periodically archives gateway audit log files to an S3 subpath named "gateway audit." DSM does not delete audit logs stored in the S3 repository, which means the archived log record persists beyond the five-day local retention window. Externalizing audit logs to S3 separates the log record from the Provider VM that generated it, reducing the risk that a compromise or failure of the Provider VM affects the integrity or availability of the audit trail. Organizations apply bucket-level access controls and retention policies on the S3 repository to restrict who can read, modify, or delete the archived logs.

DSM supports log forwarding, allowing DSM logs to be sent to a centralized log management or security information and event management (SIEM) platform via the System Logs capability. Forwarding logs to an external system provides a second copy of the audit record under access controls and integrity protections that are independent of the DSM platform itself.

For SQL Server databases provisioned through DSM, the SQL Server data service uses server-level DDL triggers to restrict the file paths available for audit logs, limiting where audit log files can be written. DSM supports enabling audit logging for SQL Server with configurable levels of auditing for system logins and a configurable log retention period. DSM can automatically create a simple audit for login events based on the Login Auditing option selected during cluster creation. For custom audit requirements, the standard T-SQL `CREATE AUDIT` and `CREATE AUDIT SPECIFICATION` DDL statements are available.

DSM can generate and download log bundles that aggregate relevant logs from database VMs, supporting diagnosis and out-of-band preservation of the operational log record.

Audit events can be exported from the System Audit Events view in CSV or XLSX format using the ACTIONS menu. Exports include all events visible to the authenticated user regardless of pagination, supporting external archival and out-of-band preservation of the audit record.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer supports the protection of event logs and audit tools through real-time log forwarding to external systems, structured audit record content, and access controls on the Controller's audit interfaces. The 9.1 release adds License Hub audit logging capabilities that are always active and optionally forwarded to external syslog.

The primary off-system protection mechanism is the Syslog-Audit-Persistence feature, configured under Alert Actions in the Avi Controller. When enabled, audit events stream to one or more external syslog servers in real time; once forwarded, the records reside outside the Controller where local administrative access cannot modify or remove them. The Controller supports syslog delivery over TCP in addition to UDP, and syslog server targets are specified in Controller Settings. Any compatible syslog receiver, including VCF Log Management configured as a log forwarder, can serve as a destination.

An important architectural boundary applies: Avi Load Balancer debug logs and Event Logs are not forwarded to remote syslog servers through the OS-level syslog configuration. Debug logs are accessible only through Tech Support generation. Event Logs require a separate API pull or a scripted push from the Controller to a remote logging system, which means organizations should configure explicit export or retrieval processes for off-system preservation.

The Avi Controller's Event Logs provide detailed records of system activities, configuration changes, and operational events, including an audit trail that identifies who made each change and when. Object-specific event logs are accessible via the Events tab on detail pages for Virtual Services, Pools, Service Engines, and Controllers, automatically filtered to the relevant object. The API supports querying events over extended periods, including a one-year lookback using the CONFIG\_DELETE endpoint with the appropriate duration parameter.

Avi WAF generates audit log records for matched security events, such as cross-site scripting detection, capturing the matched variable name and content when audit log parts are configured to include event data (ctl:auditLogParts=+E). This extends event log coverage to L7 application protection events beyond configuration and access activity.

In 9.1, the License Hub provides dedicated audit logging that records all access events and cannot be disabled. License Hub can be configured to transmit log messages to a remote syslog server. A built-in Audit local user account with the auditor role provides dedicated access to APIs and services for system auditing, separate from the administrative account.

Both the Controller and Service Engines log delivery failures when transmitting to Events, Syslog, and Splunk targets, including failed trusted channel initiations and function failures. Disruptions to the log forwarding pipeline are captured as auditable events in their own right.

## Control CR 6.1(1)

> Audit reporting: Capability to report audit records to a centralized repository. Automated reporting: Provide automated transmission of audit records.

**SCF Controls:** MON-08

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides several technical mechanisms that help protect event logs from unauthorized access, modification, and deletion, though full satisfaction of this control requires organizational policies and additional tooling beyond the platform itself.

Remote syslog forwarding is the primary mechanism for protecting log integrity across the VCF stack. VMware ESX hosts can be configured to send syslog messages to a remote log server using the `esxcli system syslog config set` command, specifying the target server IP and port. The `Syslog.global.auditRecord.remote` parameter specifically controls whether ESX audit records are forwarded to the remote hosts defined by `Syslog.global.logHost`, providing a dedicated configuration knob for audit record remote transmission separate from general message forwarding. The NSX networking components send audit logs directly to the configured remote syslog server, separating them from the local syslog file and storing them in `/var/log/audit/audit.log` on the local host. NSX API audit logging is always enabled and cannot be disabled, providing a persistent record of all API-driven administrative actions. VCF Log Management can be configured to pull tasks, events, and alerts from VMware vCenter instances and receive syslog feeds from ESX hosts, acting as a centralized log aggregation point within the VCF environment. VCF Operations itself integrates its audit log with syslog and supports forwarding user activity audit logs to an external syslog server. VCF Operations HCX similarly supports configuring a remote syslog server so that logs generated on both the HCX manager and appliance virtual machines are forwarded off-host.

Log forwarding supports encrypted transport to protect log data in transit. NSX Manager syslog supports the TLS protocol and requires port 6514 to be open on both the NSX Manager and the remote log server for TLS-encrypted log transmission. The vCenter syslog forwarding API supports TCP, UDP, TLS, and RELP protocols, and Foundation Load Balancer syslog communication requires a certificate for secure log forwarding. vCenter event streaming to a remote syslog server is separately enabled through the `vpzd.event.syslog.enabled` option in the vCenter Management Interface, providing a dedicated control for event export independent of the syslog API.

By forwarding logs to remote collection infrastructure, VCF reduces the risk that a compromised host or component could modify or delete its own log records. The local copies remain on each component, but the remote copies provide an independent record that local administrators cannot alter.

Access controls on log data are handled through the role-based access model within each VCF component. Access to VCF Log Management is governed by role-based access controls, restricting who can view and manage log data. Within VCF Operations for Networks, only users with the Admin role can view audit logs and configure whether personally identifiable information is included in those logs. For Windows-based domain controllers in the environment, read-only security log access can be enabled, and membership in the Event Log Readers group governs which accounts have permission to read event logs. The NSX documentation notes that core files and audit logs may contain sensitive information such as passwords or encryption keys.

VCF Operations provides an Audit Events page accessible through the Security menu, where administrators can review audit records that capture interactions within the platform including searches, logins, logouts, capability checks, and configuration modifications. Audit logs across VCF Operations capture actions from the API, user interface, and CLI, including regular create, read, update, and delete operations as well as login and logout events. VCF Operations audit data can be filtered by user ID, user name, authentication source, session, client IP, category, and message, supporting targeted review of specific activities. VCF Operations for Networks audit logs similarly capture API, UI, and CLI actions. These audit records include structured metadata such as timestamps, thread identifiers, source IP addresses, log levels, and component class names.

NSX audit logs use a structured syslog format with an audit flag (`audit="true"`) in the structured-data field to distinguish security-relevant events from operational messages. Dedicated log files separate audit records: `nsx-audit.log` contains all

messages with the audit flag, while `nsx-audit-write.log` contains messages flagged as both audit and update operations. NSX audit log messages include metadata fields such as component identifier, log level, request ID, split ID, and subcomponent classification, and each API-associated audit log entry includes a request ID parameter (`req-id`) to identify a specific API call. NSX logs all auditable events to support compliance requirements.

The Consumption layer of VCF, comprising VMware Kubernetes Service (VKS), vSphere Supervisor, and vSphere namespaces, adds its own audit logging mechanisms that extend event log protection to the Kubernetes control plane. The Kubernetes API server generates a security-relevant, chronological audit log documenting all actions in the cluster, including activities from users, applications using the Kubernetes API, and the control plane itself. Each audit event in the `audit.k8s.io` API group carries structured fields including `apiVersion`, `kind`, `level`, `auditID`, and `stage`, along with an `AuthenticationMetadata` block that records impersonation constraints when constrained impersonation was used to authorize a request. Audit event delivery is configured through the `kube-apiserver --audit-log-path` and `--audit-log-mode` parameters; the delivery mode defaults to blocking, which writes each event synchronously before the API server returns a response, reducing the risk of event loss under load. An audit policy file, mounted at `/etc/kubernetes/audit-policy.yaml`, governs the scope of the audit stream by defining which event types are recorded at which detail level; if the audit logging flag is omitted from the policy, no events are logged. Audit event truncation is disabled by default and must be explicitly enabled, so audit records are written in their entirety unless an administrator opts into size-based truncation. Backends include local log files written to `/var/log/kubernetes/audit/` and webhook endpoints for forwarding to external systems. Pod Security Standards (PSA) audit mode, when enabled on a VKS cluster, records security policy violations as audit annotations in the audit log stream, extending audit coverage to workload-level security events. Kubernetes seccomp profiles using the `SCMP_ACT_LOG` action allow individual syscalls of a process to be logged to syslog or auditd, providing fine-grained syscall-level audit recording for sensitive workloads.

Fluent Bit agents deployed by VCF in the `vmware-system-logging` namespace on VKS clusters collect and forward logs to external destinations, with access to that namespace restricted to cluster administrators. Fluent Bit processes Kubernetes API server audit logs separately from container logs and Linux daemon logs when forwarding to VCF Log Management, maintaining the distinction of audit records in the forwarded stream. On vSphere Supervisor control plane nodes, Fluent Bit collects `kube-apiserver` audit logs from `/var/log/vmware/audit/kube-apiserver.log` using the tag `audit.apiserver.*` and stores them in a local database at `/var/log/vmware/fluentbit/flb_audit_apiserver` before forwarding; on VKS worker cluster nodes, audit logs are collected in JSON format from `/var/log/kubernetes/kube-apiserver.log` with a 50 MB memory buffer limit. Fluent Bit also collects Linux authentication logs and container runtime interface logs from VKS cluster nodes, extending the set of log types forwarded to external destinations. The Supervisor can be configured to preserve the source IP addresses of authentication and Kubernetes API server requests for later inspection in audit logs, supporting attribution of actions to specific network origins. Supported forwarding targets include VCF Log Management, Syslog servers, Elasticsearch, Kafka, and Splunk. For workloads running in vSphere Namespaces as vSphere Pods, the `LogAgentConfiguration` custom resource can be applied to a namespace to stream syslog messages to a remote Syslog server; each forwarded message includes metadata fields for namespace name, pod UID, pod name, container name, and ESX host name. By routing audit records to an external log management system, the platform creates a copy of the audit trail outside the cluster, so that a compromise or administrative action affecting the cluster does not also destroy audit evidence.

To fully satisfy this control, organizations deploying VCF should implement log forwarding to a dedicated, hardened log management or SIEM platform where retention, integrity verification, and access controls can be managed independently of the VCF infrastructure. VCF does not provide immutable or tamper-evident log storage on its own; protection against log modification or deletion at the storage layer requires integration with a write-once-read-many (WORM) log repository or equivalent storage. For high-assurance environments, Kubernetes guidance recommends deploying cryptographic protections to produce tamper-evident and confidential logs. Organizations should also establish policies governing who may access log data, how long logs are retained, and how log integrity is validated over time.

VMware Private AI Services (PAIS) generates audit-relevant logs across its AI infrastructure components and exposes an observability subsystem with configurable retention and external forwarding that contributes to event log protection.

Deep Learning VMs record cloud-init script logs in `/var/log/dl.log` and GPU driver installation logs for the vGPU guest driver or NVIDIA data center GPU driver in `/var/log/gpu-install.log`. JupyterLab instances on Deep Learning VMs surface logs that can be examined to verify component installation and operation. Model endpoints support configurable log verbosity through the `VLLM_LOGGING_LEVEL` environment variable, letting operators control what the vLLM inference server records.

PAIS observability is activated by updating the `PAISConfiguration` custom resource. Operators set a Prometheus storage class policy through `observability.prometheusRuntime.storageClassName` and a retention period through `observability.prometheusRuntime.metricsRetention.period`, which define where collected metrics are stored and how long they are preserved. LLM traces can be forwarded to an external OpenTelemetry Collector over HTTP/protobuf or gRPC by uncommenting the `spec.observability.llmTraces` section of the `PAISConfiguration` custom resource, externalizing trace data away from the system where it was generated. PAIS Controller metrics are exposed at the Prometheus metrics endpoint on the `pais-controller-manager` service and can be streamed from the Supervisor cluster to VCF Operations for consolidated observability. PAIS integrates with Grafana for metrics visualization using OIDC provider authentication, restricting who can view the collected observability data.

Access to the underlying log files on Deep Learning VM filesystems and the persistent volumes backing Prometheus storage is governed by the Kubernetes RBAC and persistent volume access controls provided by VCF. Organizations should configure the Prometheus storage class and retention parameters to align with their log protection requirements and route LLM traces and observability data to a protected external platform such as Grafana, VCF Operations, or another OpenTelemetry-compatible backend.

### **VMware vDefend (Contributes)**

VMware vDefend generates structured security event logs across its Distributed Firewall, Gateway Firewall, IDS/IPS, and Malware Prevention components, with several mechanisms that support protection of those logs from suppression or loss.

The Security Services Platform audit log is enabled by default and cannot be disabled through configuration. All access events to the Security Services Platform, including access to the License Hub, are recorded in these audit logs, preventing log suppression at the platform level.

The Distributed IDS/IPS can be configured to forward events directly to external syslog consumers via the `ids_events_to_syslog` setting; this forwarding is disabled by default and requires explicit configuration. When remote syslog is enabled on ESX hosts, IDS/IPS events are exported directly from those hosts to external receivers. vDefend components write log files to `/var/log`, with syslog messages conforming to RFC 5424 on vDefend appliances and RFC 3164 on ESX hosts. Gateway Firewall isolation rules log blocked traffic attempts to the `unifiedsyslog.log` file on Edge Nodes.

Malware Prevention produces structured event logs in JSON format that include standard attributes (date, log level, module) along with detailed file analysis metadata: file hashes (SHA256, SHA1, and MD5), file type, file size, inspection timestamp, client and server VM identifiers, submission source, detection status, verdict, analysis status, malware classification, malware family, HTTP domain, HTTP method, and user agent. These records capture the full analysis lifecycle, including file intercepted, verdict cache hit, local static analysis submission, cloud dynamic analysis submission, verdict obtained, and policy enforced events. In 9.1, the Malware Prevention Service can be configured to forward logs to a remote logging server using TLS encryption with mutual certificate authentication (requiring server CA certificate, client CA certificate, client certificate, and client key), providing an off-appliance copy of event records with in-transit protection.

Identity Firewall event log sources can be integrated with VCF Log Management for centralized log collection and monitoring, providing an additional path to off-appliance log preservation.

Protection of forwarded or archived logs from unauthorized access and modification at rest is outside vDefend's scope and depends on the receiving log management or SIEM platform's controls.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) contributes to this control through role-based log access controls, syslog forwarding to external platforms, and audit event generation that captures user identity and action details.

PNR generates audit events across a broad range of recovery and replication operations, including configuration changes, recovery plan executions, and protection group modifications. Each audit event record includes the event name, user identity, and source IP address from which the action was initiated. Login events include timestamps, session keys, and user identity information in a structured format. Events covering protection group and consistency group changes are logged at appropriate severity levels (Info or Error), and corresponding alarms can be configured to trigger notification when specific events occur.

PNR log files are designed to exclude sensitive data such as private keys and passwords. The vSphere Replication component maintains security-relevant audit logs at defined paths on the appliance, including authorization error messages, login events, and certificate verification errors, each recorded with timestamps and component information to support forensic review.

To move logs off the local appliance, PNR supports syslog forwarding from the PNR Appliance to a remote syslog server. The Clean Room Orchestrator supports forwarding events to VCF Log Management, Microsoft Sentinel, or Splunk Cloud, enabling integration with enterprise log management and security monitoring platforms. Event forwarding to VCF Log Management requires version 9.0 or later. Centralizing logs in an external platform removes them from local storage where an attacker with appliance access could otherwise modify or delete them.

PNR does not provide write-once or tamper-evident local log storage. Organizations should forward PNR logs to a centralized SIEM or write-once-read-many log repository to protect against modification or deletion at the storage layer.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) produces structured audit logs that record platform-level and gateway-level operations, and provides several mechanisms that contribute to protecting those logs from unauthorized access and loss.

At the platform level, DSM maintains gateway audit log files on the Provider VM at `/var/log/tdm/provider/audit/`, with individual files named using a timestamp format (`gateway-audit-timestamp.log`). Local retention covers the last five days of log files; once files are successfully uploaded to a configured S3 repository, DSM removes them from the local filesystem after five days. Each log entry captures structured fields including Component, Event Type, Event Details, Event Time (UTC), Status, and Source. The Source field records the email identifier of the user who initiated the operation, or "System" for events initiated by DSM without user intervention. The System Audit Events view, accessible through the Provider interface, is the designated location for reviewing non-database-related platform operations. Access to the Provider interface is scoped to the administrative role, restricting which users can read or interact with the system-level audit record.

DSM supports offloading audit logs to external storage as the primary mechanism for durable protection. When a default S3 provider log repository is configured, DSM periodically archives gateway audit log files to an S3 subpath named "gateway audit." DSM does not delete audit logs stored in the S3 repository, which means the archived log record persists beyond the five-day local retention window. Externalizing audit logs to S3 separates the log record from the Provider VM that generated it, reducing the risk that a compromise or failure of the Provider VM affects the integrity or availability of the audit trail. Organizations apply bucket-level access controls and retention policies on the S3 repository to restrict who can read, modify, or delete the archived logs.

DSM supports log forwarding, allowing DSM logs to be sent to a centralized log management or security information and event management (SIEM) platform via the System Logs capability. Forwarding logs to an external system provides a second copy of the audit record under access controls and integrity protections that are independent of the DSM platform itself.

For SQL Server databases provisioned through DSM, the SQL Server data service uses server-level DDL triggers to restrict the file paths available for audit logs, limiting where audit log files can be written. DSM supports enabling audit logging for SQL Server with configurable levels of auditing for system logins and a configurable log retention period. DSM can automatically create a simple audit for login events based on the Login Auditing option selected during cluster creation. For custom audit requirements, the standard T-SQL `CREATE AUDIT` and `CREATE AUDIT SPECIFICATION` DDL statements are available.

DSM can generate and download log bundles that aggregate relevant logs from database VMs, supporting diagnosis and out-of-band preservation of the operational log record.



Audit events can be exported from the System Audit Events view in CSV or XLSX format using the ACTIONS menu. Exports include all events visible to the authenticated user regardless of pagination, supporting external archival and out-of-band preservation of the audit record.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer supports the protection of event logs and audit tools through real-time log forwarding to external systems, structured audit record content, and access controls on the Controller's audit interfaces. The 9.1 release adds License Hub audit logging capabilities that are always active and optionally forwarded to external syslog.

The primary off-system protection mechanism is the Syslog-Audit-Persistence feature, configured under Alert Actions in the Avi Controller. When enabled, audit events stream to one or more external syslog servers in real time; once forwarded, the records reside outside the Controller where local administrative access cannot modify or remove them. The Controller supports syslog delivery over TCP in addition to UDP, and syslog server targets are specified in Controller Settings. Any compatible syslog receiver, including VCF Log Management configured as a log forwarder, can serve as a destination.

An important architectural boundary applies: Avi Load Balancer debug logs and Event Logs are not forwarded to remote syslog servers through the OS-level syslog configuration. Debug logs are accessible only through Tech Support generation. Event Logs require a separate API pull or a scripted push from the Controller to a remote logging system, which means organizations should configure explicit export or retrieval processes for off-system preservation.

The Avi Controller's Event Logs provide detailed records of system activities, configuration changes, and operational events, including an audit trail that identifies who made each change and when. Object-specific event logs are accessible via the Events tab on detail pages for Virtual Services, Pools, Service Engines, and Controllers, automatically filtered to the relevant object. The API supports querying events over extended periods, including a one-year lookback using the CONFIG\_DELETE endpoint with the appropriate duration parameter.

Avi WAF generates audit log records for matched security events, such as cross-site scripting detection, capturing the matched variable name and content when audit log parts are configured to include event data (ctl:auditLogParts=+E). This extends event log coverage to L7 application protection events beyond configuration and access activity.

In 9.1, the License Hub provides dedicated audit logging that records all access events and cannot be disabled. License Hub can be configured to transmit log messages to a remote syslog server. A built-in Audit local user account with the auditor role provides dedicated access to APIs and services for system auditing, separate from the administrative account.

Both the Controller and Service Engines log delivery failures when transmitting to Events, Syslog, and Splunk targets, including failed trusted channel initiations and function failures. Disruptions to the log forwarding pipeline are captured as auditable events in their own right.



## Control CR 6.2

> Continuous monitoring: Capability to monitor for security-relevant events continuously.

**SCF Controls:** MON-01

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides enterprise-wide monitoring through integrated tools that span infrastructure health, performance metrics, compliance assessment, log analysis, and alerting.

VCF Operations continuously monitors the operational state of all VCF components to detect conditions that may affect availability or compliance. It collects metrics across compute, storage, and networking resources, and correlates them over time to identify trends that may lead to potential failures. VCF Health, a component of VCF Operations, continuously monitors the operational state of VCF components to verify they are functioning as expected, giving platform operators a single view of operational status. Platform operators can configure alerts, perform log analysis, run infrastructure diagnostics, and review VCF health from a unified interface.

VCF Operations provides continuous compliance management by monitoring infrastructure against industry-standard benchmarks and custom policies. Administrators can create custom compliance benchmarks that monitor selected alert definitions and policies, tailoring the monitoring posture to organizational requirements. The platform assesses objects based on selected compliance policies and activates alert definitions associated with the current scorecard, providing ongoing visibility into compliance status. Compliance-related alerts are triggered on objects and displayed through a dedicated compliance alert subtype.

The alerting subsystem in VCF Operations supports the full lifecycle of alert management. Administrators can configure and manage alert definitions to monitor infrastructure events and conditions, including performance metrics, log events, and custom conditions configured through the Management Pack Builder. The Platform Health Alerts page provides a centralized view of all alerts related to overall system health, including alerts from data sources and infrastructure nodes. Health alerts are generated for conditions that affect the health of the environment and require immediate attention. Administrators can create policies to govern how alert definitions are evaluated, including which objects to monitor and what thresholds to apply, and can apply consistent practices to optimize alert behavior across monitored objects.

VCF Operations supports flexible grouping and scheduling to align monitoring with operational realities. Custom groups can be configured with automatic membership criteria to support monitoring strategies and achieve a high level of automation. Maintenance schedules can be assigned to objects undergoing planned downtime to keep monitoring data accurate and avoid false alerts when an object is offline for scheduled maintenance.

VCF Log Management provides centralized log collection and analysis. It ingests logs from VMware vCenter, VMware ESX hosts, NSX, vSAN, VCF Automation, VCF Operations, VCF Identity Broker, VCF Operations for Networks, and VCF Operations HCX. VCF Operations and VCF Log Management must be integrated to implement the centralized log collection architecture. The platform provides dashboards and analytics for troubleshooting and security investigation. Log-based alert definitions allow administrators to trigger notifications when specific log patterns appear across monitored systems. Log forwarding to external SIEM or syslog targets is configurable, with support for TLS encryption to protect log data in transit. The vCenter syslog forwarding API supports TCP, UDP, TLS, and RELP protocols. NSX Manager syslog supports TLS and requires port 6514 for encrypted log transmission.

At the vSphere layer, the alarm infrastructure integrates with events and performance counters to provide real-time alerting. The MetricAlarmExpression data object allows administrators to set alarms on specific performance metrics with configurable monitoring intervals and aggregation functions for processing data points. The vSphere Client provides direct visibility into tasks and events for change tracking and troubleshooting. VCF ships with preconfigured vSphere alarms that are active by default and monitor specific components including the Identity Management Service, Message Bus Configuration Service, ESX Agent Manager, VMware Service Control Agent, vSphere HA host status, and host connection and power state. The vCenter Management Interface provides appliance-level health visibility through an Overall Health badge that reflects one of five states

(Good, Warning, Alert, Critical, or Unknown) across all vCenter components, and notifies administrators of available security patches through Critical or Alert badge indicators when automatic patch checks are enabled.

When VCF Automation is integrated with VCF Operations, the Insights dashboard and Alerts tab provide real-time capacity awareness and alert information for VCF Automation objects, giving IT teams visibility into resource usage across both provisioned workloads and the underlying infrastructure. The VCF Automation Monitor tab surfaces deployment health information sourced from VCF Operations, and deployment cards display Last Request Status information that tracks the most recent operation on each deployment (Create, Update, or Delete) and its outcome (Successful, In Progress, or Failed). The Management Pack Builder allows administrators to set up event definitions for integrated systems, monitoring specific conditions or thresholds and configuring alerts for proactive management. Management Packs extend monitoring to network devices, including sensor monitoring for devices such as switches and routers. Adapters can be installed on VCF Operations nodes to extend monitoring to additional systems and data sources beyond the core VCF components.

The VMware Kubernetes Service (VKS) layer of VCF extends enterprise monitoring to container workloads and Kubernetes infrastructure. VKS clusters support MachineHealthCheck objects for both control plane nodes and worker node deployments; administrators manage these through the VCF CLI `vcf cluster machinehealthcheck` command, which supports configuration of unhealthy-condition definitions, maximum unhealthy thresholds (expressed as an absolute number or a percentage), and label selectors to target specific machines. The VCF CLI `vcf cluster machinehealthcheck node get` and `vcf cluster machinehealthcheck control-plane get` commands retrieve node and control plane health status on demand. VKS cluster health conditions, including the Ready and ControlPlaneReady status indicators, reflect the operational state of each cluster and are accessible via `kubectl` and the VCF CLI. The VCF Automation Provider Management UI includes a Services overview interface that exposes VKS cluster management functions and the health status of all cluster elements, giving platform operators a centralized view of the VKS tier. The Kubernetes API server within VKS clusters exposes diagnostic and health check endpoints (`/healthz`, `/livez`, `/readyz`, `/statusz`, and `/metrics`) that external monitoring systems can scrape to track API server availability and readiness; the `kube-apiserver` also emits the `kubernetes_healthchecks_total` counter and `kubernetes_healthcheck` gauge metric, which report structured health check status for components including `etcd` and `autoregister-completion`. Access to these monitoring endpoints is governed by the built-in Kubernetes system:monitoring RBAC group, which grants read access to liveness, readiness, and metrics endpoints while restricting broader cluster access. The Kubernetes Watch API allows monitoring tools to track changes to any API object as a live stream, supporting event-driven monitoring without continuous polling. The Node Problem Detector monitors node health and surfaces node-level problems as Kubernetes events and node conditions that external monitoring systems can consume via the event stream.

For workload-level monitoring within VKS clusters, Prometheus Operator is available as a standard package and defines custom resource definitions to deploy and manage Prometheus and Alertmanager StatefulSets, enabling high-availability alerting configurations. The Prometheus Alertmanager custom resource definition configures alert grouping, inhibition, and routing to external notification systems. The Istio service mesh, also available as a standard package on VKS clusters, provides service-level monitoring alongside its traffic management capabilities. The Dynatrace Operator can be deployed on VKS clusters to automate the deployment, scaling, and maintenance of the Dynatrace monitoring stack. Historical analysis, dashboarding, and alerting on cluster-level metrics require a dedicated monitoring solution configured to scrape the VKS metrics endpoints and subscribe to the Kubernetes event stream. The `vcf cluster support-bundler create` command gathers diagnostic information from Supervisor and VKS clusters when troubleshooting is needed.

At the network layer, NSX provides monitored indicators including node uptime status, deployment status, management and controller connections, tunnel status, PNIC status, and uptime duration. NSX time series APIs allow viewing data such as VPN statistics, including metadata about the number of tunnels configured and how many are up or down at a given time.

VCF Operations also supports self-monitoring through the VCF Operations adapter, which collects internal metrics such as activity success counts. This allows administrators to verify that the monitoring platform itself is functioning correctly. The VCF Operations policy engine monitors conditions and generates events to react to changing states in the orchestrator server or connected technologies. The Adapter Instance monitoring interval is configurable, with intervals as granular as five minutes.

VMware Private AI Services (PAIS) includes a built-in observability stack that supplies monitoring data for AI infrastructure and AI workloads. PAIS Controller metrics are exposed at a Prometheus metrics endpoint on the `pais-controller-manager` service,

allowing collection by monitoring systems and visualization in observability platforms such as Grafana. Observability is activated by updating the PAISConfiguration custom resource with optional sections for Prometheus metrics collection and LLM trace forwarding. The Prometheus runtime can be configured with a specified storage class policy and a configurable metrics retention period, giving administrators control over how long monitoring data is stored. A Prometheus server pod must be in a ready and running state for observability to function.

LLM trace data from PAIS can be forwarded to an OpenTelemetry Collector endpoint over http/protobuf or gRPC, providing distributed tracing for AI model inference activity. The observability.llmTraces configuration governs trace forwarding and works alongside the Prometheus metrics pipeline to produce two signals: metrics for health and performance, and traces for inference-level behavior. PAIS Controller metrics include the Model Endpoint Controller and PAIS Configuration Controller, giving monitoring platforms component-level visibility into PAIS operations.

In 9.1, PAIS adds direct integration with VCF Operations. The controller-pod-metrics-streaming setting activates streaming of PAIS controller pod metrics from the Supervisor cluster to VCF Operations, allowing administrators to view PAIS metrics alongside other VCF infrastructure metrics. A Telegraf deployment in the vmware-system-monitoring namespace uses a telegraf-user-config ConfigMap to manage user-configured monitoring endpoints for PAIS. The VCF Consumption CLI connects to the Supervisor endpoint and enables VI administrators to stream metrics from the PAIS controller pod. DevOps engineers can visualize health and performance metrics for PAIS components running in a VCF Automation namespace using Grafana, and MLOps engineers and application developers can monitor the health, quality, and behavior of AI applications and the models used in agents through the same observability platforms.

PAIS supplies the technical monitoring mechanisms this control requires. Enterprise-wide orchestration of monitoring controls, including alert threshold definition, escalation procedures, and integration with a centralized monitoring or log management platform, remain organizational responsibilities.

#### **VMware vDefend (Contributes)**

VMware vDefend (vDefend) supports network security monitoring through multiple integrated capabilities that contribute to enterprise-wide monitoring frameworks.

Security Intelligence, hosted on the Security Services Platform (SSP), provides continuous visibility into network traffic flows across protected workloads. It monitors flows between application pairs, updates the Policy Protection Status for each application to indicate that flows are being monitored, and continuously detects new flows between environment pairs, alerting administrators to unauthorized or unexpected traffic. The Monitor & Plan Overview page presents a consolidated view of pending actions, data collection activity status, network security posture, flow trends, and a summary of suspicious traffic. The Security Intelligence Recommendations monitoring capability checks for changes in the scope of monitored entities every hour when the Monitoring toggle is enabled, allowing administrators to keep policy recommendations current as environments evolve. Segmentation Monitoring provides dedicated views of application and infrastructure flows with configurable time ranges spanning from the last hour to the last month, and displays flows related to detected infrastructure servers in a specified time interval.

The Network Detection and Response (NDR) Sensor checks the status of activated detection features every 15 minutes and updates its configuration accordingly, supporting sustained threat detection coverage. NDR Sensor health status monitoring tracks whether key components, including network traffic capture, file analysis, and data transmission, are operating properly. The NDR Sensor also sends alerts notifying administrators of conditions such as services being down, high CPU, memory, or disk usage, packet drops, or missed heartbeats. NDR provides role-based access control covering triage operations such as viewing detections and campaigns, as well as SIEM configuration management.

The Security Services Platform monitors its own health, with Yellow or Red health status indicators displayed when issues occur in subcomponents. SSP monitoring tracks the status of the flow clustering job that runs every hour and reports suspicious traffic detector execution status after each run, with values such as SUCCESS, NOT\_ENOUGH\_BASELINE, and FAILURE.

The Security dashboard provides a consolidated view for monitoring and configuring security features across network workloads. In federated deployments, the Global Manager provides centralized event management and monitoring for malicious IP blocking activity across all Local Managers.

vDefend generates Unified Security Logs that VCF Log Management can collect and analyze. These logs cover Distributed Firewall, Gateway Firewall, Identity Firewall, FQDN Analysis, TLS Inspection, Intrusion Detection/Prevention System (IDPS), and URL Filtering activity. Broader log aggregation and SIEM correlation for enterprise-wide monitoring remain the responsibility of the SIEM and logging infrastructure deployed alongside vDefend.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) provides monitoring capabilities specific to the disaster recovery domain and integrates with the broader VCF monitoring ecosystem through dedicated Protection and Recovery dashboards in VCF Operations. VCF Operations collects configuration, component health, and usage information from protection and recovery servers and site pairs, presenting this through an overview dashboard that displays all site pairs, configurations, health scores, and protection and recoverability status for sites and virtual machines. PNR provides default health alerts for protection groups, recovery plans, cloud file systems, recovery SDDCs, and protected sites through VCF Operations, and the dashboards surface alerts informing administrators about configurations and deployments that require attention. Separate dashboard views cover vSAN Snapshots and Replication monitoring, and an Inventory view provides health status, metrics, alerts, and network topology for all monitored objects.

Within its own domain, PNR provides event logging and configurable alarms for replication and recovery operations. Administrators can create alarms that trigger on specified event conditions, with support for email notifications, SNMP traps, or script execution in response. PNR monitors connections between sites by sending periodic ping requests, logs connection check events, and generates alarms when broken connections are detected. The alarm system generates warning-level alerts for expired snapshot schedules, replication errors such as missing VMs, and connectivity issues with protected and target sites. PNR also monitors resource consumption on the Protection and Recovery Server host and raises alarms when configurable resource thresholds are reached.

PNR includes a health monitoring service that checks each health finding every five minutes and records a timestamp at each check. Health findings are tracked with status history that can be reviewed over configurable time periods, defaulting to a 24-hour window. The Protection and Recovery Appliance supports syslog forwarding to a remote syslog server for log analysis and centralized monitoring, allowing appliance-level events to flow into existing log management infrastructure.

The Clean Room Orchestrator, part of PNR's Cyber Recovery capabilities, supports monitoring of clean room events, tasks, and alarms for recovery operations. Clean Room Orchestrator can forward events to VCF Operations, Microsoft Sentinel, or Splunk Cloud for centralized monitoring, supporting organizations that require DR event data to flow into enterprise SIEM and log management platforms as part of a broader monitoring strategy. The Clean Room Orchestrator integrates with endpoint detection and response tools, including CrowdStrike and Carbon Black sensor appliances, for threat detection within isolated recovery environments.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) contributes to enterprise-wide monitoring controls through a layered set of observability capabilities covering database health, infrastructure compliance, and event-driven alerting.

DSM provides a built-in monitoring framework that tracks database instance health across multiple dimensions. From the Monitoring tab in the DSM UI or the vSphere Client, administrators and authorized users can view real-time and historical metrics for database VMs, including CPU Health, Data Disk Health, System Disk Health, Max Connections, Active Connections, Write Throughput, Read Throughput, Thread Resource Utilization, Slow Queries per Second, InnoDB Buffer Usage, and InnoDB Reads and Writes. The Monitoring page defaults to the last three hours of aggregated metric data, with the time range adjustable through the Time Range Last dropdown. This visibility supports detection of resource contention, performance degradation, and connectivity anomalies at the database layer.

DSM's alert framework provides configurable thresholds for warning and critical conditions on key health indicators. Administrators and users can set separate warning and critical thresholds for CPU Health, Data Disk Health, System Disk Health, and Max Connections parameters. When thresholds are exceeded, DSM generates typed alerts such as Database Engine Health, Custom Config Health, and Backup Storage Health alerts. DSM monitors database replication status and raises an alert when a secondary database is not replicating data from the primary. DSM also raises an alert when metrics are no longer being received from a cluster, supporting detection of monitoring gaps. Alerts can be activated or deactivated per data service through the "Enable Alerts for DB Alert" setting, providing granular control over which databases are subject to active alerting. For each alert, DSM provides mitigation suggestions to help administrators resolve issues without disrupting database services.

In addition to database-level alerts, DSM generates global infrastructure alerts that surface vCenter integration and infrastructure policy issues. These include `INFRASTRUCTURE_POLICY_VC_ALARMS`, raised when vCenter resource alarms are active; `INFRASTRUCTURE_POLICY_CLUSTER_COMPLIANT` and `INVALID_SUPERVISOR_INFRASTRUCTURE_POLICY_STATUS`, which reflect cluster compliance state; `VCENTER_CONNECTIVITY`, raised when vCenter connectivity fails and triggering troubleshooting steps that include verifying connectivity and checking authentication logs; and `LicensingNotUpdated` and `vCenterBinding` API status alerts that surface license and API integration conditions. A Global Alerts page in the DSM UI provides a centralized view of these alerts during and after installation and ongoing operation.

In 9.1, DSM integrates with VCF Operations to extend observability beyond the DSM UI. When configured, VCF Operations automatically discovers and monitors PostgreSQL and MySQL clusters in the Ready state and displays a Data Services View dashboard showing active alerts and inventory items. VCF Operations presents performance data across three levels: cluster-level for overall health, node-level for infrastructure resource use, and database-level for KPIs including Active Connections, Commits, Rollbacks, and Deadlocks. This integration requires configuring the connection between DSM and VCF Operations before use.

DSM also adds SQL Server as a supported database type, with dedicated monitoring through an integrated InfluxDB instance and Telegraf agents deployed across the provider appliance and SQL Server workload clusters. The monitoring framework supports metrics-based and status-condition-based alerts for SQL Server, covering database engine degradation, non-operational status, and automated or transaction log backup failure conditions.

DSM supports event notification to external systems through a webhook integration. Webhooks transmit alert notifications containing the triggered date, triggered time, triggered user email, task status, previous status, and task type. This allows DSM alerts to feed into centralized monitoring or incident management platforms maintained by the organization. DSM generates plug-in registration logs that administrators can review for vCenter connectivity errors, and DSM's troubleshooting guidance directs administrators to review active vCenter infrastructure alarms and resolve underlying issues identified through the alert system.

Enterprise-wide monitoring requires organizational decisions about monitoring architecture, log aggregation platforms, alerting policies, and operational response procedures that lie outside DSM's scope. DSM provides the data and integration points; the organization must determine how those signals are collected, correlated, and acted upon across the broader environment.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) contributes to enterprise-wide monitoring through application-delivery-layer mechanisms built into the Avi Controller and Service Engines.

The Avi Controller monitors Service Engine metrics and sends alarms and notifications to administrators when resource utilization data exceeds defined thresholds. Administrators configure custom thresholds on Service Engine groups through the Resource Events tab in the Avi UI. The Alert feature processes system events and metrics against user-defined rules and generates alerts that trigger configured actions, including sending an email, a syslog message, or an SNMP trap. Alert notifications can be forwarded to remote syslog servers for integration with centralized log management and monitoring platforms.

ControlScripts extend this event-response capability: a ControlScript can alter Avi configuration or communicate to external systems, including through webhook integrations with external monitoring tools, in response to defined system events. This supports dynamic responses including resource scaling or automated notifications based on health scores from monitoring data.

The Avi Controller and Service Engines track health score changes and SSL certificate expiration events. Avi integrates with external monitoring tools such as Prometheus for metrics collection and analysis. User-Defined Metrics allow administrators to define and extract customized telemetry data for specific compliance or reporting needs through the API. Real-time metrics are available via API but deactivate by default after 30 minutes to conserve resources and must be reactivated when sustained real-time visibility is required.

At the data plane, Avi health monitors track the status of backend servers and Service Engines continuously. Service Engine health monitoring records detailed server operational status, including health monitor name, type, last transition timestamps, rise count, fall count, total checks, and total failed checks. Each health monitor can be assigned its own threshold and timeout period for server health validation. Monitors support HTTP, HTTPS, TCP, and UDP health-check protocols with configurable success and failure thresholds, send intervals, and receive timeouts.

In Kubernetes environments managed by the Avi Kubernetes Operator (AKO), the HealthMonitor custom resource definition (CRD) allows users to define custom health monitoring configurations for backend services with fine-grained control over health check parameters. The Avi Multi-Cluster Kubernetes Operator (AMKO) GlobalDeploymentPolicy supports health monitor references for consistent monitoring across multi-cluster deployments. Within VCF Automation, a HealthMonitor custom resource can create monitors for tracking backend pool member status using TCP, PING, and HTTP check types.



## Control CR 7.1

> Denial of service protection: Capability to maintain essential functions during a DoS attack. Resource management: Manage resources to prevent exhaustion.

**SCF Controls:** BCD-12.2, NET-02.1

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides several automated mechanisms across its networking and compute layers that help limit the effects of denial of service attacks, though dedicated perimeter DoS/DDoS mitigation (such as volumetric attack scrubbing) is outside VCF's scope and requires additional infrastructure or services.

At the network layer, VMware vDefend implements SYN proxy protection to defend TCP servers from SYN flood attacks. The SYN proxy mechanism intercepts and validates TCP handshake requests before forwarding legitimate connections to backend servers, operating automatically at the distributed firewall layer without requiring manual intervention for each connection attempt. SYN floods are among the most common forms of DoS attack, and this mechanism addresses them directly within the virtualized network.

NSX provides broadcast and multicast rate limiting capabilities that drop egress packets to protect the network and virtual machines from broadcast and multicast storms. These storm conditions, whether caused by misconfiguration or malicious activity, can saturate network bandwidth and degrade service availability. Rate limiting enforces packet thresholds that automatically discard excess traffic when configured limits are exceeded.

NSX Gateway QoS profiles define limits on traffic rates by specifying permitted information rate and burst size. Traffic that does not conform to the QoS policy is dropped. QoS profiles can be applied to both Tier-1 gateway ingress and egress traffic. NSX QoS Segment Profiles provide bandwidth controls at the segment level, configuring average and peak ingress/egress bandwidth to limit transmit rates and manage burst traffic.

SpoofGuard provides anti-spoofing protections by validating ARP, GARP, and Neighbor Discovery packets against authorized MAC and IP addresses, dropping traffic from VMs using unauthorized source addresses. IP address spoofing is a common technique used in distributed DoS attacks to obscure the source of malicious traffic, and SpoofGuard helps limit this vector within the virtualized environment. SpoofGuard tracks the number of packets dropped due to its checks.

ESX hosts support a guest BPDU filter that prevents virtual machines from sending Bridge Protocol Data Unit traffic to physical switches, protecting physical switching infrastructure from BPDU-induced DoS conditions. The vSphere distributed switch MAC Limit Policy can be configured to drop packets from unknown source MAC addresses when the port's MAC address table limit is reached, limiting the impact of MAC flooding attacks against the virtual switching layer.

vDefend policies support configurable actions including Allow, Deny, and Drop for traffic between sources and destinations. The Drop action silently rejects specified network traffic, while Deny actively informs the client that the connection is rejected. These policy actions can be configured to automatically block traffic from known malicious sources or to restrict traffic patterns associated with DoS activity.

VCF Automation supports an internal host denylist that makes specified internal hosts unreachable to tenant organizations and restricts connection testing API access, helping protect the internal management network from abuse.

At the compute layer, vSphere Distributed Resource Scheduler (DRS) automatically balances resource allocation across cluster hosts, preventing a single workload from consuming disproportionate resources. Resource pools provide hierarchical resource allocation with reservations, limits, and shares. Fixed reservations provide minimum resource availability for VMs within a pool, while limits cap how much CPU or memory a pool can consume, preventing one pool's workloads from starving others. Shares-based prioritization provides proportional allocation during contention.

Network I/O Control (NIOC) manages bandwidth distribution across different traffic types on a vSphere distributed switch, allowing administrators to reserve and prioritize bandwidth for management, vSAN, vMotion, and VM traffic separately. By



reserving bandwidth for system-critical traffic classes, NIOC helps limit the impact of traffic floods on essential platform operations.

vSAN storage policies support IOPS limits per object, throttling I/O operations when a disk exceeds its configured limit.

VCF Operations implements flow limit protection with outbound limits on TCP and UDP flows (1,000 allowed / 50 denied for TCP, 500 allowed / 50 denied for UDP), preventing individual entities from overloading the total flow capacity. VCF Operations APIs and VCF Operations for Networks APIs also implement rate limiting, returning HTTP 429 responses when clients send too many requests.

VMware vDefend extends these capabilities with the Gateway Firewall, which filters traffic at the earliest point when traffic enters the NSX overlay network, providing a perimeter enforcement point where traffic can be inspected and dropped before reaching workload segments. The distributed Layer 2 through Layer 7 firewall can enforce drop rules to deny matching traffic, and IDS/IPS capabilities detect and block attack patterns within the virtualized network. Combined with the distributed firewall's per-workload enforcement, this layered approach applies automated filtering at multiple points in the network path.

Because VCF operates within the virtualized environment and does not include dedicated perimeter DoS/DDoS mitigation appliances or services for volumetric attack scrubbing, organizations need to supplement these platform capabilities with external DoS protection infrastructure, network-level traffic scrubbing services, and incident response procedures to address large-scale or application-layer DoS attacks that originate outside the VCF boundary.

### **VMware vDefend (Contributes)**

VMware vDefend provides several automated mechanisms that directly protect against or limit the effects of denial of service attacks at the network layer within the virtualized environment.

vDefend includes built-in flood protection that imposes active session limits for ICMP, UDP, and half-open TCP flows. Configurable TCP Half Open Connection Limit and UDP Active Flow Limit settings restrict the number of active, not-fully-established TCP flows and active UDP flows allowed through the firewall. When configured thresholds are exceeded, new connection attempts are dropped, addressing TCP SYN flood conditions and UDP flood conditions by limiting connection state table exhaustion. These flood protection settings are configurable per firewall section, allowing administrators to tune thresholds based on the expected traffic profile of different workload tiers. The Distributed Firewall also includes network and transport layer protocol validation to protect against low-level IP attacks that target protocol parsing and state tracking.

vDefend IDS/IPS extends DoS protection with signature-based and behavioral detection of denial of service attack patterns. The IDS/IPS engine supports configurable thresholds and rate filters that trigger events based on traffic behavior, allowing dynamic adjustment of a rule's action when specific conditions are met. Administrators can set trigger conditions based on rule match counts within specified time periods, so that isolated matches generate alerts while sustained attack patterns trigger automatic blocking. Rate filters can be configured to track by source IP address, counting connection attempts from a specific source individually and switching to drop action when that source exceeds the configured threshold within the specified time window. When deployed in IPS mode, matching traffic is dropped in-line, providing automated mitigation without manual intervention. The IDS/IPS engine also includes an automated self-protection mechanism that selectively drops IDS/IPS events and packet capture data when high event volumes threaten out-of-memory conditions, helping maintain system availability under sustained high-traffic loads.

vDefend supports Malicious IP Filtering, which downloads and enforces lists of known malicious IP addresses. Traffic from IPs on the malicious IP feed is automatically blocked through firewall rules, reducing the attack surface by limiting known attack sources from reaching protected workloads. Exception lists allow administrators to exclude specific IPs from blocking when false positives are identified.

At the gateway perimeter, vDefend can block traffic based on application-level information including URL categories, fully-qualified domain names, and URL reputation scores. The Gateway Firewall in the Advanced Threat Prevention edition also includes gateway intrusion prevention service, extending IPS-based detection and blocking to north-south traffic at the segment boundary. The Distributed Firewall's ability to apply Allow, Drop, or Reject actions to traffic matching selected

services provides granular control over which protocols and applications are permitted, limiting the protocols available for abuse in an application-layer DoS attack.

vDefend contributes to DoS protection but does not replace dedicated perimeter DDoS mitigation infrastructure. Volumetric attacks that saturate upstream network links require scrubbing services or ISP-level mitigation before traffic reaches the virtualized environment. vDefend's protections operate within the software-defined network, addressing DoS conditions at the workload and segment level rather than at the network edge.

### **VCF Protection and Recovery (Meets)**

VCF Protection and Recovery (PNR) implements site-level failover capabilities to maintain availability of critical virtual machine workloads after a disruption.

Recovery plans automate the failover process between protected and recovery sites. During disaster recovery failover, if an error occurs on the protected site, the failover continues without interruption, prioritizing workload recovery. Recovery plans support both planned migration for orderly, scheduled failover and disaster recovery for unplanned site failures. PNR provides an API workflow through the VCF Operations Orchestrator Plug-In for PNR to initiate failover programmatically, enabling integration with automated orchestration frameworks.

PNR supports multiple replication methods that underpin the failover capability. vSphere Replication provides host-based replication with configurable recovery point objectives. A recovery option allows vSphere Replication to synchronize recent changes from the source site to the target before recovering virtual machines, reducing data loss during recovery operations. vSphere Replication supports up to 200 recovery points when using the Latest instances option, enabling selection of a specific point in time for recovery. Replication continues at the defined recovery point objective after vSphere High Availability restarts a virtual machine. Array-based replication leverages storage vendor capabilities for datastore-level failover, with a configurable option to wait for device rediscovery after failover. Automated failback is supported only for array-based replication. Virtual Volumes policy-driven replication provides a third replication method.

Failover and failback can be performed independently on sites, allowing isolated recovery operations without affecting other recovery sites. For stretched storage environments, PNR requires that the failover command isolate devices at the recovery site. IP customization is applied during the failover process, supporting DHCP, static IPv4, and IPv6 addressing so that recovered virtual machines are reachable on the recovery site network. Inventory mappings translate protected site resources to their recovery site equivalents.

PNR supports failover even when operating in degraded mode, though test failover and reprotect operations are not available in that state. Test failover allows organizations to validate their failover procedures without disrupting production replication. After a successful failover, the reprotect process reverses the direction of protection, making the recovery site the new protected site and restoring replication to prepare for failback.

PNR supports failover for Windows Server Failover Clustering (WSFC) workloads. The Cyber Recovery capability enables scheduled snapshot management through vSAN Data Protection groups for VM-level protection that integrates with the broader failover workflow.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides built-in high availability and failover mechanisms for PostgreSQL, MySQL, and SQL Server database workloads through two HA topologies that organizations can select based on their availability requirements.

The Single vSphere Cluster HA topology co-locates all database nodes within one vSphere cluster. For PostgreSQL, DSM deploys a primary instance, one database-level synchronous replica, and a dedicated monitor node. The monitor node orchestrates automatic database-level failover: when the primary fails, the monitor detects the failure and promotes the synchronous streaming replica to the primary role without requiring manual intervention. MySQL under this topology uses a three-node configuration with a single primary and two database-level synchronous replicas sharing the same cluster. SQL Server uses a three-node configuration with Microsoft Always On availability groups, two synchronous replicas, and one configuration-only replica. After a SQL Server failover, the new primary maintains the security and management

configurations from the previous primary instance. In all cases, DSM database nodes also rely on the underlying VCF platform for VM-level recovery through vSphere HA and Distributed Resource Scheduler, and on platform storage redundancy.

The Cross vSphere Cluster HA topology distributes database nodes across three separate vSphere clusters, reducing the blast radius of a single cluster failure. For PostgreSQL, this topology places a primary, a database-level replica, and a monitor on distinct clusters. For MySQL, a primary, a replica, and a monitor are distributed across three clusters. For SQL Server, Always On availability groups are configured with two synchronous replicas and one configuration-only replica, with each replica node in a separate vSphere cluster. Cross vSphere Cluster HA provides the highest availability and resilience for environments that require high availability across multiple failure domains. When configuring Cross-Cluster HA in an infrastructure policy, three clusters must be selected from the same datacenter and must share the same storage policies and distributed port group. The Enable-Cross-Cluster-HA infrastructure policy setting activates this topology. DSM raises a compliance alert if vSphere HA is not active on a cluster referenced by an infrastructure policy.

The DSM Provider Appliance management plane also relies on vSphere High Availability so the control plane can recover from a host failure without manual intervention.

For cross-site disaster recovery, DSM supports a multi-site deployment model for PostgreSQL that replicates data to a secondary site. When a failover to the secondary site is needed, DSM provides a switchover workflow to promote the secondary instance to the primary role. After the original site recovers, a failback switchover reverses the roles to restore the original topology. DSM also provides the ability to copy replication user connection strings from a primary instance so that secondary instances can be redirected to the new primary during failover operations. One documented limitation applies: PostgreSQL logical replication requires manual intervention to resume replication after the source cluster experiences a failover. When a MySQL HA cluster is used as the replication source, automatic source failover for read replicas requires the allocation of additional IP addresses from the IP pool for each HA node, and read replicas reconnect to another cluster member after several failed attempts, which can take a few minutes before the switch completes. For SQL Server, during a failover new writes are not protected with a second copy until the failed node rejoins the cluster.

DSM HA clusters require automated backups to be enabled in order to provision databases. DSM provides real-time alerts for resource and connectivity issues together with continuous performance monitoring, which supports operators in detecting conditions that may require a failover response. Webhook notifications can also be configured so that external systems are alerted on data service operation failures.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer provides application-layer DoS and DDoS protection through multiple automated mechanisms built into the Service Engine data plane. At the TCP level, Service Engines automatically apply TCP SYN cookies, idle connection reaping, and protocol validation to mitigate connection-state attacks. These protections are active on every virtual service without requiring manual configuration.

For more targeted attack patterns, Avi provides configurable DDoS threshold profiles applied at the virtual service level. Threshold profiles define minimum and maximum detection thresholds for a range of attack signatures, including SYN\_FLOOD, BAD\_RST\_FLOOD, MALFORMED\_FLOOD, FAKE\_SESSION, ZERO\_WINDOW\_STRESS, SMALL\_WINDOW\_STRESS, DOS\_HTTP\_TIMEOUT, DOS\_HTTP\_ERROR, and DOS\_HTTP\_ABORT. The Service Engine uses a dynamic threshold mechanism that starts at a configurable minimum value and scales upward as attack intensity increases, up to a configurable maximum, which reduces false positives under normal load spikes. Default threshold values can be overridden by configuring DDoS threshold profiles in the Service Engine Group settings. At the application layer, attack thresholds for SSL errors, application errors, and slow-URL patterns have configurable minimum and maximum values per proxy core per virtual service, covering attack types such as DOS\_SSL\_ERROR, DOS\_APP\_ERROR, DOS\_SLOW\_URL, TCP\_NON\_SYN\_FLOOD, DNS\_ATTACK\_REFLECTION, and DNS\_ATTACK\_AMPLIFICATION\_EGRESS. Administrators select a DDoS Policy for each virtual service on the Policies tab during virtual service configuration.

Rate limiting is available through DataScript Rate Limiters and a shared rate limiter. The DataScript Rate Limiter enforces request rate limits per virtual service. The shared rate limiter distributes configured rate limiter counts among Service Engines and enforces rate limits on incoming connections using token-based rate management across Service Engine cores. Virtual

services also support connection rate limiting to restrict the number of new connections accepted per time period. For DNS-based attacks, a DNS Amplification Egress DDoS mitigation policy can be attached to a virtual service with a deny action and a configurable maximum mitigation age.

Avi extends DDoS mitigation to IPv6 traffic, including protection against DNS Reflection Ingress attacks, which involve sending DNS queries with spoofed IP addresses of the victim to swamp the victim with unsolicited traffic through DNS server responses.

When attacks are detected, Avi can respond automatically through ControlScript-based alert actions. The Avi Controller monitors virtual services and generates DDOS events with configurable alert levels and thresholds. Alert actions can invoke a ControlScript that automatically adds the offending client IP address to a blocklist IP group attached to a network security policy, blocking or rate-shaping the attacker without manual intervention. ControlScripts can also parse DDoS attack event details including attack type and source IP addresses to automatically trigger network security policy updates that deny traffic from attacking clients. The DDoS Insights view in Application Monitoring breaks down attack data by layer 4 and layer 7 vectors, providing metrics such as the number of network attacks per second, including IP fragmentation attacks and TCP SYN flood events. The Service Engine calculates a DDoS Score for each virtual service that increases a security penalty depending on the severity of a detected DoS attack. The Service Engine also detects and logs potential DoS attacks when clients close connections prematurely during request transmission.

For Kubernetes environments, the Avi Kubernetes Operator (AKO) exposes DDoS protection through the L4 Rule CRD, which allows administrators to express custom Security Policies applied to virtual service traffic, including configuration for DDoS attack detection and mitigation. This brings Avi's DDoS capabilities to Kubernetes-managed virtual services through a declarative policy model.

Auditors should verify that DDoS threshold profiles have been configured in the Service Engine Group to reflect the environment's traffic patterns, that DDoS Policies have been assigned to virtual services, and that ControlScript alert actions are enabled for automated blocklisting on critical virtual services.

## Control CR 7.1(1)

> Denial of service protection: Capability to maintain essential functions during a DoS attack. Resource management: Manage resources to prevent exhaustion.

**SCF Controls:** NET-02.1

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides several automated mechanisms across its networking and compute layers that help limit the effects of denial of service attacks, though dedicated perimeter DoS/DDoS mitigation (such as volumetric attack scrubbing) is outside VCF's scope and requires additional infrastructure or services.

At the network layer, VMware vDefend implements SYN proxy protection to defend TCP servers from SYN flood attacks. The SYN proxy mechanism intercepts and validates TCP handshake requests before forwarding legitimate connections to backend servers, operating automatically at the distributed firewall layer without requiring manual intervention for each connection attempt. SYN floods are among the most common forms of DoS attack, and this mechanism addresses them directly within the virtualized network.

NSX provides broadcast and multicast rate limiting capabilities that drop egress packets to protect the network and virtual machines from broadcast and multicast storms. These storm conditions, whether caused by misconfiguration or malicious activity, can saturate network bandwidth and degrade service availability. Rate limiting enforces packet thresholds that automatically discard excess traffic when configured limits are exceeded.

NSX Gateway QoS profiles define limits on traffic rates by specifying permitted information rate and burst size. Traffic that does not conform to the QoS policy is dropped. QoS profiles can be applied to both Tier-1 gateway ingress and egress traffic. NSX QoS Segment Profiles provide bandwidth controls at the segment level, configuring average and peak ingress/egress bandwidth to limit transmit rates and manage burst traffic.

SpoofGuard provides anti-spoofing protections by validating ARP, GARP, and Neighbor Discovery packets against authorized MAC and IP addresses, dropping traffic from VMs using unauthorized source addresses. IP address spoofing is a common technique used in distributed DoS attacks to obscure the source of malicious traffic, and SpoofGuard helps limit this vector within the virtualized environment. SpoofGuard tracks the number of packets dropped due to its checks.

ESX hosts support a guest BPDU filter that prevents virtual machines from sending Bridge Protocol Data Unit traffic to physical switches, protecting physical switching infrastructure from BPDU-induced DoS conditions. The vSphere distributed switch MAC Limit Policy can be configured to drop packets from unknown source MAC addresses when the port's MAC address table limit is reached, limiting the impact of MAC flooding attacks against the virtual switching layer.

vDefend policies support configurable actions including Allow, Deny, and Drop for traffic between sources and destinations. The Drop action silently rejects specified network traffic, while Deny actively informs the client that the connection is rejected. These policy actions can be configured to automatically block traffic from known malicious sources or to restrict traffic patterns associated with DoS activity.

VCF Automation supports an internal host denylist that makes specified internal hosts unreachable to tenant organizations and restricts connection testing API access, helping protect the internal management network from abuse.

At the compute layer, vSphere Distributed Resource Scheduler (DRS) automatically balances resource allocation across cluster hosts, preventing a single workload from consuming disproportionate resources. Resource pools provide hierarchical resource allocation with reservations, limits, and shares. Fixed reservations provide minimum resource availability for VMs within a pool, while limits cap how much CPU or memory a pool can consume, preventing one pool's workloads from starving others. Shares-based prioritization provides proportional allocation during contention.

Network I/O Control (NIOC) manages bandwidth distribution across different traffic types on a vSphere distributed switch, allowing administrators to reserve and prioritize bandwidth for management, vSAN, vMotion, and VM traffic separately. By

reserving bandwidth for system-critical traffic classes, NIOC helps limit the impact of traffic floods on essential platform operations.

vSAN storage policies support IOPS limits per object, throttling I/O operations when a disk exceeds its configured limit.

VCF Operations implements flow limit protection with outbound limits on TCP and UDP flows (1,000 allowed / 50 denied for TCP, 500 allowed / 50 denied for UDP), preventing individual entities from overloading the total flow capacity. VCF Operations APIs and VCF Operations for Networks APIs also implement rate limiting, returning HTTP 429 responses when clients send too many requests.

VMware vDefend extends these capabilities with the Gateway Firewall, which filters traffic at the earliest point when traffic enters the NSX overlay network, providing a perimeter enforcement point where traffic can be inspected and dropped before reaching workload segments. The distributed Layer 2 through Layer 7 firewall can enforce drop rules to deny matching traffic, and IDS/IPS capabilities detect and block attack patterns within the virtualized network. Combined with the distributed firewall's per-workload enforcement, this layered approach applies automated filtering at multiple points in the network path.

Because VCF operates within the virtualized environment and does not include dedicated perimeter DoS/DDoS mitigation appliances or services for volumetric attack scrubbing, organizations need to supplement these platform capabilities with external DoS protection infrastructure, network-level traffic scrubbing services, and incident response procedures to address large-scale or application-layer DoS attacks that originate outside the VCF boundary.

### **VMware vDefend (Contributes)**

VMware vDefend provides several automated mechanisms that directly protect against or limit the effects of denial of service attacks at the network layer within the virtualized environment.

vDefend includes built-in flood protection that imposes active session limits for ICMP, UDP, and half-open TCP flows. Configurable TCP Half Open Connection Limit and UDP Active Flow Limit settings restrict the number of active, not-fully-established TCP flows and active UDP flows allowed through the firewall. When configured thresholds are exceeded, new connection attempts are dropped, addressing TCP SYN flood conditions and UDP flood conditions by limiting connection state table exhaustion. These flood protection settings are configurable per firewall section, allowing administrators to tune thresholds based on the expected traffic profile of different workload tiers. The Distributed Firewall also includes network and transport layer protocol validation to protect against low-level IP attacks that target protocol parsing and state tracking.

vDefend IDS/IPS extends DoS protection with signature-based and behavioral detection of denial of service attack patterns. The IDS/IPS engine supports configurable thresholds and rate filters that trigger events based on traffic behavior, allowing dynamic adjustment of a rule's action when specific conditions are met. Administrators can set trigger conditions based on rule match counts within specified time periods, so that isolated matches generate alerts while sustained attack patterns trigger automatic blocking. Rate filters can be configured to track by source IP address, counting connection attempts from a specific source individually and switching to drop action when that source exceeds the configured threshold within the specified time window. When deployed in IPS mode, matching traffic is dropped in-line, providing automated mitigation without manual intervention. The IDS/IPS engine also includes an automated self-protection mechanism that selectively drops IDS/IPS events and packet capture data when high event volumes threaten out-of-memory conditions, helping maintain system availability under sustained high-traffic loads.

vDefend supports Malicious IP Filtering, which downloads and enforces lists of known malicious IP addresses. Traffic from IPs on the malicious IP feed is automatically blocked through firewall rules, reducing the attack surface by limiting known attack sources from reaching protected workloads. Exception lists allow administrators to exclude specific IPs from blocking when false positives are identified.

At the gateway perimeter, vDefend can block traffic based on application-level information including URL categories, fully-qualified domain names, and URL reputation scores. The Gateway Firewall in the Advanced Threat Prevention edition also includes gateway intrusion prevention service, extending IPS-based detection and blocking to north-south traffic at the segment boundary. The Distributed Firewall's ability to apply Allow, Drop, or Reject actions to traffic matching selected



services provides granular control over which protocols and applications are permitted, limiting the protocols available for abuse in an application-layer DoS attack.

vDefend contributes to DoS protection but does not replace dedicated perimeter DDoS mitigation infrastructure. Volumetric attacks that saturate upstream network links require scrubbing services or ISP-level mitigation before traffic reaches the virtualized environment. vDefend's protections operate within the software-defined network, addressing DoS conditions at the workload and segment level rather than at the network edge.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer provides application-layer DoS and DDoS protection through multiple automated mechanisms built into the Service Engine data plane. At the TCP level, Service Engines automatically apply TCP SYN cookies, idle connection reaping, and protocol validation to mitigate connection-state attacks. These protections are active on every virtual service without requiring manual configuration.

For more targeted attack patterns, Avi provides configurable DDoS threshold profiles applied at the virtual service level. Threshold profiles define minimum and maximum detection thresholds for a range of attack signatures, including SYN\_FLOOD, BAD\_RST\_FLOOD, MALFORMED\_FLOOD, FAKE\_SESSION, ZERO\_WINDOW\_STRESS, SMALL\_WINDOW\_STRESS, DOS\_HTTP\_TIMEOUT, DOS\_HTTP\_ERROR, and DOS\_HTTP\_ABORT. The Service Engine uses a dynamic threshold mechanism that starts at a configurable minimum value and scales upward as attack intensity increases, up to a configurable maximum, which reduces false positives under normal load spikes. Default threshold values can be overridden by configuring DDoS threshold profiles in the Service Engine Group settings. At the application layer, attack thresholds for SSL errors, application errors, and slow-URL patterns have configurable minimum and maximum values per proxy core per virtual service, covering attack types such as DOS\_SSL\_ERROR, DOS\_APP\_ERROR, DOS\_SLOW\_URL, TCP\_NON\_SYN\_FLOOD, DNS\_ATTACK\_REFLECTION, and DNS\_ATTACK\_AMPLIFICATION\_EGRESS. Administrators select a DDoS Policy for each virtual service on the Policies tab during virtual service configuration.

Rate limiting is available through DataScript Rate Limiters and a shared rate limiter. The DataScript Rate Limiter enforces request rate limits per virtual service. The shared rate limiter distributes configured rate limiter counts among Service Engines and enforces rate limits on incoming connections using token-based rate management across Service Engine cores. Virtual services also support connection rate limiting to restrict the number of new connections accepted per time period. For DNS-based attacks, a DNS Amplification Egress DDoS mitigation policy can be attached to a virtual service with a deny action and a configurable maximum mitigation age.

Avi extends DDoS mitigation to IPv6 traffic, including protection against DNS Reflection Ingress attacks, which involve sending DNS queries with spoofed IP addresses of the victim to swamp the victim with unsolicited traffic through DNS server responses.

When attacks are detected, Avi can respond automatically through ControlScript-based alert actions. The Avi Controller monitors virtual services and generates DDOS events with configurable alert levels and thresholds. Alert actions can invoke a ControlScript that automatically adds the offending client IP address to a blocklist IP group attached to a network security policy, blocking or rate-shaping the attacker without manual intervention. ControlScripts can also parse DDoS attack event details including attack type and source IP addresses to automatically trigger network security policy updates that deny traffic from attacking clients. The DDoS Insights view in Application Monitoring breaks down attack data by layer 4 and layer 7 vectors, providing metrics such as the number of network attacks per second, including IP fragmentation attacks and TCP SYN flood events. The Service Engine calculates a DDoS Score for each virtual service that increases a security penalty depending on the severity of a detected DoS attack. The Service Engine also detects and logs potential DoS attacks when clients close connections prematurely during request transmission.

For Kubernetes environments, the Avi Kubernetes Operator (AKO) exposes DDoS protection through the L4 Rule CRD, which allows administrators to express custom Security Policies applied to virtual service traffic, including configuration for DDoS attack detection and mitigation. This brings Avi's DDoS capabilities to Kubernetes-managed virtual services through a declarative policy model.



Auditors should verify that DDoS threshold profiles have been configured in the Service Engine Group to reflect the environment's traffic patterns, that DDoS Policies have been assigned to virtual services, and that ControlScript alert actions are enabled for automated blocklisting on critical virtual services.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 7.2

> Resource management: Allocate resources to prioritize essential functions.

**SCF Controls:** CAP-01

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides built-in capacity and performance management through VCF Operations, which delivers capabilities organized around three functional areas: assess, optimize, and plan. These capabilities allow administrators to evaluate whether physical capacity is sufficient to meet current or forecasted demand, balance supply against consumption, and manage production capacity as organizational requirements change.

The assess capability evaluates current and projected resource utilization across compute, storage, and networking. VCF Operations includes a capacity analytics engine that analyzes historical resource utilization data and generates projected utilization metrics. The engine computes and publishes metrics in the Capacity Analytics Generated group, including a capacityRemainingPercentage that tracks the most constrained resource relative to usable capacity. Capacity dashboards display key indicators including time remaining before capacity is projected to run out, the amount of remaining capacity, the number of virtual machines that can fit in remaining capacity, and reclaimable resources that can increase available capacity. The Capacity Summary dashboard provides details on available resources and what is currently running on those resources. The recommended approach for usable capacity is to maintain a green state by calculating total capacity minus high availability reservation for the time remaining.

The optimize capability helps administrators manage resources within the virtualized infrastructure to maintain performance and maximize resource utilization while keeping capacity within acceptable limits. The Workload Optimization screen lets administrators optimize workload placement based on policy settings, accessible via the Workload Placement tab under Capacity. Capacity Optimization allows forecasting the impact of adding workloads by modeling various scenarios to determine whether desired workloads can fit in targeted data centers. Administrators configure capacity policy settings by defining requirements and tolerances for their environment, such as CPU utilization thresholds, and then configure object groups and policies accordingly. At the vSphere layer, Distributed Resource Scheduler (DRS) continuously compares cluster and host-level metrics against an ideal resource usage given resource pool attributes, current demand, and a configurable imbalance target, automatically migrating virtual machines to maintain load balance. vSphere also applies admission control when creating or modifying resource pools, rejecting reservation requests that would exceed available capacity to prevent over-commitment of guaranteed resource allocations.

The plan capability uses historical resource utilization data and real-time predictive projections to forecast future capacity needs. The capacity engine uses an industry-standard statistical analysis model of demand behavior for its projections. Infrastructure Planning shows time remaining and the impact on CPU and memory when capacity changes, including graphical representations of whether removing capacity decreases the time remaining before resources run out. Committed scenarios allow capacity to be reserved so that operations teams do not perform ad hoc resource increases on workloads while a capacity manager is engaged in resource planning for new projects.

At the storage layer, vSAN provides storage-specific capacity and performance metrics. VCF Operations collects disk I/O and disk space metrics to monitor vSAN disk group performance, including read-cached hit rate, write-buffer free percentage, cache disk destage rate, congestion metrics, and outstanding I/O statistics. vSAN displays deduplication and compression savings information when these space efficiency features are enabled. vSAN also supports auto-policy management to optimize capacity utilization based on cluster size and type. The vSAN Performance Dashboard in VCF Operations provides a consolidated view of storage performance alongside compute and network resources.

The Consumption layer within VCF extends these capacity management capabilities to Kubernetes workloads running on VMware Kubernetes Service (VKS) and the vSphere Supervisor. At the namespace level, vSphere Namespace resource capacity can be configured with explicit limits on CPU, memory, storage, and storage policy consumption. When a vSphere Namespace is configured, a resource pool is created on VMware vCenter, and administrators can apply capacity and usage

quotas to bound what workloads in that namespace may consume; by default, this resource pool is created without capacity or usage quotas, requiring administrators to configure limits explicitly. The StoragePolicyQuota resource allows per-namespace limits on storage capacity allocated under a given storage policy, specified via the `spec.limit` parameter. VKS Cluster Management quota policies provide an additional boundary at the cluster management layer, with preconfigured templates (small, medium, and large) that define common limits on CPU and memory requests, or a custom template that allows specification of CPU and memory limits and request quantities for various resource types.

Within Kubernetes namespaces, ResourceQuota objects control resource allocation across pods and containers, allowing administrators to set per-namespace quotas on CPU, memory, and storage. ResourceQuota configuration can be enabled to protect system-critical priority classes, including system-node-critical and system-cluster-critical, from resource contention. LimitRange resources work alongside ResourceQuota to set default and maximum resource requests and limits at the pod level, reducing resource exhaustion from workloads that do not specify their own constraints; cluster administrators can use LimitRange to set per-claim storage request ceilings and ResourceQuota to cap total namespace storage consumption. Kubernetes pods can be configured with resource requests and limits at both pod-level and container-level, providing fine-grained control over CPU and memory allocation.

At the node level, Kubernetes nodes report capacity and allocatable resources including CPU, memory, ephemeral storage, hugepages, and pod count limits. The kubelet `systemReserved` configuration reserves CPU cores and memory for system processes, protecting node stability from workload interference. For workloads requiring stronger latency and performance guarantees, kubelet resource management policies can be configured. Node overprovisioning capacity reservation is configured by defining resource requests and limits for placeholder pods in a Deployment manifest, with total reserved resources calculated as the product of replica count and per-pod resource quantities, keeping headroom available for newly scheduled workloads.

The Kubernetes kube-scheduler manages availability, performance, and capacity through scheduler policies and profiles. Kube-scheduler metrics can be used to build capacity planning dashboards, assess current or historical scheduling limits, and identify workloads that cannot be scheduled due to resource shortfalls. Kubernetes Dynamic Resource Allocation allows consumable device capacity to be requested via ResourceClaimTemplate objects, with requestPolicy constraints controlling how those capacities are consumed across allocations; device capacity can define a default request policy value and valid range with step increments. The Supervisor Monitor tab under vSAN provides a Capacity view for tracking how much capacity stateful service objects are consuming.

VM classes in VKS support two reservation types: guaranteed, which fully reserves configured resources so that requests equal limits, and best effort, which does not hold resources in reserve. The choice of VM class reservation type affects how compute capacity is consumed under varying load conditions and is relevant to capacity planning for VM-based workloads on the platform. Kubernetes production cluster guidance recommends maintaining a capacity expansion plan based on expected pod and container counts, with additional capacity provisioned to allow critical workloads to reschedule in the event of node failures.

VMware Private AI Services (PAIS) provides several mechanisms that support capacity management for AI workloads. These mechanisms operate at multiple layers: organization quotas, region quotas, zone configurations, VM class and storage class reservations, infrastructure policies, and observability for capacity trending.

The foundation of capacity management in PAIS is the organization quota system. Organization region quotas in VCF Automation accept only finite CPU and memory limits and reservations; unlimited resources cannot be assigned. CPU and memory reservations cannot be over-provisioned, though CPU and memory limits and storage can be. VCF Automation creates a resource pool for the namespace in the clusters that correspond to the specified zones, applying the reservation and limit values to that resource pool so the platform enforces the quota at runtime.

Within organizations, capacity is allocated through region and zone quotas. Provider administrators use the VCF Automation Provider Management portal to configure a region and assign region quotas, with quotas based on the zones selected for one or multiple supervisors. For each zone, CPU and memory limits and reservations are specified along with reserved or non-reserved VM classes and storage classes; storage limits per storage class are configurable. Region quotas can include explicit

GPU resource counts by profile (for example, H100 and L40 vGPU profiles assigned per zone). The PAIS SupervisorNamespaceClassConfig captures these per-zone CPU limits, CPU reservations, memory limits, and memory reservations as a managed configuration object. When an organization quota spans multiple supervisors in the same region, a separate region quota is required for each supervisor, so administrators account for each physical boundary in their capacity plan.

VM class reservations provide another layer of capacity control designed specifically for AI workloads. Zone capacity is configured through reserved VM class instances, where each reservation specifies a VM class name and a count of reserved instances in the zone. VM class capacity is reserved in Supervisor zones to support resource availability for the workloads that run there. For workloads requiring GPU resources, VM classes with GPU reservations require capacity configuration using the `dcli` command, which sets reserved VM class instances with a count parameter per zone. Guaranteed VM classes are not permitted in a region quota with zero reservation, and guaranteed VM classes are not allowed in a quota unless CPU and memory reservations are provided to all zones, so resource availability commitments are tied to explicit capacity configuration. GPU-enabled workload domains receive license capacity allocation, while the management domain does not.

When AI catalog items are created in VCF Automation through the Private AI Services quickstart, an infrastructure policy is selected to define the quality and quantity of resources that the PAIS DSM database can consume from vSphere clusters; the policy also determines which storage policies and VM classes are available. A default storage policy can be selected to define storage placement and resources for the database. AI Kubernetes cluster deployments require selection of a storage class that defines storage placement and resources for the virtual machines, and the PAIS VKS control plane configuration requires specification of a VM class and storage class to represent the resources allocated to control plane nodes. PAIS model endpoints similarly use storage class policies that describe storage requirements and classes of service for the VMs created for the Kubernetes cluster nodes running the model. The containerd storage volume used by PAIS workloads is sized at 100 GiB and mounted at `/var/lib/containerd`, giving administrators a fixed local-image footprint to plan against.

PAIS also supports observability for capacity planning. A Prometheus server is deployed in the Private AI Services namespace with its own metrics database backed by a Persistent Volume; storage size for that database is inferred from the metrics retention property in the PAISConfiguration custom resource, allowing administrators to plan monitoring infrastructure capacity alongside AI workload capacity. The VCF Consumption CLI connects to the Supervisor endpoint and streams metrics from the PAIS controller pod, and PAIS metrics can be visualized in Grafana for monitoring and troubleshooting, including LLM completion API response times, inference engine tokens-per-second generation rate, and GPU compute and memory utilization. These signals feed the organizational capacity planning process that the control evaluates.

### **VMware vDefend (Contributes)**

VMware vDefend includes capacity and performance management capabilities specific to its security processing infrastructure through the Security Services Platform (SSP).

The SSP provides a Platform Host Capacity view that gives administrators visibility into resource utilization across the security processing cluster. This view tracks cluster data, unexported flow data, total flows, prioritized applications and groups, the average number of flows every five minutes, and the data collection status per cluster. The SSP also displays a time-series graph showing flow distribution over an hour and generates a warning notification when unexported flows affect the prioritized collection of applications and groups. SSP monitors which hosts are operating within capacity and which are capped when flows on the host exceed capacity. The system dashboard provides flow processing capacity analysis to display cluster sizing requirements.

When approaching capacity limits, vDefend provides actionable guidance: administrators can monitor specific metrics, add worker nodes, disable flow collection on certain hosts or clusters, or rebalance virtual machines across transport nodes to manage system performance. Resource reservation is enabled by default for SSP deployments, reserving the required CPU and memory at the resource pool level. The documentation advises against oversubscribing host CPU and memory, and recommends adjusting virtual machine allocations or migrating virtual machines when resources are insufficient.

SSP includes tools to manage license pool consumption for capacity reservations in mission-critical production sites and to set hard limits for development environments, supporting capacity planning across different environment tiers.

Host Transport Node Metrics monitoring helps detect capacity constraints within the Distributed Firewall processing layer, allowing administrators to optimize resource allocation and verify that security enforcement does not affect workload performance. DFW Resource Utilization Metrics provide a dedicated view for capacity planning and performance troubleshooting within the Distributed Firewall. For IDS/IPS deployments, administrators can assess current CPU and memory load on ESX hosts to identify hosts with unused capacity that can accommodate the IDPS engine without performance degradation.

Broader infrastructure capacity planning across compute, storage, and networking is handled by VCF Operations, which provides the full assess, optimize, and plan lifecycle for the overall VCF environment.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) contributes to capacity management through Infrastructure Policies, VM classes, IP pool sizing requirements, and per-database resource monitoring. These mechanisms give administrators the tools to plan for and constrain resource consumption within the database platform.

Infrastructure Policies are the primary capacity control mechanism in DSM. Administrators configure Infrastructure Policies to define and set limits on the specific compute, storage, and network resources that database workloads can consume from vSphere clusters. The DSM console displays all configured infrastructure policies, including their compute, storage, and networking resource definitions and the VM classes each policy permits, giving administrators a view of the resource boundaries placed on DSM-managed workloads.

VM classes provide the per-database resource allocation unit. Each VM class specifies the vCPUs and memory in GiBs assigned to provisioned database VMs. When a user provisions a database, they select from the VM classes defined in the Infrastructure Policy. Administrators can adjust vCPU and memory allocations for database VMs at runtime without reprovisioning the database, supporting capacity adjustments as workload demands change. In DSM, enforcement of specific hardware allocations extends to SQL Server clusters to maintain their performance and stability within the containerized environment.

IP pool sizing is an explicit capacity planning requirement in DSM. IP pools must be sized to support the total number of database clusters expected to run simultaneously, plus reserve capacity for maintenance operations such as database engine upgrades and cluster topology changes, each of which temporarily requires additional IP addresses. DSM recommends monitoring IP pool utilization and expanding pool ranges or creating additional pools before capacity is reached. Global alerts are available to notify administrators when IP pool capacity approaches its limits.

The DSM Monitoring page provides visibility into database VM performance and resource consumption. Administrators can view metrics including Active Connections, Data Disk Usage, Write Throughput, Read Throughput, and Thread Resource Utilization for each database node VM. DSM also tracks system resource utilization metrics for CPU, memory, and data disk use through VCF Operations to help identify potential bottlenecks.

The Provider VM itself has sizing requirements tied to the volume of services it manages. Resources configured for a DSM Provider VM must be determined based on the type of environment in which it runs and the volume of services it manages, requiring upfront capacity planning when deploying the DSM management plane.

When DSM is deployed with VCF Automation as a provider, the Supervisor cluster must have sufficient CPU, memory, and storage resources available to schedule the DSM Consumption Operator pod. In this deployment model, infrastructure policies are created as cluster-scoped resources in the consumption cluster, and capacity planning for the Supervisor cluster is a prerequisite for successful scheduling.

Satisfying this control fully requires organizational processes for capacity forecasting and review that extend beyond what DSM provides at the platform level. DSM supplies the technical controls for constraining and monitoring database resource consumption; the broader organizational capacity management program, including demand forecasting, regular capacity reviews, and escalation processes, must be defined and operated by the organization.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer provides application-layer capacity management mechanisms across three areas: real-time capacity visibility through Pool and Service Engine metrics, proactive capacity management through elastic scaling and configurable resource controls, and multi-tenant capacity governance through the VCF organization-managed mode introduced in 9.1.

On the visibility side, the Pool Metrics system tracks both Estimated Capacity (the pool's maximum possible connections) and Available Capacity (currently open connections) at the pool level. Pool Metrics deliver specialized analytics and performance data representing the health and capacity of backend pool servers. At the individual server level, the `l4_server.avg_available_capacity` metric estimates connections-per-second capacity available for each server, calculated as the difference between maximum capacity and average load. The License Hub Platform Dashboard Resources tab visualizes hardware consumption overall and per controller and worker node, including System Load tracking of overall CPU demand against allocated vCPU capacity on a 15-minute average.

On the scaling and configuration side, each Service Engine has a maximum capacity for processing traffic measured in terms of traffic throughput or SSL transactions per second, which is a function of the SE VM size (number of vCPUs or memory), type of traffic, and deployment environment. CPU and memory reservations with limits set to unlimited are recommended for Service Engine virtual machines to achieve consistent and deterministic performance. Elastic HA N+M mode allows administrators to define a buffer expressed as M, an integer count of buffer Service Engines. When an N+M group is scaled out to the maximum number of Service Engines, Avi Load Balancer permits additional virtual service placements into the spare capacity represented by M. Pool configuration allows setting the maximum number of connections per server on the Advanced tab to help prevent servers from being overloaded and to maintain optimal connection handling.

In the 9.1 Avi for VCF integration, organization-managed mode adds Service Unit quota enforcement per organization. Provider administrators can assign either a quota measured in Service Units or specific Service Engine Groups to organizations through the VCF Automation Provider Management portal, establishing resource governance to help prevent organizations from over-consuming shared Avi Load Balancer resources. In organization-managed mode, the provider or Load Balancer administrator manages resource assignments to enforce quotas and resource boundaries, and organization administrators can edit a Service Engine Group to scale the underlying performance (CPU and memory) or modify high-availability behaviors. Service Engine capacity provisioned on the Avi Controller is periodically reported, and the license quota reservation mechanism can be configured to keep a specific amount of capacity always available for a tier when a consumption ceiling is set.

These mechanisms give operations teams the metrics and controls needed to assess current application delivery capacity, monitor utilization trends, and adjust Service Engine group configuration or scale out in response to demand. The organizational processes for defining capacity plans, setting review cadences, acting on threshold alerts, and making scale decisions remain the responsibility of the organization.

Not applicable for this control: VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 7.3

> Control system backup: Support the backup of configuration and user data. Verify backup integrity: Capability to verify the integrity of backups.

**SCF Controls:** BCD-11

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides built-in backup capabilities across its management components, workload layers, and storage subsystems, supporting organizations in meeting their Recovery Time Objectives (RTOs) and Recovery Point Objectives (RPOs).

#### Management component backups

VCF supports backup of all core management components. VMware vCenter supports both file-based and image-based backup methods, with file-based daily backups configured through the vCenter Management Interface of each vCenter instance. vCenter file-based backup and restore supports FTP, FTPS, HTTP, HTTPS, SFTP, NFS, or SMB as the backup server protocol. vCenter file-based backup optionally encrypts backup files using an encryption password that must be provided during restore; because vCenter backups contain the Key Derivation Key used for vSphere encryption, backup files should be stored on a secured backup server with appropriate access controls. VCF Operations similarly supports file-based and image-based backup methods. NSX Manager uses file-based backup, which VCF Operations configures automatically during the initial bring-up process. File-based backup jobs for VCF Operations and vCenter must be configured separately after deployment.

Scheduled backups should be implemented to prepare for critical failure scenarios, upgrade events, or certificate updates of management components. To capture a consistent state across related components, VCF requires creating a series of coordinated backup jobs that run simultaneously across component nodes. Simultaneous backups of component nodes help ensure that the backed-up state is logically consistent, reducing the need for integrity remediation during a restore. On-demand manual backups can be taken after a successful recovery operation or after a scheduled backup failure. VCF Operations backup is also configured to take a backup on state change, providing an additional layer of protection when configuration changes occur.

The backup file server stores backups from all NSX Manager or Global Manager nodes, and NSX includes a script to automatically delete old backups to manage storage consumption. NSX Global Manager backups should be periodically cleaned up to avoid excessive storage consumption on the backup file server. VCF Operations for Networks can store backups on local storage, retaining up to five backup files, or upload them to SSH or FTP servers. An initial on-demand backup should be performed after configuring VCF Operations to verify that backups complete successfully before relying on scheduled jobs. Sufficient storage capacity must be provisioned to accommodate backup data across all components.

#### Backup integrity and restore procedures

Backup integrity verification in VCF centers on version compatibility. When restoring a vCenter instance from a file-based backup, administrators must have a valid backup that matches the exact version of the vCenter appliance being restored to. VCF Operations must be available before restoring vCenter, and vCenter must be available before restoring VCF Operations, establishing a defined dependency order for restore operations. These dependencies require careful sequencing during recovery to restore the management plane to a consistent state.

#### Workload-level data protection

For workload-level data protection, VCF provides several mechanisms. Changed Block Tracking (CBT) tracks modified disk blocks on virtual machines, enabling incremental backups that reduce backup windows and storage requirements. VMware vSphere Storage APIs for Data Protection (VADP) provide a standardized interface for third-party backup solutions to integrate with VCF. vSphere Supervisor supports backup of VM Service VMs through backup partner solutions based on VADP, and if a VADP-based backup solution is deployed on the default management cluster, the solution must be properly started and operational before starting up or shutting down VCF. Quiesced snapshots are appropriate for automated or



periodic backups of VM Service virtual machines, as they capture a consistent state before the backup operation proceeds. VCF Automation supports snapshotting a VM group to create an object that references distinct snapshots for each member VM; restoring a VM group snapshot restores each member of the group.

vSphere Supervisor control plane backup and restore can be performed using vCenter file-based backup and restore from the Workload Management UI. For applications that provide their own backup capabilities, VCF documentation recommends using application-specific backup methods rather than relying on filesystem backup alone.

#### vSAN data protection and snapshots

vSAN Data Protection allows snapshots to be managed through protection groups with configurable snapshot retention policies and scheduling intervals. The native Snapshot Service is supported by vSAN Express Storage Architecture (ESA) but not by vSAN Original Storage Architecture (OSA). Native snapshots in vSAN ESA are much faster than traditional vSphere snapshots because a base VMDK and its snapshots are contained in one vSAN object rather than being separate objects, allowing more frequent backups and faster recovery. In vSAN OSA, snapshots are separate vSAN objects. The vSAN Distributed File System Snapshot can be used by third-party backup vendors to copy changed files incrementally to a different physical location. The vSAN Snapshot Service API provides programmatic access for operations including creating linked clones from snapshots and restoring deleted virtual machines.

#### VMware Kubernetes Service backup

VMware Kubernetes Service (VKS) and the Supervisor layer extend VCF's backup capabilities to containerized workloads, cluster state, and persistent volume data. VKS Supervisor backup and restore for cluster node VMs is activated through the vCenter backup feature available in the vCenter Management Interface, integrating Kubernetes cluster state backup with the same management-plane backup infrastructure used for other VCF components.

For workload-level backup of VKS clusters, the platform supports three complementary methods: vSphere Plugin Snapshot (CSI Snapshot), File System Backup (FSB), and the Velero Plugin for vSphere. The vSphere Plugin Snapshot method produces crash-consistent, block-level backups of persistent volumes through the Container Storage Interface (CSI) layer and supports snapshot direct access and network traffic offload; it is faster and less resource-intensive than filesystem backup for complex filesystems with large numbers of small files, making it well-suited for production databases and high-IOPS applications. The File System Backup method operates at the filesystem level, supports NFS volumes and incremental backups where unchanged files are not re-transferred between runs, and offers backup repository features including deduplication, compression, and encryption; it is appropriate for legacy volumes with non-CSI storage. The Velero Plugin for vSphere can back up and restore both stateless and stateful applications running on vSphere Pods in the Supervisor. Platform teams can use both FSB and CSI Volume Snapshots within a single protection strategy to maximize data durability across workload types. VKS cluster backup supports opt-in and opt-out approaches for FSB volume backup, giving operators control over which volumes are included in filesystem-level operations. Third-party backup software can also integrate with VKS clusters using the standard Kubernetes CSI snapshot API: tools create a VolumeSnapshot object from the target PVC, materialize a temporary PVC from the snapshot, and use a Data Mover pod to transfer data to the backup destination. Kubernetes implements Persistent Volume Claim as Snapshot Source Protection, which blocks deletion of in-use API objects while a snapshot is in progress, reducing the risk of partial or inconsistent backup data. Kubernetes also supports provisioning a new PersistentVolumeClaim directly from a VolumeSnapshot by specifying the snapshot as the dataSource, enabling volume-level point-in-time recovery without requiring a full application restore workflow.

VKS cluster management provides a Create backup wizard that allows users to create backups immediately or schedule backups of specified data resources in a cluster. Backup scheduling supports immediate execution, recurring schedule formats with targeted start times, or custom cron expressions; automated backup scheduling can also be configured using the Velero CLI with the `velero schedule create` command, specifying the interval or cron expression for recurring backup operations. For on-demand backups outside a scheduled window, the `velero backup create` command backs up Kubernetes resources and persistent volume data for stateful workloads, with support for per-namespace targeting and volume snapshot inclusion. VKS cluster backups can retain a maximum of 720 backups per backup schedule; when high-frequency schedules are required, the retention period should be reduced to remain within this limit. Backup schedules can be edited, paused, or deleted through

the management interface. File system backups are included in the restore by default, and snapshot data can be moved to a separate backup storage location. Before defining a backup, a target location and credential must be created; if a data protection credential is deleted, scheduled backups relying on that credential will fail on their next execution, creating a detectable gap in backup execution history that administrators should monitor. Creating a backup schedule requires the Organization Administrator or Project Administrator role.

VKS cluster backup and restore operations are managed through the VCF Automation Tenant Portal under Manage & Govern > VCF Services > Kubernetes Management > Clusters > Data Protection. The portal provides restore operations on specific backups, and a dedicated cross-cluster restore capability in the Data Protection tab enables restoring backups from a source cluster to a different target cluster. VKS cluster management supports backup and restore of an entire cluster, selected namespaces from the backup, or specific resources identified by a label. The CloneFromSnapshot CRD includes a status.phase field that tracks restore progress through multiple phases, providing operational visibility during restore operations.

For Kubernetes cluster state, etcd is the backing store for all Kubernetes objects, and clusters using etcd should have a backup plan. etcd backup can be performed using the built-in etcdctl snapshot save command by specifying the endpoint, CA certificate, and client credentials, or by taking a volume snapshot of the etcd storage volume when the underlying storage supports it. etcd snapshots can be restored from a previous snapshot file or from the remaining data directory, and cross-patch-version restores are supported. During kubeadm upgrades, automatic backup folders are written to /etc/kubernetes/tmp containing etcd data and manifests as a recovery point.

#### Backup automation through APIs

VCF exposes backup automation through its APIs, including the RecoveryBackupJobBackupRequest data structure for specifying backup location, protocol, authorization, and database scope, as well as the RecoveryBackupSchedulesUpdateSpec for programmatically managing backup schedules.

#### Disaster recovery planning

For disaster recovery planning, VCF supports application disaster recovery with data replication to a secondary site. Organizations should plan and test disaster recovery strategies regularly to meet their RTO and RPO targets. VCF Operations provides integration with VMware Live Recovery for periodic test runs to validate that existing recovery plans work with the underlying infrastructure and configured RPO limits. Application disaster recovery may result in some data loss during failover depending on the configured RPO.

VCF directly provides the mechanisms for creating recurring backups, managing retention, and supporting RTO/RPO requirements through multiple backup and snapshot methods. However, organizations must establish backup policies, define RTO/RPO targets, schedule regular restore testing, and maintain backup infrastructure such as external storage and third-party backup solutions to fully satisfy this control.

#### VMware vDefend (Contributes)

VMware vDefend's Security Services Platform (SSP) provides backup and restore capabilities for the vDefend control plane, covering firewall rules, microsegmentation policies, IDS/IPS configurations, Bare Metal server information, tokens, and all associated security data managed by the platform. These backups protect against data loss from hardware or software malfunctions, data corruption, malware incidents, or unintentional deletion.

SSP supports recurring backup schedules configurable through the Backup Schedule interface, with frequency options ranging from 6 hours to 720 hours (30 days), as well as weekly scheduling by selecting the day and time. Recurring backups can be activated or deactivated by toggling the Recurring setting. Best practice guidance recommends scheduling backups during periods of low platform usage to minimize operational impact. A backup should also be performed before executing scale-out operations on the SSP cluster, to preserve the current configuration state before significant topology changes. Backup data is encrypted with a passphrase and transmitted to an SFTP server. Adequate storage must be verified on the backup server before activating recurring backups, and the platform must be in a healthy state for backup operations to proceed. Backup is also blocked if NSX is not onboarded to SSP.

SSP maintains a Backup History section that displays the status of each backup operation, showing "Completed" when the operation succeeds. When backup or restore operations enter a failed state, pod logs provide detailed error messages, including database query failures and volume attachment errors, to support troubleshooting. Restoration supports deploying the backed-up configuration onto the same SSP instance or a new instance, enabling recovery when the current instance is in a non-recoverable state. Restoration requires that the same NSX instance that was onboarded at the time of backup be used, and that the platform be deployed on the same vSAN datastore and build version as the initial installation. Certificate consistency between backup and restored environments must be maintained to avoid disruptions to NDR Sensor connectivity.

These capabilities address the backup component of this control for vDefend's own configuration and policy data. vDefend does not back up general workload data or application-level information; those responsibilities fall to the broader VCF platform's replication, snapshot, and backup mechanisms, including vSAN Data Protection, Protection and Recovery (PNR), and integration with third-party backup providers. vDefend's contribution is limited to protecting the security policy and configuration data that defines the network security posture of the environment.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) provides replication, snapshot, and recovery testing capabilities that support recovery point and recovery time objectives for protected workloads.

vSphere Replication delivers host-based continuous replication with RPO configurable from 1 minute to 24 hours, set via a slider during replication configuration. PNR uses the configured RPO to determine the most recent available state to which virtual machines are recovered during disaster recovery. vSAN snapshots and replication is a native, integrated snapshot-based backup solution for VMs running on vSAN datastores, managed through the vSphere Client, and supports RPO as low as 1 minute. Snapshot retention schedules support hourly, daily, weekly, and monthly snapshots with a maximum retention duration of three years. Predefined retention templates are available for quick configuration of RPO and snapshot retention parameters. Array-based replication supports configuring replication schedule, frequency, and RPO through VM storage policies, depending on the storage array's implementation.

PNR replication classes are preconfigured with an RPO that defines the acceptable amount of data loss after a disaster. In deployments using VCF Automation integration, replication classes can be assigned at the organization level.

Cyber Recovery, the cloud-based recovery component within PNR, uses incremental snapshots that transfer only modified or new data to cloud backup. This design enables detection of ransomware encryption by comparing entropy rates between normal snapshots and snapshots containing suspect data. Ransomware recovery plans in Cyber Recovery support predefined retention templates for configuring snapshot RPO and retention parameters.

PNR's own configuration is protected through backup settings that specify a datastore path, enable or disable the backup scheduler, set a retention period, and schedule backup frequency and time. PNR REST APIs can be configured to export and upload configuration data to a datastore on a daily basis. PNR also maintains internal database backups for audit and recovery purposes, covering the replication manager database (srmdb), recovery manager database (vrmsdb), and snapshot service database (snapservice).

Recovery plan testing is the integrity verification mechanism for protected data. During disaster recovery execution, PNR first attempts storage synchronization; if that succeeds, it uses the synchronized storage state to recover virtual machines to their most recent available state per the configured RPO. RPO violation dashboards in VCF Operations report violation counts for incoming and outgoing replications, enabling monitoring of whether replication is meeting defined targets.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides a built-in backup framework for managed PostgreSQL, MySQL, and SQL Server databases, giving administrators and database owners the tools to establish recurring backup schedules aligned with organizational recovery objectives.

Backup configuration is available at database creation and can be modified at any time through the database settings interface. Administrators activate automated backups using the Enable Automated Backups toggle (also exposed as Enable Backups in MySQL flows), specify a backup location pointing to S3-compatible object storage, and define a retention period.

PostgreSQL databases support both full and incremental backups, and SQL Server databases support full and differential backup schedules. SQL Server adds a Log Backup Frequency setting in Advanced Settings with a maximum value of 30 minutes that is directly related to the Recovery Point Objective for that database. Backup schedules are expressed as cron expressions and can be customized per database or applied at the policy level.

Data Service Policies provide centralized backup governance for databases deployed within a namespace. A policy can include named backup jobs specifying backup type and schedule, and the policy backup requirement setting offers three options: Require Automated Backups (databases must have automated backups), Allow Automated Backups (users can choose to activate or deactivate automated backups), and Disallow. Allowed backup locations are defined in the policy via the `allowedBackupLocations` parameter, restricting where backups can be stored. When multiple policies apply, DSM aggregates the backup schedules into a single effective backup policy per database. High Availability clusters require automated backups to be enabled in order to provision databases, helping ensure that recoverable copies exist for replicated topologies.

DSM also supports on-demand backups for primary PostgreSQL databases, independent of the configured automated schedule, which is useful for capturing a point-in-time snapshot before a significant change. Backup storage is managed centrally through the Settings pane Storage Settings tab, which displays Database Backup Storage buckets and allows administrators to add or edit object storage targets. Provider-level control plane backups are stored in a designated Provider Backup Repo bucket configured during Provider VM setup, and recovery of the Provider VM from backup uses the `restore-provider` command with a `pgbackrest.conf` path to restore the appliance to the latest timestamp.

DSM retains automated backups of deleted databases until the configured retention period expires. When a database with configured backups is deleted, the system automatically creates an archive containing all collected backups that remains available in the Archives tab for the remainder of the retention period, allowing recovery of inadvertently deleted databases. Administrators can also manually delete retained backups of deleted databases when they are no longer needed.

Point-in-time recovery is supported from configured backup storage, restoring a backup at a specified date and time to a newly created database VM with a user-specified name and datastore location. The restore function is also accessible through the VCF Automation UI for tenant users. If any incremental backup in a chain is deleted, DSM cannot restore data to points in time that depended on that backup, and a restore from an incremental chain requires the last full backup plus all following incremental backups. Maintaining a balanced schedule of full and incremental backups reduces the risk of a gap in the recovery chain.

DSM provides operational validation of the backup target through the backup storage repository alert, which checks connection and operational status, bucket name, certificate expiration date, credentials, and sufficient available space in the repository. This helps surface configuration drift or storage failures that would otherwise compromise recoverability.

Satisfying this control fully requires that the organization define RTOs and RPOs for each database workload, configure backup policies accordingly, and periodically test restoration to verify that backup data is valid and recovery objectives are achievable. DSM provides the technical backup infrastructure and operational checks on the backup target; RTO and RPO definition and end-to-end backup integrity testing are organizational responsibilities.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) includes a built-in backup and restore capability for the Avi Controller configuration database. The configuration database defines all clouds, virtual services, users, tenants, and related entities. Protecting it is the primary mechanism for recovering the Avi control plane after a Controller failure.

Backup operations can be created on-demand or on a recurring schedule via the Controller web UI, CLI (using the `export` command), or REST API (using the `/api/scheduler/` endpoint). The scheduler supports configurable frequency units, including hourly, daily, and weekly intervals, with the run mode set to periodic. For single-node Controller deployments, enabling periodic external configuration backups is mandatory; the product documentation states that external backups are required to handle recovery from a complete VM or disk failure.

Avi supports multiple backup storage targets. Local backups are stored directly on the Controller with a configurable retention count, configured via the "Enable Local Backups (On Controller)" setting. Remote backups are transferred over Secure Copy

Protocol (SCP) or Secure File Transfer Protocol (SFTP) to an off-Controller server, configured via the "Enable Remote Server Backup" setting. In 9.1, configuration backups can also be stored in Amazon S3 buckets as an additional remote storage target. When deployed as part of VCF, Avi backups are stored by default on the same server used for VCF Operations appliance backups; the recommended best practice is to configure an external SFTP server as the backup destination. VCF Operations can be used to configure the external backup server and storage path for Avi backups. Product guidance also recommends storing backups with clear labeling that includes version and date information.

Before a restore proceeds, Avi performs a set of pre-restore checks to validate the backup image. In 9.1, these checks include verification of backup configuration version compatibility, presence of all file objects from the configuration on the Controller, FIPS mode consistency between the backup and the current Controller environment, Service Engine attachment verification, system configuration validation, and patch verification. All checks must pass before the restore operation is allowed to proceed.

Restore operations can be performed via the CLI (import command with the `keep_uid` option to restore configuration to an empty Controller), the web UI, or the REST API restore endpoint. The API method accepts a configuration file and a passphrase parameter to decrypt an encrypted backup; the passphrase must match the one used when creating the backup. Restore operations generate system reports tracking operation state, start time, end time, duration, and total checks completed. The Controller metrics database can also be exported and imported separately to prevent loss of metrics data during version rollbacks. If all three nodes of a Controller cluster are permanently unavailable, recovery must be performed using a backup and restore procedure.

For multi-site deployments, Avi GSLB (Global Server Load Balancing) supports an Active disaster recovery deployment mode, enabling traffic redirection to a healthy site when a primary site fails. Avi GSLB adaptive replication mode also protects against configuration errors propagating across data centers by preventing faulty configuration objects from being replicated to follower sites.

The Avi backup and restore capability covers the Controller configuration plane. Application data, workload VM images, and the broader RTO/RPO framework depend on organizational process decisions and on the underlying VCF platform for VM-level data protection (see VCF Coverage).

## Control CR 7.3(1)

> Control system backup: Support the backup of configuration and user data. Verify backup integrity: Capability to verify the integrity of backups.

**SCF Controls:** BCD-11.1

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides backup and restore capabilities for its management components and workload layers, along with monitoring features that support organizations in verifying backup reliability. Routine backup testing itself requires organizational processes beyond what the platform automates.

VCF supports file-based and image-based backup methods for its core management components, including VMware vCenter, VCF Operations, and NSX Manager. VCF Operations automatically configures NSX Manager backup during the initial bring-up process, while file-based backup jobs for VCF Operations and vCenter must be configured separately. vCenter file-based daily backups are configured through the vCenter Management Interface of each vCenter instance. File-based backups can be stored on FTP, FTPS, HTTP, HTTPS, SFTP, NFS, or SMB servers, and the backup can optionally include Stats, Events, and Tasks data from the vCenter database to preserve historical audit records. Backup files support optional encryption with a password that must be provided again during the restore procedure. The platform recommends creating coordinated backup jobs that capture the state of related management components at a common point in time, so that restores can proceed from a consistent baseline. Scheduled backups should be implemented to prepare for critical failure scenarios, component upgrades, or certificate updates. vSphere Supervisor control plane backup and restore is activated through the vCenter backup feature in the vCenter Management Interface, which also protects VMware Kubernetes Service (VKS) cluster node VMs; this backup is accessible from the Workload Management UI. VKS cluster backup and restore operations for workloads and namespaces are managed through the VCF Automation Tenant Portal under Manage & Govern > VCF Services > Kubernetes Management > Clusters > Data Protection. This interface supports backup and restore of the entire cluster, selected namespaces from a backup, or specific resources identified by a label, and provides a "Restore from another cluster" function that allows a backup from a source cluster to be applied to a different target cluster. For Kubernetes workloads running on VKS, volume backup status and failures can be confirmed using the Velero CLI with the details flag.

Workload data on VKS clusters is protected through Velero integration. Velero supports two backup strategies: vSphere Plugin Snapshot, which captures persistent volumes at the block level and writes data directly to object storage without requiring a backup repository, and CSI Snapshot combined with File System Backup, which captures data at the file-system level with support for deduplication, compression, and encryption when those repository features are required. The CSI Snapshot and File System Backup methods also support incremental backup, where files that have not changed since the last backup are not re-uploaded. The Velero Plugin for vSphere backs up and restores both stateless and stateful applications running on vSphere Pods in the Supervisor. Third-party backup software can integrate with the CSI snapshot API to perform volume backup and restore operations against VKS clusters. Velero supports scheduled backup operations, enabling organizations to automate regular backup cadences as a foundation for routine testing.

Velero provides inspection capabilities that support backup verification. The `velero backup describe` command with the details flag reports completed and failed Restic volume backup entries along with source file references and pod identifiers, allowing administrators to diagnose backup failures before depending on a given backup for recovery. VKS cluster backup logs record backup failures with detailed error messages that include the process ID, user ID, group ID, and timestamp of the lock holder when a backup cannot proceed due to an existing exclusive lock. Scheduled backups relying on a deleted data protection account credential will fail on their next execution, and this failure creates a detectable gap in backup execution history visible in the backup monitoring interfaces.

For Kubernetes cluster state, etcd serves as the backing store for all cluster configuration and API objects. etcd supports two backup methods: built-in snapshot via the `etcdctl snapshot save` command (specifying the endpoint, CA certificate, client certificate, and client key) and volume snapshots. Snapshots can be restored from a previous backup file or from a remaining data directory, with cross-patch-version restore supported for the same major.minor etcd version. During kubeadm upgrades,



VKS automatically writes time-stamped backup folders for etcd data and control plane manifests to `/etc/kubernetes/tmp`, providing a rollback point that can be exercised as part of upgrade validation. Kubernetes guidance recommends that clusters verify backup restorability at regular intervals.

The platform includes built-in integrity verification as part of the restore process. Before restoring a vCenter instance from a file-based backup, the administrator must verify that the vCenter version retrieved from VCF Operations matches the version associated with the backup file. The backup must be of the same version as the vCenter appliance on which the restore will be performed. These version-matching checks help confirm that the backup data is consistent and compatible before a restore operation proceeds. Management component restore operations for VCF Operations, vCenter, and NSX can be tested by performing actual restores to validate backup data integrity.

VCF Operations provides monitoring capabilities that track backup success and failure across the environment. Backup failures are counted over time and grouped by error code, giving administrators visibility into recurring issues. Successful and failed inventory backups are also monitored. NSX tracks the total number of times backup configuration was updated, as well as backup successes and failures for both cluster backups and inventory backups. Alert notifications can be configured for backup failures under Platform Health Alerts, and administrators can track backup and restore process failures on the Infrastructure page under Settings, Infrastructure and Support, Overall Health section. NSX generates a BackupFailure alert when a scheduled backup operation fails. After resolving errors in VCF components, administrators can take on-demand manual backups to verify that the backup process is functioning correctly.

The VCF documentation recommends performing an initial on-demand backup after configuring VCF Operations to verify that backups complete successfully. This practice of performing a test backup after configuration is a form of validation, but it is a one-time recommendation rather than an automated recurring test.

For workload-level data protection, vSAN Data Protection allows snapshots to be managed through protection groups with configurable retention policies (duration and time units) and scheduling intervals. Native snapshots in vSAN Express Storage Architecture (ESA) use a B-tree structure instead of a chain structure, allowing more frequent backups and faster recovery than traditional vSphere snapshots. The native Snapshot Service is supported by vSAN ESA but not by vSAN Original Storage Architecture (OSA). Snapshot restore operations can be tested to validate data integrity.

VMware vSphere Storage APIs for Data Protection (VADP) provide a standardized interface for third-party backup solutions. VADP-based backup partner solutions can automatically perform backup, full restore, and registration of VM Service VMs on a Supervisor. These partner solutions typically include their own backup verification and integrity checking capabilities.

The vCenter API provides the `RecoveryBackupJobBackupRequest` data structure for programmatic backup operations, specifying backup location, protocol, and authorization details. The `com.vmware.appliance.recovery.backup.validate` appliance API command validates backup job parameters without starting the job, allowing administrators to confirm that backup configuration is correct before execution. This API access enables organizations to build automated backup testing workflows that trigger backups and verify completion status programmatically.

VCF directly provides backup scheduling, backup monitoring with failure alerting, on-demand backup testing, version-matching integrity checks, restore capabilities, workload-level snapshot management, and VKS cluster backup and restore through the VCF Automation Tenant Portal. Establishing a routine cadence for testing backup reliability and validating restore procedures requires organizational processes, runbooks, and scheduling that go beyond the platform's built-in automation.

### **VMware vDefend (Contributes)**

VMware vDefend Security Services Platform (SSP) provides backup and restore capabilities for the security platform itself, offering several mechanisms that support routine verification of the backup process.

SSP includes a configurable backup schedule that supports recurring backups, which can be toggled on or off. Backup frequency can be set as weekly on a selected day and time, or as an interval-based schedule ranging from a minimum of 6 hours to a maximum of 720 hours. The SSP Backup History section displays backup operation status, showing Completed when a backup operation is successful, giving operators direct visibility into whether scheduled backups are completing. When a scheduled backup fails, SSP generates a Scheduled Backup Error alarm to notify operators of the failure.



Backup and restore operations generate pod logs that record detailed error messages, including database query failures and volume attachment errors, supporting diagnosis of backup failures. If an operation becomes stuck in a FAILED or QUEUED state, operators can inspect cluster-api pods and backup pod status to troubleshoot the condition.

The SSP Installer data backup produces duplicate platform configuration data that can be restored in the event of hardware malfunctions, software malfunctions, data corruption, malware incidents, or unintentional data deletion. Restore operations require adequate storage on the target server, the same NSX instance that was onboarded at the time of backup, and the same vSAN datastore and build version as the initial installation. Certificate consistency between backup and restored SSP environments is required to avoid service disruptions, including disruptions to NDR Sensors.

vDefend's contribution is to the operational health and backup completion visibility of the security platform itself. The control asks whether the organization routinely tests backups for reliability and integrity; vDefend provides the scheduling, status tracking, and alerting mechanisms that support such testing, but automated restoration testing and integrity validation of backup content require supplemental organizational procedures.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) provides non-disruptive testing capabilities that exercise the recovery process and verify the integrity and availability of replicated data at the recovery site. The cadence and scope of routine testing remain an organizational practice.

Recovery plan tests run virtual machines on a test network and against a temporary snapshot of replicated data at the recovery site, isolating test execution from production data and from ongoing operations at either site. Tests can be run as often as needed and cancelled at any time without impacting production systems. Testing exercises nearly every aspect of a recovery plan to validate disaster recovery readiness. PNR documentation recommends testing recovery plans before use for planned migration or disaster recovery to validate functionality before production execution, as failure to test may result in data loss during an actual disaster recovery. After a reprotect operation completes, running a test recovery verifies that errors are resolved before running a planned migration or disaster recovery. After testing completes, the cleanup capability removes test artifacts so they do not persist in the recovery environment.

Recovery plan testing can be configured to replicate recent changes to the recovery site so that the test exercises the latest copy of protected virtual machines, though this may increase synchronization time. PNR can generate and export reports about each run of a recovery plan, test of a recovery plan, or test cleanup for audit trail purposes, providing a record of testing activity that administrators can reference.

PNR monitors protection group and recovery plan health, detecting critical issues including loss of connectivity to protected or destination sites, snapshot replication failure, and recovery plan malfunction. The Protection and Recovery management pack integrates with VCF Operations to enable data collection from PNR Cloud instances and provide centralized dashboarding of recovery telemetry.

Traditional backup testing capabilities, including platform component backup validation and VADP-based third-party backup verification, are provided by the underlying VCF platform (see VCF Coverage). Establishing a routine test schedule, defining acceptance criteria, and recording test outcomes for audit are organizational responsibilities that sit outside PNR's automated scope.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides automated backup capabilities, backup health monitoring, integrity-verification mechanisms, and on-demand restore tooling that support an organization's routine backup verification practices. A formal backup testing program itself requires organizational process.

DSM supports automated backups for databases with configurable backup schedules and retention periods. PostgreSQL databases use configurable settings for backup location, retention period, and backup schedule, controlled by the Enable Automated Backups setting during database creation or through database settings afterward. DSM data service policies can be configured to require automated backups, allow them as optional, or disallow them, giving administrators a way to enforce backup coverage on databases under a policy. High Availability clusters require automated backups to be enabled before

databases can be provisioned. Backup chain integrity depends on retaining the last full backup and all subsequent incrementals. If any incremental in the chain is deleted, DSM cannot restore the full data, which is a consideration when designing retention policy and backup chain management procedures.

Backup health is actively monitored. DSM validates the connection and operational status of the backup storage repository, verifies the bucket name, checks credential expiration, confirms certificate expiration dates, and validates that there is sufficient available repository space. Scheduled backup failure alerts notify administrators when a scheduled backup operation fails, and the monitoring framework supports metrics-based and status-condition alerts that identify critical failure conditions including database engine degradation, non-operational status, and automated or transaction log backup failure. Webhook payloads triggered by data service operation failures can be validated using the TEST function before deployment so that failure notifications reach the intended target. A trusted root certificate must be configured for each backup location used by a data service policy that requires or allows automated backups.

For data integrity verification, DSM PostgreSQL distributions include the amcheck extension at versions 1.2, 1.3, and 1.4 across the supported PostgreSQL builds, which provides functions for verifying relation integrity directly inside the database. Administrators can use amcheck to spot-check that a restored database's internal structures are sound after a recovery exercise, giving the testing program a database-engine-level data integrity check.

Administrators have several mechanisms to verify recoverability. DSM supports point-in-time recovery (PITR) that restores a backup at a specific point in time to a new database instance, with restoration possible to any point within the configured backup retention period. The restore creates a new database VM with a user-specified name and datastore location, so the test restore does not affect the source database. Deleted database backups remain available and can be restored to a newly created database VM at the most recent timestamp or to a specific point in time, supporting test restores of historic state. On-demand backups can be triggered manually for primary PostgreSQL databases to capture a known-good state before a planned change or verification exercise. DSM Provider VM recovery from backup is supported through the restore-provider command with the pgbackrest.conf path, which restores the Provider VM to the latest timestamp after a terminal failure during non-upgrade operations. The Provider VM requires a Provider Backup Repo to be configured before recovery is possible. Tenants can also restore a database to a specified restore point through the VCF Automation UI when DSM is consumed through VCF Automation.

While DSM provides the backup infrastructure, monitoring, validation, integrity-verification, and restore capabilities that form the technical foundation of a backup testing program, the organization must conduct scheduled restore tests against non-production environments to validate actual data recoverability, document the results, and maintain a formal testing cadence. DSM does not automate test restores to a separate environment, so these organizational procedures are required to fully satisfy this control.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer provides backup and restore capabilities for the Avi Controller configuration, including automated scheduling, multiple storage destinations, pre-restore integrity validation, and post-restore verification.

The Controller backup scheduler automates periodic backup operations. Configuration data can be written to local storage on the Controller with a configurable retention count, or to a remote backup destination. Configuring a remote backup destination is recommended to protect against scenarios where the Controller cluster fails in a non-recoverable fashion. In VCF-integrated deployments, VCF Operations is used to configure the external backup server and storage path; by default, Avi Load Balancer backups are stored on the same server where VCF Operations appliance backups are stored. For single-node Controller deployments and when recovering from a complete VM or disk failure, enabling periodic external configuration backups is mandatory. Administrators can use the `show backups` CLI command to list available backup files, and the `export` command to capture a full system configuration or individual virtual service configurations. A backup must be verified as taken prior to initiating any upgrade operation. After upgrades, backups should be stored with clear labeling that includes version and date.

Restore operations can be initiated through the CLI using the `import` command with the `keep_uuid` option to restore to an empty Controller, or through the REST API by attaching a configuration backup file and providing the decryption passphrase.

Before any restore proceeds, the Avi Controller performs pre-restore validation checks that verify Controller Cluster readiness, upgrade operation status, ServiceEngineGroup upgrade status, backup configuration version compatibility, FIPS mode alignment between the backup and the target environment, the presence of required file objects, and configuration validity. These checks are logged with timestamps, including FIPS configuration status and image validation state. In VCF-integrated environments, the restored Controller instance must remain synchronized with the VCF Operations lifecycle.

After a restore completes, the Controller performs post-restore validation tasks that verify the successful restoration of configuration objects, including counts of restored Clouds and ServiceEngineGroups. This confirms that backup data has been correctly applied before the system returns to service.

Organizations should establish a defined process to periodically execute test restores, verify that recovered configurations are complete, and record results as evidence of backup program effectiveness.

## Control CR 7.4

> Control system recovery and reconstitution: Capability to recover to a known secure state.

**SCF Controls:** BCD-12

**Aggregate Coverage:** Meets

### VCF (Contributes)

VCF provides built-in capabilities for secure recovery and reconstitution of infrastructure and workloads to a known state after a disruption, compromise, or failure. These capabilities span compute-level high availability, storage-level data protection, structured backup and restore workflows for management components, and disaster recovery across sites.

At the compute layer, vSphere High Availability (HA) automatically restarts virtual machines on surviving hosts following a physical machine failure, restoring workloads to their prior operational state. vSphere Fault Tolerance provides continuous availability for individual workloads in the event of a hardware failure with no data loss or downtime. Proactive HA supports maintenance mode as a remediation method, allowing the platform to respond to hardware degradation indicators. On TPM-equipped hosts, VMware ESX supports a Secure ESX Configuration recovery key that is required to restore the host configuration following TPM failure, TPM reset, motherboard replacement, or TPM device replacement; administrators can retrieve and store this key using `esxcli system settings encryption recovery list` and should keep it in a secure, off-host location.

At the storage layer, vSAN fault domains distribute data according to storage policy so that it remains available even when hardware failures occur. vSAN stretched clusters extend a cluster across two availability zones, providing site-level resiliency against the loss of an entire availability zone. When one site becomes unavailable, the surviving site continues operations, and with local fault protection enabled, the cluster can perform repairs on missing or broken components in the available site. vSAN Data Protection uses native snapshots stored locally on a vSAN Express Storage Architecture (ESA) cluster to support point-in-time recovery from operational failures or ransomware attacks. Changed Block Tracking (CBT) tracks changes on disk sectors, enabling incremental backups through VADP-integrated partner backup solutions.

For large-scale disruptions, Application Disaster Recovery enables failover to a secondary site, with data replication so recovery options exist in the event of catastrophic failure. Disaster recovery can also be implemented for fleet-level components, covering the management plane itself. VCF documentation recommends planning and testing disaster recovery strategies regularly to validate that recovery time objectives (RTO) and recovery point objectives (RPO) can be met.

VCF includes structured backup and restore workflows for its management components. Administrators create a series of backup jobs that capture the state of related components, including VCF Operations, NSX Manager, and VMware vCenter, at a common point in time. VCF Operations backup is configured to take a backup on state change. VMware vCenter supports both file-based and image-based backup methods. The file-based restore procedure is a two-stage operation: stage 1 deploys a new vCenter appliance, and stage 2 transfers data from the backup location to that appliance; the restored appliance operates in non-HA mode, so vCenter High Availability must be reconfigured after recovery completes. Simultaneous backups of VCF Operations fleet management component nodes support restoring the nodes to a logically consistent state, so further integrity remediation is not required after a restore. The platform also recommends backing up the management domain vCenter and VCF Identity Broker appliance following any substantial modifications to the identity source configuration, such as a VCF SSO reset, to preserve a known-good recovery point.

For VMware vCenter specifically, the vCenter Management API provides a unified restore process that handles multiple vCenter components in a single operation. After a file-based or image-based restore, the reconciliation API checks whether vCenter partners in embedded linked mode are available, synchronizes vCenter data and services with the partners, and runs vCenter services. The appliance recovery API exposes a restore client interface for programmatic restore operations, supporting automation of recovery workflows.

VCF Operations supports image-based backup and restore as a recovery process for single node, primary node, replica node, data node, and collector failures. VCF Operations for Networks supports backup and restore of its deployment data to enable recovery when the existing setup is unrecoverable. VCF Operations HCX provides backup and restore operations through its

appliance management interface, and its Mobility Groups support workload migration with state tracking, retry capabilities, and reverse migration from destination back to the original source.

The Consumption layer of VCF, which encompasses VMware Kubernetes Service (VKS), vSphere namespaces, and VCF Services consumption, extends recovery and reconstitution capabilities to containerized workloads and self-service deployments. At the Kubernetes control-plane level, ReplicaSets and Deployment controllers continuously monitor pod health and reschedule workloads to healthy nodes when pods or nodes fail. Liveness probes detect application-level failures and trigger container restarts; readiness probes gate traffic until a workload recovers from transient faults or overloads. These self-healing mechanisms operate without manual intervention for transient failures. Pod Disruption Budgets allow administrators to define minimum availability constraints that the platform respects during voluntary disruptive actions such as node upgrades or certificate rotations, so those operations proceed without taking down an entire service.

VKS clusters support structured backup and restore through VKS Cluster Management and Velero-based mechanisms. VKS Cluster Management backup operations can capture an entire cluster or a subset of data resources, and restore operations allow the operator to specify the scope of resources to recover. Restore operations include file system backups by default; CSI volume snapshots can be optionally selected for restoration. CSI Volume Snapshots deliver crash-consistent backups with support for block mode and file system mode, incremental backups, deduplication, compression, and encryption, and provide a resilient safety net for VKS clusters when primary storage infrastructure fails. CloneFromSnapshot custom resources expose a status.phase field that tracks restore progress through multiple phases, giving operators visibility into the state of ongoing recovery operations. The Velero plugin for vSphere supports backup and restore of vSphere pods, and VADP-based backup partner solutions can perform backup, full restore, and registration of VM Service VMs on a Supervisor.

Deploying workloads across multiple failure zones using the Cluster API defines worker nodes across multiple failure domains with automatic healing rules that reschedule workloads when a zone becomes unavailable. Replicating control plane components across at least three failure zones limits the impact of a single-zone failure on the Kubernetes control plane itself. For VM Service workloads, the IaaS Services Console and Local Consumption Interface provide a Resources tab where users can create snapshots with optional memory capture and guest file system quiescence, and can revert a VM to a prior snapshot to restore it to a known state.

Deployment rollback is available natively in Kubernetes through Deployments with revision history, allowing administrators to return to a previous known-good revision when a newly deployed version fails. The Harness CI/CD Supervisor Service extends this with automated rollback capabilities that can revert to a previous stable state when deployment failures are detected.

Immutable infrastructure patterns supported by Kubernetes, combined with infrastructure-as-code stored in version control, provide a reproducible mechanism for reconstituting a cluster to a known, authorized state. Every change to the declared state is recorded in version control, and the cluster can be rebuilt from that record after a compromise or failure. VKS Cluster Management Policy Insights expose Sync signals that indicate when policy definitions fail to propagate from the management plane to the targeted VKS cluster estate, helping operators detect and remediate policy drift before it affects recovery posture.

Restore operations require that the target infrastructure be in place before a restore can proceed. VCF documentation recommends implementing a consistent backup strategy to safeguard the infrastructure against unforeseen disruptions and to support effective disaster recovery.

#### **VMware vDefend (Contributes)**

VMware vDefend contributes to secure recovery and reconstitution through backup and restore capabilities for its own security infrastructure, and through disaster recovery features that maintain security policy continuity across site failover events.

The Security Services Platform provides backup and restore functionality, accessible through the System > Backup and Restore interface, that allows recovery of a platform instance to its previous configuration when the current instance is in a non-recoverable state. The backup scope includes the full system inventory: tokens, Bare Metal servers, and Bare Metal server interfaces are captured as part of inventory backup, and required certificates from the backup server are restored automatically during the restore process. The platform must be in a healthy state to perform backup operations; backup and

restore cannot proceed when the platform is unhealthy. Certificate consistency between backup and restored Security Services Platform environments must be maintained to avoid NDR Sensor disruptions. Backup and restore operations enter a failed state after 30 minutes if internal services are restarted manually or due to failure. Administrators should back up the Security Services Platform before performing scale-out operations, and Bare Metal Service operations (onboarding, offboarding, or upgrading Bare Metal servers) should not be performed while backup or restore is in progress.

The Security Services Platform can be deployed on vSAN stretched configurations, which supports recovery from site failures and reduces recovery time at the infrastructure level.

For disaster recovery scenarios, NSX Federation maintains security posture during workload migration and site failover by propagating Distributed Firewall rules, security groups, and microsegmentation policies across multiple sites. When workloads fail over from a protected site to a recovery site, the vDefend security policies governing east-west traffic at the destination site are already in place. vSphere HA can be configured to handle host-level failures affecting Security Services Platform appliances.

The broader recovery of compute infrastructure, storage, virtual machines, and application workloads to a known state is handled by other VCF components such as Protection and Recovery, vSAN Data Protection, vSphere Replication, and vCenter backup and restore.

### **VCF Protection and Recovery (Meets)**

VCF Protection and Recovery (PNR) provides mechanisms for secure recovery and reconstitution of virtual machine workloads to a known state after a disruption, compromise, or failure.

Recovery is organized through protection groups and recovery plans. Protection groups define which virtual machines are protected, while recovery plans orchestrate the sequence of steps required to recover those workloads at the recovery site. The state of a recovery plan is determined by the states of the protection groups within it, and the plan state governs which operations are permitted at each stage of the DR lifecycle. Recovery plans support two primary modes: planned migration for orderly, scheduled failover, and disaster recovery for unplanned site failures. A single recovery plan can also be configured to serve both disaster recovery and cyber recovery operations, allowing organizations to maintain consistent recovery procedures across different incident types.

Recovery plan testing is a built-in capability. Recovery plans should be tested before use for planned migration or disaster recovery to validate their functionality. This confirms that the recovery process will bring protected virtual machines online at the recovery site and that inventory mappings correctly place workloads on the appropriate networks, storage, and compute resources.

Cyber Recovery extends reconstitution capabilities with an Isolated Recovery Environment (IRE) designed for compromise scenarios such as ransomware incidents. The IRE provides a dedicated, isolated segment of the recovery environment that blocks direct private network connections from production, allowing compromised workloads to be analyzed and validated before being returned to production. Cyber Recovery transitions virtual machines through a controlled sequence of states, including Entering validated, Validated, Staging, and Staged, to prepare them for recovery. This state-based workflow supports data integrity validation prior to production restoration.

Recovery plan compliance checks verify that protection groups are actively replicating, that sufficient point-in-time snapshots are available for each virtual machine in the plan, and that all workflow steps can execute without conflict. Continuous compliance monitoring provides warnings when VM configuration status deviates from expected state.

The Configuration Import/Export Tool enables administrators to export PNR server configuration data, including inventory mappings, placeholder datastores, advanced settings, array manager configurations, and protection group definitions. This capability supports configuration backup and reconstitution of the PNR environment itself after a failure or migration.

PNR integrates with VCF Operations for recovery plan lifecycle visibility. The Recovery Plans Details dashboard shows the current phase of the DR lifecycle, including whether a plan is ready, whether a test is in progress, and whether incomplete recovery or cleanup operations require attention. PNR also supports recovery of VCF Automation and VCF Operations



services: when a disaster event affects the management domain containing these services, PNR enables service recovery in another management domain.

All communications between PNR and VMware vCenter instances use TLS certificates and private keys. The PNR Appliance supports importing server certificates signed by a certificate authority to establish secure communications across both protected and recovery sites.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) provides several mechanisms that support the secure recovery and reconstitution of database services to a known state following a disruption, compromise, or failure.

For backup-based recovery, DSM supports point-in-time recovery (PITR), which restores a backup to a newly created database VM. During the restore operation, the administrator can specify the database name, choose the target datastore location, and configure management settings. DSM supports restoring backups from either local or cloud-compatible (S3-compatible) storage, with restoration permitted to any point in time within the configured backup retention period. When restoring a deleted database, DSM's Archives tab allows users to access databases with retained backups and create a new database VM from a selected restore point, including options to restore to the most recent available timestamp or a custom point in time. PITR coverage extends across PostgreSQL, MySQL, and SQL Server database engines. For SQL Server, DSM automatically manages the database recovery model based on backup status, using Full Recovery for databases with automated backups enabled and Simple Recovery otherwise. Cloning a MySQL or PostgreSQL database is functionally equivalent to PITR to the latest available time, giving operators an additional path for rebuilding a known-good database instance. For failed clusters, DSM supports restoration to a different cluster with the option to target either the latest available time or a specific custom restore point.

For active recovery and failover of PostgreSQL databases, DSM provides a multi-site disaster recovery deployment model in which a secondary instance replicates from the primary across sites. When a failover is needed, DSM provides a switchover workflow that reverses the roles of instances, redirecting the secondary to become the new primary and restoring the database service to a fully operational state. DSM also provides a switchover workflow for failback, allowing operators to reverse instance roles again after the primary site is restored. DSM requires trust validation of the new primary database certificate when redirecting secondary instances during disaster recovery switchover, and operators with the DSM user role must manually establish trust between primary and secondary instances by retrieving the Certificate Authority from the primary and adding it to the Provider VM that will host the secondary. For backup-location certificates, DSM supports a Trust On First Use (TOFU) configuration that extracts and trusts certificates for specific backup locations, and at least one backup location must be configured with a trusted root certificate when a data service policy requires or allows automated backups. DSM also preserves the backing virtual machines of removed database nodes in an idle state until an instance is promoted again, maintaining infrastructure capacity for disaster recovery operations.

The resiliency of the DSM Provider Appliance itself is supplied by the underlying VCF platform (see VCF Coverage); DSM relies on vSphere High Availability to restart the appliance on another host in the event of a host failure, and does not add a separate management-plane HA mechanism. If a DSM provider restore fails for any reason, the documented procedure is to re-deploy the appliance and then follow the recovery procedure again rather than re-attempting the restore workflow on the same appliance.

DSM tracks recovery operations through its audit and operations interfaces. Event logs record Provider Restore Success events with timestamps, allowing operators to verify that recovery operations completed. Restore operation progress is visible in the Operations tab and Operations view, and a new entry appears in the Databases view table when a restore completes. To support integrity of recovery artifacts, the DSM Consumption Operator deployment includes Signature Verification of packages.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides application-layer recovery and reconstitution mechanisms at the load balancing and application delivery tier.



The Avi Controller cluster can be deployed across two availability zones, with defined failure scenarios and documented recovery actions for each zone-loss case. Availability zones should be configured based on fault domains associated with shared power supplies, shared networking devices, or other common infrastructure to reduce the risk of virtual service unavailability. When the cluster enters manual recovery state following a failure or compromise, Avi automatically revokes all Service Engine secure channel credentials and causes Service Engines to reboot themselves, halting all application traffic delivery. This returns the data plane to a controlled, known state. Service Engines that subsequently re-establish connectivity to a recovered Controller node reconfigure themselves automatically to connect only to that node.

If all three Controller cluster nodes are permanently unavailable, recovery must be performed using a backup and restore procedure. When a Controller node fails but is not permanently lost, it can be recovered by bringing the existing node back online, which is transparent to existing data traffic, or by rebuilding a new Controller node, which is disruptive to existing connections. Factory reset recovery is irreversible and deletes all Controller configuration, requiring the database to be recreated from the configuration archive. When a Service Engine is compromised or rebooted independently, all configurations including private keys and public certificates are wiped from the Service Engine, providing a clean starting state for reconstitution.

Elastic High Availability (Elastic HA) provides fast recovery for individual virtual services following the failure of a Service Engine, redistributing workloads to surviving Service Engines without requiring a full cluster-level recovery event. In vSphere DRS-enabled clusters, virtual service recovery occurs before vSphere HA brings up a replacement Service Engine when virtual service slots are available in other Service Engines, so application services are restored in a defined sequence.

Avi GSLB (Global Server Load Balancing) supports an Active disaster recovery deployment mode, enabling global-level recovery and reconstitution for applications served across multiple sites.

Avi provides configuration backup and restore capabilities for the Controller. A remote backup destination should be specified to protect against non-recoverable Controller cluster failures. The Controller checks and obtains user consent prior to performing a restore operation. After a storage failure requiring a full restore, the Controller is restored from the configuration archive. The restore process follows a defined sequence: cloud discovery, post-restore validation, configuration unblocking, and placement resumption. Post-restore validation verifies the count of restored configuration objects, including clouds and Service Engine groups, confirming that the reconstituted state matches the backup. Restore operations report state information including the operation type, image version, and cluster node name. The Controller logs restore completion events with timestamps, node IP addresses, task status, and success or failure messages for audit trail purposes.

Avi's recovery mechanisms address the application delivery tier. VM-level backup and restore for the Controller and Service Engine VMs, storage-level recovery, and infrastructure reconstitution depend on the underlying VCF platform. See VCF Coverage for those mechanisms.

## Control CR 7.6

> Network and security configuration settings: Protect configuration settings from unauthorized changes. Automated configuration monitoring: Monitor for unauthorized configuration changes.

**SCF Controls:** CFG-02

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides secure baseline configurations for all major platform components, consistent with industry-accepted hardening standards. The VCF Security Baseline is a purpose-built document that builds on VCF security fundamentals to address frequently requested compliance standards, regulations, and frameworks. It is designed and validated to tailor security configurations without affecting VCF's ability to meet its design objectives. Organizations can use the VCF Security Baseline to evaluate both default and non-default configurations and to secure their environments in a compliance context.

VCF Operations includes a consolidated out-of-the-box benchmark called the VCF 9 Security Baseline, which provides a ready-to-use compliance profile for evaluating the security posture of a VCF environment. This benchmark enables automated assessment of configurations against the documented baseline, helping maintain a consistent configuration across VCF components. ISO Security Standards compliance benchmarks are also available from the VCF Operations repository as a native pack, giving organizations the ability to measure their environment against internationally recognized hardening standards. Broadcom also provides DISA Security Technical Implementation Guides (STIGs) for VCF components, including vSphere 7.0 and vSphere 8.0 STIGs, which provide detailed instructions for securing and configuring systems to meet specific security requirements.

Individual VCF components have their own security configuration guides that document hardening recommendations. The vSphere Security Configuration Guide provides best practices for securing VMware vCenter, including securing the installation, configuring access and authentication, defining roles and permissions, and enabling auditing and logging. VMware ESX host hardening covers authentication and authorization settings, disabling unnecessary services, enabling security features such as Secure Boot and Trusted Platform Module (TPM), and implementing network security measures. The vSphere Security Configuration Guide documents specific STIG-aligned values for the Security.PasswordQualityControlPwquality setting on ESX hosts, including parameters such as minlen=15, maxrepeat=3, maxclassrepeat=4, difok=4, and enforce\_for\_root. The NSX Security Configuration Guide provides recommendations for securing NSX components, including NSX nodes, NSX Controllers, and NSX Edge nodes, by configuring strong authentication, implementing role-based access control, and securing administrative access.

VCF Operations also provides a Configure Host Security Config Data workflow that allows selective enforcement of host security hardening rules. Beyond security settings, vSphere Lifecycle Manager maintains a desired-state cluster image that defines the ESX base build, vendor add-ons, and firmware for every host in a cluster, so the software baseline stays consistent across hosts, and VCF Operations runs scheduled drift detection that flags configuration deviations from the desired state for remediation. Administrators can configure individual configuration elements under Enforce Options, enabling granular control over which baseline settings are applied and enforced across the environment. This supports reducing organizational and business risk by maintaining configuration consistency. vSphere Host Profiles provide an additional mechanism for automated and centrally managed host configuration baseline enforcement, capturing a reference host's configuration and applying it consistently across ESX hosts, covering memory, storage, networking, and other configuration aspects.

For containerized workloads, VMware Kubernetes Service (VKS) includes preconfigured security policy templates. The Baseline security template provides a set of constraints that restrict known privilege escalations while maintaining compatibility with common containerized workloads.

DISA STIGs for Supervisor and VKRs are available to guide hardening of the Supervisor control plane and individual VKS cluster releases, providing baseline security guidance at the same standard as the DISA STIGs applied to VCF infrastructure components. The VKS cluster management interface provides out-of-the-box security policy templates that mirror industry-standard Pod Security Standards profiles, including both Baseline and Restricted configurations. Custom VKS Cluster

Management Security Policy templates are also available to fine-tune individual security context parameters and support governance controls tailored to specific organizational requirements.

Pod Security Admission (PSA), enabled in VKr 1.25 and later, enforces pod security standards at the namespace level. Three policy profiles are defined: Privileged (unrestricted, for system-level workloads), Baseline (helps prevent known privilege escalations while allowing the default minimally specified pod configuration), and Restricted (heavily restricted, following pod hardening best practices). Three enforcement modes are available: enforce, warn, and audit. The `podSecurityStandard` field in the ClusterClass API encodes the intended security posture in cluster definitions, accepting these modes with specific policy versions and namespace exemptions so that baseline security configurations are applied consistently across VKS deployments. System namespaces including `kube-system`, `tkg-system`, and `vmware-system-cloudprovider` are excluded from pod security enforcement. The PSA admission controller can also be statically configured with default enforce, audit, and warn levels for unlabeled namespaces, so namespaces created without explicit labels still receive a defined security baseline. To support gradual policy adoption, audit and warn modes can be set to a stricter level such as restricted while enforce remains at baseline, giving administrators visibility into policy violations before full enforcement is applied. VCF Automation IaaS Resource Policies provide an additional enforcement point at the management layer, supporting requirements that Kubernetes clusters operate at a minimum baseline pod security level.

Kubernetes-native kernel security mechanisms provide workload-level baseline hardening options. `seccomp` restricts the Linux kernel system call surface available inside containers using BPF-based profiles, and the `RuntimeDefault` `seccomp` profile provides a strong set of security defaults while preserving workload functionality. AppArmor and SELinux provide further kernel-level restrictions through default configurations that offer foundational protections. Kubernetes default RBAC role bindings should be reviewed as part of establishing a security baseline, as defaults may grant broader access than a hardened configuration requires. Isolating each workload in its own Kubernetes namespace allows tailored security policies to be applied per workload and restricts access to namespace-scoped resources such as ConfigMaps and Secrets.

Harbor Registry, available as a standard package for VKS clusters, supports a controlled baseline for container artifacts entering the environment. Harbor provides vulnerability scanning, content trust, and policy-based replication, along with role-based access control and artifact security policies, supporting controlled management of the container image supply chain. For stand-alone virtual machines deployed in vSphere Supervisor environments, Content Libraries can distribute VM images created with current patches and required security settings that follow corporate security policies.

VMware Private AI Services (PAIS) provides PAIS-specific declarative configuration mechanisms that organizations can use to define and apply baseline settings across the AI infrastructure stack on VCF.

The primary mechanism is the `PAISConfiguration` custom resource. Private AI Services settings are declared in a YAML file compliant with the `PAISConfiguration` schema and applied with `kubectl`, giving administrators a single resource that captures observability, identity, and trust configuration for an organization namespace. Observability includes the `spec.observability.prometheusRuntime` section for Prometheus metrics collection with a configurable storage class and metrics retention period, and an `llmTraces` section for routing large language model traces to an OpenTelemetry Collector. The `PAISConfiguration` custom resource can be patched with `kubectl patch` and verified with `kubectl get paisconfiguration`, supporting baseline change control and drift detection at the AI service layer.

Certificate trust is handled through the `spec.clientTls.caBundleRefs` section, which references ConfigMaps containing CA trust bundles. Private AI Services requires HTTPS trust bundles for the OIDC provider, the Harbor registry, content management systems hosting knowledge database data, and MCP servers that provide tools to models. Trust bundles can be organized per application, with periodic updates required before certificate expiration. Access scope is restricted through the `authorizedGroups` setting, which limits Private AI Services access to specified OIDC groups.

Harbor registry distribution carries baseline controls for model and image artifacts. Replication rules support configurable filters, target namespace, and trigger mode, including a manual trigger mode, so administrators can align artifact replication with organizational policy for connected and disconnected sites. The Model Gallery uses Harbor project access capabilities to restrict access to sensitive training and tuning data. Air-gapped deployments capture the self-hosted registry URL and registry login credentials as part of the activation flow.

Workload resource baselines are defined through `SupervisorNamespaceClassConfig`, which specifies per-zone CPU limits, CPU reservations, memory limits, and memory reservations for resource allocation across zones. GPU-capable VM classes can carry GPU reservations and be confined to a single zone or supervisor namespace, supporting isolation of GPU consumption to a specific tenant. A default storage policy selection in the Private AI Services quickstart defines storage placement and resources for the service database.

Deep Learning VM (DL VM) baselines are captured through VCF Automation self-service catalog items that parameterize VM class, data disk size, user password, and optional SSH public key authentication for remote access. The DL VM OVF property `config-json` accepts a base64-encoded configuration file that controls DCGM Exporter metrics visibility through `export_dcgml_to_public` and JupyterLab authentication through `enable_jupyter_auth`, giving operators a consistent surface to define DL VM hardening parameters.

Credentials supplied during Private AI Services activation, including the private container registry password and proxy password, are stored as secrets in VCF Automation rather than as plaintext configuration values, supporting credential hygiene as part of the deployment baseline.

### **VMware vDefend (Contributes)**

VMware vDefend contributes to secure baseline configurations within its network security domain by providing structured configuration management capabilities, predefined security profiles and strategies, and configuration integrity controls that align with industry-accepted hardening practices.

vDefend supports draft configuration management with configuration locking to prevent multiple users from simultaneously opening and editing a manual draft, helping maintain configuration integrity and reducing the risk of conflicting changes to firewall policy. Role-Based Access Control (RBAC) in vDefend governs who can create or modify firewall policies, providing control over which administrators can alter the security baseline.

In vDefend, five system-defined VPC Security Strategies are available for establishing security baselines across Virtual Private Clouds (VPCs): VPC Isolation, VPC Isolation with Essential Services, VPC External Connectivity, VPC Secure Connection, and NONE. These strategies are consumed through Security Profiles in VCF Automation, which are predefined configuration templates that define the default security posture for VPCs. VPC Security Strategies provide a declarative, consistent way to apply DFW security baselines across multiple VPCs within a project, and Project Admins are guided to select a pre-defined high-level security posture that enforces Secure by Default principles rather than manually configuring individual firewall rules. Security profiles can be applied to one or more VPCs simultaneously and updated as requirements evolve.

vDefend provides predefined TLS Decryption Action Profiles with four settings: Balanced (default), High Fidelity, High Security, and Custom. These profiles are vendor-defined baseline configurations for TLS inspection behavior, giving organizations a documented starting point aligned with different security postures. TLS Inspection policies also support an Advanced Configuration option to lock policies, preventing multiple users from making concurrent changes to TLS inspection policy baseline settings.

Security Services Platform validates configuration changes before implementation through a deployment wizard that conducts thorough validations to confirm configurations are correct and compatible. The platform enforces initial security policies that cannot be modified through the user interface after they have been published, providing an immutability mechanism for foundational policy baseline elements. Security Services Platform supports backup and restore of configuration data, and certificate consistency between backup and restored environments should be verified to maintain NDR sensor integrity.

Security Services Platform's Bare Metal Security feature extends vDefend policy enforcement to physical servers, applying L3/4 security policies to bare metal infrastructure using the same policy model and management plane as virtualized workloads. Two default DFW rules are created when Bare Metal servers are onboarded to Security Services Platform, providing an immediate baseline posture for physical infrastructure. Security policies are defined, propagated, and enforced across Bare Metal servers with consistent application and monitoring.

vDefend's configuration management capabilities operate within the network security layer. The broader secure baseline configuration program for VCF, including the VCF Security Baseline document, component hardening guides, DISA STIGs,

and compliance scanning through VCF Operations, is managed at the platform level. vDefend contributes by maintaining its own secure defaults and configuration controls within the network security domain.

### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) provides configurable security baseline mechanisms for its replication and recovery infrastructure. The PNR appliance includes a security reference section that identifies configuration files containing settings that affect environment security, and PNR documentation establishes that secure operation depends on proper configuration and maintenance of the PNR server. The security reference also documents network port requirements, providing the technical specifications needed to develop and maintain secure baseline configurations for disaster recovery infrastructure.

PNR includes a Configuration Import/Export Tool that captures the complete set of protection and recovery configuration settings, including server version, build number, local and remote site names, inventory mappings, placeholder datastores, advanced settings, and array managers. The tool supports both UI-based and scripted exports and can transfer configuration data without requiring credential entry when using the bundled generated script. This gives organizations a mechanism to document, back up, and restore known-good baseline configurations for their disaster recovery infrastructure. The underlying system configuration is stored in the `va-configurator.xml` file and is accessible through Advanced Settings on the Site Pair tab in the management interface, providing a defined location for baseline review and modification.

The PNR appliance management interface allows administrators to configure appliance security settings after deployment. The Advanced Settings system provides privileged users with the ability to adjust default values governing protection and recovery features. When modifying the minimum protocol version for TLS, PNR requires that the change be applied consistently across all configuration occurrences in the system, supporting systematic baseline management for transport security settings.

For cyber recovery deployments, PNR documentation recommends using standard VCF topology with separate workload and management domains to achieve better isolation and align with security best practices at the recovery site.

Full compliance with this control requires an organizational baseline configuration management program, documentation practices, version control, and hardening tooling extending beyond what PNR provides directly.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) contributes to secure baseline configuration management through its infrastructure policy framework, data service policy enforcement, and built-in secure defaults for authentication and certificate management.

Infrastructure policies in DSM define the compute, storage, and networking resources for each database deployment, including the list of allowed VM classes and compatible storage policies. These policies are configured by DSM administrators and act as standardized, repeatable deployment baselines that database users must follow. When administrators configure infrastructure policies, they constrain the quality and quantity of resources that users can consume from vSphere clusters, establishing guardrails that support consistent deployments across the environment. Database cluster creation applies both infrastructure policies and storage policies that enforce isolation and resource constraints, reducing the risk of misconfiguration by individual users.

Data service policies extend this baseline enforcement across multi-tenant deployments. DSM administrators can define and configure data service policies for various database services and enforce them across different tenants to maintain a consistent security and compliance posture. Data service policies constrain which infrastructure options can be applied to database instances. When DSM is integrated with VCF Automation, data service policies can be applied consistently across organizational units.

DSM also ships with predefined secure defaults for database authentication. PostgreSQL databases on DSM use `scram-sha-256` as the default password-based authentication method, as configured in the default `pg_hba.conf` entries stored at `/pgsql/data/pg_hba.conf`. Custom `pg_hba.conf` entries support `scram-sha-256`, `reject`, and `cert` authentication methods. TLS is enforced by default for database client connections, with an administrator-controlled option to relax enforcement via the `postgrescluster-allow-non-ssl-system-users` setting in the advanced-system-config ConfigMap within the `dsm-system` namespace.

Certificate management is integrated into the database creation workflow and validates certificate chains for connections to external systems, including VMware vCenter. DSM introduces a stricter default certificate validation mode, the "latest" mode, that enforces cryptographic and KeyUsage validations for all DSM endpoints, including the Provider VM and data service clusters. Certificates uploaded to DSM are validated against FIPS 140-3 cryptographic standards, and uploads containing unsupported algorithms or parameters are rejected. Custom trusted root certificates must comply with FIPS requirements. DSM also verifies the SHA256 thumbprint of vCenter during initial configuration to confirm the management connection targets the expected endpoint. Custom certificate support allows organizations to replace the default DSM certificate with organization-issued certificates for both the Provider VM and database clusters. For self-service deployments using the DSM Consumption Operator, signature verification confirms package integrity before installation.

DSM validates certificate chains using configured root CA certificates stored in ConfigMaps and supports private container registry configuration with CA certificate validation for air-gapped environments.

Addressing this control fully requires organizational processes beyond what DSM provides directly. DSM does not include tooling to document baseline configurations in a compliance-ready format, map configurations to specific industry hardening standards, or audit deviations from defined baselines over time. Organizations should supplement DSM's policy and defaults framework with configuration management documentation, change tracking processes, and integration with compliance scanning tools to satisfy the full requirements of this control.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides several built-in mechanisms that support the development and application of secure baseline configurations for the application delivery layer.

The SSL/TLS Profile system gives administrators a documented starting point for cryptographic configuration. Profile Templates define accepted SSL and TLS versions and ciphers in prioritized order, and can be associated with virtual services or the Controller itself. The Standard SSL/TLS Profile is suitable for typical deployments, while the System-Standard profile provides modern cipher suites for backend server re-encryption in Kubernetes environments managed through the Avi Kubernetes Operator (AKO). Avi includes a curated list of cipher suites considered reasonably strong for SSL/TLS configuration, and the documentation notes the trade-offs between security, compatibility, and computational expense when selecting profiles. The Service Engine disables unsafe legacy renegotiation by default, providing a built-in protocol hardening baseline that reduces the risk of protocol downgrade attacks.

The System-Secure-HTTP application profile is a pre-configured baseline for securing HTTP virtual services. It enforces HTTPS, protects cookies, manages client IP handling, and controls protocol behavior, giving administrators a hardened starting point for web application delivery. Administrators can supplement these defaults with more granular settings through policies or DataScripts.

For deployments requiring validated cryptographic modules, Avi Load Balancer implements FIPS 140-3 compliance, providing a cryptographic baseline aligned with the U.S. and Canadian government standard for cryptographic modules.

The Avi Web Application Firewall (WAF) Positive Security model supports application-layer baseline configuration through rules that define allowed application behavior. Each rule supports Enable/Disable control, mode selection between Detection and Enforcement, and configurable Paranoia levels, allowing organizations to tune an application security baseline to their risk posture. The Positive Security Rule Updates feature can be enabled to activate additional configuration options for application security policies. The Cloud Console Application Rules service extends this with rules specifically designed to address known application vulnerabilities, including those with CVEs, and updates them automatically.

For Kubernetes deployments, AKO uses Custom Resource Definitions (CRDs) to enable syntactical validation of configuration objects at creation time, which helps prevent invalid configurations from being deployed. The AKO L4 CRD Rule supports SSL profiles and PKI profiles for securing pool connections, allowing Kubernetes-native expression of Avi's cryptographic baseline settings.

Certificate lifecycle management supports maintaining cryptographic baselines over time. Certificate Management Profiles support automated certificate renewal workflows, and the Trust Protection Platform integration can automatically handle certificate and chain certificate renewals by learning the mapping of virtual service names to their certificates.

The organizational process of developing, documenting, and maintaining baseline configurations across all in-scope technology assets, including selecting appropriate profile settings, applying them consistently, and updating them as software versions and security guidance evolve, remains outside what Avi performs on the organization's behalf.



## Control CR 7.6(1)

> Network and security configuration settings: Protect configuration settings from unauthorized changes. Automated configuration monitoring: Monitor for unauthorized configuration changes.

**SCF Controls:** CFG-02.2

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides several integrated automated mechanisms for governing, monitoring, and reporting on baseline configurations across the infrastructure stack.

VCF Operations is the centralized configuration monitoring engine through its Configuration Drifts feature. Administrators define desired-state configuration templates and assign them to VMware vCenter instances and vSphere clusters. When a configuration value changes from the assigned template value, VCF Operations generates a drift notification, allowing administrators to identify and remediate unauthorized or unintended changes. The Configuration Drift dashboard provides a consolidated view of all detected drifts, and drift status reports can be downloaded for offline review or evidence collection. Configuration templates can be integrated with a source control repository, enabling versioned and auditable management of configuration baselines. This integration supports change control workflows with reviews and automated validation through continuous integration and continuous delivery pipelines. Drift detection is skipped if the template does not exist in the source control repository, so maintaining the repository is a prerequisite for continuous monitoring.

VCF Operations alert definitions provide an additional layer of configuration monitoring. Administrators can configure alerts that use Metric/Property symptoms based on operational or performance values and configuration properties collected from target objects. These alert definitions monitor infrastructure components and notify administrators when configuration properties deviate from expected baselines. Associated remediation recommendations are attached to each alert, providing actionable guidance when violations are detected. Custom groups can be designated to update automatically with membership criteria, supporting structured monitoring strategies and enabling automated policy application across managed objects.

VCF Operations Diagnostics extends configuration verification by providing integrated issue detection and remediation. The Diagnostic Findings feature discovers known issues and applies fixes based on industry-standard best practices and knowledge base articles. Diagnostics derives its findings from properties and product log data collected across the full VCF stack, including VMware vCenter, VMware ESX, NSX, VCF Operations, vSAN, and VCF Automation. This cross-component visibility enables centralized verification of configuration baselines across all major platform services. Diagnostic Findings consolidates log-based and property-based findings with specific resolution guidance, integrating with VCF Log Management for additional depth in identifying configuration drift or misalignment.

VCF includes built-in compliance benchmarks based on the VCF Security Baseline and the Security Configuration Guide. These benchmarks evaluate both default and non-default settings across infrastructure components and report on compliance status. DISA STIG regulatory benchmarks are available for securing and configuring systems, and organizations can create, import, and export custom compliance benchmarks that monitor selected alert definitions and policies. Compliance management continuously monitors infrastructure objects against these benchmarks and provides actionable remediation recommendations when violations are detected.

For automated remediation and configuration enforcement, Automation Central in VCF Operations allows administrators to create scheduled jobs that automate management actions. Jobs with ten or fewer resources run immediately, while larger jobs run in parallel groups of ten. The VCF Automation plug-in provides out-of-the-box workflows for common tasks such as host configuration and infrastructure management, reducing reliance on manual configuration processes. Infrastructure operations can be further automated through integration with tools such as Terraform and Ansible for agile templating and workflow-driven configuration management.

VCF supports centralized agent-based configuration distribution. Once connected to its server, the agent automatically retrieves and applies configuration settings based on defined Agent Groups, providing a push-based mechanism for maintaining baseline configurations across managed endpoints.

At the host level, vSphere Lifecycle Manager (vLCM) defines desired software state images for ESX clusters and supports validation of the desired state and detecting drifts from it. VCF Operations supports up to 25 simultaneous cluster compliance checks per instance, and the VCF Operations API provides endpoints for performing image compliance checks programmatically. vSphere Host Profiles provide a complementary automated mechanism for host configuration compliance. Administrators define a reference host configuration that is captured as a Host Profile and applied across managed hosts; scheduled compliance checks then determine whether any configuration parameters on a host differ from those specified in the profile, and deviations are flagged for remediation.

VCF Automation integrates with VCF Operations to expose metrics and monitoring data for deployments. When monitoring is enabled, VCF Automation retrieves VCF Operations health status and associated alerts, providing a unified view of deployment health and configuration state. Tags applied to deployments support governance by expressing capabilities and constraints that define how resources are provisioned and managed. VCF Automation IaaS Resource Policies extend this governance to the Kubernetes cluster tier, supporting baseline pod security level enforcement that requires provisioned clusters to meet a defined pod security standard before resources are deployed into them.

The VMware Kubernetes Service (VKS) and vSphere Supervisor layer extends baseline configuration governance to containerized workloads through a declarative model. All Kubernetes resources, including Deployments, ConfigMaps, NetworkPolicies, Pod Security Admission labels, ResourceQuotas, and LimitRanges, are defined as desired-state manifests enforced through the Kubernetes API server. Pod Security Admission is applied at the namespace level in three configurable modes: enforce mode rejects non-compliant pods at admission time, audit mode records violations in the cluster audit log without blocking admission, and warn mode generates warnings for non-compliant submissions. The security standard applied to a namespace can be set to privileged, baseline, or restricted using `pod-security.kubernetes.io/` labels; VKS applies Pod Security Admission to namespaces by default, with the standard configurable per cluster and namespace. System namespaces such as `kube-system`, `tkg-system`, and `vmware-system-cloudprovider` are excluded from enforcement by default. The three-mode design enables automated reporting on pod security baseline violations through the audit and warn modes independently of enforcement, supporting continuous assessment of compliance posture across the workload layer. Controllers continuously reconcile the actual cluster state against the declared desired state, automatically correcting configuration drift without manual intervention. This continuous reconciliation loop addresses the real-time verification requirement for the workload consumption layer in the same way that CDM-style programs require continuous verification of all technology assets.

The Argo CD Supervisor Service extends this declarative model to GitOps-based configuration governance. Desired configurations are stored in Git as the authoritative baseline, and Argo CD continuously compares the live cluster state against that baseline, surfacing discrepancies through its dashboard, health status indicators, and alerting integrations. Depending on the sync policy configured, Argo CD can automatically reconcile the cluster to the declared state or flag deviations for operator review. The service is designed to minimize manual intervention through automation. When configuration is managed as immutable infrastructure in version control, the combination of Git history and Kubernetes audit logs creates a durable, chronological record of every authorized change to the cluster configuration.

VKS Cluster Management provides policy-driven governance of the VKS cluster estate. Security policies can be assigned to clusters and enforced across the organization, and custom policies are implemented using OPA Gatekeeper as the policy engine. VKS cluster management proactively monitors the cluster estate to verify that assigned policies are enforced correctly and generates Policy Insights when configuration conflicts or environmental errors are detected, providing automated reporting of policy compliance status across all managed clusters.

Admission controllers built into the Supervisor API server enforce baseline security and configuration requirements at the point of resource admission. A validating admission webhook rejects Kubernetes resources that do not conform to the organization's defined configuration baseline before they are persisted to etcd, making off-baseline configurations inexpressible at submission time rather than relying on after-the-fact detection. Kubernetes Operators extend this model to

application-level configuration governance by automating repeatable configuration tasks and continuously monitoring and enforcing configuration requirements for the applications they manage.

The Kubernetes audit logging subsystem records all API server interactions, including configuration creates, updates, and deletes, according to a configurable audit policy. Audit events can be captured at multiple stages and shipped to an external log backend via webhook, providing an automated and continuous record of all configuration changes across the cluster. Node Problem Detector runs as a DaemonSet across cluster nodes, continuously monitoring node-level health conditions and reporting anomalies through Kubernetes Events and Node status conditions, providing automated detection of node-level configuration deviations that may indicate drift from the expected baseline.

#### **VMware vDefend (Contributes)**

VMware vDefend contributes to automated baseline configuration management through several network-security-layer mechanisms in 9.1.

Security Intelligence, part of the Security Services Platform, runs an automated classification cron job at 2:00 AM local time and every 24 hours thereafter. The job analyzes traffic flows from the past 30 days to classify compute entities as infrastructure service providers or workloads. Administrators can review and accept or modify the inferred classifications through the Security Explorer interface, and can supplement automatic detection by providing CSV files listing known infrastructure servers. Security Intelligence also classifies detected application assets as critical or regular applications, and a Rerun Analysis function re-initiates classification analysis automatically when NSX inventory changes occur, such as deleted groups or manually updated tags.

Platform Inventory monitoring in the Security Services Platform collects infrastructure asset status across Regions, Zones, Environments, and Virtual Machines, showing success, timeout, or out-of-sync conditions. Administrators can filter by name and status to identify specific assets and expand environment rows to view assigned tags and verify hierarchy accuracy.

Security Intelligence Policy Recommendations includes a Monitoring capability that checks for changes in the scope of input entities every hour when the Monitoring toggle is enabled, supporting continuous awareness of scope drift.

In federated deployments, the Global Manager centralizes management of IDS/IPS signatures, profiles, Distributed Firewall (DFW) and Gateway Firewall (GFW) policies, oversubscription settings, and syslog settings across all Local Managers. vDefend also supports centralized management of custom IDS/IPS signatures from the Global Manager across multiple Local Managers.

These capabilities address automated discovery, workload classification, asset status monitoring, and centralized policy management within the network security layer. They do not constitute a full CDM platform and should be supplemented with broader configuration management tooling for complete TAAS baseline governance.

#### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer provides technical mechanisms that contribute to automated baseline configuration governance. Avi Load Balancer events are the building blocks of configuration automation: every change made through the web interface, CLI, or REST API is recorded as an event and automatically propagated to all Controller cluster members, giving operators a continuous record of what changed and maintaining cluster consistency.

User-Defined Metrics allow administrators to define and extract highly customized telemetry data for specific enterprise compliance or reporting needs via API calls. This supports continuous monitoring of configuration state against defined thresholds and provides the raw data for compliance-oriented reporting workflows.

Avi's REST API and CLI provide the interfaces for infrastructure-as-code configuration management. Official Ansible and Terraform integrations allow organizations to declare Avi Load Balancer baseline configurations as code, enforce those baselines through CI/CD pipeline runs, and detect drift when the running configuration diverges from the declared state. Ansible can be integrated with CI/CD pipelines to trigger automated Controller deployments upon code changes. Terraform modules support automatic scaling of service pool members through Network Infrastructure Automation (NIA) integration with Consul backend state management.

In VCF deployments, Avi integrates with VCF Automation's Provider Management Portal, which enables provider administrators to define infrastructure boundaries, create regions and organizations, and set region quotas for capacity allocation. This integration extends Avi's configuration governance surface to include multi-tenant resource governance and capacity planning for organizations consuming load-balancing services through the VCF stack.

Defining the configuration baselines, scheduling continuous comparison across the broader environment, and reporting deviation findings to management are organizational responsibilities fulfilled outside Avi through tooling and processes that operate against Avi's API and audit data.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control CR 7.7

> Least functionality: Disable unnecessary ports, protocols, and services.

**SCF Controls:** CFG-03

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides multiple layers of configuration controls that restrict systems to only essential capabilities by controlling ports, protocols, and services across the platform stack.

At the hypervisor layer, VMware ESX hosts include a built-in host firewall that controls which ports and services are accessible. The firewall supports specification of allowed hosts through the `allowedHosts` parameter, so administrators can limit which IP addresses may connect to specific services. Services such as SSH (TSM-SSH) can be individually started or stopped through the VMware Host Client, giving administrators direct control over which management services are running. ESX lockdown mode restricts host access so that all management is performed through VMware vCenter, with an Exception Users list for controlled overrides when direct host access is needed. ESX also supports disabling unexposed features through isolation configuration parameters, removing functionality that is not needed in the environment.

Port groups on distributed switches are communication boundaries where security policies can be configured. Each port group carries its own security policy settings, including controls over promiscuous mode, MAC address changes, and forged transmits. Administrators can modify these policies to restrict which types of network traffic virtual machines are permitted to generate or receive. Each port group can also be assigned a VLAN ID to restrict traffic to a logical Ethernet segment. SpoofGuard and switch security profiles can be applied to distributed port groups to prevent traffic spoofing and enforce network security policies at the virtual switch layer. Distributed port groups also support Port Blocking Policies, which allow administrators to selectively block ports from sending or receiving data, including blocking all ports within a group simultaneously. Traffic Filtering and Marking Policy can be applied at the individual distributed port or uplink port level to permit or stop traffic flows through virtual machines, VMkernel adapters, and physical adapters. Port-level configuration overrides on distributed virtual switches are deactivated by default, so permitting per-port policy exceptions requires deliberate administrator action. The virtual serial port proxy provides an additional gateway that enforces security for remote serial port connections, restricting access to only authorized communication paths.

Management traffic is isolated on dedicated networks. The Fleet Level Components on Dedicated Management Network model uses VLAN-backed distributed port groups to provide isolation from other networks. NSX allows configuration of services with port and protocol pairing, and these services can be referenced in firewall rules to allow or block specific types of traffic.

The platform documents explicit port requirements for each component. When vSAN is enabled on a cluster, required ports are added to ESX firewall rules and configured automatically, keeping firewall configuration aligned with actual service requirements rather than leaving unnecessary ports open. VCF Operations Cloud Proxy requires only ports 4505, 4506, and 8443 via TCP for application monitoring, establishing a minimal set of allowed network communication paths. These defined port lists give administrators a clear baseline of what must be open rather than relying on permissive defaults.

VCF Operations supports firewall hardening, which when activated restricts network access to internal services, limiting exposure to only the ports and protocols required for operation. A service discovery policy setting controls whether the system actively discovers services on the network; disabling this policy reduces the attack surface by preventing unnecessary service enumeration, though administrators should be aware that turning it off changes the security posture of the operating system. For orchestration functions, VCF Operations enforces privilege requirements where ports below 1024 can only be opened with superuser privileges on Linux systems, providing an inherent restriction on which processes can bind to well-known service ports. VCF Operations also collects ESX security properties including service running status, firewall rulesets, service policies, and disabled protocols when the vSphere Hardening Guide policy is applied, giving administrators visibility into the least-functionality posture of each host.

The Security Configuration Guide provides recommendations for disabling unnecessary services and enabling security features on ESX hosts as a baseline for least-functionality configuration.

At the workload consumption layer, VMware Kubernetes Service (VKS) and vSphere Supervisor enforce least functionality across compute, kernel, and network dimensions for containerized workloads. Pod Security Admission (PSA), enabled by default in Supervisor clusters running VKS 1.25 and later, enforces Kubernetes Pod Security Standards at the namespace level. Administrators configure each namespace with an enforcement profile from three options: Privileged (no restrictions), Baseline (blocks known privilege escalation paths), and Restricted (heavily restricted, following pod hardening practices). Each profile supports three independent modes: enforce (rejects non-compliant pods at admission), audit (records violations in the audit log without blocking), and warn (surfaces policy warnings to the requesting user without blocking), allowing operators to gain visibility into policy gaps before tightening enforcement. The Restricted PSA level requires containers to drop all Linux capabilities, blocks host networking, host PID, and host IPC namespaces, restricts volume types to a safe subset, disallows privilege escalation by setting `allowPrivilegeEscalation` to false, and requires a defined Seccomp profile. Supervisor applies the `vmware-system-restricted` Pod Security Policy by default to VKS clusters, which blocks privileged container access, helps prevent root escalation, and restricts permitted volume types; standard packages such as Contour also reference this policy so that system components operate under the same restricted constraints. VKS Cluster Management Security Policies provide an additional management layer for pod security within VKS clusters; when creating a security policy, the Restricted pod security policy template offers a preconfigured tight security context that constrains pod privileges and capabilities across the cluster.

Kubernetes pod and container security context configuration allows administrators to add or drop specific POSIX kernel capabilities through the `securityContext.capabilities` field, restricting which kernel operations individual containers may exercise. Pods can be configured with Seccomp profiles that filter the system calls available to containers; the `RuntimeDefault` profile applies the container runtime's default system call filter and blocks calls not required by typical applications, while custom profiles allow more precise control. The Restricted Pod Security Standard makes a defined Seccomp profile mandatory for compliant workloads. Supervisor also supports SELinux, which restricts access to files, applications, ports, and processes through labels and security policies enforced on tagged processes, and AppArmor profiles that restrict a container's abilities by enforcing a defined set of rules and restrictions on process behavior at the kernel level. On Ubuntu and Photon nodes in VKS clusters, AppArmor additionally controls file system access and network access, extending mandatory access control across multiple resource dimensions beyond process behavior alone. These mechanisms operate independently of Kubernetes admission controls and provide a defense-in-depth layer for workload capability restriction. Kubernetes ephemeral containers disallow ports, `livenessProbe`, and `readinessProbe` fields, further restricting their capabilities relative to standard containers.

Network-level least functionality for VKS workloads is enforced through Kubernetes NetworkPolicy objects, implemented by Antrea or NSX-based CNI plugins in Supervisor, which restrict ingress and egress traffic at the pod and namespace level by port, protocol, and peer selector. Cluster administrators can restrict the IP address ranges allowed for Kubernetes Services through centralized management of ServiceCIDR resources, reducing the scope of reachable service endpoints across the cluster. Access to control plane ports such as the etcd API port should be restricted at the network level to only specified and trusted IP address ranges. VKS control plane nodes enforce a set of default inbound firewall rules that administrators cannot override, covering only the ports required for control plane operation, including TCP 6443 for the kube-apiserver, TCP 2379-2380 for etcd, TCP 22 for SSH, TCP 179 for BGP, and TCP 10100 for the kapp-controller, among others. The kubelet `allow-privileged` flag controls whether privileged containers are permitted on cluster nodes. The Istio CNI plugin, configurable through the `istioCNI.enabled` setting, manages pod network interfaces and enforces network policies without requiring privileged containers, reducing the need for elevated privileges in service mesh deployments. Kubernetes guidance documented in Supervisor materials states that access control components operating with high privileges should be isolated from general workloads using namespace boundaries, so that privilege is scoped to where it is operationally required.

VCF Automation provides security groups that define isolation settings for on-demand private or outbound networks, allowing administrators to restrict which services and ports are accessible within network segments. Network profiles support isolation policies defined as either on-demand or existing security groups, enabling granular control over network communication boundaries.



For ongoing compliance verification, VCF Network Operations provides a search query capability to identify firewall rules that permit any port, querying for rules where the action is set to allow and the service is configured as "any." This allows administrators to systematically find and remediate overly permissive rules that violate least-functionality principles, restricting access to only the ports and protocols that are explicitly required.

### **VMware vDefend (Meets)**

VMware vDefend provides the primary enforcement mechanism for least functionality at the network layer, allowing administrators to configure systems so that only essential ports, protocols, and services are permitted.

vDefend (DFW) operates per workload and supports Allow, Drop, or Reject actions on traffic matching defined services with specific port and protocol pairings. Administrators build policies that permit only the traffic required for a workload's function while denying everything else. DFW policies can explicitly block insecure protocols such as Telnet, FTP, and TFTP across the data center, actively blocking protocols that should not be present in the environment. DFW governance implements a least-privilege posture by preventing overly permissive rules such as "ANY/ANY/ANY ALLOW," guiding administrators toward specific port and protocol restrictions rather than broad allow policies. Rules are processed in precedence order across multiple policy sections, giving administrators fine-grained control over which traffic categories are evaluated first.

vDefend extends least-functionality enforcement to inter-project and perimeter boundaries. Gateway Firewall rules support configurable service selection, allowing administrators to specify a blanket block on all protocols or access restricted to specific ports for selective inter-project communication. Enterprise administrators can configure inter-project isolation policies that allow only essential infrastructure protocols while denying all other inter-project communication, defining perimeter-level traffic boundaries based on what each segment actually requires.

Security Intelligence, part of the Security Services Platform, provides traffic flow analysis with protocol and port filtering. Administrators can analyze observed flows using Exclude or Match criteria, supporting up to 15 entries of single ports, port ranges, or protocols. This visibility helps administrators verify that only essential ports and protocols carry traffic and identify unexpected communication patterns that may indicate unnecessary services remain enabled. Security Intelligence also provides guidance on restricting risky application protocols that transport data in clear text, flagging these for restriction or replacement as part of the overall least-functionality posture.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) contributes to the Least Functionality control through its infrastructure policy framework and data service policy allowlists, which restrict what network resources, compute configurations, and service parameters are available to database consumers.

Infrastructure policies are the primary mechanism. Each infrastructure policy requires the selection of specific network port groups for each compute resource (network-port-group setting), and at least one IP pool must be defined for each selected port group (ip-pools setting). This constrains which network segments database instances can attach to, restricting consumers from selecting arbitrary network endpoints. Infrastructure policies must be explicitly enabled before they become available for database provisioning; policies that are not yet ready for use can be kept deactivated (infrastructure-policy-enabled setting), which limits which network configurations are offered to consumers at any given time. DSM also validates infrastructure policy IP ranges against system configuration networks, detecting and rejecting overlapping configurations before they take effect.

Data service policies build on infrastructure policies to create a tiered allowlisting mechanism. Each data service policy requires the selection of at least one infrastructure policy, and in the self-service consumption model, the DataServicePolicy custom resource's allowedInfrastructurePolicies field enumerates precisely which infrastructure policies authorized consumers may use. Database users cannot select infrastructure configurations outside the approved set defined by administrators. Infrastructure policies also filter storage policies to display only those compatible with the designated compute resources, further narrowing the options available at provisioning time.

Infrastructure policies create broader resource guardrails as well: the overall policy framework restricts the quality and quantity of resources DSM users can consume from vSphere clusters, and database clusters are deployed with infrastructure and storage policies that enforce isolation and resource constraints from the moment they are provisioned.



Data service policies support an "Allow with a List" option, which creates a specific allowlist of approved parameters that users can select from during provisioning. SQL Server data service policies extend this pattern: they require selection of at least one infrastructure policy and restrict instance provisioning and server-level configuration, including Active Directory integration and high availability settings, to administrators only.

For PostgreSQL databases, DSM provides an additional connection control capability. The Block Database Connections operation can stop existing client connections and block new ones from being established, with exemptions only for monitoring, replication, and internal service traffic. This allows administrators to restrict access to a database instance to only the services required for continued operation.

DSM's contribution to this control is scoped to the database-as-a-service layer. Restrictions on ports, protocols, and services at the network perimeter, host, or hypervisor level depend on the underlying VCF platform, NSX networking configuration, and vDefend firewall policies outside DSM's direct control. Organizations should complement DSM's policy framework with platform-level network security controls to fully address the Least Functionality requirement.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides mechanisms for restricting ports, protocols, and services at the application delivery layer, supporting least-functionality configurations for application traffic and management access.

At the application traffic layer, each virtual service accepts traffic only on explicitly configured service ports. The per-Service-Port profile configuration (ApplicationProfile or NetworkProfile) can be overridden on a per-service-port basis for advanced use cases. Virtual Service Network Security Policies can match on service port to filter traffic based on TCP service ports on which the virtual service is listening, allowing administrators to explicitly permit or block traffic by protocol and port on a per-virtual-service basis.

For Controller management access, Avi provides several mechanisms to restrict access to management channels. The Client Management Access configuration allows administrators to define allowed SSH clients, allowed CLI shell clients, allowed external HTTP(S) clients, and allowed external SNMP clients separately, each scoped to specific IP addresses, ranges, subnets, or IP groups. Management channels not included in the configured allowlist are blocked. This configuration is accessible through the Securing Management IP Access workflow in the Controller UI under System settings. Trusted host profiles can be configured to restrict which hosts are permitted to access the Controller, providing an additional layer of host-identity-based access restriction. The Controller also limits the allowed ciphers and HMACs for management sessions, reducing the cryptographic surface exposed on each management channel.

Security groups for Avi Controller and Service Engines specify the protocol ports for which traffic is permitted between components, providing a defined port inventory that administrators use to scope surrounding network controls.

These capabilities apply at the application delivery boundary. Least-functionality controls for the underlying compute, storage, and network virtualization layers are provided by VCF, which is rated Meets for this control.

Not applicable for this control: VCF Protection and Recovery. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control CR 7.8

> Component inventory: Capability to provide a list of hardware and software components.

**SCF Controls:** AST-02

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF contributes to asset inventory processes by maintaining a detailed, real-time inventory of technology assets across compute, storage, and networking layers.

VMware vCenter maintains the authoritative inventory of all managed objects in the environment. The vSphere inventory organizes objects in a hierarchical structure beginning at the root folder and extending through datacenters, clusters, hosts, virtual machines, networks, and datastores. Each object carries metadata including configuration details, resource allocations, and operational state. Administrators can extend this metadata by defining custom attributes, which store organization-specific fields against any inventory object in vCenter. Custom attribute values are held centrally by vCenter rather than within the individual objects, allowing teams to annotate VMs, hosts, datastores, and other inventory items with organization-defined information such as ownership, classification, or business justification details. This inventory reflects the live state of the environment as objects are created, modified, or removed. Multiple vCenter instances can be linked for centralized inventory management across workload domains.

Programmatic access to the full inventory is available through the vSphere Web Services API. The Property Collector service uses Property Filter Specifications to identify inventory elements either directly or through object property traversal. The `RetrievePropertiesEx` method collects properties from inventory objects, allowing external systems and automation scripts to enumerate assets, gather configuration details, and build reports at whatever level of granularity is needed. The SimpleClient sample application demonstrates how to connect to a vCenter instance and obtain a listing of top-level inventory entities along with their properties and references.

VCF Operations extends inventory visibility by retrieving inventory details from each vCenter, including associated clusters, VMware ESX hosts, datastores, and virtual machines. Discovered objects are grouped logically in detailed inventory lists, and the inventory management interface supports sorting, searching, and object tagging to organize assets at scale. Custom object groups are containers that collect data from included objects and report on the data collected, allowing administrators to create dashboards and widgets that display resource information for grouped assets. VCF Operations also applies policies to newly discovered objects, such as objects added to an object group. ResourceAttribute elements define the metrics that appear in the monitoring interface, each identified by a Metric Key, providing continuous visibility into asset utilization, capacity, and health. The vSphere Compute Inventory Dashboard provides interactive views of clusters, hosts, and virtual machines associated with any selected object.

For application-level visibility, the VCF Operations Service Discovery adapter discovers running applications and services along with their CPU and memory metrics. Discovery rules can be defined using object types including metrics, relationships, properties, object names, and tag-based criteria. Discovered applications can be grouped by object type and selected attributes through Application Definition criteria, and the results can be exported in CSV format for use in external asset tracking systems.

VCF Automation adds a consumption-oriented layer to asset tracking. Projects link users, catalog items, and infrastructure resources where deployments are provisioned. The Overview tab displays infrastructure and consumption metrics across the organization. Resources visible in inventory views are scoped by the rights and project roles assigned to each user, providing role-appropriate visibility for audit and review. When catalog items are deployed, they are provisioned against the infrastructure available to the selected project, creating a traceable relationship between the request, the deployment, and the underlying resources. Provider accounts can create and track assets including virtual machines, catalogs, templates, and media. Organizations in VCF Automation support policy-based governance on resource usage, allowing IT teams to monitor resource consumption, control permissions, and provide managed services with network isolation. Each organization maintains a secure boundary from other organizations. Resource quota policies can be defined through the Policies API to

limit resources consumed by each user, project, or organization. Day 2 Action policies restrict which users have access to specific lifecycle operations on deployed resources. Tags within VCF Automation express capabilities and constraints that define deployments, and cloud administrators can place constraint tags on projects to exercise governance over resource provisioning, supporting asset classification and controlled placement of workloads across the infrastructure.

For integration with external asset registries, VCF provides multiple paths. The Management Pack for ServiceNow in VCF Operations discovers and synchronizes asset details including CPU count, CPU speed, CPU type, disk space, RAM, serial number, manufacturer, model ID, OS version, IP address, and vCenter reference through the ServiceNow Table API and CMDB Meta API. These properties are marked as required discovery fields, keeping the external CMDB aligned with the actual virtual infrastructure. VCF Operations for Networks supports adding ServiceNow as a CMDB data source to fetch configuration items and their relationships. VCF Automation also integrates with ServiceNow for ITSM and CMDB synchronization, configurable through extensibility action scripts or through VCF Operations Orchestrator Client hosted workflows. The ServiceNow ITSM Plug-in for VCF Automation captures extensive asset metadata, including resource name, operational status, project assignment, storage details, account and region information, endpoint type, zone, environment, and custom properties. The plug-in stores machine resource records in custom ServiceNow CMDB tables that track creation timestamps, deployment IDs, machine IDs, entitled users, and network schema. Deployment records similarly capture blueprint version, project ID, catalog ID, organization ID, operational status, owner, requester, and expiration tracking. The plug-in supports day-2 resource lifecycle actions such as Add Disk, Resize, Power On/Off, Create Snapshot, and Delete, and maintains CMDB accuracy through reconciliation.

Software asset tracking is supported through vSphere Lifecycle Manager, where software depots contain both the payloads and metadata for software updates. VCF Lifecycle Management tracks different versions of software components to help maintain compatibility across the infrastructure. The VCF Operations API includes methods to register associated products with their product name, version, and deployment details, contributing to a software-level view of the environment.

The Consumption layer of VCF extends asset inventory to containerized workloads running on VMware Kubernetes Service (VKS) and vSphere Supervisor. The Kubernetes API server maintains a live registry of every object within a cluster or Supervisor namespace, covering pods, deployments, services, ConfigMaps, Secrets, PersistentVolumeClaims, CustomResourceDefinitions, and other resource types. Every resource carries standard metadata including name, namespace, labels, annotations, creation timestamp, and owning references that administrators can query or export at any level of granularity. Labels and annotations allow teams to attach arbitrary metadata to any Kubernetes resource, such as application name, business unit, or data classification, and these fields are queryable via label selectors, supporting filtered inventory exports by category. Supervisor namespaces map to vCenter namespaces, associating Kubernetes workloads with the organizational units defined during provisioning, and resource quotas and limit ranges attached to each namespace record the allocated capacity and support tracking of resource consumption by application or team.

For workloads that consume specialized hardware such as accelerators or dedicated network devices, the Kubernetes Device Plugin framework provides hardware-level asset visibility within VKS clusters. Administrators or device owners use DeviceClasses to define sets of devices available in the cluster, and workloads claim specific hardware resources by creating ResourceClaims that filter for device parameters declared in a DeviceClass. Monitoring agents for device plugin resources discover which devices are in use on each node and collect metadata that associates each device allocation with the specific container consuming it, extending hardware asset tracking to the workload level.

The Harbor Registry Supervisor Service provides an inventory mechanism for container images and artifacts. Harbor organizes images in a Projects structure by team or application and secures them with policies and role-based access control, allowing administrators to track which container images are authorized for use within the environment. Container images deployed through VCF consumption paths should be scanned, signed, and approved according to organizational software supply chain policies before use; digital certificates and validation mechanisms support supply chain assurance of images and artifacts. Where GitOps practices are in use, the Argo CD Supervisor Service extends this inventory capability by recording application synchronization events with timestamps, capturing the resource group, kind, namespace, name, sync status, and health status of each synchronized resource. Git repositories serve as the single source of truth for desired application and infrastructure

state in a GitOps workflow, and infrastructure as code stored in version control creates a durable audit log of every authorized change to that state.

These capabilities give VCF a strong technical foundation for tracking virtual infrastructure assets with accurate, real-time data at multiple levels of granularity. However, the broader requirements of this control, including business justification tracking, organization-defined accountability information, and formal review and audit processes for the full scope of technology assets, applications, services, and data, require organizational policies and processes that operate alongside the platform's inventory capabilities.

### **VMware vDefend (Contributes)**

VMware vDefend's Security Services Platform (SSP) maintains an inventory of network-visible workloads organized into a hierarchical asset collection spanning regions, zones, environments, applications, and application tiers. The Publish Inventory Assets tab in SSP displays infrastructure asset details including asset names, asset types, IP addresses, and total VM counts for assets sourced from imported CSV files, discovered services, or published infrastructure assets, providing accurate, granular records of network-visible assets.

Platform Inventory monitoring supports ongoing inventory accuracy through several mechanisms. Administrators can filter infrastructure assets by Name and Status, identifying assets that are timed out or out of sync. The inventory hierarchy view shows regions, zones, and VMs so administrators can verify correct definition. SSP Inventory displays assigned tags for each infrastructure asset, and environment rows can be expanded to view tag assignments and confirm hierarchy accuracy. The platform also collects and displays VM tags with their respective scopes, supporting classification and tracking. Published applications and their application types are verifiable in the Inventory > Applications interface.

SSP identifies and publishes application assets, classifying them as either critical or regular applications. Application Asset Collections represent groups of workloads delivering enterprise software functions. Security Intelligence within SSP displays application associations for compute entities, indicating the tiers within each application to which a given compute entity belongs. Administering compute entity classifications in Security Intelligence requires appropriate privileges, supporting access control over classification changes. Consistent naming conventions are recommended when preparing inventory data for import, and the Asset Prefix setting enables standardized naming during segmentation planning and monitoring.

In disconnected mode, usage files include an `asset_id` field providing a unique asset identifier, supporting asset tracking in environments without continuous connectivity.

vDefend's inventory scope is limited to network-visible workloads and does not replace enterprise-wide TAASD inventory tools, authorized software catalogs with business justification records, or data asset registries required for full AST-02 compliance.

### **VMware Data Services Manager (Contributes)**

VMware Data Services Manager (DSM) contributes to asset inventory requirements by providing built-in visibility into the database instances, infrastructure resources, and users it manages. This visibility supports tracking and accountability for the subset of technology assets that DSM governs. Organization-wide TAASD inventory across all asset categories requires organizational processes and additional tooling beyond DSM itself.

DSM tracks the association between deployed databases and the infrastructure policies under which they were provisioned. Administrators can review databases and their configuration details through the Databases tab in the Infrastructure Policies view of the DSM Administrator UI, which displays each policy alongside the databases bound to it. Because infrastructure policies define the compute, storage, and networking resources allocated to databases, including allowed VM classes, this view gives administrators a structured picture of how platform resources are being consumed. The About the DSM UI Views section confirms the Infrastructure Policies view lists all policies available in the environment and the databases associated with each.

DSM administrators can define and configure data service policies for the data services DSM manages, supporting compliance with organizational standards across tenants. Each policy can be tailored to specific requirements and assigned to one or more organizations to enforce consistent configurations. For tenant-facing self-service, the DSM Consumption Operator

creates infrastructure policies as cluster-scoped resources in the consumption cluster and requires an `allowedInfrastructurePolicies` list that defines which policies a tenant cluster may use, giving administrators a clear inventory of the policies in scope per cluster. Tenant users create and consume databases under a Supervisor namespace through the VCF Automation Cloud Consumption Interface (CCI), and the DSM portal exposes a Consumption Operator endpoint that provides access to deployment artifacts.

For database-level visibility, the Database Health view in the DSM UI collects metric data for each database instance and cross-references alerts with resolution paths. DSM deploys a Telegraf agent on the Provider Appliance and a local Telegraf agent on each workload cluster to gather VM-level metrics such as CPU, memory, and disk usage, plus engine-specific metrics. DSM can forward metrics to an external Metrics target server, supporting integration with external monitoring or inventory aggregation systems.

For auditors, DSM's inventory-relevant information is accessible through the Administrator UI and is available to personnel with the Admin role. The Databases tab, Infrastructure Policies view, Database Health view, and Consumption Operator policy inventory collectively provide the granularity needed to account for DSM-managed database assets and the policies governing them. The documented DSM user roles (vSphere administrator, VCF Automation organization administrator, VCF Automation project user) clarify which personnel have visibility into which inventory views.

DSM provides resource tracking and visibility within its own management scope. DSM does not perform organization-wide asset discovery and does not inventory physical infrastructure, non-database applications, or data assets outside its management domain. A TAASD inventory program spanning all asset categories requires a CMDB or dedicated asset management solution that can aggregate DSM resource data alongside information from other platform components.

Not applicable for this control: VCF Protection and Recovery, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control EDR 2.4

> Mobile code integrity: Verify the integrity of mobile code before execution. Mobile code authenticity: Verify the authenticity of mobile code before execution. Mobile code authorization: Allow only authorized mobile code to execute. Mobile code security: Provide the capability to enforce a security policy for mobile code. Mobile code execution environment: Run mobile code in a confined environment.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control EDR 2.4(1)

> Mobile code integrity: Verify the integrity of mobile code before execution. Mobile code authenticity: Verify the authenticity of mobile code before execution. Mobile code authorization: Allow only authorized mobile code to execute. Mobile code security: Provide the capability to enforce a security policy for mobile code. Mobile code execution environment: Run mobile code in a confined environment.

**SCF Controls:** CHG-04.2

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF enforces digital signature verification for software and firmware components at multiple layers of the stack, from hypervisor boot through application deployment.

During VCF deployment, each component is assigned a certificate from a default signing certificate authority. The VMware Certificate Authority (VMCA) is an internal CA that comes integral with vSphere deployment and has been purpose-built to serve the certificate needs of the vSphere environment. From initial installation onward, all VCF components operate within a certificate-based trust framework rooted in a known signing authority. Administrators can configure VCF to use certificates from VMCA, Microsoft CA, OpenSSL CA, or external third-party certificate authorities, giving organizations flexibility to integrate their approved CA hierarchy into the VCF signing and verification workflow. All communication between the VCF Installer and ESX hosts is performed over HTTPS with certificate identity validation, and the installer validates the identity of each host by comparing the certificate common name against the provided FQDN.

At the hypervisor level, VMware ESX uses UEFI Secure Boot to verify that the boot loader and kernel modules are digitally signed before execution. The host acceptance level setting controls which vSphere Installation Bundles (VIBs) are permitted: it can be configured to accept only VMware-certified VIBs, restricting installation to components signed by VMware's certificate authority. The vSphere Security Configuration Guide includes recommendations for enabling Secure Boot and configuring TPM on ESX hosts. When UEFI Secure Boot is enabled on an ESX host, the host performs full signature verification of each package installed on the host at every boot, not only at initial installation. vSphere also supports UEFI Secure Boot for individual virtual machines, where the bootloader, operating system kernel, and operating system drivers must each carry a valid signature before the virtual machine is permitted to boot.

vSphere Lifecycle Manager (vLCM) provides desired-state image management for ESX hosts. Administrators define a target image specification for each cluster, and vLCM checks host compliance against that image. When hosts deviate from the desired image, vLCM can remediate them back to the approved configuration. This approach restricts ad-hoc installation of unsigned or unapproved software components on managed hosts.

VCF Operations enforces digital signing for its management packs and orchestrator packages. Management pack PAK files are verified against a digital signature provided by VMware. If a PAK file lacks a valid digital signature, the system displays a warning indicating the file cannot be verified as authentic, and the digital signature indicates the original developer or publisher. For orchestrator packages, all packages exported from a VCF Operations orchestrator server are digitally signed, and the orchestrator's certificate infrastructure allows signing of all elements created within the system. When a client receives an element from the orchestrator server, the client verifies the server identity and decides whether to trust the signature. Package signing certificates are a form of digital identification used to support encrypted communication and provide a signature for orchestrator packages. For production environments, VCF recommends using a certificate signed by a certificate authority rather than a self-signed certificate, allowing organizations to align package signing with their approved CA infrastructure.

For container workloads, VMware Kubernetes Service (VKS) validates TLS certificates when fetching container images from registries and requires that registry certificates be signed by a trusted certificate authority. The platform requires that a certificate signing request be generated, signed by a certificate authority, and applied to the Workload Platform Management layer to secure the Kubernetes control plane. TLS certificates for the Supervisor layer are derived from the VMCA certificate, anchoring the Kubernetes control plane trust chain in the same root authority as the broader VCF infrastructure. The VCF CLI



imgpkg tool supports CA certificate validation for registry connections through its --registry-ca-cert-path flag, which can be specified multiple times to add CA certificates for registry API communication. VKS clusters can also be configured to trust any number of CA certificates for private registry endpoints, directing image pulls to controlled, organization-approved sources.

Harbor is available as a standard package for VKS clusters and provides enterprise container registry capabilities including content trust and image signing. Organizations can configure Harbor to enforce signature requirements at image push or pull time, restricting deployments to images signed under an organization-approved certificate authority. Admission controllers are the primary enforcement mechanism for image signing policies within VKS clusters. The ImagePolicyWebhook admission controller allows administrators to configure an external image verification backend that is consulted before any pod is scheduled; if the backend determines that an image lacks a valid signature from an approved authority, the pod is rejected before it runs. ValidatingWebhookConfiguration objects can integrate third-party image signing solutions with the Kubernetes API server, creating an enforcement layer that intercepts pod creation requests and validates image signatures as part of the admission control chain.

The Kubernetes control plane components used by VKS are themselves distributed as signed artifacts. Kubernetes release artifacts, including standalone binaries and Software Bill of Materials files, are signed using cosign keyless signing, and control plane component images can be verified using cosign with certificate-based identity and OIDC issuer validation. Kubernetes CertificateSigningRequest signers can deny requests that do not meet approval conditions, and CertificateSigningRequests with unknown signerName values are never auto-approved, restricting unauthorized certificate issuance within the cluster. For workload certificate lifecycle management, the cert-manager standard package creates TLS certificates for workloads and automatically renews them before expiration, with VKS cluster control plane certificate rotation configured through the ClusterClass API and defaulting to renewal 90 days before expiry. The renewal window is configurable through the renewalDaysBeforeExpiry parameter in the ClusterClass specification, allowing organizations to set this period to match their certificate management policies.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control EDR 2.4(a)

> Mobile code integrity: Verify the integrity of mobile code before execution. Mobile code authenticity: Verify the authenticity of mobile code before execution. Mobile code authorization: Allow only authorized mobile code to execute. Mobile code security: Provide the capability to enforce a security policy for mobile code. Mobile code execution environment: Run mobile code in a confined environment.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control EDR 2.4(b)

> Mobile code integrity: Verify the integrity of mobile code before execution. Mobile code authenticity: Verify the authenticity of mobile code before execution. Mobile code authorization: Allow only authorized mobile code to execute. Mobile code security: Provide the capability to enforce a security policy for mobile code. Mobile code execution environment: Run mobile code in a confined environment.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control EDR 2.4(c)

> Mobile code integrity: Verify the integrity of mobile code before execution. Mobile code authenticity: Verify the authenticity of mobile code before execution. Mobile code authorization: Allow only authorized mobile code to execute. Mobile code security: Provide the capability to enforce a security policy for mobile code. Mobile code execution environment: Run mobile code in a confined environment.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control EDR 2.13

> Physical tamper resistance and detection: Capability to detect physical tampering of the embedded device. Tamper response: Take defined actions (e.g., zeroize keys) upon tamper detection.

**SCF Controls:** TDA-05.1

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control EDR 2.13(1)

> Physical tamper resistance and detection: Capability to detect physical tampering of the embedded device. Tamper response: Take defined actions (e.g., zeroize keys) upon tamper detection.

**SCF Controls:** TDA-05.2

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides several diagnostic and test interfaces across its component products, along with event logging and alerting capabilities that enable organizations to monitor access to those interfaces.

VCF's diagnostic interfaces include the vSphere DiagnosticManager, a managed object available on both VMware ESX (ESX) and VMware vCenter (vCenter) systems. The DiagnosticManager operates service-wide rather than on a per-session basis, providing operations for generating diagnostic bundles and obtaining information about system logs. The DiagnosticManager API can be accessed through the Managed Object Browser (MOB), and all MOB access is authenticated and logged. The vCenter REST API also exposes an endpoint for validating the current log forwarding configuration, which can optionally send a test diagnostic log message from the vCenter instance to all configured logging servers. On the storage side, vSAN has a network diagnostic mode that, when enabled, creates a ramdisk on a host to collect and save network metrics generated during performance testing. NSX provides the Online Diagnostic System (ODS), which exposes APIs to check debugging reports generated by runbooks.

Access to these diagnostic and test interfaces is captured through VCF's event logging infrastructure. vCenter records management operations and API calls as events, which can be used to track when diagnostic interfaces are invoked. Access to the DiagnosticManager and other vSphere API endpoints is subject to role-based access control and is logged through the vCenter event system. Administrators can define alarm rules in vCenter to monitor specific events, with each alarm rule targeting a single event type. When alarms are triggered, vCenter can send SNMP traps and email notifications to designated recipients with configurable alarm subject information. The SNMP agent included with vCenter sends traps when the vCenter system is started or when an alarm is triggered. The vCenter SNMP agent supports SNMPv1, v2c, and v3 trap formats, with SNMPv3 supporting all security levels including authentication and privacy. The agent can also be configured to filter specific notification types, allowing administrators to control which categories of events are forwarded to SNMP management systems.

VCF Log Management collects and aggregates log data from across the VCF stack, including ESX hosts, vCenter, VCF Operations, and NSX. Log-based alerts can be configured through Symptom Definitions in Infrastructure Operations under Configurations. These alerts can detect patterns in log data that indicate access to diagnostic interfaces and generate corresponding notifications. VCF Operations supports outbound SNMP trap notifications as a mechanism for sending alerts to external SNMP management systems, with the ability to send alert notifications to up to four SNMP trap destinations. Filtering is available when defining notifications using an SNMP Trap destination, allowing administrators to focus alerts on diagnostic interface access events. Alert definitions can be created based on log events from any monitored system, allowing operators to build detection rules tailored to their environment.

VCF Operations for Networks captures audit logs for actions performed through its API, user interface, and CLI, providing visibility into administrative access to network diagnostic tools. VCF Operations for Networks can also forward problem alerts and platform or collector syslogs to remote syslog servers for centralized monitoring.

NSX Manager provides monitoring dashboards showing details regarding system health, networking and security, and compliance reporting, which add visibility into diagnostic activity across the network infrastructure. Platform Operators can configure alerts, perform log analysis and troubleshooting, and run infrastructure diagnostics and health monitoring through the VCF management plane, with all actions logged for audit purposes.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.





## Control EDR 3.2

> Protection from malicious code: Apply malware protection specifically to embedded resources.

**SCF Controls:** EMB-06

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

### Control EDR 3.10

> Support for updates: Embedded device capability to support secure software/firmware updates. Update authenticity: Verify authenticity of updates via digital signatures.

**SCF Controls:** TDA-01.1

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

### **Control EDR 3.10(1)**

> Support for updates: Embedded device capability to support secure software/firmware updates. Update authenticity: Verify authenticity of updates via digital signatures.

**SCF Controls:** EMB-16

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

### Control EDR 3.11

> Physical tamper resistance and detection (SI): Integrity-based tamper detection for hardware. Hardware-based integrity: Use hardware (e.g., TPM) to verify device integrity.

**SCF Controls:** AST-08

**Aggregate Coverage:** Contributes

#### VCF (Contributes)

VCF provides monitoring, alerting, compliance assessment, and forensic capabilities that support periodic inspection of systems and system components for indicators of compromise (IoC). While VCF does not include a dedicated IoC detection engine or endpoint detection and response (EDR) tool, its operational monitoring and change-detection features contribute to an organization's ability to identify anomalous conditions that may signal compromise.

VCF Operations continuously monitors the managed environment by evaluating collected data against defined policies and assigning a compliance score to each object or group based on adherence to those policies. This compliance monitoring enables discovery of non-compliant objects and associated compliance violations, which can surface configuration drift or unauthorized changes that may indicate tampering or compromise. The Anomalies widget displays metric deviations from both dynamic and static thresholds, and the Potential Evidence tab surfaces events, property changes, and anomalous metrics that may be relevant to a given alert. Risk alerts flag conditions that are not immediate threats but warrant prompt investigation.

Configuration drift detection in VCF Operations compares current settings against assigned configuration template values and notifies administrators of any deviations. The Configuration Drift dashboard allows administrators to view and compare drifts and identify settings that have changed, while the Detect Drift function detects drift across all configuration templates and displays results in the Drifts table. This change-detection capability supports identification of unauthorized modifications to managed infrastructure.

VCF Operations provides a layered alerting framework. Log-based alerts can be configured through the Symptom Definitions card in Infrastructure Operations, and alert definitions can be created to monitor infrastructure events and conditions. The system correlates a subset of events published by monitored systems and delivers them as faults, while fault symptoms can be defined based on those events and added to alert definitions. This allows administrators to build detection logic around events that may indicate compromise.

VCF Operations collects security properties for virtual machines, including the connection state of CD-ROM, USB controller, parallel port, and serial port devices, which supports detection of unauthorized device connections to virtual infrastructure. Administrators can filter virtual machines by encryption status, vCenter name, host name, and VM name to assess compliance posture across the environment.

Audit event reporting in VCF Operations records configuration changes, user actions, and authentication events across the managed infrastructure. These audit events can be filtered by authentication status, user, vCenter instance, object type, and event description, giving investigators a detailed view of activity across the environment for conducting security forensics.

VCF Operations includes a Forensics widget that allows administrators to compare data across two time periods, search for and filter objects within the environment, and display widget data based on filtered object sets. This capability supports forensic comparison and analysis when investigating potential indicators of compromise. Guided Network Troubleshooting highlights anomalous entities and metrics within the network path, aiding root-cause analysis during incident investigation.

OS and Application Monitoring Properties are collected for operating systems, application services, remote checks, Linux processes, and Windows services. These monitoring data points can be used to create reports, views, and dashboards for ongoing system inspection, providing visibility into the operational state of managed components and flagging deviations from expected behavior.

VMware ESX hosts generate detailed audit logs, and multiple VCF components support syslog forwarding to external log aggregation and SIEM systems. VMware vCenter provides both API and UI methods to configure syslog forwarding. vSphere

Supervisor can forward control plane and Kubernetes API server audit logs to external syslog servers via Fluent Bit in RFC 5424 format. These log forwarding capabilities allow VCF telemetry to feed into external IoC detection and correlation tools. ESX also enforces integrity checks on installed VIBs through the Host Image Profile Acceptance Level, governing what software is permitted to run on the host and supporting detection of unauthorized additions to the hypervisor. vSphere provides preconfigured alarms that monitor the status of host hardware system boards, allowing notification when hardware-level anomalies are detected.

The NSX networking layer includes monitoring dashboards showing details about system health, networking, security, and compliance reporting. NSX supports logging and auditing capabilities that can be integrated with external logging and monitoring solutions to monitor for security events and anomalies. The System Health Agent on NSX components monitors component statistics, analyzes them, alerts on abnormalities, and can trigger remediation actions when specific conditions are met. VCF Operations for Networks monitors network flows and can detect outlier VMs based on flow metrics including bytes, packets, sessions, and traffic rate, supporting detection of anomalous traffic patterns within the environment. At the host layer, ESX measures boot components against a Trusted Platform Module (TPM) 2.0 hardware root of trust, and VMware vCenter exposes the resulting boot measurements and TPM event logs through its trusted-infrastructure API, so administrators can confirm a host booted in a known-good state and detect firmware or boot-level tampering. Secure Boot independently validates the cryptographic signatures of the ESX kernel and modules at startup, so unsigned or altered components are surfaced as an integrity failure rather than loaded silently.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer contributes to this control through its application-layer threat detection and event logging capabilities, which provide the data security teams need during periodic Indicators of Compromise (IoC) inspections.

Avi's Web Application Firewall (WAF) analyzes all incoming and outgoing application traffic using signature-based detection, a positive security model, client reputation scoring, machine learning, and outlier analysis. WAF signatures include detection for OWASP Top 10 attack vectors and other threat categories. The WAF operates in either Detection mode or Enforcement mode; Detection mode allows new rules to be evaluated without blocking legitimate traffic, making it suitable for baseline establishment and traffic anomaly review during periodic inspections. Partial detection allows individual rules to be placed in Enforcement mode while the policy as a whole remains in Detection mode, supporting incremental adoption.

The Avi Controller generates structured security events that auditors and security teams can review as part of periodic IoC inspections. The Controller tracks denial-of-service attack events with attack count metrics and generates alerts at configurable severity levels. The scan detection system dynamically classifies client IPs into Good, Bad, and Unknown groups based on observed traffic patterns, adjusting automatically as patterns change. The Controller also logs the initiator and target of failed trusted channel establishment attempts, including cases where an SSL certificate is absent, providing visibility into authentication anomalies.

Avi events serve as an audit trail for user logins, configuration changes, and runtime state changes, and can be forwarded to Splunk for centralized audit trail analysis. Client request logs can be collected automatically based on categories of significance for application-level review. System events provide a historical record of faults within the system. DataScripts support custom identification of suspicious traffic and execution of appropriate mitigation actions, extending detection coverage beyond built-in signatures.

Avi provides several tools suited for periodic review workflows. The application security report can be generated on demand through the Controller UI or CLI, with an optional duration parameter to define the report period, producing a point-in-time record of detected threats suitable for scheduled IoC review cycles. The Config Audit Trail, accessible at Operations > Events > Config Audit Trail, presents a filtered view of user activity and configuration change events over a selected period, supporting review of whether configuration changes preceded observed anomalies. Alert events can be streamed to external syslog servers using Syslog Audit Persistence, enabling integration with SIEM platforms that support automated IoC correlation. The Controller also generates system reports for upgrade operations containing pre-check results and operational status, which support verification of system state before and after changes.

The organization is responsible for defining the inspection cadence, determining what constitutes a compromise indicator in context, and performing the host-level and physical inspection that Avi does not address as an application delivery controller. Platform-level system integrity inspection relies on the underlying VCF platform.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

### **Control EDR 3.11(1)**

> Physical tamper resistance and detection (SI): Integrity-based tamper detection for hardware. Hardware-based integrity: Use hardware (e.g., TPM) to verify device integrity.

**SCF Controls:** EMB-05

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.



## Control EDR 3.12

> Provisioning Product: Ensure secure unique identity provisioning during manufacturing.

**SCF Controls:** AST-18

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF contributes to roots of trust protection through cryptographic key management, certificate trust chain infrastructure, hardware-based integrity verification, and software package signing across its component stack.

VCF manages cryptographic keys through integration with external Key Management Servers (KMS) that support the Key Management Interoperability Protocol (KMIP) 1.1 standard, or through the built-in vSphere Native Key Provider. VMware vCenter communicates with external KMS instances using the KMIP protocol, and the KMS must be capable of acting as a symmetric key foundry and server. The vSphere Native Key Provider can coexist with an existing key server infrastructure, giving organizations flexibility in how they provision and manage encryption keys. Trust between vCenter and a KMS can be established through multiple methods: uploading a KMS certificate directly, using the vCenter Root CA certificate, providing an uploaded certificate and private key, or generating a Certificate Signing Request for the KMS to sign. The vSphere Web Services API exposes an `establishTrust()` method that enables mutual trust between vCenter and a KMS server using self-signed certificates, signed certificate signing requests, or certificate and key pair exchange. For stronger security, administrators should upload a leaf certificate or intermediate CA certificate rather than a root CA certificate, since uploading a root CA causes vCenter to trust all certificates signed by that authority.

vCenter maintains a trusted root certificate store where administrators can add root certificates or certificate chains from enterprise or third-party certificate authorities. Adding a root certificate to this store establishes trust with that CA for all subsequent certificate validation across the vCenter environment. The `TrustedRootChains` API provides programmatic access to manage and retrieve root certificate chains by identifier, supporting automation of trust chain verification. The underlying certificate storage uses the VMware Endpoint Certificate Store (VECS), which maintains named stores including `TRUSTED_ROOTS` for all trusted root certificates and `TRUSTED_ROOTS_CRLS` for certificate revocation list data; administrators can manage these stores with the `vecs-cli` command-line tool. The `TRUSTED_ROOTS_CRLS` store must not be deleted or modified, as doing so can damage the certificate infrastructure. Control plane VMs in vSphere Supervisor environments rely on a Trusted Roots bundle that must be kept current with the latest vCenter root certificates.

At the hardware level, VCF supports Secure Boot and Trusted Platform Module (TPM) to help verify that hosts boot in known-good configurations. The vSphere Security Configuration Guide recommends enabling Secure Boot and TPM as part of VMware ESX host hardening. vCenter provides API interfaces for retrieving TPM event logs for configuring firmware trust with an attestation service or validating boot-time TPM measurements, and for listing configured TPM endorsement keys on a host. These capabilities support establishing a hardware-rooted chain of trust from firmware through the hypervisor.

Beyond core key and certificate management, several VCF components maintain their own trust infrastructure. VCF Operations orchestrator supports the import, export, and generation of package signing certificates, providing a mechanism for verifying the integrity and authenticity of software packages deployed within the environment. VCF Operations also maintains an SSL truststore with a password-protected keystore for managing certificates used in secure communications. VCF Automation can establish trust relationships with external servers by verifying and accepting certificate information for SSL communication, and it exchanges SAML service provider metadata with external identity providers to establish federated trust relationships. NSX components establish trust through certificate exchange between Local Manager and Global Manager instances during federation setup. VCF PowerCLI maintains its own Trusted Certificate Store where trusted certificates are saved and used when connecting to server systems, extending the trust chain to administrative automation tooling.

VCF provides the cryptographic infrastructure for establishing, storing, and verifying trust relationships through certificates and keys across all its major components. However, the organizational processes for provisioning supplier keys, verifying supply chain integrity of cryptographic material, defining key protection policies, and physically protecting key storage remain the responsibility of the deploying organization.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control EDR 3.13(a)

> Integrity of boot process: Verify firmware integrity before execution. Boot process authenticity: Verify firmware authenticity before execution. Secure boot: Provide a hardware-rooted secure boot process.

**SCF Controls:** AST-18

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF contributes to roots of trust protection through cryptographic key management, certificate trust chain infrastructure, hardware-based integrity verification, and software package signing across its component stack.

VCF manages cryptographic keys through integration with external Key Management Servers (KMS) that support the Key Management Interoperability Protocol (KMIP) 1.1 standard, or through the built-in vSphere Native Key Provider. VMware vCenter communicates with external KMS instances using the KMIP protocol, and the KMS must be capable of acting as a symmetric key foundry and server. The vSphere Native Key Provider can coexist with an existing key server infrastructure, giving organizations flexibility in how they provision and manage encryption keys. Trust between vCenter and a KMS can be established through multiple methods: uploading a KMS certificate directly, using the vCenter Root CA certificate, providing an uploaded certificate and private key, or generating a Certificate Signing Request for the KMS to sign. The vSphere Web Services API exposes an `establishTrust()` method that enables mutual trust between vCenter and a KMS server using self-signed certificates, signed certificate signing requests, or certificate and key pair exchange. For stronger security, administrators should upload a leaf certificate or intermediate CA certificate rather than a root CA certificate, since uploading a root CA causes vCenter to trust all certificates signed by that authority.

vCenter maintains a trusted root certificate store where administrators can add root certificates or certificate chains from enterprise or third-party certificate authorities. Adding a root certificate to this store establishes trust with that CA for all subsequent certificate validation across the vCenter environment. The `TrustedRootChains` API provides programmatic access to manage and retrieve root certificate chains by identifier, supporting automation of trust chain verification. The underlying certificate storage uses the VMware Endpoint Certificate Store (VECS), which maintains named stores including `TRUSTED_ROOTS` for all trusted root certificates and `TRUSTED_ROOTS_CRLS` for certificate revocation list data; administrators can manage these stores with the `vecs-cli` command-line tool. The `TRUSTED_ROOTS_CRLS` store must not be deleted or modified, as doing so can damage the certificate infrastructure. Control plane VMs in vSphere Supervisor environments rely on a Trusted Roots bundle that must be kept current with the latest vCenter root certificates.

At the hardware level, VCF supports Secure Boot and Trusted Platform Module (TPM) to help verify that hosts boot in known-good configurations. The vSphere Security Configuration Guide recommends enabling Secure Boot and TPM as part of VMware ESX host hardening. vCenter provides API interfaces for retrieving TPM event logs for configuring firmware trust with an attestation service or validating boot-time TPM measurements, and for listing configured TPM endorsement keys on a host. These capabilities support establishing a hardware-rooted chain of trust from firmware through the hypervisor.

Beyond core key and certificate management, several VCF components maintain their own trust infrastructure. VCF Operations orchestrator supports the import, export, and generation of package signing certificates, providing a mechanism for verifying the integrity and authenticity of software packages deployed within the environment. VCF Operations also maintains an SSL truststore with a password-protected keystore for managing certificates used in secure communications. VCF Automation can establish trust relationships with external servers by verifying and accepting certificate information for SSL communication, and it exchanges SAML service provider metadata with external identity providers to establish federated trust relationships. NSX components establish trust through certificate exchange between Local Manager and Global Manager instances during federation setup. VCF PowerCLI maintains its own Trusted Certificate Store where trusted certificates are saved and used when connecting to server systems, extending the trust chain to administrative automation tooling.

VCF provides the cryptographic infrastructure for establishing, storing, and verifying trust relationships through certificates and keys across all its major components. However, the organizational processes for provisioning supplier keys, verifying

supply chain integrity of cryptographic material, defining key protection policies, and physically protecting key storage remain the responsibility of the deploying organization.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control EDR 3.13(b)

> Integrity of boot process: Verify firmware integrity before execution. Boot process authenticity: Verify firmware authenticity before execution. Secure boot: Provide a hardware-rooted secure boot process.

**SCF Controls:** AST-18

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF contributes to roots of trust protection through cryptographic key management, certificate trust chain infrastructure, hardware-based integrity verification, and software package signing across its component stack.

VCF manages cryptographic keys through integration with external Key Management Servers (KMS) that support the Key Management Interoperability Protocol (KMIP) 1.1 standard, or through the built-in vSphere Native Key Provider. VMware vCenter communicates with external KMS instances using the KMIP protocol, and the KMS must be capable of acting as a symmetric key foundry and server. The vSphere Native Key Provider can coexist with an existing key server infrastructure, giving organizations flexibility in how they provision and manage encryption keys. Trust between vCenter and a KMS can be established through multiple methods: uploading a KMS certificate directly, using the vCenter Root CA certificate, providing an uploaded certificate and private key, or generating a Certificate Signing Request for the KMS to sign. The vSphere Web Services API exposes an `establishTrust()` method that enables mutual trust between vCenter and a KMS server using self-signed certificates, signed certificate signing requests, or certificate and key pair exchange. For stronger security, administrators should upload a leaf certificate or intermediate CA certificate rather than a root CA certificate, since uploading a root CA causes vCenter to trust all certificates signed by that authority.

vCenter maintains a trusted root certificate store where administrators can add root certificates or certificate chains from enterprise or third-party certificate authorities. Adding a root certificate to this store establishes trust with that CA for all subsequent certificate validation across the vCenter environment. The `TrustedRootChains` API provides programmatic access to manage and retrieve root certificate chains by identifier, supporting automation of trust chain verification. The underlying certificate storage uses the VMware Endpoint Certificate Store (VECS), which maintains named stores including `TRUSTED_ROOTS` for all trusted root certificates and `TRUSTED_ROOTS_CRLS` for certificate revocation list data; administrators can manage these stores with the `vecs-cli` command-line tool. The `TRUSTED_ROOTS_CRLS` store must not be deleted or modified, as doing so can damage the certificate infrastructure. Control plane VMs in vSphere Supervisor environments rely on a Trusted Roots bundle that must be kept current with the latest vCenter root certificates.

At the hardware level, VCF supports Secure Boot and Trusted Platform Module (TPM) to help verify that hosts boot in known-good configurations. The vSphere Security Configuration Guide recommends enabling Secure Boot and TPM as part of VMware ESX host hardening. vCenter provides API interfaces for retrieving TPM event logs for configuring firmware trust with an attestation service or validating boot-time TPM measurements, and for listing configured TPM endorsement keys on a host. These capabilities support establishing a hardware-rooted chain of trust from firmware through the hypervisor.

Beyond core key and certificate management, several VCF components maintain their own trust infrastructure. VCF Operations orchestrator supports the import, export, and generation of package signing certificates, providing a mechanism for verifying the integrity and authenticity of software packages deployed within the environment. VCF Operations also maintains an SSL truststore with a password-protected keystore for managing certificates used in secure communications. VCF Automation can establish trust relationships with external servers by verifying and accepting certificate information for SSL communication, and it exchanges SAML service provider metadata with external identity providers to establish federated trust relationships. NSX components establish trust through certificate exchange between Local Manager and Global Manager instances during federation setup. VCF PowerCLI maintains its own Trusted Certificate Store where trusted certificates are saved and used when connecting to server systems, extending the trust chain to administrative automation tooling.

VCF provides the cryptographic infrastructure for establishing, storing, and verifying trust relationships through certificates and keys across all its major components. However, the organizational processes for provisioning supplier keys, verifying

supply chain integrity of cryptographic material, defining key protection policies, and physically protecting key storage remain the responsibility of the deploying organization.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control EDR 3.14

> Integrity of data at rest: Protect embedded storage from unauthorized modification. Cryptographic integrity: Use cryptography to protect data at rest integrity.

**SCF Controls:** END-06.5

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides automated mechanisms to verify the integrity of the boot process for both physical hosts and virtual machines, spanning hardware-rooted trust, software specification validation, and post-boot health verification.

ESX hosts support UEFI Secure Boot, which uses cryptographic signature verification to validate each component loaded during the boot sequence. When Secure Boot is enabled, the host firmware checks the digital signature of the bootloader, and each subsequent stage verifies the next, creating a chain of trust from firmware through the hypervisor kernel. The vSphere Security Configuration Guide recommends enabling Secure Boot on all ESX hosts as part of standard hardening. ESX also supports an `execInstalledOnly` enforcement mode, built on top of UEFI Secure Boot, which restricts execution to software installed through the managed package process and prevents unauthorized binaries from running on the host.

VCF also supports Trusted Platform Module (TPM) integration on ESX hosts. TPM devices perform measured boot, recording cryptographic hashes of each boot component into platform configuration registers. VMware vCenter provides programmatic access to TPM device information through the trusted infrastructure API, including retrieval of TPM event logs that record boot-time measurements. These event logs can be used with an attestation service to validate that hosts booted with known-good firmware and software configurations. The Tpm interface from the trusted infrastructure package allows administrators to list TPM devices on a host and retrieve detailed information including manufacturer, model, and firmware version. vCenter also supports unsealing secrets bound to an endorsement key to verify reported measurements from the TPM event log, providing a second verification path for boot integrity. ESX host TPM modules must run the latest 2.0 firmware when in use, and must be disabled in the BIOS if not in use, keeping the hardware trust baseline consistent and verifiable.

Remote attestation capabilities allow vCenter to verify host boot integrity by comparing TPM measurements against expected values, providing an automated mechanism to detect tampering or unexpected changes in the boot process of managed hosts.

For virtual machines, VCF supports UEFI Secure Boot at the VM level, allowing guest operating systems to verify boot component integrity through the same cryptographic chain-of-trust mechanism used on physical hardware. VM boot firmware can be configured to EFI type using the vSphere Automation SDK `Boot.UpdateSpec()` method, and boot options including EFI firmware type and boot behavior settings are configurable through both the vSphere Client and the vSphere Automation API. VM-level Secure Boot is supported starting with Hardware Version 13. For Windows virtual machines, vSphere also supports Configurable Code Integrity under Virtualization-Based Security (VBS), which enforces that only trusted, signed code runs from the boot loader onwards within the guest.

At the software specification level, VCF's lifecycle management includes automated validation before any software is applied to hosts. The Drafts service offers a `validate_Task` method that checks whether a draft software specification is complete and identifies any conflicts between draft components or unresolved dependencies. Before saving a draft and turning it into the desired state for a cluster or standalone host, the specification should be validated to confirm it is complete and valid. This automated pre-deployment validation helps ensure that hosts receive verified, conflict-free software configurations, reducing the risk of boot failures or integrity issues caused by incompatible components. VCF also supports the creation of exported ISO images from desired software specifications for use in clean installs and bootstrapping, allowing administrators to provision systems from a known, validated software baseline.

For post-boot verification, VCF provides automated checking mechanisms across its management components. The Supportability and Serviceability tool (SoS) on VCF Operations includes a health check function that verifies system state after operations such as a vCenter instance restore, confirming that all components have started correctly and are operating as expected. VCF Operations Cloud Proxy implements automated first boot verification by checking for the presence of a



"Succeeded" file in /var/log/firstboot, providing a programmatic way to confirm that the boot process completed without error.

Auditors evaluating a VCF deployment for this control should verify that UEFI Secure Boot is enabled on all ESX hosts, that TPM 2.0 modules are active and running current firmware, that lifecycle management workflows use the software specification validation step before applying updates, that SoS health checks are run after restore or maintenance operations, and that boot verification logs are monitored for component deployments.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control EDR 3.14(1)

> Integrity of data at rest: Protect embedded storage from unauthorized modification. Cryptographic integrity: Use cryptography to protect data at rest integrity.

**SCF Controls:** END-06.6

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides multiple automated mechanisms that protect the integrity of boot firmware across both physical VMware ESX hosts and virtual machines.

ESX hosts support UEFI Secure Boot, which validates the cryptographic signature of each component in the boot chain before allowing it to execute. When Secure Boot is enabled, the host firmware restricts the boot process to only authorized, signed code, protecting the hypervisor from unauthorized modification. The vSphere Security Configuration Guide recommends enabling Secure Boot on all ESX hosts as a standard hardening measure, alongside Trusted Platform Module (TPM) enablement. Secure Boot enforcement can be activated or deactivated through ESXCLI, and ESX also supports `execInstalledOnly` enforcement, a control built on top of UEFI Secure Boot that restricts execution to only installed binaries.

TPM 2.0 integration on ESX hosts provides measured boot capabilities. The TPM records cryptographic hashes of each boot stage into hardware-protected platform configuration registers, creating a tamper-evident chain of trust from firmware through the hypervisor. If any boot component is altered, the TPM measurements will differ from expected values. The TPM event log can be retrieved for configuring firmware trust with an attestation service or for validating boot time measurements. VMware vCenter provides programmatic access to TPM device information, including manufacturer and firmware version details, through the trusted infrastructure API. TPM endorsement keys used to establish hardware identity and trust are also managed through this API. An attestation service can compare measurements against known-good values to confirm that boot firmware and subsequent components have not been modified.

For virtual machines, VCF supports the Extensible Firmware Interface (EFI) as a virtual hardware feature, enabling UEFI Secure Boot for guest operating systems. Virtual machine firmware type can be configured to EFI through the vSphere Client or programmatically through the vSphere Automation SDK using the `Boot.UpdateSpec` API or the `BootTypes.UpdateSpec.Builder setType()` method. UEFI Secure Boot at the VM level protects guest operating system boot firmware using the same signature verification approach as physical hosts. For Windows virtual machines, vSphere supports Virtualization-based Security (VBS) with Configurable Code Integrity, which restricts execution to trusted code from the boot loader onward within the guest operating system. VCF also supports hardware memory encryption for confidential workloads through technologies like SEV-ES, which protects VM state from inspection or modification at the hardware level.

vSphere Lifecycle Manager provides automated firmware lifecycle management for ESX hosts. Firmware add-ons can be added, replaced, removed, upgraded, or downgraded through vSphere Lifecycle Manager images, allowing administrators to maintain firmware at vendor-validated versions across the host fleet. vSphere Lifecycle Manager images consist of a base image, vendor add-ons, components, and firmware add-ons that can be independently retrieved and applied to hosts. This automated approach helps keep host firmware aligned with tested baselines and reduces the window during which outdated firmware is in use.

VCF Operations provides monitoring for firmware integrity through management packs. The Management Pack for Lenovo Compute generates alerts for firmware unrecoverable boot device failure events detected by UEFI, as well as firmware update failed alerts when a firmware update does not complete normally and the target image is partially written. These automated alerts allow operations teams to detect and respond to firmware integrity issues.

VMware publishes security advisories for known firmware and software vulnerabilities affecting VCF components, with remediation guidance for each advisory.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control HDR 2.4

> Host mobile code integrity: Verify integrity of mobile code on host. Host mobile code authenticity: Verify authenticity of mobile code on host. Host mobile code authorization: Enforce authorized execution on host. Host mobile code security: Enforce security policies for host-based mobile code. Sandboxing: Execute host mobile code in a restricted environment.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control HDR 2.4(1)

> Host mobile code integrity: Verify integrity of mobile code on host. Host mobile code authenticity: Verify authenticity of mobile code on host. Host mobile code authorization: Enforce authorized execution on host. Host mobile code security: Enforce security policies for host-based mobile code. Sandboxing: Execute host mobile code in a restricted environment.

**SCF Controls:** EMB-16

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control HDR 2.4(a)

> Host mobile code integrity: Verify integrity of mobile code on host. Host mobile code authenticity: Verify authenticity of mobile code on host. Host mobile code authorization: Enforce authorized execution on host. Host mobile code security: Enforce security policies for host-based mobile code. Sandboxing: Execute host mobile code in a restricted environment.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control HDR 2.4(b)

> Host mobile code integrity: Verify integrity of mobile code on host. Host mobile code authenticity: Verify authenticity of mobile code on host. Host mobile code authorization: Enforce authorized execution on host. Host mobile code security: Enforce security policies for host-based mobile code. Sandboxing: Execute host mobile code in a restricted environment.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.



## Control HDR 2.4(c)

> Host mobile code integrity: Verify integrity of mobile code on host. Host mobile code authenticity: Verify authenticity of mobile code on host. Host mobile code authorization: Enforce authorized execution on host. Host mobile code security: Enforce security policies for host-based mobile code. Sandboxing: Execute host mobile code in a restricted environment.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control HDR 2.13

> Host physical tamper detection: Capability to detect host chassis tampering. Host tamper response: Audit and alert upon host physical breach.

**SCF Controls:** TDA-05.1

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control HDR 2.13(1)

> Host physical tamper detection: Capability to detect host chassis tampering. Host tamper response: Audit and alert upon host physical breach.

**SCF Controls:** TDA-05.2

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides several diagnostic and test interfaces across its component products, along with event logging and alerting capabilities that enable organizations to monitor access to those interfaces.

VCF's diagnostic interfaces include the vSphere DiagnosticManager, a managed object available on both VMware ESX (ESX) and VMware vCenter (vCenter) systems. The DiagnosticManager operates service-wide rather than on a per-session basis, providing operations for generating diagnostic bundles and obtaining information about system logs. The DiagnosticManager API can be accessed through the Managed Object Browser (MOB), and all MOB access is authenticated and logged. The vCenter REST API also exposes an endpoint for validating the current log forwarding configuration, which can optionally send a test diagnostic log message from the vCenter instance to all configured logging servers. On the storage side, vSAN has a network diagnostic mode that, when enabled, creates a ramdisk on a host to collect and save network metrics generated during performance testing. NSX provides the Online Diagnostic System (ODS), which exposes APIs to check debugging reports generated by runbooks.

Access to these diagnostic and test interfaces is captured through VCF's event logging infrastructure. vCenter records management operations and API calls as events, which can be used to track when diagnostic interfaces are invoked. Access to the DiagnosticManager and other vSphere API endpoints is subject to role-based access control and is logged through the vCenter event system. Administrators can define alarm rules in vCenter to monitor specific events, with each alarm rule targeting a single event type. When alarms are triggered, vCenter can send SNMP traps and email notifications to designated recipients with configurable alarm subject information. The SNMP agent included with vCenter sends traps when the vCenter system is started or when an alarm is triggered. The vCenter SNMP agent supports SNMPv1, v2c, and v3 trap formats, with SNMPv3 supporting all security levels including authentication and privacy. The agent can also be configured to filter specific notification types, allowing administrators to control which categories of events are forwarded to SNMP management systems.

VCF Log Management collects and aggregates log data from across the VCF stack, including ESX hosts, vCenter, VCF Operations, and NSX. Log-based alerts can be configured through Symptom Definitions in Infrastructure Operations under Configurations. These alerts can detect patterns in log data that indicate access to diagnostic interfaces and generate corresponding notifications. VCF Operations supports outbound SNMP trap notifications as a mechanism for sending alerts to external SNMP management systems, with the ability to send alert notifications to up to four SNMP trap destinations. Filtering is available when defining notifications using an SNMP Trap destination, allowing administrators to focus alerts on diagnostic interface access events. Alert definitions can be created based on log events from any monitored system, allowing operators to build detection rules tailored to their environment.

VCF Operations for Networks captures audit logs for actions performed through its API, user interface, and CLI, providing visibility into administrative access to network diagnostic tools. VCF Operations for Networks can also forward problem alerts and platform or collector syslogs to remote syslog servers for centralized monitoring.

NSX Manager provides monitoring dashboards showing details regarding system health, networking and security, and compliance reporting, which add visibility into diagnostic activity across the network infrastructure. Platform Operators can configure alerts, perform log analysis and troubleshooting, and run infrastructure diagnostics and health monitoring through the VCF management plane, with all actions logged for audit purposes.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.



## Control HDR 3.2

> Host malware protection: Protection from malicious code for host devices. Centralized management: Centrally manage host malware signatures.

**SCF Controls:** END-04

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides several platform capabilities that support malicious code protection, though organizations must deploy additional tools to achieve full antimalware coverage across endpoints.

At the hypervisor layer, VMware ESX supports Secure Boot and code signing to restrict unauthorized code from executing on hosts. Trusted Platform Module (TPM) support enables boot-time integrity measurement, and VMware vCenter provides APIs to retrieve TPM event logs for validating boot-time measurements or configuring firmware trust with an attestation service. Administrators can verify that hosts have booted in a known good configuration by checking TPM device status and event logs through the vCenter trusted infrastructure API. ESX host TPM modules must be verified as running the latest 2.0 firmware, and if not in use, disabled in the BIOS. Virtual machines can be configured with UEFI firmware for guest operating system boot integrity. Windows guest operating systems support Virtualization Based Security (VBS), which enables Device Guard to help prevent malware from running and Credential Guard to protect credential secrets; these features are configurable through Group Policy on VMs with ESX 6.7 or later compatibility. Virtual machines can also have the Guest Integrity (GI) security device enabled through the VMware Host Client, which is the platform foundation for VMware AppDefense.

The vSphere Security Configuration Guide provides hardening recommendations for ESX hosts, including configuring authentication and authorization settings, disabling unnecessary services, and enabling security features like Secure Boot and TPM. These configuration baselines are industry-standard best practices for host hardening.

vSphere Lifecycle Manager and VCF Operations Configuration Profiles maintain desired-state configuration for ESX hosts. Configuration Drifts in VCF Operations provides a proactive method to monitor configuration changes and identify when vCenter instances and vSphere clusters deviate from their approved baselines. Drift status is updated through a collection cycle that runs every 8 hours. This helps identify when host software or configuration has been modified from its approved state, which can indicate unauthorized changes or tampering.

VCF's networking layer, NSX, supports integration with third-party endpoint protection solutions for virtual machine security. The platform documentation references specific partner integrations including Symantec Endpoint Protection and ESET Antivirus/Security software, which provide anti-malware, intrusion prevention, and firewall capabilities for guest virtual machines. NSX App IDs can be used in conjunction with these third-party solutions for network management and security policy enforcement. Organizations can implement security groups, define security policies, and use NSX's integration with third-party security solutions for virtual machine protection.

NSX Guest Introspection provides an agent-based framework that supports endpoint security within virtual machines. The Guest Introspection Agent must be installed and running in each guest VM, and logged-in users should not have the privilege to remove or stop the agent. This architecture enables third-party security solutions to inspect guest operating system activity without requiring agents to be installed directly on the hypervisor. The security and integrity of the guest operating system must be maintained for these integrations to function correctly.

For container workloads, the VCF CLI supports cosign signature verification for container images through the `imgpkg` tool, which enables finding and copying cosign signatures. Image pull authentication through `imagePullSecrets` restricts container image pulls to authenticated private registries. Plugin discovery image signature verification is supported, with configurable public key paths for the cosign verifier. These capabilities help restrict deployment of images that have been tampered with.

VCF Operations supports the deployment and monitoring of agents on endpoint virtual machines, with guest operation privileges required to install agents. Automated software update capabilities in vCenter help keep systems current with security patches, reducing the window of exposure to known malware exploits.

For organizations that deploy VMware vDefend as an add-on to VCF, distributed malware prevention capabilities become available. The NSX Application Platform hosts the Malware Prevention for vDefend feature, which can be deployed from the Local Manager UI in NSX Federation environments. When host clusters are activated for Distributed Malware Prevention, a service virtual machine (SVM) is deployed on each host to perform malware scanning. vDefend also supports IDS/IPS capabilities alongside distributed malware prevention. The DistributedMps runbook is available to verify the health of the Malware Prevention Service pipeline and diagnose issues such as protection not working on a particular VM or certain files not being scanned.

Malware prevention activities generate log data for audit and forensic purposes. The Malware Prevention for vDefend feature writes log files that can be used for troubleshooting and security monitoring, and remote logging can be configured on the malware prevention SVMs for centralized log collection.

VMware Private AI Services (PAIS) includes model validation capabilities that scan machine learning model files for malicious content before they are accepted into a Model Gallery. The validation runs as a sandbox process on Deep Learning VMs and covers integrity verification through hash code checking, malware scanning, serialization attack scanning, and inference behavior testing with NVIDIA Triton Inference Server. Both the malware and serialization checks run as part of the validation workflow required before a model can be uploaded to a Model Gallery and made available to downstream model endpoints, knowledge bases, or AI agents.

Machine learning model files are a category of artifact that traditional endpoint security tools may not inspect. Serialized models in formats such as Python pickle files can carry executable payloads that activate when an application or researcher loads the model. The PAIS Model Gallery intake workflow treats this upload boundary as a security checkpoint, requiring each artifact to pass hash, malware, serialization, and inference behavior tests before it is published for use. PAIS guidance also directs organizations to distribute only models that have been validated internally on a Deep Learning VM rather than pulling artifacts directly from the Internet, where models may contain malicious code or be tuned for malicious behavior. The validation is operated by MLOps engineers, who confirm that onboarded models meet the organization's security, privacy, and technical requirements.

For platform-level malware protection covering virtual machines, VMware ESX hosts, and general workloads running on the PAIS infrastructure, organizations rely on VCF platform controls and, where deployed, VMware vDefend. PAIS adds the specialized scanning layer applied to AI model artifacts at the point of intake, addressing a threat surface that general-purpose antimalware tools do not typically cover.

### **VMware vDefend (Contributes)**

VMware vDefend provides multiple layers of malicious code detection and prevention through its Malware Prevention Service and IDS/IPS capabilities. Together these address the detection aspect of this control directly and contribute to the eradication aspect by restricting malicious code from executing and spreading across the network.

The Malware Prevention Service deploys as a service virtual machine (SVM) on each host in a cluster, with Guest Introspection thin agents installed on each protected guest VM. This architecture separates the detection engine from the protected workload, giving the service visibility into file activity, process execution, and network events from outside the guest. On Linux guests, a File and Network Agent handles file and network traffic introspection. A Host Agent (MUX) component running on ESX hosts supports the Guest Introspection functionality.

The service detects both known and unknown malicious files, including zero-day threats. Supported file categories include Windows executables (.exe, self-extracting), Linux executables (.elf), installer files (.msi), and shortcut files (.lnk), among others. When a file is analyzed, Malware Prevention assigns a verdict and classifies the threat by malware class (virus, trojan horse, worm, adware, ransomware, spyware) and malware family. A single detection event can be associated with multiple malware families and classes. Upon detection, the service can take direct remediation actions, including denying access to suspicious files or deleting files identified as malicious. An allowlist capability enables administrators to override verdicts for files known to be legitimate, suppressing false positives without disabling broader detection.

Beyond file-based detection, Malware Prevention Service includes fileless malware detection to identify attacks that operate entirely within system memory using existing binaries and in-memory operations such as PowerShell or VBScript, without

leaving executable artifacts on disk. The service monitors Indicators of Compromise including process command lines, process hashes, process execution trees, and in-memory executions. In-memory script buffer executions are scored for threat severity and assigned a verdict of malicious or suspicious, with the associated malware family and class displayed on the Malware Prevention Dashboard.

When malware is detected, the service extracts artifacts and in-memory script buffers that security teams can use to define incidents of compromise and create new detection rules for similar future threats. This extends malware coverage as new threat patterns are identified within the environment.

The Malware Prevention Dashboard in Security Services Platform aggregates detection events for malicious files, suspicious files, and uninspected files over configurable time periods. Each unique file is tracked by hash, and events can be filtered by verdict criteria. This centralized visibility allows security teams to identify malware incidents, track detection trends, and coordinate eradication efforts across the virtualized environment.

vDefend IDS/IPS provides signature-based and behavioral intrusion detection and prevention at the distributed firewall level, complementing Malware Prevention with network-layer detection of malware delivery attempts, command-and-control communications, and lateral movement patterns associated with malware infections. IDS/IPS operates at the virtual network interface level, providing visibility into east-west traffic between workloads that perimeter security tools cannot inspect.

While vDefend provides strong detection and file-level remediation capabilities, complete eradication of malicious code from infected guest operating systems typically requires endpoint security agents operating inside the guest. vDefend restricts malware from spreading across the network and blocks or removes detected malicious files, but cleaning an already-infected system requires host-level remediation tooling.

#### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer contributes to malicious code protection at the application delivery layer through WAF-based payload inspection, IP Reputation threat intelligence, Application Rules, and Bot Management.

The Avi WAF inspects inbound HTTP/HTTPS request bodies and parameters against signature-based rules, including ModSecurity rules, to detect and block exploit payloads, injection attacks, and known malware delivery patterns. WAF rules are configurable per virtual service and can be tuned to reduce false positives while maintaining coverage of malicious traffic patterns. Configuration is managed through the Avi Controller UI under the WAF Policy section for each virtual service.

The Application Rules service, available through Avi Cloud Console Live Security Threat Intelligence, provides automatically updated WAF rules targeting known vulnerabilities with CVE mappings across more than 5,000 applications, including WordPress, Drupal, and Apache. These rules are pushed automatically to Avi Controllers, reducing the window between vulnerability disclosure and protection.

Avi's IP Reputation service provides a threat intelligence feed that classifies IP addresses known to be infected with malware or identified as contacting malware distribution points (threat category code 6). The service also identifies IP addresses known to distribute malware, shell code, rootkits, worms, or viruses, including Windows exploit IP addresses. An "All threats" classification (threat category code 32) can be applied to block any suspicious IP addresses at the application delivery layer. IP Reputation data is synchronized via Avi Cloud Services and can be referenced in virtual service policies to block malicious source traffic before it reaches protected workloads. This is configured under Administration > External Services > Cloud Services.

Avi Bot Management provides a default System-BotDetectionPolicy that can be bound to virtual services to detect and mitigate malicious bots used in online fraud, credential stuffing, and automated data scraping. Bot Management performs heuristic scanning of User-Agent strings to detect malicious patterns such as SQL injections, marking such requests as bad bot of type web attack. In Kubernetes deployments, the Avi Kubernetes Operator (AKO) supports assigning a BotDetectionPolicy to virtual services via the botPolicyRef parameter, extending bot detection to Kubernetes ingress workloads.

Avi WAF does not include built-in virus or file-content malware scanning. Avi documentation explicitly recommends deploying a dedicated virus and malware scanning tool on the upload directories of protected applications to detect and mitigate malicious uploads, including uploads that may bypass WAF inspection. Malware detection and eradication for workloads



running behind Avi is the responsibility of endpoint protection tools deployed on those workloads. Organizations should configure WAF policies, Application Rules, IP Reputation, and Bot Management on Avi virtual services while also deploying dedicated antimalware solutions on application workloads and their supporting infrastructure.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control HDR 3.2(1)

> Host malware protection: Protection from malicious code for host devices. Centralized management: Centrally manage host malware signatures.

**SCF Controls:** END-04.3

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides infrastructure-level capabilities that support the centralized management of antimalware technologies, though it does not itself function as a general-purpose endpoint antimalware solution. VCF's contribution spans host integrity management, guest introspection frameworks for third-party security integration, distributed malware prevention through VMware vDefend, agent deployment infrastructure, and operational monitoring.

VMware vCenter provides centralized management of the virtualization environment, including host attestation monitoring and Secure Boot enforcement. vSphere Lifecycle Manager manages all hosts in a cluster collectively with a single image, enabling desired-state configuration with drift detection. Upon remediation, the defined image is installed on all hosts in the cluster, and the manager can compare the current image on a host against a desired image to define the compatibility status. VCF Operations displays drift status for vSphere clusters that have vSphere Configuration Profiles enabled, with a collection cycle that updates drift data every eight hours. These capabilities help maintain a known-good baseline across all hosts, supporting antimalware efforts by detecting unauthorized changes to host software.

NSX includes a Guest Introspection framework that enables third-party endpoint security products to operate within the virtualized environment. The NSX Guest Introspection Agent is installed in each guest virtual machine to provide security context, including user identity information. Security administrators are responsible for ensuring the agent is installed and running in each guest VM, and logged-in users should not have the privilege to remove or stop the agent. This framework allows third-party antimalware products (such as Symantec Endpoint Protection, which provides anti-malware, intrusion prevention, and firewall capabilities) to integrate with the NSX platform for centralized deployment and policy enforcement.

vSphere also supports enabling Virtualization-Based Security (VBS) on Windows guest virtual machines with VMware ESX 6.7 and later compatibility, allowing administrators to configure features such as Device Guard and Credential Guard from VM hardware settings. Device Guard provides a set of hardware and software features designed to help prevent malware from running on Windows systems, and these protections are available to Windows workloads through centrally managed VM configurations.

VMware vDefend extends VCF's networking foundation with distributed malware detection and prevention, IDS/IPS, and network detection and response capabilities. The NSX Application Platform hosts the Malware Prevention for vDefend feature, and Malware Prevention can be deployed and managed from the Local Manager UI in NSX Federation environments. When host clusters are activated for Distributed Malware Prevention, a service virtual machine is deployed on each host. vDefend is managed centrally through NSX Manager, which provides a single management interface for security policy configuration and enforcement across multiple locations. NSX Manager supports activation of distributed security on selected clusters and provides APIs for malware prevention and security intelligence integration. The DistributedMps runbook is available to verify the health of the Malware Prevention Service pipeline and diagnose issues such as protection not working on a particular VM or certain files not being scanned. Malware Prevention logging is available at defined log file paths on the NSX Application Platform worker nodes for operational visibility.

ESX Agent Manager provides infrastructure for deploying and configuring agent virtual machines on ESX hosts. Solutions integrated with ESX Agent Manager can pass configuration properties (such as IP addresses and credentials) so that agents connect back to their management solution. When a user adds a host to a cluster of hosts that require an ESX agent, ESX Agent Manager deploys the required agents on the new host automatically. This mechanism supports centralized deployment of third-party security agents alongside the antimalware capabilities provided by vDefend.

VCF Operations contributes additional monitoring capability by discovering and monitoring application services on endpoint virtual machines where agents are installed. Guest operation privileges in VCF Operations are required to install agents on endpoint virtual machines, and the platform displays guest OS services authentication status for visibility into agent health.

General-purpose endpoint antimalware management, including agent deployment to guest operating systems, signature distribution, scan scheduling, and quarantine management, requires separate tooling outside of VCF. Organizations must select, deploy, and centrally manage an antimalware product that integrates with VCF's Guest Introspection framework and ESX Agent Manager infrastructure.

#### **VMware vDefend (Contributes)**

VMware vDefend contributes to centralized management of antimalware technologies through the Security Services Platform and NSX Manager, which together provide a single management interface for deploying, configuring, and maintaining network-based malware prevention across the data center.

The Distributed Malware Prevention service is deployed and managed centrally through NSX Manager. Administrators activate malware prevention on selected ESX host clusters, and the platform handles deployment of a service virtual machine (SVM) on each host in the cluster. The Guest Introspection framework extends this architecture by deploying lightweight thin agents on protected guest VMs; these agents intercept file activity within the guest and route it to the host-resident SVM for inspection and analysis, enabling centrally managed malware detection without requiring independent endpoint agent management on each guest. Guest Introspection supports thin agents on both Windows and Linux virtual machines, with the Host Agent (MUX) component running on ESX hosts to coordinate introspection functionality. Security policies and malware prevention rules are authored in NSX Manager and published from the management plane to host clusters, providing consistent policy enforcement without per-host configuration. The Malware Prevention Service can also be deployed on Gateway Firewalls for north-south traffic inspection; both deployment models are configured through the Security > IDS/IPS & Malware Prevention > Settings interface in NSX Manager.

The Security Services Platform coordinates malware analysis across the environment. Deployment status can be monitored per cluster and by individual transport nodes within the cluster. When a file is identified as suspicious or malicious, the platform can enforce remediation actions on the endpoint, including denying access to the file or deleting it. The Malware Prevention Service also includes fileless malware detection to identify threats that operate within system memory and use existing binaries and in-memory operations. A dashboard and logs are available for Security Operations teams to extract artifacts associated with in-memory malware execution, providing visibility into the antimalware policy outcomes across the environment.

For IDS/IPS, vDefend supports centralized management through a Global Manager that manages custom signatures across multiple Local Managers, enabling consistent signature distribution in multi-site deployments.

vDefend's centralized management scope is limited to network-based malware detection and prevention for virtual workloads. General-purpose endpoint antimalware management, including agent deployment, signature distribution for host-based antivirus, scan scheduling, and quarantine management, requires separate endpoint protection tooling outside of vDefend.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

### Control HDR 3.10

> Host update support: Capability to update host OS and applications. Verified host updates: Verify authenticity/integrity of host updates.

**SCF Controls:** TDA-01.1

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control HDR 3.10(1)

> Host update support: Capability to update host OS and applications. Verified host updates: Verify authenticity/integrity of host updates.

**SCF Controls:** CHG-04.2

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF enforces digital signature verification for software and firmware components at multiple layers of the stack, from hypervisor boot through application deployment.

During VCF deployment, each component is assigned a certificate from a default signing certificate authority. The VMware Certificate Authority (VMCA) is an internal CA that comes integral with vSphere deployment and has been purpose-built to serve the certificate needs of the vSphere environment. From initial installation onward, all VCF components operate within a certificate-based trust framework rooted in a known signing authority. Administrators can configure VCF to use certificates from VMCA, Microsoft CA, OpenSSL CA, or external third-party certificate authorities, giving organizations flexibility to integrate their approved CA hierarchy into the VCF signing and verification workflow. All communication between the VCF Installer and ESX hosts is performed over HTTPS with certificate identity validation, and the installer validates the identity of each host by comparing the certificate common name against the provided FQDN.

At the hypervisor level, VMware ESX uses UEFI Secure Boot to verify that the boot loader and kernel modules are digitally signed before execution. The host acceptance level setting controls which vSphere Installation Bundles (VIBs) are permitted: it can be configured to accept only VMware-certified VIBs, restricting installation to components signed by VMware's certificate authority. The vSphere Security Configuration Guide includes recommendations for enabling Secure Boot and configuring TPM on ESX hosts. When UEFI Secure Boot is enabled on an ESX host, the host performs full signature verification of each package installed on the host at every boot, not only at initial installation. vSphere also supports UEFI Secure Boot for individual virtual machines, where the bootloader, operating system kernel, and operating system drivers must each carry a valid signature before the virtual machine is permitted to boot.

vSphere Lifecycle Manager (vLCM) provides desired-state image management for ESX hosts. Administrators define a target image specification for each cluster, and vLCM checks host compliance against that image. When hosts deviate from the desired image, vLCM can remediate them back to the approved configuration. This approach restricts ad-hoc installation of unsigned or unapproved software components on managed hosts.

VCF Operations enforces digital signing for its management packs and orchestrator packages. Management pack PAK files are verified against a digital signature provided by VMware. If a PAK file lacks a valid digital signature, the system displays a warning indicating the file cannot be verified as authentic, and the digital signature indicates the original developer or publisher. For orchestrator packages, all packages exported from a VCF Operations orchestrator server are digitally signed, and the orchestrator's certificate infrastructure allows signing of all elements created within the system. When a client receives an element from the orchestrator server, the client verifies the server identity and decides whether to trust the signature. Package signing certificates are a form of digital identification used to support encrypted communication and provide a signature for orchestrator packages. For production environments, VCF recommends using a certificate signed by a certificate authority rather than a self-signed certificate, allowing organizations to align package signing with their approved CA infrastructure.

For container workloads, VMware Kubernetes Service (VKS) validates TLS certificates when fetching container images from registries and requires that registry certificates be signed by a trusted certificate authority. The platform requires that a certificate signing request be generated, signed by a certificate authority, and applied to the Workload Platform Management layer to secure the Kubernetes control plane. TLS certificates for the Supervisor layer are derived from the VMCA certificate, anchoring the Kubernetes control plane trust chain in the same root authority as the broader VCF infrastructure. The VCF CLI `imgpkg` tool supports CA certificate validation for registry connections through its `--registry-ca-cert-path` flag, which can be

specified multiple times to add CA certificates for registry API communication. VKS clusters can also be configured to trust any number of CA certificates for private registry endpoints, directing image pulls to controlled, organization-approved sources.

Harbor is available as a standard package for VKS clusters and provides enterprise container registry capabilities including content trust and image signing. Organizations can configure Harbor to enforce signature requirements at image push or pull time, restricting deployments to images signed under an organization-approved certificate authority. Admission controllers are the primary enforcement mechanism for image signing policies within VKS clusters. The ImagePolicyWebhook admission controller allows administrators to configure an external image verification backend that is consulted before any pod is scheduled; if the backend determines that an image lacks a valid signature from an approved authority, the pod is rejected before it runs. ValidatingWebhookConfiguration objects can integrate third-party image signing solutions with the Kubernetes API server, creating an enforcement layer that intercepts pod creation requests and validates image signatures as part of the admission control chain.

The Kubernetes control plane components used by VKS are themselves distributed as signed artifacts. Kubernetes release artifacts, including standalone binaries and Software Bill of Materials files, are signed using cosign keyless signing, and control plane component images can be verified using cosign with certificate-based identity and OIDC issuer validation. Kubernetes CertificateSigningRequest signers can deny requests that do not meet approval conditions, and CertificateSigningRequests with unknown signerName values are never auto-approved, restricting unauthorized certificate issuance within the cluster. For workload certificate lifecycle management, the cert-manager standard package creates TLS certificates for workloads and automatically renews them before expiration, with VKS cluster control plane certificate rotation configured through the ClusterClass API and defaulting to renewal 90 days before expiry. The renewal window is configurable through the renewalDaysBeforeExpiry parameter in the ClusterClass specification, allowing organizations to set this period to match their certificate management policies.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control HDR 3.11

> Host integrity verification: Verify host hardware/software integrity. Host hardware root of trust: Use hardware to anchor host integrity.

**SCF Controls:** AST-08

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides monitoring, alerting, compliance assessment, and forensic capabilities that support periodic inspection of systems and system components for indicators of compromise (IoC). While VCF does not include a dedicated IoC detection engine or endpoint detection and response (EDR) tool, its operational monitoring and change-detection features contribute to an organization's ability to identify anomalous conditions that may signal compromise.

VCF Operations continuously monitors the managed environment by evaluating collected data against defined policies and assigning a compliance score to each object or group based on adherence to those policies. This compliance monitoring enables discovery of non-compliant objects and associated compliance violations, which can surface configuration drift or unauthorized changes that may indicate tampering or compromise. The Anomalies widget displays metric deviations from both dynamic and static thresholds, and the Potential Evidence tab surfaces events, property changes, and anomalous metrics that may be relevant to a given alert. Risk alerts flag conditions that are not immediate threats but warrant prompt investigation.

Configuration drift detection in VCF Operations compares current settings against assigned configuration template values and notifies administrators of any deviations. The Configuration Drift dashboard allows administrators to view and compare drifts and identify settings that have changed, while the Detect Drift function detects drift across all configuration templates and displays results in the Drifts table. This change-detection capability supports identification of unauthorized modifications to managed infrastructure.

VCF Operations provides a layered alerting framework. Log-based alerts can be configured through the Symptom Definitions card in Infrastructure Operations, and alert definitions can be created to monitor infrastructure events and conditions. The system correlates a subset of events published by monitored systems and delivers them as faults, while fault symptoms can be defined based on those events and added to alert definitions. This allows administrators to build detection logic around events that may indicate compromise.

VCF Operations collects security properties for virtual machines, including the connection state of CD-ROM, USB controller, parallel port, and serial port devices, which supports detection of unauthorized device connections to virtual infrastructure. Administrators can filter virtual machines by encryption status, vCenter name, host name, and VM name to assess compliance posture across the environment.

Audit event reporting in VCF Operations records configuration changes, user actions, and authentication events across the managed infrastructure. These audit events can be filtered by authentication status, user, vCenter instance, object type, and event description, giving investigators a detailed view of activity across the environment for conducting security forensics.

VCF Operations includes a Forensics widget that allows administrators to compare data across two time periods, search for and filter objects within the environment, and display widget data based on filtered object sets. This capability supports forensic comparison and analysis when investigating potential indicators of compromise. Guided Network Troubleshooting highlights anomalous entities and metrics within the network path, aiding root-cause analysis during incident investigation.

OS and Application Monitoring Properties are collected for operating systems, application services, remote checks, Linux processes, and Windows services. These monitoring data points can be used to create reports, views, and dashboards for ongoing system inspection, providing visibility into the operational state of managed components and flagging deviations from expected behavior.

VMware ESX hosts generate detailed audit logs, and multiple VCF components support syslog forwarding to external log aggregation and SIEM systems. VMware vCenter provides both API and UI methods to configure syslog forwarding. vSphere



Supervisor can forward control plane and Kubernetes API server audit logs to external syslog servers via Fluent Bit in RFC 5424 format. These log forwarding capabilities allow VCF telemetry to feed into external IoC detection and correlation tools. ESX also enforces integrity checks on installed VIBs through the Host Image Profile Acceptance Level, governing what software is permitted to run on the host and supporting detection of unauthorized additions to the hypervisor. vSphere provides preconfigured alarms that monitor the status of host hardware system boards, allowing notification when hardware-level anomalies are detected.

The NSX networking layer includes monitoring dashboards showing details about system health, networking, security, and compliance reporting. NSX supports logging and auditing capabilities that can be integrated with external logging and monitoring solutions to monitor for security events and anomalies. The System Health Agent on NSX components monitors component statistics, analyzes them, alerts on abnormalities, and can trigger remediation actions when specific conditions are met. VCF Operations for Networks monitors network flows and can detect outlier VMs based on flow metrics including bytes, packets, sessions, and traffic rate, supporting detection of anomalous traffic patterns within the environment. At the host layer, ESX measures boot components against a Trusted Platform Module (TPM) 2.0 hardware root of trust, and VMware vCenter exposes the resulting boot measurements and TPM event logs through its trusted-infrastructure API, so administrators can confirm a host booted in a known-good state and detect firmware or boot-level tampering. Secure Boot independently validates the cryptographic signatures of the ESX kernel and modules at startup, so unsigned or altered components are surfaced as an integrity failure rather than loaded silently.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer contributes to this control through its application-layer threat detection and event logging capabilities, which provide the data security teams need during periodic Indicators of Compromise (IoC) inspections.

Avi's Web Application Firewall (WAF) analyzes all incoming and outgoing application traffic using signature-based detection, a positive security model, client reputation scoring, machine learning, and outlier analysis. WAF signatures include detection for OWASP Top 10 attack vectors and other threat categories. The WAF operates in either Detection mode or Enforcement mode; Detection mode allows new rules to be evaluated without blocking legitimate traffic, making it suitable for baseline establishment and traffic anomaly review during periodic inspections. Partial detection allows individual rules to be placed in Enforcement mode while the policy as a whole remains in Detection mode, supporting incremental adoption.

The Avi Controller generates structured security events that auditors and security teams can review as part of periodic IoC inspections. The Controller tracks denial-of-service attack events with attack count metrics and generates alerts at configurable severity levels. The scan detection system dynamically classifies client IPs into Good, Bad, and Unknown groups based on observed traffic patterns, adjusting automatically as patterns change. The Controller also logs the initiator and target of failed trusted channel establishment attempts, including cases where an SSL certificate is absent, providing visibility into authentication anomalies.

Avi events serve as an audit trail for user logins, configuration changes, and runtime state changes, and can be forwarded to Splunk for centralized audit trail analysis. Client request logs can be collected automatically based on categories of significance for application-level review. System events provide a historical record of faults within the system. DataScripts support custom identification of suspicious traffic and execution of appropriate mitigation actions, extending detection coverage beyond built-in signatures.

Avi provides several tools suited for periodic review workflows. The application security report can be generated on demand through the Controller UI or CLI, with an optional duration parameter to define the report period, producing a point-in-time record of detected threats suitable for scheduled IoC review cycles. The Config Audit Trail, accessible at Operations > Events > Config Audit Trail, presents a filtered view of user activity and configuration change events over a selected period, supporting review of whether configuration changes preceded observed anomalies. Alert events can be streamed to external syslog servers using Syslog Audit Persistence, enabling integration with SIEM platforms that support automated IoC correlation. The Controller also generates system reports for upgrade operations containing pre-check results and operational status, which support verification of system state before and after changes.

The organization is responsible for defining the inspection cadence, determining what constitutes a compromise indicator in context, and performing the host-level and physical inspection that Avi does not address as an application delivery controller. Platform-level system integrity inspection relies on the underlying VCF platform.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control HDR 3.11(1)

> Host integrity verification: Verify host hardware/software integrity. Host hardware root of trust: Use hardware to anchor host integrity.

**SCF Controls:** CFG-06.1

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides automated mechanisms to detect unauthorized deviations from approved baselines and to remediate them, operating across three complementary layers: configuration compliance monitoring, host software state management, and preventive policy enforcement. The VCF consumption layer extends these mechanisms to Kubernetes workloads through VMware Kubernetes Service (VKS), adding declarative self-healing, admission-based baseline enforcement, and workload-level audit logging.

VCF Operations includes a compliance management engine that continuously monitors infrastructure objects against defined baselines. The scope of monitoring covers VMware vCenter instances, vSAN, NSX, and cloud environments, with monitored objects including hosts, virtual machines, distributed port groups, and distributed switches. VCF Operations supports both built-in and custom compliance benchmarks, including VMware SDDC, VCF, regulatory, and custom benchmarks. Administrators can create custom benchmarks tailored to organizational requirements and edit alert definitions and policies at any time after creation. When VCF Operations detects that an object deviates from its compliance policy, it generates alerts and notifications. Depending on the severity of the non-compliance, administrators can configure automated remediation actions to bring the object back into compliance. For violations that require manual review, the compliance engine highlights risk areas and provides actionable remediation recommendations.

VCF Operations also includes a Configuration Drifts feature that proactively monitors configuration changes across vCenter instances and vSphere clusters. Administrators define desired values through configuration templates, and VCF Operations compares the current configuration of each managed object against those templates on a scheduled basis. When a deviation is detected, the system generates a drift notification identifying the specific settings that changed. The Configuration Drifts Overview page lets administrators view and compare drifts, and drift status reports can be downloaded for review and record-keeping. Configuration templates can be integrated with a source control repository for versioning, auditing, and change management, enabling accurate configuration updates and supporting change control with reviews.

For host software state, vSphere Lifecycle Manager takes a desired-state approach. Administrators define a desired software specification, including base image, components, and add-ons, for each cluster or standalone host through Image Manager. The system checks host compliance against this specification and, when the current state differs from the desired state, cluster remediation applies the desired specification to bring all hosts into alignment. Pre-check validation runs before remediation to confirm hosts are in a good state to be updated. Global and per-cluster remediation policies govern how the remediation process behaves. At the individual host level, VMware ESX also enforces software integrity through the Host Image Profile Acceptance Level, which governs the integrity validation requirements for VIBs (VMware Installation Bundles) before they can be installed on a host, providing a preventive control that complements Lifecycle Manager's desired-state remediation.

VCF Automation extends baseline enforcement to provisioned infrastructure through Policy as Code, which lets organization administrators enforce instance-centric policies on Infrastructure as a Service (IaaS) resources from a central location. IaaS Resource Policies support validation expressions that enforce constraints on resource creation and updates, such as requiring specific parameter values during provisioning. This preventive enforcement complements VCF Operations' detective and corrective controls by blocking non-compliant resource configurations before they are deployed.

At the Kubernetes workload layer, VKS clusters operate on a declarative state model where desired workload configurations are persisted as Kubernetes objects. The Kubernetes control plane continuously reconciles actual cluster state against the declared desired state: when a running pod crashes, is deleted, or is evicted from a node, the relevant controller automatically reschedules it to restore the declared replica count. Deployments maintain revision history and support rollback when a new rollout fails readiness or liveness checks. Kubernetes liveness probes provide a further container-level detection mechanism:

when a container's health check fails, Kubernetes automatically restarts that container to restore correct application state. At the node level, VKS MachineHealthCheck objects specify the conditions under which a cluster node is considered unhealthy, including condition types such as MemoryPressure and DiskPressure with configurable timeout durations. When a node meets those conditions, the Cluster API triggers automated machine replacement to restore the declared cluster state. MachineHealthCheck is configurable for both control plane and worker node pools and can be updated at runtime. This continuous reconciliation provides automated resiliency at the workload layer, complementing the infrastructure-level remediation provided by VCF Operations and vSphere Lifecycle Manager.

VKS clusters support Kubernetes ValidatingAdmissionPolicy objects, which define CEL-based (Common Expression Language) rules evaluated against each API request before it is persisted. The validationActions field on a ValidatingAdmissionPolicyBinding specifies how violations are handled: Audit records the violation in the Kubernetes audit log, Warn returns a warning to the requesting client, and Deny rejects the request outright. Policy failures are recorded in audit events with a validation.policy.admission.k8s.io/validation\_failure annotation, enabling automated monitoring pipelines to detect unauthorized changes. The Pod Security Admission controller provides enforce, audit, and warn modes for pod workload baselines, configurable per namespace or VKS cluster through the ClusterClass podSecurityStandard field. In enforce mode, pod submissions that violate the configured profile are rejected at admission; in audit mode, violations are recorded without blocking the pod. VCF Automation mutation policies with the pod-security template extend this further by automatically modifying pods to meet security standards before admission, providing active remediation rather than only rejection. The VKS Policy Insights page surfaces incompatibility detections, including pre-existing Gatekeeper installations not deployed via cluster management, flagging configuration deviations for administrative review.

For audit trail purposes, VCF Operations tracks each configuration change to the authenticated user who initiated it or scheduled the job that performed the change, providing accountability for changes that may cause compliance deviations. NSX Manager audit logs capture both old and new values for modified security policies, recording rule changes including operation status, timestamps, and user information. VCF Operations also provides the isolation.device.edit.disable setting to restrict unauthorized modification of device configurations on managed objects. At the Kubernetes layer, VKS audit logging records all API server activity with authorization.k8s.io/decision and authorization.k8s.io/reason annotations that capture authorization outcomes and their justification, providing a chronological record of all actions across the VKS cluster that can feed automated monitoring pipelines for drift detection.

### **VMware vDefend (Contributes)**

VMware vDefend (vDefend) contributes to integrity assurance at the network policy and security detection layers but does not perform host-level configuration baseline deviation detection or automated remediation of endpoint settings.

Security Intelligence, part of the Security Services Platform (SSP), continuously monitors network traffic flows and alerts administrators to new or unexpected flows between environments, enabling detection of deviations from established microsegmentation baselines. Security Intelligence evaluates whether security rules have been implemented and generates a security score to track policy enforcement progress, helping administrators identify gaps between the desired baseline and current policy state.

Within the IDS/IPS subsystem, vDefend tracks user modifications to system signatures, allowing administrators to filter and view only those signatures that have been customized or altered from their default state. Global Signature Management records signatures that have been changed or excluded by users. When deviations from the signature baseline need to be corrected, the bulk edit feature supports a Reset action that restores signatures to their default configuration, providing a resiliency action within the security detection layer.

For malware-related events, Intelligent Assist provides automated resiliency. Targeted remediation enables all IDS signatures for the affected workload; Comprehensive remediation automatically creates DFW policies to restrict access for affected systems. Applied remediation policies can be reviewed under Security > IDS/IPS & Malware Prevention > Emergency Threat Rules. Administrators can also implement lockdown protection rules to restrict traffic leakage between environment pairs as an additional resiliency action.

Automated remediation of unauthorized changes to system configurations or OS-level settings is outside vDefend's scope.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer.  
This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control HDR 3.12

> Host provisioning: Securely provision host unique identifiers.

**SCF Controls:** AST-18

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF contributes to roots of trust protection through cryptographic key management, certificate trust chain infrastructure, hardware-based integrity verification, and software package signing across its component stack.

VCF manages cryptographic keys through integration with external Key Management Servers (KMS) that support the Key Management Interoperability Protocol (KMIP) 1.1 standard, or through the built-in vSphere Native Key Provider. VMware vCenter communicates with external KMS instances using the KMIP protocol, and the KMS must be capable of acting as a symmetric key foundry and server. The vSphere Native Key Provider can coexist with an existing key server infrastructure, giving organizations flexibility in how they provision and manage encryption keys. Trust between vCenter and a KMS can be established through multiple methods: uploading a KMS certificate directly, using the vCenter Root CA certificate, providing an uploaded certificate and private key, or generating a Certificate Signing Request for the KMS to sign. The vSphere Web Services API exposes an `establishTrust()` method that enables mutual trust between vCenter and a KMS server using self-signed certificates, signed certificate signing requests, or certificate and key pair exchange. For stronger security, administrators should upload a leaf certificate or intermediate CA certificate rather than a root CA certificate, since uploading a root CA causes vCenter to trust all certificates signed by that authority.

vCenter maintains a trusted root certificate store where administrators can add root certificates or certificate chains from enterprise or third-party certificate authorities. Adding a root certificate to this store establishes trust with that CA for all subsequent certificate validation across the vCenter environment. The `TrustedRootChains` API provides programmatic access to manage and retrieve root certificate chains by identifier, supporting automation of trust chain verification. The underlying certificate storage uses the VMware Endpoint Certificate Store (VECS), which maintains named stores including `TRUSTED_ROOTS` for all trusted root certificates and `TRUSTED_ROOTS_CRLS` for certificate revocation list data; administrators can manage these stores with the `vecs-cli` command-line tool. The `TRUSTED_ROOTS_CRLS` store must not be deleted or modified, as doing so can damage the certificate infrastructure. Control plane VMs in vSphere Supervisor environments rely on a Trusted Roots bundle that must be kept current with the latest vCenter root certificates.

At the hardware level, VCF supports Secure Boot and Trusted Platform Module (TPM) to help verify that hosts boot in known-good configurations. The vSphere Security Configuration Guide recommends enabling Secure Boot and TPM as part of VMware ESX host hardening. vCenter provides API interfaces for retrieving TPM event logs for configuring firmware trust with an attestation service or validating boot-time TPM measurements, and for listing configured TPM endorsement keys on a host. These capabilities support establishing a hardware-rooted chain of trust from firmware through the hypervisor.

Beyond core key and certificate management, several VCF components maintain their own trust infrastructure. VCF Operations orchestrator supports the import, export, and generation of package signing certificates, providing a mechanism for verifying the integrity and authenticity of software packages deployed within the environment. VCF Operations also maintains an SSL truststore with a password-protected keystore for managing certificates used in secure communications. VCF Automation can establish trust relationships with external servers by verifying and accepting certificate information for SSL communication, and it exchanges SAML service provider metadata with external identity providers to establish federated trust relationships. NSX components establish trust through certificate exchange between Local Manager and Global Manager instances during federation setup. VCF PowerCLI maintains its own Trusted Certificate Store where trusted certificates are saved and used when connecting to server systems, extending the trust chain to administrative automation tooling.

VCF provides the cryptographic infrastructure for establishing, storing, and verifying trust relationships through certificates and keys across all its major components. However, the organizational processes for provisioning supplier keys, verifying supply chain integrity of cryptographic material, defining key protection policies, and physically protecting key storage remain the responsibility of the deploying organization.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.



## Control HDR 3.13(a)

> Host boot integrity: Verify integrity of the host OS bootloader. Host boot authenticity: Verify authenticity of the host OS bootloader.

**SCF Controls:** AST-18

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF contributes to roots of trust protection through cryptographic key management, certificate trust chain infrastructure, hardware-based integrity verification, and software package signing across its component stack.

VCF manages cryptographic keys through integration with external Key Management Servers (KMS) that support the Key Management Interoperability Protocol (KMIP) 1.1 standard, or through the built-in vSphere Native Key Provider. VMware vCenter communicates with external KMS instances using the KMIP protocol, and the KMS must be capable of acting as a symmetric key foundry and server. The vSphere Native Key Provider can coexist with an existing key server infrastructure, giving organizations flexibility in how they provision and manage encryption keys. Trust between vCenter and a KMS can be established through multiple methods: uploading a KMS certificate directly, using the vCenter Root CA certificate, providing an uploaded certificate and private key, or generating a Certificate Signing Request for the KMS to sign. The vSphere Web Services API exposes an `establishTrust()` method that enables mutual trust between vCenter and a KMS server using self-signed certificates, signed certificate signing requests, or certificate and key pair exchange. For stronger security, administrators should upload a leaf certificate or intermediate CA certificate rather than a root CA certificate, since uploading a root CA causes vCenter to trust all certificates signed by that authority.

vCenter maintains a trusted root certificate store where administrators can add root certificates or certificate chains from enterprise or third-party certificate authorities. Adding a root certificate to this store establishes trust with that CA for all subsequent certificate validation across the vCenter environment. The `TrustedRootChains` API provides programmatic access to manage and retrieve root certificate chains by identifier, supporting automation of trust chain verification. The underlying certificate storage uses the VMware Endpoint Certificate Store (VECS), which maintains named stores including `TRUSTED_ROOTS` for all trusted root certificates and `TRUSTED_ROOTS_CRLS` for certificate revocation list data; administrators can manage these stores with the `vecs-cli` command-line tool. The `TRUSTED_ROOTS_CRLS` store must not be deleted or modified, as doing so can damage the certificate infrastructure. Control plane VMs in vSphere Supervisor environments rely on a Trusted Roots bundle that must be kept current with the latest vCenter root certificates.

At the hardware level, VCF supports Secure Boot and Trusted Platform Module (TPM) to help verify that hosts boot in known-good configurations. The vSphere Security Configuration Guide recommends enabling Secure Boot and TPM as part of VMware ESX host hardening. vCenter provides API interfaces for retrieving TPM event logs for configuring firmware trust with an attestation service or validating boot-time TPM measurements, and for listing configured TPM endorsement keys on a host. These capabilities support establishing a hardware-rooted chain of trust from firmware through the hypervisor.

Beyond core key and certificate management, several VCF components maintain their own trust infrastructure. VCF Operations orchestrator supports the import, export, and generation of package signing certificates, providing a mechanism for verifying the integrity and authenticity of software packages deployed within the environment. VCF Operations also maintains an SSL truststore with a password-protected keystore for managing certificates used in secure communications. VCF Automation can establish trust relationships with external servers by verifying and accepting certificate information for SSL communication, and it exchanges SAML service provider metadata with external identity providers to establish federated trust relationships. NSX components establish trust through certificate exchange between Local Manager and Global Manager instances during federation setup. VCF PowerCLI maintains its own Trusted Certificate Store where trusted certificates are saved and used when connecting to server systems, extending the trust chain to administrative automation tooling.

VCF provides the cryptographic infrastructure for establishing, storing, and verifying trust relationships through certificates and keys across all its major components. However, the organizational processes for provisioning supplier keys, verifying

supply chain integrity of cryptographic material, defining key protection policies, and physically protecting key storage remain the responsibility of the deploying organization.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control HDR 3.13(b)

> Host boot integrity: Verify integrity of the host OS bootloader. Host boot authenticity: Verify authenticity of the host OS bootloader.

**SCF Controls:** CFG-07

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides several automated provisioning and configuration capabilities that contribute to Zero-Touch Provisioning (ZTP) objectives, though full ZTP for arbitrary network devices requires additional organizational processes and tooling beyond VCF itself.

VCF Operations automates the commissioning of VMware ESX hosts into the VCF environment. When new hosts are added, administrators specify host details and select a network pool; the platform validates each host and commissions it into the free pool for use with VCF domains and clusters. Network pools automatically assign static IP addresses to vSAN, NFS, iSCSI, and vMotion VMkernel ports when hosts are added to a domain or cluster, removing the need for manual network configuration of each host interface. This commissioning workflow applies a consistent, repeatable configuration to each host as it joins the environment. SDK samples are available for programmatic host commissioning, cluster creation, and domain creation, enabling further automation of the onboarding process.

Once hosts are commissioned, vSphere Lifecycle Manager enforces a desired-state software model. Administrators define an approved software image specification (base image, components, vendor add-ons) for each cluster, and Lifecycle Manager applies that specification to all hosts collectively. When a new host joins a cluster, it is checked against the desired state and remediated to match if it differs. This approach means newly added hosts automatically receive the correct software configuration without individual manual setup.

vSphere also supports Auto Deploy, a PXE-based provisioning mechanism that allows ESX hosts to boot from the network and receive their complete software image automatically from a central deployment server without manual intervention. Because the deployment server delivers the boot image over the network, Auto Deploy requires a secure separation between the production network and the management or deployment network; custom certificates are supported to help secure the provisioning process.

VCF Automation extends automated provisioning through cloud templates and network profiles. Administrators can define cloud-specific network profiles that govern the behavior of network provisioning for deployments, specifying how networks are assigned and configured when new workloads or infrastructure components are instantiated. On-demand network provisioning is supported through external IPAM integrations, allowing new networks to be created automatically in the external IPAM system as part of the deployment process. Cloud template properties allow administrators to define storage provisioning types and other infrastructure settings declaratively, so that deployed resources receive their configuration from a template rather than through manual steps. Administrators can test template syntax and placement before release, then publish versioned templates to the self-service catalog for repeatable, policy-governed provisioning.

Policy as Code lets organization administrators enforce constraints on provisioned IaaS resources centrally, so deployments automatically comply with organizational standards without manual review of each provisioning request.

VCF Automation also supports onboarding existing unmanaged virtual machines into managed deployments through its API, bringing them under the same lifecycle and policy governance as newly provisioned resources. Day 2 customizations for VM Apps support network configuration changes similar to initial deployment, extending the automated configuration model beyond initial provisioning into the operational lifecycle.

At the Kubernetes consumption layer, VMware Kubernetes Service (VKS) provides declarative, automated cluster provisioning that follows a similar zero-touch model. When a platform administrator creates a VKS cluster using the ClusterClass API, the cluster is automatically provisioned with pre-defined configurations covering the Kubernetes version, node count, resource limits, CNI plugin, storage classes, and security settings without requiring manual node-by-node configuration. The

TkgServiceConfiguration object allows administrators to define cluster-wide defaults for the container network interface, global proxy server settings, and trusted TLS certificates; these settings apply automatically to all VKS clusters created within the Supervisor namespace, so newly provisioned clusters receive a consistent, pre-approved configuration from the moment they are created.

When an external identity provider is registered with a Supervisor, all VKS clusters subsequently created on that Supervisor are automatically configured with that identity provider through the Pinnipid authentication service. VKS runs the Pinnipid OIDC client on both the Supervisor and VKS cluster nodes, so configuring an external OIDC provider for the Supervisor automatically configures the Pinnipid components across all clusters without per-cluster administrator action. Node-level configuration is handled through ClusterClass variables and the osConfiguration parameter; the osConfiguration trust setting applies system-wide certificate trust automatically to all nodes in a cluster, supporting secure configuration of nodes as they join. Pod Security Admission, with Baseline and Restricted profiles, automatically applies security policies to workloads as they are created, so newly deployed containers receive consistent security constraints without per-deployment administrator intervention. Validating and mutating webhooks can extend this enforcement to apply organization-specific configuration policies automatically at admission time.

VCF Operations provides network auto-discovery capabilities that support SSH, SNMP v2, SNMPv3, and API-based access credential types for discovering devices on the network. This automated discovery helps identify and catalog new devices as they appear, which is a foundational step in any ZTP workflow.

These capabilities collectively provide an automated, template-driven provisioning model for infrastructure and workload components within the VCF environment. For physical network equipment such as switches, routers, and access points, organizations will need vendor-specific ZTP frameworks or external orchestration tools that operate outside VCF's scope. Organizations should pair VCF's provisioning automation with network device management platforms and configuration management tools to achieve a full ZTP strategy across the infrastructure stack.

### **VMware vDefend (Contributes)**

VMware vDefend contributes to zero-touch provisioning by automating the application of network security policies to devices as they are onboarded to the infrastructure, though it does not provide a standalone ZTP solution covering initial device bootstrapping.

For physical workloads, vDefend Bare Metal Security provides an automated onboarding workflow. Administrators generate a registration token through the Security Services Platform, install the BMS installer service, and deploy Bare Metal Agents on target servers. Once deployed, the Bare Metal Agents receive security policy configurations from the Bare Metal Orchestrator (BMO) and Bare Metal Controller (BMC), translate those policies, and program the data path. Onboarded bare metal servers and their interfaces are integrated with NSX Manager for centralized management and visibility. When bare metal servers are onboarded, the Security Services Platform automatically creates a default Bare Metal server group and default DFW rules, providing an immediate security policy baseline without per-server manual configuration.

For virtualized workloads, vDefend policy groups enable automated security orchestration by moving away from static, manual IP management. Security policies update automatically as workloads are provisioned, moved, or modified, so newly placed virtual machines receive their intended security posture without requiring per-workload firewall rule configuration.

The Security Services Platform is deployed as an OVA appliance, and the SSP Installer supports adding control plane and worker nodes to scale the security management infrastructure as environments grow.

In 9.1, vDefend Distributed IDS/IPS supports automatic signature downloads as a configurable option, so intrusion detection and prevention profiles can be maintained without manual intervention as new workloads join protected segments. The Network Detection and Response (NDR) Sensor follows the same token-based onboarding model used by Bare Metal Security, registering to the Security Services Platform through a two-step process: generating a registration token in the Security Services Platform UI and completing registration via CLI.

The broader aspects of ZTP, such as initial operating system installation, base network configuration, and hardware initialization, fall outside vDefend's scope and are handled by other VCF components or vendor-specific provisioning frameworks.

#### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) contributes to Zero-Touch Provisioning through automatic Service Engine lifecycle management and cloud integration automation. The Avi Controller automatically provisions and integrates Service Engine virtual machines as load balancing capacity is required, deploying them into the network without per-instance manual configuration steps. Service Engines can be rapidly provisioned on virtual infrastructure and de-provisioned when no longer needed, with the Controller managing placement, network attachment, and policy application.

For NSX-integrated deployments, the NSX Cloud Quick Start Wizard provides a guided configuration walkthrough for establishing a basic NSX Cloud environment using NSX Overlay networking, reducing the manual steps required to integrate Avi with the underlying NSX networking fabric. This wizard is accessible from the Avi Controller interface and completes configuration with a minimal set of inputs.

When an NSX cloud connector is configured, the Avi Controller can automatically generate vDefend objects using the `automate\_dfw\_objects` setting. As virtual services are provisioned through the Controller, their corresponding network security objects can be created automatically without requiring separate manual firewall configuration steps in vDefend.

Avi also supports automated certificate lifecycle management through integration with Venafi Trust Protection Platform. The Avi Controller Onboard Discovery feature automates the import of certificates from network devices into Venafi for monitoring, validation, and provisioning. Once discovered, the Controller can generate corresponding certificate and application objects in Venafi, enabling certificate provisioning to virtual services immediately after discovery.

Provisioning of infrastructure hosts, network switches, and other platform components outside the application delivery tier is managed by VCF platform capabilities. Organizations deploying Avi within VCF can combine Avi's Service Engine and virtual service automation with VCF platform provisioning workflows to address the full scope of this control.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control HDR 3.14

> Host data at rest integrity: Protect host file system integrity. Host storage encryption: Use cryptography for host data at rest.

**SCF Controls:** END-06.5

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides automated mechanisms to verify the integrity of the boot process for both physical hosts and virtual machines, spanning hardware-rooted trust, software specification validation, and post-boot health verification.

ESX hosts support UEFI Secure Boot, which uses cryptographic signature verification to validate each component loaded during the boot sequence. When Secure Boot is enabled, the host firmware checks the digital signature of the bootloader, and each subsequent stage verifies the next, creating a chain of trust from firmware through the hypervisor kernel. The vSphere Security Configuration Guide recommends enabling Secure Boot on all ESX hosts as part of standard hardening. ESX also supports an `execInstalledOnly` enforcement mode, built on top of UEFI Secure Boot, which restricts execution to software installed through the managed package process and prevents unauthorized binaries from running on the host.

VCF also supports Trusted Platform Module (TPM) integration on ESX hosts. TPM devices perform measured boot, recording cryptographic hashes of each boot component into platform configuration registers. VMware vCenter provides programmatic access to TPM device information through the trusted infrastructure API, including retrieval of TPM event logs that record boot-time measurements. These event logs can be used with an attestation service to validate that hosts booted with known-good firmware and software configurations. The Tpm interface from the trusted infrastructure package allows administrators to list TPM devices on a host and retrieve detailed information including manufacturer, model, and firmware version. vCenter also supports unsealing secrets bound to an endorsement key to verify reported measurements from the TPM event log, providing a second verification path for boot integrity. ESX host TPM modules must run the latest 2.0 firmware when in use, and must be disabled in the BIOS if not in use, keeping the hardware trust baseline consistent and verifiable.

Remote attestation capabilities allow vCenter to verify host boot integrity by comparing TPM measurements against expected values, providing an automated mechanism to detect tampering or unexpected changes in the boot process of managed hosts.

For virtual machines, VCF supports UEFI Secure Boot at the VM level, allowing guest operating systems to verify boot component integrity through the same cryptographic chain-of-trust mechanism used on physical hardware. VM boot firmware can be configured to EFI type using the vSphere Automation SDK `Boot.UpdateSpec()` method, and boot options including EFI firmware type and boot behavior settings are configurable through both the vSphere Client and the vSphere Automation API. VM-level Secure Boot is supported starting with Hardware Version 13. For Windows virtual machines, vSphere also supports Configurable Code Integrity under Virtualization-Based Security (VBS), which enforces that only trusted, signed code runs from the boot loader onwards within the guest.

At the software specification level, VCF's lifecycle management includes automated validation before any software is applied to hosts. The Drafts service offers a `validate_Task` method that checks whether a draft software specification is complete and identifies any conflicts between draft components or unresolved dependencies. Before saving a draft and turning it into the desired state for a cluster or standalone host, the specification should be validated to confirm it is complete and valid. This automated pre-deployment validation helps ensure that hosts receive verified, conflict-free software configurations, reducing the risk of boot failures or integrity issues caused by incompatible components. VCF also supports the creation of exported ISO images from desired software specifications for use in clean installs and bootstrapping, allowing administrators to provision systems from a known, validated software baseline.

For post-boot verification, VCF provides automated checking mechanisms across its management components. The Supportability and Serviceability tool (SoS) on VCF Operations includes a health check function that verifies system state after operations such as a vCenter instance restore, confirming that all components have started correctly and are operating as expected. VCF Operations Cloud Proxy implements automated first boot verification by checking for the presence of a

"Succeeded" file in /var/log/firstboot, providing a programmatic way to confirm that the boot process completed without error.

Auditors evaluating a VCF deployment for this control should verify that UEFI Secure Boot is enabled on all ESX hosts, that TPM 2.0 modules are active and running current firmware, that lifecycle management workflows use the software specification validation step before applying updates, that SoS health checks are run after restore or maintenance operations, and that boot verification logs are monitored for component deployments.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.



## Control HDR 3.14(1)

> Host data at rest integrity: Protect host file system integrity. Host storage encryption: Use cryptography for host data at rest.

**SCF Controls:** END-06.6

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides multiple automated mechanisms that protect the integrity of boot firmware across both physical VMware ESX hosts and virtual machines.

ESX hosts support UEFI Secure Boot, which validates the cryptographic signature of each component in the boot chain before allowing it to execute. When Secure Boot is enabled, the host firmware restricts the boot process to only authorized, signed code, protecting the hypervisor from unauthorized modification. The vSphere Security Configuration Guide recommends enabling Secure Boot on all ESX hosts as a standard hardening measure, alongside Trusted Platform Module (TPM) enablement. Secure Boot enforcement can be activated or deactivated through ESXCLI, and ESX also supports `execInstalledOnly` enforcement, a control built on top of UEFI Secure Boot that restricts execution to only installed binaries.

TPM 2.0 integration on ESX hosts provides measured boot capabilities. The TPM records cryptographic hashes of each boot stage into hardware-protected platform configuration registers, creating a tamper-evident chain of trust from firmware through the hypervisor. If any boot component is altered, the TPM measurements will differ from expected values. The TPM event log can be retrieved for configuring firmware trust with an attestation service or for validating boot time measurements. VMware vCenter provides programmatic access to TPM device information, including manufacturer and firmware version details, through the trusted infrastructure API. TPM endorsement keys used to establish hardware identity and trust are also managed through this API. An attestation service can compare measurements against known-good values to confirm that boot firmware and subsequent components have not been modified.

For virtual machines, VCF supports the Extensible Firmware Interface (EFI) as a virtual hardware feature, enabling UEFI Secure Boot for guest operating systems. Virtual machine firmware type can be configured to EFI through the vSphere Client or programmatically through the vSphere Automation SDK using the `Boot.UpdateSpec` API or the `BootTypes.UpdateSpec.Builder setType()` method. UEFI Secure Boot at the VM level protects guest operating system boot firmware using the same signature verification approach as physical hosts. For Windows virtual machines, vSphere supports Virtualization-based Security (VBS) with Configurable Code Integrity, which restricts execution to trusted code from the boot loader onward within the guest operating system. VCF also supports hardware memory encryption for confidential workloads through technologies like SEV-ES, which protects VM state from inspection or modification at the hardware level.

vSphere Lifecycle Manager provides automated firmware lifecycle management for ESX hosts. Firmware add-ons can be added, replaced, removed, upgraded, or downgraded through vSphere Lifecycle Manager images, allowing administrators to maintain firmware at vendor-validated versions across the host fleet. vSphere Lifecycle Manager images consist of a base image, vendor add-ons, components, and firmware add-ons that can be independently retrieved and applied to hosts. This automated approach helps keep host firmware aligned with tested baselines and reduces the window during which outdated firmware is in use.

VCF Operations provides monitoring for firmware integrity through management packs. The Management Pack for Lenovo Compute generates alerts for firmware unrecoverable boot device failure events detected by UEFI, as well as firmware update failed alerts when a firmware update does not complete normally and the target image is partially written. These automated alerts allow operations teams to detect and respond to firmware integrity issues.

VMware publishes security advisories for known firmware and software vulnerabilities affecting VCF components, with remediation guidance for each advisory.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 1.6

> Network wireless access: Authenticate all users on network wireless ports. Wireless intrusion detection: Detect unauthorized wireless access points.

**SCF Controls:** NET-15

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 1.6(1)

> Network wireless access: Authenticate all users on network wireless ports. Wireless intrusion detection: Detect unauthorized wireless access points.

**SCF Controls:** NET-15.1

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 1.13

> Untrusted network access: Control network traffic from untrusted zones. Network access control (NAC): Implement NAC for untrusted devices.

**SCF Controls:** NET-01.1

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides several foundational capabilities that support a Zero Trust Architecture approach, though a complete ZTA implementation requires organizational policies, identity governance processes, continuous verification mechanisms, and additional tooling beyond what VCF delivers on its own.

VCF enforces authentication as a prerequisite to all access. The platform's authentication framework determines whether users are who they claim to be through credentials including smart cards, usernames, and passwords. VCF SSO integrates with the VCF Identity Broker to provide centralized authentication for VCF components, with support for identity provider federation to enterprise directories such as Active Directory Federation Services. Federating vCenter to an enterprise identity provider provides flexible options such as multi-factor authentication. OAuth 2.0 support allows information exchange among web services without exposing sensitive data such as usernames and passwords, reducing credential exposure during authentication flows. NSX network devices require authentication prior to establishing any management connection for administrative access, and all administrative accounts must be password protected. VMware ESX and vCenter systems restrict connections to authenticated users and they can access only those objects for which they have the required privileges and are authorized to use or view. Direct access to the ESX host console can be restricted to prevent administrators from overriding policy enforcement, and ESX supports configuration of users who can override lockdown mode and access the DCUI. For Windows virtual machines, vSphere Virtualization-based Security supports Credential Guard, which isolates and hardens key system and user secrets in the guest operating system against compromise. At the workload consumption layer, VMware Kubernetes Service (VKS) clusters support multiple authentication mechanisms for the Kubernetes API server, including OIDC federation with VCF SSO, X.509 client certificates, service account tokens, and webhook token authentication. Namespace-level access in VKS follows the same VCF SSO identity model as the broader vSphere platform. VKS implements a zero-trust token validation posture for external integrations such as AWS IRSA, trusting tokens only when issuer, signature, audience, and subject conditions are verified. Kubernetes guidance recommends enabling as few authentication mechanisms as necessary to simplify user management and prevent retained access through unused pathways, and it recommends setting short certificate lifetimes with automated rotation to limit the window of opportunity for credential misuse.

Once authenticated, VCF applies authorization controls that restrict what each principal can do. The vCenter AuthorizationManager controls access to specific server objects based on the permissions associated with each object. Authenticated users can view objects or invoke operations on the server only when the permissions associated with their account allow it. Administrators can create, modify, and manage roles and permissions through the AuthorizationManager, enabling granular role-based access control across the vSphere environment. The CloudAdmin role restricts access to platform-level operations, and the No Cryptography Administrator system role supports all Administrator privileges except for Cryptographic Operations privileges, allowing organizations to separate encryption key management from general administration. You can clone the No Cryptography Administrator role and create custom roles with only some of the Cryptographic Operations privileges to further limit what users can do. NSX provides granular role separation within the VPC scope, where you can assign VPC Admin, Network Admin, Security Admin, Network Operator, and Security Operator roles. A VPC Admin has full access to all networking and security objects inside an NSX VPC, while other roles provide narrower access. This "authenticate first, then authorize" model aligns with core ZTA principles of never granting implicit trust. For containerized workloads, Kubernetes RBAC provides Role, ClusterRole, RoleBinding, and ClusterRoleBinding objects that define what each authenticated identity may perform at the namespace or cluster level. Least-privileged access applies even to administrators; elevated access is reserved for break-glass scenarios while standard operations use narrower roles. Authorization webhooks support fine-grained, attribute-based authorization decisions by passing field and label selector information to the webhook, enabling per-resource access control based on object attributes such as node name and labels. Dynamic Resource Allocation APIs carry elevated privilege implications and should be controlled using RBAC to restrict access

based on user persona. Pod Security Admission enforces workload security profiles at the namespace level, with Baseline and Restricted profiles limiting privilege escalation, host namespace access, and container capabilities. Security contexts allow administrators to require that containers run as non-root users, drop Linux capabilities, apply seccomp profiles, or use AppArmor to restrict system call access at runtime. Cluster administrators can create custom pod security policies beyond the platform defaults and bind them to specific users, applying different trust levels to different teams or namespaces. Argo CD, available as a standard package for VKS clusters, supports local account authentication backed by global RBAC policies that define access control through policy rules and group mappings, extending authenticated and authorized access controls to the continuous delivery pipeline. Harbor, also deployable as a VKS standard package, enforces role-based access control for artifact management, requiring authentication and authorization before users can pull or push container images and charts.

Solution-to-solution communication within VCF uses certificate-based trust rather than passwords. VCF Automation establishes trust relationships with servers by verifying and trusting certificate information for SSL communication. When a client receives an element from VCF Operations, it verifies the server identity and decides whether to trust the signature before proceeding. Mutual TLS authentication based on self-signed or CA certificates is required between hosts and the NSX Central Control Plane. The VCF Identity Broker provides a centralized authentication source for VCF components, and vCenter and NSX components connect to it for SSO-based access. This certificate-based trust model means that VCF's own internal communications follow a verify-then-trust pattern rather than assuming trust based on network location. At the consumption layer, the Secret Store Supervisor Service enforces RBAC for administrators, DevOps engineers, and application tenants managing and distributing sensitive data, providing controlled access to credentials without broad exposure. Kubernetes guidance recommends using short-lived secrets to limit the impact of unintentional credential exposure, and audit rules can alert on anomalous access patterns such as concurrent reading of multiple secrets by a single user. Scoping IAM role trust policies to specific service accounts and namespaces enforces least-privilege at the credential level within VKS deployments.

VCF's networking layer provides segmentation capabilities that support ZTA network isolation. NSX projects enable multi-tenancy by isolating security and networking objects across tenants within a single NSX deployment, so that each tenant's resources are separated from others. NSX VPC enables tenant-level isolation of both networking and security objects, with each VPC operating as a self-contained boundary. Security groups can further define isolation settings for on-demand private or outbound networks. Kubernetes network policies can be imported to NSX and converted to vDefend policies, extending consistent security boundaries to containerized workloads alongside traditional VMs. The vSphere Distributed Services Engine supports zero-trust security by offloading network security processing to a dedicated data processing unit, separating security enforcement from the host operating system. These segmentation capabilities allow organizations to build network architectures where lateral movement between zones requires explicit authorization. VMware vDefend extends this foundation with microsegmentation and identity-based firewall rules that integrate with Active Directory for user-aware access enforcement at the workload level. At the Kubernetes networking layer, Cilium, a supported CNI option, enforces network policies at layers three through seven using an identity-based security model that is decoupled from network addressing, allowing policy to follow workload identity rather than IP addresses. Istio's ambient data plane mode, available as a standard package for VKS clusters, uses a per-node ztunnel component to provide mutual authentication and encrypted transport for east-west traffic within the mesh, extending verify-before-trust semantics to workload-to-workload communication. Service meshes operating at layer seven can enforce policies based on workload identity and frequently provide mutual TLS to maintain data confidentiality even in the presence of a compromised node.

VCF Operations provides visibility into platform activities through audit event collection. VCF Operations reports on authentication and security-related audit events, and each audit event has a detailed view containing information that helps analyze events in the virtual infrastructure for suspicious user actions or system issues. This supports the continuous monitoring aspect of zero trust by correlating authentication events, configuration changes, and related activity across the environment.

To realize a full Zero Trust Architecture, organizations need to supplement VCF's technical controls with an overarching ZTA policy framework that defines trust boundaries, continuous verification requirements, and least-privilege access principles across all systems. Device posture assessment, continuous session monitoring, and adaptive access policies based on risk signals typically require integration with identity governance platforms and security analytics tools beyond what VCF provides natively.

Zero Trust Architecture requires that every user and device be authenticated and authorized before accessing data or resources, with no implicit trust based on network location or prior access. VMware Private AI Services (PAIS) implements authentication and access control mechanisms across its AI services layer that support this model.

PAIS enforces access through OIDC-based group membership. The PAISConfiguration custom resource requires an issuer URL, a client ID, and scopes including openid, groups, and offline\_access, with a groups claim carried in the access token. Access is restricted to user accounts that belong to authorized groups configured via the authorizedGroups setting, and users outside those groups are denied access regardless of network position. Communication with the OIDC provider is required over HTTPS, and an issuer certificate must be present for that provider. PAIS supports a single OIDC provider per configuration.

Certificate-based mutual trust is enforced across PAIS component connections. CA trust bundles are required for the OIDC provider, the Harbor registry hosting the model gallery, content management systems used as data sources in Data Indexing and Retrieval, MCP servers providing tools to models, and the pgvector database server storing vector embeddings for Knowledge Bases. Trust bundles are configured through VCF Automation by uploading a certificate file or referencing a ConfigMap in the namespace, and the worker.clientTls.caBundleRefs setting controls the CA bundle references used for client TLS connections. Separate trust bundles per application are recommended, with refreshes required before certificate expiration.

At the model and data plane, the Model Gallery uses Harbor project access controls to restrict access to sensitive model training and tuning data, and model endpoints authenticate to private registries through a harbor-pull-secret-readonly secret. Remote MCP servers connected to PAIS can use static authentication tokens, and Knowledge Base data sources such as Confluence and SharePoint require a user name and password or a personal access token configured per administrator settings. Grafana dashboards visualizing PAIS metrics also authenticate through the OIDC provider, extending OIDC-mediated access into the observability path.

### **VMware vDefend (Contributes)**

VMware vDefend (vDefend) is a foundational technology for implementing Zero Trust Architecture (ZTA) within VCF. While a full ZTA implementation also requires identity governance, organizational policy, and continuous verification processes beyond any single product, vDefend provides the network security enforcement layer that treats all east-west traffic as untrusted by default and enforces explicit authorization at the workload level.

vDefend enforces security policies at the vNIC level, meaning every virtual machine's network interface is individually firewalled regardless of its network location. Network location does not confer trust, and every communication must be explicitly permitted. The default-deny posture of Distributed Firewall rules means workloads cannot communicate unless a policy explicitly allows that traffic, reducing the risk of lateral movement by treating all internal traffic as untrusted. The Distributed Firewall Connectivity Strategy feature simplifies micro-segmentation rule creation for applications, lowering the operational barrier to deploying a Zero Trust model.

Security Services Platform provides a guided security segmentation journey that walks administrators through a staged implementation of zero trust using vDefend. This workflow moves from broad infrastructure segmentation through environment-level isolation to fine-grained application ringfencing, where policies are defined down to the individual application tier. Security Intelligence complements this journey by providing the visibility and analytics needed to understand application security policy requirements, supporting identification of gaps in the zero trust posture. Security Services Platform generates Security Segmentation Reports and Blast Radius Reports that break down at-risk workloads, communication chains, and network flows to support Zero Trust journey planning. Security Services Platform also enforces role-based access control by assigning platform roles to remote users and user groups, subjecting access to the security management plane to the same authorization principles that govern workload-level policies.

vDefend Identity Firewall extends the zero trust model beyond IP-based rules by integrating with Active Directory to create firewall rules based on user identity and group membership. Administrators can define policies that grant or restrict access based on authenticated user identity rather than IP address. Identity Firewall rules can specify source users or groups, destination resources, services, and context profiles to allow or deny traffic based on user identity. Identity Firewall also



supports simultaneous logins by multiple users on the same virtual machine, with each user's application access controlled based on their role requirements. Administrators can use Identity Firewall with Remote Desktop Session Host to create Active Directory security groups that allow or deny users access to application servers based on their assigned role. Identity Firewall supports two login detection methods: Event Log Scraper (ELS), which reads Windows security event logs from domain controllers, and Guest Introspection, which captures authentication events from within the guest operating system. The choice between these methods depends on the workload type and deployment environment.

#### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer contributes to Zero Trust Architecture by enforcing authentication and authorization at the application delivery layer before client requests reach protected workloads. Avi's SSO Policy, configured per virtual service, supports Authentication Rules and Authorization Rules that validate JSON Web Tokens, OAuth 2.0 access tokens, and OIDC claims on every incoming request. Clients that present invalid or absent credentials are denied access before any request is forwarded to the backend resource. This per-virtual-service enforcement model means each application can independently require successful authentication as a condition of access.

Auth Mapping Profiles extend this authentication model into the Avi Controller's administrative plane. These profiles map authenticated user identities, including identities sourced from LDAP, SAML, or other external directory services, to specific tenants and roles with varying permission levels. Auth Mapping rules can be configured to deny all privileges to specific users, blocking them from obtaining any role or tenant assignment on login. When remote authentication is configured, administrators can independently enable or disable local user login, supporting stricter authentication policy enforcement. A user without a matching rule in the Auth Mapping Profile receives no privileges on login. Auth Mapping Profiles should be configured to enforce tenant isolation except for designated super users who require cross-tenant access.

At the management plane, administrators can restrict the source IP address ranges permitted to reach the Avi Controller and enforce strong password requirements. User Account Lockout is implemented on the Controller as a mechanism that blocks access after repeated failed authentication attempts. User accounts can be scoped to specific tenants and granted different roles within each tenant, limiting the blast radius of a compromised credential. HTTP security policies attached to virtual services are configurable to meet the security requirements applicable to each deployed application.

In VCF-integrated deployments, the Avi PKI Profile contains the list of Trusted Authorities allowed to sign client certificates and can be used to implement Zero Trust security between the server and the client. This certificate-based mechanism supports mutual authentication at the application delivery tier, complementing token-based authentication for workloads that rely on client certificate validation rather than OAuth or OIDC flows.

Full Zero Trust Architecture across a VCF environment requires capabilities beyond what Avi provides at the application delivery tier. Network-layer microsegmentation between workloads is provided by VMware vDefend, which is a separate product. OAuth, OIDC, and SAML federation require an external identity provider. The organizational policies that govern how authentication is enforced for all users and all devices must be defined and maintained by the organization; Avi provides the enforcement point for application traffic but does not itself constitute a complete ZTA implementation.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 1.13(1)

> Untrusted network access: Control network traffic from untrusted zones. Network access control (NAC): Implement NAC for untrusted devices.

**SCF Controls:** NET-04, NET-04.1

**Aggregate Coverage:** Meets

### VCF (Contributes)

VCF provides multiple layers of access control list (ACL) mechanisms and network traffic restriction capabilities that contribute to data flow enforcement, though full ACL governance across physical and virtual infrastructure requires organizational processes and may benefit from VMware vDefend for advanced stateful firewall rule sets.

At the network infrastructure layer, VCF uses vSphere Distributed Switches to centrally manage VLAN assignments, port group policies, and traffic shaping rules across all hosts in a workload domain. Distributed switches support traffic shaping policies that limit the bandwidth of incoming traffic, helping restrict which traffic types and volumes can traverse each network segment. Distributed port groups and uplink port groups also support a Traffic Filtering and Marking Policy, where per-rule actions can be set to Allow, Drop, or Tag traffic based on defined match criteria, providing rule-based traffic enforcement at the virtual switch level. Traffic segmentation is built into VCF's architecture: management, vMotion, vSAN, and NFS traffic each run on dedicated VLANs and VMkernel ports, and the vMotion network must be distinct from the vSAN network. This separation restricts which traffic types can traverse each segment by design. Private VLANs (PVLANs) provide additional Layer 2 isolation within a single VLAN through virtual network segmentation on distributed virtual switches, limiting communication between VMs even on the same segment. Distributed switches also support VLAN-based SPAN (VSPAN) for virtual distributed port mirroring, which aids in monitoring and validating traffic flows. At the ESX host level, a built-in firewall operates on a deny-by-default basis, closing all ports except those an administrator explicitly designates as authorized, giving each host an independent network access control point.

NSX extends these capabilities with software-defined networking constructs including segments, Tier-0 and Tier-1 gateways, and Virtual Private Clouds (VPCs). NSX Tier-0 gateways can be configured with linked Tier-1 gateways to create multi-tier topologies, enabling route-based traffic restrictions between network tiers. NSX edge gateways handle north-south routing. With vDefend deployed, North-South Firewall Rules on these gateways provide traffic protection, and an NSX project's default gateway firewall and distributed firewall rules (a vDefend capability) govern the default behavior of north-south and east-west traffic for workloads in that project. VPCs are self-contained virtual network environments that provide an abstraction layer for setting up isolated virtual private cloud networks with their own topology, IP address management, and permission models, allocatable to discrete projects in a self-service consumption model. NSX Manager enforces approved authorizations for controlling the flow of management information within network devices based on information flow control policies.

NSX also provides data flow enforcement at the load balancer tier through its Access List Control (ALC) feature. The NSX load balancer compares all traffic flowing through it with ACL statements, which either drop or allow the traffic based on configured rules. This gives administrators another enforcement point for restricting network traffic to only what is authorized.

At the Consumption layer, VMware Kubernetes Service (VKS) and the Supervisor extend ACL enforcement to containerized workloads through Kubernetes NetworkPolicy resources, which function as Layer 3/4 ACLs for pod-level traffic. NetworkPolicy objects specify allowed ingress and egress traffic by pod selector, namespace selector, IP block, port, and protocol; pods not selected by any NetworkPolicy receive no restriction by default, making explicit policy deployment necessary to enforce a default-deny posture across a cluster. VKS clusters run Antrea as the default CNI plugin, which implements NetworkPolicy enforcement through the AntreaPolicy feature gate and provides per-policy and per-rule packet and byte count statistics through the NetworkPolicyStats capability, giving administrators visibility into which policies are actively restricting traffic. For VKS clusters using NSX VPC networking, NSX-backed network policies apply distributed ACLs at the hypervisor level, extending enforcement beyond the CNI layer. The ovn-kubernetes Transparent network mode similarly supports distributed ACLs applied via Kubernetes NetworkPolicy resources. For Layer 7 traffic control within VKS clusters, the Istio service mesh standard package provides a pluggable policy layer and configuration API supporting access controls, rate

limits, and quotas based on workload identity; when deployed in ambient mode with strict mutual TLS, all service-to-service traffic within the mesh is subject to authentication before authorization checks apply, restricting lateral movement to explicitly permitted paths.

For file-level and storage data flow isolation, the Supervisor uses ACLs to isolate file share access within supervisor namespaces, restricting data access between workloads running in different namespaces. vSAN file services support managing ACLs on SMB shares using the MMC tool, giving administrators granular control over which users and systems can access shared storage resources. Administrators should be aware that ACL-based file share isolation in the Supervisor carries a risk of IP spoofing, which should be addressed through complementary network controls when deploying file shares in multi-tenant environments. Argo CD deployments on the Supervisor implement network segmentation and firewall rules to control communication between Argo CD components, following the principle of restricting inter-component traffic to only what each component requires. The Harbor container registry standard package enforces role-based access control over artifact access, complementing network-layer ACLs with image-level access restrictions.

For object-level access control, VCF Automation maintains an ACL table that defines which users can access specific defined entities. System administrators with the Manage General ACL right can create, modify, and remove ACL entries through the accessControls API. Those with the View General ACL right can audit ACL assignments on specific entities. Access to runtime defined entities is controlled through rights bundles and ACL entries. This ACL mechanism works alongside role-based access controls so that rights determine what actions a user can perform while ACLs determine which objects those actions apply to.

VCF Operations for Networks supports ACL visibility through its network assurance and verification capabilities, which allow administrators to validate that ACL configurations are applied as intended across the managed environment. This includes the ability to verify ACL configurations on supported physical network equipment such as Cisco ASR routers, helping validate that physical and virtual network access controls are consistent.

For workload-level traffic enforcement with stateful firewall rules, microsegmentation, and deep packet inspection, VMware vDefend and Gateway Firewall extend VCF's networking foundation with fine-grained Layer 2-7 policy enforcement. Organizations that require ACL governance beyond what VCF provides natively should establish processes for ACL review cycles, change management for firewall rules, and regular validation of both physical and virtual network access controls.

### **VMware vDefend (Meets)**

VMware vDefend provides the primary data flow enforcement mechanism within VCF through its Distributed Firewall, Gateway Firewall, and Security Intelligence capabilities. These capabilities function as software-defined access control lists governing traffic between workloads, segments, and external networks.

The vDefend distributed firewall enforces Layer 2-7 access policies at the vNIC level on every workload. Administrators define firewall rules that explicitly permit or deny traffic based on source and destination groups, ports, protocols, application identity, user identity via Active Directory integration through Identity Firewall, and fully qualified domain names (FQDN filtering). Because enforcement happens at the workload's virtual network interface, traffic is filtered before it reaches the physical network, and policies follow the workload as it moves between hosts. The Application Connectivity Strategy's "Allow Intra, Deny Other Traffic" mode automatically generates an intra-group ALLOW rule with optional logging plus a default ANY/ANY/DROP rule to isolate the group, providing a structured default-deny posture that restricts traffic to only what is explicitly authorized.

URL filtering provides an additional access control layer, enabling rule-based restrictions based on URL categories, URL reputation scores, and custom URL lists applied to both HTTP and HTTPS traffic. Identity Firewall rules can specify source users or groups, destination resources, services, and context profiles to allow or deny traffic based on Active Directory group membership. These rules support Allow and Reject actions and can be applied to physical devices where users log in, as well as to tier-0 and tier-1 gateways. Role-based access control within the Distributed Firewall governs who can create or modify firewall rules, restricting policy changes to authorized personnel.

The vDefend gateway firewall provides stateful North-South traffic inspection at Tier-0 and Tier-1 gateways, enforcing ACL-equivalent rules on traffic entering or leaving network segments. The Gateway Firewall supports Identity Firewall integration

with Active Directory for identity-based access control at the perimeter. Together, the Distributed and Gateway Firewalls address both East-West (inter-workload) and North-South (perimeter) data flow enforcement.

Security Intelligence, hosted on the Security Services Platform, supports ACL governance by observing traffic flows between applications and environments. Security Intelligence classifies flows as allowed, blocked, or unprotected and generates policy recommendations with segmentation strategy actions of Allow (creates a default traffic flow allow rule) or Drop (creates a default traffic flow drop rule). When a security policy is published, a rule is automatically generated to allow intra-application traffic. The View Impacted Flows capability displays unprotected traffic flows used to generate policy recommendations, helping administrators identify unauthorized communication patterns and tighten ACL governance over time.

The Security Services Platform governs access to Security Intelligence operations through role-based access control with five defined roles: Enterprise Admin, Auditor, Security Engineer, Security Operator, and Support Bundle Collector. Three permission levels apply across operations: Full Access (Create, Read, Update, and Delete), Read, and None. This restricts policy recommendation and enforcement actions to authorized security personnel.

### **VMware Avi Load Balancer (Contributes)**

Avi Load Balancer provides network-layer access control list (ACL) enforcement at the application delivery layer through Network Security Policies attached to virtual services. These policies evaluate incoming client traffic based on network-level parameters and determine whether a connection should be accepted, discarded, or rate-limited. Rules reference named IP Groups, which are reusable collections of IP addresses, IP ranges, and IP network prefixes. A Network Security Policy that denies traffic stops further policy evaluation, so ACL denials take effect before any downstream HTTP Security, Request, or Response policies execute.

Administrators configure Network Security Policies on individual virtual services through the Avi Controller. IP Groups are managed centrally and can be referenced across multiple virtual services, providing a scalable method for applying consistent ACL rules across the environment. The Avi Controller API supports programmatic creation and modification of Network Security rules that match on IP address and apply IP Groups to control access, enabling automated ACL governance through scripts or orchestration systems.

At Layer 7, Avi WAF rule actions support pass (allow) and deny (block) options for controlling traffic based on policy matches, extending data-flow enforcement to the application layer. Avi Load Balancer also supports OAuth/OIDC authorization policies that can match token scope attributes and request paths using CONTAINS and BEGINS\_WITH match criteria, enabling granular, path-based access control for API endpoints.

The Avi Controller enforces IP Access Lists on its own management interfaces by programming Linux IP tables on the Controller host, restricting network-layer access to specified management endpoints. These lists, configured under Administration > Controller Access Settings, limit which client IP addresses may reach the Controller API and UI.

These mechanisms scope Avi's ACL enforcement to traffic transiting its virtual services and to Controller management-plane access. ACL governance for east-west and other network traffic not flowing through Avi virtual services relies on VMware vDefend and VCF network controls.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 2.4

> Network mobile code integrity: Verify integrity of mobile code in transit. Network mobile code authenticity: Verify authenticity of mobile code in transit. Network mobile code authorization: Block unauthorized mobile code at boundary. Network mobile code policy: Enforce mobile code security via network devices. Deep packet inspection: Inspect network traffic for unauthorized mobile code.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 2.4(1)

> Network mobile code integrity: Verify integrity of mobile code in transit. Network mobile code authenticity: Verify authenticity of mobile code in transit. Network mobile code authorization: Block unauthorized mobile code at boundary. Network mobile code policy: Enforce mobile code security via network devices. Deep packet inspection: Inspect network traffic for unauthorized mobile code.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 2.4(a)

> Network mobile code integrity: Verify integrity of mobile code in transit. Network mobile code authenticity: Verify authenticity of mobile code in transit. Network mobile code authorization: Block unauthorized mobile code at boundary. Network mobile code policy: Enforce mobile code security via network devices. Deep packet inspection: Inspect network traffic for unauthorized mobile code.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.



## Control NDR 2.4(b)

> Network mobile code integrity: Verify integrity of mobile code in transit. Network mobile code authenticity: Verify authenticity of mobile code in transit. Network mobile code authorization: Block unauthorized mobile code at boundary. Network mobile code policy: Enforce mobile code security via network devices. Deep packet inspection: Inspect network traffic for unauthorized mobile code.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 2.4(c)

> Network mobile code integrity: Verify integrity of mobile code in transit. Network mobile code authenticity: Verify authenticity of mobile code in transit. Network mobile code authorization: Block unauthorized mobile code at boundary. Network mobile code policy: Enforce mobile code security via network devices. Deep packet inspection: Inspect network traffic for unauthorized mobile code.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 2.13

> Network device tamper detection: Detect physical tampering of network hardware. Network physical security: Prevent unauthorized physical port access. Port lockdown: Disable unused physical network ports.

**SCF Controls:** TDA-05.1

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 2.13(1)

> Network device tamper detection: Detect physical tampering of network hardware. Network physical security: Prevent unauthorized physical port access. Port lockdown: Disable unused physical network ports.

**SCF Controls:** TDA-05.2

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides several diagnostic and test interfaces across its component products, along with event logging and alerting capabilities that enable organizations to monitor access to those interfaces.

VCF's diagnostic interfaces include the vSphere DiagnosticManager, a managed object available on both VMware ESX (ESX) and VMware vCenter (vCenter) systems. The DiagnosticManager operates service-wide rather than on a per-session basis, providing operations for generating diagnostic bundles and obtaining information about system logs. The DiagnosticManager API can be accessed through the Managed Object Browser (MOB), and all MOB access is authenticated and logged. The vCenter REST API also exposes an endpoint for validating the current log forwarding configuration, which can optionally send a test diagnostic log message from the vCenter instance to all configured logging servers. On the storage side, vSAN has a network diagnostic mode that, when enabled, creates a ramdisk on a host to collect and save network metrics generated during performance testing. NSX provides the Online Diagnostic System (ODS), which exposes APIs to check debugging reports generated by runbooks.

Access to these diagnostic and test interfaces is captured through VCF's event logging infrastructure. vCenter records management operations and API calls as events, which can be used to track when diagnostic interfaces are invoked. Access to the DiagnosticManager and other vSphere API endpoints is subject to role-based access control and is logged through the vCenter event system. Administrators can define alarm rules in vCenter to monitor specific events, with each alarm rule targeting a single event type. When alarms are triggered, vCenter can send SNMP traps and email notifications to designated recipients with configurable alarm subject information. The SNMP agent included with vCenter sends traps when the vCenter system is started or when an alarm is triggered. The vCenter SNMP agent supports SNMPv1, v2c, and v3 trap formats, with SNMPv3 supporting all security levels including authentication and privacy. The agent can also be configured to filter specific notification types, allowing administrators to control which categories of events are forwarded to SNMP management systems.

VCF Log Management collects and aggregates log data from across the VCF stack, including ESX hosts, vCenter, VCF Operations, and NSX. Log-based alerts can be configured through Symptom Definitions in Infrastructure Operations under Configurations. These alerts can detect patterns in log data that indicate access to diagnostic interfaces and generate corresponding notifications. VCF Operations supports outbound SNMP trap notifications as a mechanism for sending alerts to external SNMP management systems, with the ability to send alert notifications to up to four SNMP trap destinations. Filtering is available when defining notifications using an SNMP Trap destination, allowing administrators to focus alerts on diagnostic interface access events. Alert definitions can be created based on log events from any monitored system, allowing operators to build detection rules tailored to their environment.

VCF Operations for Networks captures audit logs for actions performed through its API, user interface, and CLI, providing visibility into administrative access to network diagnostic tools. VCF Operations for Networks can also forward problem alerts and platform or collector syslogs to remote syslog servers for centralized monitoring.

NSX Manager provides monitoring dashboards showing details regarding system health, networking and security, and compliance reporting, which add visibility into diagnostic activity across the network infrastructure. Platform Operators can configure alerts, perform log analysis and troubleshooting, and run infrastructure diagnostics and health monitoring through the VCF management plane, with all actions logged for audit purposes.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.



## Control NDR 3.2

> Network malware protection: Filter malicious code at network boundaries.

**SCF Controls:** END-04

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides several platform capabilities that support malicious code protection, though organizations must deploy additional tools to achieve full antimalware coverage across endpoints.

At the hypervisor layer, VMware ESX supports Secure Boot and code signing to restrict unauthorized code from executing on hosts. Trusted Platform Module (TPM) support enables boot-time integrity measurement, and VMware vCenter provides APIs to retrieve TPM event logs for validating boot-time measurements or configuring firmware trust with an attestation service. Administrators can verify that hosts have booted in a known good configuration by checking TPM device status and event logs through the vCenter trusted infrastructure API. ESX host TPM modules must be verified as running the latest 2.0 firmware, and if not in use, disabled in the BIOS. Virtual machines can be configured with UEFI firmware for guest operating system boot integrity. Windows guest operating systems support Virtualization Based Security (VBS), which enables Device Guard to help prevent malware from running and Credential Guard to protect credential secrets; these features are configurable through Group Policy on VMs with ESX 6.7 or later compatibility. Virtual machines can also have the Guest Integrity (GI) security device enabled through the VMware Host Client, which is the platform foundation for VMware AppDefense.

The vSphere Security Configuration Guide provides hardening recommendations for ESX hosts, including configuring authentication and authorization settings, disabling unnecessary services, and enabling security features like Secure Boot and TPM. These configuration baselines are industry-standard best practices for host hardening.

vSphere Lifecycle Manager and VCF Operations Configuration Profiles maintain desired-state configuration for ESX hosts. Configuration Drifts in VCF Operations provides a proactive method to monitor configuration changes and identify when vCenter instances and vSphere clusters deviate from their approved baselines. Drift status is updated through a collection cycle that runs every 8 hours. This helps identify when host software or configuration has been modified from its approved state, which can indicate unauthorized changes or tampering.

VCF's networking layer, NSX, supports integration with third-party endpoint protection solutions for virtual machine security. The platform documentation references specific partner integrations including Symantec Endpoint Protection and ESET Antivirus/Security software, which provide anti-malware, intrusion prevention, and firewall capabilities for guest virtual machines. NSX App IDs can be used in conjunction with these third-party solutions for network management and security policy enforcement. Organizations can implement security groups, define security policies, and use NSX's integration with third-party security solutions for virtual machine protection.

NSX Guest Introspection provides an agent-based framework that supports endpoint security within virtual machines. The Guest Introspection Agent must be installed and running in each guest VM, and logged-in users should not have the privilege to remove or stop the agent. This architecture enables third-party security solutions to inspect guest operating system activity without requiring agents to be installed directly on the hypervisor. The security and integrity of the guest operating system must be maintained for these integrations to function correctly.

For container workloads, the VCF CLI supports cosign signature verification for container images through the `imgpkg` tool, which enables finding and copying cosign signatures. Image pull authentication through `imagePullSecrets` restricts container image pulls to authenticated private registries. Plugin discovery image signature verification is supported, with configurable public key paths for the cosign verifier. These capabilities help restrict deployment of images that have been tampered with.

VCF Operations supports the deployment and monitoring of agents on endpoint virtual machines, with guest operation privileges required to install agents. Automated software update capabilities in vCenter help keep systems current with security patches, reducing the window of exposure to known malware exploits.

For organizations that deploy VMware vDefend as an add-on to VCF, distributed malware prevention capabilities become available. The NSX Application Platform hosts the Malware Prevention for vDefend feature, which can be deployed from the Local Manager UI in NSX Federation environments. When host clusters are activated for Distributed Malware Prevention, a service virtual machine (SVM) is deployed on each host to perform malware scanning. vDefend also supports IDS/IPS capabilities alongside distributed malware prevention. The DistributedMps runbook is available to verify the health of the Malware Prevention Service pipeline and diagnose issues such as protection not working on a particular VM or certain files not being scanned.

Malware prevention activities generate log data for audit and forensic purposes. The Malware Prevention for vDefend feature writes log files that can be used for troubleshooting and security monitoring, and remote logging can be configured on the malware prevention SVMs for centralized log collection.

VMware Private AI Services (PAIS) includes model validation capabilities that scan machine learning model files for malicious content before they are accepted into a Model Gallery. The validation runs as a sandbox process on Deep Learning VMs and covers integrity verification through hash code checking, malware scanning, serialization attack scanning, and inference behavior testing with NVIDIA Triton Inference Server. Both the malware and serialization checks run as part of the validation workflow required before a model can be uploaded to a Model Gallery and made available to downstream model endpoints, knowledge bases, or AI agents.

Machine learning model files are a category of artifact that traditional endpoint security tools may not inspect. Serialized models in formats such as Python pickle files can carry executable payloads that activate when an application or researcher loads the model. The PAIS Model Gallery intake workflow treats this upload boundary as a security checkpoint, requiring each artifact to pass hash, malware, serialization, and inference behavior tests before it is published for use. PAIS guidance also directs organizations to distribute only models that have been validated internally on a Deep Learning VM rather than pulling artifacts directly from the Internet, where models may contain malicious code or be tuned for malicious behavior. The validation is operated by MLOps engineers, who confirm that onboarded models meet the organization's security, privacy, and technical requirements.

For platform-level malware protection covering virtual machines, VMware ESX hosts, and general workloads running on the PAIS infrastructure, organizations rely on VCF platform controls and, where deployed, VMware vDefend. PAIS adds the specialized scanning layer applied to AI model artifacts at the point of intake, addressing a threat surface that general-purpose antimalware tools do not typically cover.

### **VMware vDefend (Contributes)**

VMware vDefend provides multiple layers of malicious code detection and prevention through its Malware Prevention Service and IDS/IPS capabilities. Together these address the detection aspect of this control directly and contribute to the eradication aspect by restricting malicious code from executing and spreading across the network.

The Malware Prevention Service deploys as a service virtual machine (SVM) on each host in a cluster, with Guest Introspection thin agents installed on each protected guest VM. This architecture separates the detection engine from the protected workload, giving the service visibility into file activity, process execution, and network events from outside the guest. On Linux guests, a File and Network Agent handles file and network traffic introspection. A Host Agent (MUX) component running on ESX hosts supports the Guest Introspection functionality.

The service detects both known and unknown malicious files, including zero-day threats. Supported file categories include Windows executables (.exe, self-extracting), Linux executables (.elf), installer files (.msi), and shortcut files (.lnk), among others. When a file is analyzed, Malware Prevention assigns a verdict and classifies the threat by malware class (virus, trojan horse, worm, adware, ransomware, spyware) and malware family. A single detection event can be associated with multiple malware families and classes. Upon detection, the service can take direct remediation actions, including denying access to suspicious files or deleting files identified as malicious. An allowlist capability enables administrators to override verdicts for files known to be legitimate, suppressing false positives without disabling broader detection.

Beyond file-based detection, Malware Prevention Service includes fileless malware detection to identify attacks that operate entirely within system memory using existing binaries and in-memory operations such as PowerShell or VBScript, without



leaving executable artifacts on disk. The service monitors Indicators of Compromise including process command lines, process hashes, process execution trees, and in-memory executions. In-memory script buffer executions are scored for threat severity and assigned a verdict of malicious or suspicious, with the associated malware family and class displayed on the Malware Prevention Dashboard.

When malware is detected, the service extracts artifacts and in-memory script buffers that security teams can use to define incidents of compromise and create new detection rules for similar future threats. This extends malware coverage as new threat patterns are identified within the environment.

The Malware Prevention Dashboard in Security Services Platform aggregates detection events for malicious files, suspicious files, and uninspected files over configurable time periods. Each unique file is tracked by hash, and events can be filtered by verdict criteria. This centralized visibility allows security teams to identify malware incidents, track detection trends, and coordinate eradication efforts across the virtualized environment.

vDefend IDS/IPS provides signature-based and behavioral intrusion detection and prevention at the distributed firewall level, complementing Malware Prevention with network-layer detection of malware delivery attempts, command-and-control communications, and lateral movement patterns associated with malware infections. IDS/IPS operates at the virtual network interface level, providing visibility into east-west traffic between workloads that perimeter security tools cannot inspect.

While vDefend provides strong detection and file-level remediation capabilities, complete eradication of malicious code from infected guest operating systems typically requires endpoint security agents operating inside the guest. vDefend restricts malware from spreading across the network and blocks or removes detected malicious files, but cleaning an already-infected system requires host-level remediation tooling.

#### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer contributes to malicious code protection at the application delivery layer through WAF-based payload inspection, IP Reputation threat intelligence, Application Rules, and Bot Management.

The Avi WAF inspects inbound HTTP/HTTPS request bodies and parameters against signature-based rules, including ModSecurity rules, to detect and block exploit payloads, injection attacks, and known malware delivery patterns. WAF rules are configurable per virtual service and can be tuned to reduce false positives while maintaining coverage of malicious traffic patterns. Configuration is managed through the Avi Controller UI under the WAF Policy section for each virtual service.

The Application Rules service, available through Avi Cloud Console Live Security Threat Intelligence, provides automatically updated WAF rules targeting known vulnerabilities with CVE mappings across more than 5,000 applications, including WordPress, Drupal, and Apache. These rules are pushed automatically to Avi Controllers, reducing the window between vulnerability disclosure and protection.

Avi's IP Reputation service provides a threat intelligence feed that classifies IP addresses known to be infected with malware or identified as contacting malware distribution points (threat category code 6). The service also identifies IP addresses known to distribute malware, shell code, rootkits, worms, or viruses, including Windows exploit IP addresses. An "All threats" classification (threat category code 32) can be applied to block any suspicious IP addresses at the application delivery layer. IP Reputation data is synchronized via Avi Cloud Services and can be referenced in virtual service policies to block malicious source traffic before it reaches protected workloads. This is configured under Administration > External Services > Cloud Services.

Avi Bot Management provides a default System-BotDetectionPolicy that can be bound to virtual services to detect and mitigate malicious bots used in online fraud, credential stuffing, and automated data scraping. Bot Management performs heuristic scanning of User-Agent strings to detect malicious patterns such as SQL injections, marking such requests as bad bot of type web attack. In Kubernetes deployments, the Avi Kubernetes Operator (AKO) supports assigning a BotDetectionPolicy to virtual services via the botPolicyRef parameter, extending bot detection to Kubernetes ingress workloads.

Avi WAF does not include built-in virus or file-content malware scanning. Avi documentation explicitly recommends deploying a dedicated virus and malware scanning tool on the upload directories of protected applications to detect and mitigate malicious uploads, including uploads that may bypass WAF inspection. Malware detection and eradication for workloads

running behind Avi is the responsibility of endpoint protection tools deployed on those workloads. Organizations should configure WAF policies, Application Rules, IP Reputation, and Bot Management on Avi virtual services while also deploying dedicated antimalware solutions on application workloads and their supporting infrastructure.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

### Control NDR 3.10

> Network device updates: Support secure updates for network device firmware. Verified network updates: Cryptographically verify network firmware updates.

**SCF Controls:** TDA-01.1

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 3.10(1)

> Network device updates: Support secure updates for network device firmware. Verified network updates: Cryptographically verify network firmware updates.

**SCF Controls:** CHG-04.2

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF enforces digital signature verification for software and firmware components at multiple layers of the stack, from hypervisor boot through application deployment.

During VCF deployment, each component is assigned a certificate from a default signing certificate authority. The VMware Certificate Authority (VMCA) is an internal CA that comes integral with vSphere deployment and has been purpose-built to serve the certificate needs of the vSphere environment. From initial installation onward, all VCF components operate within a certificate-based trust framework rooted in a known signing authority. Administrators can configure VCF to use certificates from VMCA, Microsoft CA, OpenSSL CA, or external third-party certificate authorities, giving organizations flexibility to integrate their approved CA hierarchy into the VCF signing and verification workflow. All communication between the VCF Installer and ESX hosts is performed over HTTPS with certificate identity validation, and the installer validates the identity of each host by comparing the certificate common name against the provided FQDN.

At the hypervisor level, VMware ESX uses UEFI Secure Boot to verify that the boot loader and kernel modules are digitally signed before execution. The host acceptance level setting controls which vSphere Installation Bundles (VIBs) are permitted: it can be configured to accept only VMware-certified VIBs, restricting installation to components signed by VMware's certificate authority. The vSphere Security Configuration Guide includes recommendations for enabling Secure Boot and configuring TPM on ESX hosts. When UEFI Secure Boot is enabled on an ESX host, the host performs full signature verification of each package installed on the host at every boot, not only at initial installation. vSphere also supports UEFI Secure Boot for individual virtual machines, where the bootloader, operating system kernel, and operating system drivers must each carry a valid signature before the virtual machine is permitted to boot.

vSphere Lifecycle Manager (vLCM) provides desired-state image management for ESX hosts. Administrators define a target image specification for each cluster, and vLCM checks host compliance against that image. When hosts deviate from the desired image, vLCM can remediate them back to the approved configuration. This approach restricts ad-hoc installation of unsigned or unapproved software components on managed hosts.

VCF Operations enforces digital signing for its management packs and orchestrator packages. Management pack PAK files are verified against a digital signature provided by VMware. If a PAK file lacks a valid digital signature, the system displays a warning indicating the file cannot be verified as authentic, and the digital signature indicates the original developer or publisher. For orchestrator packages, all packages exported from a VCF Operations orchestrator server are digitally signed, and the orchestrator's certificate infrastructure allows signing of all elements created within the system. When a client receives an element from the orchestrator server, the client verifies the server identity and decides whether to trust the signature. Package signing certificates are a form of digital identification used to support encrypted communication and provide a signature for orchestrator packages. For production environments, VCF recommends using a certificate signed by a certificate authority rather than a self-signed certificate, allowing organizations to align package signing with their approved CA infrastructure.

For container workloads, VMware Kubernetes Service (VKS) validates TLS certificates when fetching container images from registries and requires that registry certificates be signed by a trusted certificate authority. The platform requires that a certificate signing request be generated, signed by a certificate authority, and applied to the Workload Platform Management layer to secure the Kubernetes control plane. TLS certificates for the Supervisor layer are derived from the VMCA certificate, anchoring the Kubernetes control plane trust chain in the same root authority as the broader VCF infrastructure. The VCF CLI `imgpkg` tool supports CA certificate validation for registry connections through its `--registry-ca-cert-path` flag, which can be

specified multiple times to add CA certificates for registry API communication. VKS clusters can also be configured to trust any number of CA certificates for private registry endpoints, directing image pulls to controlled, organization-approved sources.

Harbor is available as a standard package for VKS clusters and provides enterprise container registry capabilities including content trust and image signing. Organizations can configure Harbor to enforce signature requirements at image push or pull time, restricting deployments to images signed under an organization-approved certificate authority. Admission controllers are the primary enforcement mechanism for image signing policies within VKS clusters. The ImagePolicyWebhook admission controller allows administrators to configure an external image verification backend that is consulted before any pod is scheduled; if the backend determines that an image lacks a valid signature from an approved authority, the pod is rejected before it runs. ValidatingWebhookConfiguration objects can integrate third-party image signing solutions with the Kubernetes API server, creating an enforcement layer that intercepts pod creation requests and validates image signatures as part of the admission control chain.

The Kubernetes control plane components used by VKS are themselves distributed as signed artifacts. Kubernetes release artifacts, including standalone binaries and Software Bill of Materials files, are signed using cosign keyless signing, and control plane component images can be verified using cosign with certificate-based identity and OIDC issuer validation. Kubernetes CertificateSigningRequest signers can deny requests that do not meet approval conditions, and CertificateSigningRequests with unknown signerName values are never auto-approved, restricting unauthorized certificate issuance within the cluster. For workload certificate lifecycle management, the cert-manager standard package creates TLS certificates for workloads and automatically renews them before expiration, with VKS cluster control plane certificate rotation configured through the ClusterClass API and defaulting to renewal 90 days before expiry. The renewal window is configurable through the renewalDaysBeforeExpiry parameter in the ClusterClass specification, allowing organizations to set this period to match their certificate management policies.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 3.11

> Network device integrity: Verify integrity of network device configuration. Hardware integrity (Network): Use hardware-based integrity for network assets.

**SCF Controls:** AST-08

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides monitoring, alerting, compliance assessment, and forensic capabilities that support periodic inspection of systems and system components for indicators of compromise (IoC). While VCF does not include a dedicated IoC detection engine or endpoint detection and response (EDR) tool, its operational monitoring and change-detection features contribute to an organization's ability to identify anomalous conditions that may signal compromise.

VCF Operations continuously monitors the managed environment by evaluating collected data against defined policies and assigning a compliance score to each object or group based on adherence to those policies. This compliance monitoring enables discovery of non-compliant objects and associated compliance violations, which can surface configuration drift or unauthorized changes that may indicate tampering or compromise. The Anomalies widget displays metric deviations from both dynamic and static thresholds, and the Potential Evidence tab surfaces events, property changes, and anomalous metrics that may be relevant to a given alert. Risk alerts flag conditions that are not immediate threats but warrant prompt investigation.

Configuration drift detection in VCF Operations compares current settings against assigned configuration template values and notifies administrators of any deviations. The Configuration Drift dashboard allows administrators to view and compare drifts and identify settings that have changed, while the Detect Drift function detects drift across all configuration templates and displays results in the Drifts table. This change-detection capability supports identification of unauthorized modifications to managed infrastructure.

VCF Operations provides a layered alerting framework. Log-based alerts can be configured through the Symptom Definitions card in Infrastructure Operations, and alert definitions can be created to monitor infrastructure events and conditions. The system correlates a subset of events published by monitored systems and delivers them as faults, while fault symptoms can be defined based on those events and added to alert definitions. This allows administrators to build detection logic around events that may indicate compromise.

VCF Operations collects security properties for virtual machines, including the connection state of CD-ROM, USB controller, parallel port, and serial port devices, which supports detection of unauthorized device connections to virtual infrastructure. Administrators can filter virtual machines by encryption status, vCenter name, host name, and VM name to assess compliance posture across the environment.

Audit event reporting in VCF Operations records configuration changes, user actions, and authentication events across the managed infrastructure. These audit events can be filtered by authentication status, user, vCenter instance, object type, and event description, giving investigators a detailed view of activity across the environment for conducting security forensics.

VCF Operations includes a Forensics widget that allows administrators to compare data across two time periods, search for and filter objects within the environment, and display widget data based on filtered object sets. This capability supports forensic comparison and analysis when investigating potential indicators of compromise. Guided Network Troubleshooting highlights anomalous entities and metrics within the network path, aiding root-cause analysis during incident investigation.

OS and Application Monitoring Properties are collected for operating systems, application services, remote checks, Linux processes, and Windows services. These monitoring data points can be used to create reports, views, and dashboards for ongoing system inspection, providing visibility into the operational state of managed components and flagging deviations from expected behavior.

VMware ESX hosts generate detailed audit logs, and multiple VCF components support syslog forwarding to external log aggregation and SIEM systems. VMware vCenter provides both API and UI methods to configure syslog forwarding. vSphere

Supervisor can forward control plane and Kubernetes API server audit logs to external syslog servers via Fluent Bit in RFC 5424 format. These log forwarding capabilities allow VCF telemetry to feed into external IoC detection and correlation tools. ESX also enforces integrity checks on installed VIBs through the Host Image Profile Acceptance Level, governing what software is permitted to run on the host and supporting detection of unauthorized additions to the hypervisor. vSphere provides preconfigured alarms that monitor the status of host hardware system boards, allowing notification when hardware-level anomalies are detected.

The NSX networking layer includes monitoring dashboards showing details about system health, networking, security, and compliance reporting. NSX supports logging and auditing capabilities that can be integrated with external logging and monitoring solutions to monitor for security events and anomalies. The System Health Agent on NSX components monitors component statistics, analyzes them, alerts on abnormalities, and can trigger remediation actions when specific conditions are met. VCF Operations for Networks monitors network flows and can detect outlier VMs based on flow metrics including bytes, packets, sessions, and traffic rate, supporting detection of anomalous traffic patterns within the environment. At the host layer, ESX measures boot components against a Trusted Platform Module (TPM) 2.0 hardware root of trust, and VMware vCenter exposes the resulting boot measurements and TPM event logs through its trusted-infrastructure API, so administrators can confirm a host booted in a known-good state and detect firmware or boot-level tampering. Secure Boot independently validates the cryptographic signatures of the ESX kernel and modules at startup, so unsigned or altered components are surfaced as an integrity failure rather than loaded silently.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer contributes to this control through its application-layer threat detection and event logging capabilities, which provide the data security teams need during periodic Indicators of Compromise (IoC) inspections.

Avi's Web Application Firewall (WAF) analyzes all incoming and outgoing application traffic using signature-based detection, a positive security model, client reputation scoring, machine learning, and outlier analysis. WAF signatures include detection for OWASP Top 10 attack vectors and other threat categories. The WAF operates in either Detection mode or Enforcement mode; Detection mode allows new rules to be evaluated without blocking legitimate traffic, making it suitable for baseline establishment and traffic anomaly review during periodic inspections. Partial detection allows individual rules to be placed in Enforcement mode while the policy as a whole remains in Detection mode, supporting incremental adoption.

The Avi Controller generates structured security events that auditors and security teams can review as part of periodic IoC inspections. The Controller tracks denial-of-service attack events with attack count metrics and generates alerts at configurable severity levels. The scan detection system dynamically classifies client IPs into Good, Bad, and Unknown groups based on observed traffic patterns, adjusting automatically as patterns change. The Controller also logs the initiator and target of failed trusted channel establishment attempts, including cases where an SSL certificate is absent, providing visibility into authentication anomalies.

Avi events serve as an audit trail for user logins, configuration changes, and runtime state changes, and can be forwarded to Splunk for centralized audit trail analysis. Client request logs can be collected automatically based on categories of significance for application-level review. System events provide a historical record of faults within the system. DataScripts support custom identification of suspicious traffic and execution of appropriate mitigation actions, extending detection coverage beyond built-in signatures.

Avi provides several tools suited for periodic review workflows. The application security report can be generated on demand through the Controller UI or CLI, with an optional duration parameter to define the report period, producing a point-in-time record of detected threats suitable for scheduled IoC review cycles. The Config Audit Trail, accessible at Operations > Events > Config Audit Trail, presents a filtered view of user activity and configuration change events over a selected period, supporting review of whether configuration changes preceded observed anomalies. Alert events can be streamed to external syslog servers using Syslog Audit Persistence, enabling integration with SIEM platforms that support automated IoC correlation. The Controller also generates system reports for upgrade operations containing pre-check results and operational status, which support verification of system state before and after changes.



The organization is responsible for defining the inspection cadence, determining what constitutes a compromise indicator in context, and performing the host-level and physical inspection that Avi does not address as an application delivery controller. Platform-level system integrity inspection relies on the underlying VCF platform.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 3.11(1)

> Network device integrity: Verify integrity of network device configuration. Hardware integrity (Network): Use hardware-based integrity for network assets.

**SCF Controls:** CFG-06.1

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides automated mechanisms to detect unauthorized deviations from approved baselines and to remediate them, operating across three complementary layers: configuration compliance monitoring, host software state management, and preventive policy enforcement. The VCF consumption layer extends these mechanisms to Kubernetes workloads through VMware Kubernetes Service (VKS), adding declarative self-healing, admission-based baseline enforcement, and workload-level audit logging.

VCF Operations includes a compliance management engine that continuously monitors infrastructure objects against defined baselines. The scope of monitoring covers VMware vCenter instances, vSAN, NSX, and cloud environments, with monitored objects including hosts, virtual machines, distributed port groups, and distributed switches. VCF Operations supports both built-in and custom compliance benchmarks, including VMware SDDC, VCF, regulatory, and custom benchmarks. Administrators can create custom benchmarks tailored to organizational requirements and edit alert definitions and policies at any time after creation. When VCF Operations detects that an object deviates from its compliance policy, it generates alerts and notifications. Depending on the severity of the non-compliance, administrators can configure automated remediation actions to bring the object back into compliance. For violations that require manual review, the compliance engine highlights risk areas and provides actionable remediation recommendations.

VCF Operations also includes a Configuration Drifts feature that proactively monitors configuration changes across vCenter instances and vSphere clusters. Administrators define desired values through configuration templates, and VCF Operations compares the current configuration of each managed object against those templates on a scheduled basis. When a deviation is detected, the system generates a drift notification identifying the specific settings that changed. The Configuration Drifts Overview page lets administrators view and compare drifts, and drift status reports can be downloaded for review and record-keeping. Configuration templates can be integrated with a source control repository for versioning, auditing, and change management, enabling accurate configuration updates and supporting change control with reviews.

For host software state, vSphere Lifecycle Manager takes a desired-state approach. Administrators define a desired software specification, including base image, components, and add-ons, for each cluster or standalone host through Image Manager. The system checks host compliance against this specification and, when the current state differs from the desired state, cluster remediation applies the desired specification to bring all hosts into alignment. Pre-check validation runs before remediation to confirm hosts are in a good state to be updated. Global and per-cluster remediation policies govern how the remediation process behaves. At the individual host level, VMware ESX also enforces software integrity through the Host Image Profile Acceptance Level, which governs the integrity validation requirements for VIBs (VMware Installation Bundles) before they can be installed on a host, providing a preventive control that complements Lifecycle Manager's desired-state remediation.

VCF Automation extends baseline enforcement to provisioned infrastructure through Policy as Code, which lets organization administrators enforce instance-centric policies on Infrastructure as a Service (IaaS) resources from a central location. IaaS Resource Policies support validation expressions that enforce constraints on resource creation and updates, such as requiring specific parameter values during provisioning. This preventive enforcement complements VCF Operations' detective and corrective controls by blocking non-compliant resource configurations before they are deployed.

At the Kubernetes workload layer, VKS clusters operate on a declarative state model where desired workload configurations are persisted as Kubernetes objects. The Kubernetes control plane continuously reconciles actual cluster state against the declared desired state: when a running pod crashes, is deleted, or is evicted from a node, the relevant controller automatically reschedules it to restore the declared replica count. Deployments maintain revision history and support rollback when a new rollout fails readiness or liveness checks. Kubernetes liveness probes provide a further container-level detection mechanism:

when a container's health check fails, Kubernetes automatically restarts that container to restore correct application state. At the node level, VKS MachineHealthCheck objects specify the conditions under which a cluster node is considered unhealthy, including condition types such as MemoryPressure and DiskPressure with configurable timeout durations. When a node meets those conditions, the Cluster API triggers automated machine replacement to restore the declared cluster state. MachineHealthCheck is configurable for both control plane and worker node pools and can be updated at runtime. This continuous reconciliation provides automated resiliency at the workload layer, complementing the infrastructure-level remediation provided by VCF Operations and vSphere Lifecycle Manager.

VKS clusters support Kubernetes ValidatingAdmissionPolicy objects, which define CEL-based (Common Expression Language) rules evaluated against each API request before it is persisted. The validationActions field on a ValidatingAdmissionPolicyBinding specifies how violations are handled: Audit records the violation in the Kubernetes audit log, Warn returns a warning to the requesting client, and Deny rejects the request outright. Policy failures are recorded in audit events with a validation.policy.admission.k8s.io/validation\_failure annotation, enabling automated monitoring pipelines to detect unauthorized changes. The Pod Security Admission controller provides enforce, audit, and warn modes for pod workload baselines, configurable per namespace or VKS cluster through the ClusterClass podSecurityStandard field. In enforce mode, pod submissions that violate the configured profile are rejected at admission; in audit mode, violations are recorded without blocking the pod. VCF Automation mutation policies with the pod-security template extend this further by automatically modifying pods to meet security standards before admission, providing active remediation rather than only rejection. The VKS Policy Insights page surfaces incompatibility detections, including pre-existing Gatekeeper installations not deployed via cluster management, flagging configuration deviations for administrative review.

For audit trail purposes, VCF Operations tracks each configuration change to the authenticated user who initiated it or scheduled the job that performed the change, providing accountability for changes that may cause compliance deviations. NSX Manager audit logs capture both old and new values for modified security policies, recording rule changes including operation status, timestamps, and user information. VCF Operations also provides the isolation.device.edit.disable setting to restrict unauthorized modification of device configurations on managed objects. At the Kubernetes layer, VKS audit logging records all API server activity with authorization.k8s.io/decision and authorization.k8s.io/reason annotations that capture authorization outcomes and their justification, providing a chronological record of all actions across the VKS cluster that can feed automated monitoring pipelines for drift detection.

### **VMware vDefend (Contributes)**

VMware vDefend (vDefend) contributes to integrity assurance at the network policy and security detection layers but does not perform host-level configuration baseline deviation detection or automated remediation of endpoint settings.

Security Intelligence, part of the Security Services Platform (SSP), continuously monitors network traffic flows and alerts administrators to new or unexpected flows between environments, enabling detection of deviations from established microsegmentation baselines. Security Intelligence evaluates whether security rules have been implemented and generates a security score to track policy enforcement progress, helping administrators identify gaps between the desired baseline and current policy state.

Within the IDS/IPS subsystem, vDefend tracks user modifications to system signatures, allowing administrators to filter and view only those signatures that have been customized or altered from their default state. Global Signature Management records signatures that have been changed or excluded by users. When deviations from the signature baseline need to be corrected, the bulk edit feature supports a Reset action that restores signatures to their default configuration, providing a resiliency action within the security detection layer.

For malware-related events, Intelligent Assist provides automated resiliency. Targeted remediation enables all IDS signatures for the affected workload; Comprehensive remediation automatically creates DFW policies to restrict access for affected systems. Applied remediation policies can be reviewed under Security > IDS/IPS & Malware Prevention > Emergency Threat Rules. Administrators can also implement lockdown protection rules to restrict traffic leakage between environment pairs as an additional resiliency action.

Automated remediation of unauthorized changes to system configurations or OS-level settings is outside vDefend's scope.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer.  
This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 3.12

> Network provisioning: Securely provision network device identities.

**SCF Controls:** AST-18

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF contributes to roots of trust protection through cryptographic key management, certificate trust chain infrastructure, hardware-based integrity verification, and software package signing across its component stack.

VCF manages cryptographic keys through integration with external Key Management Servers (KMS) that support the Key Management Interoperability Protocol (KMIP) 1.1 standard, or through the built-in vSphere Native Key Provider. VMware vCenter communicates with external KMS instances using the KMIP protocol, and the KMS must be capable of acting as a symmetric key foundry and server. The vSphere Native Key Provider can coexist with an existing key server infrastructure, giving organizations flexibility in how they provision and manage encryption keys. Trust between vCenter and a KMS can be established through multiple methods: uploading a KMS certificate directly, using the vCenter Root CA certificate, providing an uploaded certificate and private key, or generating a Certificate Signing Request for the KMS to sign. The vSphere Web Services API exposes an `establishTrust()` method that enables mutual trust between vCenter and a KMS server using self-signed certificates, signed certificate signing requests, or certificate and key pair exchange. For stronger security, administrators should upload a leaf certificate or intermediate CA certificate rather than a root CA certificate, since uploading a root CA causes vCenter to trust all certificates signed by that authority.

vCenter maintains a trusted root certificate store where administrators can add root certificates or certificate chains from enterprise or third-party certificate authorities. Adding a root certificate to this store establishes trust with that CA for all subsequent certificate validation across the vCenter environment. The `TrustedRootChains` API provides programmatic access to manage and retrieve root certificate chains by identifier, supporting automation of trust chain verification. The underlying certificate storage uses the VMware Endpoint Certificate Store (VECS), which maintains named stores including `TRUSTED_ROOTS` for all trusted root certificates and `TRUSTED_ROOTS_CRLS` for certificate revocation list data; administrators can manage these stores with the `vecs-cli` command-line tool. The `TRUSTED_ROOTS_CRLS` store must not be deleted or modified, as doing so can damage the certificate infrastructure. Control plane VMs in vSphere Supervisor environments rely on a Trusted Roots bundle that must be kept current with the latest vCenter root certificates.

At the hardware level, VCF supports Secure Boot and Trusted Platform Module (TPM) to help verify that hosts boot in known-good configurations. The vSphere Security Configuration Guide recommends enabling Secure Boot and TPM as part of VMware ESX host hardening. vCenter provides API interfaces for retrieving TPM event logs for configuring firmware trust with an attestation service or validating boot-time TPM measurements, and for listing configured TPM endorsement keys on a host. These capabilities support establishing a hardware-rooted chain of trust from firmware through the hypervisor.

Beyond core key and certificate management, several VCF components maintain their own trust infrastructure. VCF Operations orchestrator supports the import, export, and generation of package signing certificates, providing a mechanism for verifying the integrity and authenticity of software packages deployed within the environment. VCF Operations also maintains an SSL truststore with a password-protected keystore for managing certificates used in secure communications. VCF Automation can establish trust relationships with external servers by verifying and accepting certificate information for SSL communication, and it exchanges SAML service provider metadata with external identity providers to establish federated trust relationships. NSX components establish trust through certificate exchange between Local Manager and Global Manager instances during federation setup. VCF PowerCLI maintains its own Trusted Certificate Store where trusted certificates are saved and used when connecting to server systems, extending the trust chain to administrative automation tooling.

VCF provides the cryptographic infrastructure for establishing, storing, and verifying trust relationships through certificates and keys across all its major components. However, the organizational processes for provisioning supplier keys, verifying supply chain integrity of cryptographic material, defining key protection policies, and physically protecting key storage remain the responsibility of the deploying organization.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 3.13(a)

> Network boot integrity: Verify network device bootloader integrity. Network boot authenticity: Verify network device bootloader authenticity.

**SCF Controls:** AST-18

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF contributes to roots of trust protection through cryptographic key management, certificate trust chain infrastructure, hardware-based integrity verification, and software package signing across its component stack.

VCF manages cryptographic keys through integration with external Key Management Servers (KMS) that support the Key Management Interoperability Protocol (KMIP) 1.1 standard, or through the built-in vSphere Native Key Provider. VMware vCenter communicates with external KMS instances using the KMIP protocol, and the KMS must be capable of acting as a symmetric key foundry and server. The vSphere Native Key Provider can coexist with an existing key server infrastructure, giving organizations flexibility in how they provision and manage encryption keys. Trust between vCenter and a KMS can be established through multiple methods: uploading a KMS certificate directly, using the vCenter Root CA certificate, providing an uploaded certificate and private key, or generating a Certificate Signing Request for the KMS to sign. The vSphere Web Services API exposes an `establishTrust()` method that enables mutual trust between vCenter and a KMS server using self-signed certificates, signed certificate signing requests, or certificate and key pair exchange. For stronger security, administrators should upload a leaf certificate or intermediate CA certificate rather than a root CA certificate, since uploading a root CA causes vCenter to trust all certificates signed by that authority.

vCenter maintains a trusted root certificate store where administrators can add root certificates or certificate chains from enterprise or third-party certificate authorities. Adding a root certificate to this store establishes trust with that CA for all subsequent certificate validation across the vCenter environment. The `TrustedRootChains` API provides programmatic access to manage and retrieve root certificate chains by identifier, supporting automation of trust chain verification. The underlying certificate storage uses the VMware Endpoint Certificate Store (VECS), which maintains named stores including `TRUSTED_ROOTS` for all trusted root certificates and `TRUSTED_ROOTS_CRLS` for certificate revocation list data; administrators can manage these stores with the `vecs-cli` command-line tool. The `TRUSTED_ROOTS_CRLS` store must not be deleted or modified, as doing so can damage the certificate infrastructure. Control plane VMs in vSphere Supervisor environments rely on a Trusted Roots bundle that must be kept current with the latest vCenter root certificates.

At the hardware level, VCF supports Secure Boot and Trusted Platform Module (TPM) to help verify that hosts boot in known-good configurations. The vSphere Security Configuration Guide recommends enabling Secure Boot and TPM as part of VMware ESX host hardening. vCenter provides API interfaces for retrieving TPM event logs for configuring firmware trust with an attestation service or validating boot-time TPM measurements, and for listing configured TPM endorsement keys on a host. These capabilities support establishing a hardware-rooted chain of trust from firmware through the hypervisor.

Beyond core key and certificate management, several VCF components maintain their own trust infrastructure. VCF Operations orchestrator supports the import, export, and generation of package signing certificates, providing a mechanism for verifying the integrity and authenticity of software packages deployed within the environment. VCF Operations also maintains an SSL truststore with a password-protected keystore for managing certificates used in secure communications. VCF Automation can establish trust relationships with external servers by verifying and accepting certificate information for SSL communication, and it exchanges SAML service provider metadata with external identity providers to establish federated trust relationships. NSX components establish trust through certificate exchange between Local Manager and Global Manager instances during federation setup. VCF PowerCLI maintains its own Trusted Certificate Store where trusted certificates are saved and used when connecting to server systems, extending the trust chain to administrative automation tooling.

VCF provides the cryptographic infrastructure for establishing, storing, and verifying trust relationships through certificates and keys across all its major components. However, the organizational processes for provisioning supplier keys, verifying



supply chain integrity of cryptographic material, defining key protection policies, and physically protecting key storage remain the responsibility of the deploying organization.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 3.13(b)

> Network boot integrity: Verify network device bootloader integrity. Network boot authenticity: Verify network device bootloader authenticity.

**SCF Controls:** AST-18

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF contributes to roots of trust protection through cryptographic key management, certificate trust chain infrastructure, hardware-based integrity verification, and software package signing across its component stack.

VCF manages cryptographic keys through integration with external Key Management Servers (KMS) that support the Key Management Interoperability Protocol (KMIP) 1.1 standard, or through the built-in vSphere Native Key Provider. VMware vCenter communicates with external KMS instances using the KMIP protocol, and the KMS must be capable of acting as a symmetric key foundry and server. The vSphere Native Key Provider can coexist with an existing key server infrastructure, giving organizations flexibility in how they provision and manage encryption keys. Trust between vCenter and a KMS can be established through multiple methods: uploading a KMS certificate directly, using the vCenter Root CA certificate, providing an uploaded certificate and private key, or generating a Certificate Signing Request for the KMS to sign. The vSphere Web Services API exposes an `establishTrust()` method that enables mutual trust between vCenter and a KMS server using self-signed certificates, signed certificate signing requests, or certificate and key pair exchange. For stronger security, administrators should upload a leaf certificate or intermediate CA certificate rather than a root CA certificate, since uploading a root CA causes vCenter to trust all certificates signed by that authority.

vCenter maintains a trusted root certificate store where administrators can add root certificates or certificate chains from enterprise or third-party certificate authorities. Adding a root certificate to this store establishes trust with that CA for all subsequent certificate validation across the vCenter environment. The `TrustedRootChains` API provides programmatic access to manage and retrieve root certificate chains by identifier, supporting automation of trust chain verification. The underlying certificate storage uses the VMware Endpoint Certificate Store (VECS), which maintains named stores including `TRUSTED_ROOTS` for all trusted root certificates and `TRUSTED_ROOTS_CRLS` for certificate revocation list data; administrators can manage these stores with the `vecs-cli` command-line tool. The `TRUSTED_ROOTS_CRLS` store must not be deleted or modified, as doing so can damage the certificate infrastructure. Control plane VMs in vSphere Supervisor environments rely on a Trusted Roots bundle that must be kept current with the latest vCenter root certificates.

At the hardware level, VCF supports Secure Boot and Trusted Platform Module (TPM) to help verify that hosts boot in known-good configurations. The vSphere Security Configuration Guide recommends enabling Secure Boot and TPM as part of VMware ESX host hardening. vCenter provides API interfaces for retrieving TPM event logs for configuring firmware trust with an attestation service or validating boot-time TPM measurements, and for listing configured TPM endorsement keys on a host. These capabilities support establishing a hardware-rooted chain of trust from firmware through the hypervisor.

Beyond core key and certificate management, several VCF components maintain their own trust infrastructure. VCF Operations orchestrator supports the import, export, and generation of package signing certificates, providing a mechanism for verifying the integrity and authenticity of software packages deployed within the environment. VCF Operations also maintains an SSL truststore with a password-protected keystore for managing certificates used in secure communications. VCF Automation can establish trust relationships with external servers by verifying and accepting certificate information for SSL communication, and it exchanges SAML service provider metadata with external identity providers to establish federated trust relationships. NSX components establish trust through certificate exchange between Local Manager and Global Manager instances during federation setup. VCF PowerCLI maintains its own Trusted Certificate Store where trusted certificates are saved and used when connecting to server systems, extending the trust chain to administrative automation tooling.

VCF provides the cryptographic infrastructure for establishing, storing, and verifying trust relationships through certificates and keys across all its major components. However, the organizational processes for provisioning supplier keys, verifying

supply chain integrity of cryptographic material, defining key protection policies, and physically protecting key storage remain the responsibility of the deploying organization.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 3.14

> Network data at rest: Protect sensitive data stored on network devices. Network config encryption: Encrypt stored configuration files.

**SCF Controls:** END-06.5

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides automated mechanisms to verify the integrity of the boot process for both physical hosts and virtual machines, spanning hardware-rooted trust, software specification validation, and post-boot health verification.

ESX hosts support UEFI Secure Boot, which uses cryptographic signature verification to validate each component loaded during the boot sequence. When Secure Boot is enabled, the host firmware checks the digital signature of the bootloader, and each subsequent stage verifies the next, creating a chain of trust from firmware through the hypervisor kernel. The vSphere Security Configuration Guide recommends enabling Secure Boot on all ESX hosts as part of standard hardening. ESX also supports an `execInstalledOnly` enforcement mode, built on top of UEFI Secure Boot, which restricts execution to software installed through the managed package process and prevents unauthorized binaries from running on the host.

VCF also supports Trusted Platform Module (TPM) integration on ESX hosts. TPM devices perform measured boot, recording cryptographic hashes of each boot component into platform configuration registers. VMware vCenter provides programmatic access to TPM device information through the trusted infrastructure API, including retrieval of TPM event logs that record boot-time measurements. These event logs can be used with an attestation service to validate that hosts booted with known-good firmware and software configurations. The Tpm interface from the trusted infrastructure package allows administrators to list TPM devices on a host and retrieve detailed information including manufacturer, model, and firmware version. vCenter also supports unsealing secrets bound to an endorsement key to verify reported measurements from the TPM event log, providing a second verification path for boot integrity. ESX host TPM modules must run the latest 2.0 firmware when in use, and must be disabled in the BIOS if not in use, keeping the hardware trust baseline consistent and verifiable.

Remote attestation capabilities allow vCenter to verify host boot integrity by comparing TPM measurements against expected values, providing an automated mechanism to detect tampering or unexpected changes in the boot process of managed hosts.

For virtual machines, VCF supports UEFI Secure Boot at the VM level, allowing guest operating systems to verify boot component integrity through the same cryptographic chain-of-trust mechanism used on physical hardware. VM boot firmware can be configured to EFI type using the vSphere Automation SDK `Boot.UpdateSpec()` method, and boot options including EFI firmware type and boot behavior settings are configurable through both the vSphere Client and the vSphere Automation API. VM-level Secure Boot is supported starting with Hardware Version 13. For Windows virtual machines, vSphere also supports Configurable Code Integrity under Virtualization-Based Security (VBS), which enforces that only trusted, signed code runs from the boot loader onwards within the guest.

At the software specification level, VCF's lifecycle management includes automated validation before any software is applied to hosts. The Drafts service offers a `validate_Task` method that checks whether a draft software specification is complete and identifies any conflicts between draft components or unresolved dependencies. Before saving a draft and turning it into the desired state for a cluster or standalone host, the specification should be validated to confirm it is complete and valid. This automated pre-deployment validation helps ensure that hosts receive verified, conflict-free software configurations, reducing the risk of boot failures or integrity issues caused by incompatible components. VCF also supports the creation of exported ISO images from desired software specifications for use in clean installs and bootstrapping, allowing administrators to provision systems from a known, validated software baseline.

For post-boot verification, VCF provides automated checking mechanisms across its management components. The Supportability and Serviceability tool (SoS) on VCF Operations includes a health check function that verifies system state after operations such as a vCenter instance restore, confirming that all components have started correctly and are operating as expected. VCF Operations Cloud Proxy implements automated first boot verification by checking for the presence of a

"Succeeded" file in /var/log/firstboot, providing a programmatic way to confirm that the boot process completed without error.

Auditors evaluating a VCF deployment for this control should verify that UEFI Secure Boot is enabled on all ESX hosts, that TPM 2.0 modules are active and running current firmware, that lifecycle management workflows use the software specification validation step before applying updates, that SoS health checks are run after restore or maintenance operations, and that boot verification logs are monitored for component deployments.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 3.14(1)

> Network data at rest: Protect sensitive data stored on network devices. Network config encryption: Encrypt stored configuration files.

**SCF Controls:** END-06.6

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides multiple automated mechanisms that protect the integrity of boot firmware across both physical VMware ESX hosts and virtual machines.

ESX hosts support UEFI Secure Boot, which validates the cryptographic signature of each component in the boot chain before allowing it to execute. When Secure Boot is enabled, the host firmware restricts the boot process to only authorized, signed code, protecting the hypervisor from unauthorized modification. The vSphere Security Configuration Guide recommends enabling Secure Boot on all ESX hosts as a standard hardening measure, alongside Trusted Platform Module (TPM) enablement. Secure Boot enforcement can be activated or deactivated through ESXCLI, and ESX also supports `execInstalledOnly` enforcement, a control built on top of UEFI Secure Boot that restricts execution to only installed binaries.

TPM 2.0 integration on ESX hosts provides measured boot capabilities. The TPM records cryptographic hashes of each boot stage into hardware-protected platform configuration registers, creating a tamper-evident chain of trust from firmware through the hypervisor. If any boot component is altered, the TPM measurements will differ from expected values. The TPM event log can be retrieved for configuring firmware trust with an attestation service or for validating boot time measurements. VMware vCenter provides programmatic access to TPM device information, including manufacturer and firmware version details, through the trusted infrastructure API. TPM endorsement keys used to establish hardware identity and trust are also managed through this API. An attestation service can compare measurements against known-good values to confirm that boot firmware and subsequent components have not been modified.

For virtual machines, VCF supports the Extensible Firmware Interface (EFI) as a virtual hardware feature, enabling UEFI Secure Boot for guest operating systems. Virtual machine firmware type can be configured to EFI through the vSphere Client or programmatically through the vSphere Automation SDK using the `Boot.UpdateSpec` API or the `BootTypes.UpdateSpec.Builder setType()` method. UEFI Secure Boot at the VM level protects guest operating system boot firmware using the same signature verification approach as physical hosts. For Windows virtual machines, vSphere supports Virtualization-based Security (VBS) with Configurable Code Integrity, which restricts execution to trusted code from the boot loader onward within the guest operating system. VCF also supports hardware memory encryption for confidential workloads through technologies like SEV-ES, which protects VM state from inspection or modification at the hardware level.

vSphere Lifecycle Manager provides automated firmware lifecycle management for ESX hosts. Firmware add-ons can be added, replaced, removed, upgraded, or downgraded through vSphere Lifecycle Manager images, allowing administrators to maintain firmware at vendor-validated versions across the host fleet. vSphere Lifecycle Manager images consist of a base image, vendor add-ons, components, and firmware add-ons that can be independently retrieved and applied to hosts. This automated approach helps keep host firmware aligned with tested baselines and reduces the window during which outdated firmware is in use.

VCF Operations provides monitoring for firmware integrity through management packs. The Management Pack for Lenovo Compute generates alerts for firmware unrecoverable boot device failure events detected by UEFI, as well as firmware update failed alerts when a firmware update does not complete normally and the target image is partially written. These automated alerts allow operations teams to detect and respond to firmware integrity issues.

VMware publishes security advisories for known firmware and software vulnerabilities affecting VCF components, with remediation guidance for each advisory.

Not applicable for this control: VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.



## Control NDR 5.2

> Zone boundary protection: Control flow between different security zones. Deny by default: Implement a "deny all, permit by exception" policy. Stateful inspection: Use stateful traffic filtering at boundaries. Fail close: Ensure boundary protection fails to a secure (closed) state.

**SCF Controls:** NET-03

**Aggregate Coverage:** Meets

### VCF (Contributes)

VCF provides networking infrastructure and monitoring capabilities that support boundary protection at both external network boundaries and key internal boundaries within the virtual environment.

At the external boundary, NSX Edge nodes serve as the communication gateway for inter-location and northbound traffic, establishing a defined perimeter through which external communications traverse. NSX Edge supports policy-based and route-based IPsec VPN services on Tier-0, Tier-0 VRF, and Tier-1 gateways, enabling secure connectivity between sites and to external networks. NAT rules can be configured at the VPC level to perform IP translation between internal and external networks, controlling which workloads are reachable from outside the environment. External IP configurations allow outside connectivity for virtual machines on private networks by performing address translation through 1:1 NAT, granting access to specific workloads rather than exposing entire network segments.

For internal boundary control, VCF provides several layers of network segmentation. NSX segments, VLANs, and Private VLANs (PVLANS) allow administrators to create logical network boundaries that separate traffic at Layer 2. VMware distributed virtual switches support Private VLAN and VLAN-based SPAN for virtual distributed port mirroring. ESX port groups function as communication boundaries and can be configured with security policies to segment traffic between different groups of virtual machines. vSphere Distributed Switch port groups also support a Traffic Filtering and Marking Policy, which can permit or stop traffic on a per-port-group or per-port basis; port blocking policies provide a complementary mechanism to selectively prevent individual distributed ports from sending or receiving data. vSphere Distributed Switches provide centralized management of these boundaries across all hosts in a workload domain, applying consistent VLAN configurations and traffic shaping rules.

VCF supports multiple distributed switch models that isolate traffic at the physical layer. The Storage Separation Distributed Switch model isolates storage traffic onto a separate vSphere Distributed Switch from management and virtual networking traffic. The Workload Separation Distributed Switch model isolates VM Management traffic onto a separate switch from ESX Management, vMotion, Storage, and Virtual Networking traffic. In secure environments, these separation models can provide physical air-gapped separation of network traffic.

NSX Virtual Private Clouds (VPCs) provide isolated network environments within a shared infrastructure. VPCs support on-demand logical networks that can be advertised as public subnets or exposed through networking services like External IPs or NAT, providing project-level boundary isolation with their own topology, IP address management, and permission models. VCF Automation maintains secure organizational boundaries that isolate tenants from one another, providing additional logical separation within the environment.

Network I/O Control (NIOC) determines the bandwidth that different network traffic types are given on a vSphere distributed switch. When NIOC is enabled, distributed switch traffic is divided into predefined network resource pools covering Fault Tolerance, iSCSI, vMotion, management, vSphere Replication, NFS, and virtual machine traffic. Bandwidth reservation and shares allocation enforce performance boundaries between traffic types under contention. Avi Load Balancer provides Layer 4 load balancing for services, distributing traffic across server pools with configurable load balancing algorithms and health monitoring.

VCF provides network monitoring through multiple components. NSX supports IPFIX profile configuration on segments to enable flow-based network monitoring, giving administrators visibility into traffic traversing the network. At the vSphere layer, NetFlow can be enabled on individual distributed port groups or ports through the vSphere Distributed Switch monitoring policy, providing IP packet flow data independent of NSX. The `pktcap-uw` command-line utility on ESX hosts supports packet

capture at specific points in the network stack, including at physical adapters and at the boundary between the distributed switch and physical adapters, enabling packet-level traffic inspection without additional tools. NSX Manager monitors control plane connections between transport nodes and manager nodes, providing awareness of the network infrastructure state.

VCF Operations extends monitoring capabilities with network metrics including inbound and outbound connection tracking (net|allInboundTotal and net|allOutboundTotal). The Management Pack for Network Devices provides alerts and notifications for key network events including throughput totals and packet counts, and monitors outgoing errors on network interfaces. VCF Operations can detect protected flows, identifying traffic that has a specific firewall rule associated with it beyond a default allow-all policy, which helps administrators understand which communications are governed by explicit security policies. Outbound flow limits serve as a protection mechanism, with TCP flows limited to 1,000 allowed and 50 denied outbound, and UDP flows limited to 500 allowed and 50 denied outbound. VCF Operations also supports firewall hardening that restricts network access to its own internal services, demonstrating boundary protection for management interfaces.

VCF Operations for Networks adds flow analysis and troubleshooting capabilities, including application discovery based on flows, network topology visualization through the Topology Graph Widget, and traffic flow visualization between applications, application tiers, shared services, and the Internet. VCF Operations for Networks provides intents for network design and verification, helping validate that the network configuration matches the intended design. Distributed virtual switches support health check operations for NIC teaming and VLAN/MTU verification, while vSAN network partition health checks detect VLAN misconfigurations and uplink losses. VCF Operations for Networks also provides visibility into vDefend distributed firewall rule activity, including identification of inactive rules that have had no matching flows.

VMware Kubernetes Service (VKS) clusters and the vSphere Supervisor control plane extend boundary protection to the workload consumption layer. Within VKS clusters, Kubernetes NetworkPolicy objects provide declarative packet-filtering rules at OSI Layer 3 and Layer 4, defining permitted ingress and egress traffic based on pod label selectors, namespace selectors, and CIDR blocks. By default, pods accept all traffic; NetworkPolicy objects restrict communication to only the flows required for each workload or namespace, establishing internal boundaries between application tiers. Antrea, the CNI plugin integrated with Supervisor in NSX VPC deployments, enforces these NetworkPolicy rules and exposes NetworkPolicyStats, giving administrators measurable visibility into how boundary controls are applied across cluster-internal traffic. At the cluster perimeter, distributed firewall isolation rules on cluster nodes block external networks from reaching VKS cluster pods by default; Supervisor control plane nodes can reach cluster pods, but traffic originating outside the Supervisor control plane is dropped. Connections from worker nodes and pods to the Kubernetes API server use the API server's secure port by default, and SSH tunnels restrict control plane-to-node communications from exposure outside the cluster network. Kubernetes Ingress resources backed by ingress controllers perform TLS termination and route external requests to backend services, establishing a defined and controlled entry point at each cluster's external-facing edge. Network segmentation and firewall rules are applied to secure inter-component communication within Supervisor Service deployments such as Argo CD.

For stateful firewall inspection, microsegmentation, IDS/IPS, and deep packet inspection at network boundaries, organizations can deploy VMware vDefend, which extends VCF's networking foundation with Layer 2 through Layer 7 firewall capabilities and advanced threat prevention. VCF's networking layer provides the segmentation, routing, and monitoring infrastructure on which boundary enforcement policies operate, but the active enforcement of traffic at those boundaries requires either vDefend or integration with external security tools. Organizational processes for defining boundary policies, reviewing traffic logs, managing firewall rule sets, and maintaining network diagrams are also required.

VMware Private AI Services (PAIS) contributes to boundary protection through AI-infrastructure-specific network isolation patterns for both connected and air-gapped deployments. The core boundary enforcement capabilities, including distributed firewall rules, IDS/IPS inspection, and traffic analysis, are provided by VCF and VMware vDefend.

PAIS setup is configured according to environment Internet connectivity, supporting either connected or disconnected air-gapped deployment models. In a disconnected air-gapped environment, files are downloaded from the Internet to a bastion host VM and then transferred to an admin host VM that has no Internet access, from which VI administrators interact with the environment. This pattern removes direct Internet exposure from the AI infrastructure components and constrains the path by which external content enters the trusted enclave.

In air-gapped deployments, PAIS configures components to pull container images from a local internal registry rather than from public sources. The NVIDIA GPU Operator deploys validator, operator, driver, manager, toolkit, and devicePlugin components as separate container images pulled from the local registry. PAIS model endpoints authenticate to private registries through harbor-pull-secret-readonly to control access to container images.

PAIS integrates with Harbor registries as a model storage backend, requiring authentication and certificate-based trust for model retrieval. The Model Gallery uses Harbor project access capabilities to restrict access to sensitive data used to train and tune models, providing an internal boundary between AI workload data stores and other platform consumers. PAIS uses a CA trust bundle for secure communication with external services and databases.

The NSX Edge cluster or Virtual Network Appliance (VNA) cluster provides two-tier north-south routing for the Supervisor instance and the workloads it runs, while the Workload Domain NSX Manager provides VPC networking services to private AI workloads. VPC networking for the VCF Automation organization is configured with a Public IP range, a transit gateway IP range, and a VPC private IP range to delineate external, transit, and internal address space for AI tenants.

Organizations should refer to the VCF and vDefend Coverages for the underlying boundary monitoring and enforcement infrastructure.

### **VMware vDefend (Meets)**

VMware vDefend provides monitoring and control of communications at both external and internal network boundaries within the virtual infrastructure through stateful firewall inspection, microsegmentation, and behavioral traffic analysis.

At the external boundary, vDefend enforces zone boundaries and provides stateful North-South traffic inspection at the network perimeter. Gateway Firewall policies apply to traffic entering and leaving edge gateways, and Gateway Firewall IDS/IPS events can be monitored on nodes, providing inspection-layer visibility at the external boundary.

At internal boundaries, vDefend enforces East-West traffic policies between virtual machines, containers, and workloads regardless of their physical location. Distributed Firewall rules operate at the vNIC level, creating per-workload enforcement points that function as internal boundary controls. Distributed Firewall manages East-West traffic control for both inter-VPC and intra-VPC communication within Organizations, and Distributed Firewall rules protect traffic between objects within a VPC, enforcing boundaries within project-scoped network environments. This microsegmentation approach allows organizations to define fine-grained internal boundaries beyond traditional VLAN-based segmentation. The policy-level Applied-To field defines a clear boundary around a set of VMs, such as an entire application, allowing targeted policies for specific zones, tenants, or applications without interfering with policies defined for other applications, tenants, or zones. Distributed Firewall policies can explicitly block unsecure protocols such as Telnet, FTP, and TFTP across the data center, supporting protocol-level boundary enforcement. Distributed Firewall also supports targeted deny rules scoped to infrastructure groups via the Applied-To field to protect critical services, facilitating phased Zero Trust adoption. Distributed Firewall governance implements least-privilege security by preventing overly permissive rules such as any-to-any-allow configurations. Distributed Firewall policies manage security for Bare Metal servers through traffic-control rules, extending boundary enforcement beyond virtual workloads. For federated environments, Distributed Firewall policies applied globally allow sources or destinations to be any object without span restrictions, supporting multi-site boundary enforcement.

Security Services Platform adds environment-level boundary enforcement through Protection Rules, which define how traffic flows between environment pairs such as production, development, and test. Protection Rule actions can be set to Drop or Reject to block all traffic between specified environment pairs, or Allow for more selective enforcement. Security Services Platform Environment Protection evaluates whether workloads are protected by environment-specific segmentation policies and measures coverage including the percentage of environment pairs in blocked mode. Application Protection in Strict Mode enforces application-level firewall rules on all application workloads except infrastructure, with scoring based on workload coverage, traffic coverage, and lock-down policies. Security Intelligence, a capability within Security Services Platform, defines private IP ranges as the boundary for internal workloads, treating traffic that crosses this boundary as external. The Secure External Communications project identifies workloads with inbound or outbound connections to external IP addresses to mitigate vulnerability to attacks. Security Intelligence recommends identifying all inter-environment flows and securing them with specific policies before enforcing strict block rules, supporting a methodical approach to boundary hardening.

Network Detection and Response (NDR) monitors traffic across boundaries and collects detection events representing security-relevant activity that has occurred in the network. NDR can be integrated with SIEMs to send security event logs about network infrastructure for centralized boundary monitoring. In sensitive or classified environments, NDR can be configured for air-gapped networks, executing all detection, correlation, and response activities entirely within closed networks using local storage and processing without requiring external connectivity. NDR remediations for events detected on the Gateway Firewall are applied on the Distributed Firewall, providing a unified enforcement response across both the internal and external boundary layers.

vDefend does not define which boundaries an organization should establish or set the policies governing boundary placement decisions. Those are organizational architecture and governance responsibilities. The underlying networking infrastructure of VCF, comprising NSX segments, VLANs, and edge gateways, provides the segmentation and routing fabric on which vDefend's boundary enforcement operates.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer provides boundary protection mechanisms at the application delivery layer for traffic it proxies. Network Security Policies in Avi evaluate incoming client traffic based on network-level parameters and determine whether a connection should be accepted, discarded, or rate-limited. These policies apply IP-based traffic filtering at the virtual service level; a deny rule stops further policy evaluation, preventing downstream HTTP Security, Request, or Response policies from executing for that traffic. Network security policies are configurable per virtual service and per Virtual IP address, giving administrators auditable and consistent access controls across the deployment.

Avi's Web Application Firewall (WAF) inspects application-layer traffic at the proxy boundary using positive and negative security models. Positive Security rules define what constitutes allowed application behavior and enforce restrictions on traffic that does not conform to that model. WAF Application Rules provide protection against known attack patterns, including rules targeting specific CVEs; rules are automatically updated through the Controller Cloud Services connection, and existing rules continue to protect applications when that connection is unavailable. WAF rule actions support pass and deny options for controlling traffic based on policy matches. Administrators can also enable the Application Rules service through the Avi Cloud Console, which delivers automatically updated CVE-targeted rules directly to Avi Controllers.

For DDoS mitigation, Avi's DDoS Policy detects and mitigates Layer 4 through Layer 7 network attacks through the Network Security policy framework. This includes protection against DNS Reflection Ingress attacks, which involve sending DNS queries with spoofed IP addresses to flood a victim with unsolicited DNS server responses.

In Kubernetes deployments, the Avi Kubernetes Operator (AKO) extends boundary controls to Kubernetes ingress traffic. The L4 Rule custom resource definition (CRD) in AKO allows administrators to express custom Security Policies applied to virtual service traffic, including configuration for DDoS attack detection and mitigation. The `'insecureEdgeTerminationPolicy'` setting in AKO controls handling of insecure traffic, with a redirect option that enforces secure traffic routing through HTTP Request policies applied per FQDN.

Avi Service Engines implement traffic isolation between the data plane and the management and control plane using a dedicated management vNIC bound to the default Linux network namespace, with health monitor traffic further isolated through Linux network namespace features in the kernel. Service Engine IP addresses egressing the Data Plane vNIC are routed through the external network and subjected to routing isolation and ACL/firewalling controls present in that external network. A dedicated management network is required for communication between Avi Controllers and Service Engines. In isolated network deployments, each Avi Controller supports a second management IP address or DNS-resolvable FQDN for Service Engines connecting from an isolated segment, configured via the `'isolated-network-management-address'` setting.

In VCF deployments using the Avi-for-VCF integration, the Management Network hosts the Avi Controller and provides connectivity to vCenter, ESX hosts, and Supervisor control plane nodes, maintaining a defined boundary between management and workload traffic.

Incoming HTTP(S) traffic can be replicated to logging and sideband servers at the protocol level via the Virtual Service Sideband Profile, enabling deep traffic inspection by external appliances such as web application firewalls and supporting compliance and auditing requirements.

Avi also supports a Firewall Sandwich Topology, where virtual services and firewall pool members are chained so that application traffic traverses configured firewall members on its path from the Internet to internal servers. In this topology, a sample of traffic is inspected by the firewall while remaining traffic considered trusted is directly routed to the internal network.

Boundary protection for segments and layers outside Avi's application delivery scope, including network-level perimeter controls, east-west enforcement between application tiers, and internal boundary protection across the broader VCF environment, requires the VCF networking infrastructure and VMware vDefend, which is a separate product.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 5.2(1)

> Zone boundary protection: Control flow between different security zones. Deny by default: Implement a "deny all, permit by exception" policy. Stateful inspection: Use stateful traffic filtering at boundaries. Fail close: Ensure boundary protection fails to a secure (closed) state.

**SCF Controls:** NET-04.1

**Aggregate Coverage:** Meets

### VCF (Contributes)

VCF supports deny-by-default and allow-by-exception network traffic patterns at multiple layers, including the ESX host firewall, vSphere Distributed Switch traffic filtering, and NSX routing layer. Full workload firewall deny-by-default enforcement at distributed and gateway firewall levels is provided by VMware vDefend, which is a separate product.

ESX hosts include a built-in firewall that controls incoming and outgoing connections for management services, with service groups including SSH and SNMP. Each firewall service allows access from all IP addresses by default; administrators can restrict access by specifying allowed IP addresses in CIDR notation (for example, 192.168.0.0/24) or as individual comma-separated addresses. The ``esxcli network firewall ruleset set --allowed-all`` command disables open access for a ruleset and enforces an explicit IP allowlist per service, implementing a deny-by-default posture at the management-service level on every ESX host.

vSphere Distributed Switches support Traffic Filtering and Marking policies at the distributed port and uplink port level. Network traffic rules can be configured with Allow or Drop actions to control traffic flowing through individual ports, and port-level override options allow policy exceptions to be applied at the individual port level without modifying the parent distributed port group policy. These rules operate at the virtual switch tier and provide a deny-by-default mechanism independent of firewall functionality.

At the routing layer, NSX Tier-0 Gateways support IP prefix lists that control route redistribution with permit and deny actions. The default action for a route in an IP prefix list is Deny, which implements a deny-by-default posture for route advertisement. Administrators must explicitly add a Permit entry for each desired route, with all others denied by default. This deny-by-default routing posture restricts which traffic flows can be advertised and reachable through the NSX infrastructure.

VCF Automation allows administrators to associate security groups with network profiles. When a network profile includes security groups, all machines provisioned with that profile inherit the associated network policies, providing a way to apply consistent deny/allow policies at provisioning time. The security group constructs integrate with vDefend firewall enforcement when vDefend is deployed, surfacing rule-based access controls through the automation layer.

VCF Operations for Networks provides a Security Planning capability through the Plan and Assess interface. This feature analyzes flows and displays traffic flow patterns and micro-segmentation data, giving administrators visibility into which traffic is currently permitted and helping identify gaps in deny-by-default coverage. Planning queries can analyze data center security by planning security policies for specified entities before they are applied.

At the workload consumption layer, VMware Kubernetes Service (VKS) clusters use Kubernetes NetworkPolicy resources as the primary mechanism for deny-by-default traffic control between pods. A NetworkPolicy selects pods by label selector and specifies which ingress and egress traffic is permitted; once at least one NetworkPolicy applies to a pod, all traffic not explicitly permitted is dropped for that pod. To establish a deny-by-default posture in a namespace, administrators create a NetworkPolicy that selects all pods but specifies empty ingress and egress rules, then layer explicit allow rules on top using pod label selectors, namespace selectors, destination ports, or IP block CIDR ranges with optional exclusions. VKS uses Antrea as its default Container Network Interface (CNI) plugin; Antrea implements NetworkPolicy enforcement using Open vSwitch, enabling per-flow filtering at the virtual switch layer. The AntreaPolicy feature gate enables Antrea-native policies, which support tiered policy ordering and cluster-scoped rules beyond what standard NetworkPolicy resources support.

Beginning with VKS 3.6, VKS clusters support configurable inbound firewall rules for cluster nodes through the ``firewall`` subvariable of the ClusterClass node variable. The ``firewall.inboundRules`` variable can be set on control plane nodes and as



variable overrides on each worker node `machineDeployment`; each rule accepts an optional source IP address or CIDR block and a target protocol specified as an IANA protocol number or alias string such as `tcp` or `udp`. This provides a declarative API for controlling node-level inbound access, complementing the pod-level NetworkPolicy layer.

Each VKS cluster is provisioned with a built-in distributed firewall rule for kube-dns, applied to the CoreDNS pod logical port at cluster creation, restricting CoreDNS access to its designated port. Antrea NetworkPolicyStats is enabled by default on VKS clusters, providing statistics on which network policies are active and what connection volumes they have matched. VKS Cluster Management Security Policies include a Policy enforcement setting whose default action is Deny, which fully enforces the policy and denies admission requests that contain violations; a Dry run mode is available for validation before enforcement is activated. The `DenyServiceExternalIPs` admission controller restricts use of `LoadBalancer` and `ExternalIPs` service types, limiting which services can advertise external endpoints.

Kubernetes itself enforces deny-by-default access control at the API server level: all parts of an API request must be explicitly permitted by at least one authorization mechanism in order to proceed. One scope boundary applies at the pod networking layer: Kubernetes NetworkPolicy resources are specified to restrict TCP, UDP, and SCTP traffic; for other protocols such as ARP and ICMP, enforcement behavior depends on the CNI plugin in use, and administrators should verify handling for those protocols in their specific deployment. A namespace with no NetworkPolicy applied allows all pod traffic by default; achieving deny-by-default behavior requires deliberate creation and deployment of default-deny NetworkPolicy objects in every namespace, which organizations must establish as an operational practice.

VMware vDefend extends these capabilities with full stateful firewall enforcement at both the gateway and workload tiers. vDefend applies security policies at tier-1 gateway service interfaces, controlling traffic as it enters and exits network segments. vDefend firewall rules provide granular control through configurable source groups, destination groups, services, and direction settings (inbound, outbound, or both). Each rule supports three actions: Allow permits matching traffic, Deny actively rejects it with a notification to the client, and Drop silently discards the packet. This rule structure enables administrators to build policies that block all traffic by default and then define specific exceptions for permitted flows.

Rule ordering within vDefend firewall policies is significant for implementing deny-by-default patterns. Rules that permit specific exceptions should be placed before rules that drop traffic, so that the allowed exceptions are evaluated first and all remaining unmatched traffic falls through to the default deny rule. vDefend preserves the order in which policy rules are added, and precedence is determined at both the policy level and the individual rule level.

vDefend Layer 7 access profiles support URL category filtering with configurable Allow, Reject, or Reject with Response actions for matched traffic. Administrators can define FQDN-based firewall rules to allow or reject traffic going to specific domains, using both pre-defined and custom FQDN lists. URL categories support wildcard pattern matching for domain filtering, including single-level and multi-level wildcards and anchored patterns to match specific URL structures.

vDefend includes a Firewall Exclusion List that allows administrators to designate up to 100 groups that should bypass firewall rule evaluation. This should be used sparingly, as excluded objects are not subject to the deny-by-default policy.

vDefend extends deny-by-default enforcement to east-west traffic between virtual machines. vDefend DFW is a software-defined Layer 2-7 firewall that provides stateful firewalling with distributed firewall policies supporting Allow, Drop, and Reject actions, direction controls, and context-aware rule parameters including sources, destinations, services, and context profiles. vDefend functionality can be configured on virtual distributed switches in data centers and workloads where segmentation, visibility, or advanced security capabilities are desired.

Organizational governance of firewall rule policies, change management for rule updates, and periodic review processes to verify that deny-by-default posture remains intact across the environment must be established outside the platform.

### VMware vDefend (Meets)

VMware vDefend (VMware vDefend) implements a deny-by-default, allow-by-exception traffic policy for both East-West and North-South traffic in a VCF environment.

The vDefend distributed firewall (DFW) enforces stateful Layer 2-7 security policies at the virtual network interface level of each workload. Each firewall rule is configured with one of three actions: Allow, Drop, or Reject. DFW automatically generates



a Default Drop rule that denies all traffic from any source to any destination for any service. This catch-all rule sits at the bottom of the rule table and enforces policy for packets not matching any other rule. VMware recommends changing the DFW default rule to a block action and enforcing access control through a positive control model where only traffic explicitly defined in firewall rules is permitted. Specific rules should be placed above more general rules in the rule table so that more targeted permits take precedence. DFW supports Context Profiles for Layer 7-aware rule matching, enabling allow-by-exception rules that specify application-layer context in addition to source, destination, and port.

The Application Connectivity Strategy in DFW provides structured, API-driven approaches to configuring the default traffic posture. The Allow Intra, Deny Other Traffic strategy uses the ALLOW\_INTRA and ALLOWLIST APIs to generate an intra-group Allow rule paired with a default ANY/ANY/DROP rule, isolating the group from all other traffic. The Allow List connectivity preference configures a Drop action as the default, denying all traffic not explicitly permitted. For traffic not covered by named strategies, DFW supports custom policies that provide granular control of lateral East-West traffic.

FQDN filtering rules in DFW support Allow, Drop, or Reject actions, extending the deny-by-exception model to domain-name-based traffic control. The Identity Firewall enforces the allow-by-exception discipline for user-identity-based access: each Identity Firewall rule that permits traffic from a user group to a destination must be paired with a corresponding DFW or Gateway Firewall rule applied to the destination group. Identity Firewall rules also support a Reject action, allowing administrators to deny traffic matching specified user groups, services, and destinations.

The vDefend gateway firewall provides deny-by-default enforcement for North-South traffic at the NSX edge. Gateway Firewall rules support Allow, Drop, and Reject actions. In federated deployments, DFW and Gateway Firewall rules can be created from the Global Manager with global, regional, or local spans and are synchronized to Local Managers. The default gateway firewall rule is editable from the Global Manager and can be configured as allow-all or deny-all, giving administrators control over the baseline North-South posture. The L7 Access Profile extends allow-by-exception to application-layer traffic, with rule actions of Allow, Reject, and Reject with Response.

The Security Services Platform (SSP) and Security Intelligence support the transition to and maintenance of a deny-by-default posture. After establishing exception policies for an environment pair, administrators can change the protection rule action to Drop or Reject to block new, unknown traffic. Policy recommendations can be scoped with a Source Exclusions List, allowing fine-tuned control over which traffic sources are included in generated allowlists. The Security Overview Dashboard provides views of DFW policies, including policy actions (allow, drop, and reject) and total rule counts, giving administrators visibility into the active enforcement posture. Security Policies in vDefend DFW can be reviewed and modified through the Policies and Rules field.

The Firewall Rule Analysis Report identifies potential security misconfigurations in the distributed firewall configuration, helping administrators find shadowed or overly permissive rules that could undermine the deny-by-default posture.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides deny-by-default traffic controls at the application delivery layer through Network Security Policies, HTTP Policy Sets, and Web Application Firewall (WAF) policies in Enforcement mode.

Avi's Network Security Policy is evaluated for every client connection to a virtual service. The policy determines whether a connection is accepted, discarded, or rate-limited based on network-level parameters. Multiple ordered rules can be configured within each Network Security Policy, with explicit Allow and Deny actions available. When a rule with a Deny action matches a connection, further policy evaluation stops, so downstream HTTP Security, Request, or Response policies do not execute for that traffic. Deny rules support a log parameter that, when set to True, records all denied traffic. ControlScripts can dynamically create Network Security Policy rules that deny traffic from specific client IP addresses identified during DDoS events, using IS\_IN match criteria against a configured list of source addresses. Network Security Policy can also be configured to deny connection attempts from addresses identified as malicious through IP Reputation data, dropping TCP SYN packets before connections are established. Both Network Security and HTTP Security policies support a Blocklist mechanism, where a policy rule references a configured IP group named Blocklist to discard clients matching that group.

The HTTP Policy Set provides an additional allow-or-deny mechanism at the application layer. Match conditions classify traffic and apply explicit allow or deny actions; denied traffic receives a TCP RST unless the system is under a volumetric denial-of-service attack, in which case the connection is silently discarded.

When configured in Enforcement mode, Avi's WAF applies a positive security model to application-layer traffic. WAF rule actions support pass (allow) and deny (block) options; requests that satisfy configured rules are permitted, and those that do not are blocked by default. Administrators can author custom WAF rules using SecRule syntax with chain operators, enabling multi-condition enforcement such as allowlisting specific source IP ranges for specific URI paths. WAF Exceptions allow specific traffic to be excluded from rule enforcement based on Client IP, Path, or Match Element filter combinations. WAF Allowlist rules support a BYPASS action for trusted IP address groups, including IP Group-based allowlisting, providing an allow-by-exception mechanism alongside the default-deny posture.

Avi's traffic controls apply only to application traffic passing through its virtual services and Service Engines. Deny-by-default policy enforcement for general east-west and north-south network traffic across the VCF platform is a function of VMware vDefend, which is a separate product. An organization implementing deny-by-default network traffic controls in a VCF environment will typically use both Avi and vDefend together, with Avi addressing application-layer traffic at the load balancer boundary.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 5.2(2)

> Zone boundary protection: Control flow between different security zones. Deny by default: Implement a "deny all, permit by exception" policy. Stateful inspection: Use stateful traffic filtering at boundaries. Fail close: Ensure boundary protection fails to a secure (closed) state.

**SCF Controls:** NET-04.1

**Aggregate Coverage:** Meets

### VCF (Contributes)

VCF supports deny-by-default and allow-by-exception network traffic patterns at multiple layers, including the ESX host firewall, vSphere Distributed Switch traffic filtering, and NSX routing layer. Full workload firewall deny-by-default enforcement at distributed and gateway firewall levels is provided by VMware vDefend, which is a separate product.

ESX hosts include a built-in firewall that controls incoming and outgoing connections for management services, with service groups including SSH and SNMP. Each firewall service allows access from all IP addresses by default; administrators can restrict access by specifying allowed IP addresses in CIDR notation (for example, 192.168.0.0/24) or as individual comma-separated addresses. The ``esxcli network firewall ruleset set --allowed-all`` command disables open access for a ruleset and enforces an explicit IP allowlist per service, implementing a deny-by-default posture at the management-service level on every ESX host.

vSphere Distributed Switches support Traffic Filtering and Marking policies at the distributed port and uplink port level. Network traffic rules can be configured with Allow or Drop actions to control traffic flowing through individual ports, and port-level override options allow policy exceptions to be applied at the individual port level without modifying the parent distributed port group policy. These rules operate at the virtual switch tier and provide a deny-by-default mechanism independent of firewall functionality.

At the routing layer, NSX Tier-0 Gateways support IP prefix lists that control route redistribution with permit and deny actions. The default action for a route in an IP prefix list is Deny, which implements a deny-by-default posture for route advertisement. Administrators must explicitly add a Permit entry for each desired route, with all others denied by default. This deny-by-default routing posture restricts which traffic flows can be advertised and reachable through the NSX infrastructure.

VCF Automation allows administrators to associate security groups with network profiles. When a network profile includes security groups, all machines provisioned with that profile inherit the associated network policies, providing a way to apply consistent deny/allow policies at provisioning time. The security group constructs integrate with vDefend firewall enforcement when vDefend is deployed, surfacing rule-based access controls through the automation layer.

VCF Operations for Networks provides a Security Planning capability through the Plan and Assess interface. This feature analyzes flows and displays traffic flow patterns and micro-segmentation data, giving administrators visibility into which traffic is currently permitted and helping identify gaps in deny-by-default coverage. Planning queries can analyze data center security by planning security policies for specified entities before they are applied.

At the workload consumption layer, VMware Kubernetes Service (VKS) clusters use Kubernetes NetworkPolicy resources as the primary mechanism for deny-by-default traffic control between pods. A NetworkPolicy selects pods by label selector and specifies which ingress and egress traffic is permitted; once at least one NetworkPolicy applies to a pod, all traffic not explicitly permitted is dropped for that pod. To establish a deny-by-default posture in a namespace, administrators create a NetworkPolicy that selects all pods but specifies empty ingress and egress rules, then layer explicit allow rules on top using pod label selectors, namespace selectors, destination ports, or IP block CIDR ranges with optional exclusions. VKS uses Antrea as its default Container Network Interface (CNI) plugin; Antrea implements NetworkPolicy enforcement using Open vSwitch, enabling per-flow filtering at the virtual switch layer. The AntreaPolicy feature gate enables Antrea-native policies, which support tiered policy ordering and cluster-scoped rules beyond what standard NetworkPolicy resources support.

Beginning with VKS 3.6, VKS clusters support configurable inbound firewall rules for cluster nodes through the ``firewall`` subvariable of the ClusterClass node variable. The ``firewall.inboundRules`` variable can be set on control plane nodes and as

variable overrides on each worker node `machineDeployment`; each rule accepts an optional source IP address or CIDR block and a target protocol specified as an IANA protocol number or alias string such as `tcp` or `udp`. This provides a declarative API for controlling node-level inbound access, complementing the pod-level NetworkPolicy layer.

Each VKS cluster is provisioned with a built-in distributed firewall rule for kube-dns, applied to the CoreDNS pod logical port at cluster creation, restricting CoreDNS access to its designated port. Antrea NetworkPolicyStats is enabled by default on VKS clusters, providing statistics on which network policies are active and what connection volumes they have matched. VKS Cluster Management Security Policies include a Policy enforcement setting whose default action is Deny, which fully enforces the policy and denies admission requests that contain violations; a Dry run mode is available for validation before enforcement is activated. The `DenyServiceExternalIPs` admission controller restricts use of `LoadBalancer` and `ExternalIPs` service types, limiting which services can advertise external endpoints.

Kubernetes itself enforces deny-by-default access control at the API server level: all parts of an API request must be explicitly permitted by at least one authorization mechanism in order to proceed. One scope boundary applies at the pod networking layer: Kubernetes NetworkPolicy resources are specified to restrict TCP, UDP, and SCTP traffic; for other protocols such as ARP and ICMP, enforcement behavior depends on the CNI plugin in use, and administrators should verify handling for those protocols in their specific deployment. A namespace with no NetworkPolicy applied allows all pod traffic by default; achieving deny-by-default behavior requires deliberate creation and deployment of default-deny NetworkPolicy objects in every namespace, which organizations must establish as an operational practice.

VMware vDefend extends these capabilities with full stateful firewall enforcement at both the gateway and workload tiers. vDefend applies security policies at tier-1 gateway service interfaces, controlling traffic as it enters and exits network segments. vDefend firewall rules provide granular control through configurable source groups, destination groups, services, and direction settings (inbound, outbound, or both). Each rule supports three actions: Allow permits matching traffic, Deny actively rejects it with a notification to the client, and Drop silently discards the packet. This rule structure enables administrators to build policies that block all traffic by default and then define specific exceptions for permitted flows.

Rule ordering within vDefend firewall policies is significant for implementing deny-by-default patterns. Rules that permit specific exceptions should be placed before rules that drop traffic, so that the allowed exceptions are evaluated first and all remaining unmatched traffic falls through to the default deny rule. vDefend preserves the order in which policy rules are added, and precedence is determined at both the policy level and the individual rule level.

vDefend Layer 7 access profiles support URL category filtering with configurable Allow, Reject, or Reject with Response actions for matched traffic. Administrators can define FQDN-based firewall rules to allow or reject traffic going to specific domains, using both pre-defined and custom FQDN lists. URL categories support wildcard pattern matching for domain filtering, including single-level and multi-level wildcards and anchored patterns to match specific URL structures.

vDefend includes a Firewall Exclusion List that allows administrators to designate up to 100 groups that should bypass firewall rule evaluation. This should be used sparingly, as excluded objects are not subject to the deny-by-default policy.

vDefend extends deny-by-default enforcement to east-west traffic between virtual machines. vDefend DFW is a software-defined Layer 2-7 firewall that provides stateful firewalling with distributed firewall policies supporting Allow, Drop, and Reject actions, direction controls, and context-aware rule parameters including sources, destinations, services, and context profiles. vDefend functionality can be configured on virtual distributed switches in data centers and workloads where segmentation, visibility, or advanced security capabilities are desired.

Organizational governance of firewall rule policies, change management for rule updates, and periodic review processes to verify that deny-by-default posture remains intact across the environment must be established outside the platform.

### VMware vDefend (Meets)

VMware vDefend (VMware vDefend) implements a deny-by-default, allow-by-exception traffic policy for both East-West and North-South traffic in a VCF environment.

The vDefend distributed firewall (DFW) enforces stateful Layer 2-7 security policies at the virtual network interface level of each workload. Each firewall rule is configured with one of three actions: Allow, Drop, or Reject. DFW automatically generates

a Default Drop rule that denies all traffic from any source to any destination for any service. This catch-all rule sits at the bottom of the rule table and enforces policy for packets not matching any other rule. VMware recommends changing the DFW default rule to a block action and enforcing access control through a positive control model where only traffic explicitly defined in firewall rules is permitted. Specific rules should be placed above more general rules in the rule table so that more targeted permits take precedence. DFW supports Context Profiles for Layer 7-aware rule matching, enabling allow-by-exception rules that specify application-layer context in addition to source, destination, and port.

The Application Connectivity Strategy in DFW provides structured, API-driven approaches to configuring the default traffic posture. The Allow Intra, Deny Other Traffic strategy uses the ALLOW\_INTRA and ALLOWLIST APIs to generate an intra-group Allow rule paired with a default ANY/ANY/DROP rule, isolating the group from all other traffic. The Allow List connectivity preference configures a Drop action as the default, denying all traffic not explicitly permitted. For traffic not covered by named strategies, DFW supports custom policies that provide granular control of lateral East-West traffic.

FQDN filtering rules in DFW support Allow, Drop, or Reject actions, extending the deny-by-exception model to domain-name-based traffic control. The Identity Firewall enforces the allow-by-exception discipline for user-identity-based access: each Identity Firewall rule that permits traffic from a user group to a destination must be paired with a corresponding DFW or Gateway Firewall rule applied to the destination group. Identity Firewall rules also support a Reject action, allowing administrators to deny traffic matching specified user groups, services, and destinations.

The vDefend gateway firewall provides deny-by-default enforcement for North-South traffic at the NSX edge. Gateway Firewall rules support Allow, Drop, and Reject actions. In federated deployments, DFW and Gateway Firewall rules can be created from the Global Manager with global, regional, or local spans and are synchronized to Local Managers. The default gateway firewall rule is editable from the Global Manager and can be configured as allow-all or deny-all, giving administrators control over the baseline North-South posture. The L7 Access Profile extends allow-by-exception to application-layer traffic, with rule actions of Allow, Reject, and Reject with Response.

The Security Services Platform (SSP) and Security Intelligence support the transition to and maintenance of a deny-by-default posture. After establishing exception policies for an environment pair, administrators can change the protection rule action to Drop or Reject to block new, unknown traffic. Policy recommendations can be scoped with a Source Exclusions List, allowing fine-tuned control over which traffic sources are included in generated allowlists. The Security Overview Dashboard provides views of DFW policies, including policy actions (allow, drop, and reject) and total rule counts, giving administrators visibility into the active enforcement posture. Security Policies in vDefend DFW can be reviewed and modified through the Policies and Rules field.

The Firewall Rule Analysis Report identifies potential security misconfigurations in the distributed firewall configuration, helping administrators find shadowed or overly permissive rules that could undermine the deny-by-default posture.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides deny-by-default traffic controls at the application delivery layer through Network Security Policies, HTTP Policy Sets, and Web Application Firewall (WAF) policies in Enforcement mode.

Avi's Network Security Policy is evaluated for every client connection to a virtual service. The policy determines whether a connection is accepted, discarded, or rate-limited based on network-level parameters. Multiple ordered rules can be configured within each Network Security Policy, with explicit Allow and Deny actions available. When a rule with a Deny action matches a connection, further policy evaluation stops, so downstream HTTP Security, Request, or Response policies do not execute for that traffic. Deny rules support a log parameter that, when set to True, records all denied traffic. ControlScripts can dynamically create Network Security Policy rules that deny traffic from specific client IP addresses identified during DDoS events, using IS\_IN match criteria against a configured list of source addresses. Network Security Policy can also be configured to deny connection attempts from addresses identified as malicious through IP Reputation data, dropping TCP SYN packets before connections are established. Both Network Security and HTTP Security policies support a Blocklist mechanism, where a policy rule references a configured IP group named Blocklist to discard clients matching that group.

The HTTP Policy Set provides an additional allow-or-deny mechanism at the application layer. Match conditions classify traffic and apply explicit allow or deny actions; denied traffic receives a TCP RST unless the system is under a volumetric denial-of-service attack, in which case the connection is silently discarded.

When configured in Enforcement mode, Avi's WAF applies a positive security model to application-layer traffic. WAF rule actions support pass (allow) and deny (block) options; requests that satisfy configured rules are permitted, and those that do not are blocked by default. Administrators can author custom WAF rules using SecRule syntax with chain operators, enabling multi-condition enforcement such as allowlisting specific source IP ranges for specific URI paths. WAF Exceptions allow specific traffic to be excluded from rule enforcement based on Client IP, Path, or Match Element filter combinations. WAF Allowlist rules support a BYPASS action for trusted IP address groups, including IP Group-based allowlisting, providing an allow-by-exception mechanism alongside the default-deny posture.

Avi's traffic controls apply only to application traffic passing through its virtual services and Service Engines. Deny-by-default policy enforcement for general east-west and north-south network traffic across the VCF platform is a function of VMware vDefend, which is a separate product. An organization implementing deny-by-default network traffic controls in a VCF environment will typically use both Avi and vDefend together, with Avi addressing application-layer traffic at the load balancer boundary.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control NDR 5.2(3)

> Zone boundary protection: Control flow between different security zones. Deny by default: Implement a "deny all, permit by exception" policy. Stateful inspection: Use stateful traffic filtering at boundaries. Fail close: Ensure boundary protection fails to a secure (closed) state.

**SCF Controls:** SEA-07.2

**Aggregate Coverage:** Meets

### VCF (Meets)

VCF provides multiple fail-secure mechanisms across its component stack that enable systems to transition to a known, predictable state during failures while preserving data integrity and system state information.

At the compute layer, vSphere Fault Tolerance (FT) provides continuous availability for protected virtual machines by maintaining a synchronized secondary VM running in lockstep on a separate host. If the primary host fails, the secondary VM takes over immediately with no data loss and no downtime, maintaining the application's in-memory state and active connections. This is a direct fail-to-known-state capability where the system transitions to a pre-established secondary instance. When Fault Tolerance encryption mode is set to Required, the platform restricts secondary VM placement to only those hosts that support FT encryption, so that failover can occur only on hosts that maintain the encryption posture of the protected VM.

vSphere High Availability (HA) automatically restarts VMs on surviving hosts following a physical host failure. HA uses network heartbeat monitoring to detect host failures, with a 15-second heartbeat timeout triggering failure detection. Admission control policies reserve cluster capacity so that sufficient resources are available for VM restart even after a host failure. For stretched cluster deployments, HA admission control can reserve 50% of the cluster's aggregate CPU and memory resources to allow all VMs on one site to restart on the other site during a site failure. NSX Manager nodes are similarly protected through vSphere HA with high restart priority, so that management plane components recover quickly after a host failure. For hosts using vSphere Distributed Services Engine with data processing units (DPUs), lifecycle operations are isolated within each DPU's boundaries, so that errors on one DPU do not affect the state of another; the overall operation result is still marked as failed to preserve visibility into the event.

At the storage layer, vSAN fault domains distribute data objects across independent failure boundaries based on storage policy. Each object is stored across at least two nodes (primary and replica) in separate fault domains, providing redundancy against localized failures. During a component failure, write operations for the failed component are redirected to a durability component, so the system continues operating in a degraded but data-safe state. When vSAN detects a permanent component failure, it begins rebuilding affected components immediately if adequate cluster resources exist. For transient failures where a component enters an absent state, vSAN waits 60 minutes before initiating a rebuild, preserving the opportunity for the original component to return. These behaviors return storage objects to their policy-defined state after failures. For stretched cluster deployments, the vSAN site disaster tolerance setting on the vSAN configuration page defines the data redundancy method used to handle a site failure, allowing administrators to configure the system's fail-secure behavior for site-level disruptions.

VCF Operations clusters are designed with fault domain awareness. Each object within the VCF Operations data store is replicated across fault domains. The cluster can tolerate the loss of one fault domain or individual nodes without data loss, continuing to operate in a degraded state until the issue is resolved. This preserves all monitoring and analytics state information through a failure event. VCF Operations recommends configuring host failure settings to restart VMs as part of vSphere HA setup, applying the same fail-secure approach to monitoring infrastructure.

At the infrastructure orchestration level, Cluster Availability associates each node with a fault domain, enabling the platform to track which components are affected by a failure and to coordinate the transition to a degraded-but-functional state. vSphere Supervisor failure domains allow each zone to be treated as an independent failure domain, extending this model to Kubernetes workloads. VMware Kubernetes Service (VKS) clusters require at least one control plane instance per failure zone to provide fault tolerance, and each VKS failure domain maps to a vSphere Zone associated with a single vSphere cluster. The



Cluster API enables sets of worker node machines to be defined across multiple failure domains with rules to automatically heal the cluster in the case of whole-zone service disruption.

At the Kubernetes workload layer, the admission control framework enforces fail-closed policy behavior for VKS clusters running on vSphere Supervisor. Both `MutatingAdmissionPolicy` and `ValidatingAdmissionPolicy` resources support a `failurePolicy` field that controls how the API server responds when a policy cannot be evaluated due to CEL expression parse errors, type check errors, runtime errors, or misconfigured policy definitions. The default value is `Fail`, causing the API server to deny the request rather than allow it through when evaluation fails. When a `ValidatingAdmissionPolicyBinding` is configured with `failurePolicy` set to `Fail`, the `validationActions` field defines how failures are enforced; setting `parameterNotFoundAction` to `Deny` causes the API server to reject requests when a referenced policy parameter is absent. Pod Security Admission applies a fail-closed model at the pod level: if any container fails to meet the `securityContext` requirements of the active Pod Security Standard profile (Baseline, Restricted, or Privileged), the entire pod is rejected, and partial enforcement is not permitted. For workload scheduling across failure boundaries, VKS validates that each `MachineDeployment` includes a `failureDomain` specification when multiple vSphere Zones are configured and denies requests that omit this field.

For batch workloads, the Kubernetes Job resource supports a Pod Failure Policy that uses custom conditions on pod status to distinguish transient failures from permanent misconfigurations. Transient failures can trigger retry behavior while permanent misconfigurations cause the job to terminate in a defined state rather than looping indefinitely. VKS cluster management provides structured visibility into failure states: a VKS cluster reports `Unknown` status when one or more control plane nodes or core system components are in an undetermined state, and the VKS Controller reports phase and component readiness conditions with `Info`, `Warning`, and `Error` severity classifications. VKS Cluster Management Policy Insights include Sync signals that indicate when policy definitions fail to propagate from the management plane to the targeted VKS cluster estate, making policy divergence visible rather than silent. For post-failure state preservation and analysis, VKS Linux nodes maintain system state information including cloud-init logs, network socket status, kernel parameters, and process information, all of which can be collected via support bundles.

In the networking layer, NSX maintains a desired state model where the platform continuously tracks the difference between the intended configuration (desired state) and the actual configuration (realized state). When an error occurs and the realized state diverges from the desired state, this divergence is surfaced so that administrators can identify and remediate the inconsistency. This model supports fail-secure operations by making configuration drift visible rather than silently allowing degraded state to persist.

For broader disruption scenarios, Application Disaster Recovery enables failover to a secondary site to maintain operations during major disruptions, providing a defined recovery target and a known-state transition path for site-level failures.

VCF provides fault detection and alerting capabilities that support fail-secure operations. VCF Operations correlates events published by monitored systems and delivers them as faults, signifying events that affect the availability of objects in the environment. Administrators can define custom fault symptoms based on these events and incorporate them into alert definitions for automated notification. At the host level, VMware ESX generates problem observations when storage devices enter a permanent error state, providing early warning of hardware degradation that could affect system availability.

VMware ESX hosts support host encryption safe mode, which means the host can encrypt virtual machines and is designed to keep encryption keys protected during failure scenarios. Once host encryption mode is enabled, it cannot be disabled easily, which limits accidental exposure of encrypted data during host recovery operations.

Authentication subsystems in VCF implement authentication failure handling, so that repeated authentication failures result in account lockout rather than continued access attempts. The VCF SSO lockout policy is the specific mechanism that governs this behavior: administrators can configure the maximum number of failed login attempts and the time interval between failures before lockout is applied. This is a fail-secure pattern at the security boundary, where the system transitions to a more restrictive state when anomalous behavior is detected.

vSphere Lifecycle Manager supports recovery from configuration drift by detecting when a host's current state differs from its desired specification. Administrators can check the compliance of a cluster or host against the desired state and remediate it

back to the defined configuration, returning the host to its known-good baseline after localized failures or configuration changes.

#### **VMware vDefend (Contributes)**

VMware vDefend (vDefend) contributes to fail-secure system behavior through mechanisms in the Security Services Platform and the IDS/IPS subsystem.

The Security Services Platform (SSP) provides backup and restore capabilities that preserve platform configuration and security policy state. Backup and restore operations are gated on platform health; when the platform is not in a healthy state, these operations are prevented rather than allowed to proceed against a degraded system. When a scheduled backup fails, SSP triggers a Scheduled Backup Error alarm to alert administrators. The platform also provides continuous monitoring of deployment status through an alarms interface that surfaces errors requiring attention. If internal services are restarted during a backup or restore, the operation enters a failed state after 30 minutes, establishing a defined failure boundary rather than an indefinite hang. SSP deployments should use vSphere High Availability to support host-level failure recovery; deploying on vSAN stretched configurations provides additional resilience against multiple simultaneous host failures within a site.

The vDefend distributed firewall follows a Zero Trust staged deployment model that prioritizes securing common infrastructure services, establishing controlled east-west traffic postures as part of a progressive fail-secure architecture.

For IDS/IPS policy objects, the policy realization state must reach `REALIZED` with a `runtime_status` of `SUCCESS` to confirm that the policy has reached its intended known state on the enforcement plane. When high volumes of IDS/IPS events and PCAP data create out-of-memory conditions that could destabilize the system, vDefend applies an automated protective mechanism that selectively drops lower-priority events and PCAP data to maintain system stability, prioritizing operational continuity over complete event capture under load.

#### **VCF Protection and Recovery (Contributes)**

VCF Protection and Recovery (PNR) is designed to transition to known, deterministic states when unexpected conditions arise during disaster recovery operations. Recovery plan state is determined by the states of the protection groups within the plan, and the plan state in turn determines which operations are allowed, so the system progresses through defined states rather than entering undefined conditions. When a recovery is canceled or datastore errors occur, the plan enters an Incomplete recovery state and requires administrators to either resolve the errors and rerun the recovery or remove protection for the affected virtual machines. Reprotect operations have a corresponding state machine with a Ready state on success and an Incomplete Reprotect state when failures occur during reverse replication or placeholder virtual machine creation.

PNR handles split-brain scenarios deterministically. The Protection and Recovery dashboards in VCF Operations generate critical warnings when a plan was running while two sites were disconnected, when a recovery plan is in an error state, or when reprotect fails for a plan, alerting administrators that the condition must be investigated before further operations proceed. During disaster recovery, if the protected site goes offline or errors occur on it, the failure is reported but the recovery operation continues to the recovery site without aborting, allowing recovery to proceed even when the protected site is partially or fully unreachable.

System state information is preserved through several mechanisms in 9.1. The Protection and Recovery Global Console alerts users to expired snapshot schedules, replication errors such as missing virtual machines, connectivity issues with protected or destination sites, and site misconfigurations that require investigation. The Protection and Recovery dashboards in VCF Operations expose the current phase of the disaster recovery lifecycle for each plan, including whether a plan is ready, a test is in progress, or incomplete recovery or cleanup operations are pending, and which protection groups are in which state. When errors are resolved and recovery is rerun, operations that succeeded previously are skipped and successfully recovered virtual machines continue running, supporting partial-success failure recovery without restarting the entire plan.

#### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides application-delivery-specific fail-secure behavior through defined failure states for its data-plane components. When a Service Engine host fails, Service Engines automatically transition to one of two known operational states: `OPER_PARTITIONED` when virtual services on that host remain operational, and `OPER_DOWN` when no

operational virtual services exist. These discrete, named states give operators and the platform a deterministic view of failure conditions within the application delivery layer, supporting the organization's ability to define and respond to known failure outcomes. When a Service Engine is unable to register with an unavailable Controller, it comes up stateless and ignores data-plane heartbeats from other Service Engines, providing bounded behavior in a degraded-Controller scenario.

The Avi Controller cluster documents failure scenarios and recovery actions for deployments across two availability zones. Fault domains are mapped to Availability Zones, and within each Service Engine Group operators can specify two or three Availability Zones across which Service Engines and virtual services are placed to improve fault tolerance, so that a zone failure produces predictable, bounded degradation. When a virtual service enters a failed state due to invalid configuration or resource constraints, the failed state reflects the state of the virtual service at a specific moment in time; operators can disable the virtual service in that state and re-enable it after running corrective actions.

The ICAP Profile provides configurable failure action handling for ICAP server errors, with two explicit options: Fail Closed, which sends an HTTP 503 to the client on error, and Fail Open, which allows traffic to pass. This mechanism gives operators direct control over the known state Avi enters when a content inspection dependency is unavailable.

System state information is preserved at failure through the inventoryfaultconfig mechanism. The Avi Controller tracks and generates alerts across independently configurable fault categories, including license faults, migration faults, backup scheduler faults, SSL profile faults, deprecated API version faults, and Service Engine faults. The Avi Controller and Service Engine log audit record failures to Events, Syslog, and Splunk, covering initiation and termination of trusted channels, failure of trusted channel functions, and unsuccessful certificate validation attempts, including the ObjectDoesNotExist condition when an SSL certificate does not exist during trusted channel establishment.

Infrastructure-level fail-secure capabilities for compute, storage, and network are provided by VCF rather than Avi (see VCF Coverage). The organization must also define the specific known failure states, recovery procedures, and failure scope that this control requires.

Not applicable for this control: VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of this product.

## Control NDR 5.3

> Network communication restriction: Block general purpose person-to-person traffic.

**SCF Controls:** NET-04.1

**Aggregate Coverage:** Meets

### VCF (Contributes)

VCF supports deny-by-default and allow-by-exception network traffic patterns at multiple layers, including the ESX host firewall, vSphere Distributed Switch traffic filtering, and NSX routing layer. Full workload firewall deny-by-default enforcement at distributed and gateway firewall levels is provided by VMware vDefend, which is a separate product.

ESX hosts include a built-in firewall that controls incoming and outgoing connections for management services, with service groups including SSH and SNMP. Each firewall service allows access from all IP addresses by default; administrators can restrict access by specifying allowed IP addresses in CIDR notation (for example, 192.168.0.0/24) or as individual comma-separated addresses. The `esxcli network firewall ruleset set --allowed-all` command disables open access for a ruleset and enforces an explicit IP allowlist per service, implementing a deny-by-default posture at the management-service level on every ESX host.

vSphere Distributed Switches support Traffic Filtering and Marking policies at the distributed port and uplink port level. Network traffic rules can be configured with Allow or Drop actions to control traffic flowing through individual ports, and port-level override options allow policy exceptions to be applied at the individual port level without modifying the parent distributed port group policy. These rules operate at the virtual switch tier and provide a deny-by-default mechanism independent of firewall functionality.

At the routing layer, NSX Tier-0 Gateways support IP prefix lists that control route redistribution with permit and deny actions. The default action for a route in an IP prefix list is Deny, which implements a deny-by-default posture for route advertisement. Administrators must explicitly add a Permit entry for each desired route, with all others denied by default. This deny-by-default routing posture restricts which traffic flows can be advertised and reachable through the NSX infrastructure.

VCF Automation allows administrators to associate security groups with network profiles. When a network profile includes security groups, all machines provisioned with that profile inherit the associated network policies, providing a way to apply consistent deny/allow policies at provisioning time. The security group constructs integrate with vDefend firewall enforcement when vDefend is deployed, surfacing rule-based access controls through the automation layer.

VCF Operations for Networks provides a Security Planning capability through the Plan and Assess interface. This feature analyzes flows and displays traffic flow patterns and micro-segmentation data, giving administrators visibility into which traffic is currently permitted and helping identify gaps in deny-by-default coverage. Planning queries can analyze data center security by planning security policies for specified entities before they are applied.

At the workload consumption layer, VMware Kubernetes Service (VKS) clusters use Kubernetes NetworkPolicy resources as the primary mechanism for deny-by-default traffic control between pods. A NetworkPolicy selects pods by label selector and specifies which ingress and egress traffic is permitted; once at least one NetworkPolicy applies to a pod, all traffic not explicitly permitted is dropped for that pod. To establish a deny-by-default posture in a namespace, administrators create a NetworkPolicy that selects all pods but specifies empty ingress and egress rules, then layer explicit allow rules on top using pod label selectors, namespace selectors, destination ports, or IP block CIDR ranges with optional exclusions. VKS uses Antrea as its default Container Network Interface (CNI) plugin; Antrea implements NetworkPolicy enforcement using Open vSwitch, enabling per-flow filtering at the virtual switch layer. The AntreaPolicy feature gate enables Antrea-native policies, which support tiered policy ordering and cluster-scoped rules beyond what standard NetworkPolicy resources support.

Beginning with VKS 3.6, VKS clusters support configurable inbound firewall rules for cluster nodes through the `firewall` subvariable of the `ClusterClass` node variable. The `firewall.inboundRules` variable can be set on control plane nodes and as variable overrides on each worker node `machineDeployment`; each rule accepts an optional source IP address or CIDR block

and a target protocol specified as an IANA protocol number or alias string such as ``tcp`` or ``udp``. This provides a declarative API for controlling node-level inbound access, complementing the pod-level NetworkPolicy layer.

Each VKS cluster is provisioned with a built-in distributed firewall rule for kube-dns, applied to the CoreDNS pod logical port at cluster creation, restricting CoreDNS access to its designated port. Antrea NetworkPolicyStats is enabled by default on VKS clusters, providing statistics on which network policies are active and what connection volumes they have matched. VKS Cluster Management Security Policies include a Policy enforcement setting whose default action is Deny, which fully enforces the policy and denies admission requests that contain violations; a Dry run mode is available for validation before enforcement is activated. The ``DenyServiceExternalIPs`` admission controller restricts use of ``LoadBalancer`` and ``ExternalIPs`` service types, limiting which services can advertise external endpoints.

Kubernetes itself enforces deny-by-default access control at the API server level: all parts of an API request must be explicitly permitted by at least one authorization mechanism in order to proceed. One scope boundary applies at the pod networking layer: Kubernetes NetworkPolicy resources are specified to restrict TCP, UDP, and SCTP traffic; for other protocols such as ARP and ICMP, enforcement behavior depends on the CNI plugin in use, and administrators should verify handling for those protocols in their specific deployment. A namespace with no NetworkPolicy applied allows all pod traffic by default; achieving deny-by-default behavior requires deliberate creation and deployment of default-deny NetworkPolicy objects in every namespace, which organizations must establish as an operational practice.

VMware vDefend extends these capabilities with full stateful firewall enforcement at both the gateway and workload tiers. vDefend applies security policies at tier-1 gateway service interfaces, controlling traffic as it enters and exits network segments. vDefend firewall rules provide granular control through configurable source groups, destination groups, services, and direction settings (inbound, outbound, or both). Each rule supports three actions: Allow permits matching traffic, Deny actively rejects it with a notification to the client, and Drop silently discards the packet. This rule structure enables administrators to build policies that block all traffic by default and then define specific exceptions for permitted flows.

Rule ordering within vDefend firewall policies is significant for implementing deny-by-default patterns. Rules that permit specific exceptions should be placed before rules that drop traffic, so that the allowed exceptions are evaluated first and all remaining unmatched traffic falls through to the default deny rule. vDefend preserves the order in which policy rules are added, and precedence is determined at both the policy level and the individual rule level.

vDefend Layer 7 access profiles support URL category filtering with configurable Allow, Reject, or Reject with Response actions for matched traffic. Administrators can define FQDN-based firewall rules to allow or reject traffic going to specific domains, using both pre-defined and custom FQDN lists. URL categories support wildcard pattern matching for domain filtering, including single-level and multi-level wildcards and anchored patterns to match specific URL structures.

vDefend includes a Firewall Exclusion List that allows administrators to designate up to 100 groups that should bypass firewall rule evaluation. This should be used sparingly, as excluded objects are not subject to the deny-by-default policy.

vDefend extends deny-by-default enforcement to east-west traffic between virtual machines. vDefend DFW is a software-defined Layer 2-7 firewall that provides stateful firewalling with distributed firewall policies supporting Allow, Drop, and Reject actions, direction controls, and context-aware rule parameters including sources, destinations, services, and context profiles. vDefend functionality can be configured on virtual distributed switches in data centers and workloads where segmentation, visibility, or advanced security capabilities are desired.

Organizational governance of firewall rule policies, change management for rule updates, and periodic review processes to verify that deny-by-default posture remains intact across the environment must be established outside the platform.

### VMware vDefend (Meets)

VMware vDefend (VMware vDefend) implements a deny-by-default, allow-by-exception traffic policy for both East-West and North-South traffic in a VCF environment.

The vDefend distributed firewall (DFW) enforces stateful Layer 2-7 security policies at the virtual network interface level of each workload. Each firewall rule is configured with one of three actions: Allow, Drop, or Reject. DFW automatically generates a Default Drop rule that denies all traffic from any source to any destination for any service. This catch-all rule sits at the

bottom of the rule table and enforces policy for packets not matching any other rule. VMware recommends changing the DFW default rule to a block action and enforcing access control through a positive control model where only traffic explicitly defined in firewall rules is permitted. Specific rules should be placed above more general rules in the rule table so that more targeted permits take precedence. DFW supports Context Profiles for Layer 7-aware rule matching, enabling allow-by-exception rules that specify application-layer context in addition to source, destination, and port.

The Application Connectivity Strategy in DFW provides structured, API-driven approaches to configuring the default traffic posture. The Allow Intra, Deny Other Traffic strategy uses the ALLOW\_INTRA and ALLOWLIST APIs to generate an intra-group Allow rule paired with a default ANY/ANY/DROP rule, isolating the group from all other traffic. The Allow List connectivity preference configures a Drop action as the default, denying all traffic not explicitly permitted. For traffic not covered by named strategies, DFW supports custom policies that provide granular control of lateral East-West traffic.

FQDN filtering rules in DFW support Allow, Drop, or Reject actions, extending the deny-by-exception model to domain-name-based traffic control. The Identity Firewall enforces the allow-by-exception discipline for user-identity-based access: each Identity Firewall rule that permits traffic from a user group to a destination must be paired with a corresponding DFW or Gateway Firewall rule applied to the destination group. Identity Firewall rules also support a Reject action, allowing administrators to deny traffic matching specified user groups, services, and destinations.

The vDefend gateway firewall provides deny-by-default enforcement for North-South traffic at the NSX edge. Gateway Firewall rules support Allow, Drop, and Reject actions. In federated deployments, DFW and Gateway Firewall rules can be created from the Global Manager with global, regional, or local spans and are synchronized to Local Managers. The default gateway firewall rule is editable from the Global Manager and can be configured as allow-all or deny-all, giving administrators control over the baseline North-South posture. The L7 Access Profile extends allow-by-exception to application-layer traffic, with rule actions of Allow, Reject, and Reject with Response.

The Security Services Platform (SSP) and Security Intelligence support the transition to and maintenance of a deny-by-default posture. After establishing exception policies for an environment pair, administrators can change the protection rule action to Drop or Reject to block new, unknown traffic. Policy recommendations can be scoped with a Source Exclusions List, allowing fine-tuned control over which traffic sources are included in generated allowlists. The Security Overview Dashboard provides views of DFW policies, including policy actions (allow, drop, and reject) and total rule counts, giving administrators visibility into the active enforcement posture. Security Policies in vDefend DFW can be reviewed and modified through the Policies and Rules field.

The Firewall Rule Analysis Report identifies potential security misconfigurations in the distributed firewall configuration, helping administrators find shadowed or overly permissive rules that could undermine the deny-by-default posture.

### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer (Avi) provides deny-by-default traffic controls at the application delivery layer through Network Security Policies, HTTP Policy Sets, and Web Application Firewall (WAF) policies in Enforcement mode.

Avi's Network Security Policy is evaluated for every client connection to a virtual service. The policy determines whether a connection is accepted, discarded, or rate-limited based on network-level parameters. Multiple ordered rules can be configured within each Network Security Policy, with explicit Allow and Deny actions available. When a rule with a Deny action matches a connection, further policy evaluation stops, so downstream HTTP Security, Request, or Response policies do not execute for that traffic. Deny rules support a log parameter that, when set to True, records all denied traffic. ControlScripts can dynamically create Network Security Policy rules that deny traffic from specific client IP addresses identified during DDoS events, using IS\_IN match criteria against a configured list of source addresses. Network Security Policy can also be configured to deny connection attempts from addresses identified as malicious through IP Reputation data, dropping TCP SYN packets before connections are established. Both Network Security and HTTP Security policies support a Blocklist mechanism, where a policy rule references a configured IP group named Blocklist to discard clients matching that group.

The HTTP Policy Set provides an additional allow-or-deny mechanism at the application layer. Match conditions classify traffic and apply explicit allow or deny actions; denied traffic receives a TCP RST unless the system is under a volumetric denial-of-service attack, in which case the connection is silently discarded.

When configured in Enforcement mode, Avi's WAF applies a positive security model to application-layer traffic. WAF rule actions support pass (allow) and deny (block) options; requests that satisfy configured rules are permitted, and those that do not are blocked by default. Administrators can author custom WAF rules using SecRule syntax with chain operators, enabling multi-condition enforcement such as allowlisting specific source IP ranges for specific URI paths. WAF Exceptions allow specific traffic to be excluded from rule enforcement based on Client IP, Path, or Match Element filter combinations. WAF Allowlist rules support a BYPASS action for trusted IP address groups, including IP Group-based allowlisting, providing an allow-by-exception mechanism alongside the default-deny posture.

Avi's traffic controls apply only to application traffic passing through its virtual services and Service Engines. Deny-by-default policy enforcement for general east-west and north-south network traffic across the VCF platform is a function of VMware vDefend, which is a separate product. An organization implementing deny-by-default network traffic controls in a VCF environment will typically use both Avi and vDefend together, with Avi addressing application-layer traffic at the load balancer boundary.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.



## Control SAR 2.4

> App mobile code integrity: Verify integrity of app-level mobile code. App mobile code authenticity: Verify authenticity of app-level mobile code. App mobile code authorization: Enforce authorized execution for applications. App mobile code policy: Enforce security policies within the software app. Application sandboxing: Run application code in restricted containers.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control SAR 2.4(1)

> App mobile code integrity: Verify integrity of app-level mobile code. App mobile code authenticity: Verify authenticity of app-level mobile code. App mobile code authorization: Enforce authorized execution for applications. App mobile code policy: Enforce security policies within the software app. Application sandboxing: Run application code in restricted containers.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control SAR 2.4(a)

> App mobile code integrity: Verify integrity of app-level mobile code. App mobile code authenticity: Verify authenticity of app-level mobile code. App mobile code authorization: Enforce authorized execution for applications. App mobile code policy: Enforce security policies within the software app. Application sandboxing: Run application code in restricted containers.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control SAR 2.4(b)

> App mobile code integrity: Verify integrity of app-level mobile code. App mobile code authenticity: Verify authenticity of app-level mobile code. App mobile code authorization: Enforce authorized execution for applications. App mobile code policy: Enforce security policies within the software app. Application sandboxing: Run application code in restricted containers.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control SAR 2.4(c)

> App mobile code integrity: Verify integrity of app-level mobile code. App mobile code authenticity: Verify authenticity of app-level mobile code. App mobile code authorization: Enforce authorized execution for applications. App mobile code policy: Enforce security policies within the software app. Application sandboxing: Run application code in restricted containers.

**SCF Controls:** END-10

**Aggregate Coverage:** Not Applicable

Not applicable for this control: VCF, VMware vDefend, VCF Protection and Recovery, VMware Data Services Manager, VMware Avi Load Balancer. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

## Control SAR 3.2

> Application malware protection: Use app-level scanning for malicious code.

**SCF Controls:** END-04

**Aggregate Coverage:** Contributes

### VCF (Contributes)

VCF provides several platform capabilities that support malicious code protection, though organizations must deploy additional tools to achieve full antimalware coverage across endpoints.

At the hypervisor layer, VMware ESX supports Secure Boot and code signing to restrict unauthorized code from executing on hosts. Trusted Platform Module (TPM) support enables boot-time integrity measurement, and VMware vCenter provides APIs to retrieve TPM event logs for validating boot-time measurements or configuring firmware trust with an attestation service. Administrators can verify that hosts have booted in a known good configuration by checking TPM device status and event logs through the vCenter trusted infrastructure API. ESX host TPM modules must be verified as running the latest 2.0 firmware, and if not in use, disabled in the BIOS. Virtual machines can be configured with UEFI firmware for guest operating system boot integrity. Windows guest operating systems support Virtualization Based Security (VBS), which enables Device Guard to help prevent malware from running and Credential Guard to protect credential secrets; these features are configurable through Group Policy on VMs with ESX 6.7 or later compatibility. Virtual machines can also have the Guest Integrity (GI) security device enabled through the VMware Host Client, which is the platform foundation for VMware AppDefense.

The vSphere Security Configuration Guide provides hardening recommendations for ESX hosts, including configuring authentication and authorization settings, disabling unnecessary services, and enabling security features like Secure Boot and TPM. These configuration baselines are industry-standard best practices for host hardening.

vSphere Lifecycle Manager and VCF Operations Configuration Profiles maintain desired-state configuration for ESX hosts. Configuration Drifts in VCF Operations provides a proactive method to monitor configuration changes and identify when vCenter instances and vSphere clusters deviate from their approved baselines. Drift status is updated through a collection cycle that runs every 8 hours. This helps identify when host software or configuration has been modified from its approved state, which can indicate unauthorized changes or tampering.

VCF's networking layer, NSX, supports integration with third-party endpoint protection solutions for virtual machine security. The platform documentation references specific partner integrations including Symantec Endpoint Protection and ESET Antivirus/Security software, which provide anti-malware, intrusion prevention, and firewall capabilities for guest virtual machines. NSX App IDs can be used in conjunction with these third-party solutions for network management and security policy enforcement. Organizations can implement security groups, define security policies, and use NSX's integration with third-party security solutions for virtual machine protection.

NSX Guest Introspection provides an agent-based framework that supports endpoint security within virtual machines. The Guest Introspection Agent must be installed and running in each guest VM, and logged-in users should not have the privilege to remove or stop the agent. This architecture enables third-party security solutions to inspect guest operating system activity without requiring agents to be installed directly on the hypervisor. The security and integrity of the guest operating system must be maintained for these integrations to function correctly.

For container workloads, the VCF CLI supports cosign signature verification for container images through the `imgpkg` tool, which enables finding and copying cosign signatures. Image pull authentication through `imagePullSecrets` restricts container image pulls to authenticated private registries. Plugin discovery image signature verification is supported, with configurable public key paths for the cosign verifier. These capabilities help restrict deployment of images that have been tampered with.

VCF Operations supports the deployment and monitoring of agents on endpoint virtual machines, with guest operation privileges required to install agents. Automated software update capabilities in vCenter help keep systems current with security patches, reducing the window of exposure to known malware exploits.

For organizations that deploy VMware vDefend as an add-on to VCF, distributed malware prevention capabilities become available. The NSX Application Platform hosts the Malware Prevention for vDefend feature, which can be deployed from the Local Manager UI in NSX Federation environments. When host clusters are activated for Distributed Malware Prevention, a service virtual machine (SVM) is deployed on each host to perform malware scanning. vDefend also supports IDS/IPS capabilities alongside distributed malware prevention. The DistributedMps runbook is available to verify the health of the Malware Prevention Service pipeline and diagnose issues such as protection not working on a particular VM or certain files not being scanned.

Malware prevention activities generate log data for audit and forensic purposes. The Malware Prevention for vDefend feature writes log files that can be used for troubleshooting and security monitoring, and remote logging can be configured on the malware prevention SVMs for centralized log collection.

VMware Private AI Services (PAIS) includes model validation capabilities that scan machine learning model files for malicious content before they are accepted into a Model Gallery. The validation runs as a sandbox process on Deep Learning VMs and covers integrity verification through hash code checking, malware scanning, serialization attack scanning, and inference behavior testing with NVIDIA Triton Inference Server. Both the malware and serialization checks run as part of the validation workflow required before a model can be uploaded to a Model Gallery and made available to downstream model endpoints, knowledge bases, or AI agents.

Machine learning model files are a category of artifact that traditional endpoint security tools may not inspect. Serialized models in formats such as Python pickle files can carry executable payloads that activate when an application or researcher loads the model. The PAIS Model Gallery intake workflow treats this upload boundary as a security checkpoint, requiring each artifact to pass hash, malware, serialization, and inference behavior tests before it is published for use. PAIS guidance also directs organizations to distribute only models that have been validated internally on a Deep Learning VM rather than pulling artifacts directly from the Internet, where models may contain malicious code or be tuned for malicious behavior. The validation is operated by MLOps engineers, who confirm that onboarded models meet the organization's security, privacy, and technical requirements.

For platform-level malware protection covering virtual machines, VMware ESX hosts, and general workloads running on the PAIS infrastructure, organizations rely on VCF platform controls and, where deployed, VMware vDefend. PAIS adds the specialized scanning layer applied to AI model artifacts at the point of intake, addressing a threat surface that general-purpose antimalware tools do not typically cover.

### **VMware vDefend (Contributes)**

VMware vDefend provides multiple layers of malicious code detection and prevention through its Malware Prevention Service and IDS/IPS capabilities. Together these address the detection aspect of this control directly and contribute to the eradication aspect by restricting malicious code from executing and spreading across the network.

The Malware Prevention Service deploys as a service virtual machine (SVM) on each host in a cluster, with Guest Introspection thin agents installed on each protected guest VM. This architecture separates the detection engine from the protected workload, giving the service visibility into file activity, process execution, and network events from outside the guest. On Linux guests, a File and Network Agent handles file and network traffic introspection. A Host Agent (MUX) component running on ESX hosts supports the Guest Introspection functionality.

The service detects both known and unknown malicious files, including zero-day threats. Supported file categories include Windows executables (.exe, self-extracting), Linux executables (.elf), installer files (.msi), and shortcut files (.lnk), among others. When a file is analyzed, Malware Prevention assigns a verdict and classifies the threat by malware class (virus, trojan horse, worm, adware, ransomware, spyware) and malware family. A single detection event can be associated with multiple malware families and classes. Upon detection, the service can take direct remediation actions, including denying access to suspicious files or deleting files identified as malicious. An allowlist capability enables administrators to override verdicts for files known to be legitimate, suppressing false positives without disabling broader detection.

Beyond file-based detection, Malware Prevention Service includes fileless malware detection to identify attacks that operate entirely within system memory using existing binaries and in-memory operations such as PowerShell or VBScript, without



leaving executable artifacts on disk. The service monitors Indicators of Compromise including process command lines, process hashes, process execution trees, and in-memory executions. In-memory script buffer executions are scored for threat severity and assigned a verdict of malicious or suspicious, with the associated malware family and class displayed on the Malware Prevention Dashboard.

When malware is detected, the service extracts artifacts and in-memory script buffers that security teams can use to define incidents of compromise and create new detection rules for similar future threats. This extends malware coverage as new threat patterns are identified within the environment.

The Malware Prevention Dashboard in Security Services Platform aggregates detection events for malicious files, suspicious files, and uninspected files over configurable time periods. Each unique file is tracked by hash, and events can be filtered by verdict criteria. This centralized visibility allows security teams to identify malware incidents, track detection trends, and coordinate eradication efforts across the virtualized environment.

vDefend IDS/IPS provides signature-based and behavioral intrusion detection and prevention at the distributed firewall level, complementing Malware Prevention with network-layer detection of malware delivery attempts, command-and-control communications, and lateral movement patterns associated with malware infections. IDS/IPS operates at the virtual network interface level, providing visibility into east-west traffic between workloads that perimeter security tools cannot inspect.

While vDefend provides strong detection and file-level remediation capabilities, complete eradication of malicious code from infected guest operating systems typically requires endpoint security agents operating inside the guest. vDefend restricts malware from spreading across the network and blocks or removes detected malicious files, but cleaning an already-infected system requires host-level remediation tooling.

#### **VMware Avi Load Balancer (Contributes)**

VMware Avi Load Balancer contributes to malicious code protection at the application delivery layer through WAF-based payload inspection, IP Reputation threat intelligence, Application Rules, and Bot Management.

The Avi WAF inspects inbound HTTP/HTTPS request bodies and parameters against signature-based rules, including ModSecurity rules, to detect and block exploit payloads, injection attacks, and known malware delivery patterns. WAF rules are configurable per virtual service and can be tuned to reduce false positives while maintaining coverage of malicious traffic patterns. Configuration is managed through the Avi Controller UI under the WAF Policy section for each virtual service.

The Application Rules service, available through Avi Cloud Console Live Security Threat Intelligence, provides automatically updated WAF rules targeting known vulnerabilities with CVE mappings across more than 5,000 applications, including WordPress, Drupal, and Apache. These rules are pushed automatically to Avi Controllers, reducing the window between vulnerability disclosure and protection.

Avi's IP Reputation service provides a threat intelligence feed that classifies IP addresses known to be infected with malware or identified as contacting malware distribution points (threat category code 6). The service also identifies IP addresses known to distribute malware, shell code, rootkits, worms, or viruses, including Windows exploit IP addresses. An "All threats" classification (threat category code 32) can be applied to block any suspicious IP addresses at the application delivery layer. IP Reputation data is synchronized via Avi Cloud Services and can be referenced in virtual service policies to block malicious source traffic before it reaches protected workloads. This is configured under Administration > External Services > Cloud Services.

Avi Bot Management provides a default System-BotDetectionPolicy that can be bound to virtual services to detect and mitigate malicious bots used in online fraud, credential stuffing, and automated data scraping. Bot Management performs heuristic scanning of User-Agent strings to detect malicious patterns such as SQL injections, marking such requests as bad bot of type web attack. In Kubernetes deployments, the Avi Kubernetes Operator (AKO) supports assigning a BotDetectionPolicy to virtual services via the botPolicyRef parameter, extending bot detection to Kubernetes ingress workloads.

Avi WAF does not include built-in virus or file-content malware scanning. Avi documentation explicitly recommends deploying a dedicated virus and malware scanning tool on the upload directories of protected applications to detect and mitigate malicious uploads, including uploads that may bypass WAF inspection. Malware detection and eradication for workloads

running behind Avi is the responsibility of endpoint protection tools deployed on those workloads. Organizations should configure WAF policies, Application Rules, IP Reputation, and Bot Management on Avi virtual services while also deploying dedicated antimalware solutions on application workloads and their supporting infrastructure.

Not applicable for this control: VCF Protection and Recovery, VMware Data Services Manager. This control addresses an end user activity or process, a design decision, or activity outside the scope of these products.

